Faire face au "Pogo vert"

January 18, 2024 9:18 AM



Publié le 30/11/2022

Avez-vous déjà été témoin ou vécu les situations suivantes dans le cadre de votre travail

- Chut! il ne faut pas dire que ça ne va pas bien
- Ce n'est pas vraiment un "failed" c'est plutôt un "pass whatever"
- Arrêtons de chercher des bugs, car il va toujours en avoir
- Est-ce qu'on peut livrer, hier, tout est beau?!
- Coupons dans les tests, on a pas besoin de ça !

Et que votre opinion n'est pas prise en compte sur le fait que ça ne va pas du tout, qu'il ne faut pas livrer !

Définition

- "Pogo" => est relatif à "Pas Go!" pour livrer.
- et "Vert" => est relatif aux résultats des exécutions de tests.

La raison d'utiliser "Pogo vert" est simplement que parfois, il n'est pas permis, suggéré fortement ou difficilement possible de dire selon nous, que ce n'est pas un Go! pour la livraison, que nous devons faire disparaître les rouges des résultats et livrer quand même.

On s'enttend que cela reste, qu'un Pogo, que c'est du "fast food", même en ajoutant du vert (salade).

Comment l'éviter ou du moins le négocier ?

Pour nous QA Agile, c'est normal de livrer avec des bugs, avec les résultats sans cacher ou traffiquer les résultats.

On nous demande de fermer les yeux, nous sommes, donc, dans une situation délicate, que nous tournions gauche ou à droite, c'est le piège assuré !

Vos choix :

1. Se battre au nom de la qualité satisfaction, et risquer gros !
2. Fermer les yeux, on peut juste se dire que c'est encore un "Pogo vert" et juste dire que si la livraison est de la "crape", que le client n'est pas satisfait,

ce n’est pas notre faute pour esuite entrevoir la chute et de se préparer à proposer des idées de solutions par la suite. Comme on dit : -"*On traversera le pont une fois rendus à la rivière*"

3. **Demandez de l'aide rapidement à votre coach ou votre gestionnaire Alithya. C'est par expérience la meilleure option, car c'est, ici, une situation politique.**

Vous a vez-vous une idée de solution ?
Lassez-nous vos commentaire

Acrostiche Q.U.A.L.I.T.É.

January 18, 2024 9:19 AM

Qualification
Utile
Acceptation
Loyauté
Imagination
Transparence
Émerveillement

Et vous, ce serait quoi vos mots ?

L'expérimentation est une condition préalable de la qualité

January 18, 2024 9:19 AM

Pour bien expérimenter, il faut savoir bien échouer

Dans le Larousse

Résultat négatif d'une tentative, d'une entreprise, manque de réussite ; défaite, insuccès, revers : Subir un échec.
Synonymes : faillite - fiasco (familier) - four (familier) - veste (familier)
Contraires : réussite - succès - victoire

Wikipédia "Échec"

Échec est l'état ou la condition qui fait que l'objectif désiré ou prévu n'est pas atteint, et peut être vu comme l'opposé de succès, mais cela peut aussi désigner un sentiment qui surgit lorsque l'on regrette quelque chose, au même titre que le regret.

Dans son sens le plus général, un échec est une situation qui résulte d'une action n'ayant pas abouti !

Résultat

Donc, il est normal que, spontanément, échec donne souvent l'impression d'avoir perdu. Ce n'est, donc, certainement pas l'idéalisation que l'on se fait de la qualité.

Suivi rapidement de la perte de confiance, perte sécurité, perte de succès et perte de satisfaction… c'est devenu une nuisance un problème, un conflit…

Donc réaction → on se débarrasse, on jette à la poubelle !

On ne veut, absolument, pas échouer, surtout devant les autres, la honte !

Mais est-ce que nous passerions pas à côté d'un trésor de qualité inestimable sans le savoir ?!

Nous faisons face à un paradoxe :
L'échec est vu comme ne pas être à la hauteur, tandis que la qualité est vu comme éviter l'échec, qui ne passe pas le test, etc.
Quand, pourtant, nous devrions voir l'échec comme l'expérience faisant, intégralement, partie du processus de la qualité.

Pourquoi il est important, crucial et vital d'échouer ?

Expérimenter, échouer, apprendre, répéter.

L'échec est une opportunité

Commençons par dire que échec n’est pas un synonyme de fatalité
c’est un résultat qui a différentes perspectives
et différents contextes
que nous pouvons continuer
en amélioration continu.

Certaines personnes pourraient dire que l'échec est le pire résultat possible, mais cela peut être l'une des choses les plus importantes qui se produisent dans la vie de quelqu'un .

Je n'ai pas échoué. Je viens juste de trouver 10 000 façons qui ne fonctionnent pas.

La première étape vers le succès est l'échec. Nous avons tous peur d'échouer et ce n'est pas mal d'avoir peur. Mais nous devons apprendre de nos erreurs afin de pouvoir grandir en tant que personne et avoir plus de succès à l'avenir.

Cela peut être difficile à gérer, mais si nous le voyons sous un jour positif , nous pouvons en sortir plus forts de l'autre côté. Cet article de blog va vous montrer pourquoi l'échec est important pour votre croissance en tant qu'individu.

L'échec offre une opportunité de croissance et d'apprentissage , et il peut nous aider à devenir de meilleures personnes. Voici quelques-unes des raisons pour lesquelles vous devriez apprendre de l'échec.

Il y a une raison pour laquelle l'échec est considéré comme l'un des ingrédients les plus cruciaux du succès. Lorsque vous échouez, vous apprenez ce qui ne fonctionne pas – et cette information est essentielle pour déterminer ce qui fonctionne. Chaque personne qui réussit a échoué un nombre incalculable de fois, mais elle a également appris de ses échecs et a continué à avancer . Alors n'ayez pas peur d'échouer – adoptez-le,

apprenez-en et utilisez-le pour vous rendre encore plus performant.

FAIL : Première tentative d'apprentissage.

L'échec nous aide à apprendre de nos erreurs

Vous avez besoin d'un échec pour vous développer. C'est la plus grande leçon de la vie, et elle se produira que vous essayiez de l' empêcher ou non. Peu importe à quel point vous essayez de l'éviter, un échec se produira, alors ayez une attitude sans crainte à son égard. Cela ne signifie pas que vous devez vous attendre à échouer ; plutôt, acceptez-le quand cela vous arrive.

Il y a quelque chose à apprendre de l'échec. Vous apprendrez à mieux vous connaître et apprendrez de vos erreurs à la suite d'un échec. Nous reconsidérons, repensons et découvrons de nouvelles façons et techniques pour atteindre nos objectifs à cause des échecs.

Personne n'est parfait et nous faisons tous des erreurs. L'échec peut nous aider à apprendre de nos erreurs afin de ne pas les refaire à l'avenir.

Échouer c’est apprendre rapidement.

L'échec est le moyen le plus rapide d'apprendre

La plupart d'entre nous apprenons mieux de nos échecs. Nous ne pouvons pas apprendre si nous n'essayons pas, et nous ne pouvons pas essayer si nous avons peur d'échouer.

L'échec est le moyen le plus rapide d'apprendre , car vous obtiendrez un retour immédiat sur ce qui fonctionne et ce qui ne fonctionne pas . Vous apprendrez également à gérer le stress et la déception.

Il n'y a tout simplement pas de meilleur instructeur que l'échec. Ce n'est que par essais et erreurs que vous découvrez ce qui fonctionne et ce qui ne fonctionne pas.

Garde l'espoir vivant.

L'échec peut nous aider à devenir plus résilients

Lorsque vous échouez à quelque chose, être résilient peut vous aider à l'accepter et à vous relever. Il faut être dur pour surmonter les effets négatifs de l'échec.

Certaines personnes permettent aux échecs de les abattre, tandis que d'autres y voient l'opportunité d'une vie. Ceux qui continuent d'essayer après avoir subi plusieurs défaites peuvent être qualifiés de résilients.

L'échec peut vous aider à devenir plus résilient ; vous apprendrez à vous dépoussiérer et à continuer.

Un aspect essentiel de la créativité est de ne pas avoir peur d'échouer.

L'échec stimule la créativité

La plupart des grandes inventions dans le monde sont issues d'un échec. Thomas Edison, par exemple, a échoué plus de 1 000 fois avant de créer l'ampoule.

L'échec peut nous aider à devenir plus créatifs car il nous oblige à proposer de nouvelles idées et solutions. Cela peut aussi nous aider à être plus innovants.

La créativité et l'innovation proviennent d'une combinaison d'expérience, de connaissances et d'échecs. Il ne peut y avoir d'innovation sans échec.

Intelligemment : spécifique, mesurable, atteignable, réaliste, d’une durée déterminée.

L'échec nous aide à fixer et à atteindre des objectifs

Il est difficile d'atteindre des objectifs si vous êtes réticent à l'échec. Vous ne saurez jamais ce qui est possible si vous n'essayez pas.

L'échec fait partie du processus qui mène au succès. Les gens qui ont accompli de grandes choses ont échoué plusieurs fois . Vous devez fixer des objectifs élevés et être prêt à faire tout ce qu'il faut pour les atteindre, y compris échouer.

Tu veux des résultats, alors entraîne toi à la mesure de tes attentes.

L'échec vous rend plus fort

Les obstacles que nous surmontons renforcent notre force et notre caractère, ce qui nous rend plus forts.

L'échec peut vous rendre plus fort physiquement, émotionnellement et mentalement. Cela peut également vous aider à devenir plus indépendant.

Selon une nouvelle étude , vous avez plus de chances de réussir à l'avenir si vous faites face à un échec tôt dans la vie.

Lorsque vous rencontrez un échec, ce n'est pas la fin du monde. C'est juste un revers temporaire que vous devez surmonter. Vous devez croire en vous et croire que vous pouvez le faire.

L'échec nous rend plus ingénieux

Si vous ne savez pas comment résoudre un problème ou ce que vous avez fait de mal, concentrez-vous à nouveau sur le processus et résolvez -le la prochaine fois .

La débrouillardise est la capacité à trouver des solutions aux problèmes. Si vous utilisez l'échec comme une expérience d'apprentissage, vous deviendrez plus ingénieux.

Quand quelque chose ne marche pas tout de suite, n'abandonnez pas. Continuez d'essayer jusqu'à ce que vous réussissiez. L'échec est une opportunité pour nous d'apprendre et de devenir plus ingénieux.

Quiconque n'a jamais fait d'erreur n'a jamais essayé quelque chose de nouveau.

L'échec peut nous aider à développer de nouvelles compétences et forces

L'échec vous aide à apprendre à gérer les défis de manière positive. Cela peut également vous aider à devenir plus apte à résoudre des problèmes et à faire face à des situations difficiles.

Lorsque l'échec vous offre un choix alternatif , il est tentant d'abandonner. Peut-être que reconsidérer et prendre du recul vous aiderait à découvrir la meilleure approche pour utiliser vos forces.

Lorsqu'un échec survient, cela devient une opportunité pour nous de développer de nouvelles compétences et forces qui nous aideront à l'avenir. Certaines de nos plus grandes forces proviennent des échecs tout au long de la vie.

L'échec met les choses en perspective

Lorsque vous échouez à quelque chose, cela vous montre ce qui est vraiment important dans la vie. Pourquoi s'acharner sur des choses qui n'ont pas d'importance alors qu'il y a tant d'autres choses importantes sur lesquelles se concentrer ?

Lorsque vous rencontrez un échec, cela aide à mettre les choses en perspective. Vous vous rendrez compte que certaines des choses qui vous préoccupaient ne sont pas importantes du tout.

L'échec peut nous aider à devenir plus reconnaissants pour ce que nous avons et à nous concentrer sur les choses qui comptent le plus.

Ne sois pas effrayé d'être différent.

L'échec nous aide à nous débarrasser de la peur

D'une manière ou d'une autre, bon nombre des préoccupations que nous avons dans nos vies sont liées à l'échec. C'est la peur d'échouer qui nous empêche d'explorer d'autres possibilités et de sortir de nos zones de confort.

Lorsque nous faisons face à nos peurs, il devient plus facile pour nous de les surmonter. L'échec peut nous aider à nous débarrasser de nos peurs. Une fois que vous vous êtes habitué à perdre, vous n'avez plus peur de poursuivre vos objectifs. Vous devenez plus disposé à prendre des risques et quittez votre zone pour réaliser tout ce que vous désirez. Cela devient moins effrayant lorsque nous en faisons l'expérience et réalisons que ce n'est pas la fin du monde.

Reste humble tu n'es pas là pour impressionner personne sauf toi-même.

L'échec nous apprend l'humilité

Il est important de se rappeler que l'échec ne fait pas de vous un échec, mais plutôt une personne qui a vécu une défaite momentanée. Si nous ne pouvons pas rire de nous-mêmes, nous devenons incapables de passer à autre chose après les échecs.

Lorsque nous expérimentons un échec, cela peut nous apprendre l'humilité et comment accepter la défaite avec grâce. L'échec est une expérience humiliante et nous rend plus humains.

L'échec peut nous aider à être humbles après avoir réussi. Il est facile d'être arrogant après avoir réussi quelque chose, mais l'échec peut rapidement remettre les choses en perspective.

L’appréciation infiniment plus que la reconnaissance.

L'échec vous aide à apprécier le succès

L'échec nous rend avide de succès et nous travaillons plus fort pour y parvenir. Nous apprécions également davantage le succès parce que nous savons à quel point il est difficile d'y parvenir.

Nous voulons tous réussir dans la vie, mais nous ne pouvons pas oublier que l'échec fait partie du processus. Il est important de connaître à la fois l'échec et le succès et de les apprécier tous les deux.

Lorsque vous réussissez après avoir connu un échec, cela a un goût beaucoup plus sucré. Vous vous rendrez compte que cela valait tout le travail acharné et les efforts que vous avez déployés.

L'échec nous enseigne des leçons de vie importantes

La leçon que l'on peut apprendre.

Une grande partie de ce qui est dit depuis le début de cet article revient aussi à dire que l'expérimentation est remplis d'enseignements.

Bon nombre des leçons de vie les plus importantes que nous apprenons proviennent de nos échecs. Nous apprenons à gérer la déception, à gérer le stress et à être résilients.

Rien d'autre que l'échec ne peut nous fournir une compréhension plus profonde de la vie et modifier la façon dont nous percevons tout ce qui se passe autour de nous. Cela nous transforme en de meilleures personnes qui sont plus conscientes de leur existence.

L'échec peut vous apprendre de nombreuses leçons importantes sur la vie que vous n'auriez pas apprises autrement. Cela peut nous aider à comprendre ce qui nous manque et nous aider à devenir de meilleures personnes.

L'échec favorise la croissance personnelle

Les personnes qui survivent à l'échec sont plus motivées pour améliorer leur vie et devenir de meilleures personnes.

Ceux qui échouent une, deux ou plusieurs fois sont plus susceptibles de continuer d'essayer jusqu'à ce qu'ils réussissent. Ils n'abandonnent pas facilement parce qu'ils ont déjà échoué au moins une fois.

Lorsque vous rencontrez un échec, c'est une opportunité de croissance personnelle et d'amélioration. Cela nous aide à devenir plus motivés et

engagés à réussir.

Célébrer les petites victoires.

L'échec peut être un tremplin vers le succès

L'échec peut être un tremplin vers le succès. Tout dépend de la façon dont vous y réagissez.

Si vous apprenez de vos échecs et devenez plus ingénieux, vous vous rapprocherez un peu plus de la réalisation de vos objectifs et de vos rêves .

Ne laissez jamais l'échec prendre le dessus sur vous. Il peut nous apprendre de nombreuses leçons importantes sur la vie que nous n'aurions pas apprises autrement. N'ayez donc pas peur d'échouer, cela peut être un tremplin vers le succès .

7 mentions qui sont à l'opposé de "échouer"

1. Échouer n'est pas évitable
2. Échouer n'est pas un évènement… c'est un processus
3. Échouer n'est pas un objectif
4. Échouer n'est pas l'ennemi
5. Échouer n'est pas irréversible
6. Échouer n'est pas stigmatisation
7. Échouer n'est pas final

Un chemin en images vers le succès passant à travers l'échec

Pourquoi l'échec est bon.

Pourquoi avons-nous besoin d'échec.

L'échec est important et essentiel.

L'échec est le succès en cours.

L'échec n'est pas le contraire du succès. ça fait partie du succès.

Pourquoi apprendre de l'échec est important.

Déjà essayé, toujours échoué, peu importe, essayer à nouveau, échouer à nouveau, échouer mieux.

Échouer tôt, échouer souvent, échouer en continuant d'avancer.

Échouer rapidement, apprendre plus vite.

Valider vite, échouer vite, ajuster vite.

Chaque échec est un pas vers le succès.

Échouer est l'opportunité de recommencer, plus intelligemment cette fois.

Échouer est l'opportunité de recommencer, plus intelligemment cette fois.

Échouer rapidement, souvent, sans relâche et tu y retrouveras le succès.

Échouer/Rapidement.

Réussir plus rapidement.

Conclusion

"L'expert de sa qualité expérimente son défaut"

Nous avons vu ensemble que l'**expérimentation** est une étape que nous devons passer par "**échouer**" pour arriver à faire de la **qualité**.

Avant de terminer, je trouvais important de souligner brièvement de faire de l'expérimentation/changement/innovation/amélioration de façon intelligente et sécuritaire.

Mieux vaut faire souvent de petits échecs dans un environnement sécuritaire que d'y laisser sa peau

Mais attention, changer de place les meubles, les outils, les choses en général… Quand on est seul ça va, mais quand on est plusieurs ça dérange, même si l'intention est d'améliorer l'organisation.
…Surtout si l'autre est aveugle, car cela brise les habitudes et les habitudes font partie de nos repères.

Donc, 3 conseils à garder à l'esprit lors de l'expérimentation :

8. Faire l'expérimentation dans un bac à sable,
9. Noter, analyser et valider constamment par petites bouchées,
10. Assister la transition, la pérennité et l'accoutumance.

…

Il y aurait encore beaucoup de beaux sujets à explorer :

- Est-ce qu'il y a de l'expérimentation si on ne fait que du contrôle de la qualité ?
- Dans un contexte où les gens ne connaissent pas les vertus de l'échec, est-il souvent très difficile d'échouer sans perdre en crédibilité ?

Avez-vous des astuces pour expérimenter de façon efficace ?
Comment feriez-vous ?
Partagez avec la communauté !

3 changements d'état d'esprit pour réussir avec l'automatisation des tests

January 18, 2024 9:20 AM

L'automatisation des tests est une condition préalable nécessaire pour un développement moderne de haute qualité. Les tests et l'automatisation des tests doivent être intégrés dans les processus de livraison de logiciels pour donner à l'équipe le bon niveau de retour d'information, au bon moment, et pour garantir un logiciel de haute qualité.

L'un des plus grands défis de la construction d'un cadre d'automatisation des tests qui fonctionne sur une base évolutive à long terme est qu'il nécessite des conditions fondamentalement différentes par rapport à une approche de test principalement manuelle.

Le passage à une approche axée sur l'automatisation nécessite de multiples changements d'état d'esprit, d'ensembles de compétences et des outils et processus nécessaires pour réussir. Le premier changement est d'un biais en faveur des tests manuels à un biais en faveur des tests automatisés. Le deuxième changement de mentalité consiste à passer des tests à la demande aux « tests en continu ». L'état d'esprit final est de se déplacer vers la gauche et d'écrire les tests avant le début du développement.

Dans l'article suivant, nous vous guidons à travers les différents changements d'état d'esprit, ainsi que les compétences et les emplois/responsabilités typiques pour réussir à chaque phase. Il n'y a bien sûr pas de solution miracle et chaque équipe aura des besoins différents, néanmoins, à mesure que l'automatisation des tests mûrit, le modèle mental que nous utilisons pour examiner les tests doit en faire autant.

3 changements d'état d'esprit pour réussir l'automatisation des tests

Créer un changement durable nécessite des changements dans les outils, les processus et, surtout, l'état d'esprit des parties prenantes impliquées. D'après notre expérience, ces 3 changements d'état d'esprit doivent être atteints pour réussir l'automatisation des tests sur le long terme.

Examinons chaque étape en détail et quelques-uns des défis à relever.

Commencer avec des tests manuels bien définis et structurés en vue de l'automatisation.

Dans les tests manuels, la capacité d'exécuter des tests manuels est toujours freinée par le manque de main-d'œuvre et de temps. Par conséquent, les équipes doivent limiter le nombre de tests manuels aux seuls tests à haut risque. Ils testent ce qui présente le plus grand risque d'échec, pas nécessairement la plus grande valeur pour le client.

Compétences requises : la connaissance du domaine est prioritaire sur l'expertise technique.

Objectif typique : création, exécution, suivi et surveillance. Avec l'exécution manuelle des tests, le coût d'exécution est élevé, il faut simplement du temps pour exécuter chaque test un par un. Pour éviter des retouches inutiles de mise en place des conditions de test, les tests sont souvent étendus pour inclure des types de contrôles multiples et variés. De plus, il y a beaucoup de mises à jour "à la volée" des tests et/ou des données de test. Ces retouches ne sont souvent pas suivies et le coût de maintenance des tests est donc sous-estimé.

Outils : Avec une focalisation sur les tests manuels, le principal outil requis est un outil de gestion des tests et peut-être un pour aider aux tests exploratoires .

Défi : Le défi des tests manuels est d'équilibrer la validation des nouvelles fonctionnalités au fur et à mesure de leur mise en œuvre par rapport à la nécessité d'avoir suffisamment de tests de régression des fonctionnalités existantes afin d'être suffisamment sûr que la version est suffisamment stable pour être livrée/publiée - d'où un risque approche basée sur les tests .

En raison de la pression temporelle constante, au lieu de vérifier chaque cas, un cas de chaque groupe de tests est sélectionné, si ce test réussit, le risque d'échec du groupe est « faible » et, par conséquent, les autres tests sont ignorés.

Très typiquement dans un contexte agile, ce scénario ressemble à une

cascade itérative.

Les tests ne commencent qu'une fois le développement terminé ou sur le point de l'être. Cela conduit également à une mentalité de silo de test, et les testeurs sont souvent externalisés ou délocalisés. Malheureusement, cela aboutit souvent à ce que les tests soient dépriorisés et/ou considérés comme une faible valeur ajoutée. Pas exactement le résultat que nous souhaitons.

Voyons comment **passer des tests manuels aux tests automatisés** .

Changement **#1** : Qualitatif plutôt que quantitatif

Si vous pouvez automatiser les nombreux tests chronophages qui "doivent" être exécutés, alors au lieu de vous concentrer sur l'exécution du plus grand nombre de tests (quantitatifs), vous pouvez plutôt vous concentrer sur des tests de plus grande valeur (qualitatifs). De cette façon, vous pouvez tirer parti de la connaissance du domaine et du savoir-faire du produit pour créer un meilleur produit.

Compétences requises : la connaissance du domaine est toujours nécessaire, mais également une compréhension technique du système testé. De plus, des compétences en automatisation des tests et/ou en développement sont également nécessaires.

Mission type : Création et maintenance des tests automatisés. Les cas de test manuels sont traduits en tests automatisés. Les tests automatisés dépendent également davantage des données stables. Par conséquent, une compréhension plus approfondie du système cible au niveau technique est nécessaire. Les tests qui échouent doivent faire l'objet d'une enquête afin d'identifier s'il y a un problème avec le test, les données, l'environnement ou l'application.

Améliorations : Le plus grand avantage des tests automatisés est que l'exécution est bon marché. Cela permet à la majeure partie des tests de s'exécuter automatiquement, ce qui permet aux testeurs d'avoir un objectif qualitatif beaucoup plus élevé. Cela peut également signifier plus de temps pour tester de manière exploratoire afin de mieux comprendre et valider le système testé. Plus de temps consacré à l'analyse qualitative signifie des conceptions de test plus avancées, conduisant à une meilleure solution de

test et donc à des niveaux de qualité plus élevés.

Outils requis : Gestion des tests , Outil de tests exploratoires , Outil d'automatisation des tests

Défis : La complexité de l'automatisation des tests est souvent sous-estimée et nécessite une compréhension technique beaucoup plus approfondie que les tests manuels. Cela signifie que nous avons besoin de personnes ayant des compétences en automatisation des tests dans l'équipe, ainsi que des ressources techniques pour aider à construire le cadre et les fondations des tests.

Avoir les tests est une chose, les utiliser en est une autre. L'un des problèmes classiques est que la capacité d'exécuter les tests « fréquemment » n'est pas exploitée. Les tests automatisés deviennent une extension de la suite de tests manuels, une case à cocher, plutôt que quelque chose qui peut être utilisé pour améliorer la qualité du système.

Un autre problème qui se pose est la redoutable pollution/contamination des tests d'automatisation. Il était une fois-là que les tests qui s'exécutaient avec succès commencent à échouer… et avant que vous ne vous en rendiez compte, le temps passé à maintenir l'automatisation des tests dépasse le temps passé à en créer de nouveaux. De multiples facteurs de contamination entrent en jeu et sont exacerbés, que ce soit du côté de l'obsolescence ou du maintien des tests par exemple... De plus, si les tests sont découplés des pipelines de construction et de déploiement, vous aurez perdu le contrôle et le combat. Ce sera fatal avant même d'avoir commencé à chercher des solutions.

C'est souvent le moment où les exercices d'automatisation des tests échouent, l'effort de maintenance semble l'emporter sur les avantages apportés par le framework.

Il sera gargantuesque et même pratiquement impossible, la mission, de regagner la confiance de l'automatisation des tests.

C'est pourquoi, il est d'autant plus important de miser sur la qualité plutôt que sur la quantité

Alors, comment passe-t-on **du test automatisé au test continu** ?

Changement **#2** : Tout tester, tout le temps au fur et

à mesure

Compétences requises : en plus des compétences nécessaires pour les tests automatisés, nous avons également besoin d'une bonne compréhension des outils et des approches DevOps . Cela inclut une amélioration itérative et continue basée sur les retours constants et rapides des tests.

Objectif typique : les tests doivent être intégrés dans les pipelines et nous devons suivre sans relâche les tests qui échouent. Une tolérance zéro pour les tests qui échouent est le seul moyen de rester au top. Les tests doivent être repensés pour être plus résilients et le dialogue avec les développeurs « s'intensifie ».

Améliorations : Les tests deviennent un outil de surveillance actif. Ils nous donnent maintenant davantage d'informations et non plus juste nous indiquer si l'application fonctionne correctement. Maintenant, nous pouvons voir à quel point l'application ou le paysage applicatif est stable, nous validons nos données, nous pouvons voir comment le système se comporte à tout moment. Des vidéos, des captures d'écran et des fichiers journaux sont rassemblés à chaque exécution et enregistrés dans notre outil de gestion des tests . Nous pouvons créer des tableaux de bord que nous pouvons offrir à n'importe quel niveau de la gestion de projet afin de voir exactement à quel point le système est vraiment stable.

Outils nécessaires : maintenant, le paysage technique commence à paraître beaucoup plus complexe

1. **Outil d'intégration continue** - Avec un cadre de test continu, le déclencheur est généralement l' outil d'intégration continue, c'est-à-dire Jenkins .
2. **Référentiel de code** - Le code de test est stocké dans le référentiel de code, il doit être traité avec le même respect que le code produit, sous contrôle de version.
3. **Automatisation des tests** - Les tests commencent alors (vous pouvez utiliser CAST, un outil systématique d'automatisation des tests à grande échelle , ou ForeCAST, un outil de génération de tests autonome basé sur l'IA)
4. **Gestion des tests** - Tous les tests et résultats doivent être poussés vers l'outil de gestion des tests

5. **Gestion des tâches** - Bien sûr, la beauté de Xray est qu'il s'intègre nativement à Jira pour notre gestion des tâches et des projets.
6. **Environnement de test** - Les tests s'exécutent dans un environnement de test
7. **Système cible** - Le système de test est notre proxy pour le système réel testé.
8. **Outil de surveillance** des journaux - Par exemple, ElasticStack où les résultats sont poussés vers un tableau de bord central, où nous collectons également tous les journaux d'application et approfondissons chaque test défaillant, voyons les modèles d'échec et avons une vue objective sur la santé de notre application.
9. **Wiki** - Confluence (si nous en avons besoin)

Défis : Un anti-modèle typique qui se pose est que le même modèle de cascade itératif continue. Les tests commencent toujours après le début du développement et ce n'est que lorsque la fonctionnalité "se stabilise" que l'automatisation des tests commence.

Comme illustré dans le diagramme, l'automatisation des tests à la fin du processus de développement rend l'automatisation des tests coûteuse, fragile et apporte le moins de valeur à l'équipe.

Il est impossible de tout tester, cependant il est alors judicieux de bien découper les validations et les exécuter au fur et à mesure sans négliger aucune des phases dans le cycle du développement.

Nous avons donc un pas de plus à franchir, **passer du test continu au test d'abord** .

Changement **#3** : basculer vers la gauche pour automatiser, intégrer et collaborer

Compétence requise : Chaque membre de l'équipe est acteur de la qualité. Nous avons besoin d'un mélange de compétences techniques et commerciales, de connaissances du domaine et, bien sûr, de DevOps.

Objectif typique : tester les exigences et non le code. Cela signifie créer des tests en amont et en faire la manière standard de faire les choses. La « plomberie » du cadre de test continu doit être maintenue à tout moment. Les tests peuvent passer au rouge jusqu'à ce qu'ils passent au vert, puis ils passent toujours au vert.

Améliorations : Maintenant, nous avons vraiment allumé les lumières. Les tests sont maintenant une documentation vivante, ils étendent la documentation des exigences pour en faire une spécification complète. Les documents vivants sont validés à chaque exécution des tests. Maintenant, nous obtenons un retour rapide sur chaque cycle.

Cette boucle de rétroaction nous aide à savoir deux choses :

10. Pour une nouvelle fonctionnalité, nous découvrons immédiatement si la fonctionnalité est "terminée"
11. Si nous avons cassé quelque chose parce que chaque document vivant est automatiquement converti dans notre suite de régression.

Outils requis : nous avons déjà tout ce dont nous avons besoin du point de vue des outils, mais nous devons être en mesure de tester et de définir toutes les couches de l'application, notamment :

- L'extrémité avant
- Back-End
- Tout ce qui se trouve entre les deux (files d'attente msg, base de données, fichiers journaux)

L'outil que nous allons utiliser pour automatiser nos tests doit être :

- Facile à utiliser
  - Nous devons permettre aux membres non techniques et techniques de pouvoir contribuer pleinement
- Techniquement complet
  - Nous devons soutenir le développement full-stack
  - Si vous avez un développement full-stack, vous devez prendre en charge les tests full-stack
- Entièrement intégré aux outils DevOps existants
  - Nous voulons tirer parti des outils DevOps existants, pas les concurrencer.
  - Nous devrions tirer parti de la relation entre les tests et les exigences pour obtenir une couverture complète

Enfin, nous devons construire l'écosystème environnant pour rendre les tests efficaces.

- Données
  - Notre approche de la gestion des données de test doit être résiliente et flexible
- Environnements
  - Nous devons avoir le contrôle total des environnements de test

- Visible
  - Nous devons nous assurer que nos efforts sont compris et peuvent être vus par toutes les parties prenantes du projet.

    C'est un travail difficile à maintenir et il faut une approche disciplinée pour le maintenir à jour et fonctionner correctement. Les avantages sont énormes, avec des améliorations massives en termes de vitesse et de qualité.

    Défis : à mesure que le nombre de tests augmente, il n'est plus possible d'exécuter tous les tests, tout le temps. Par conséquent, nous devons également "décaler vers la droite". En mettant en place des passerelles de qualité étagées, les premiers ensembles ont un parti pris vers la rapidité et les infos minimales mais suffisantes, ensuite on peut tester la fonctionnalité de manière de plus en plus exhaustive, puis passer aux tests non fonctionnels ; performances, charge, reprise après sinistre, sécurité, etc.

    C'est un travail difficile, mais si nous le faisons correctement, tout code qui sort de l'autre côté de nos pipelines est un morceau de code livrable, entièrement validé et prêt à être publié.

Pour prioriser l'automatisation des tests, il faut changer d'état d'esprit

Quel que soit le type de changement, son succès est déterminé par les outils , les processus et l'état d'esprit. Les états d'esprit que nous avons décrits ne sont pas absolus et peuvent coexister dans un projet. Si vous essayez d'atteindre les mentalités les plus avancées, les avantages sont très tangibles, ce qui signifie plus rapide, moins cher et de meilleure qualité.

Moins tangible, mais non moins importante, est l'amélioration du flux dans l'équipe de développement logiciel. Cela signifie moins de friction dans les processus, ce qui se traduit par moins de frustration et plus de plaisir au travail.

Peut-être que vous n'avez pas besoin de tout tester, tout le temps, mais avec une mentalité de test d'abord soutenue par des tests continus, la qualité et la vitesse de votre logiciel s'amélioreront considérablement.

E1. Faire rayonner le côté sombre de la pyramide - Mystérieuse

January 18, 2024 9:20 AM

Hey, les aventuriers de la pyramide perdue des mystérieuses cités d'or... la pyramide, c'est simple, non sorcier, pas magique, ni mystérieux ! 😉

Avant tout, connaissez-vous la pyramide ?

Pour mieux apprécier cette séries de "blogs", je vous suggère de connaitre d'abord ce qu'est la Pyramide de test (par niveaux)

Connaissez-vous le côté obscur de la pyramide ?

La vérité sur la pyramide c'est qu'Il est facile de tout mélanger…

Même si je comprends la tendance naturelle. Il ne faut pas s'empêcher de penser différemment, d'innover, de réinventer et de proposer quelque chose d'adapté à notre contexte. Cependant, si une pyramide devient un sapin de Noël ou une recette de pâté chinois à la banane, peut-on encore nommer ça une pyramide ? 😜

Connaît-on vraiment la vérité sur la pyramide ?! C'est peut-être une éternelle quête… Pourquoi il y a autant de versions différentes qui n'ont aucun sens ?! Voici une introduction pour en comprendre un peu, le pourquoi Histoire et évolution de la pyramide

J'ai pris le temps de me mettre dans la peau d'un archéologue pour tenter de démystifier le secret de la pyramide, mais surtout de révéler les fouilles et mettre les découvertes à jour en vue d'en savoir plus.

J'espère que cela nous permettra de trouver des solutions pour éviter de tomber dans des pièges ou même d'être victime de spéculations.

Il y a de nombreux pièges à éviter !!!

Voyons ensemble, dans ce blog, si on peut **faire rayonner le côté sombre de la pyramide**.

Avons-nous plusieurs autres options pyramidales ?

Jetons un coup d'œil aux options les plus connues ou aux souhaits qu'une pyramide ne peut pas réaliser.

Pyramide par phases (SDLC)

Est-ce seulement possible, par ordre de phases ? Est-ce que les phases sont de plus en plus petites, et il y aurait moins de tests par exemple ? Quel serait son utilité ? Quel serait l'ordre idéal selon vous ? Voyons voir...

Ici, le besoin, à mon avis, est de vouloir tester plus tôt avec une quantité de plus en plus réduite allant vers la fin en évitant de re-tester ou de sur-tester ! Même si il est vrai que le fait de découper par niveau nous permet de ne pas "sur exécuter" les tests... En est-il réellement le cas pour les phases ?! Nous avons même réussi à dénaturer le Shift-Left testing ! Donc, je préconiserais de revoir Le principe du respect du cycle de vie (Continu, Préventif et Réactif)

Pyramide par Types de test, est-ce possible ?

Manuel, UI, E2E, analyses statiques, contrôle, unitaire, uat... c'est quoi un type de test ?, est-ce la même chose qu'un niveau ou une approche ou une tactique ou une technique une méthode, un framework... ? Commençons par connaître les Types de test et les Niveaux de test 😊 Quel serait l'ordre idéal selon vous ? Lors de la construction d'une maison quel est l'ordre idéal pour la pose du plancher, des murs et du toit ? Ici, le besoin, à mon avis, est de vouloir un portrait global de couverture pour mieux faire le suivi, donc être proactif et cibler les efforts. Trouver les bonne ressources spécialistes du domaine relatif au test... donc, oui le quadrant plus que la pyramide... Je préconiserais plutôt la pratique WD-6 (le 6W du Lean Six Sigma ou le 5W1H) qui est, pour moi, plus instinctive et plus efficace. Mais un empêche pas l'autre 😊 l'important c'est de bien maîtriser les types !

Pyramide sur mesure ?

J'aime cette vision qu'une pyramide est un graphique de résultats statistiques... Est-ce réellement une représentation des résultats (diagrammes et métriques) ? À quoi ressemblerait la pyramide si nous mesurions le contexte sur une période donnée, et quelles seraient les statistiques idéales à atteindre, quelle forme aurait-elle ? La forme absolue, la priorisation absolue, le pourcentage absolu, ou juste un contexte actuel & futur. Est-ce que vous seriez étonné si l'on vous disait que la pyramide (modèle stratégique de la qualité) est variable selon le contexte des connaissances, pratiques ou technologies utilisées. Tout va dépendre de la maturité/capacité de la modélisation et la gestion des validations, environnements de test, mock, automatisation (pipelines et outils) ... Ex. Restons dans la forme d'un triangle pour garder le "low, medium, high" voici l'exemple qui est difficile à croire, mais qui est totalement VRAI selon les différents contextes de qualité optimisée mesurable :

Pyramide parfaite ?

Mais que faisons-nous au moment où nous avons réalisé qu'il n'y a pas de pyramide parfaite absolue qui fonctionne pour toutes les situations et qu'en plus elle se transforme, évolue et s'adapte… ça sonne très Agile.

Pyramide Agile ?

Si nous voulions une pyramide Agile, elle serait claire, itérative, évolutive, flexible, aidante à la collaboration et à l'entre-aide… donc, alors là, quel serait le but d'une pyramide ?!

Pyramide quantitative ?

Une plus grosse pyramide ou une plus petite ? Si on part du cornet et qu'on veut devenir une pyramide… Est-ce qu'on garde la même couverture des tests E2E (système) et il devrait juste avoir EN PLUS à retester en intégration et la même chose en unitaire ?

Pyramide sapin de noël ?

Revenons encore un peu au "Sapin de noël", quel serait l'ordre et regroupement idéal selon vous de cet arbre de Noël ?

Quels seraient les cadeaux sous ce sapin ?

Une pyramide One-size-fits-all, le grand mixte (couteau suisse) existe, mais je dois vous dire que ce n'est d'aucune utilité.

Ici, le besoin, à mon avis, est de vouloir un portrait global de couverture pour mieux faire le suivi, donc être proactif et cibler les efforts.
...Je préconiserais plutôt la pratique de la qualité par la modélisation/architecture (digital twins)
En Agile nous avons le framework de Développement piloté par la fonctionnalité (FDD) un parent de DDD (software design approach), avec la méthodologie Agile modeling (AM)

La pyramide mobile ou spécifique ?

Si vous êtes en constante évolution ou avez plusieurs équipe travaillant dans de différents contextes ou êtes en amélioration continue, que devrions-nous faire ? Une pyramide pour chaque contexte ? Une pour quand on fait du JavaScript, une autre pour le mobile ? Et oui, nous voyons l'existence de nouvelles pyramides comme le saint Graal (trophy) ou la pyramide mobile, etc. Que nous parlions de l'utilisation du saint Graal (trophy) ou de la mythique Pyramide... Ce sont des théories, des idées générales... Si nous devons adapter la pyramide au contexte, alors, quel est le but de son existence ?

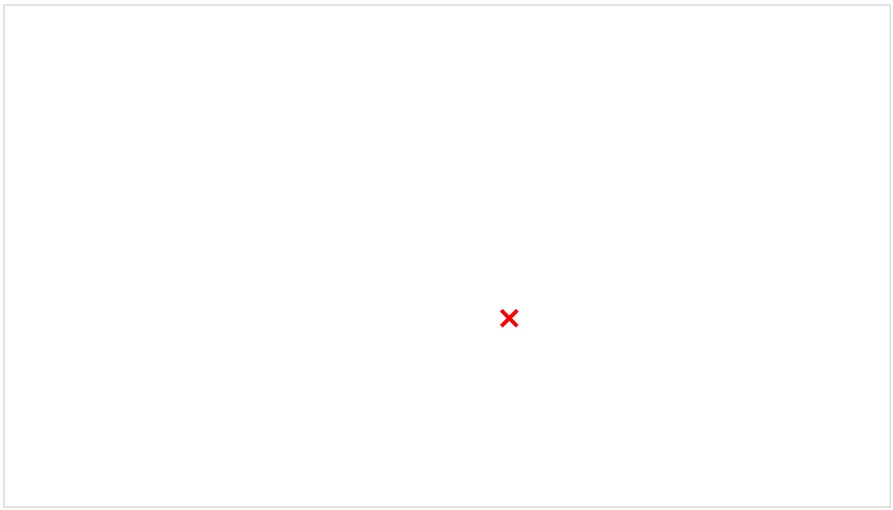
Avons-nous réellement d'autres options ou simplement une option valable !?

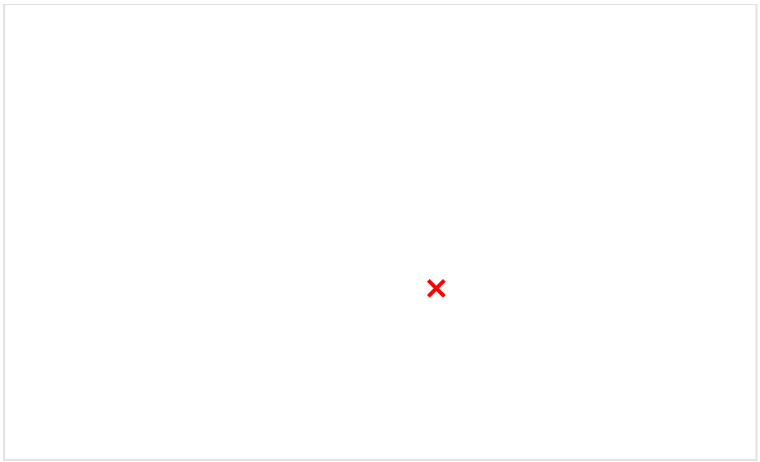
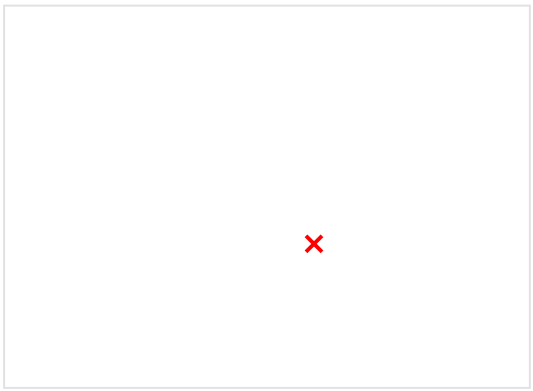
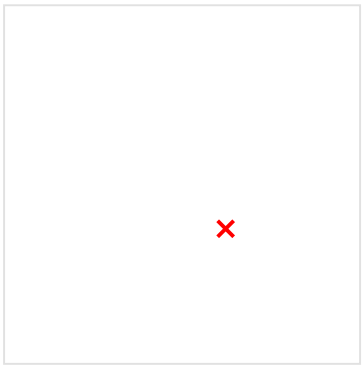
à suivre dans la prochaine épisode...

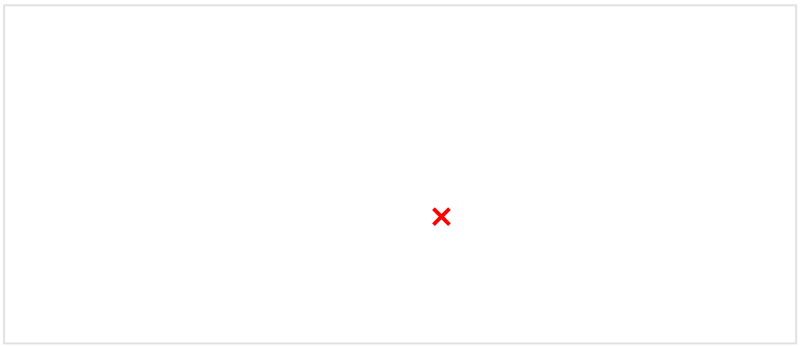
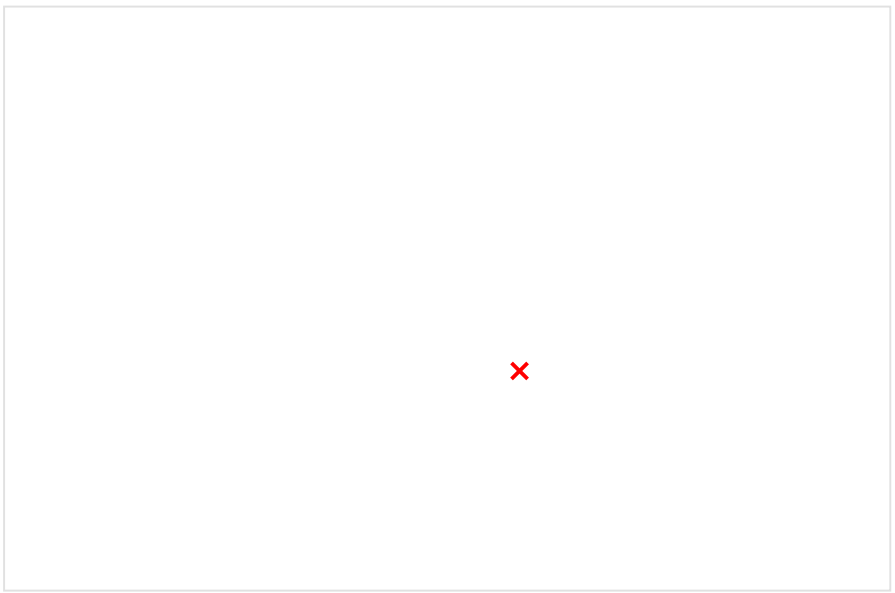
E0. Histoire et évolution de la pyramide

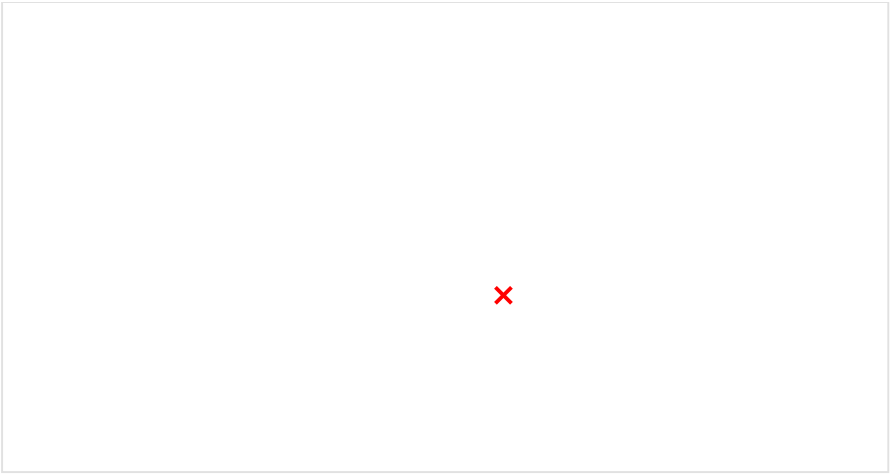
January 18, 2024 9:21 AM

?. Pourquoi existe-t-il un nombre infini de pyramides?
?. D'où vient t-elle?
?. C'est quoi le lien avec l'automatisation?
?. Quel lien avec l'Agile?
...
!. Cet article pourrait vous éclairer

	À l'aube de l'automatisation des tests, la qualité n'était encore qu'un bébé, nous étions au contraire voire très peu automatisés. S'il y avait automatisation, l'accent était mis uniquement sur l'automatisation des tests utilisateurs. Bon au début c'était mieux que le manuel, mais cette piste, nous savons, est parsemée de multiples pièges très chers. Nous nous sommes vite rendu compte que ce n'était pas en un claquement de doigt que cela ce ferait et que nous avions maintenant besoin de nouveaux spécialistes rares du genre hybride QA-DEV (SDET). L'automatisation des tests est une nouvelle porte de pandori dangereuse !
	Une solution simple, nous pouvions sauver instantanément beaucoup, c'était l'évidence même,

	nous pouvions sauver instantanément beaucoup, c'était l'évidence même, qu'il fallait investir sur les tests unitaires avant les tests manuels… Dans le but d'aider la cause de l'automatisation, c'est ainsi qu'est née la fameuse pyramide !
	Il était clair dès le départ que les choses n'allaient pas avec la pyramide, mais la métaphore était encore partiellement vraie et démontrait l'idéal de l'économie et d'efficacité. Chacun l'adaptait en fonction de son contexte, pas standardisé, blizzard de compréhension, etc. La perte de confiance généralisé, on avait tous envie de lancer la pyramide à l'adoption !
	L'intervention "à Gille" (Agile) est arrivée ! Voici la pyramide des **tests automatisés** illustrée par Mike Cohn, l'un des fondateurs du mouvement Agile, présentent cela en particulier dans le livre Succeeding with Agile (en anglais). *En utilisant cette métaphore, l'auteur préconise la répartition de tests dans un projet de développement agile*

	qu'il avait participé. Il partage que cela l'avait grandement aidé !
	Bien qu'en proposant une simplification et enlevant les tests manuels de la donne, cela semblait encore une fois incompris. "Belle théorie, mais impossible à appliquer." le cône le modèle anti-paterne revient de plus belle, il y a de plus en plus de confusion !!!! Chacun se fait sa propre pyramide mélangeant/incorporant toutes sortes de concepts de la qualité, stratégie, phases, mesures, etc. !
	Je ne crois pas que c'est le genre de pyramide que nous recherchions !

	En ce jour, les niveaux de test ont été standardisés, les règles sont plus encadrées, c'est plus clair et les solutions à l'automatisation et les meilleurs pratiques sont nombreuses. Vous pouvez voir, ici, la Pyramide de test (par niveaux)

CONCLUSION

La pyramide persiste encore aujourd'hui dans sa popularité parce quelle est mythique !
Ceci en dépit d'être pratiquement inutile ou pour causer plus de mal que de bien avec des pièges très coûteux au détriment de la qualité.

La cause de tout ce chaos ?!

C'était bien sûr parce qu'on mélangeait les phases du cycle de développement, les types de tests, on voulait un couteau suisse… donc bref, l'incompréhension générale des 3 niveaux de tests par dessus tout !

Rassurez-vous, vous pouvez éviter les nombreux écueils ou éviter de vous retrouver dans une pyramide indésirable en consultant l'épisode suivante !
E1. Faire rayonner le côté sombre de la pyramide - Mystérieuse

Enfin nous avons belle et bien une pyramide aujourd'hui, claire et qui n'est ni un entrepôt de n'importe quoi, ni une règle illogique à suivre, ni un cornet… C'est tout simplement la Pyramide de test (par niveaux)

10 règles d'or de l'assurance qualité

January 18, 2024 9:21 AM

Ce ne sont pas des commandements bien sûr, mais une énumération non priorisée de points importants pour viser le succès dans la pratique de l'assurance qualité.

1. Tester est un vrai métier, il faut de vrais rôles de testeurs distincts des programmeurs.

2. Le test ne doit pas être sous-évalué, de l'ordre de 25 à 30% de l'effort d'un projet de développement.

3. L'environnement de test et les jeux de données de test doivent être définis, configurables, à la demande, stables et accessibles en mock ou non le plus tôt possible dans le cycle de vie pour éliminer les erreurs de conception et faciliter le travail des équipes de test/développement.

4. Une application doit toujours être conçue de manière à être testable, et un bon découpage en composants est une aide précieuse sur ce point.

5. Il convient d'investir sur toutes les dimensions du test y compris l'acceptation par les utilisateurs et l'ergonomie.

6. Il faut définir les conditions d'exploitation du système et tester les configurations, la montée en charge ou le comportement en cas de coupure réseau et non le moindre, qui est le moins testé → "la sécurité".

7. C'est toujours une bonne idée d'investir sur l'automatisation pour l'ensemble des tests et en particulier pour valider la non-régression. L'automatisation des tests est aussi un vrai métier, il faut de vrais rôles d'automaticiens distincts des QA et programmeurs.

8. Les plans de test sont du développement qui font partie de la gestion des

versions/livraisons.

9. Comme pour le développement, un travail partiel bien ciblé permet de couvrir l'essentiel selon une règle de 80/20. Il vaut mieux commencer par cela et améliorer par itération, que chercher à tout prix une exhaustivité utopique.

10. Être stratégique en établissant nos conventions, maîtrisant l'adoption des meilleurs pratiques et évitant les anti-patterns.
    Voici l'exemple d'une super bonne pratique :
    Le décalage vers la gauche présente de nombreux avantages autres que de faciliter les validations et les efforts de développement sur une base continue (valider au fur et à mesure).

Un sourire est le premier geste de la qualité

January 18, 2024 9:21 AM

La qualité n'est pas seulement un "niveau d'excellence" ou un "respect des exigences" ou un "diagnostic de ce qui ne va pas"

Ci-dessous figure une liste non exhaustive d'éléments à prendre en compte lorsqu'on parle de qualité :

- Le bonheur des clients qui reçoivent le produit et qui sont ravis de l'utiliser.

- La plupart des modèles de qualité se concentrent fortement sur le "processus" et moins sur la "bonne chimie de l'équipe".

- Le bonheur des personnes travaillant sur le développement de produits a un impact direct sur la satisfaction des clients.

- N'oublions pas les clients internes satisfaits (pairs, subordonnés, membres de l'équipe de projet, service de test...) conduisent finalement à des clients externes satisfaits.

- Lorsque les gens croient en la vision, l'objectif et ont une fierté du produit qu'ils développent, ils en donneront plus tout en continuant d'offrir la qualité.

- Une bonne cohésion, un esprit d'équipe, une bonne communication et la complicité sont également synonyme de la qualité.

- Gardons simplement à l'esprit, que les gens heureux sont plus susceptibles apprécier le niveau de qualité et/ou de produire une meilleure qualité.

  Nous avons survolé la gentillesse, la fierté, l'entraide, la confiance, la satisfaction...
  Veiller sur la qualité c'est bien plus que tester, c'est d'abord trouver la façon d'être positif en commençant par un simple sourire 😊

Coaching et relation de confiance

January 18, 2024 9:22 AM

Être au service de l'autre c'est seulement possible lorsqu'on est en mesure d'établir un rapport de confiance avec la personne qui va recevoir le service. Pensez au pompier qui doit convaincre un homme ayant le vertige de monter dans son échelle. Le pompier est prêt à donner son aide mais s'il n'arrive pas à gagner la confiance de cet homme, il ne pourra pas le sauver du feu. C'est un cas extrême de vie ou mort et, au même temps, il nous montre qu'un pompier expérimenté saura établir la confiance en quelques secondes. Comment arrive-t-on à installer cette confiance? Eh bien, c'est ce dont je veux vous parler dans cette publication.

Le rapport de confiance constitue la base des relations significatives, étroites et harmonieuses, entre les personnes. C'est le sentiment de connexion que vous obtenez lorsque vous rencontrez quelqu'un d'agréable et qui vous inspire confiance. Vous avez le sentiment que cette personne comprend vos points de vue. C'est le lien qui se forme lorsque vous découvrez que vous partagez des valeurs et des priorités avec une autre personne.

Lors du coaching individuel il est très important d'établir un rapport de confiance, de sécurité et de respect avec le coaché. S'il n'y a pas ce lien, l'intervention du coach est vouée à l'échec.

Afin d'aider notre coaché, il nous faut participer activement dans la construction d'un rapport positif. Comme les bases du rapport commenceront à s'établir naturellement pendant la première rencontre, il est important d'avoir une vision générale des techniques de communication. Si ces techniques ne vous sont pas familières, cette vidéo saura vous éclairer: Le rapport en PNL (9:30 min)



Le rapport doit être une connexion à double sens entre le coach et le coaché, donc ce n'est pas quelque chose que vous pouvez créer vous-même unilatéralement.

Vous pouvez cependant apprendre à le stimuler en suivant ces étapes :

1.- Créer une ambiance accueillante

Les premières impressions comptent et votre apparence doit vous aider à vous connecter avec les gens et non à créer une barrière. Soyez conscient de votre apparence et de l'environnement dans lequel vous vous trouvez avec votre coaché. Votre posture, une allure soignée et détendue, le ton et le rythme vocal que vous utilisez, sont tous des éléments qui agrémenteront et faciliteront la rencontre. Votre environnement doit également être chaleureux et accueillant (prenez soin de choisir un fond d'écran virtuel propice à cette discussion).

2.- Rappelez-vous les bases de la bonne communication

- Soyez culturellement approprié. Il faut comprendre le contexte culturel de votre coaché.
- Lors de présentations souriez.
- Relaxez.
- Rappelez-vous le nom et prénom de votre coaché, son titre, son pays d'origine, etc. Si vous n'avez pas ces informations, posez des questions ouvertes.
- Gardez une posture positive et constructive pendant l'entretien.
- Écoutez attentivement.

Ces principes de base constituent le fondement d'une excellente communication, ils vous aideront à établir la confiance et l'empathie chez les gens que vous écoutez.

3.- Trouver un terrain d'entente

Identifier un terrain d'entente peut aider à établir un rapport, alors utilisez de petites discussions pour trouver quelque chose que vous partagez avec votre coaché.

La plupart des gens aiment parler d'eux-mêmes, et plus vous vous intéressez sincèrement à eux, plus ils sont susceptibles de se détendre et de « s'ouvrir ». Utilisez des questions ouvertes pour découvrir des informations personnelles : vous partagez peut-être les mêmes passe-temps, ou aimez les mêmes fruits. Le simple fait d'exprimer votre frustration commune face à la circulation qui a retardé vos trajets vers le travail peut vous aider à vous rapprocher de quelqu'un.

Il est important d'être authentique et sincère et d'éviter d'en faire trop. N'inventez pas un intérêt juste pour créer un rapport. Non seulement cela peut sembler rebutant, mais cela peut aussi nuire à votre crédibilité !

4.- Créez des discussions sur un thème commune

Le rapport ne peut pas grandir sans interaction humaine, et une excellente façon d'interagir est de focaliser sur des thèmes intéressant le coaché. Le visionnement des vidéos motivationnels ou la lecture des livres ou des articles sur internet, peut servir d'objet de discussion dans les rencontres et apporter une vision commune. N'hésitez pas à partager des lectures ou vidéos qui vous ont inspiré.

5.- Soyez empathique

L'empathie consiste à comprendre les autres en voyant les choses de leur point de vue et en reconnaissant leurs émotions. Ainsi, pour comprendre et partager le point de vue d'une autre personne, vous devez apprendre ce qui la motive. Comme nous l'avons déjà mentionné, beaucoup de gens aiment parler de ce qu'ils aiment et n'aiment pas, de leurs besoins et de leurs désirs, de leurs problèmes et de leurs succès, alors posez des questions ouvertes et donnez-leur de l'espace pour parler.

Vous devez vraiment entendre ce qu'ils disent, afin de pouvoir répondre intelligemment et avec curiosité. Il est donc important d'avoir une bonne écoute et d'affiner votre intelligence émotionnelle.

6.- Prenez conscience de la complexité de la communication

La façon dont vous communiquez ne se limite pas à ce que vous dites. Les mots ne représentent que sept pour cent de notre communication sur les émotions ou les attitudes. Le ton, le volume et le rythme de notre voix représentent un pourcentage plus élevé (38 %). Notre langage corporel représente jusqu'à 55 %. Donc, vous manquerez une astuce si vous ne considérez pas "l'image globale" de la communication humaine.

7.- Soyez conscient de vos propres sentiments et pensées.

Le coach est aussi un être humain: il a des émotions, des préjugées, des frustrations, du stress, etc. C'est pourquoi il est important d'apprendre des techniques tels que la pleine conscience, la méditation, le tai-chi, le yoga, etc. Ces techniques nous aident à comprendre nos propres limites et à les gérer efficacement. Il peut aussi s'avérer intéressant pour un coach de se faire de coacher par un collègue dans certaines situations. Plus le coach est ouvert à comprendre et à intégrer calmement son propre univers des sentiments et pensées, plus il sera disponible pour accueillir les sentiments et pensées de ses

coachés.

La suite?

Pensez au pompier qui a convaincu un homme ayant le vertige de monter dans son échelle. Le pompier et cet homme doivent se rendre maintenant au pied de l'échelle. Ils sont deux à descendre petit à petit vers la sécurité.

Le coach et son coaché sont deux également. A chaque pas, ils sont mis à une épreuve: éveiller la confiance, la bienveillance et le courage dans la relation du coaching et dans ses résultats positifs.

"Grooming" un mot à ne plus utiliser ?

January 18, 2024 9:22 AM

Vous allez me dire, c’est quoi le lien du "Grooming" avec l’assurance qualité ?!

En assurance qualité, c’est une étape cruciale!
On parlera souvent de la définition des spécifications, du besoin, solutions, de la couverture…
Vous avez surement entendu aussi, "3 amigos" ou BDD, ou la définition de terminer (DoD) ou la "Définition de Prêt" (DoR).

ex. une "story" qui doit avoir tout ce dont elle a besoin pour être "testable" et prêt pour s'engager dans un prochain sprint.

Origine selon le Scrum guide









Voici la référence web du Scrum guide: [Scrum Guide | Scrum Guides]

POURQUOI donc, ne pas utiliser le mot

"GROOMING" quand même dans le cadre de la mêlée de la planification?!

Ok, on pourrait s'arrêter là et dire simplement, soyons fidèles aux mises à jour du Scrum guide et donc utilisons, maintenant, le bon terme soit "**Refinement**"
En français: raffinement ou affinage.
Mais pour comprendre le pourquoi, allons voir plus loin ensemble!

Si vous faite une traduction mot pour mot on vous donnera le mot inoffensif "TOILETTAGE"

"TOILETTAGE" pour ceux qui ne connaisse pas ça, est un mot, ancien, utilisé pour se laver, se mettre propre, se parfumer avec de l'eau de toilette, se maquiller, se trimer, se couper/peigner/brosser les cheveux et/ou la barbe, couper/limer les ongles, prendre soin, se soigner, etc. donc, bref, se refaire une beauté à la toilette.
Oh!, bien oui, la toilette n'étant pas le bol ou la cuvette de toilette mais bien une pièce comparable à la salle de bain d'aujourd'hui.
Néanmoins, aujourd'hui, ce mot est plus utilisé et représentatif dans le contexte des animaux, autres que les humains et on ne fait pas du toilettage dans une toilette.
C'est pourquoi, "faire du raffinement" fait, donc, plus de sens dans le contexte du logiciel, plutôt que dire: "allons refaire une beauté au 'backlog'". (Quoi que j'aime bien, c'est amusant)
À titre informatif, c'est la même chose en anglais.

Cependant, la raison la plus convaincante à mes yeux est la suivante…

ATTENTION!

Certain mot peut cacher une subtilité dans son évolution, comme le mot "rat", "requin", "chien/chienne", "vache", "cochon/cochonne" ou encore "gai" par exemple, qui veut dire qu'une personne rit, s'amuse, est joyeuse, de bonne humeur, qui est enjouée, souvent prête à rire, toujours de bonne humeur… par contre aujourd'hui nous n'utiliserons pas ce mot pour dire "joyeux" mais plutôt pour dire "homosexuel"
Ici le mot en anglais est très proche -> "Gay" donc l'évolution c'est fait plus rapidement.

Dans d'autres cas un mot peut choquer, être vulgaire, et même être proscrit !
C'est pourquoi, pour le mot "Grooming", je ne peux pas vous dire la définition ici.
Cela dépasse le niveau professionnel.
Je peux juste vous dire que si vous faite une simple recherche sur Google

par exemple, vous allez vite tomber sur le sujet.

Maintenant que nous savons…

C’est l’une des raisons du pourquoi, que j’ai écrit ce blogue aujourd’hui ! Je n’avais aucune idée de l'inconfort/irritant que je pouvais causer lorsque je parlais à un anglophone ?!
Personnellement, depuis le jour où j'ai su ça, à chaque fois que j'entends le mot "Grooming", ça me met à l’envers et me bouleverse à chaque fois ! J’ai beaucoup de difficulté de ne rien dire, juste de ravaler et subir mes émotions…

Et vous ?

Êtes-vous en accord, est-il important non seulement d'arrêter de dire ce mot mais aussi de le proscrire et même de se forcer et d'insister pour qu'on n'utilise plus ce mot pour une industrie mondiale anglophone (software development), même ici au Québec !?

N'hésitez pas à laisser vos commentaires :)

Quiz éclaire - Pyramide de test

January 18, 2024 9:28 AM

Section 1

1.Quelles sont tous les niveaux de la pyramide de test ?Réponse requise. Choix multiples.
(15 points)

Manuel
UAT
UI
Bout en bout
Fonctionnel
Intermédiaire
Intégrité
Intégration
Système
API
Composent
Automatisé
Affaire
Domaine
Flow
Non-fonctionnel
Test de fumée
Test de déploiement
Test d'installation
Test de donnée
Test de configuration
Test unitaire de code
Analyse de code statique
Analyse des flux de données
Test d'analyses de contrôle du développement
Analyse du DOM
Analyse des métriques
Analyse de couverture de code
La traçabilité
La journalisation
Compatibilité
Internationalisation et localisation
Test destructifs/choas (résilience)
Test d'utilisation
Test de performance
Test d'accessibilité
Test de sécurité
Test de concurrence/collision
Test de non-régression
Test d'acceptation

Test continus
Test A/B
Test simultanés
Test de conformité
Test de conformité
métamorphique
par propriété
Définition
Spécification
Critères d'acceptation
Unitaire
Veiller
Opérer
Diffuser
Analyser
QA
Planifier
Définir

Section 2

Section

2.Combiens il y a de pyramide différentes ?Réponse requise. Choix unique.

(5 points)

Une seule
Une dizaine
Une cinquantaine
Une centaines
Une infinité

Section 3

Section

3.La pyramide est divisée en combiens de types de test ?Réponse requise. Choix unique.

(3 points)

3
4
entre 5 et 10
Au choix
Aucune de c'est réponses

Section 4

Section

4.La pyramide est divisée en combiens de phases ?Réponse requise. Choix unique.

(3 points)

3
4
entre 5 et 10
Au choix
Aucune de c'est réponses

Section 5

Section

5.La pyramide est divisée en combiens de niveaux ?Réponse requise. Choix unique.

(5 points)

3

4

entre 5 et 10

Au choix

Aucune de c'est réponses

Section 6

Section

6.Quel est l'ordre de ces niveaux de haut vers le basRéponse requise. Classement.

(5 points)

Unitaire

Intégration

Système

Section 7

Section

7.Est-il vrai que la pyramide de test est un outil, une approche pour planifier et prioriser les testsRéponse requise. Choix unique.

(5 points)

Oui

Non

Pas seulement ça

Section 8

Section

8.Est-il vrai que la pyramide de test est une représentation visuelle des différents niveaux de tests dans un projet de développement logiciel. Elle montre comment les tests unitaires, les tests d'intégration et les tests système s'imbriquent les uns dans les autres pour fournir une couverture de test complète. La pyramide de test peut aider les équipes à comprendre le découpage des niveaux de test et à optimiser la quantité de tests en éliminant les tests superflus.Réponse requise. Choix unique.

(15 points)

Oui

Non

Pas seulement ça

Section 9

Section

9.Est-il vrai que le pourcentage idéal de la pyramide est 10-20-70 ?Réponse requise. Choix unique.

(5 points)

Oui

Non

Oui, mais le bon pourcentage

Section 10

Section

10.Est-il vrai que la pyramide de test est un "graph" selon des statistiques de la couverture ?Réponse requise. Choix unique.
(4 points)
Oui
Non
Oui, mais pas seulement ça
Section 11
Section
11.Est-il vrai que la pyramide de test mobile et JS ressemble plus à un trophée ou une coupe ?Réponse requise. Choix unique.
(5 points)
Oui
Non
Oui, mais pas seulement ça
Section 12
Section
12.Est-il vrai que la pyramide de test est un concept de fondation du genre plancher, murs et toit ou pyramidale ?Réponse requise. Choix unique.
(5 points)
Oui
Non
Oui, mais pas seulement ça
Section 13
Section
13.Est-il vrai que le niveau unitaire est F.I.R.S.T. ?

- Foudroyant (Fast/Rapid)
- Isolé (Isolated/Independent)
- Répétable (Repeatable)
- Souverain (Self-Validated/Verification)
- Temps opportun (Timely/Carefully)

Réponse requise. Choix unique.
(5 points)
Oui, juste pour unitaire
Non
Oui, mais pas seulement unitaire, c'est pour tous les niveaux
Section 14
Section
14.Est-ce pour tous les contextes/réalités de développement ?Réponse requise. Choix unique.
(5 points)
Oui
Non

Section 15

Section

15.Est-ce possible d'utiliser la pyramide dans un contexte de développement que nous pouvons seulement tester en boite noire ?Réponse requise. Choix unique.
(5 points)
Oui
Non

Section 16

Section

16.Est-ce que le mock à un lien direct avec les niveaux de la pyramide ?Réponse requise. Choix unique.
(5 points)
Oui
Non

Section 17

Section

17.Est-ce vrai que l'on peut tester des mêmes types de test sur les trois niveaux, comme par exemple les types UI, Performance, UAT ou acceptation, etc.Choix unique.
(5 points)
Oui
Non

Section 18

Section

Question bonus !
18.Est-il vrai que les tests unitaires sont plus importants, plus prioritaires que les autres niveaux ?Choix unique.
(5 points)
Oui
Non

Section 19

Section

Question bonus !
19.Est-il vrai que plus que nous allons vers le haut de la pyramide, plus que les tests sont lents et coûteux ?Choix unique.
(5 points)
Oui
Non

Section 20

Il reste à soumettre le quiz :)

Ajouter

3 amigos (Y El Guapo)

January 18, 2024 9:29 AM

Le but est de rassembler tous les acteurs impliqués afin de comprendre le besoin et définir le comportement attendu du fonctionnel logiciel, suivi de la solution adoptée.

Ensemble, nous allons créer les spécifications sous forme de validations ce qui nous donnera un plan de test.

Le développement sera ensuite piloté par ce plan.

Implicite au plan de test :

- Story/Epic
- Version
- Requis
- Exigences
- Critères de succès
- Critères d'acceptation
- DoD (définition de terminer)
- Release notes
- ...

Les 3 amigos sont un trio constitué de **représentants** développeurs, de testeurs et de Business Analysts représentant le métier sous-jacent du produit à construire. Nous parlons ici de **représentants** (les acteurs impliqués), cela ne se limite pas à toujours les mêmes 3-4 personnes (PO, DEV, QA), cela peut aussi être le UX, le DevOps, l'Architecte, l'utilisateur, un SME, c'est à dire toute personne qui sera utile pour répondre aux questions lié aux livrables.

Ils travaillent ensemble pour exprimer les besoins dans un langage commun, compréhensible par tous, peu importe le niveau de maîtrise fonctionnel de chacun. Cet échange permet aux intervenants d’apporter leur vision, qu’elle soit fonctionnelle ou technique, et de mettre en exergue certains points clés devant être définis pour mettre en place la fonctionnalité.

C'est une "réunion/atelier/cérémonie/mêlée Agile" qui est **récurrente** au début du cycle de développement, à la phase "DÉFINIR".

Règles du jeu

3 AMIGOS			**L'APOCALYPSE**
Représentant DevOps, Architecte	Représentant Affaire, Utilisateur	Représentant Assurance qualité, Agile	Représentant Chaos, Hacker éthique (ingénierie du chaos)
Assure la définition de la solution (développement/intégration, diffusion/opération…)	Assure la définition du besoin (comportements, feedback/satisfaction…)	Assure la définition de la prévention et mesure (performance, efficacité, standard…)	Assure la définition de la force/fiabilité (fortification/protection, sécurité/confiance, solidité/durabilité…)

    1. Selon le tableau, chaque séance commence par la création des équipes pour chaque rôles
    2. Les 3 amigos devront étoffer ensemble le plan de test de la livraison au fur et a mesure de la séance.
1. À tout moment, le représentant du chaos cherche les ennuis, il pense en dehors des sentiers battus.
   L'intention n'est pas de ralentir la séance mais d'interpeller toutes les parties avec des sujets souvent oubliés ou non évoqués comme la sécurité, la performance, est-ce une valeur ajoutée, comment ça fonctionne si je suis aveugle, si j'ai entré une balise html dans la zone de texte, si je crée 2 comptes et que je fais...
2. Le représentant CLIENT explique son besoin
    1. À tour de rôle chacun apportera sa vision avec des exemples du besoin dans le but de tous s'entendre sur le même besoin
3. Le représentant DEV explique la solution
    1. À tour de rôle chacun apportera sa vision avec des exemples de la solution dans le but de tous s'entendre sur la solution
4. Le représentant QA explique la prévention de la qualité
    1. À tour de rôle chacun apportera sa vision avec des exemples de la prévention dans le but de tous s'entendre sur la solution
5. Compléter et mettre au propre le plan
6. Le jeu est terminé lorsque les 3 amis votent pour et acceptent de s'engager

sur le plan.

N.B. Cela peut s'étendre sur plusieurs séances

Analyse de la cause première (RCA) & La prévention des bugs (bug prevention)

January 18, 2024 9:29 AM

RCA signifie “Root Cause Analysis” en anglais et se traduit en français par “Analyse de la cause première” ou “Analyse des causes profondes”. C’est une méthode d’analyse qui vise à identifier les causes profondes d’un problème plutôt que les symptômes. L’objectif est de trouver la cause première du problème pour pouvoir le résoudre de manière permanente.

La prévention des bugs est une approche qui vise à prévenir les bugs plutôt que de les corriger après coup. Elle consiste à mettre en place des processus et des pratiques pour éviter que des bugs ne se produisent dans le code. Cela peut inclure des pratiques telles que l’utilisation de tests automatisés, la revue de code par les pairs et l’utilisation de normes de codage strictes.

Anti-patterns

January 18, 2024 9:30 AM

Les anti-patterns sont la mauvaise utilisation des pratiques courantes qui sont considérées comme inefficaces ou contre-productives. Les anti-patterns peuvent inclure la mauvaise utilisation des pratiques telles que la documentation excessive, la sur-ingénierie, la réactivité excessive et la dépendance excessive aux outils.

En revanche, les patterns (meilleurs praiques) d'assurance qualité sont des pratiques courantes qui sont considérées comme efficaces et productives dans le domaine de l'assurance qualité. Les patterns peuvent inclure des pratiques telles que l'automatisation des tests, la collaboration étroite entre les équipes de développement et de test et l'utilisation d'outils de gestion de projet/test.

ref. :

- 27 Sprint Anti-Patterns | Scrum.org
- Anti-patterns You Should Avoid in Your Code (freecodecamp.org)
- AntiPatterns (sourcemaking.com)
- Anti-pattern - Wikipedia
- 5 Agile Anti-Patterns to Avoid with Solutions | Net Solutions
- DevOps Fragility, Antipatterns, and Consequences - Developer Support (microsoft.com)
- LCK#100 (tardate.com)
- Anti-pattern - Wikipedia
- How to Avoid 8 Common Agile Anti-Patterns Hurting Your Team - BuildingBetterSoftware

Artefact (artifact)

January 18, 2024 9:30 AM

En développement logiciel, un artefact est un terme général désignant toute sorte d'information créée, produite, modifiée ou utilisée par les travailleurs dans la mise au point du système. Les artefacts sont des produits tangibles qui résultent du processus de développement logiciel et peuvent aider à décrire la fonction, l'architecture ou la conception même du logiciel. En UML, un artéfact désigne n'importe quelle entité qui intervient dans la réalisation ou l'utilisation d'un système logiciel : code source, graphiques, schémas de base de données, documents texte, diagrammes, modèles, etc.

Artifact (software development) - Wikipedia

ATDD (Acceptance Test-Driven Development)

January 18, 2024 9:30 AM

L'ATDD est une méthode de développement logiciel qui consiste à écrire des tests automatisés pour vérifier que l'application répond aux exigences des utilisateurs. Les tests sont écrits en collaboration avec les parties prenantes pour s'assurer que les exigences sont claires et précises.

BDD versus ATDD

Le BDD (Behavior Driven Development) et l'ATDD (Acceptance Test Driven Development) sont deux méthodes de développement logiciel qui se concentrent sur le comportement attendu d'un système plutôt que sur les détails techniques de sa mise en œuvre. Cependant, il y a une différence subtile entre les deux.

Le BDD se concentre sur le comportement du système dans son ensemble, tandis que l'ATDD se concentre sur le comportement de chaque fonctionnalité individuelle.

Le BDD est souvent utilisé pour les tests d'acceptation à haut niveau, tandis que l'ATDD est souvent utilisé pour les tests d'acceptation à bas niveau.

Les tests d'acceptation à haut niveau sont des tests qui vérifient que le système répond aux exigences globales de l'application. Ils sont souvent utilisés pour les tests d'acceptation utilisateur et se concentrent sur les fonctionnalités de haut niveau de l'application.

Les tests d'acceptation à bas niveau sont des tests qui vérifient que chaque fonctionnalité individuelle de l'application répond aux exigences spécifiques de cette fonctionnalité. Ils sont souvent utilisés pour les tests unitaires et se concentrent sur les détails techniques de la mise en œuvre de chaque fonctionnalité.

- J'aime
- Commentaire
- 4 vues
- Enregistrer pour plus tard

Audit

January 18, 2024 9:31 AM

Il s'agit d'une activité d'inspection/évaluation qui vérifie la conformité aux plans, politiques et procédures,
et veille à la préservation des ressources. L'audit est une fonction du personnel; il sert "d'yeux et d'oreilles" à
gestion.

Auto-documentation (self-documenting)

January 18, 2024 9:31 AM

“Self-documenting” est un terme utilisé pour décrire un test, du code ou tout autre documentation qui est facile à comprendre et à maintenir sans documentation supplémentaire. Le code est écrit de manière à ce que les noms des variables, des fonctions et des classes soient explicites et que le code soit structuré de manière à être facilement compréhensible sans avoir besoin du recours de la description et/ou les commentaires. Cela permet de comprendre rapidement et de le modifier facilement si nécessaire.

Automatisation

January 18, 2024 9:31 AM

Exécution totale ou partielle de tâches techniques par des machines fonctionnant **sans intervention humaine**. Comme dans automatiser l'exécution d'une tâche, d'une suite d'opérations, etc.
L’automatisation est le fait d’automatiser, de rendre automatique l’exécution d’une tâche ou d’une suite d’opérations, sans intervention de l’homme. Cela inclut l’automatisation des processus métier (BPA, Business Process Automation), l’automatisation de l’informatique, les applications personnelles telles que la domotique et plus encore. L’automatisation est un procédé ou un ensemble de techniques visant à réduire ou rendre inutile l’intervention de ressources humaines dans un processus où celle-ci était coutumière.

Automatisation des tests

January 18, 2024 9:31 AM

L'automatisation des tests logiciels décrit tout processus impliquant l'utilisation d'outils logiciels distincts pour tester le développement de logiciels. Ces outils utilisent des séquences scénarisées pour examiner et valider les produits avec beaucoup moins d'intervention humaine que les techniques de test traditionnelles. Pendant l'automatisation des tests, les outils logiciels d'automatisation surveilleront les tests, compareront les résultats aux résultats attendus et rapporteront les résultats. Les tests de logiciels automatisés réduisent les délais de mise sur le marché et offrent une plus grande efficacité dans les tests de produits. L'automatisation des tests logiciels permet de tester et de livrer un produit en continu.

Bug

January 18, 2024 9:32 AM

Un Bug est tout comportement indésirable qui nécessite une intervention de l'équipe projet.

Ce type de problème peut survenir dans n'importe quel environnement et n'est pas nécessairement rattaché au développement de nouvelles fonctionnalités. C'est tout simplement un constat que quelque chose ne répond pas aux attentes.

Bonnes pratiques

Dans le métier d'AQ, les bonnes pratiques de gestion de Bug incluent:

- Un processus défini pour traiter la découverte de Bugs
- Documentation complète de l'état actuel
- Documentation complète de l'état visé
- Liste de validations des changements prévus
- Stratégie pour mitiger les risques

Utilisation du terme et alias

Le terme Bug peut être utilisé pour englober tout problème qui doit être corrigé. D'autres termes peuvent être utilisés mais risquent d'induire en erreur tout ceux qui ont une définition existante dans leur contexte.

Alias

Le terme reconnu en français est “Bogue”. Tenez en compte que nous ne pouvons pas toujours modifier les termes présents dans les outils. Dans la mesure du possible, privilégier la clarté dans vôtre contexte.

Autres termes

Avant d'utiliser ces termes, considérer le risque d'induire en erreur vos collègues. Un vocabulaire trop riche apporte ses propres problèmes.

- Anomalie
- Défaut
- Erreur
- Défaillance
- Faute

Ressources:

- Comment rédiger une anomalie

L'origine du premier bug

En 1947, une équipe à Harvard a découvert la cause d'erreurs constantes dans leur ordinateur: Un papillon de nuit dans la machine. 

Build

January 18, 2024 9:32 AM

Build (construction) désigne le processus de compilation du code source en un exécutable ou une bibliothèque utilisable.

Cartographie d’exemples (example mapping)

January 18, 2024 9:32 AM

L’Example Mapping est un exercice qui permet de travailler sur les user-stories en définissant les différents exemples qui répondront aux différentes règles de vos user-stories. C’est une façon simplifiée pour faire maturer les user-stories de votre backlog. L’objectif de cet atelier est d’encourager l’échange entre les business owners et l’équipe de développement autour d’un même référentiel.

Pour effectuer cet exercice, vous aurez besoin de cartes (post-it) de différentes couleurs: jaunes, bleus, verts et rouges. Vous commencez par prendre une première user-story que vous mettrez sur un post-it jaune. Ensuite, vous définissez les règles que devra respecter votre user-story. Chaque règle sera mise sur un post-it bleu et placée sous la user-story. Vous allez ensuite définir des exemples pour chacune des règles qui ont été énoncées sur des post-it verts; vous écrirez un exemple par post-it. Enfin, vous pouvez utiliser des post-it rouges pour noter les questions qui se posent lors de l’exercice.

L’Example Mapping est un exercice très intéressant car il permet de mettre rapidement en place des possibilités de découpage ou d’ajout de user-stories répondant aux éventuelles questions posées. Cet exercice permet de mettre en évidence des choses que vous auriez pu oublier dans votre backlog. Il peut être intéressant de faire cet exercice avec les 3 amigos et/ou pendant le raffinement car il permet rapidement de mettre en avant des cas de tests et la définition du plan de test (livraison) de la story.

Sachant que l’Example Mapping permet de faire maturer des user-stories, il est conseillé au product owner de faire cet exercice avant le démarrage estimé, avant la plannification des user-stories. Cet exercice ne remplace pas la mise en place d’un story-mapping qui a pour but de faire émerger un premier backlog (sans détail).

Cas de test

January 18, 2024 9:32 AM

Un cas de test est une série d'actions que nous effectuons pour améliorer une fonction ou une fonctionnalité particulière de notre application à déterminer. Pour mémoire, un cas de tests correspond à une typologie de tests à exécuter afin de mesurer la bonne réalisation, ou non, du résultat attendu, selon une donnée de tests (Jeux de données ou JDD).

Les cas de test sont utilisés pour vérifier si le produit répond à certaines exigences spécifiques et à des critères qualité prédéfinis. Donc, ce sont des éléments importants dans le développement de logiciels. Ils sont utilisés pour tester les fonctionnalités d'une application et pour s'assurer que tout fonctionne correctement. Un cas de test est composé de divers éléments tels que des étapes de test, des données de test et des conditions préalables et postérieures développées pour un scénario de test particulier.

Les cas de test sont une documentation détaillée qui affiche tous les détails sur le résultat, l'entrée ou le processus du test.

Cas d’utilisation

January 18, 2024 9:33 AM

Un cas d’utilisation, ou cas d’usage, définit en génie logiciel et en ingénierie des systèmes une manière d’utiliser un système qui a une valeur ou une utilité pour les acteurs impliqués. Le cas d’utilisation correspond à un ensemble d’actions réalisées par le système en interaction avec les acteurs en vue d’une finalité (E2E). L’ensemble des cas d’utilisation permet ainsi de décrire les exigences fonctionnelles d’un système en adoptant le point de vue et le langage de l’utilisateur final.

Un cas d’utilisation est utilisé pour définir comment utiliser un système pour effectuer une tâche spécifique. Il décrit les interactions entre les acteurs et le système pour atteindre un objectif spécifié.

Un cas d’utilisation est une manière d’utiliser un système qui a une valeur ou une utilité pour les acteurs impliqués. Le cas d’utilisation correspond à un ensemble d’actions réalisées par le système en interaction avec les acteurs en vue d’une finalité. L’ensemble des cas d’utilisation permet ainsi de décrire les exigences fonctionnelles d’un système en adoptant le point de vue et le langage de l’utilisateur final.

Les cas d’utilisation sont souvent représentés sous forme de diagrammes pour visualiser les interactions entre les acteurs et le système. Ces diagrammes peuvent être utilisés pour communiquer les exigences fonctionnelles du système aux parties prenantes et pour guider la conception et le développement du système.

Code freeze

January 18, 2024 9:33 AM

Le code freeze est une période pendant laquelle les développeurs ne sont plus autorisés à modifier le code source d'un logiciel. Le code freeze est généralement mis en place avant la sortie d'une nouvelle version d'un logiciel et permet de s'assurer que le code est stable et prêt pour la production. Pendant cette période, les développeurs peuvent se concentrer sur la correction de bugs et l'amélioration de la qualité du code.

Correctif (patch)

January 18, 2024 9:33 AM

Patch (correctif) désigne une mise à jour mineure du logiciel destinée à corriger des bugs ou des problèmes de sécurité.

Courbe de Bell de Roger (Roger’s Bell Curve)

January 18, 2024 9:33 AM

La courbe de Bell de Roger est une méthode d’assurance qualité logicielle qui permet de mesurer la qualité d’un logiciel en fonction de la quantité de défauts qu’il contient. Elle est basée sur la courbe de Gauss et permet de déterminer le nombre de défauts dans un logiciel en fonction de la qualité attendue. Cette méthode est utilisée pour évaluer la qualité des logiciels et pour déterminer les zones à améliorer.

CRUD

January 18, 2024 9:34 AM

CRUD est un acronyme pour Create (Créer), Read (Lire), Update (Mettre à jour) et Delete (Supprimer), qui sont les quatre opérations de base pour la gestion des données dans une base de données ou une application.

Les tests CRUD sont des tests qui vérifient que ces quatre opérations de base fonctionnent correctement dans une application ou un système. Ces tests peuvent inclure la création de nouveaux enregistrements, la lecture des données existantes, la mise à jour des enregistrements existants et la suppression des enregistrements.

Les tests CRUD sont importants pour s'assurer que les données sont gérées correctement dans une application ou un système et que les utilisateurs peuvent effectuer les opérations de base nécessaires pour interagir avec les données.

Cycle de vie du développement logiciel (SDLC)

January 18, 2024 9:34 AM

Le SDLC (Software Development Life Cycle) permet une analyse approfondie à chaque phase du cycle de vie du développement logiciel, ce qui permet de mesurer l'efficacité de votre processus de développement logiciel et de l'améliorer . Les systèmes de gestion SDLC contrôlent chaque étape du processus tout en ajoutant des analyses, des systèmes de gestion du travail et un suivi des bogues qui permettent d'améliorer les parties du cycle de vie ne fonctionnant pas efficacement. En résumé, analyser et tester à chaque phase permet de maximiser l'efficacité, accélérer la vitesse et réduire les coûts à toutes les phases.

Les systèmes d'information et l'ingénierie logicielle, le cycle de vie du développement des systèmes (SDLC), également appelé cycle de vie du développement d'applications, est un processus de planification, de création, de test et de déploiement, donc, cycle de livraison d'un système d'information. Le concept SDLC s'applique à une gamme de configurations matérielles et logicielles, car un système peut être composé uniquement de matériel, de logiciel uniquement ou d'une combinaison des deux.

Les phases

Il y a généralement six phases dans ce cycle :

1. **DÉFINIR** (analyse des exigences/spécifications/besoins/solutions)
   Cette phase consiste à définir les exigences et les objectifs du projet. Les exigences sont collectées auprès des parties prenantes et sont utilisées pour créer une spécification des exigences.

2. **PLANIFIER** (conception, plan de test…)
   Cette phase consiste à planifier le projet en fonction des exigences définies dans la phase précédente. Les activités de planification comprennent la définition des tâches, l'estimation des coûts et la planification des ressources.

3. **INTÉGRER** (développement)
   Cette phase consiste à intégrer les différents composants du système pour créer un système complet. Les tests d'intégration sont effectués pour s'assurer que tous les composants fonctionnent correctement ensemble.

4. **DIFFUSER** (mise en œuvre)

Cette phase consiste à déployer le système dans l'environnement de production. Les tests de déploiement sont effectués pour s'assurer que le système fonctionne correctement dans l'environnement de production.

5. **OPÉRER**
   Cette phase consiste à exploiter le système en production. Les activités d'exploitation comprennent la surveillance du système, la gestion des incidents et la maintenance du système.

6. **VEILLER** (Satisfaction, alertes et statistiques)
   Cette phase consiste à surveiller le système en production pour s'assurer qu'il fonctionne correctement et qu'il répond aux exigences des utilisateurs. Notez que "**veiller**", à la satisfaction, focus plus sur le positif → "assurer", "protéger", "prendre soin de"…
   plutôt que "mesurer", "évaluer", "contrôler", "surveiller", "punir ou avoir des conséquences"…

Les états

Un cycle est une boucle itérative et aussi chacune des phases…

Il y a généralement 3 états par phase :

7. **Analyser**
8. **Faire**
9. **Valider** (tester)

Le "flow"

La documentation/métriques scientifique

Si vous suivez la méthode du chercheur scientifique en amélioration continue, lors de chaque phase, vous obtiendrez donc le succès, la confiance et la satisfaction !

Prendre des notes (comme un chercheur scientifique) est important et sous-estimé. Les traces est le levier de l'amélioration.

Il est important de recommencer toutes les phases lorsque une phase échoue ! Cela peut paraître intense, mais ce ne l'est pas, et surtout que c'est très payant

Le principe est de repositionner rapidement le besoin en continue si il y a lieu, plutôt que d'attendre à la fin pour le réaliser et trop tarder pour réviser. (ex. de questions après l'évolution du contexte : la priorisation, la valeur ajouté, le RoI, mise-à-jour de la documentation, etc.)

Vous remarquerez qu'en général nous passons instantanément les phases précédentes pour continuer là où nous devions continuer

Dans un cadre d’un sprint Agile, un récit (story) ou un bug devra avoir passé par toutes les phases avant de se terminer. Le terminer est la définition (DoD)

Voici le "flow" du retour au début :

Voici différentes façons de représenter le "FLOW"

Linéaire

"SHELL" or "STAIR"

(il y a aussi l’escalier montant, ce n’est qu’aller vers le haut plutôt que le bas)

"WHEEL" or "DONUT"

"INFINI" or "8" or "DevOps"

"V" or "BOL" or "Shift-Left"

D’autres profils SDLC

Il y a d’autres profils des phases et états comme le "Waterfall" par exemple, qui fait les validations/tests en tant qu’une des phases à la fin, souvent dit "UAT" (**important : ce n’est pas la définition exacte de UAT**) ou QA (test)

ref.: Systems development life cycle - Wikipedia

Cycle de test

January 18, 2024 9:34 AM

“Test cycles” est un terme utilisé par Zephyr

Un cycle de test est un ensemble ciblé de cas de test qui sont regroupés pour atteindre des objectifs de test spécifiques. Les cycles de test ont une portée plus longue et plus large que les cas de test, et ils peuvent être attribués à des testeurs et à des environnements de test spécifiques. Des exemples de cycles de test incluent des tests de régression, des tests de vérification de construction, des tests de bout en bout, etc.

! Ne pas confondre avec Cycle de vie

Le terme le plus utilisé est un plant de test, il y a aussi une livraison (release) ou un projet de test.

Définition de prêt (DoR)

January 18, 2024 9:35 AM

DoR signifie “Definition of Ready” en anglais et se traduit en français par “Définition de Prêt”. C’est une pratique de l’agilité qui consiste à définir les critères qui doivent être remplis avant qu’une tâche puisse être considérée comme prête à être travaillée. Les critères peuvent inclure des éléments tels que la définition des exigences, la disponibilité des ressources et la définition des critères d’acceptation.

- J’aime
- Commentaire
- 4 vues
- Enregistrer pour plus tard

Définition de Terminé (DoD)

January 18, 2024 9:35 AM

DoD signifie “Definition of Done” en anglais et se traduit en français par “Définition de Terminé”. C’est une pratique de l’agilité qui consiste à définir les critères qui doivent être remplis avant qu’une tâche puisse être considérée comme terminée. Les critères peuvent inclure des éléments tels que la réalisation des tests unitaires, la revue de code par les pairs et la documentation.

Démarche scientifique pour le test exploratoire

January 18, 2024 9:35 AM

La “démarche scientifique” fait généralement référence à une méthode systématique pour acquérir des connaissances basée sur des observations, des expériences et des preuves. Dans le contexte des tests exploratoires de logiciels, cela peut signifier l’utilisation d’une approche structurée pour découvrir comment le logiciel fonctionne réellement, en posant des questions sur la façon dont il gérera les cas difficiles et faciles. La qualité des tests dépend de la capacité du testeur à inventer des cas de test et à trouver des défauts. Plus le testeur en sait sur le produit et les différentes méthodes de test, meilleurs seront les tests.

L’utilisation d’une démarche scientifique dans les tests exploratoires peut offrir plusieurs avantages. Tout d’abord, cela peut aider à garantir que les tests sont effectués de manière systématique et rigoureuse, en suivant une méthode structurée pour découvrir comment le logiciel fonctionne réellement. Cela peut également aider à garantir que les tests sont basés sur des observations, des expériences et des preuves, plutôt que sur des suppositions ou des intuitions.

Deuxièmement, l’utilisation d’une démarche scientifique peut aider à améliorer la qualité des tests en encourageant les testeurs à inventer des cas de test et à trouver des défauts en utilisant leur connaissance du produit et des différentes méthodes de test. Cela peut également aider à garantir que les tests sont effectués de manière objective et impartiale, en se basant sur des faits plutôt que sur des opinions ou des préférences personnelles.

Enfin, l’utilisation d’une démarche scientifique peut aider à améliorer la crédibilité et la fiabilité des résultats des tests en fournissant une méthode rigoureuse pour documenter et présenter les résultats. Cela peut aider à renforcer la confiance dans les résultats des tests et à faciliter la prise de décision en se basant sur des preuves solides.

Pour appliquer la démarche scientifique dans les tests exploratoires, vous pouvez suivre ces étapes:

1. **Définir une question de recherche:** Commencez par définir une question de recherche claire et précise que vous souhaitez répondre à travers vos

tests exploratoires. Par exemple, vous pourriez vous demander comment le logiciel gère les cas difficiles ou comment il répond à des entrées de données inhabituelles.

2. **Collecter des données:** Ensuite, utilisez des méthodes de collecte de données appropriées pour recueillir des informations sur le logiciel que vous testez. Cela peut inclure l'observation du comportement du logiciel, l'exécution de tests, la collecte de commentaires des utilisateurs, etc.
3. **Analyser les données:** Une fois que vous avez collecté des données, analysez-les pour identifier les tendances, les modèles et les relations. Utilisez des méthodes d'analyse appropriées pour tirer des conclusions à partir de vos données.
4. **Tirer des conclusions:** Sur la base de votre analyse, tirez des conclusions sur le comportement du logiciel et répondez à votre question de recherche. Assurez-vous que vos conclusions sont étayées par des preuves solides et documentez-les de manière claire et concise.
5. **Communiquer les résultats:** Enfin, communiquez vos résultats aux parties prenantes concernées, en présentant vos conclusions de manière claire et concise. Assurez-vous d'inclure suffisamment de détails pour permettre aux autres de comprendre comment vous êtes arrivé à vos conclusions et comment ils peuvent utiliser vos résultats pour prendre des décisions éclairées.

Déploiement

January 18, 2024 9:36 AM

Déploiement désigne le processus d’installation et de configuration du logiciel sur les systèmes des utilisateurs finaux.

Développement Agile piloté par le modèle (AMDD)

January 18, 2024 9:36 AM

Agile Model Driven Development (AMDD) est une version agile de Model Driven Development (MDD), un processus où les modèles sont générés avant que le code ne soit écrit. Avec AMDD, des modèles agiles sont créés où les modèles sont suffisamment bons pour commencer à écrire le code, au lieu de passer trop de temps à créer des modèles généraux avec MDD.

En AMDD, les efforts de conception sont répartis entre la modélisation dans les deux sprints et le codage.

ref. : What is Test Driven Development (TDD)? Example

Développement piloté par la fonctionnalité (FDD)

January 18, 2024 9:36 AM

Le FDD (**F**eature **D**riven **D**evelopment) est une méthode de développement logiciel qui se concentre sur la création de fonctionnalités spécifiques pour l'application. Elle utilise des processus itératifs pour développer les fonctionnalités et se concentre sur la collaboration entre les développeurs et les parties prenantes pour s'assurer que les fonctionnalités répondent aux besoins des utilisateurs.

Développement piloté par le comportement (BDD)

January 18, 2024 9:36 AM

Le BDD (**B**ehavior **D**riven **D**evelopment) est une méthode de développement logiciel qui se concentre sur le comportement attendu d'un système plutôt que sur les détails techniques de sa mise en œuvre. Elle est basée sur la collaboration entre les développeurs, les testeurs et les parties prenantes pour s'assurer que le système répond aux besoins des utilisateurs.

Développement piloté par les tests (TDD)

January 18, 2024 9:37 AM

Le TDD (**T**est **D**riven **D**evelopment) est une méthode de développement logiciel qui consiste à définir les tests avant de faire le design et le code de l'application. Les tests sont les exigences/requis, les critères de succès/acceptations.

TDD est mieux connu des développeurs comme un test unitaire automatisé du code qui doit être effectué avant de coder la fonction, mais il est important de comprendre qu'il ne s'agit pas seulement que de cela.

Développement piloté par les tests d'acceptation (ATDD)

January 18, 2024 9:37 AM

L’ATDD (**A**cceptance **T**est **D**riven **D**evelopment) est une méthode de développement logiciel qui consiste à écrire des tests pour vérifier que l’application répond aux exigences des utilisateurs. Les tests sont écrits en collaboration avec les parties prenantes pour s’assurer que les exigences sont claires et précises.

DevOps

January 18, 2024 9:37 AM

Le but du DevOps est de "Colle-laborer"

DevOps est une pratique, une approche, visant à améliorer le travail dans le processus de cycle de vie de développement de logiciels (SDLC). DevOps se concentre sur la réduction de la taille des lots, la création d'équipes verticales autonomes et l'automatisation de votre pipeline de déploiement.

Entre le développement et l'opérationnel… il y a l'intention de partager, réutiliser, simplifier, uniformiser, solidifier… améliorer les attentes, les lenteurs, les droits d'accès, processus, procédures… Donc, un but commun, briser les silos et faire de la QUALITÉ ensemble !

Le DevOps est un mouvement en ingénierie informatique et une pratique technique visant à l'unification du développement logiciel (dev) et de l'administration des infrastructures informatiques (ops), notamment l'administration système. C'est un ensemble de pratiques qui met l'accent sur la collaboration et la communication entre les développeurs de logiciels et les professionnels des opérations informatiques, en automatisant le processus de livraison de logiciels et les changements d'infrastructure.

En combinant développement (Dev) et opérations (Ops), DevOps est l'union des personnes, des processus et des technologies destinés à fournir continuellement de la valeur aux clients. DevOps est une culture qui encourage la collaboration entre tous les rôles impliqués dans le développement et la maintenance des logiciels.

Oui, mais qu'en est-il du Dev**Sec**Ops ou du Dev**Qa**Ops ? Et bien c'est juste là pour rappeler que certains aspects sont parfois oubliés ou négligés et qu'il faut mettre l'accent sur eux à ce moment-là. Consultez la section en bas de page pour en savoir plus.

Il ya la pratique, mais aussi le rôle

Le rôle et les responsabilités d'un DevOps (expert/spécialiste) varient en fonction de l'entreprise et de l'équipe dans laquelle il travaille. En général, les DevOps sont responsables de la collaboration entre les équipes de développement et d'exploitation pour améliorer la qualité des logiciels et accélérer le déploiement. Les DevOps sont également responsables de la mise en place d'outils et de processus pour automatiser les tâches répétitives et améliorer l'efficacité. Les DevOps sont également responsables de la surveillance des systèmes pour identifier les problèmes avant qu'ils ne deviennent critiques. En fin de compte, le rôle des DevOps est de faciliter la communication et la collaboration entre les équipes pour améliorer la qualité des logiciels et accélérer le déploiement.

Voici le cycle de vie complet du développement "DevOps"

En DevOps :

- Il y un seul SDLC (cycle de vie du développement), un seul flux pour tous les types de tâches.
- Chacune des 6 phases passe toutes par une étape d'analyse et se termine par une étape de validation.
- Et c'est en continu.
-

Tableau périodique des outils DevOps

Les technologies DevOps évoluent constamment. Pour vous aider à vous y retrouver, la base de connaissance est catégorisée par le fameux tableau périodique des outils suivants :
Attention c'est pour le plaisir d'avoir une liste d'outils, cette liste est porté à être désuet rapidement.

Voici la liste des sous-catégories du DevOps

DevOps va au-delà du développement et des opérations pour incorporer les contributions de toutes les parties prenantes de l'application - y compris l'ingénierie de plateforme et d'infrastructure, la sécurité, la conformité, la gestion des risques, la ligne de métier, les utilisateurs finaux et les clients - dans le cycle de vie du développement de logiciels.

Il est important de noter que DevOps est plus qu'une simple méthodologie ou collection de processus. C'est fondamentalement un ensemble de pratiques - à la fois culturelles et technologiques. Plutôt que de prescrire chaque détail, DevOps rassemble un ensemble de pratiques, d'idées culturelles et d'outils qui fonctionnent dans un cadre qui est malléable pour

les besoins d'une équipe et d'une organisation données.

Cependant, dans certains contextes, **vous pouvez voir des termes spécifiques utilisés pour mettre l'accent sur certaines parties de DevOps**.

🎯 Amplifier l'emphase sur les aspects fragiles dans le but d'améliorer la maturité actuel du **DevOps** !

***Il est important de noter que ces termes ne sont pas standardisés et peuvent être interprétés différemment par différentes personnes ou organisations. Il est donc toujours préférable de clarifier ce que l'on entend par ces termes lorsqu'ils sont utilisés dans un contexte spécifique.

- **DevSecurityOps ou DevSecOps** : DevSecOps est une approche qui intègre les équipes de développement et de sécurité.
  Il s'agit d'une évolution de DevOps qui met l'accent sur la sécurité dans la conception de logiciels ou d'applications. DevSecOps est l'abréviation de Development, Security, and Operations. En plus de fournir des applications de qualité livrées rapidement, les DevSecOps s'assurent également de leur sécurité.
- **SecOps** : SecOps, ou Security Operations, est une approche qui intègre les pratiques de sécurité dans les opérations informatiques. Cela permet une meilleure collaboration entre les équipes de sécurité et les équipes d'exploitation.
- **QAOps** : QAOps, ou Quality Assurance Operations, est une pratique qui intègre l'assurance qualité (QA) dans les pipelines de livraison de logiciels. Dans QAOps, l'équipe des opérations collabore continuellement avec l'équipe QA pour garantir que des produits de qualité sont continuellement livrés.
- **GitOps** : GitOps est une sous-catégorie de DevOps qui utilise Git comme source unique de vérité pour le déploiement de l'infrastructure et des applications. Cela permet une gestion de version claire et un historique des changements.
- **DataOps** : DataOps est une méthodologie automatisée, orientée processus, pour améliorer la qualité et réduire le cycle de livraison des données. Elle applique des principes de DevOps tels que l'automatisation, l'intégration continue et le déploiement continu à la gestion des données.
- **MLOps** : MLOps, ou DevOps pour l'apprentissage automatique, est une pratique pour la collaboration et la communication entre les data scientists et les professionnels des opérations pour aider à gérer la production de modèles d'apprentissage automatique.
- **AIOps** : AIOps, ou Artificial Intelligence for IT Operations, est une méthode qui utilise l'IA et l'apprentissage automatique pour automatiser et améliorer les opérations informatiques.
- **ChatOps** : ChatOps est une méthode de collaboration et de gestion des tâches techniques et commerciales via une plateforme de chat. Il permet aux équipes de travailler ensemble et de gérer les systèmes à partir d'un seul point, généralement une plateforme de chat.
- **NoOps** : NoOps, ou No Operations, est un concept selon lequel une organisation peut devenir si automatisée et abstraite de son infrastructure qu'il n'y a pas besoin d'une équipe d'opérations dédiée.
- **ArchOps** : SecOps, ou Security Operations, est une approche qui intègre les pratiques de sécurité dans les opérations informatiques. Cela permet une meilleure collaboration entre les équipes de sécurité et les équipes d'exploitation.
- **IaC (Infrastructure as Code)** : IaC est une pratique qui permet aux développeurs de définir et de gérer l'infrastructure informatique à l'aide de fichiers de code. Cela permet une gestion plus efficace et automatisée de l'infrastructure.
- **SRE (Site Reliability Engineering)** : SRE est une discipline qui applique les principes de l'ingénierie logicielle à la gestion des opérations informatiques. Cela permet une meilleure fiabilité et une meilleure performance des systèmes.
- **SysOps** : SysOps, ou System Operations, est une approche qui se

concentre sur la gestion et la maintenance des systèmes informatiques dans un environnement de production.
    - **DbaOps** : DbaOps, ou Database Operations, est une approche qui intègre les pratiques de gestion de base de données dans le pipeline DevOps. Cela permet une meilleure collaboration entre les administrateurs de bases de données et les équipes de développement et d'exploitation.
    - **Rugged DevOps** : Rugged DevOps est une approche qui met l'accent sur la sécurité tout au long du cycle de vie du développement de logiciels. Il s'agit d'intégrer les pratiques de sécurité dès le début du processus de développement et de les maintenir tout au long du cycle de vie.
    - **BizDevOps** : BizDevOps, ou Business DevOps, est une approche qui intègre les équipes commerciales dans le pipeline DevOps. Cela permet une meilleure collaboration entre les équipes de développement, d'exploitation et commerciales, ce qui peut conduire à une meilleure compréhension des besoins des clients et à une livraison plus rapide de produits de haute qualité.
    - **WinOps** : WinOps est une approche de DevOps spécifiquement pour les environnements Windows. Elle vise à apporter les principes de DevOps aux équipes qui travaillent avec la technologie Microsoft.
- **ChatOps** : ChatOps est une méthode de collaboration et de gestion des tâches techniques et commerciales via une plateforme de chat. Il permet aux équipes de travailler ensemble et de gérer les systèmes à partir d'un seul point, généralement une plateforme de chat.
- **DocOps** : ChatOps est une méthode de collaboration et de gestion des tâches techniques et commerciales via une plateforme de chat. Il permet aux équipes de travailler ensemble et de gérer les systèmes à partir d'un seul point, généralement une plateforme de chat.
- **HROps** : HROps, ou Human Resources Operations, est une approche qui intègre les équipes des ressources humaines dans le pipeline DevOps. Cela permet une meilleure collaboration entre les équipes de développement, d'exploitation et des ressources humaines.
- **QADevOps** : Ce terme est utilisé pour décrire une plateforme de DevOps pour l'écosystème SAP. Il est basé sur le programme DORA "Devops Research and Assessment" de Google. La plateforme utilise l'apprentissage automatique pour automatiser les processus de gouvernance dans la gestion des codes sources des entreprises avec des développements ABAP.
- **DevQAOps** : Ce terme est utilisé pour décrire le rôle d'un expert en tests automatisés dans la mise en place de tests automatisés. Le rôle de DevQAOps consiste à construire des ponts efficaces entre Dev et Ops et QA pour assurer une coopération et une productivité fructueuses. DevQAOps devrait mettre en œuvre l'automatisation des processus dans le développement et les tests de logiciels où cela est possible pour atteindre la vitesse et l'agilité nécessaires.
- **FinOps** : FinOps est une approche qui se concentre sur la gestion financière des services cloud.
- **TestOps** : TestOps est une approche qui intègre les tests de logiciels dans le pipeline DevOps.
- **OpsDev** : OpsDev est une approche qui met l'accent sur les opérations dès le début du processus de développement.
- **DevBizOps** : DevBizOps est une approche qui intègre les équipes de développement, d'exploitation et commerciales.
- **DevDataOps** : DevDataOps est une approche qui intègre les équipes de développement et de données.
- **DevNetOps** : DevNetOps est une approche qui intègre les équipes de développement et de réseau.
- **DevCloudOps** : DevCloudOps est une approche qui intègre les équipes de développement et de cloud.
- **DevUXOps** : DevUXOps est une approche qui intègre les équipes de développement et d'expérience utilisateur.
- **DevMarketingOps** : DevMarketingOps est une approche qui intègre les équipes de développement et de marketing.
- **DevSupportOps** : DevSupportOps est une approche qui intègre les équipes de développement et de support.
- **DevSalesOps** : DevSalesOps est une approche qui intègre les équipes de développement et de vente.

- **DevHROps** : DevHROps est une approche qui intègre les équipes de développement et de ressources humaines.
- **DevLegalOps** : DevLegalOps est une approche qui intègre les équipes de développement et juridiques.
- **DevFinanceOps** : DevFinanceOps est une approche qui intègre les équipes de développement et de finance.
- **DevPRops** : DevPRops est une approche qui intègre les équipes de développement et de relations publiques.
- **DevDesignOps** : DevDesignOps est une approche qui intègre les équipes de développement et de design.
- **DevProductOps** : DevProductOps est une approche qui intègre les équipes de développement et de produit.
- **DevResearchOps** : DevResearchOps est une approche qui intègre les équipes de développement et de recherche.
- **DevTrainingOps** : DevTrainingOps est une approche qui intègre les équipes de développement et de formation.
- **DevComplianceOps** : DevComplianceOps est une approche qui intègre les équipes de développement et de conformité.
- **DevAuditOps** : DevAuditOps est une approche qui intègre les équipes de développement et d'audit.
- **DevRiskOps** : DevRiskOps est une approche qui intègre les équipes de développement et de gestion des risques.
- **DevStrategyOps** : DevStrategyOps est une approche qui intègre les équipes de développement et de stratégie.
- **DevPlanningOps** : DevPlanningOps est une approche qui intègre les équipes de développement et de planification.
- **DevAnalyticsOps** : DevAnalyticsOps est une approche qui intègre les équipes de développement et d'analyse.
- **DevCustomerOps** : DevCustomerOps est une approche qui intègre les équipes de développement et de service à la clientèle.
- **DevPartnerOps** : DevPartnerOps est une approche qui intègre les équipes de développement et de partenariat.
- **DevVendorOps** : DevVendorOps est une approche qui intègre les équipes de développement et de gestion des fournisseurs.

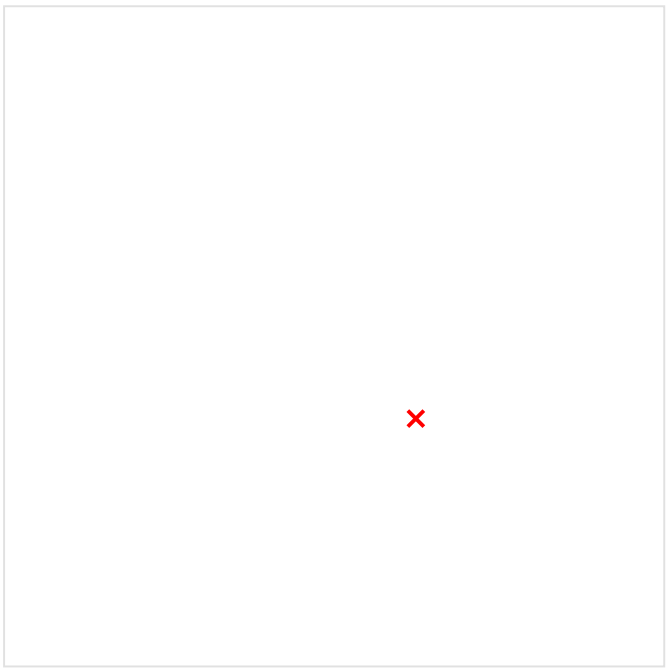
...ainsi de suite...

Diagramme de Kiviat, en étoile, toile d'araignée ou radar.

January 18, 2024 9:37 AM

Le diagramme de Kiviat, également connu sous le nom de diagramme en radar, en étoile ou en toile d'araignée, est utilisé pour représenter sur un plan en deux dimensions au moins trois ensembles de données multivariées.

Chaque axe, qui part d'un même point, représente une caractéristique quantifiée.



Ce type de diagramme facilite une analyse détaillée de plusieurs objets, ainsi que leur comparaison générale (comparaison des surfaces) ou point par point. Il est particulièrement utile si les axes sont correctement normés selon l'importance donnée à chaque caractéristique.

Un des exemples d'utilisation est la représentation de la répartition des vents à un lieu géographique donné sur une période donnée. Le diagramme de Kiviat a une limite physique autour de 8 axes, pour représenter des données multivariées sur encore plus de dimensions, il faut alors utiliser d'autres graphiques tels que les coordonnées parallèles.

Il a été créé en 1877 par le statisticien allemand Georg von Mayr.

REF :
2019-02-05,Wikipédia,https://fr.wikipedia.org/wiki/Diagramme_de_Kiviat

Documentation vivante

January 18, 2024 9:38 AM

La documentation **vivante** est d'avoir une documentation à jour à chaque instant et donc toujours le même document versionné passant par le brouillons jusqu'à la publication officiel.

Important de savoir… la documentation vivante est une approche qui consiste à regrouper les tests et les spécifications. Les tests et la spécification ne font plus qu'un et il est fortement conseillé de la rédiger avec Gherkin. La documentation vivante permet aux développeurs de se concentrer sur la qualité de leur travail tout en créant la documentation.

Elle peut aider une équipe à améliorer la conception d'un système et l'adoption du ATDD, TDD, BDD, FDD et de la conception pilotée par le domaine (DDD, ou Domain-Driven Design).

Domain-driven design (DDD)

January 18, 2024 9:38 AM

DDD signifie “Domain-Driven Design” en anglais. C’est une approche/méthode majeure de conception de logiciels qui se concentre sur la modélisation des logiciels pour correspondre à un domaine selon les entrées des experts de ce domaine (domaine métier de l’application). Selon la conception pilotée par le domaine, la structure et le langage du code logiciel (noms de classe, méthodes de classe, variables de classe) doivent correspondre au domaine métier. Elle utilise des concepts tels que les entités, les valeurs d’objet et les agrégats pour modéliser le domaine métier.

ref. : Domain-driven design - Wikipedia

DRY

January 18, 2024 9:38 AM

Don’t **R**epeat **Y**ourself. C’est un principe qui consiste à éviter d’avoir de la redondance de code dans votre application. Cela s'applique aussi à l'automatisation.

Cette redondance est source de nombreux problèmes dont :

- L’augmentation du volume de code
- La diminution de sa lisibilité
- L’introduction de bugs suite à une évolution du code

Pour éviter ce fléau, il est possible de factoriser le code redondant dans des classes , méthodes ou fonctions et de l’appeler quand on en a besoin.

N.B. : **Ce n’est pas appliqué juste au code** 😉

Émulateur

January 18, 2024 9:38 AM

Dans le contexte des tests de logiciels, un émulateur est un outil logiciel ou matériel qui reproduit le comportement d'un système spécifique, permettant à un système d'exécuter des logiciels ou des applications conçus pour un autre système. Un émulateur peut être utilisé pour tester la fonctionnalité.

Les avantages de l'utilisation d'un émulateur incluent la possibilité de tester la fonctionnalité d'un logiciel ou d'une application conçue pour un autre système. Cela peut faciliter le développement ou le débogage d'un système ou remplacer un système obsolète ou inutilisable par un autre.

Les émulateurs et les simulateurs ont des objectifs différents dans le développement de logiciels, les émulateurs se concentrant sur la compatibilité et la réplication, tandis que les simulateurs se concentrent sur la modélisation et l'analyse.

Cependant, il y a aussi des limites à l'utilisation de ces outils. Par exemple, un émulateur ne peut reproduire que le comportement d'un modèle dont toutes les variables sont connues.

Fuite de bugs (Bug leakage)

January 18, 2024 9:39 AM

“Bug leakage” ou “fuite de bugs” se produit lorsqu’un bug est découvert par l’utilisateur final ou le client, qui aurait pu être détecté par l’équipe de test. Cela signifie que le bug a été manqué par l’équipe de test et n’a pas été corrigé avant la publication du logiciel.

Gravité

January 18, 2024 9:39 AM

La gravité est une mesure de l'impact [1] et est définie par des niveaux de gravité. *Les niveaux de gravité des incidents constituent une mesure de l'impact d'un incident sur l'entreprise*, ou sur le projet ou sur le logiciel. *En règle générale, plus le niveau de gravité est faible, plus l'impact de l'incident est important*[1].
Pour définir le niveau de gravité d'un incident, on peut se poser les questions suivantes:
Quel est l'impact d'un incident sur les utilisateurs ? Affecte-t-il l'ensemble du système ? Les empêche-t-il d'effectuer une tâche essentielle ? Ou peut-être crée-t-il simplement du stress et rend-il les tâches plus difficiles ? [1]

[1] Atlassian. (n.d.). Comprendre les niveaux de gravité des incidents. Atlassian. Retrieved May 4, 2023, from https://www.atlassian.com/fr/incident-management/kpis/severity-levels

Hackathon (Qualiathon)

January 18, 2024 9:39 AM

Un **hackathon** (également connu sous le nom de hack day, innovation day, hackfest, datathon ou codefest ; une fusion de hacking et marathon) est un événement où les gens s'engagent dans une ingénierie collaborative rapide sur une période de temps relativement courte, comme 24 ou 48 heures, un week-end par exemple ou la longueur d'un "sprint" (1 à 4 semaines de travail)

Il peut y avoir des prix, de la gamification, des concours, des boîtes à lunch en télétravail ou des buffets/drinks en présentiel. Même que pour les gagnants, l'entreprise peut signer un contrat de démarrage de projet et/ou acheter la preuve de concepte. C'est un "Win-Win" pour l'entreprise et les employées. Un événement qui offre beaucoup d'expérience, de plaisir et un retour sur investissement profitable :)

Ils sont souvent exécutés à l'aide de pratiques de développement logiciel Agile, telles que la conception dans laquelle les joueurs informatiques et les autres personnes impliquées dans le développement de logiciels, y compris les graphistes, les concepteurs d'interface, les chefs de produit, les chefs de projet, les testeurs ou les représentants de l'assurance qualité, les experts du domaine et d'autres collaborent largement sur projets d'ingénierie, tels que le génie logiciel (R&D).

L'objectif d'un hackathon est de créer un logiciel fonctionnel, du matériel ou une preuve de concept d'ici la fin de l'événement. Les hackathons ont tendance à avoir un objectif spécifique, qui peut inclure le langage de programmation utilisé, le système d'exploitation, une application, une API, ou le sujet et la démographie des programmeurs, des contraintes et un scénario. Dans d'autres cas, il n'y a pas de restrictions.

Bref, un hackathon est un événement où des groupes de volontaires du développement se réunissent pendant une période de temps définie pour travailler sur des concepts informatiques de manière collaborative. C'est un processus de création aux objectifs larges et variés souvent axés sur la proposition de solutions informatiques innovantes, l'amélioration de logiciels existants ou la conception de nouvelles applications ou encore l'exploration de nouvelles approches/méthodologies dans le domaine des technologies numériques.

Un "**Qualiathon**" est un hackathon avec l'objectif d'innover ou de faire découvrir l'assurance qualité.

Hacker éthique

January 18, 2024 9:39 AM

Un hacker éthique est un expert en sécurité informatique qui utilise ses compétences pour identifier les vulnérabilités dans les systèmes informatiques et les réseaux de l'entreprise. Les hackers éthiques sont également connus sous le nom de pirates informatiques éthiques ou de testeurs de pénétration. Les hackers éthiques sont souvent embauchés par des entreprises pour tester la sécurité de leurs systèmes informatiques et réseaux.

Hotfix (correctif à chaud)

January 18, 2024 9:40 AM

Hotfix (correctif à chaud) désigne un correctif urgent destiné à résoudre un problème critique dans le logiciel.

Hyper-automatisation

January 18, 2024 9:40 AM

Automatiser le maximum possible, mais comment ?

1. Le coffre à outils (les bons outils non limitatifs)
2. Le niveau des compétences/connaissances

L'hyper-automatisation est un terme qui fait référence à l'utilisation de technologies avancées telles que l'intelligence artificielle (IA), l'apprentissage automatique (ML) et l'automatisation des processus robotiques (RPA) pour automatiser les tâches qui étaient autrefois effectuées par des humains. Elle permet d'augmenter l'automatisation des processus métier en introduisant l'intelligence artificielle (IA), l'apprentissage machine (ML) et l'automatisation des processus par la robotique (RPA). Les organisations qui adoptent l'hyper-automatisation visent à rationaliser les processus dans l'ensemble de leur entreprise en utilisant l'intelligence artificielle (IA), l'automatisation des processus robotiques (RPA) et d'autres technologies.

Ingénierie du chaos (chaos engineering)

January 18, 2024 9:40 AM

L'ingénierie du chaos est une pratique de l'ingénierie logicielle qui consiste à tester la résilience d'un système en introduisant des pannes ou des perturbations contrôlées dans le système. L'objectif est de découvrir les points faibles du système et de les corriger avant qu'ils ne causent des problèmes dans un environnement de production réel.

L'ingénierie du chaos peut être utilisée pour tester la résilience des systèmes distribués tels que les applications cloud et les microservices. Elle peut également être utilisée pour tester la résilience des systèmes non distribués tels que les applications monolithiques.

Chaos engineering - Wikipedia

Intégration continue (continuous integration)

January 18, 2024 9:40 AM

C'est le ci dans ci/cd...

L’intégration continue (Continuous Integration en anglais) est une pratique de développement logiciel qui consiste à fusionner fréquemment les modifications de code dans un référentiel central. L’objectif de l’intégration continue est de détecter rapidement les problèmes d’intégration et de permettre une correction rapide. L’intégration continue est souvent associée à la pratique de livraison continue (Continuous Delivery en anglais), qui consiste à automatiser le processus de déploiement des applications.

ref. : Continuous integration - Wikipedia

Intégrité des données (data integrity)

January 18, 2024 9:40 AM

La Data Integrity (intégrité des données en français) est la garantie que les données sont complètes, exactes et fiables tout au long de leur cycle de vie. La Data Integrity est maintenue par une collection de processus, de règles et de normes mises en œuvre lors de la phase de conception. La Data Integrity est également liée à la sécurité des données en ce qui concerne la conformité réglementaire - telle que la conformité GDPR - et la sécurité.

Intelligence artificielle

January 18, 2024 9:41 AM

L'IA désigne la possibilité pour une machine de reproduire des **comportements liés aux humains**, tels que le raisonnement, la planification et la créativité.

L'IA permet à des systèmes techniques de percevoir leur environnement, gérer ces perceptions, résoudre des problèmes et entreprendre des actions pour atteindre un but précis. L'ordinateur reçoit des données (déjà préparées ou collectées via ses capteurs - une caméra, par exemple) les analyse et réagit.

Les systèmes dotés d'IA sont capables d'adapter leurs comportements (plus ou moins) en analysant les effets produits par leurs actions précédentes, travaillant de manière autonome.

L'intelligence artificielle (IA) est une branche de l'informatique qui se concentre sur la création de machines intelligentes qui peuvent effectuer des tâches qui nécessitent normalement l'intelligence humaine. Un ordinateur est une machine électronique qui peut effectuer des calculs et des traitements de données à grande vitesse. L'automatisation est le fait d'automatiser, de rendre automatique l'exécution d'une tâche ou d'une suite d'opérations, sans intervention de l'homme.

KISS

January 18, 2024 9:41 AM

K*eep* **I***t* **S***imple,* **S***tupid*

Non, il ne s’agit pas d’embrassade ou d’un band de rock 🤘

Il s'agit bien d'un acronyme anglophone.

Bon, je préfère **Smart**, d'autre utilisent **Soft**, **So**, **Silly**, **Super**, **Short, Significant**, **Sweet** et, pourtant, originalement c'est **Stupid**. Je pense que l'intention était de déranger/provoquer pour semer avec impacte !

En français, on pourrait le traduire par "**Ne complique pas les choses même si ce n'est pas jolie/original**".

C’est un principe popularisé en développement logiciel qui préconise de privilégier autant que possible la simplicité lors de la construction d’une application.

En effet, la complexité (non justifiée) a les conséquences suivantes, pour des gains parfois discutables :

- Surcoût de développement
- Surcoût de maintenance

Cette complexité peut prendre plusieurs formes comme :

- Une architecture surdimensionnée par rapport aux besoins
- L’ajout de fonctionnalités secondaires ou non prioritaires

On ne peut pas être trop simple quand les objectifs sont atteints.
C’est rarement simple d'atteindre la simplicité 🤪

Libération de bugs (bug release)

January 18, 2024 9:41 AM

“Bug release” ou “libération de bugs” se produit lorsqu’une version d’un logiciel est publiée avec un ensemble de bugs connus. Ces bugs sont généralement de faible gravité/priorité. Cela signifie que l’équipe de développement a pris la décision de publier le logiciel malgré la présence de ces bugs, en estimant que leur impact sur l’expérience utilisateur serait minime.

Livraison

January 18, 2024 9:41 AM

Livraison désigne le processus de distribution du logiciel aux utilisateurs finaux.

Manifeste Agile

January 18, 2024 9:42 AM

Le ***Manifeste pour le développement agile de logiciels*** (*The Manifesto for Agile Software Development*, titre original en anglais) est un texte rédigé par dix-sept experts du développement d'applications informatiques sous la forme de plusieurs méthodes dites agiles. Ces experts estimaient que le traditionnel cycle de développement en cascade ne correspondait plus aux contraintes et aux exigences des organisations en évolution rapide. Les méthodes agiles ne sont pas apparues avec le Manifeste en 2001 mais celui-ci détermine leurs dénominateurs communs et consacre le terme d'« agile » pour les référencer. Les valeurs et principes du Manifeste agile sont défendus par l'Agile Alliance.

Les 4 valeurs

- **aux individus et leurs interactions** plutôt qu'aux processus et aux outils ;
- **à un logiciel fonctionnel** plutôt qu'à une documentation exhaustive ;
- **à la collaboration avec les clients** plutôt qu'à la négociation contractuelle ;
- **à l'adaptation au changement** plutôt qu'à l'exécution d'un plan.

Les 12 principes AGILE

1. **SATISFACATION**
   Notre priorité absolue est de satisfaire grâce à la livraison rapide et continue de logiciels de valeur.
2. **FLEXIBILITÉ**
   Accueillez les exigences changeantes, même tardivement dans le développement. Les processus agiles exploitent le changement pour l'avantage concurrentiel du client.
3. **CONSTANCE**
   Fournissez fréquemment des logiciels fonctionnels, de quelques semaines à quelques mois, en privilégiant les délais les plus courts.
4. **UNICITÉ**
   Métiers et développeurs doivent travailler ensemble au quotidien tout au long du projet.
5. **CONFIANCE**

Construire des projets autour d'individus motivés.
Donnez-leur l'environnement et le soutien dont ils ont besoin,
et faites-leur confiance pour faire le travail.

6. **DISCUSSION**
   La méthode la plus efficiente et la plus efficace pour
   transmettre des informations à une
   équipe de développement et au sein de celle-ci est la conversation en face à face.
7. **EXPÉRIMENTATION**
   Retravailler un logiciel est la principale mesure de progrès.
8. **CONTINU**
   Les processus agiles favorisent le développement durable.
   Les sponsors, développeurs et utilisateurs doivent pouvoir
   maintenir indéfiniment un rythme constant.
9. **MESURE**
   Une attention continue à l'excellence technique
   et à une bonne conception améliore l'agilité.
10. **SIMPLICITÉ**
    La simplicité - l'art de maximiser la quantité
    de travail non fait - est essentielle.
11. **AUTONOMIE**
    Les meilleures architectures, exigences et conceptions
    émergent d'équipes auto-organisées.
12. **AMÉLIORATION**
    À intervalles réguliers, l'équipe réfléchit à la manière
    de devenir plus efficace, puis ajuste et ajuste
    son comportement en conséquence.

ref.

Manifesto for Agile Software Development (agilemanifesto.org)

Manifeste agile — Wikipédia (wikipedia.org)

Matrice de test Agile (quadrants de test)

January 18, 2024 9:42 AM

L'objectif de la matrice est précisément d'effectuer les tests minimaux qui peuvent fournir la valeur maximale dans tous les contextes.

Les quadrants de test Agile aident toute l'équipe à communiquer et valider avec les experts dans le domaine et à fournir un produit de haute qualité en un rien de temps. La matrice c'est une cartographie qui divise l'ensemble de la méthodologie de test agile en quatre quadrants de base. Le processus global est séparé en quatre quadrants qui permettent de comprendre la façon dont les tests Agile sont exécutés. Donc, toute l'équipe de développement vise à travailler plus efficacement ensemble..

Cela aide à voir rapidement dans quelle direction prendre et le "comment" automatiser. Nous aide également à déterminer vers quelles ressources (personnes) se tourner pour chaque test ou groupe de tests.

La matrice nous fournit une taxonomie (science de la classification) utile pour aider les équipes à identifier, définir et planifier les tests nécessaires.

L'ingrédient commun des 4 quadrants est le besoin, d'une équipe travaillant ensemble vers un objectif commun, d'obtenir la meilleure qualité de produit et la satisfaction optimale du client en explorant toutes les différentes validations possibles.

Vous voulez en savoir plus...

- La matrice est une cartographie , un diagramme de "Venn" qui place différents tests sur deux axes : **métier/affaire** versus **technologie** et **programmation** versus **produit**.
- Son nom original est la matrice de test Marick (Marick Test Matrix), plus connu sous le nom des quadrants de test Agile.
- Dans cet article, lorsque je parle d'**exemple**, je veux dire des exemples de la manière dont les codeurs utiliseront la chose en cours de développement.
- Vous remarquerez qu'on ne dit pas "**le quadrant**" mais bien "**les quadrants**", voilà pourquoi les "Q1, Q2, Q3 et Q4" (ce n'est pas chronologique c'est un identifiant référentiel)
- L'origine de la matrice de test Agile provient du besoin de répondre à cette question : "avons-nous couvert tous les contextes, les bases, les sphères, les départements, avons-nous testé sous tous les angles ?"

Voici la matrice de test par quadrants :

Avez-vous remarqué la séparation entre prévention (gauche) et détection (droite)

Il n'y a pas, non plus, de types de test ou des tests automatisés ou manuels, etc.
Pas d'inquiétude, tout est normal, car la matrice n'est pas une recette à suivre. Tout sera clair lorsque vous pratiquerez son utilisation.

Définition des axes

Métier/affaire

Un test orienté affaire est un test que vous pourriez décrire à un expert en affaires dans des termes qui l'intéresseraient (ou devraient l'intéresser). Si vous parliez au téléphone et que vous vouliez décrire les questions auxquelles le test répond, vous utiliseriez des mots tirés du domaine des affaires : « Si vous *retirez* plus *d'argent* que vous n'en avez sur votre *compte* , le système vous accorde-t-il automatiquement un *prêt* pour la différence?"
(Ces catégories ont des limites floues, comme beaucoup le font. Par exemple, le choix des configurations de navigateur à tester est en partie une décision métier (affaire)

Technologiques

Un test orienté sur la technologie est celui que vous décrivez avec des mots tirés du domaine des programmeurs : "Différents navigateurs implémentent *Javascript* différemment, nous testons donc si notre produit fonctionne avec les plus importants." Ou : "PersistentUser#delete ne doit pas se plaindre si l'enregistrement de l'utilisateur n'existe pas."

Programmation

Par "appuyer la programmation", j'entends que les programmeurs utilisent les tests comme partie intégrante de l'acte de programmation. Par exemple, certains programmeurs écrivent un test avant le développement pour déterminer quel code écrire ensuite. En écrivant ce code, ils modifient une partie du comportement du programme. Exécuter le test après le changement les rassure sur le fait qu'ils ont changé ce qu'ils voulaient. L'exécution de tous les *autres* tests les rassure sur le fait qu'ils n'ont pas modifié le comportement qu'ils n'avaient pas l'intention d'altérer.

Produit

Les tests qui critiquent le produit ne sont pas centrés sur l'acte de programmer. Au lieu de cela, ils regardent un produit fini avec l'intention de découvrir des insuffisances, anomalies, omissions, oublis...

Détails des quadrants

Q1 | Utiliser la technologie par l'**exemple** pour appuyer la programmation

Dans ce quadrant, on se concentre sur la qualité interne du code, avec des cas de tests pilotés par la technologie et mis en œuvre pour aider l'équipe.

- C'est en symbiose avec le développement piloté par les tests l'exemple (Shift-left)
  - Le ATDD/TDD/BDD/Spécifications/Requis/Exigences (on ne parle pas juste ici du TDD de tests unitaires qui valident le code)
- Valider/orienter notre code de façon fonctionnelle
- Il y a aussi des outils comme SonarQube qui peuvent aider à avoir plus de confiance sur notre code
- Il y a aussi l'intelligence artificielle qui peut également aider à avoir plus de confiance sur notre code

Q2 | Utiliser des **exemples** orientés métier pour soutenir le travail de toute l'équipe (pas seulement les programmeurs)

Dans ce quadrant, on se concentre sur la qualité externe du code, avec des cas de tests pilotés par le métier et mis en œuvre pour aider l'équipe.

Il s'agit ici d'inciter les programmeurs à écrire le bon code, améliorer les conversations entre les experts en technologie et les experts métier et aider les experts métier à réaliser plus rapidement les possibilités inhérentes au produit.

- **Provoquer le bon code**
  Il s'agit d'une extrapolation simple de la conception standard basée sur les tests (ou basée sur des exemples) des interfaces internes à l'interface du produit entier. Pour ajouter une nouvelle fonctionnalité, commencez par un ou plusieurs exemples de son fonctionnement. Le programmeur fait alors correspondre le code à cet exemple. Le flux d'exemples continue jusqu'à ce que le code requis soit en place. Ensuite, la fonctionnalité est terminée (pour l'instant).
  Bien que l'extrapolation soit simple, nous avons du chemin à parcourir avant d'avoir aplani les détails, avant que la pratique ne soit bien comprise.
- **Améliorer les conversations de projet**
  Cela n'a pas plus de sens de jeter des exemples par-dessus le mur à un programmeur et de s'attendre à ce qu'il écrive le bon code que de le faire avec des exigences. Les programmeurs ont besoin de contexte, d'arrière-

plan et de connaissances tacites. Ils obtiennent cela grâce à des conversations avec des experts en affaires. Les exemples peuvent améliorer cette conversation, ne serait-ce qu'en donnant aux gens de quoi parler . Ils ancrent la conversation dans le concret.
Là où les exemples peuvent aider particulièrement, je pense, c'est à forger un vocabulaire commun. Je suis fan de l'idée que la terminologie de domaine devrait être "réifiée" en étant transformée en objets dans le code. Pas dans le style naïf « écrivez sur le domaine et soulignez tous les noms » de la conception orientée objet, mais dans le style plus sophistiqué du Domain-Driven Design d'Eric Evans .
Donc, ce que nous devons avoir, c'est un processus par lequel la connaissance du domaine comprise de manière floue est rendue très concrète, transformée en 1 et en 0. Il me semble que les exemples sont une étape intermédiaire, un moyen de s'approprier progressivement des connaissances. Mais, comme pour l'utilisation d'exemples pour guider les programmeurs, il reste beaucoup de connaissances à apprendre.

- **Rendre les possibilités plus perceptibles**
  Nous voulons que les experts commerciaux aient "Aha!" moments, dans lesquels ils se rendent compte que parce que le produit fait A de manière B et aussi X de manière Y , il est logique qu'il fasse un nouveau Z qui n'avait pas été imaginé auparavant. Nous souhaitons également que d'autres personnes de l'équipe aient le même type de réalisations, qu'elles pourront ensuite proposer aux experts métiers. Bref, nous voulons de la créativité.
  La meilleure façon de libérer la créativité est probablement de mettre la main sur un logiciel fonctionnel et de l'essayer. (J'écrirai plus à ce sujet dans un article ultérieur.) Mais une autre façon est d'expliquer un exemple à quelqu'un d'autre. Avez-vous déjà eu du mal à trouver un bogue, puis l'erreur vous a-t-elle sauté aux yeux dès que vous avez commencé à expliquer le code à quelqu'un d'autre ? Pour moi, écrire de la documentation utilisateur a un effet similaire : j'utilise des exemples pour expliquer quelles sont les idées fondamentales derrière le logiciel et comment elles s'articulent. Très souvent, je me rends compte qui ne le font pas. C'est le même sentiment qu'avec les bugs, même si la personne à qui je l'explique est un lecteur imaginaire, pas une vraie personne assise à côté de moi.
  Ainsi, la manière dont nous créons des exemples et en discutons peut accélérer l'évolution du produit.

Q3 | Utiliser le **test** orientés métier pour concevoir les produits

Dans ce quadrant, on se concentre sur la qualité externe du code, avec des

cas de tests pilotés par le métier et mis en œuvre pour critiquer le produit.

- Ici c'est très en mode exploratoire en jumelage avec un utilisateur, sme, designer, qa, analyste affaire, fonctionnel ou même n'importe qui n'étant pas un programmeur.
  - Cela ne fonctionne plus lorsque l'utilisateur connait et/ou s'adapte au produit
- Ce ne sont pas ici des UAT ou des tests de règles d'affaires

Q4 | Utiliser le **test** orienté technologie pour critiquer le produits

Dans ce quadrant, on se concentre sur la qualité interne du code, avec des cas de tests pilotés par la technologie et mis en œuvre pour critiquer le produit.

- C'est la partie technique, mécanique… Nous allons tester l'efficacité et les limites technologiques et du produit (stabilité, fiabilité, maintenabilité, etc.)
- Des tests non fonctionnels

Utilisation

Utilisation des quadrants de test Agile

Suivi de la couverture de test avec la matrice de test Agile

Suivi de la couverture de test avec la matrice de test Agile (quadrants)

Relation des rôles avec la matrice par quadrants de test Agile

Relation des rôles avec la matrice par quadrants de test Agile

ATTENTION !

On voit souvent des adaptations douteuses des quadrants sans avoir au préalable fait l'effort d'en comprendre l'utilité, ce qui peut nous réserver de mauvaises surprises et ajouter à l'incompréhension.

Chez Alithya, nous n'encourageons pas de complexifier les quadrants pour répondre à d'autres besoins.

- Que le test soit automatisé ou manuel, il se retrouve dans tous les quadrants, donc les quadrants qui spécifie le manuel versus automatisé ne sont pas des quadrants Agile originaux
- Mixer la pyramide, les quadrants, et les meilleurs pratique n'est certainement pas le but des quadrants
- Les niveaux, les types, les tactiques, techniques des tests ne sont pas non plus l'objectif des quadrants

Voici des exemples à ne pas suivre :



Références :

- Exploration Through Example (exampler.com)

La matrice (les quadrants) a été initialement décrite par Brian Marick et a été popularisée par Lisa Crispin et Janet Gregory dans leur livre Agile Testing : A Practical Guide for Testers and Agile Teams.

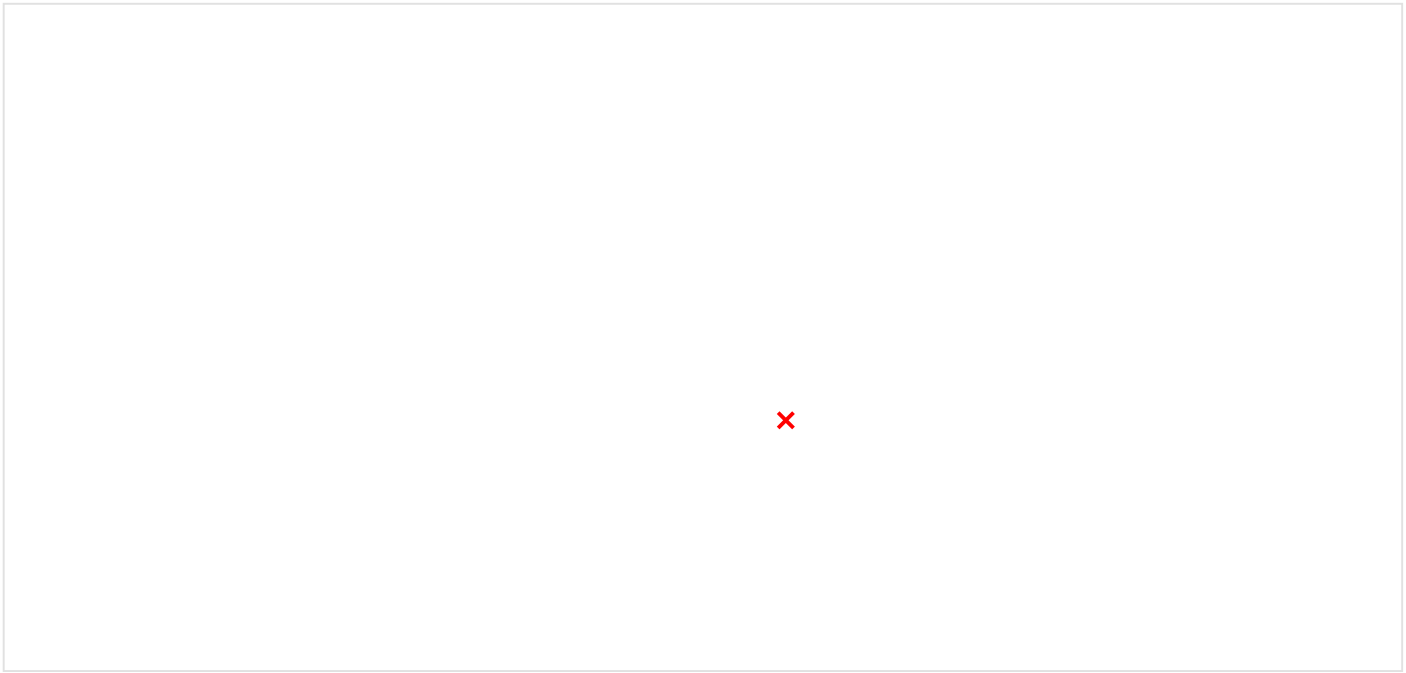
- Using the Agile Testing Quadrants - Agile Testing with Lisa Crispin and Janet Gregory

Matrice de traçabilité (RTM)

January 18, 2024 9:43 AM

En assurance qualité du logiciel, une matrice de traçabilité (RTM) est un document qui permet de suivre les exigences tout au long du cycle de vie du développement logiciel. Elle permet de s'assurer que toutes les exigences sont satisfaites et que les tests sont effectués pour chaque exigence. La matrice de traçabilité est généralement créée à partir d'un tableau qui relie les exigences aux cas de test.

Voici un exemple de matrice de traçabilité des exigences :



Maturité (le delta en assurance qualité logicielle)

January 18, 2024 9:44 AM

Le delta en assurance qualité logicielle fait référence à la différence entre l'état actuel de l'assurance qualité et l'état souhaité. Il peut être utilisé pour mesurer l'efficacité des processus d'assurance qualité et pour identifier les domaines qui nécessitent des améliorations.

La maturité en assurance qualité logicielle fait référence au niveau de maturité des processus d'assurance qualité dans une organisation. Elle peut être mesurée à l'aide de modèles de maturité tels que le modèle Capability Maturity Model Integration (CMMI). Les organisations peuvent utiliser ces modèles pour évaluer leur niveau de maturité actuel et identifier les domaines qui nécessitent des améliorations.

ref. : Critères d'évaluation de maturité des professionnels QA

Mock

January 18, 2024 9:44 AM

Certains appellent ça des bouchons, d'autres des stubs, ou encore des mocks, des fakes, des switch, des splits, émulateur/simulateur… bref ce que la littérature appelle ça des "Test Double" ("Doublure" pour les Tests, en français). Il s'agit d'un objet sur lequel nous avons le contrôle et qui va se substituer à une dépendance de notre objet testé et nous permettre de valider différents comportements attendus en fonction des retours de la doublure (cas nominal, aux limites ou d'erreurs, etc.).

J'aime commencer en disant : "Même environnement complet de dev est un moque de la prod 😳"

Mais on moque quoi au juste ? …

1. Le data et la logique

Prérequis :

- Modulaire? Si votre application n'est pas modulaire, il sera difficile de moquer
- Bien moquer avec suivi du changement de l'original (si l'original change, le mock aussi)
- Un système de gestion des données commun et efficace (ex. json, xml, csv, sqlite, etc…)
- Avoir accès à tous les protocoles

Doit être mis en œuvre en premier et avec réaction rapide, sans trop de maintenance!

Risqué	**Suffisant**	**Trop**
Non fiable et non aligné (déconnecté) avec le vrai  Non fiable et non aligné (déconnecté) avec le vrai	Contrat de confiance et de besoin réel (connecté et mis à jour en continu)  Contrat de confiance et de besoin réel (connecté et mis à jour en continu)	Même effort ou plus que le vrai. Ne pas confondre avec les "jumeaux numériques"  Même effort ou plus que le vrai. Ne pas confondre avec les "jumeaux numériques"

Modularité (Modularisation)

January 18, 2024 9:45 AM

La modularité sont, dans un cadre de test, de petits modules indépendants de parties d'un test. Ces modules individuels sont construits ensemble pour former un test réalisant un cas de test ou scénario particulier. La modularité des tests permet une plus grande flexibilité, une meilleure réutilisation, une clarté, maintenabilité efficace et des exécutions simultanées.

Niveaux de test

January 18, 2024 9:45 AM

Un niveau est un point sur une échelle, par exemple une échelle de quantité, de qualité ou de difficulté. Dans le contexte des tests de logiciels, les niveaux font référence aux différents niveaux d'une application. Les niveaux d'application sont en couches et peuvent être “mock-ées”. Les “mock” au niveau de l'application sont implicites et nécessaires pour définir les niveaux de test. C'est pourquoi on peut valider un test sur plusieurs niveaux. Dans le contexte où nous n'avons pas de couches “mock-ables“, il n'est pas possible de tester par niveaux et donc on peut envisager des tests boîte noire.

Il y a trois niveaux de test :

- Unitaire
- Intégration
- Système

Un niveau de test est l'entrée et la sortie d'une unité, d'un groupe d'unités ou sous-systèmes/systèmes qui est défini par son mock *Tous les environnements autres que "PROD" sont moqués !

Attention un niveau n’est pas un type ou une phase. On remarque cette erreur fréquente en français contrairement en Anglais qui est bien défini.

C'est pourquoi "acceptation" n'est pas/plus un niveau, même si on le voit encore trop sur internet et même du côté de l'ISTQB. Cette amélioration du côté de l'ISTQB est en cours mais n'est pas encore officielle cependant car c'est assez logique... du côté d'Alithya, nous suivons le cap de l'amélioration.
Un type de test peut se retrouver dans un ou plusieurs ou tous les niveaux de test et une ou plusieurs ou toutes les phases de test.
Donc "acceptation" n'est tout simplement pas un niveau !

Ça peut servir à quoi les niveaux ?

L’architecture, la modélisation possède les niveaux du corps, le cœur et l'esprit

- Peut grandement faciliter la traçabilité, la proactivité, la notification et la résolution profonde des "bugs"

- Cette modularité nous permet d'appliquer de manière fragmentée, éclatée, autonome et indépendante, au fur et à mesure tout en ayant une vision claire et commune de la couverture/évolution du système global et donc aider à faire de la “qualité/intégration-livraison” facilité par l’approbation par module, plus efficacement et rapidement.
- Peut aider à attacher la couverture de la qualité à l’architecture/modèle avant l’intégration
- Peut servir d'éviter de sur-tester
- Peut servir d’entrée de jeu aux meilleures pratiques de la qualité (ex. “shift-left”, Ci-Cd, DevOps, etc.)
- Peut aider dans l'élaboration des ***prérequis*** d’une stratégie de test.
    - Valoriser l’automatisation.
    - Servir de levier pour souligner l'importance de solidifier/stabiliser les déploiements et les environnements de test
    - **Faire voir l'indispensable besoin de moquer avant de commencer à tester**
- Peut également servir d’encouragement pour briser les silos de la qualité
- Peut voir l’impact de faire le même test en system, intégration et unitaire

Il est important de savoir qu'un niveau n’est…

- **pas** une phase, une “gate”, une promotion
- **pas** une catégorie, classification, type, plan de test
- **pas** une méthode, tactique, manuel/automatisé
- **pas** un environnement, contexte, E2E
- **pas** une stratégie

Niveau de test d'intégration

January 18, 2024 9:45 AM

Moqué à un ensemble de plusieurs unités (fonctionnalité/sous-système)
Validations de boulonnage/assemblage/connection

Moqué à l'entrée et la sortie d'un ensemble d'unités

ex. Une fonction global, un produit, une application, une suite logicielle, une librairie, service/api

Test d'intégration — Wikipédia (wikipedia.org)



Niveau de test unitaire

January 18, 2024 9:46 AM

Moqué à l'unité (validation du component)

ex. un composant, un module, une partie… non divisible

Unitaire :
Moqué à l'unité, oui mais, qui est surtout **autonome/indépendant des autres tests !**

Le niveau unitaire vérifie si chaque module autonome fonctionne correctement.

Le but de ces tests de niveau unitaire est d'isoler chaque module du système et de les tester séparément.

En français on entend aussi **tests d'unités**

Les tests unitaires sont de très bas niveau, près de la source de votre application. Ils consistent à tester les méthodes et fonctions individuelles des classes, des composants ou des modules utilisés par votre logiciel. Les tests unitaires sont en général assez bon marché à automatiser et peuvent être exécutés très rapidement par un serveur d'intégration continue.

En programmation informatique, le test unitaire (ou « T.U. », ou « U.T. » en anglais) est une procédure permettant de vérifier le bon fonctionnement d'une partie précise d'un logiciel ou d'une portion d'un programme (appelée « unité » ou « module »).

Attention, dans le jargon de l'assurance qualité, il est important de différencier le **niveau,** la **méthode** et **le type.**

Par exemple : Le développeur utilise le mot "unit test" et va souvent dire que c'est un test unitaire ou un test de type unitaire et rarement un test de niveau unitaire… Qui à vrai dire, est une méthode pour tester au niveau unitaire (unitairement). Nous pouvons dire tester unitairement que le code est conforme… C'est donc ici, un type de test sanitaire ou d'intégrité.

De plus si nous faisons du TDD, nous validons l'intégrité du code mais aussi les spécifications/comportements fonctionnels unitairement, car le code est

structuré de façon à respecter le fonctionnel (outch!)
Oncle Bob est une super référence

Validation unitaire de l'intégrité de l'interface utilisateur

Le test d'interface utilisateur unitaire est un type de test qui permet de vérifier le comportement d'une interface utilisateur. Les tests unitaires permettent de vérifier le bon fonctionnement d'une petite partie bien précise (unité ou module) d'une application. L'entrée à utiliser dans un test unitaire doit être la plus simple possible afin de vérifier le comportement que vous testez actuellement.

Validation unitaire de l'intégrité de la performance

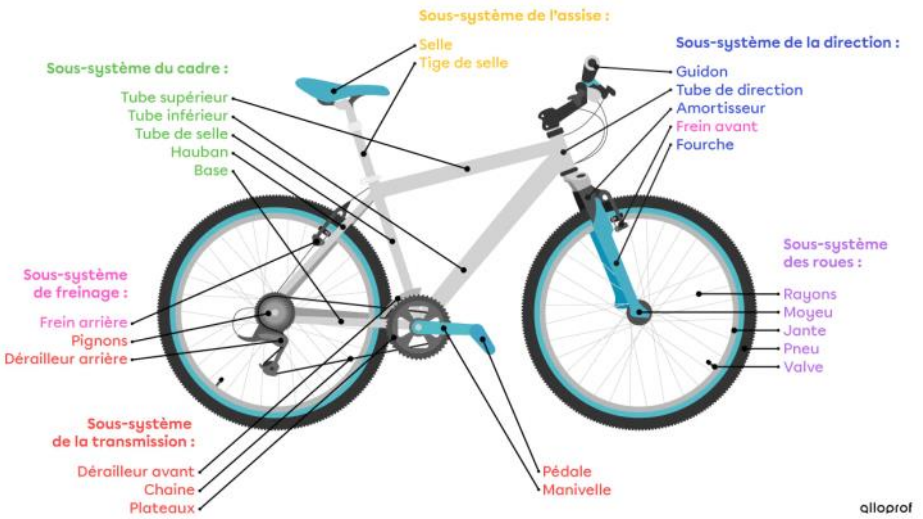
...

Validation unitaire de l'intégrité des données

...

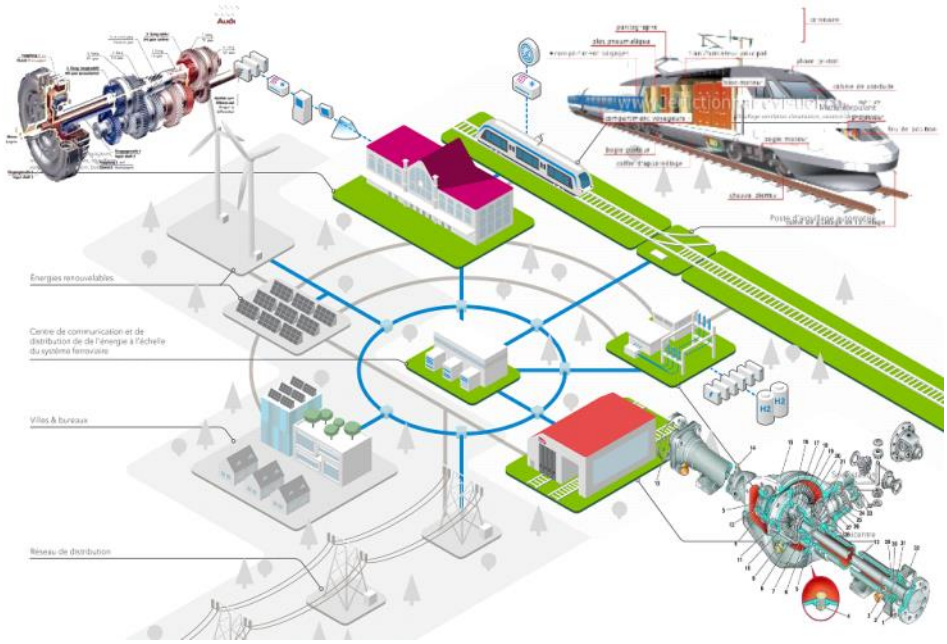
Validation unitaire de l'intégrité du code (unit test)

...



Niveau de test système

January 18, 2024 9:46 AM



Moqué à un ensemble de plusieurs sous-systèmes ou fonctionnalités (système)
C’est ici les validations de mise en service

Moqué les sous-systèmes

ex. système d’authentification/éligibilité/membre, système de session, système de traçabilité, système de rapports, un sous-système ex. système d’alertes/sécurité, système électrique, système de chauffage, système de commutation, système de temps (calendrier/période/chronomètre), système économique, système de traitement de l'information, système d’exploitation, système de contrôle, système solaire, système ferroviair…

Un système peut être ouvert, fermé, ou isolé selon son degré d’interaction avec son environnement.

Il y a aussi la nature conceptuelle et concrète.

Pourquoi "SYSTEM" et non pas "E2E" !

Simplement parce que les "E2E" sont inclues dans SYSTEM

Tests système — Wikipédia (wikipedia.org)

Ordinateur

January 18, 2024 9:47 AM

latin: ordinator, celui qui met en ordre.

Un ordinateur est un système de traitement de l'information programmable par la lecture séquentielle d'un ensemble d'instructions, organisées en programmes, qui lui font exécuter des opérations logiques et arithmétiques.

Machine automatique de traitement de l'information, obéissant à des programmes formés par des suites d'opérations arithmétiques et logiques.

Un ordinateur est un système de traitement de l’information programmable tel que défini par Alan Turing et qui fonctionne par la lecture séquentielle d’un ensemble d’instructions, organisées en programmes, qui lui font exécuter des opérations logiques et arithmétiques. C’est une machine dotée d’une unité de traitement lui permettant d’exécuter des programmes enregistrés. C’est un ensemble de circuits électroniques permettant de manipuler des données sous forme binaire ou bits.

Paradoxe du pesticide

January 18, 2024 9:47 AM

Le paradoxe du pesticide est un principe de test logiciel qui stipule que si les mêmes tests sont répétés encore et encore, ils finiront par ne plus être efficaces pour trouver de nouveaux défauts1. Cela est dû au fait que les tests sont conçus pour détecter des défauts spécifiques, et une fois que ces défauts ont été corrigés, les mêmes tests ne seront plus en mesure de détecter de nouveaux défauts qui pourraient être introduits dans le logiciel.

Ce principe est pertinent dans les tests de non-régression, car ces tests sont souvent répétés à chaque nouvelle version du logiciel pour s'assurer que les changements n'ont pas introduit de nouveaux défauts. Pour éviter le paradoxe du pesticide dans les tests de non-régression, il est important de mettre à jour régulièrement les tests pour s'assurer qu'ils couvrent les nouvelles fonctionnalités et les changements apportés au logiciel. Cela peut être réalisé en ajoutant de nouveaux tests, en modifiant les tests existants ou en remplaçant les tests obsolètes par de nouveaux tests plus pertinents.

En résumé, pour éviter le paradoxe du pesticide dans les tests de non-régression, il est important de mettre à jour régulièrement les tests pour s'assurer qu'ils couvrent les nouvelles fonctionnalités et les changements apportés au logiciel

Paradoxe du pesticide

January 18, 2024 9:47 AM

Le paradoxe du pesticide est un principe de test logiciel qui stipule que si les mêmes tests sont répétés encore et encore, ils finiront par ne plus être efficaces pour trouver de nouveaux défauts. Cela est dû au fait que les tests sont conçus pour détecter des défauts spécifiques, et une fois que ces défauts ont été corrigés, les mêmes tests ne seront plus en mesure de détecter de nouveaux défauts qui pourraient être introduits dans le logiciel.

Ce principe est pertinent dans les tests de non-régression, car ces tests sont souvent répétés à chaque nouvelle version du logiciel pour s'assurer que les changements n'ont pas introduit de nouveaux défauts. Pour éviter le paradoxe du pesticide dans les tests de non-régression, il est important de mettre à jour régulièrement les tests pour s'assurer qu'ils couvrent les nouvelles fonctionnalités et les changements apportés au logiciel. Cela peut être réalisé en ajoutant de nouveaux tests, en modifiant les tests existants ou en remplaçant les tests obsolètes par de nouveaux tests plus pertinents.

En résumé, pour éviter le paradoxe du pesticide dans les tests de non-régression, il est important de mettre à jour régulièrement les tests pour s'assurer qu'ils couvrent les nouvelles fonctionnalités et les changements apportés au logiciel

Plan de test

January 18, 2024 9:47 AM

Un **plan de test** (livraison/release/delivery test plan) est un document détaillé qui décrit et met en action les objectifs de la stratégie de test, les objectifs, les ressources nécessaires, le calendrier et les critères de réussite pour assurer la qualité d'une livraison/version d'une fonctionnalité ou une suite ou une librairie ou une epic ou une story ou un logiciel spécifique, etc…

On y retrouve par exemple la portée, la couverture, objectif de test, où quels environnements, document de références (rapports de suivi, stratégie, release notes…), approches, analyses, types de tests couverts, horaire, ressources, suivi des bugs, complétion status.

L'objectif principal de la stratégie et du plan de test, c'est bien sûr, être pro-actif, prévenir, découvrir les défauts, les erreurs et toute autre lacune qui pourrait empêcher le logiciel d'agir comme prévu ou de fournir une mauvaise expérience à vos utilisateurs.

N'oublions pas que le plan de test est l'engagement en action de la stratégie et garantit à votre logiciel une qualité satisfaction client selon une livraison, donc un nouveau plan une nouvelle planification à chaque livraisons/versions avec la même stratégie qui n'est pas inclus dans le plan de test mais implicite.

Un plan de test (ou plan d'essai) décrit généralement les objectifs des tests, les fonctionnalités à tester, les étapes de test à suivre, les données d'entrée à utiliser et les critères d'acceptation pour chaque test. Le plan de test peut également inclure des informations sur les ressources nécessaires pour effectuer les tests, telles que le matériel, les logiciels et le personnel.

Le plan de test est un document important qui aide à organiser et à planifier les tests pour s'assurer que toutes les fonctionnalités du logiciel sont testées de manière approfondie et rigoureuse. Il peut être utilisé par les testeurs, les développeurs et les gestionnaires pour suivre l'avancement des tests et s'assurer que tous les objectifs de test sont atteints.


Roadtrip avec enfants
CHECKLIST
JEUX
Ni oui, ni non
Devine à qui je pense
Jacques à dit
J'ai emporté dans ma valise
Pierre, papier, ciseaux
Roi du silence
NOURRITURE/BOISSON
Eau
Jus
Gâteaux
Fruits
Snacks
Sandwichs
Barres de céréales
ET N'OUBLIEZ PAS
De faire un arrêt toutes les deux heures, même si ce n'est que pour 10 minutes
Essayez de garder une température agréable dans la voiture
Vérifiez les sièges, ceintures et autres avant le voyage
Faites réviser la voiture entièrement (moteur, roues, etc)
Vérifiez la météo sur la route que vous allez parcourir
Prenez une tétine de rechange si vous voyagez avec un bébé
Ne vous pressez pas et profitez du trajet
Pour chanter ensemble
- Un éléphant, qui se balançait
- Il était un petit navire
- Un kilomètre à pied
- Une souris verte
LES INDISPENSABLES
Mouchoirs
Musique
Trousse de pharmacie
Vêtements de rechange
Chargeurs
Médicaments contre la nausée
GPS ou carte
Coussin
Appareil photo
Sac poubelle
POUR SE DISTRAIRE
Livre de coloriage
Feuilles blanches
Poupées, peluches...
Lecteur DVD portable ou Tablette
Films, séries ou dessins animés
Contes
AINSI QUE
Et beaucoup de patience !
muchosol

Poka-Yoke

January 18, 2024 9:48 AM

Le Poka-Yoke est une méthode de prévention des erreurs qui signifie « éviter les erreurs » ou plus littéralement « éviter les erreurs involontaires ». Le Poka-Yoke vise à s'assurer que les bonnes conditions existent avant qu'une étape de processus ne soit exécutée, empêchant ainsi les défauts de se produire en premier lieu. C'est un outil fondamental du Lean Six Sigma qui se réfère à des stratégies qui empêchent les erreurs de se produire dans un processus.

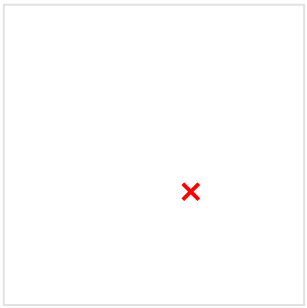
Programmation en binôme ou programmation en pair (pair programming)

January 18, 2024 9:48 AM

Accéder directement au contenu principal

SharePoint

MB



Pratique AQ (Tous)

▪ Groupe privé

Accueil Terminologies Académie Diffusions Événements Organigramme

▪

Modifier

Suivi

86 membres

Créer

Promouvoir

Détails de la page

Lecteur immersif

Analyse

Publié le 11/05/2023

Partager

Modifier

▪

La programmation en binôme (de l'anglais pair programming), parfois appelée programmation par pairs ou binômage, est une méthode de travail dans laquelle deux développeurs travaillent ensemble sur un même poste de travail. La personne qui rédige le code est appelée conducteur (driver).

La seconde personne, appelée observateur (observer), assiste le conducteur en décelant les imperfections, en vérifiant que le code implémente correctement le design et en suggérant des alternatives de développement. Les rôles s'échangent régulièrement pendant la séance de programmation.

La programmation en binôme peut offrir plusieurs avantages. Par exemple, elle peut améliorer la qualité du code, car deux personnes travaillent ensemble pour écrire et examiner le code. Cela peut également réduire le nombre d'erreurs et de bugs, car les deux personnes peuvent se relire mutuellement et détecter les erreurs plus rapidement. De plus, la programmation en binôme peut favoriser la collaboration et l'apprentissage entre les membres de l'équipe, car ils travaillent ensemble et partagent leurs connaissances et leur expertise.

Principes de l'assurance qualité

January 18, 2024 9:49 AM

L’assurance qualité du logiciel (AQL) est un ensemble d’activités planifiées et systématiques visant à fournir une assurance suffisante que la qualité logicielle d’un nouveau logiciel ou d’une nouvelle version d’un logiciel est conforme aux exigences et aux attentes établies.

Nous parlons ici des principes de la politique ou de la stratégie de l'Assurance qualité, donc il ne faut pas confondre avec les principes de la stratégie de test.

Les principes incluent:

1. Respect des normes et standards internationaux : Les normes et standards internationaux en matière de qualité logicielle doivent être respectés pour garantir la qualité des produits logiciels.
2. Qualité dans les processus logiciels : Les processus de développement, de test et de maintenance des logiciels doivent être gérés de manière à garantir la qualité des produits logiciels.
3. Mesures et métriques : Des mesures et des métriques doivent être utilisées pour évaluer la qualité des produits logiciels et pour identifier les domaines à améliorer.
4. Validation et vérification : Des tests doivent être effectués pour valider et vérifier que les produits logiciels répondent aux exigences spécifiées et fonctionnent comme prévu.
5. Ces principes visent à garantir que les produits logiciels répondent aux exigences de qualité spécifiées et à améliorer continuellement la qualité pour satisfaire les besoins et les attentes des clients.

Priorité

January 18, 2024 9:49 AM

Priorité d’une anomalie

La priorité d’une anomalie est utilisée pour refléter le niveau d'urgence à laquelle l’anomalie doit être corrigée. Les anomalies de priorité 1 et 2 doivent être résolues avant la mise en production. Quant aux anomalies avec la priorité 3, une décision doit être prise, afin de déterminer si le projet procède ou non à la mise en production, ou si elle doit être traitée en tant que problème post-implantation.

La priorité *est une mesure de l'urgence*[1].
Pour définir le niveau de priorité d'un incident, on peut se poser les questions suivantes:
À quelle vitesse devons-nous corriger ce problème ? Quel problème résoudre en premier?[1]

[1] Atlassian. (n.d.). Comprendre les niveaux de gravité des incidents. Atlassian. Retrieved May 4, 2023, from https://www.atlassian.com/fr/incident-management/kpis/severity-levels

Priorité	Description de la priorité
1- Élevé	L’anomalie survenue est considérée comme une priorité absolue, elle doit être résolue pour la prochaine version de code.
2- Moyen	L’anomalie survenue est considérée comme une priorité moindre que la priorité 1, celle-ci nécessite une correction d’ici la fin du niveau de test en cours ou d’ici la prochaine livraison du code.
3- Bas	L’anomalie survenue est considérée comme acceptable pour procéder à la mise en production ou est traitée en tant que problème post-implantation.

Programmation en groupe/équipe (mob programming)

January 18, 2024 9:49 AM

Le Mob Programming est une approche de développement logiciel où toute l'équipe travaille sur la même chose, en même temps, dans le même espace et sur le même ordinateur. Cela étend le concept de la programmation en binôme, où deux personnes travaillent ensemble, à toute l'équipe collaborant en continu sur un seul ordinateur pour livrer un seul élément de travail à la fois.

Programmation extrême (extreme programming ou XP)

January 18, 2024 9:49 AM

Ce n'est pas juste le le pair programming ou le pair testing.

L'Extreme Programming (XP), en français « la programmation extrême », est une méthode agile de génie logiciel qui privilégie l'aspect réalisation d'une application, sans pour autant négliger l'aspect gestion de projet. Elle pousse à l'extrême des principes simples, d'où son nom.

C'est une méthodologie de gestion de projet Agile qui mise sur la rapidité et la simplicité avec des cycles de développement courts et s'articule autour de 5 valeurs fondamentales, 5 règles et 12 pratiques de programmation. Les 5 valeurs fondamentales sont la communication, la simplicité, le feedback, le courage et le respect.

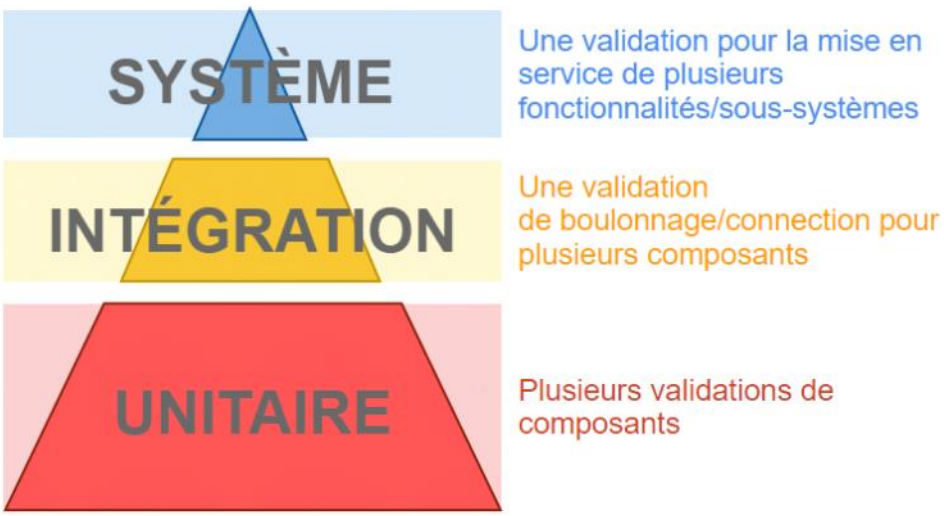
Cela s'articule autour de 5 valeurs fondamentales, 5 règles et 12 pratiques de programmation. Les 12 pratiques de l'Extreme Programming sont la planification progressive, le pair programming, les tests unitaires, la conception simple, le refactoring, l'intégration continue, les itérations courtes, le code standardisé, le client sur site, le code collectif, les 40 heures par semaine et les tests d'acceptation.

La planification progressive est une des 12 pratiques de l'Extreme Programming (XP). Les tâches sont planifiées par petits lots pour permettre une adaptation rapide aux changements. Aussi tôt faites, elles sont livrées pour obtenir des feedbacks aussi fréquemment que possible

La conception simple est une des 12 pratiques de l'Extreme Programming (XP). Elle consiste à privilégier le simple et l'essentiel pour obtenir un résultat efficace. Cette pratique pousse la simplicité et la réactivité au maximum et assure un contrôle continu des versions.

Pyramide de test (par niveaux)

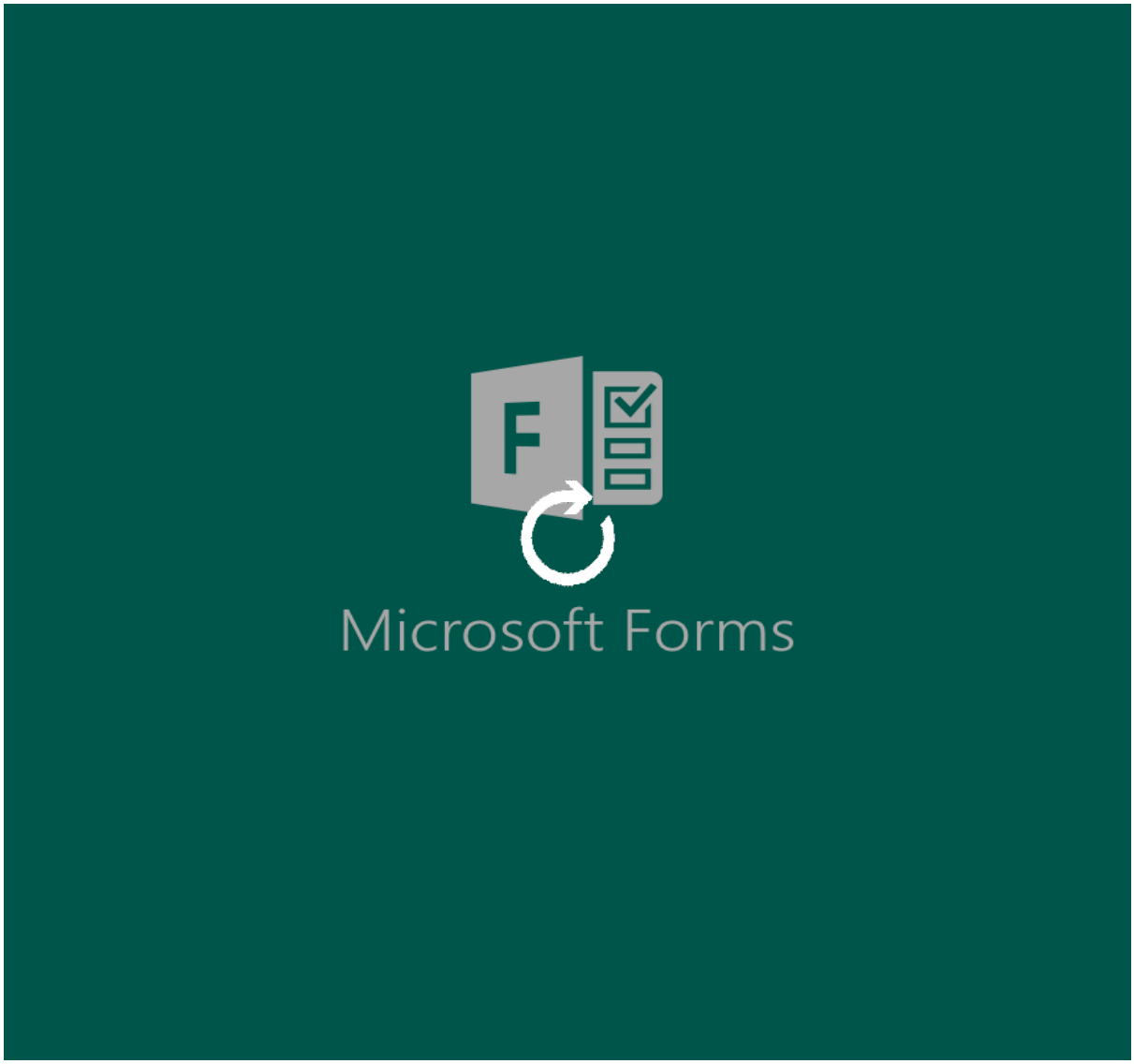
January 18, 2024 9:50 AM



La pyramide de test est une représentation visuelle des différents niveaux de tests dans le développement logiciel. Elle montre comment les tests unitaires, les tests d'intégration et les tests système s'imbriquent les uns dans les autres pour fournir une couverture de test complète. C'est une métaphore cartographique des niveaux de test applicatif (mock) complémentarisés sous forme d'un triangle, dans un découpage des validations de la couverture global idéalisé pour un retour sur investissement. Il est en triangle pour la simple raison que le fait que nous réutilisons la même validation/exécution de test système pour plusieurs validations intégration et que pour unitaire c'est 1:1 donc pas réutilisé/validé. La pyramide de test peut aider les équipes à comprendre le découpage des niveaux de test et à optimiser la quantité de tests en éliminant les tests superflus (un des principes de la stratégie de test).

Bref : Décomposer tous les tests en composants pour éviter de retester le boulonnage/connexion et la mise en service pour tous les tests !

Êtes-vous curieux de connaitre votre niveau de connaissance avec le Quiz éclaire sur la pyramide de test ?



IMPOTANT :

- **Chaque niveau a son importance cruciale !!! (**la pyramide est **FIRST** à tous les niveaux !)
  - **F**oudroyant (Fast/Rapid),
  - **I**solé (Isolated/Independent),
  - **R**épétable (Repeatable),
  - **S**ouverain (Self-Validated/Verification),
  - **T**emps opportun (Timely/Carefully)
- Il est important de comprendre ici que le but n'est pas d'éviter de faire les validations d'intégration et de système !
  Le but est d'éviter de réexécuter les mêmes validations, donc RÉUTILISATION, moins d'exécutions/moins de maintenance… au final ! (Ce n'est que du découpage des validations de la couverture global des tests)
- Les tests d'acceptation utilisateur (UAT), bout en bout ou UI ne font pas partie des niveaux de la pyramide de test, il n'y a pas plus, pas moins que trois niveaux. Voyez les Niveaux de test.

- Il n'y a pas d'ordre dans le temps pour valider les niveaux. Par exemple, si disponible, nous pouvons valider "système" dès le début du cycle de développement avant même implémentation du code (avant unitaire et intégration). Même que c'est la meilleur pratique que de valider dans l'ordre inverser (1.Système, 2.Intégration, 3.Unitaire) pour éviter les mauvaises surprises. Par contre certains tests demande de faire l'intégration après unitaire pour une raison de dépendance de composants.
- La pyramide "les niveaux de test" ne s'appliquent pas dans un contexte de **boîte noire**.
- Si vous voulez en savoir plus sur l'histoire connaître et éviter les pièges...
  ...il y a une série dédiée à ce sujet E0. Histoire et évolution de la pyramide

Exemple :

Unitaire

Lumière... valider la composante.

Dans notre cas, nous validons que la résistance produit de la lumière.

Intégration

Éclairage de la résidence (maison, garage, cour)... valider l'interconnexion des composants.

Système

Réseau électrique... valider le système dans l'ensemble.

Dans notre cas si l'électricité est bien distribuée.

Qu'arrive-t 'il si nous avons trois composants ?

Ajoutons les composants "interrupteur" et "lampe"

En suivant l'idée de la pyramide, nous n'aurons pas besoin de valider intégration à chaque fois que nous testons un composant et devoir aussi valider système à chaque fois que nous testons un composant et/ou intégration.

Nous aurons besoin de valider qu'une seule fois le système, une ou deux fois l'intégration et trois fois unitaire (composants). Donc **un total de 6, plutôt que 12 validations** dans le cas que nous devrions refaire les validations intégrations et système pour chaque validation d'un composant.

Imaginez les économies exponentielles si nous ajoutions d'autres composants qui utilisent la même intégration et le même système.

C'est pourquoi, il est intéressant de découper les validations pour optimiser la quantité de tests (validations).

De plus ! Cela va sans dire que le niveau de couverture et la qualité du produit n'augmenteront ou ne diminueront pas.

Rapport sur l’approche (approach report)

January 18, 2024 9:50 AM

Un “approach report” est un rapport qui décrit les différentes approches possibles pour résoudre un problème donné en matière d’assurance qualité. Ce type de rapport peut être utilisé pour décrire les différentes approches possibles pour la mise en place d’un processus d’assurance qualité dans le cadre d’un projet de développement de logiciel.

Release

January 18, 2024 9:50 AM

Release (version) désigne une version stable et testée du logiciel qui est prête à être distribuée aux utilisateurs.

Refactoring (code)

January 18, 2024 9:50 AM

le refactoring est une technique disciplinée pour restructurer un code existant sans modifier son comportement externe. Le but du refactoring est d'améliorer la qualité interne du code en effectuant de nombreux petits changements sans altérer le comportement externe du code. Les programmeurs informatiques et les développeurs de logiciels refactorisent le code pour améliorer la conception, la structure et la mise en œuvre du logiciel.

ref : Code refactoring - Wikipedia

Régression (regression bug)

January 18, 2024 9:51 AM

Une régression est un type de bug qui démontre qu'une fonctionnalité qui fonctionnait comme prévu ne fonctionne plus comme prévu ou ne fonctionne simplement plus.

Robotisation

January 18, 2024 9:51 AM

Substitution de robots à des opérateurs humains pour l'accomplissement de tâches répétitives humaine.

La robotisation est un processus d’automatisation qui utilise des robots pour effectuer des tâches à la place des humains. Elle est souvent utilisée dans les usines pour automatiser les tâches répétitives et dangereuses pour les humains.

Scénario de test

January 18, 2024 9:51 AM

Un scénario de test est une destination, le parcours, l'itinéraire... c'est, donc, un ensemble d'étapes à suivre pour valider l'arrivée à destination.

Dans cette procédure de test détaillée, on y retrouve le nom du scénario de test, la condition préalable (point A), les étapes/conditions d'entrée (itinéraire) et le résultat attendu (point B).

Il est utilisé pour tester une fonctionnalité de bout en bout et est généralement dérivé des cas d'utilisation. Les scénarios de test **peuvent** servir de base à la création de cas de test de niveau inférieur. Un seul scénario de test **peut** couvrir un ou plusieurs cas de test.

Son but est de tester une fonctionnalité ou un cas d'utilisation. C'est la déclaration décrivant la fonctionnalité de l'application à tester.

n.b. "Scenario" est aussi utilisé dans Cucumber (BDD) pour définir la validation d'un comportement attendu (scenario) dans une fonctionnalité (feature)

Scénario de test

January 18, 2024 9:51 AM

Un scénario de test est une destination, le parcours, l'itinéraire... c'est, donc, un ensemble d'étapes à suivre pour valider l'arrivée à destination.

Dans cette procédure de test détaillée, on y retrouve le nom du scénario de test, la condition préalable (point A), les étapes/conditions d'entrée (itinéraire) et le résultat attendu (point B).

Il est utilisé pour tester une fonctionnalité de bout en bout et est généralement dérivé des cas d'utilisation. Les scénarios de test **peuvent** servir de base à la création de cas de test de niveau inférieur. Un seul scénario de test **peut** couvrir un ou plusieurs cas de test.

Son but est de tester une fonctionnalité ou un cas d'utilisation. C'est la déclaration décrivant la fonctionnalité de l'application à tester.

n.b. "Scenario" est aussi utilisé dans Cucumber (BDD) pour définir la validation d'un comportement attendu (scenario) dans une fonctionnalité (feature)

Serveur d’application d'environnement de test

January 18, 2024 9:51 AM

Un serveur d’application d'environnement de test permet de tester les changements apportés à une application dans un environnement réaliste. Il permet de créer des environnements identiques à chaque fois que vous avez besoin de tester votre produit. Cela peut inclure la configuration du matériel et du réseau. Un environnement de test bien conçu est crucial pour avoir confiance dans les résultats des tests et pour fournir des commentaires précis sur la qualité et le comportement de l’application testée.

Sévérité

January 18, 2024 9:52 AM

Sévérité d’une anomalie

La sévérité d’une anomalie est utilisée pour refléter la sévérité du problème qui a été identifié lors de l’exécution des différents types de tests.

Sévérité	Description de la sévérité
1-Critique	Problème sérieux qui rend impossible tout test / travail supplémentaire jusqu'à ce qu'il soit résolu (« Showstopper »). Le système est inutilisable et/ou les coûts engendrés peuvent être importants et/ou l'intégrité des données est dégradée La fonction échouée est inutilisable et il n'y a pas de méthode alternative acceptable pour atteindre les résultats requis Ces anomalies doivent être corrigées en premier (1er).
2-Sévère	Une erreur significative qui empêche le système d'être opérationnel, mais d'autres tests peuvent continuer. Le système n'est pas complètement exploitable ou est instable. Il existe une solution de contournement (« Workaround »).e Ces anomalies doivent être corrigées en deuxième (2e).
3-Pénalisante	Une erreur qui n’est pas bloquante (autre que sévérité 1 et 2), mais assez importante pour mener à bien le test du scénario. Une solution de contournement a été fournie pour un problème et l’erreur est toujours persistante. Le résultat attendu n'a pas été obtenu (résultat incomplet, incorrect ou inconsistant). Ces anomalies doivent être corrigées en troisième (3e).
4-Mineure	Un problème mineur est survenu et n'empêche pas le système d'être opérationnel, mais il peut gêner l'utilisateur (ex. : problème d’affichage). Les résultats attendus ne sont pas respectés, mais n'entravent pas l’utilisation de la fonctionnalité. Ces anomalies doivent être corrigées en dernier lieu.

Shift-Right (décalage vers la satisfaction)

January 18, 2024 9:52 AM

“Shift-Right” est une approche de l’assurance qualité logicielle qui vise à améliorer la satisfaction des utilisateurs en se concentrant sur les tests et la surveillance en production. L’approche consiste à déplacer les tests et la surveillance vers la droite du cycle de vie du développement logiciel, c’est-à-dire vers la fin du cycle de vie, plutôt que de se concentrer uniquement sur les tests et la surveillance en amont. Cela permet de s’assurer que le logiciel fonctionne correctement dans un environnement de production réel et que les utilisateurs sont satisfaits.

Simulateur

January 18, 2024 9:52 AM

Un simulateur, en revanche, est un outil logiciel ou matériel qui modélise le comportement d'un système spécifique, fournissant une approximation de ses performances et de ses caractéristiques à des fins de test et d'analyse. Un simulateur est principalement utilisé pour tester l'interface utilisateur et l'expérience utilisateur.

Les avantages de l'utilisation d'un simulateur incluent la possibilité de tester l'interface utilisateur et l'expérience utilisateur. Les simulateurs peuvent également être utilisés pour modéliser et analyser le comportement d'un système spécifique.

Les émulateurs et les simulateurs ont des objectifs différents dans le développement de logiciels, les émulateurs se concentrant sur la compatibilité et la réplication, tandis que les simulateurs se concentrent sur la modélisation et l'analyse.

Cependant, il y a aussi des limites à l'utilisation de ces outils. Par exemple, un simulateur doit extrapoler une partie des variables qui lui sont inconnues.

De plus, les performances de ces outils peuvent varier en fonction de la qualité des scripts de test utilisés.

SOLID

January 18, 2024 9:52 AM

SOLID est un acronyme mnémonique qui regroupe cinq principes de conception destinés à produire des architectures logicielles plus compréhensibles, flexibles et maintenables. Les principes sont un sous-ensemble de nombreux principes promus par l'ingénieur logiciel et instructeur américain Robert C. Martin1. Les cinq principes sont les suivants:

- **S**ingle Responsibility Principle (SRP) : Une classe ne doit avoir qu'une seule raison de changer.
- **O**pen/Closed Principle (OCP) : Les entités logicielles (classes, modules, fonctions, etc.) doivent être ouvertes à l'extension mais fermées à la modification.
- **L**iskov Substitution Principle (LSP) : Les objets d'une classe dérivée doivent être utilisables partout où les objets de la classe de base sont utilisés.
- **I**nterface Segregation Principle (ISP) : Les clients ne doivent pas être forcés de dépendre d'interfaces qu'ils n'utilisent pas.
- **D**ependency Inversion Principle (DIP) : Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau. Les deux doivent dépendre d'abstractions.

Stratégie d'automatisation

January 18, 2024 9:52 AM

La stratégie d’automatisation n’est pas une stratégie en soi, c’est un plan d’implémentation de l’automatisation dans un contexte n’ayant pas d’automatisation.
La vrai stratégie d’automatisation est inclus dans la stratégie de test automatisé un test fait partie de la stratégie de test.

Stratégie de test

January 18, 2024 9:53 AM

C'est la politique de test, les ententes, les conventions, les règles, nos façons de faire, nos guides/modèles, nos ligne directrices…
Stratégie de test est enfant de la Stratégie d'assurance qualité (La politique de l'assurance qualité)

- **!** Ne pas confondre avec un plan de test qui est l'engagement par l'action de la stratégie de test
  Une stratégie de test est plutôt de haut niveau et évolue lentement, contrairement au plan de test qui est nouveau pour chaque livraison.
- **!** Attention il peut y avoir des stratégies sœurs/frères comme stratégie d'audition (audit), stratégie méthodologie, stratégie automatisation, stratégie de bugs…

Nous retrouvons généralement dans une stratégie de test les catégories de conventions suivantes :

- Principes
- Approches
- Tactiques
- Planification/Livraison
- Les métriques
- Outils / coffre à outils
- Le comment faire / mode d'emploi / utilisation
- …

Tactiques et techniques de test

January 18, 2024 9:53 AM

Les tactiques et techniques de test logiciel sont des approches et méthodes utilisées pour tester les logiciels afin de s'assurer qu'ils fonctionnent correctement et répondent aux exigences de qualité.
Les **tactiques** de test logiciel sont des **approches** générales pour tester un logiciel,
Les **techniques** de test logiciel sont des **méthodes spécifiques** pour tester un logiciel.

Les tâches ou étapes communes du processus de test logiciel comprennent la définition de l'environnement de test, le développement de cas de test, la rédaction de scripts, l'analyse des résultats de test et la soumission de rapports de défauts

- Test de non-régression
  - Le test de non-régression est un processus essentiel dans le monde du développement logiciel qui permet de vérifier que les caractéristiques ou les fonctions testées et validées précédemment fonctionnent comme prévu après toute modification du code. Le test de non-régression est un test informatique qui a pour but de vérifier que les changements ou évolution apportés à un système (un site ou une application) n'ont pas induit des bugs dans celui-ci. Le TNR est donc une pratique de test logiciel qui permet de vérifier qu'une application fonctionne toujours comme prévu après toute modification, mise à jour ou encore amélioration du code. Les Tests de Non Régression (TNR) ont pour but de s'assurer que les modifications et évolutions effectuées par les développeurs lors du dernier sprint n'ont pas entrainé d'effet de bord, en altérant les parties du code non modifiées4.
- Test d'acceptation
  - Le test d'acceptation est un type de test logiciel qui permet de vérifier si un système répond aux exigences spécifiées et s'il est prêt à être utilisé par les utilisateurs finaux. Le test d'acceptation est effectué pour s'assurer que le système répond aux exigences fonctionnelles et non fonctionnelles spécifiées dans les spécifications.
- Test fonctionnel et non-fonctionnels
  - Les tests fonctionnels et non-fonctionnels sont deux types de tests logiciels différents. Les tests fonctionnels sont des tests qui vérifient si le système répond aux exigences fonctionnelles spécifiées dans les spécifications. Les tests non-fonctionnels sont des tests qui vérifient si le système répond aux exigences non-fonctionnelles spécifiées dans les spécifications. Les exigences non-fonctionnelles peuvent inclure des exigences de performance, de sécurité, d'accessibilité et d'autres exigences qui ne sont

pas liées à la fonctionnalité du système.

- Test continus
  - Le test continu est un processus de test logiciel qui met en œuvre des outils de test automatisé et d'automatisation en tant que composants du pipeline de développement logiciel. Le test continu fournit un retour immédiat sur les risques commerciaux pouvant exister pendant le développement et lors de la publication éventuelle du logiciel. Le test continu est un principe de test logiciel dans lequel tous vos tests sont exécutés en permanence, fournissant une rétroaction continue sur la qualité et la santé de vos applications. Pour réaliser des tests continus, les organisations doivent d'abord maîtriser le manuel pour adopter l'automatisation des tests.
- Test A/B
  - Le test A/B est une technique de test logiciel qui permet de comparer deux versions différentes d'un système pour déterminer laquelle est la plus efficace. Le test A/B est souvent utilisé pour tester des changements mineurs dans les interfaces utilisateur ou les fonctionnalités du système. Dans un test A/B, un groupe d'utilisateurs est sélectionné au hasard et présenté avec une version du système (version A) tandis qu'un autre groupe d'utilisateurs est présenté avec une version légèrement différente du système (version B). Les résultats sont ensuite comparés pour déterminer quelle version est la plus efficace.
- Test simultanés
  - Les tests simultanés sont une technique de test logiciel qui permet de tester plusieurs fonctionnalités ou scénarios en même temps. Les tests simultanés sont souvent utilisés pour tester des systèmes complexes qui ont de nombreuses fonctionnalités interdépendantes. Les tests simultanés peuvent être effectués manuellement ou à l'aide d'outils de test automatisés.
- Test de conformité
  - Le test de conformité est une technique de test logiciel qui permet de vérifier si un système répond aux normes et aux réglementations spécifiées. Le test de conformité est souvent utilisé pour les systèmes qui doivent respecter des normes strictes en matière de sécurité, de confidentialité ou d'accessibilité. Le test de conformité peut être effectué manuellement ou à l'aide d'outils de test automatisés.
- Test par comparaison (actual → expected)
  Le test par comparaison est une technique de test logiciel qui permet de comparer les résultats d'un système avec les résultats attendus pour déterminer si le système fonctionne correctement. Le test par comparaison est souvent utilisé pour tester des systèmes qui ont des entrées et des sorties bien définies. Dans un test par comparaison, les entrées sont fournies au système et les sorties sont comparées aux résultats attendus

pour déterminer si le système fonctionne correctement.

- **métamorphique** (défini)
  - Le test métamorphique est une technique de test logiciel qui permet de tester un système en utilisant des transformations de données. Dans un test métamorphique, les données d'entrée sont transformées avant d'être fournies au système pour tester si le système répond correctement aux transformations. Le test métamorphique est souvent utilisé pour tester des systèmes qui ont des entrées et des sorties bien définies.
- **par propriété** (généré)
  - Le test par propriété est une technique de test logiciel qui permet de vérifier si un système satisfait une propriété spécifiée. Dans un test par propriété, une propriété est spécifiée pour le système et le système est testé pour vérifier si la propriété est satisfaite. Le test par propriété est souvent utilisé pour tester des systèmes qui ont des comportements complexes ou des propriétés de sécurité.

Test

January 18, 2024 9:53 AM

Le test logiciel est le processus qui consiste à évaluer et à vérifier qu'un produit ou une application logicielle fait ce qu'il ou elle est censé (e) faire. Les avantages du test comprennent la prévention des bogues, la réduction des coûts de développement et l'amélioration des performances. Il existe de nombreux types de tests de logiciels, chacun ayant des objectifs et des stratégies spécifiques : Tests d'acceptation : ils permettent de vérifier si l'ensemble du système fonctionne comme prévu.

TEST = Un test est un ensemble de cas à tester (état de l'objet à tester avant exécution du test, actions ou données en entrée, valeurs ou observations attendues, et état de l'objet après exécution), éventuellement accompagné d'une procédure d'exécution (séquence d'actions à exécuter). Il est lié à un objectif.

Un test est défini par

- son contexte,
- une action
- et son résultat attendu

Bien rédiger un test devrait passer par les phases suivantes de la modularisation :

1. clarté (concis)
2. maintenance efficace
3. réutilisable
4. autonome et exécutable en simultané

Plusieurs termes peuvent êtres mentionnés liés aux tests de logiciels et ont des significations différentes.

Voici une brève explication de chacun d'eux :

- Test : Le processus de vérification d'un produit ou d'une application logicielle pour s'assurer qu'il fonctionne correctement et qu'il répond aux exigences spécifiées.
- Essai : Un test effectué pour évaluer les performances d'un produit ou d'une

application logicielle dans des conditions spécifiques.

- Validation : Le processus de vérification que le produit ou l'application logicielle répond aux exigences spécifiées et aux besoins des utilisateurs.
- Vérification : Le processus de vérification que le produit ou l'application logicielle répond aux exigences spécifiées et aux normes de qualité.
- Exécution : Le processus d'exécution des tests pour vérifier le fonctionnement du produit ou de l'application logicielle.
- Scénario : Une séquence d'étapes qui décrit comment un utilisateur interagit avec un produit ou une application logicielle dans un contexte spécifique.
  Ici on va parler d'un téméraire et surtout que le résultat attendu est de se rendre du point A au point B
- Cas : Une description détaillée des étapes à suivre pour effectuer un test spécifique.
- Cas de test : Un ensemble de cas qui sont utilisés pour tester une fonctionnalité spécifique d'un produit ou d'une application logicielle.
- Cas d'utilisation : Une description détaillée des interactions entre un utilisateur et un produit ou une application logicielle dans un scénario spécifique.
- Scénario de cas d'utilisation : Une séquence d'étapes qui décrit comment un utilisateur interagit avec un produit ou une application logicielle dans un scénario spécifique.
- Suite de test : Un ensemble de cas de test qui sont utilisés pour tester une fonctionnalité spécifique ou plusieurs fonctionnalités d'un produit ou d'une application logicielle.
- Couverture : La mesure du nombre de cas de test qui ont été exécutés par rapport au nombre total de cas de test disponibles.
- Plan de test : Un document qui décrit les objectifs, les stratégies et les ressources nécessaires pour effectuer les tests sur un produit ou une application logicielle.
- Stratégie de test : Un "plan" global pour la gestion des tests sur un produit ou une application logicielle ou la gestion de test dans l'ensemble de la politique de l'assurance qualité.

Test Agile

January 18, 2024 9:53 AM

Les tests Agile suivent les principes de développement Agile. Les exigences évoluent continuellement en raison de la collaboration entre les différents services de l'entreprise entre eux et avec le client. Cela peut accélérer les processus de développement et de test des produits, car chacun contribue à l'assurance qualité.

Test basé sur l’image (image-based testing)

January 18, 2024 9:54 AM

Le test basé sur l’image (Image-Based Testing en anglais) est une méthode de test qui consiste à reconnaître les éléments de l’interface utilisateur (UI) d’une application testée par leur image plutôt que par un ensemble de leurs propriétés internes. Un test basé sur l’image se compose d’une séquence d’opérations sur les éléments de l’interface utilisateur de votre application testée : clics (pour les applications de bureau et Web), touches (pour les applications mobiles), opérations de glisser-déposer, points de contrôle, etc. Le test basé sur l’image est mieux utilisé pour les applications avec des contrôles personnalisés, des applications construites avec des boîtes à outils non prises en charge pour lesquelles les propriétés d’objet ne sont pas facilement disponibles ou des graphiques 2D/3D ou des graphiques, tels que des données d’imagerie médicale.

Tests basés sur le contexte (Context-Based Testing)

January 18, 2024 9:54 AM

Le context-based testing est une méthode de test logiciel qui consiste à tester un logiciel dans des conditions réelles d'utilisation par les utilisateurs finaux. Les développeurs peuvent ainsi voir la sortie réelle de l'utilisation du logiciel et corriger les bogues éventuels trouvés lors des tests.

Le context-driven testing est une approche pratique du test logiciel exécutée par les utilisateurs finaux dans des conditions d'utilisation réelles1. Cette méthode de test se concentre sur l'utilisation d'approches de test dynamiques dans différentes conditions.

Test basé sur le modèle (MBT)

January 18, 2024 9:54 AM

Le model-based testing (MBT) est une méthode de test logiciel qui consiste à concevoir et à dériver des cas de test à partir d'un modèle abstrait et haut niveau du système sous test (SUT). Les modèles peuvent être utilisés pour représenter le comportement souhaité d'un SUT ou pour représenter des stratégies de test et un environnement de test. Le MBT peut être utilisé pour améliorer la qualité des tests en augmentant la couverture des tests et en réduisant les coûts de test.

Test de confirmation

January 18, 2024 9:54 AM

Test de confirmation: il s'agit d'un type de test associé à un changement. Il a pour but de confirmer que le défaut initial a été correctement corrigé.

Testeur Certifié Syllabus Niveau Fondation. (n.d.). *https://www.cftl.fr/wp-content/uploads/2018/10/CTFL-Syllabus-2018-FR.pdf*

Test de gorille (gorilla testing)

January 18, 2024 9:54 AM

Le “gorilla testing” est une technique de test logiciel où l’utilisateur teste l’application ou le système en fournissant des entrées extrêmes et inhabituelles pour vérifier si l’application ou le système peut gérer des situations imprévues. Le gorilla testing est généralement effectué après que le monkey testing a été effectué.

Test de non-régression (ou test de régression)

January 18, 2024 9:55 AM

Le test de non-régression est un type de test qui détermine si le logiciel continue de fonctionner comme prévu après une mise à jour du code. Le fait de ne pas fournir le résultat prévu crée une régression (bug de régression). Ce type de test a pour but de vérifier que les changements ou évolution apportés à un système (un site ou une application) n'ont pas induit des bugs dans celui-ci. Le test de non-régression permet de limiter les anomalies, les effets de bords et les coûts de maintenance.

Les tests de non-régression sont généralement effectués après chaque modification du code pour s'assurer que les changements n'ont pas affecté les fonctionnalités existantes. Ces tests peuvent être automatisés pour gagner du temps et de la fiabilité.

ref. : Les meilleures pratiques pour les tests de régression (stardust-testing.com)

Test de singe (monkey testing)

January 18, 2024 9:55 AM

Le “monkey testing” est une technique de test logiciel où l’utilisateur teste l’application ou le système en fournissant des entrées aléatoires et en vérifiant le comportement, ou en vérifiant si l’application ou le système va planter. Le monkey testing est généralement mis en œuvre sous forme exploratoire.

Test de stabilisation

January 18, 2024 9:55 AM

Les tests de stabilisation peut être relatif à une branche ou pas. C'est pour bien définir, séparer et éliminer les erreurs d'environnements versus les erreurs logiciels. C'est une tactique et son but principal est de stabiliser au minimum le fonctionnement de l'application global (pas nécessaire d'être complet). Donc cela peut aussi inclure des tests sanitaires mais aussi des tests de non-régression.

Test en binôme ou test en pair (pair testing)

January 18, 2024 9:55 AM

Le pair testing est une technique de développement de logiciels dans laquelle deux membres de l'équipe travaillent ensemble sur un clavier pour tester l'application logicielle. L'un effectue les tests et l'autre analyse ou examine les tests. Cela peut être particulièrement efficace lorsque un programmeur travaille avec un testeur.

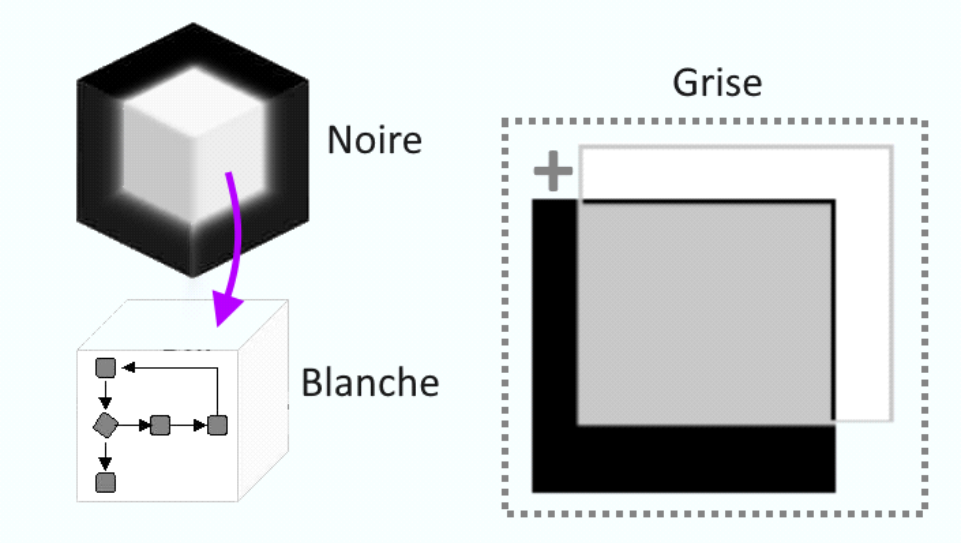
Le test en binôme peut offrir plusieurs avantages. Par exemple, il peut permettre une détection plus rapide des bugs et des erreurs, car deux personnes examinent le code en même temps. Cela peut également améliorer la qualité du code, car les deux personnes peuvent discuter et échanger des idées sur la façon de résoudre les problèmes et d'implémenter les fonctionnalités. De plus, le test en binôme peut favoriser la collaboration et l'apprentissage entre les membres de l'équipe, car ils travaillent ensemble et partagent leurs connaissances et leur expertise.

Test en boîte noire, blanche et grise (Box testing)

January 18, 2024 9:55 AM

n matière de qualité logicielle, les tests sont réalisés selon trois méthodes différentes : les tests en "boîte noire", en "boîte blanche" et en "boîte grise".

- Les tests en boîte noire sont utilisés pour tester un programme en vérifiant que les sorties obtenues sont bien celles prévues pour des entrées données.
- Les tests en boîte blanche sont utilisés pour tester la structure interne d'un programme.
- Les tests en boîte grise sont une combinaison des deux méthodes précédentes et permettent de tester la structure interne d'un programme tout en vérifiant que les sorties obtenues sont bien celles prévues pour des entrées données.



Test en groupe/équipe (mob testing)

January 18, 2024 9:56 AM

Le Mob Testing est similaire au Mob Programming, mais il se concentre sur les activités de test. Dans le Mob Testing, un groupe de testeurs utilise un seul ordinateur, en forçant un véritable flux de pièce unique. Chaque membre du groupe s'assoit devant cet ordinateur, à tour de rôle, en tournant sur une minuterie toutes les quelques minutes, en continuant à travailler sur la même tâche jusqu'à ce qu'elle soit terminée.

Test fonctionnel

January 18, 2024 9:56 AM

Un test fonctionnel est un **type** de test qui vise à vérifier que chaque fonctionnalité de l'application logicielle fonctionne conformément aux exigences du logiciel et aux attentes de l'utilisateur final. Il s'effectue principalement en boîte noire, sans accès au code source, et peut couvrir des scénarios ou des modèles d'utilisation de bout en bout.

Les tests fonctionnels visent à vérifier que le logiciel répond aux exigences fonctionnelles spécifiées. Ces tests utilisent des entrées valides et vérifient que le logiciel produit les résultats attendus. Les tests fonctionnels peuvent inclure des tests de validation des données, des tests de navigation, des tests d'interface utilisateur et d'autres types de tests qui vérifient les fonctionnalités du logiciel.

Test manuel

January 18, 2024 9:56 AM

Les tests manuels décrivent les tests menés par l'homme pour détecter les défauts d'un produit logiciel. Ces tests informent les acteurs du projet sur la qualité du produit. En règle générale, le testeur agit en tant qu'utilisateur final et utilise les fonctionnalités pour déterminer s'il fonctionne correctement. De plus, le testeur suit un plan de test pour travailler sur des cas de test spécifiques. Les tests manuels peuvent augmenter les coûts monétaires et de main-d'œuvre des tests mieux adaptés à l'automatisation. Cependant, les sondages qui nécessitent des opinions et des commentaires aléatoires, tels que la convivialité, bénéficient de tests manuels. La plupart des produits nécessitent une combinaison de tests automatisés et manuels pour s'assurer qu'ils sont prêts pour le marché.

Test non fonctionnel

January 18, 2024 9:56 AM

Un test non fonctionnel est une catégorie de test logiciel effectué pour vérifier les exigences non fonctionnelles de l'application.

Il existe plusieurs types de tests non fonctionnels, parmi lesquels on peut citer les tests :

- de performances
- de charge
- de contraintes
- de volume
- de sécurité
- de compatibilité
- d'installation
- de récupération
- etc.

Les tests non fonctionnels sont tous les types de tests de logiciels où les aspects non fonctionnels de la construction du logiciel sont testés. Parmi les exemples de tests non fonctionnels, on peut citer les tests destinés à évaluer la capacité, les performances, la facilité d'utilisation, la récupération et la portabilité. Par exemple, un test de performance vérifie si le logiciel répond rapidement aux demandes des utilisateurs même lorsqu'il y a un grand nombre d'utilisateurs simultanés. Un test de sécurité vérifie si le logiciel est protégé contre les accès non autorisés...

Les tests non fonctionnels visent à vérifier les aspects non fonctionnels du logiciel, tels que la performance, la fiabilité, la sécurité et l'utilisabilité. Ces tests peuvent inclure des tests de charge, des tests de stress, des tests de sécurité et d'autres types de tests qui vérifient les caractéristiques non fonctionnelles du logiciel.

Test sanitaire/intégrité (santé) - Test de code propre (clean code)

January 18, 2024 9:56 AM

Le "Clean Code" est un type de test qui assure que le code ne contient pas de bogues ou d’erreurs.

Oui, les tests de code unitaire contribuent grandement au "clean code", mais il y a plus que cela !

Cela peut inclure des tests, pour s’assurer que le code est claire, concis, est facile à lire et à comprendre, à maintenir, respect les modèles et le domaine, qu’il est bien structuré, modularisé, qu’il suit les normes de codage établies, les meilleurs pratiques, qu'il est sécuritaire...

Avoir du code propre est très large, cela peu même passer par l'AI et l'analytique.

Test sanitaire/intégrité (santé) - Test de configuration

January 18, 2024 9:57 AM

Le test de configuration est un type de test de logiciel qui vérifie les performances du système en cours de développement par rapport à diverses combinaisons de logiciels et de matériel pour trouver la meilleure configuration dans laquelle le système peut fonctionner sans défauts ni problèmes tout en répondant à ses exigences fonctionnelles. Les performances du système ou d'une application sont testées par rapport à chacune des configurations matérielles et logicielles prises en charge.

Test sanitaire/intégrité (santé) - Test de déploiement, installation ou publication

January 18, 2024 9:57 AM

Le test de déploiement est un type de test qui vérifie que le logiciel peut être déployé correctement. Il peut inclure des tests d'intégration pour vérifier que les différents composants du logiciel fonctionnent ensemble. Le processus de déploiement peut être automatisé pour assurer un passage rapide du code en déploiement.

C'est la validation de la relation, prêt à l'utilisation, entre l'application et son environnement.

Le terme "test d'installation" peut être utilisé pour décrire un test d'intégration qui vérifie que le logiciel peut être installé correctement. Il peut également être utilisé pour décrire un test de déploiement qui vérifie que le logiciel peut être déployé correctement.

Publication est un déploiement web.

Test sanitaire/intégrité (santé) - Test de donnée

January 18, 2024 9:57 AM

Le test de donnée est un processus qui vise à évaluer la qualité des données. Il peut inclure des étapes telles que l'exploration des données, l'évaluation de la qualité des données et le nettoyage des données. Les bonnes pratiques pour assurer la qualité des données comprennent la définition des données, leur nomenclature et leur modèle de gestion.

Test sanitaire/intégrité (santé) - Test de fumée (smoke)

January 18, 2024 9:58 AM

Le test de fumée est un type de test logiciel qui se concentre sur la vérification de la fonctionnalité de base d'une application logicielle. Les tests de fumée viennent avant d'autres tests approfondis et consistent en des tests fonctionnels ou unitaires de fonctions logicielles critiques. Les tests de fumée répondent à des questions telles que "Est-ce que le programme démarre correctement?" et "Est-ce que les boutons de contrôle principaux fonctionnent?". Le test de fumée est généralement utilisé comme première étape d'un processus de test plus complet.

Les tests de fumée sont des tests fonctionnels et/ou unitaires de fonctions logicielles critiques qui se concentrent sur la vérification de la fonctionnalité de base d'une application logicielle1. Les tests de fumée viennent avant d'autres tests approfondis et répondent à des questions telles que "Est-ce que le programme démarre correctement?" et "Est-ce que les boutons de contrôle principaux fonctionnent?

- J'aime
- Commentaire
- 3 vues
- Enregistrer pour plus tard

Tests statiques

January 18, 2024 9:58 AM

Ils ne nécessitent pas l'exécution du logiciel et l'un de leurs avantages est qu'ils peuvent être réalisés dès les premières phases du projet. Tant le code que la documentation peuvent bénéficier de ce type de test et, en fonction du produit à tester, c'est le rôle responsable de la conception des tests, de leur exécution et de l'évaluation des résultats.

Testeur Certifié Syllabus Niveau Fondation. (n.d.). https://www.cftl.fr/wp-content/uploads/2018/10/CTFL-Syllabus-2018-FR.pdf

Version

January 18, 2024 9:58 AM

Version désigne une étape spécifique dans l’évolution du logiciel, identifiée par un numéro de version unique.

- J’aime
- Commentaire
- 3 vues
- Enregistrer pour plus tard

WD-6 (le 6W du Lean Six Sigma ou le 5W1H)

January 18, 2024 9:58 AM



WD-6 est une série logique de questions utilisées pour garantir qu'un problème ou un sujet en discussion dans le but d'analysé sous toutes les perspectives (angles différents) pertinentes, un ensemble de questions séquentielles logiques que vous posez lors de l'analyse et résolution d'un problème.

L'idée de poser six questions pour analyser un projet ou un problème est venue d'un poème du célèbre poète Rudyard Kipling. Vous remarquerez que le poème original mentionnait vraiment les 5W et un H.
Le 5W1H qui pose les même questions en série passant par la question : "Comment ?" (how?)

fr. : "Je garde six serviteurs honnêtes (ils m'ont appris tout ce que je savais) Leurs noms sont quoi, pourquoi, où, quand, qui et comment."

Différents auteurs les présenteront dans une variété de séquences, mais le terme utilisé à ce jour, restait le "5W1H".

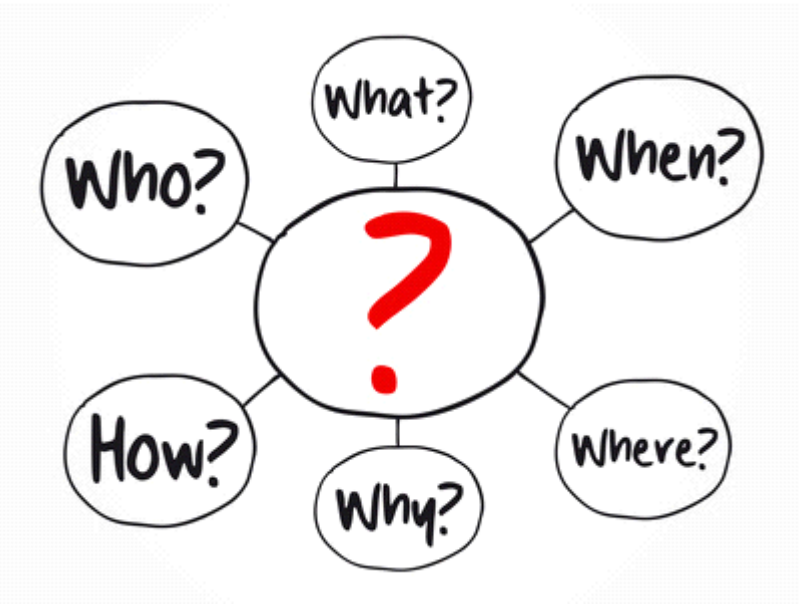
Le 6W a commencé à faire son apparition… Certain vont chercher le 6ème "W" en utilisant le "Which" par exemple, mais il manquera toujours le "How" (comment) …

C'est seulement lorsque le "**Way**" remplaça le "**How**" que le 6W fût utilisé dans le **Lean Six Sigma**.

Le **D** dans W**D**-6, c'est l'ajout du "to **D**o ou **D**oes" qui est devenu rapidement une référence humoristique au produit WD-40 → WD-6 😄

Donc, rien ici de réinventé, c'n'est que la bonne veille technique du "5W1H" d'analyse qui a évoluée

Présentation : qu'est-ce que WD-6 ?

Anglais	**Français**	
the **W**hat to **D**o	le quoi faire	
the **W**hy to **D**o	le pourquoi faire	
the **W**here to **D**o	le où faire	
the **W**hen to **D**o it	le quand le faire	
the **W**ho **D**oes	le qui fait	
the **W**ay to **D**o	la façon de faire (comment)	

Différents contextes

Vous pouvez utiliser une variété de séquences en fonction de la situation. Voici 2 exemples…

Dans le contexte de la résolution de problèmes, vous pouvez utiliser cette séquence de questions :

- Qui a remarqué le problème ?
- Quel est le problème?
- Quand le problème a-t-il commencé?
- Où est le problème?
- Pourquoi le problème est-il survenu ?
- **Comment** résoudre le problème ?

Dans le cadre de la gestion de projet, vous pouvez essayer cette approche :

- Qui doit effectuer les tâches requises ?
- Que doivent-ils faire, concrètement ?
- Quand les tâches doivent-elles être terminées ?
- Où les tâches doivent-elles être accomplies ?
- Pourquoi cette tâche est-elle requise ?
- **Comment** la tâche doit-elle être accomplie ?

3 avantages de WD-6

Vous bénéficierez du questionnement structuré.

Simple, complet et rapide

Vous n'avez pas besoin d'utiliser des statistiques complexes ou des méthodologies avancées pour utiliser cette technique. Poser les questions et analyser les réponses vous donnera un aperçu rapide de la façon dont vous devez procéder pour résoudre votre problème.

Flexible

Vous pouvez utiliser l'approche WD-6 pour résoudre des problèmes professionnels ainsi que des problèmes personnels. Il peut être appliqué dans toute une organisation, dans n'importe quelle fonction ou service.

Pensée structurée

La séquence des questions est logique et vous permet de décomposer le problème en éléments gérables. Cela vous permet également d'attaquer le problème sous différents angles.

Pourquoi est-il important de comprendre WD-6 ?

Comprendre l'utilisation des multiples questions de WD-6 vous permettra d'approfondir et plus complètement votre problème.

Décompose le problème

Tous les problèmes ont plusieurs dimensions. Vous comprendrez mieux votre problème et comment le résoudre si vous le décomposez en plusieurs points : que se passe-t-il, où cela se passe-t-il, pourquoi cela se passe-t-il et comment vous pouvez résoudre le problème.

Fournit une approche par étapes

La séquence WD-6 n'est pas tout à fait aléatoire. C'est une construction logique sans début précis mais qui se termine toujours par "comment". Si vous comprenez cela, les questions s'appuieront sur elles-mêmes - de sorte que le "comment" aura un sens et que l'évidence naitra d'elle-même.

Améliore la communication

Vous voudrez souvent utiliser l'approche WD-6 avec une équipe de personnes. Étant donné que les questions et les réponses seront une expérience partagée, la communication au sein de l'équipe sera améliorée et conduira à une solution plus robuste et meilleure.

Un exemple industriel de WD-6

Une entreprise de verre subissait une augmentation des bris de ses panneaux de verre. Bien qu'il existe de nombreuses approches de résolution de problèmes, telles que les 5 pourquoi et le diagramme en arête de poisson , ils ont choisi l'approche WD-6.

Voici ce qu'ils ont fait :

1. **Que** se passe-t-il ? Augmentation des bris de verre.
2. **Où** est-ce que ça se passe ? La machine à plastifier de la ligne 4.
3. **Quand** le problème a-t-il commencé ? QC a remarqué une augmentation significative il y a deux semaines.
4. **Pourquoi** le problème se produit-il ? Après enquête, il a été conclu que les feuilles de verre avaient une occurrence accrue de bulles d'air, ce qui affaiblissait le verre lorsqu'il était mis sous pression.
5. **Qui** devrait s'en occuper ? CQ, opérateurs de ligne, ingénierie et approvisionnement.
6. **Comment** cela sera-t-il résolu ? L'inspection entrante doublera son taux d'inspection, et l'approvisionnement et l'ingénierie se rendront chez le fournisseur pour voir s'ils peuvent l'aider à résoudre le problème de l'augmentation des bulles d'air dans le verre.

3 bonnes pratiques en pensant à WD-6

Le processus de poser les questions est facile. Votre défi sera de rester sur la tâche et d'approfondir les questions pour découvrir la solution.

7. **Résoudre la cause profonde et non traiter le symptôme**
   Vous devez vous efforcer de trouver une solution à long terme plutôt qu'une solution de pansement temporaire.
8. **Solution objective**
   Appuyez-vous sur autant de données que possible. Le problème et la solution

éventuelle doivent être exprimés quantitativement plutôt que subjectivement.

9. **Explorez plusieurs solutions**
   Ne sautez pas à la première série de conclusions. Explorez plusieurs opportunités d'amélioration, puis choisissez la meilleure ou synthétisez parmi plusieurs de vos solutions possibles.

WD-6 est une série logique de questions utilisées pour garantir qu'un problème ou un sujet en discussion dans le but d'analysé sous toutes les perspectives (angles différents) pertinentes, un ensemble de questions séquentielles logiques que vous posez lors de l'analyse et résolution d'un problème.

L'idée de poser six questions pour analyser un projet ou un problème est venue d'un poème du célèbre poète Rudyard Kipling. Vous remarquerez que le poème original mentionnait vraiment les 5W et un H.
Le 5W1H qui pose les même questions en série passant par la question : "Comment ?" (how?)

fr. : "Je garde six serviteurs honnêtes (ils m'ont appris tout ce que je savais) Leurs noms sont quoi, pourquoi, où, quand, qui et comment."

Différents auteurs les présenteront dans une variété de séquences, mais le terme utilisé à ce jour, restait le "5W1H".

Le 6W a commencé à faire son apparition… Certain vont chercher le 6ème "W" en utilisant le "Which" par exemple, mais il manquera toujours le "How" (comment) …

C'est seulement lorsque le "**Way**" remplaça le "**How**" que le 6W fût utilisé dans le **Lean Six Sigma**.

Le **D** dans W**D**-6, c'est l'ajout du "to **D**o ou **D**oes" qui est devenu rapidement une référence humoristique au produit WD-40 → WD-6 😄

Donc, rien ici de réinventé, c'n'est que la bonne veille technique du "5W1H" d'analyse qui a évoluée

Présentation : qu'est-ce que WD-6 ?

Anglais	Français	
the **W**hat to **D**o	le quoi faire	
the **W**hy to **D**o	le pourquoi faire	
the **W**here to **D**o	le où faire	
the **W**hen to **D**o it	le quand le faire	
the **W**ho **D**oes	le qui fait	
the **W**ay to **D**o	la façon de faire (comment)	

Différents contextes

Vous pouvez utiliser une variété de séquences en fonction de la situation. Voici 2 exemples…

Dans le contexte de la résolution de problèmes, vous pouvez utiliser cette séquence de questions :

- Qui a remarqué le problème ?
- Quel est le problème?
- Quand le problème a-t-il commencé?
- Où est le problème?
- Pourquoi le problème est-il survenu ?
- **Comment** résoudre le problème ?

Dans le cadre de la gestion de projet, vous pouvez essayer cette approche :

- Qui doit effectuer les tâches requises ?
- Que doivent-ils faire, concrètement ?
- Quand les tâches doivent-elles être terminées ?
- Où les tâches doivent-elles être accomplies ?
- Pourquoi cette tâche est-elle requise ?
- **Comment** la tâche doit-elle être accomplie ?

3 avantages de WD-6

Vous bénéficierez du questionnement structuré.

Simple, complet et rapide

Vous n'avez pas besoin d'utiliser des statistiques complexes ou des méthodologies avancées pour utiliser cette technique. Poser les questions et analyser les réponses vous donnera un aperçu rapide de la façon dont vous devez procéder pour résoudre votre problème.

Flexible

Vous pouvez utiliser l'approche WD-6 pour résoudre des problèmes professionnels ainsi que des problèmes personnels. Il peut être appliqué dans toute une organisation, dans n'importe quelle fonction ou service.

Pensée structurée

La séquence des questions est logique et vous permet de décomposer le problème en éléments gérables. Cela vous permet également d'attaquer le problème sous différents angles.

Pourquoi est-il important de comprendre WD-6 ?

Comprendre l'utilisation des multiples questions de WD-6 vous permettra d'approfondir et plus complètement votre problème.

Décompose le problème

Tous les problèmes ont plusieurs dimensions. Vous comprendrez mieux votre problème et comment le résoudre si vous le décomposez en plusieurs points : que se passe-t-il, où cela se passe-t-il, pourquoi cela se passe-t-il et comment vous pouvez résoudre le problème.

Fournit une approche par étapes

La séquence WD-6 n'est pas tout à fait aléatoire. C'est une construction logique sans début précis mais qui se termine toujours par "comment". Si vous comprenez cela, les questions s'appuieront sur elles-mêmes - de sorte que le "comment" aura un sens et que l'évidence naitra d'elle-même.

Améliore la communication

Vous voudrez souvent utiliser l'approche WD-6 avec une équipe de personnes. Étant donné que les questions et les réponses seront une expérience partagée, la communication au sein de l'équipe sera améliorée et conduira à une solution plus robuste et meilleure.

Un exemple industriel de WD-6

Une entreprise de verre subissait une augmentation des bris de ses panneaux de verre. Bien qu'il existe de nombreuses approches de résolution de problèmes, telles que les 5 pourquoi et le diagramme en arête de poisson , ils ont choisi l'approche WD-6.

Voici ce qu'ils ont fait :

10. **Que** se passe-t-il ? Augmentation des bris de verre.
11. **Où** est-ce que ça se passe ? La machine à plastifier de la ligne 4.
12. **Quand** le problème a-t-il commencé ? QC a remarqué une augmentation significative il y a deux semaines.
13. **Pourquoi** le problème se produit-il ? Après enquête, il a été conclu que les feuilles de verre avaient une occurrence accrue de bulles d'air, ce qui affaiblissait le verre lorsqu'il était mis sous pression.
14. **Qui** devrait s'en occuper ? CQ, opérateurs de ligne, ingénierie et approvisionnement.
15. **Comment** cela sera-t-il résolu ? L'inspection entrante doublera son taux d'inspection, et l'approvisionnement et l'ingénierie se rendront chez le fournisseur pour voir s'ils peuvent l'aider à résoudre le problème de l'augmentation des bulles d'air dans le verre.

3 bonnes pratiques en pensant à WD-6

Le processus de poser les questions est facile. Votre défi sera de rester sur la tâche et d'approfondir les questions pour découvrir la solution.

16. **Résoudre la cause profonde et non traiter le symptôme**
    Vous devez vous efforcer de trouver une solution à long terme plutôt qu'une solution de pansement temporaire.
17. **Solution objective**
    Appuyez-vous sur autant de données que possible. Le problème et la solution

éventuelle doivent être exprimés quantitativement plutôt que subjectivement.

18. **Explorez plusieurs solutions**
    Ne sautez pas à la première série de conclusions. Explorez plusieurs opportunités d'amélioration, puis choisissez la meilleure ou synthétisez parmi plusieurs de vos solutions possibles.

XPath

January 18, 2024 10:00 AM

XPath signifie XML Path Language. Il utilise une syntaxe non-XML pour fournir un moyen flexible de pointer vers différentes parties d'un document XML. C'est un langage de requête pour localiser une portion d'un document XML.

XPath peut être utilisé pour naviguer à travers les éléments et les attributs d'un document XML. Il utilise une syntaxe de type "chemin" pour identifier et naviguer les nœuds dans un document XML. Vous pouvez utiliser la fonction evaluate de l'objet document pour évaluer les expressions XPath contre un document basé sur XML (y compris les documents HTML) et renvoyer un objet "XPathResult", qui peut être un seul nœud ou un ensemble de nœuds.

Il y a plusieurs avantages à utiliser XPath. Par exemple, XPath vous permet de naviguer dans le DOM pour rechercher des éléments à tester ou à extraire. Il est également compatible avec les anciens navigateurs. La création en XPath est plus flexible que celle en CSS Selector. Lorsque vous ne connaissez pas le nom d'un élément, vous pouvez utiliser contains pour rechercher des correspondances possibles.

XPath peut être utilisé pour sélectionner des nœuds ou des ensembles de nœuds dans un document XML en utilisant des expressions de chemin. Par exemple, vous pouvez sélectionner tous les titres avec l'expression //bookstore/book/title. Vous pouvez également sélectionner le titre du premier livre avec l'expression /bookstore/book[1]/title. Vous pouvez également sélectionner tous les prix avec l'expression /bookstore/book/price[text()].

Pour écrire une expression XPath, commencez par identifier le nœud racine du document XML ou HTML, puis utilisez une combinaison de noms d'éléments, d'attributs et de valeurs pour naviguer vers l'élément souhaité. La syntaxe des expressions XPath implique généralement une série de barres obliques, de noms d'éléments et de noms d'attributs, avec des prédicats facultatifs pour filtrer les résultats.

Par exemple, pour sélectionner tous les éléments book qui sont des enfants de l'élément bookstore, vous pouvez utiliser l'expression /bookstore/book.

Vous pouvez également utiliser des prédicats pour filtrer les résultats en fonction de certaines conditions. Par exemple, pour sélectionner le premier élément book qui est un enfant de l'élément bookstore, vous pouvez utiliser l'expression /bookstore/book[1].

exemple :

Pour retrouver un texte spécifique dans un document XML en utilisant XPath, vous pouvez utiliser la fonction text() et la fonction contains(). Par exemple, pour sélectionner tous les éléments book qui contiennent le texte "Harry Potter" dans leur élément title, vous pouvez utiliser l'expression //book[contains(title, 'Harry Potter')].

Bonne pratique :

L'utilisation de l'étoile (*) dans une expression XPath est appelée caractère générique. Un astérisque dans XPath peut représenter n'importe quel nom de nœud ou d'attribut d'un document XML ou DOM. Cela peut être utile lorsque vous ne connaissez pas les noms des éléments ou des attributs à l'avance, ou lorsque vous souhaitez sélectionner plusieurs éléments ou attributs en même temps. Et peu réduire de beaucoup la maintenance du au changement fréquent du type de l'élément.

YAGNI

January 18, 2024 10:00 AM

You **A**in’t **G**onna **N**eed **I**t (tu n'en auras pas besoin)

YAGNI consiste à ne pas se baser sur d’hypothétiques évolutions futures pour faire les choix du présent, au risque d’une complexification inutile. Il faut réaliser l’application au plus simple et en fonction des besoins actuels.

Attention, ne pas prévoir versus trop de "Au cas zZo0O!" ce n’est pas mieux !



Analyse SWOT

January 18, 2024 10:40 AM

L'**analyse** ou **matrice SWOT,** de l'anglais *Strengths (*forces), *Weaknesses* (faiblesses), *Opportunities* (opportunités), *Threats* (menaces) est un outil de stratégie d'entreprise permettant de déterminer les options offertes dans un domaine d'activité stratégique.

REF: Wikipédia, 2019-02-04, https://fr.wikipedia.org/wiki/SWOT_(m%C3%A9thode_d%27analyse)

Toutefois, bien que cet outil est issue du domaine du marketing, nous l'avons adopté et ajusté afin d'en faire un outil d'analyse stratégique afin d'élaborer un plan d'amélioration, principalement en assurance qualité.

L'acronyme SWOT désigne 4 mots anglais: *Strengths (*forces), *Weaknesses* (faiblesses), *Opportunities* (opportunités), *Threats* (menaces). L'analyse SWOT permet de poser un regard sur une entité (projet, département, entreprise...). L'analyse SWOT est très facile et rapide à utiliser.

Les forces et les faiblesses sont habituellement liés aux facteurs interne de l'entité. Les opportunités et les menaces sont, quand à elles, habituellement les facteurs dépendant de l'environnement de l'entité.

Forces:

Les facteurs internes qui affectent positivement la performance de l'entité. Ces forces constituent la fondation pour bâtir le futur de l'entité ou l'atteinte d'un objectif défini.

Posez-vous ce genre de questions:

- Qu'est-ce qu'on fait exceptionnellement bien?
- Quelles sont nos meilleures ressources? technologique, spécialiste humain, etc.
- Quelles sont vos forces aux yeux de vos clients/utilisateurs?

Faiblesses:

Les facteurs internes qui affectent négativement votre performance, vos réalisations? Ces faiblesses offrent d'importants potentiels d'amélioration.

- Que pourriez-vous faire de mieux?
- Quelles sont les principales critiques formulées par vos clients, vos utilisateurs, vos collaborateurs, les autres départements, etc.?
- Où vous sentez-vous vulnérable? moins en contrôle?

Opportunités:

Les facteurs externes dont vous pouvez tirer parti pour améliorer votre performance.

- Quelles sont les opportunités que vous connaissez mais que vous n'avez pas exploitées?
- Quelles sont les tendances émergentes (marché, technologie, management, etc.) dont vous

pourriez tirer profit?

- Quels sont les partenariats que vous pourriez mettre en place?

Menaces:

Les facteurs externes qui peuvent nuire à votre performance et même à l'atteinte des cibles de l'entité?

1. Quels sont les changements internes et externes à venir qui sont susceptibles de vous rendre vulnérables?
2. Quels sont les obstacles externes qui bloquent votre progrès?

L'analyse SWOT est fondée sur la perception des parties prenantes. Elle est donc par nature subjective et qualitative.

Preview unavailable

(done) Suivi des bugs (bug tracking)

January 18, 2024 10:40 AM

Bug tracking est le processus de suivi des bugs ou des problèmes dans un logiciel. Les outils de suivi des bogues sont utilisés pour suivre les bogues et les problèmes qui ont été identifiés lors des tests et pour s'assurer que ces bogues sont corrigés avant la sortie du logiciel.

Les outils de suivi des bogues peuvent également aider à suivre les bogues qui ont été signalés par les utilisateurs après la sortie du logiciel et à s'assurer que ces bogues sont corrigés dans les versions futures du logiciel.

ATDD (Acceptance Test-Driven Development)

January 18, 2024 10:41 AM

Le développement piloté par les tests (TDD), le développement piloté par les tests d'acceptation (ATDD) et le développement piloté par le comportement (BDD) sont des méthodes de développement de logiciels qui se concentrent sur l'écriture de tests avant l'écriture du code. Le TDD est une méthode de développement de logiciels qui repose sur la répétition d'un cycle de développement court : les exigences se transforment en cas de test très spécifiques. Le code est écrit pour que le test réussisse. Contrairement au développement piloté par les tests - où les tests d'acceptation sont créés du point de vue du développeur - ATDD préconise l'automatisation des tests du point de vue de l'utilisateur. BDD est une extension de TDD et ATDD qui encourage la collaboration entre les développeurs, les testeurs et les intervenants métier tout au long du cycle de vie du logiciel.

FOIRE AUX QUESTIONS

Il est important de savoir que l'automatisation n'est pas une obligation, nous pourrions simplement avoir des tests manuels et faire du développement piloté par les tests d'acceptation (ATDD)

Cependant, nous savons qu'il est préférable d'automatiser

😉

Nous savons que la technique TDD (Test-driven development) et BDD (Behavior-Driven Development) c'est pour l'automatisation de l'ATDD, mais savons-nous que…

- nous pouvons **écrire** les validations (tests) avant le développement !
- nous pouvons aussi **automatiser** les validations (tests) avant le développement !
- nous pouvons aussi **exécuter** l'automatisation des validations (tests) avant le développement !

- Terminer le debug
- agnostique du code

UTILISATION

Le développement piloté par les tests d'acceptation (**ATDD**)

C'est une méthodologie de développement basée sur la communication entre les clients, les développeurs et les QA (3 amigos (Y El Guapo)ARCHIVED).

ATDD englobe bon nombre de pratiques comme

- la modalisation/architecture passant du DDD par le développement axé sur les fonctionnalités (FDD)
- la spécification par l'exemple (SBE),
- le développement piloté par l'exemple (EDD),
- le développement piloté par le support développement également appelé développement piloté par les tests (SDD).
- L'exigence du développement piloté par les tests (TDD (Test-driven development))
- L'exigence du développement piloté par le comportement (BDD (Behavior-Driven Development)),

Tous ces processus aident les développeurs et les testeurs à comprendre les besoins du client avant la mise en œuvre et permettent aux clients de pouvoir converser dans leur propre langue de domaine.

ATDD englobe les tests d'acceptation , mais met l'accent sur l'écriture des tests d'acceptation avant que les développeurs ne commencent à coder.

On pourrait aussi s'amuser à dire **A**rcitecture **T**est-**D**riven **D**evelopment !
Car, le but est de définir nos tests attachés à l'architecture, "first".

L'essence même, le but de l'ATDD

Nous parlons ici du Shift-left, du DevOps, du cycle de vie Agile et de tester en continu

ÉCHOUER *puis échouer* **ENCORE** *puis échouer* **MIEUX** *jusqu'au* **SUCCÈS**

1. Définir les tests
   1.1. Développer selon les tests
   1.2. Valider unitairement au fur et à mesure (avec toutes et tous, mais surtout le client)
2. Si tout est **VERT** → c'est donc que c'est **LIVRÉ** et que le client est satisfait

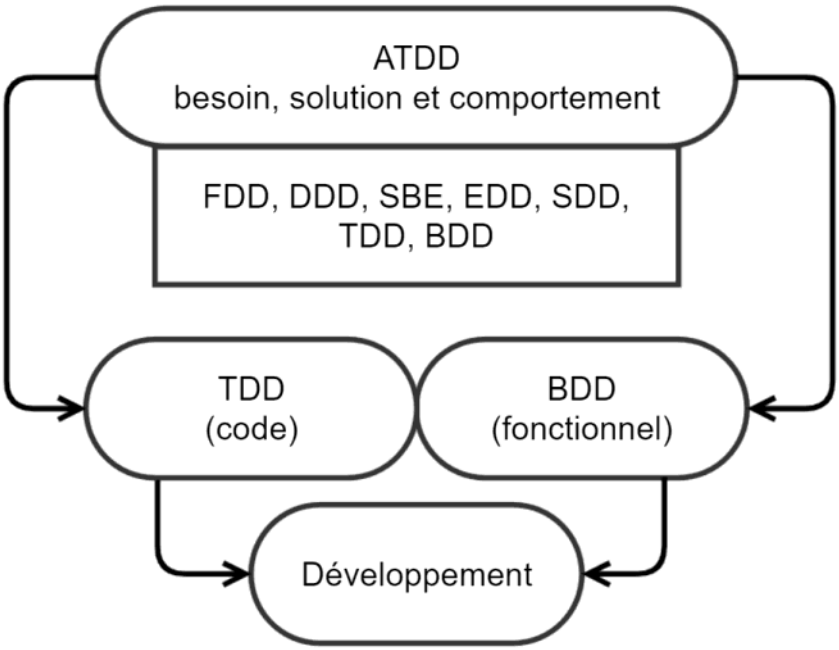
…C'est du développement piloté par tous les tests d'acceptation.

A*cceptance* **T***est-***D***riven* ***D****evelopment*

🙂

ILLUSTRATION

Avec ce diagramme on peut mieux situer le BDD et le TDD en amont du développement et en aval de ATDD



Scenario

January 18, 2024 10:42 AM

Un scénario peut avoir plusieurs sens !

- Quels sont tous les scénarios possibles ?
- Cela est aussi un synonyme de E2E ou source/destination
- Quel est le déroulement (donc les étapes)
-

C’est le "SCRIPT"

Canevas, plan détaillé

Un scénario est le canevas de mise en scène

C'est une trame écrite et détaillée des différentes scènes d’un film, comprenant généralement le découpage et les dialogues.

Il y a la trame à choix multiple (interactive) et la trame linéaire

Un roman traditionnel par exemple et linéaire, c’est premier c’est le chemin de passer du point A au point B, donc, une source, une destination, le résultat attendu est toujours l'atteinte de la destination

Attention, plusieurs itinéraires (les étapes à suivre) peuvent différer pour un même scénario

Comme par exemple les livres dont vous êtes le héro… on commence par A mais on ne sait pas où nous allons arriver

Est-ce que toutes les options possibles sont comme dire tous les scénarios possibles ?

Nous parlons ici de différents CAS de scénarios possibles, le bout en bout (End to End)

Un cas de scénario de test est défini comme toute fonctionnalité pouvant être testée. Elle est également appelée condition de test ou possibilité de test. En tant que testeur, vous devez vous mettre à la place de l'utilisateur final et comprendre les cas scénarios réels donc les cas d'utilisation de l'application sous test.

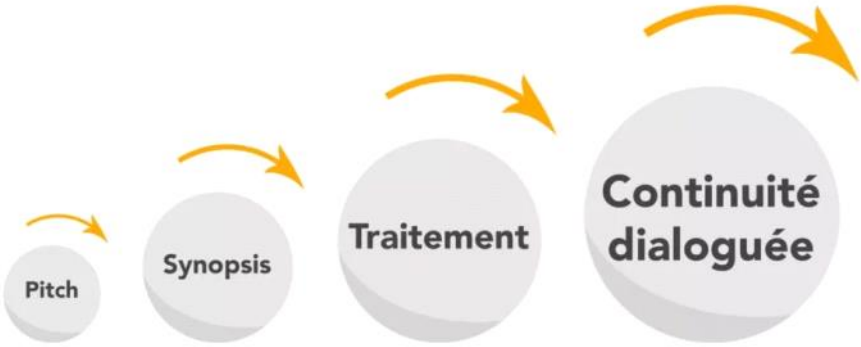
Quand nous parlons d’un scénario entendons-nous aussi de l'itinéraire, les checkpoints, etc.

Comme dans la phrase suivante… Quel serait le meilleur scénario selon les différents scénarios possibles pour se rendre à la maison ?

L’art de scénariser les tests c’est exactement comme dans les films, musique, théâtre… Il y a de la préparation et des méthodologie

Ajoutons la complexité des different cas d’utilisation

Le scénario linéaire en assurance qualité nous appelons ça le "happy path"


Pitch
Synopsis
Traitement
Continuité dialoguée

Le principe d'optimiser la quantité (éliminer le superflu)

January 18, 2024 10:42 AM

- La pyramide (découpage par les niveaux de test)
- et la gestion du risque

Sont 2 concepts pour optimiser la quantité de test à valeur ajouté.

Comment faire

1. Valeur ajoutée
2. Le risque
3. Affaire ou équipe versus technologie ou produit
4. Coder/Structurer de façon fonctionnel et sans avoir besoin de faire des unittests

Le SDLC (Software Development Life Cycle) permet une analyse approfondie à chaque phase du cycle de vie du développement logiciel, ce qui permet de mesurer l'efficacité de votre processus de développement logiciel et de l'améliorer . Les systèmes de gestion SDLC contrôlent chaque étape du processus tout en ajoutant des analyses, des systèmes de gestion du travail et un suivi des bogues qui permettent d'améliorer les parties du cycle de vie ne fonctionnant pas efficacement. En résumé, analyser et tester à chaque phase permet de maximiser l'efficacité, accélérer la vitesse et réduire les coûts à toutes les phases.

Les systèmes d'information et l'ingénierie logicielle, le cycle de vie du développement des systèmes (SDLC), également appelé cycle de vie du développement d'applications, est un processus de planification, de création, de test et de déploiement, donc, cycle de livraison d'un système d'information. Le concept SDLC s'applique à une gamme de configurations matérielles et logicielles, car un système peut être composé uniquement de matériel, de logiciel uniquement ou d'une combinaison des deux.

Les phases

Il y a généralement six phases dans ce cycle :

1. **DÉFINIR** (analyse des exigences/spécifications/besoins/solutions)
   Cette phase consiste à définir les exigences et les objectifs du projet. Les exigences sont collectées auprès des parties prenantes et sont utilisées pour créer une spécification des exigences.

2. **PLANIFIER** (conception, plan de test…)
   Cette phase consiste à planifier le projet en fonction des exigences définies dans la

phase précédente. Les activités de planification comprennent la définition des tâches, l'estimation des coûts et la planification des ressources.

3. **INTÉGRER** (développement)
   Cette phase consiste à intégrer les différents composants du système pour créer un système complet. Les tests d'intégration sont effectués pour s'assurer que tous les composants fonctionnent correctement ensemble.

4. **DIFFUSER** (mise en œuvre)
   Cette phase consiste à déployer le système dans l'environnement de production. Les tests de déploiement sont effectués pour s'assurer que le système fonctionne correctement dans l'environnement de production.

5. **OPÉRER**
   Cette phase consiste à exploiter le système en production. Les activités d'exploitation comprennent la surveillance du système, la gestion des incidents et la maintenance du système.

6. **VEILLER** (Satisfaction, alertes et statistiques)
   Cette phase consiste à surveiller le système en production pour s'assurer qu'il fonctionne correctement et qu'il répond aux exigences des utilisateurs. Notez que "**veiller**", à la satisfaction, focus plus sur le positif → "assurer", "protéger", "prendre soin de"…
   plutôt que "mesurer", "évaluer", "contrôler", "surveiller", "punir ou avoir des conséquences"…

Les états

Un cycle est une boucle itérative et aussi chacune des phases…

Il y a généralement 3 états par phase :

7. **Analyser**
8. **Faire**
9. **Valider** (tester)

Le "flow"

La documentation/métriques scientifique

Si vous suivez la méthode du chercheur scientifique en amélioration continue, lors de chaque phase, vous obtiendrez donc le succès, la confiance et la satisfaction !

Prendre des notes (comme un chercheur scientifique) est important et sous-estimé. Les traces est le levier de l'amélioration.

Il est important de recommencer toutes les phases lorsque une phase échoue ! Cela peut paraître intense, mais ce ne l'est pas, et surtout que c'est très payant

Le principe est de repositionner rapidement le besoin en continue si il y a lieu, plutôt que d'attendre à la fin pour le réaliser et trop tarder pour réviser. (ex. de questions après l'évolution du contexte : la priorisation, la valeur ajouté, le RoI, mise-à-jour de la documentation, etc.)

Vous remarquerez qu'en général nous passons instantanément les phases précédentes pour continuer là où nous devions continuer

Dans un cadre d'un sprint Agile, un récit (story) ou un bug devra avoir passé par toutes les phases avant de se terminer. Le terminer est la définition (DoD)

Voici le "flow" du retour au début :

Voici différentes façons de représenter le "FLOW"

Linéaire

"SHELL" or "STAIR"

(il y a aussi l'escalier montant, ce n'est qu'aller vers le haut plutôt que le bas)

"WHEEL" or "DONUT"

"INFINI" or "8" or "DevOps"

"V" or "BOL" or "Shift-Left"

D'autres profils SDLC

Il y a d'autres profils des phases et états comme le "Waterfall" par exemple, qui fait les validations/tests en tant qu'une des phases à la fin, souvent dit "UAT" (**important : ce n'est pas la définition exacte de UAT**) ou QA (test)

ref.: Systems development life cycle - Wikipedia

, ic

Comment rédiger une anomalie

January 18, 2024 10:03 AM

Objectif d’une anomalie

Il est tout aussi important de rapporter une anomalie que de savoir comment bien la décrire pour que celle-ci soit corrigée le plus rapidement possible et que ce soit bien la bonne chose qui soit réglée. Plus le rapport d'anomalie est court et clairs, moins il y aura d’interrogations sur celui-ci.
Certain système de gestion de tests nous permettent de lier une anomalies à une couverture des tests faits.

Audiences:

- Gestion de projet / produit:
- Faciliter la priorisation des taches à compléter avec des ressources limitées
- Comprendre l’impact sur les utilisateurs et le produit
- Comprendre la complexité de correction
- Équipe de développement:
- Identifier la cause du comportement à corriger
- Faciliter l’analyse de complexité de la correction
- Assurance Qualité:
- Faciliter la création de cas de test
- Faciliter l’exécution des tests reliés
- Générer des métriques de qualité pour le projet

Élément important d’une anomalie

Voici plusieurs éléments qui peuvent être mis dans le billet d’une anomalies. Les différents points ne seront pas toujours tous présents puisque cela dépends du genre de bogue rencontré, la situation dans laquelle on l’a trouvé et plusieurs autres facteurs.

Titre

Le titre doit être un résumé court et précis du problème encouru. Il est préférable de mettre la fonctionnalité touché en début de titre suivi d'une description brève du problème.

Exemples
À éviter

“Impossible d’envoyer un message sur la page de clavardage, c’est complètement brisé.”

- Vague.
- Agressif.

Bonne pratiques

“CLAVARDAGE - Impossible d'écrire dans le textbox.”

- L’emplacement problématique est localisé (CLAVARDAGE).
- Concentration sur le problème technique.

Autres exemples de bonnes pratique

Clavardage - Un utilisateur ne peut pas accéder au champ texte du Clavardage

Navigation - L’option Profil du menu de navigation n’est pas fonctionnelle

Résumé

En cas d'anomalies complexe dont le titre ne suffit pas à bien décrire le problème, un

résumé court pourrait être ajouté.

Utiliser un langage clair et informel. Ex: `tbd`

Prérequis

Dans quel cas peut-on reproduire l'anomalie? Y a-t-il quelques chose de spécifique à avoir/reproduire pour trouver le bogue?

Points à considérer:

- Accès. Par exemple, avoir les droits de consulter une base de données directement.
- Données. Partager des comptes particuliers qui pourraient être difficiles à recréer (Une police d'assurance avec 7 ans d'historique)
- Les étapes pour arriver au point de départ. Par exemple, comment s'authentifier ou accéder à une fonctionnalité obscure

Gravité

Résume l'impact fonctionnel sur le système avec des niveaux prédéfinis. Celui peut être d'une panne totale du système jusqu'à un impact seulement visuel.

Audience: Équipes de développement

Alias: Sévérité / Severity

Priorité

Ces attributs sont utilisés dans les anomalies pour communiquer des aspects importants à une audience spécifique.

Décrit l'impact sur le produit avec des niveaux prédéfinis. Cet attribut est souvent utilisé pour plusieurs type d'items de travail pour faciliter la priorisation des taches.

Audience: Équipe de gestion de projet

Alias: Priority

Exemples

Exemple 1: Les utilisateurs du Yukon ne peuvent pas accéder à l'application

- ***Gravité***: Critique - Puisque l'impact cause un panne totale pour ces utilisateurs sans contournement possible.
- ***Priorité***: Bas - Le nombre bas d'utilisateurs impactés réduit son urgence de correction.

Exemple 2: L'année affichée des droits d'auteurs de la marque de commerce n'est pas l'année actuelle

- ***Gravité***: Mineur - Puisque l'affichage n'a aucun impact fonctionnel sur le système
- ***Priorité***: Élevé - L'importance de l'aspect légal combiné avec une correction simple à implémenter augmente le niveau de priorité.

Ressources additionnelles

Confluence de la Pratique AQ:
https://alithya.atlassian.net/wiki/spaces/PAQ/pages/16662856...

Environnement

Les environnements typiques sont:

- Local
- Dev
- QA
- UAT
- Pre-prod / Staging

- Production

  Chaque environnement de développement a généralement des particularités uniques. Par exemple, un environnement de Dev pourrait avoir une base de données restreinte tandis qu'un environnement UAT pourrait avoir la base de données de production complète (anonymisée)

Étapes à reproduire

Décrire chaque étapes qui ont été faite pour reproduire le comportement. On peut montrer plusieurs méthodes de reproduire le bogue si tel est le cas.

Lister le détail minimum mais précis pour reproduire l'anomalie.

Exemple:

À éviter

1. Installer l'app
2. Tapper sur l'icone de l'app pour la démarrer
3. S'authentifier avec un compte valide

   Bonne pratiques
4. Accéder à la page accueil

Résultat attendu

Expliqué ce qui devrait se produire lorsque l'anomalie est corrigée.

Inclure les spécifications non respectées si possible (Récit Jira OU Règle d'affaire OU document de spécification)

Résultat actuel

Expliquer ce qui se produit actuellement.

Contournement

Décrire les méthodes de contournements découvertes durant les tests s'il y a lieu.

Un contournement possible peut aider l'équipe de gestion de projet à mieux prioriser les taches (une anomalie sévère peut être moins prioritaire si un contournement peut être partagé avec les utilisateurs impactés)

Notes

Permet d'expliquer quel tests ont été fait, les points observés qui semble affecter l'anomalie et ce qui ne semble pas produire l'anomalie. Indiquer si l'anomalie est encore reproductible.

Éléments techniques

Unique dans chaque contexte. Ajouter les détails pertinents qui peuvent assister less efforts de correction.

Exemples:

- Navigateurs utilisés
- Système d'exploitation (OS)
- la version de l'application et/ou de ses services

Exemple: Numéro de version de l'application, détails de l'appareil utilisé pour reproduire le comportement, fichier Log

Pièces jointes

Possible d'ajouter des captures d'écrans et vidéo avec le problème encouru.

Il est aussi possible de lier notre système de gestion de test à notre rapports d'anomalie.

Ressources

Gravité, sévérité et priorité de bug - Pratique Assurance Qualité - Confluence (atlassian.net)

BU Videotron

January 18, 2024 10:43 AM

	Sous-tâche (story et bug)		**DOD**	**Estimation**	**Moyenne tmp**
1	Définir le besoin	Investigation			
2	Définir la solution	Analyse			
3	Plan de test	• 3 amigos • Template • Documentation des test • Version de livraison	• Quoi tester ou pas • In-Scope • Hors-Scope • Prerequis • Données de test • Autres implémentations nécessaires • Élimination des bloquants • Étape précédantes du flow sdlc) • Quel environment • Acc4 • PPX (Sanity si besoin/applicable) • Prod (Sanity si besoin/applicable) • Comment • Cas de test • Technique de test • Étapes de reproduire • Documentation dans Go Test • Lien Go test	• Si on défini le plan de test dans les 3 amigos • qu'il y a une partie de temps pour remplir le Template • et combiens de temps pour documenter les tests • Il faut prendre du temps pour packager les releases (versionning)	2h
4	Implementation	Designer, Programmer, Coder, Configurer, Intégrer			
5	Validation LOCAL DEV				
6	Code review				
7	Validation PREV par un autre DEV				
8	Validation PREV	Validation du besoin/solution (UAT) PO-BA-SN-QA…			

		(colonne review)			
9	Automatisation de test				
10	Merge en DEV4	*(colonne review)*			
11	*Validation DEV4 DEV*	Pas obligatoire si déjà validé en PREV *(colonne review)*	versionné, packager		
12	~~Validation DEV4 QA~~	Le QA ne test pas en DEV			
13	Validation ACC	(colonne QA) ou ACC4	• Test de confirmité • Test de regression **de fonctionnalité (non pas regression complète de la release)** • NB: Détails d'éxecution + preuves de test dans les commentaires sur la story		2h
15	(colonne Ready to prod)				
16	~~Validation PPX QA~~	Le QA ne test pas en PPX			

Refresh (contenu SN)

La planification de test se fait avec une rencontre récurrente selon le besoin. Il est important de ne pas prendre du retard ou de la dette.

Définition d'un plan de test → Test plan / Plan de test

On utilise la méthode de la rencontre des 3 amigos (Y El Guapo)

Comment documenter un plan de test ?

Plan de test pour la Story ou Bug

Les éléments à avoir dans un plan de test pour une story ou un bug sont les suivants :

- Une introduction ou fiche descriptive qui présente le contexte, le périmètre, les objectifs et les livrables du plan de test.
- Une description du "scope" et du "out of scope" qui définit les fonctionnalités à tester et celles qui ne le sont pas, ainsi que les critères d'inclusion et d'exclusion.
- Une analyse des risques que permettent de couvrir les tests exécutés, en identifiant les scénarios critiques, les impacts potentiels et les mesures de mitigation.
- Les ressources matérielles et horaires nécessaires à la réalisation du plan de test, en estimant le temps, le coût, la charge et les compétences requises pour chaque activité de test.
- Les outils et environnements nécessaires à l'exécution des tests, en précisant les logiciels, les plateformes, les données, les configurations et les accès utilisés pour réaliser les tests.
- La stratégie et la méthodologie de test, en décrivant les types, les niveaux, les techniques et

les critères de test utilisés pour valider les fonctionnalités.
- Le plan de test détaillé, en listant les cas de test, les étapes, les données, les résultats attendus et les critères de succès pour chaque fonctionnalité à tester.
- Le rapport de test, en documentant les résultats, les anomalies, les mesures correctives et les recommandations issus de l'exécution des tests.

1. Bien comprendre le **besoin**
2. Bien comprendre la **solution**
3. **Couverture des tests** (lister tous les tests qui devront être exécutés)
   Prérequis :

a. Chaque test doit avoir le contexte, l'action (si nécessaire) et le résultat attendu
b. Où chacun des tests devront êtres exécutés (env.) ?
c. Qui exécutera chacun des tests ?
d. Avons-nous besoin de contributeur(s) ou d'appariement ?
e. Dépendance des jeux de données de test

1. Documenter les tests dans le gestionnaire de test, avec le quand et par qui.

a. révision par une autre personne incluse

1. Automatisation des tests dans l'outil d'automatisation, avec le quand et par qui.

a. révision par une autre personne incluse

1. Hors portée
2. todo…

Gabarit

todo…

Utilisation GoTest

Type	Nomenclature	Exemple
Test Plan	• Upper Case - Upper Case first letter	**CONTENT** - **A**pps **BEHAVIOR** - **I**nternet ✕ Alithya GoTest
Functionalité	GIVEN…THEN….EXIST GIVEN…WHEN TO…….THEN….EXIST	**GIVEN** mobility **THEN** shop **EXIST** ✕ Alithya GoTest **GIVEN** short pon address **WHEN TO** check eligibility **THEN** message **EXIST** ✕ Alithya GoTest
Scenario	all lower case • Environnement • User (anonymos, csr ….) • Mobile/Desktop • Navigator (Edge ,	**acc4-11, anonymous, mobile, chrome, en** **dev4-10, anonymous, mobile, firefox, fr** ✕ Alithya GoTest

	Chrome) • Language : Fr or En	
Steps	VALIDATION TEARDOWN must always have close browser	
Keyword combined (Modules)	GIVEN…WHEN…….THEN ….EXIST (le même nom que le “test”) ou validate for submodule	**GIVEN** address **WHEN** TO check eligibility **THEN** message **EXIST** ✕ Alithya GoTest
Variable (List , dictionnary , environment)	**UPPERCASE** for system **lowercase** the rest	${**OS**} ${**width**}

- Ne pas utiliser des données en dur , utiliser des variables
- Les Xpath
- utiliser **//*** exemple **//*[contains(text(),"${message}")]**
- utiliser des **[contains(text()…]**
- `[contains(@value.'Feeling')]`

Environnements test

- [WIP] Drupal QA work flow

	Development		**Release Candidate**		**Release**
Branche	Branche de DEV (story/bug)	DXP-SPRINT (master) Contient tous les changements.	RC, déploiement du code sélectionner (tag) seulement pour une version.		
Environnement	**PREV (If the code is Drupal only)**	**ou DEV4 (Eventually we will just use PREV, we are waiting for the magic browser to completely use PREV)**	**ACC (If the code is Drupal only)**	**ou ACC4 (Eventually we will just use ACC, we are waiting for the magic browser to completely use ACC)**	**PROD**
Qui valide ?	• Analyste affaire • PO • Front-end/Desegner/	“”	• QA • SN	“”	

	UI/UX • SN • Pair testing with QA or an other Dev				
Créer ou mettre à jour la documentation des tests	✓	devrait être fait en amont si possible	devrait être fait en amont si possible	devrait être fait en amont si possible	devrait être fait en amont si possible
Automatisation	✓	devrait être fait en amont si possible	devrait être fait en amont si possible	devrait être fait en amont si possible	devrait être fait en amont si possible
Garder des traces de ce qui à été testé	Exécutez la validation GoTest et joignez le résultat dans la story/bug ou si pas documenter dans GoTest, mettez une preuve vidéo/captures d'écran dans la section des commentaires	Exécutez la validation GoTest et joignez le résultat dans la story/bug ou si pas documenter dans GoTest, mettez une preuve vidéo/captures d'écran dans la section des commentaires	Exécutez la validation GoTest et joignez le résultat dans la story/bug ou si pas documenter dans GoTest, mettez une preuve vidéo/captures d'écran dans la section des commentaires	Exécutez la validation GoTest et joignez le résultat dans la story/bug ou si pas documenter dans GoTest, mettez une preuve vidéo/captures d'écran dans la section des commentaires	Exécutez la validation GoTest et joignez le résultat dans la story/bug ou si pas documenter dans GoTest, mettez une preuve vidéo/captures d'écran dans la section des commentaires
Quoi	• Smokes • Unitaire du code • Valider si le besoin est comblé • Valider si la solution est la bonne • Valider si le comportement est celui attendu	“”	• Smokes • Les fonctionnalités touchées • Les dépendances • Non-régression • L’accessibilité • La compatibilité	“”	

TODO…

- How will this be done in the Jira board?
- When/Where test documentation and test automation?
- Comment, quoi… Le **DoD** par environnements

Gestion de la sécurité

La sécurité du logiciel est l’ensemble des mesures et des pratiques visant à protéger les logiciels contre les attaques malveillantes, les accès non autorisés, les modifications indésirables et les pertes de données. La sécurité des logiciels est importante pour garantir la confidentialité, l’intégrité, la disponibilité et la qualité des applications et des données.

Pour intégrer la sécurité des logiciels dans l'assurance qualité et dans toutes les phases et étapes du cycle de vie du développement, il faut adopter une approche de sécurité par la conception (secure by design), qui consiste à intégrer la sécurité dès le début du projet et à la maintenir tout au long du processus. Voici quelques exemples de bonnes pratiques pour mettre en œuvre cette approche (avec les phases du cycle DevOps) :

- **Définir** : il faut définir les exigences de sécurité en fonction des besoins et des attentes des clients, des utilisateurs et des parties prenantes. Il faut également réaliser une analyse des risques pour identifier les menaces potentielles et les mesures de prévention ou de mitigation.
- **Planifier** : il faut planifier le développement du logiciel en tenant compte des aspects de sécurité, tels que le choix des technologies, des outils, des méthodes et des standards à utiliser. Il faut également définir les rôles et les responsabilités des équipes impliquées dans la sécurité du logiciel.
- **Intégrer** : il faut intégrer la sécurité dans le code du logiciel, en utilisant des techniques de programmation sécurisée, telles que le chiffrement, l'authentification, l'autorisation, la validation des entrées, etc. Il faut également utiliser des outils d'analyse statique ou dynamique pour détecter les vulnérabilités dans le code.
- **Diffuser** : il faut diffuser le logiciel en respectant les normes et les procédures de sécurité, telles que la signature numérique, le contrôle d'accès, la vérification d'intégrité, etc. Il faut également effectuer des tests de sécurité avant et après le déploiement, tels que les tests d'intrusion, les tests de performance, les tests d'acceptation, etc.
- **Opérer** : il faut opérer le logiciel en assurant sa maintenance et sa mise à jour régulière, en appliquant les correctifs de sécurité nécessaires, en surveillant son fonctionnement et en réagissant aux incidents de sécurité.
- **Veiller/mesurer** : il faut veiller sur la sécurité du logiciel en collectant et en analysant les données relatives aux performances, aux incidents, aux retours d'expérience, etc. Il faut également mesurer l'efficacité du processus de sécurité et l'améliorer en continu.

En résumé, la sécurité des logiciels est un aspect essentiel du développement et de l'exploitation des applications. Pour intégrer la sécurité dans l'assurance qualité et dans toutes les phases et étapes du cycle de vie du développement, il faut adopter une approche de sécurité par la conception, qui implique la prise en compte de la sécurité dès le début du projet et tout au long du processus. J'espère que cela vous a été utile. 😊

Ces niveaux de maturité ne sont pas figés ni linéaires, mais ils peuvent servir de repères pour évaluer la situation actuelle et identifier les axes d'amélioration.

3. **Niveau 1 : Initial**. Il n'y a pas de processus clair ni de méthode de sécurité. Les livraisons sont irrégulières, imprévisibles et non testées. Les équipes sont isolées et ne communiquent pas entre elles. Les clients sont insatisfaits et les utilisateurs sont frustrés.
4. **Niveau 2 : Réactif**. Il y a une prise de conscience de la nécessité d'un processus de sécurité, mais il n'est pas formalisé ni partagé. Les livraisons sont plus fréquentes, mais pas toujours fiables. Les équipes commencent à collaborer, mais il y a encore des silos et des conflits. Les clients sont impliqués, mais pas toujours écoutés.
5. **Niveau 3 : Proactif**. Il y a un processus de sécurité défini, documenté et appliqué avec rigueur. Les livraisons sont rapides, régulières et testées. Les équipes travaillent ensemble, avec des rôles et des responsabilités clairs. Les clients sont co-créateurs du produit et les utilisateurs sont satisfaits.
6. **Niveau 4 : Performant**. Il y a un processus de sécurité mesuré, analysé et amélioré en continu. Les livraisons sont optimisées, adaptées et sécurisées. Les équipes sont autonomes, auto-organisées et apprenantes. Les clients sont fidélisés et les utilisateurs sont enchantés.
7. **Niveau 5 : Innovant**. Il y a un processus de sécurité innovant, qui anticipe les besoins du

marché et crée de la valeur ajoutée. Les livraisons sont disruptives, différenciantes et personnalisées. Les équipes sont visionnaires, créatives et inspirantes. Les clients sont ambassadeurs et les utilisateurs sont fans.

Il nous reste qu'a nous donner des objectifs

Rôle	Coach Qualité	Automaticien Qualité	Analyste Qualité	Contrôleur (testeur) Qualité
Responsabilités	• Rendre autonome et efficace, l'équipe dans les fonctions de l'assurance qualité. • Proposer des pistes d'amélioration. • Assister à la mise en place des pistes d'améliorations. • Faire des ateliers, de l'entraînement et des formations.	• Automatiser les tests.	• Assurer l'analyse (planification) et assurer le suivi de la qualité des livraisons (plan de test). • Assurer la documentation, référencer les tests et les livraisons. • Assurer les jeux de données de tests. • Assurer la satisfaction de l'utilisateur.	• Valider des bugs et les stories • Exécutions de test (x. non-régressions)
Nos besoins, attentes et/ou demandes	• Avoir un roadmap • Avoir des ateliers QA			
Quoi améliorer ?				
Solutions				

Toolbox / Boîte à outils

- Model-Based Testing / Tests basés sur un modèle (Videotron.com)
- GoTest

Qualimétrie

- Audit

PRG-180801 - Darwin ProgramDarwin - DrupalReport / Rapport (Drupal)

Share

Report / Rapport (Drupal)



Owned by Michel Belanger

Last updated: Oct 24, 2023

12 min read4 people viewed

- Audit interne
- Terminologie
- Rôles et responsabilités
- Coffre/boîte à outils
- Gestion de test
- Gestion de bug
- Planification
- Qualimétrie

Audit interne

Impact

- **Risques**

L'assurance qualité du logiciel est un processus qui vise à garantir que le logiciel développé répond aux exigences fonctionnelles, techniques et de performance attendues par les utilisateurs et les clients. Elle implique des activités de planification, de conception, de codage, de test, de revue, de mesure et d'amélioration continue du logiciel. Si il n'y avait pas d'audit interne de l'assurance qualité du logiciel, les risques et les impacts possibles seraient les suivants:

- Une diminution de la qualité et de la fiabilité du logiciel, qui pourrait entraîner des dysfonctionnements, des bugs, des failles de sécurité ou des non-conformités aux normes et aux réglementations.
- Une insatisfaction et une perte de confiance des utilisateurs et des clients, qui pourraient se plaindre ou se tourner vers d'autres solutions logicielles plus performantes et plus sûres.
- Une augmentation des coûts et des délais de développement, de maintenance et de correction du logiciel, qui pourrait affecter la rentabilité et la compétitivité du projet ou de l'organisation.
- Une réduction de l'efficacité et de l'innovation du logiciel, qui pourrait limiter sa capacité à répondre aux besoins évolutifs du marché et aux attentes des utilisateurs.
- Une exposition à des risques opérationnels, financiers, juridiques ou stratégiques, qui pourraient compromettre la réussite ou la pérennité du projet ou de l'organisation.

L'audit interne de l'assurance qualité du logiciel est donc un élément essentiel pour assurer la conformité, la sécurité et la performance du logiciel, ainsi que pour renforcer la valeur ajoutée et la satisfaction des utilisateurs et des clients. Il contribue à la maîtrise des risques, au respect des obligations légales et éthiques, et à l'amélioration continue de la qualité

- **Niveaux de maturité**

Niveau 1: Il n'y a pas d'audit défini ni documenté pour le projet logiciel. Les activités d'audit sont inexistantes ou inadaptées. L'évaluation et l'amélioration de la conformité ne sont pas réalisées ni prises en compte.

Niveau 2: Il existe un audit basique, qui est réalisé de manière inconsistante. Les activités d'audit sont planifiées et exécutées selon des règles simples, sans suivi ni vérification

systématiques. L'évaluation et l'amélioration de la conformité sont réalisées de manière subjective ou arbitraire.
Niveau 3: Il existe un audit standardisé, qui est réalisé de manière rigoureuse. Les activités d'audit sont planifiées et exécutées selon des méthodes et des techniques reconnues, avec un suivi et une vérification systématiques. L'évaluation et l'amélioration de la conformité sont réalisées de manière objective et systématique.
Niveau 4: Il existe un audit optimisé, qui est adapté aux besoins spécifiques du projet. Les activités d'audit sont planifiées et exécutées selon des méthodes et des techniques adaptées au contexte du projet, avec un suivi et une vérification automatisés. L'évaluation et l'amélioration de la conformité sont réalisées de manière optimale et ajustée en fonction de l'avancement du projet.
Niveau 5: Il existe un audit innovant, qui est constamment amélioré selon les meilleures pratiques du marché. Les activités d'audit sont planifiées et exécutées selon des outils et des méthodes avancées, qui permettent de couvrir tous les aspects du projet. L'évaluation et l'amélioration de la conformité sont réalisées de manière innovante et flexible en fonction des opportunités du projet.

- **Avantages**
  Avoir une audit mature de l'assurance qualité du logiciel présente de nombreux avantages pour une organisation, tels que :
- Une amélioration de la qualité et de la fiabilité du logiciel, qui réduit les risques de dysfonctionnements, de bugs, de failles de sécurité ou de non-conformités aux normes et aux réglementations.
- Une augmentation de la satisfaction et de la fidélité des utilisateurs et des clients, qui apprécient un logiciel performant, sécurisé et adapté à leurs besoins.
- Une réduction des coûts et des délais de développement, de maintenance et de correction du logiciel, qui optimise la rentabilité et la compétitivité du projet ou de l'organisation.
- Une stimulation de l'efficacité et de l'innovation du logiciel, qui renforce sa capacité à répondre aux besoins évolutifs du marché et aux attentes des utilisateurs.
- Une maîtrise des risques opérationnels, financiers, juridiques ou stratégiques, qui assure la réussite et la pérennité du projet ou de l'organisation.
  L'audit mature de l'assurance qualité du logiciel implique la mise en place d'un système d'assurance qualité basé sur des normes, des audits, des revues, des tests, des mesures, des politiques, des processus et des procédures clairement définis et efficaces. Il permet également d'évaluer et d'améliorer en continu la qualité du logiciel grâce à une analyse et une collecte des erreurs, une gestion de la configuration logicielle et une gestion du risque.
  État actuel
- Constats et observations
- Il n'y a pas d'audit interne
- Non-conformités
- …
  Recommandation
- Opportunités d'amélioration…
  Évaluation

  Maturité : 1/5
  Priorité : Haute
  Effort : Haute

Terminologie

Impact

- **Risques**

Ne pas avoir de terminologie de l'assurance qualité du logiciel peut entraîner des risques et des impacts négatifs pour une organisation, tels que :

- Une confusion et une incompréhension entre les parties prenantes du projet logiciel, qui peuvent avoir des attentes, des besoins et des exigences différents ou contradictoires.
- Une difficulté à établir et à respecter les normes, les règles et les procédures de qualité, qui peuvent varier selon les contextes, les domaines ou les réglementations.
- Une perte de temps et d'argent à cause des erreurs, des défauts, des non-conformités ou des litiges liés à la qualité du logiciel.
- Une dégradation de la réputation et de la compétitivité de l'organisation, qui peut perdre la confiance et la fidélisation de ses clients ou de ses partenaires.
  Avoir une terminologie de l'assurance qualité du logiciel est donc un élément essentiel pour assurer la clarté, la cohérence et la communication entre les acteurs impliqués dans le développement, le déploiement et l'utilisation du logiciel. Elle permet également de faciliter l'application et l'évaluation des processus et des pratiques de qualité, ainsi que d'améliorer la performance et la satisfaction du logiciel.
- **Niveaux de maturité**
  Niveau 1: Il n'y a pas de terminologie définie ni documentée pour le projet logiciel. Les termes utilisés sont ambigus, imprécis ou contradictoires. La communication entre les acteurs du projet est difficile ou incomplète.
  Niveau 2: Il existe une terminologie basique, qui est utilisée de manière inconsistante. Les termes utilisés sont clairs, mais pas toujours adaptés ou cohérents. La communication entre les acteurs du projet est possible, mais pas toujours efficace.
  Niveau 3: Il existe une terminologie standardisée, qui est utilisée de manière rigoureuse. Les termes utilisés sont pertinents, précis et cohérents. La communication entre les acteurs du projet est fluide et complète.
  Niveau 4: Il existe une terminologie optimisée, qui est adaptée aux besoins spécifiques du projet. Les termes utilisés sont spécifiques, complets et harmonisés. La communication entre les acteurs du projet est optimale et sans ambiguïté.
  Niveau 5: Il existe une terminologie innovante, qui est constamment améliorée selon les meilleures pratiques du marché. Les termes utilisés sont originaux, exhaustifs et normalisés. La communication entre les acteurs du projet est excellente et sans perte d'information.
- **Avantages**
  Avoir la terminologie en assurance qualité du logiciel présente plusieurs avantages pour une organisation, tels que :

- Une meilleure communication et compréhension entre les parties prenantes du projet logiciel, qui peuvent utiliser un langage commun et précis pour exprimer leurs besoins, leurs exigences et leurs attentes.
- Une facilité à établir et à respecter les normes, les règles et les procédures de qualité, qui peuvent être clairement définies et documentées selon les standards internationaux.
- Une réduction des erreurs, des défauts, des non-conformités ou des litiges liés à la qualité du logiciel, qui peuvent être évités ou détectés plus rapidement grâce à des techniques, des pratiques et des outils appropriés.
- Une amélioration de la réputation et de la compétitivité de l'organisation, qui peut offrir un logiciel de qualité, performant, sécurisé et adapté aux besoins des utilisateurs et des clients.
  L'assurance qualité du logiciel est donc un processus qui vise à garantir que le logiciel développé répond aux exigences fonctionnelles, techniques et de performance attendues par les utilisateurs et les clients. Elle implique des activités de planification, de conception, de codage, de test, de revue, de mesure et d'amélioration continue du logiciel.
  État actuel

- Constats et observations
- Il n'y a pas de terminologie
- Non-conformités

- …

Recommandation

- Opportunités d'amélioration…

Évaluation

Maturité : 1/5
Priorité : Haute
Effort : Haute

Rôles et responsabilités

Impact

- **Risques**
- Une confusion ou une incompréhension entre les parties prenantes du projet logiciel, qui peuvent avoir des attentes, des besoins et des exigences différents ou contradictoires.
- Une difficulté à établir et à respecter les normes, les règles et les procédures de qualité, qui peuvent varier selon les contextes, les domaines ou les réglementations.
- Une perte de temps et d'argent à cause des erreurs, des défauts, des non-conformités ou des litiges liés à la qualité du logiciel.
- Une dégradation de la réputation et de la compétitivité de l'organisation, qui peut perdre la confiance et la fidélisation de ses clients ou de ses partenaires.
- Une exposition à des risques opérationnels, financiers, juridiques ou stratégiques, qui pourraient compromettre la réussite ou la pérennité du projet ou de l'organisation.
- **Niveaux de maturité**

  …Tous les rôles du logiciel et les responsabilités envers l'assurance qualité

  Niveau 1: Il n'y a pas de rôles ni de responsabilités définis ni documentés pour le projet logiciel. Les acteurs du projet sont indéfinis, interchangeables ou redondants. La répartition des tâches est aléatoire ou arbitraire.

  Niveau 2: Il existe des rôles et des responsabilités basiques, qui sont attribués de manière inconsistante. Les acteurs du projet sont identifiés, mais pas toujours qualifiés ou disponibles. La répartition des tâches est basée sur des critères simples, mais pas toujours respectée ou contrôlée.

  Niveau 3: Il existe des rôles et des responsabilités standardisés, qui sont attribués de manière rigoureuse. Les acteurs du projet sont qualifiés, disponibles et impliqués. La répartition des tâches est basée sur des critères objectifs, et est respectée et contrôlée systématiquement.

  Niveau 4: Il existe des rôles et des responsabilités optimisés, qui sont attribués de manière adaptée aux besoins spécifiques du projet. Les acteurs du projet sont compétents, motivés et autonomes. La répartition des tâches est basée sur des méthodes et des techniques reconnues, et est optimisée et ajustée en fonction de l'avancement du projet.

  Niveau 5: Il existe des rôles et des responsabilités innovants, qui sont attribués de manière constamment améliorée selon les meilleures pratiques du marché. Les acteurs du projet sont experts, engagés et coopératifs. La répartition des tâches est basée sur des outils et des méthodes avancées, et est innovante et flexible en fonction des opportunités du projet.
- **Avantages**
- Une meilleure communication et compréhension entre les parties prenantes du projet logiciel, qui peuvent utiliser un langage commun et précis pour exprimer leurs besoins, leurs exigences et leurs attentes.
- Une facilité à établir et à respecter les normes, les règles et les procédures de qualité, qui peuvent être clairement définies et documentées selon les standards internationaux.
- Une réduction des erreurs, des défauts, des non-conformités ou des litiges liés à la qualité du logiciel, qui peuvent être évités ou détectés plus rapidement grâce à des techniques, des

pratiques et des outils appropriés.
- Une amélioration de la réputation et de la compétitivité de l'organisation, qui peut offrir un logiciel de qualité, performant, sécurisé et adapté aux besoins des utilisateurs et des clients.
- Une maîtrise des risques opérationnels, financiers, juridiques ou stratégiques, qui assure la réussite et la pérennité du projet ou de l'organisation.

État actuel

- Constats et observations
- Il n'y a pas de responsabilité envers l'assurance qualité
- Non-conformités
- …

Recommandation

- Opportunités d'amélioration…

Évaluation

Maturité : 1/5
Priorité : Haute
Effort : Haute

Coffre/boîte à outils

Impact

- **Risques**

…

- **Niveaux de maturité**

Niveau 1: Il n'y a pas de boîte à outils définie ni documentée pour le projet logiciel. Les outils utilisés sont obsolètes, inadaptés ou inexistants. L'utilisation des outils n'est pas maîtrisée ni contrôlée.
Niveau 2: Il existe une boîte à outils basique, qui est utilisée de manière inconsistante. Les outils utilisés sont courants, mais pas toujours appropriés ou cohérents. L'utilisation des outils n'est pas optimale ni vérifiée.
Niveau 3: Il existe une boîte à outils standardisée, qui est utilisée de manière rigoureuse. Les outils utilisés sont pertinents, précis et cohérents. L'utilisation des outils n'est pas seulement efficace mais aussi mesurée et contrôlée.
Niveau 4: Il existe une boîte à outils optimisée, qui est adaptée aux besoins spécifiques du projet. Les outils utilisés sont spécifiques, complets et harmonisés. L'utilisation des outils n'est pas seulement mesurée et contrôlée mais aussi optimisée et ajustée en fonction de l'avancement du projet.
Niveau 5: Il existe une boîte à outils innovante, qui est constamment améliorée selon les meilleures pratiques du marché. Les outils utilisés sont originaux, exhaustifs et normalisés. L'utilisation des outils n'est pas seulement optimisée et ajustée mais aussi innovante et flexible en fonction des opportunités du projet.

- **Avantages**
- …

État actuel

- Constats et observations
- Il n'y a pas d'outils
- Non-conformités
- …

Recommandation

- Opportunités d'amélioration…

Évaluation

Maturité : 1/5

Priorité : Haute
Effort : Haute

Gestion de test

Impact

- **Risques**

…

- **Niveaux de maturité**

Niveau 1: Il n'y a pas de processus de test défini ni documenté. Les tests sont effectués de manière ad hoc, sans planification ni suivi. Les résultats des tests ne sont pas analysés ni communiqués. La qualité du logiciel n'est pas mesurée ni contrôlée.
Niveau 2: Il existe un processus de test basique, qui est appliqué de manière inconsistante. Les tests sont planifiés et exécutés selon des objectifs et des critères d'acceptation. Les résultats des tests sont enregistrés et rapportés. La qualité du logiciel est évaluée de manière subjective.
Niveau 3: Il existe un processus de test standardisé, qui est suivi de manière rigoureuse. Les tests sont conçus et exécutés selon des méthodes et des techniques reconnues. Les résultats des tests sont analysés et utilisés pour améliorer le processus de test. La qualité du logiciel est mesurée et contrôlée selon des indicateurs objectifs.
Niveau 4: Il existe un processus de test optimisé, qui est adapté aux besoins spécifiques du projet. Les tests sont automatisés et intégrés au processus de développement. Les résultats des tests sont exploités pour optimiser la performance et la fiabilité du logiciel. La qualité du logiciel est prédite et garantie selon des normes élevées.
Niveau 5: Il existe un processus de test innovant, qui est constamment amélioré selon les meilleures pratiques du marché. Les tests sont réalisés avec des outils et des méthodes avancées, qui permettent de couvrir tous les aspects de la qualité du logiciel. Les résultats des tests sont partagés et valorisés au sein de l'organisation. La qualité du logiciel est un facteur clé de succès et de compétitivité.

- **Avantages**
- …

État actuel

- Constats et observations
- Il n'y a pas de gestion de test
- Non-conformités
- …

Recommandation

- Opportunités d'amélioration…

Évaluation

Maturité : 1/5
Priorité : Haute
Effort : Haute

Gestion de bug

Impact

- **Risques**

…

- **Niveaux de maturité**

Niveau 1: Il n'y a pas de processus de gestion de bug défini ni documenté. Les bugs sont détectés et corrigés de manière aléatoire, sans priorisation ni traçabilité. Les causes et les

impacts des bugs ne sont pas analysés ni communiqués. La satisfaction du client n'est pas mesurée ni prise en compte.
Niveau 2: Il existe un processus de gestion de bug basique, qui est appliqué de manière inconsistante. Les bugs sont enregistrés et corrigés selon des règles simples, sans suivi ni vérification. Les causes et les impacts des bugs sont évalués de manière subjective. La satisfaction du client est écoutée mais pas systématiquement prise en compte.
Niveau 3: Il existe un processus de gestion de bug standardisé, qui est suivi de manière rigoureuse. Les bugs sont classifiés et corrigés selon des critères objectifs, avec un suivi et une vérification systématiques. Les causes et les impacts des bugs sont analysés et utilisés pour améliorer le processus de développement. La satisfaction du client est mesurée et contrôlée selon des indicateurs objectifs.
Niveau 4: Il existe un processus de gestion de bug optimisé, qui est adapté aux besoins spécifiques du projet. Les bugs sont prévenus et corrigés selon des méthodes et des techniques reconnues, avec un suivi et une vérification automatisés. Les causes et les impacts des bugs sont exploités pour optimiser la performance et la fiabilité du logiciel. La satisfaction du client est prédite et garantie selon des normes élevées.
Niveau 5: Il existe un processus de gestion de bug innovant, qui est constamment amélioré selon les meilleures pratiques du marché. Les bugs sont détectés et corrigés avec des outils et des méthodes avancées, qui permettent de couvrir tous les aspects du logiciel. Les causes et les impacts des bugs sont partagés et valorisés au sein de l'organisation. La satisfaction du client est un facteur clé de succès et de compétitivité.

- **Avantages**
- …

État actuel

- Constats et observations
- Il y a une gestion de bug de base
- Non-conformités
- …

Recommandation

- Opportunités d'amélioration…

Évaluation

Maturité : 1/5
Priorité : Haute
Effort : Haute

Planification

Impact

- **Risques**

…

- **Niveaux de maturité**

…Plan de test, gestion de livraison

Niveau 1: Il n'y a pas de planification définie ni documentée pour le projet logiciel. Les activités de test et de livraison sont effectuées de manière ad hoc, sans objectifs ni critères clairs. Le suivi et le contrôle du projet sont inexistants ou inefficaces.
Niveau 2: Il existe une planification basique, qui est appliquée de manière inconsistante. Les activités de test et de livraison sont planifiées et exécutées selon des règles simples, sans suivi ni vérification systématiques. Le suivi et le contrôle du projet sont possibles, mais pas toujours efficaces.
Niveau 3: Il existe une planification standardisée, qui est suivie de manière rigoureuse. Les activités de test et de livraison sont planifiées et exécutées selon des méthodes et des

techniques reconnues, avec un suivi et une vérification systématiques. Le suivi et le contrôle du projet sont efficaces et complets.
Niveau 4: Il existe une planification optimisée, qui est adaptée aux besoins spécifiques du projet. Les activités de test et de livraison sont planifiées et exécutées selon des méthodes et des techniques adaptées au contexte du projet, avec un suivi et une vérification automatisés. Le suivi et le contrôle du projet sont optimisés et ajustés en fonction de l'avancement du projet.
Niveau 5: Il existe une planification innovante, qui est constamment améliorée selon les meilleures pratiques du marché. Les activités de test et de livraison sont planifiées et exécutées selon des outils et des méthodes avancées, qui permettent de couvrir tous les aspects du projet. Le suivi et le contrôle du projet sont innovants et flexibles en fonction des opportunités du projet.

- **Avantages**
- …

État actuel

- Constats et observations
- Il n'y a pas de planification
- Non-conformités
- …

Recommandation

- Opportunités d'amélioration…

Évaluation

Maturité : 1/5
Priorité : Haute
Effort : Haute

Qualimétrie

Impact

- **Risques**

…

- **Niveaux de maturité**

…Se mesurer pour s'améliorer

Niveau 1: Il n'y a pas de qualimétrie définie ni documentée pour le projet logiciel. Les indicateurs de qualité sont inexistants ou inadaptés. La mesure et l'amélioration de la qualité ne sont pas réalisées ni prises en compte.
Niveau 2: Il existe une qualimétrie basique, qui est utilisée de manière inconsistante. Les indicateurs de qualité sont courants, mais pas toujours appropriés ou cohérents. La mesure et l'amélioration de la qualité sont réalisées de manière subjective ou arbitraire.
Niveau 3: Il existe une qualimétrie standardisée, qui est utilisée de manière rigoureuse. Les indicateurs de qualité sont pertinents, précis et cohérents. La mesure et l'amélioration de la qualité sont réalisées de manière objective et systématique.
Niveau 4: Il existe une qualimétrie optimisée, qui est adaptée aux besoins spécifiques du projet. Les indicateurs de qualité sont spécifiques, complets et harmonisés. La mesure et l'amélioration de la qualité sont réalisées de manière optimale et ajustée en fonction de l'avancement du projet.
Niveau 5: Il existe une qualimétrie innovante, qui est constamment améliorée selon les meilleures pratiques du marché. Les indicateurs de qualité sont originaux, exhaustifs et normalisés. La mesure et l'amélioration de la qualité sont réalisées de manière innovante et flexible en fonction des opportunités du projet.

- **Avantages**

- …
  État actuel
- Constats et observations
- Il n'y a pas de qualimétrie
- Non-conformités
- …
  Recommandation
- Opportunités d’amélioration…
  Évaluation

  Maturité : 1/5
  Priorité : Haute
  Effort : Haute

PRG-180801 - Darwin ProgramDarwin - DrupalAction plan (road map) / Plan d'action (feuille de route)

Share

Action plan (road map) / Plan d'action (feuille de route)



Owned by Michel Belanger

Last updated: Nov 22, 2023

6 min read6 people viewed

Positionnement d'amélioration du processus assurance qualité (actions, réalisations et projections)

Il est important de souligner que les améliorations sont itératives et que nous devons être fonctionnel dès le premier jour.

Priorités :

8. Amélioration des environnements de test (stable, private for Drupal and test data control)
9. Avoir un coffre d’outils (il n’y a rien de bon chez Vidéotron, il faut se débrouiller par nous même) **! URGENT !**
10. Amélioration, avoir de la documentation de test (avant d’intégrer des nouvelles ressources) → voir pour une solution temporaire.

11. Mettre en place l’audit et le plan d’actions
12. Suivre le plan…

Attention

- Pour éviter les pièges, ne pas automatiser avant d’avoir une stratégie et un plan d’automatisation **(pour faire une stratégie il faut avoir une couverture et pour avoir une couverture il faut documenter les tests et pour documenter il faut un outil professionnel prêt à l’utilisation conçu pour ça et de préférence pensé en vue d’automatiser)**
-

Agenda des jalons

OKR (objectifs et résultats clés)	État	Date de début	Estimation du temps	Date cible (prévue)	Tâches (Jira)

OKR (objectifs et résultats clés)	État	Date de début	Estimation du temps	Date cible (prévue)	Tâches (Jira)
QAP | Audition - Quality assurance process improvement	IN PROGRESS	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	Presentation of the roadmap, creation of the first Jira tasks (to the team)
					Workshop “What does a QA Coach do?”
					Onboarding QA (SDET profile)
					Projection over 3 weeks (by sprint)
					Long-term projection (per PI/Q and year)
					Positioning report
					Roadmaps
					Quality Assurance Roles, Responsibilities, and Expectations
					Terminer la politique AQ
					Première analyse de l'état actuel (Pistes et Propositions d’améliorations) Case départ,niveau 1 de la maturité, il n’y a aucune documentation des tests et manque d’outils fonctionnels QA (**URGENCE !**) L'équipe Alithya se retrouve sans outils QA car l'équipe Appnovation était autonome et a quittée avec leurs outils QA. Videotron est en restructuration QA, il n’y a rien de bon à proposer pour nos besoins et contextes présentement. • Trop d’environnements de test et ultra instables. Tous les tests sont dans des environnements E2E (Pas de test unitaire, intégration et système) Une demande d’aide d’Alithya pour

					l'équipe Drupal pour pallier au manque que nous n'avons plus d'outils chez Videotron : **Solution → GoTest**
					Analyse profonde et complète de l'état (Pistes et Propositions d'améliorations)
QAP | Audition - Quality assurance process improvement - Maturity level 2/5					
QAP | Audition - Quality assurance process improvement - Maturity level 3/5					
QAP | Audition - Quality assurance process improvement - Maturity level 4/5					
QAP | Audition - Quality assurance process improvement - Maturity level 5/5					
QAP | Bug management - Tracking, Root cause and documented by tests		Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	CYCLE DE VIE (GESTION) DES ANOMALIES
					Bug tracking
					Bug distribution (dispatch)
QAP | Bug management - Maturity level 2/5					
QAP | Bug					

management - Maturity level 3/5					
QAP | Bug management - Maturity level 4/5					
QAP | Bug management - Maturity level 5/5					
QAP | Qualimetry - Measurement reference	IN PROGRESS	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	Établir les métriques et les objectifs de la Qualimétrie
					Monitorer les exécutions
					Monitoring
					Traçabilité, journalisation
					Test monitoring (Grafana) - First board + at less 1 notification alert
QAP | Qualimetry - Maturity level 2/5					
QAP | Qualimetry - Maturity level 3/5					
QAP | Qualimetry - Maturity level 4/5					
QAP | Qualimetry - Maturity level 5/5					
QAP | Test management - Maturity level 1/5	IN PROGRESS	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	Establish test management conventions
					Reduce, privatize, stabilize and define test types for all test environments (priority) (gain confidence in what has already been tested and avoid over-testing)
					Write all non-regression tests (manual) with expected results
					Workshop - TMMI
					Workshop - Test plan & documented story/bug by test
					Workshop - Test control

					Workshop - Test monitoring
					Workshop - Test environment
					Workshop - Test management (testing strategy & policy)
					Workshop - Test documentation
					Workshop - Testing terminology
					Renommer les tests de la non-régression avec la nomenclature de test selon la politique. (incluent les résultats attendus) TMO Content Suscription
					Écrire les tests sanitaires prioritaires avec la nomenclature de test selon la politique. (incluent les résultats attendus) Voici les validations que je dois faire avant de commencer les tests “QA” API (ON) Le site et sa navigation (main) est accessible Login(s) (je peux me connecter)
					Automatiser les tests sanitaires (one shot)
					Automatiser progressivement les tests de la non-régression (couverture global)
					Établir la procédure/plan, pour automatiser les tests de la non-régression pendant l’implémentation des stories (automatisation en just in time) cela fait partie du CT (continous testing)
					Atelier du Shift-left → Les bases avec les 3 amigos et les plans de test Raffiner les stories et les bugs avec les trois amigos avant de planifier et s’engager dans les sprints. (Jira+GoTest) → ce avant de pointer Planifier la qualité (Plan de test) et faire le suivi qualité des livraisons (Jira+GoTest) → ce durant le planning
					Inclure et découper les règles d’affaires comme résultats attendus dans les tests
					Mock - Implantation Mock - Premier test sanitaire API Mock - Premier test sanitaire UI Mock - Faire la liste de tous les tests sanitaires et les planifier (docu+auto)
					Faire les tests sur ACC

					Contrôle des jeux de donner avoir un client que le mot de passe ne change jamais avoir un csr qui ne change jamais
					Palier au captcha Palier au token API
					Test basé sur le modèle (Model Based Testing)
					Collaboration et partage des tests GoTest avec SN Impliquer le front-end (UI-UX) dans la loop QA (Tester les components Bifrost (manuels et automatisés)) Impliquer DevOps dans la loop QA Impliquer le Dev dans la loop QA Impliquer l'analyste dans la loop QA Impliquer le PO dans la loop QA Impliquer SN dans la loop QA Impliquer ETIYA et BSI dans la loop QA
					Les 3 principes des fondation de gestion de test Le 1er principe : Respect du cycle complet de la vie du développement Le 2eme principe : Optimiser la quantité (éliminer le superflu) Le 3eme principe : Hyper-automatisation
					Contrôle sur les jeux de données
					Écrire une story incluant l'assurance qualité Écrire un bug/anomalie incluant l'assurance qualité (RCA+bug tracking)
					Environnements de test stable et sur demande
					Documentation vivante et Documentation intelligente
					Transparence sur les résultats
					Cycle de vie livraison Post mortem livraison
					Avoir le contrôle de nos data de test / Mock
					Moins de promotions d'environnements de tests
					Conception/documentation de test Gestion des données
					Environnements de test Planification et Organisation (plan de test)
					Exécution de test

					GoTest | Exécuter un test API ETIYA
					GoTest | Exécuter un test API BSI
					GoTest | Écrire tous les tests de non-régression (manuel), documenter les tests manuels est prioritaires (l'automatisation viendra par priorité)
QAP | Test management - Maturity level 2/5	TODO	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	
QAP | Test management - Maturity level 3/5	TODO	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	
QAP | Test management - Maturity level 4/5	TODO	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	
QAP | Test management - Maturity level 5/5	TODO	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	
Terminology -					

Maturity level 1/5					
Terminology - Maturity level 2/5					
Terminology - Maturity level 3/5					
Terminology - Maturity level 4/5					
Terminology - Maturity level 5/5					
Roles and responsibilities - Maturity level 1/5					
Roles and responsibilities - Maturity level 2/5					
Roles and responsibilities - Maturity level 3/5					
Roles and responsibilities - Maturity level 4/5					
Roles and responsibilities - Maturity level 5/5					
Planification - Maturity level 1/5					
Planification - Maturity level 2/5					
Planification - Maturity level 3/5					
Planification - Maturity level 4/5					
Planification - Maturity level 5/5					
QAP | Tool box - Basics	IN PROGRE	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	GoTest | Configurer les SUT GoTest pour accéder aux environnements de

	SS				test de Vidéotron
					Demande de GoTest
					Stabiliser les exécutions par les plans de test
					Formations et ateliers GoTest
					Écrire les conventions de structure et de nomenclature et les partager GoTest
					Coffre d'outils
					Correction, Prevention, Réaction
					Gestion de changement Gestion de risque Auto audition/évaluation
					Politiques et Processus
					Rôles et responsabilités
					Terminology
					Formations/ateliers Qu'est-ce qu'un Coach QA ? Shift-left & Shift-Right 3 amigos & ATDD, TDD et BDD Stratégie & Politique RCA Bien documenter un bug ou un récit lié à des tests QA Agile Qualité en continu Type de test Les niveaux de test Qualité en continu Gestion de bug Gestion de test Qualimétrie Planification L'affaire de tous ! (Rôles et responsabilités QA) Les meilleurs pratiques qualité Livrer rapidement de la qualité Les termes de la qualité Qualité Ci-Cd-Ct Qualité moderne VS old school
					~~Suivi de la non-régression dans Excel (temporairement)~~ (remplacé par GoTest)
					Prise en charge des tests Cypress dans le pipeline
					Canal de déploiement, suivi des promotions
					Santé global de la satisfactionêtre pro-actif dans nos stratégie QAAnalytics : AppDynamics
					Grafana

					Plan test : Implémenter le concept de Plan de test pour les bugs et le stories
					Donner de la visibilité de l’accomplissement du niveau de la maturité de l’assurance qualité. Le roadmap QA
					Faire un bilan de l’avancement hebdomadaire avec la feuille de route
					Créer un Epic “Implémentation/utilisation GoTest“
					GoTest | Conventions
					GoTest | Demande d’amélioration pour avoir la fonctionnalité Jira au niveau des test plans
					GoTest | Importer les tests existant dans la nouvelle instance et s’assurer qu’ils fonctionnes
					GoTest | Ajouter les utilisateurs de l'équipe (j’ai déjà ajouter Ali et Laura)
					GoTest | Voir avec David Lagacé pour que les SUTs soient sur le VPN Videotron
					GoTest | Voir avec David Lagacé pour exécuter les tests sur Browserstack
					GoTest | Error pour les exécutions des test plans
					GoTest | Valider les exécutions multiples
					GoTest | Ajouter les exécutions de nuit pour les stats de santé/performances
					GoTest | Exécuter des tests et recevoir le résultat par un url pour Appdynamic
					GoTest | Exécuter et recueillir les tests sanitaires + résultats dans les pipelines après le déploiement de l’environnement
QAP | Tool box - Maturity level 2/5					
QAP | Tool box - Maturity level 3/5					
QAP | Tool box -					

Maturity level 4/5					
QAP | Tool box - Maturity level 5/5					

Add label

suivi par semaine, par sprint, par pi

Amélioration continu

Les développeurs on dû aller faire des tâches QA pour livrer plus vite

-
- 

Owned by Michel Belanger

Last updated: Nov 01, 2023

15 min read8 people viewed

https://groupemedia.atlassian.net/browse/DWN-31620

Mise en situation/contexte : Une livraison surprise beaucoup plus grande que prévue.

Respecter l’engagement du sprint

Impact

Ne pas respecter l’engagement des sprints peut avoir un impact négatif sur la réussite du projet, la satisfaction du client et la motivation de l’équipe et même donner l’impression de ne jamais livrer dans les temps.

Risques

- Ne pas respecter l’engagement des sprints peut entraîner un retard dans la livraison du produit, une perte de confiance du client et une mauvaise réputation pour l’équipe ou l’organisation.
- Ne pas respecter l’engagement des sprints peut diminuer la qualité du produit, augmenter les risques de dysfonctionnement ou de non-conformité aux exigences, et réduire la valeur ajoutée pour le client.
- Ne pas respecter l’engagement des sprints peut affecter la collaboration au sein de l’équipe, générer du stress, de la frustration ou du découragement, et diminuer la performance et la créativité des membres.
- Ne pas respecter l’engagement des sprints peut remettre en cause la pertinence du framework Scrum, qui repose sur la transparence, l’inspection et l’adaptation, et compromettre les principes et les valeurs agiles.

Niveaux de maturité

13. **Niveau 1** : Initial. L’équipe Scrum n’a pas de méthode formelle ni de processus défini pour gérer le sprint. Le succès dépend essentiellement des efforts individuels et du talent des membres. L’équipe ne respecte pas toujours l’engagement du sprint, car elle n’est pas capable de planifier, d’estimer, de prioriser, de communiquer ou de s’adapter efficacement.
14. **Niveau 2** : Reproductible. L’équipe Scrum a un consensus sur la méthode et le processus à

suivre pour gérer le sprint. Un management de projet est mis en place, fondé sur la réussite de projets antérieurs. L'équipe respecte généralement l'engagement du sprint, car elle est capable de stabiliser le développement et de contrôler les risques.

15. **Niveau 3** : Défini. L'équipe Scrum a formalisé, documenté et standardisé le processus de gestion du sprint. Une structure Qualité & Méthodes est responsable de mettre à jour et d'améliorer le processus. L'équipe respecte systématiquement l'engagement du sprint, car elle est capable de renforcer la collaboration interne et externe, et de solliciter le feedback des parties prenantes.
16. **Niveau 4** : Géré. L'équipe Scrum a institué un processus formel de mesure et d'analyse du processus de gestion du sprint. Des indicateurs clés sont utilisés pour suivre l'avancement, la performance et la qualité du sprint. L'équipe respecte efficacement l'engagement du sprint, car elle est capable d'utiliser les données pour prendre des décisions éclairées et optimiser le processus.
17. **Niveau 5** : Optimisé. L'équipe Scrum a atteint un niveau d'excellence dans la gestion du sprint. Elle exploite les opportunités offertes par les risques pour innover et se différencier. Elle crée de la valeur ajoutée pour le produit et le client. Elle cherche à s'améliorer continuellement en se basant sur les indicateurs et les retours d'expérience.

Avantages

- Respecter l'engagement du sprint permet de livrer un incrément de produit potentiellement livrable à la fin du sprint, en respectant l'objectif du sprint et le backlog du sprint. Cela contribue à la satisfaction du client, qui peut voir les progrès réalisés et donner son feedback.
- Respecter l'engagement du sprint permet de gérer plus facilement les projets, en décomposant les grands projets complexes en petites tâches. Cela permet à l'équipe de se concentrer sur les priorités, de contrôler les risques et de s'adapter aux changements.
- Respecter l'engagement du sprint permet d'améliorer la qualité du produit, en appliquant les principes et les valeurs agiles. Cela implique de privilégier la collaboration, la transparence, l'inspection et l'adaptation, ainsi que la recherche de l'excellence technique.
- Respecter l'engagement du sprint permet de renforcer la motivation de l'équipe, en créant un sentiment de responsabilité collective et de confiance mutuelle. Cela favorise la performance, la créativité et l'innovation de l'équipe.
- Respecter l'engagement du sprint permet de créer un rythme de travail soutenable, qui évite le surmenage et le burnout des membres de l'équipe.
- Respecter l'engagement du sprint permet d'apporter de la valeur au client, en livrant des fonctionnalités utiles et pertinentes, qui répondent à ses besoins et à ses attentes.
- Respecter l'engagement du sprint permet de travailler avec un cadre léger, qui ne nécessite pas de documentation excessive ni de processus rigides, mais qui favorise la simplicité et la flexibilité.
- Respecter l'engagement du sprint permet de se focaliser sur la qualité, plutôt que sur la quantité, en s'assurant que chaque livrable respecte les normes de qualité définies par l'équipe et le client.
- Respecter l'engagement du sprint permet d'améliorer la prédictibilité de l'équipe, en utilisant des indicateurs clés pour mesurer l'avancement, la performance et la satisfaction du client.
- Respecter l'engagement du sprint permet de favoriser le feedback, en sollicitant régulièrement l'avis du client et des parties prenantes, et en intégrant leurs retours dans les améliorations du produit.
- Respecter l'engagement du sprint permet de raccourcir le cycle d'apprentissage, en expérimentant rapidement des solutions et en validant leur efficacité auprès des utilisateurs.
- Respecter l'engagement du sprint permet de développer l'empirisme, en basant les décisions sur les données réelles et les faits observés, plutôt que sur les hypothèses ou les

opinions.

État actuel

Constats et observations

Drupal n’a jusqu’à présent réussi à respecter aucun engagement de sprint.

Recommandation

Solutions	**Terme**
Lors d’une “Mise à jour Drupal” ou d’un gros engagement, si on ne peut pas livrer en petites parties ou que cela prend plus d’un sprint… Il faut faire un engagement de 6 semaines et plus. non-pas 3 semaines. Sinon pas le choix, il faut trouver le moyen de découper pour de petites livraisons	Immédiat
Il faut prendre en considération la situation actuel du présent sans les amélioration du future ! La limite de la capacité est limité par le chainon le plus faible ! Soit qu’on fait 1:1 pour DEV et QA, donc si cela prend 3 jours pour un DEV cela va aussi prendre 3 jours pour le QA ou il faudrait réduire la capacité de moitié de ce qu’on peut faire ou pointer le point plus haut si 3 alors 5 si 5 alors 8 si 8 alors 13… ou vraiment se limiter à la capacité du QA, ne pas dépasser la capacité possible du QA ou le plus petit chaînon !	Immédiat
Rencontre des 3 amigos, cela permettra de mieux pointer et avoir une meilleur clarté sur la capacité	Moyen, car il y a de la dette à rattraper

Évaluation

Maturité : 1/5
Priorité : Haute
Effort : Haute

Livrer plus vite

Impact

L’impact de livrer rapidement dans le domaine du logiciel est un sujet qui intéresse de nombreux acteurs du secteur informatique, tels que les développeurs, les clients et les utilisateurs. Livrer rapidement signifie être capable de fournir un produit logiciel aux clients dans un délai court, en respectant les exigences de qualité et de performance. Voici quelques éléments de réponse que j’ai trouvés en utilisant ma recherche web :

Risques

- Le risque de perdre des opportunités de marché, en étant devancé par la concurrence ou en ne répondant pas aux besoins du marché. Cela peut entraîner une perte de revenus, de parts de marché ou de réputation pour le site e-commerce.
- Le risque de décevoir les clients, en ne leur fournissant pas les fonctionnalités les plus récentes et les plus utiles du logiciel. Cela peut entraîner une perte de fidélité, de feedback ou d’implication des clients dans le processus de développement.
- Le risque de compromettre la qualité du logiciel, en accumulant les bugs, les erreurs ou les

anomalies qui peuvent affecter la fiabilité, la sécurité ou la performance du logiciel. Cela peut entraîner des coûts supplémentaires, des sanctions financières ou des dommages pour le site e-commerce.

- Le risque de démoraliser les développeurs, en leur imposant un rythme de travail insoutenable ou en ne reconnaissant pas leur travail. Cela peut entraîner une baisse de la créativité, de l'innovation ou de l'amélioration continue des développeurs.

Il est important de ne pas ajouter de nouvelles ressources avant d'avoir documenter les tests, sinon il y aura un effet contraire aux attentes.
Sauf si les ressources sont là pour ajouter de la documentation de test



Il faut juste se dire que ce n'est pas une solution à court terme, qu'on ne livrera pas plus vite

L'ajout de plusieurs QA (Quality Assurance) n'est pas forcément la meilleure solution pour améliorer la qualité et la rapidité de livraison d'un logiciel. En effet, il existe d'autres facteurs qui peuvent influencer l'efficacité et l'efficience du processus de test, tels que :

- L'organisation de l'équipe QA : il est important de définir clairement le rôle, les responsabilités et les interactions de chaque membre de l'équipe QA, ainsi que les relations avec les autres équipes du projet (développeurs, product owners, clients, etc.). Il faut également favoriser la collaboration, la communication et le feedback au sein de l'équipe QA et avec les autres parties prenantes.
- La méthodologie de test : il est essentiel de choisir une méthode de test adaptée au contexte agile du projet, qui permette de tester le produit de manière continue, itérative et incrémentale. Il faut également définir des critères de qualité clairs et mesurables, ainsi que des indicateurs de suivi et de reporting.
- Les outils et les techniques de test : il est important de sélectionner les outils et les techniques de test les plus appropriés au produit, à son niveau de complexité, à ses exigences fonctionnelles et non fonctionnelles, et à ses utilisateurs cibles. Il faut également veiller à optimiser l'utilisation des outils et des techniques de test, en évitant par exemple les doublons, les tests inutiles ou les tests trop longs.
- La formation et la compétence des testeurs : il est indispensable de s'assurer que les testeurs disposent des connaissances et des compétences nécessaires pour réaliser les tests de manière efficace et efficiente. Il faut également leur fournir des formations régulières pour maintenir et améliorer leurs compétences, ainsi que pour les sensibiliser aux bonnes pratiques et aux évolutions du domaine du test.
- La culture et la motivation de l'équipe : il est important de créer un climat de confiance, de respect et de reconnaissance au sein de l'équipe QA, ainsi qu'avec les autres équipes du projet. Il faut également encourager l'initiative, la créativité et l'amélioration continue des testeurs.
- La gestion des risques et des incidents : il est essentiel d'identifier, d'analyser et de traiter les risques potentiels qui peuvent affecter la qualité du produit ou la performance du processus de test. Il faut également mettre en place des mécanismes de suivi, d'alerte et de résolution des incidents qui surviennent pendant le test.
- L'optimisation des ressources et des coûts : il est important de planifier, d'allouer et de contrôler les ressources humaines, matérielles et financières nécessaires pour réaliser les tests. Il faut également chercher à réduire les coûts liés au test, en évitant par exemple le gaspillage, la duplication ou la sous-utilisation des ressources.
- etc.
  Niveaux de maturité

18. **Niveau 1 : Chaotique**. Il n'y a pas de processus clair ni de méthode agile. Les livraisons sont

irrégulières, imprévisibles et non testées. Les équipes sont isolées et ne communiquent pas entre elles. Les clients sont insatisfaits et les utilisateurs sont frustrés.

19. **Niveau 2 : Réactif**. Il y a une prise de conscience de la nécessité d'un processus agile et DevOps, mais il n'est pas formalisé ni partagé. Les livraisons sont plus fréquentes, mais pas toujours fiables. Les équipes commencent à collaborer, mais il y a encore des silos et des conflits. Les clients sont impliqués, mais pas toujours écoutés et nous sommes en mode contrôle de la qualité plutôt que prévention de la qualité.
20. **Niveau 3 : Proactif**. Il y a un processus agile et DevOps défini, documenté et appliqué avec rigueur. Les livraisons sont rapides, régulières et testées. Les équipes travaillent ensemble, avec des rôles et des responsabilités clairs. Les clients sont co-créateurs du produit et les utilisateurs sont satisfaits.
21. **Niveau 4 : Performant**. Il y a un processus agile et DevOps mesuré, analysé et amélioré en continu. Les livraisons sont optimisées, adaptées et sécurisées. Les équipes sont autonomes, auto-organisées et apprenantes. Les clients sont fidélisés et les utilisateurs sont enchantés.
22. **Niveau 5 : Innovant**. Il y a un processus agile et DevOps innovant, qui anticipe les besoins du marché et crée de la valeur ajoutée. Les livraisons sont disruptives, différenciantes et personnalisées. Les équipes sont visionnaires, créatives et inspirantes. Les clients sont ambassadeurs et les utilisateurs sont fans.

Avantages

- **Satisfaire le client**. En livrant des fonctionnalités à grande valeur ajoutée de manière fréquente et régulière, on répond mieux aux besoins et aux attentes du client, qui peut donner son feedback et voir son produit évoluer.
- **Réduire les risques**. En livrant des petits lots de logiciel testés et validés, on minimise les erreurs, les bugs et les retards qui peuvent compromettre la qualité du produit final.
- **Gagner en flexibilité**. En livrant des petits blocs fonctionnels indépendants, on peut facilement adapter le produit aux changements du marché, des technologies ou des exigences du client.
- **Optimiser les coûts**. En livrant plus petit et rapidement, on réduit les coûts de développement, de maintenance et de correction du logiciel, tout en augmentant sa valeur ajoutée.
- **Innover davantage**. En livrant plus petit et rapidement, on favorise la créativité, l'apprentissage et l'amélioration continue des équipes de développement, qui peuvent expérimenter de nouvelles solutions et proposer des fonctionnalités différenciantes.
- **Améliorer la satisfaction des utilisateurs**. En livrant plus petit et rapidement, on peut offrir aux utilisateurs des fonctionnalités utiles et pertinentes, qui répondent à leurs besoins et leurs attentes. On peut également recueillir leur feedback et leurs suggestions, ce qui permet de les fidéliser et de les impliquer dans le processus de développement.
- **Faciliter la maintenance et l'évolution du logiciel**. En livrant plus petit et rapidement, on peut réduire la complexité du code et la dette technique, ce qui facilite la correction des bugs, l'ajout de nouvelles fonctionnalités et l'adaptation aux changements technologiques.
- **Renforcer la confiance et la collaboration des équipes**. En livrant plus petit et rapidement, on peut favoriser la communication, la coordination et l'alignement des équipes de développement, de test, d'exploitation et de gestion de projet. On peut également valoriser le travail accompli, reconnaître les efforts fournis et célébrer les succès obtenus.
- Livrer rapidement permet de réduire le time-to-market, c'est-à-dire le délai entre la conception et la mise sur le marché d'un logiciel. Cela permet de devancer la concurrence, de répondre aux besoins du marché et de générer des revenus plus rapidement.
- Livrer rapidement permet d'augmenter la satisfaction des clients, qui peuvent bénéficier des fonctionnalités les plus récentes et les plus utiles du logiciel. Cela permet de fidéliser les clients, de recueillir leurs feedbacks et de les impliquer dans le processus de développement.
- Livrer rapidement permet d'améliorer la qualité du logiciel, en évitant les bugs, les erreurs

ou les anomalies qui peuvent s'accumuler lors d'un développement trop long. Cela permet de garantir la fiabilité, la sécurité et la performance du logiciel.

- Livrer rapidement permet de renforcer la motivation des développeurs, qui peuvent voir le résultat de leur travail plus rapidement et plus fréquemment. Cela favorise la créativité, l'innovation et l'amélioration continue des développeurs.

État actuel

Constats et observations

Il y a trop d'environnements de test (Drupal et autoroutes) partagés et instables. Il y a du sur-testage répétition des mêmes tests sur plusieurs environnements et il n'y a pas de documentation de test pour réutiliser les mêmes tests en exécution et savoir quoi tester, comment et laisser des traces… Il manque aussi de l'automatisation.

Recommandation

Pour livrer rapidement dans le domaine du logiciel, il est recommandé de suivre les meilleures pratiques, telles que :

- Recruter les meilleurs dans le domaine
- Fournir les bons outils et équipements de livraison de logiciels
- Formuler des objectifs et des attentes clairs
- Choisir les méthodes de livraison de logiciels qui conviennent le mieux au projet
- Tester rigoureusement le logiciel
- Célébrer les étapes importantes
- Être flexible et prévoir le changement
- Envisager la gestion des livraisons de logiciels

Solutions	Terme

Solutions	Terme
Mettre en place l'amélioration continu (Audit : rapport et plan d'action)	Immédiat
Avoir juste comme promotions… PREV et ACC pour aller en PROD (blue/green) Et ce en privé/exclusif à Drupal seulement. Je parle d'avoir des autoroutes privées aussi C'est un MUST que d'avoir des environnements de développement, de test et de déploiement contrôlés, stables, sécurisés et efficaces. Les environnements de test (dev) doivent êtres aussi sinon plus stable que la prod ! Ne pas avoir de downtime de paperasse pour les promotions.	Court/Moyen
Établir une confiance de test… cibler quoi tester sur quel environnement et ne pas retester ou sur-tester	Court
Faire du pair testing (QA+DEV)	Immédiat
Valider avec les experts dans le domaine (définir l'expert en amont et valider le besoin le plus tôt possible)	Immédiat
Découper en plus petites livraisons	Immédiat
Impliquez le QA dès le début du projet. Il est important que le QA participe à la définition des besoins, des spécifications et des critères d'acceptation du produit. Cela permet de s'assurer que le produit répond aux attentes du client et de prévenir les défauts en amont.	Immédiat
Faire les tests de stories et bugs seulement en paring (QA) ou pas sur PREV et/ou DEV4 valider par un autre dev que celui qui a fait l'implémentation Cela doit inclure les preuves des résultats. Éviter de refaire les mêmes tests dans les autres env. QA ne test plus sur DEV4 sauf paring au besoin et création/ajustement des tests auto	Immédiat

Les dev prépare la livraison (version) et la pousse en ACC pour le QA On met les stories/bugs de la même version (RC) dans la colonne QA (ne test pas les stories/bugs sur cet environement) Les QA feront les smokes et ensuite la régression en vue de créer la couverture de la release	
Faire les tests de non-régression sur ACC et/ou ACC4	Immédiat
Augmenter la documentation de tests au fur et à mesure Avec la documentation tout l'monde pourra exécuter des tests manuels, pas juste Ali.	Court/Long terme
Augmenter les tests automatisés L'automatisation des tests est une pratique qui consiste à utiliser des outils logiciels pour exécuter des tests sur un logiciel en développement, afin de vérifier sa qualité, sa fonctionnalité et sa performance. L'automatisation des tests présente de nombreux avantages, tels que : • La réduction du temps et des coûts de test • La détection rapide des erreurs et des bugs • La fiabilité et la répétabilité des tests • La couverture étendue des tests • La facilitation de la livraison continue Cependant, l'automatisation des tests n'est pas une solution magique et miraculeuse. Elle comporte également des défis et des limites, tels que : • Le choix des outils et des techniques de test adaptés au projet. Il existe plusieurs types de frameworks de tests automatiques, tels que les frameworks basés sur les mots-clés, les frameworks basés sur les données, les frameworks hybrides, etc. Il faut choisir celui qui correspond le mieux au contexte, aux ressources et aux attentes du projet. • Le coût et temps initial élevé de la mise en place de l'automatisation. Il faut investir du temps et de l'argent pour développer, configurer et maintenir les scripts de test, ainsi que pour former les testeurs à l'utilisation des outils d'automatisation. Il faut également calculer le retour sur investissement de l'automatisation et s'assurer qu'elle soit rentable à long terme. • La maintenance et la mise à jour des scripts de test pour garder une pérennité. Il faut veiller à ce que les scripts de test soient robustes, modulaires et réutilisables, afin qu'ils puissent s'adapter aux changements du logiciel ou de l'environnement de test. Il faut également éviter les tests redondants, obsolètes ou inutiles. • La difficulté à tester certaines fonctionnalités ou certains scénarios complexes. Il n'est pas toujours possible ni pertinent d'automatiser tous les types de tests, tels que les tests d'interface utilisateur, les tests d'expérience utilisateur, les tests exploratoires, etc. Il faut donc compléter les tests automatisés par des tests manuels pour couvrir tous les aspects du logiciel. • La nécessité de compléter les tests automatisés par des tests manuels. L'automatisation des tests ne remplace pas le rôle du testeur humain, qui reste indispensable pour évaluer la qualité globale du logiciel, ainsi que pour apporter son expertise, sa créativité et son jugement critique. Il faut donc trouver le bon équilibre entre l'automatisation et le manuel. • L'automatisation des tests sans stratégie ni planification. Il ne suffit pas de choisir un outil et de commencer à automatiser les tests au hasard. Il faut définir une stratégie d'automatisation qui réponde aux objectifs du projet, aux besoins du client et aux contraintes techniques. Il faut également planifier les activités d'automatisation, en déterminant le périmètre, les priorités, les ressources, les délais et les indicateurs de suivi. • L'automatisation des tests sans documentation ni normes. Il est important de documenter les tests automatisés, en indiquant leur but, leur contenu, leur résultat attendu et leur mode d'exécution. Il faut également respecter des normes de qualité et de lisibilité pour le code des scripts de test, en utilisant des commentaires, des noms explicites, des structures modulaires et des bonnes pratiques de programmation. • L'automatisation des tests sans validation ni revue. Il est essentiel de valider les tests automatisés, en vérifiant qu'ils fonctionnent correctement, qu'ils produisent les	Court/Long terme

résultats attendus et qu'ils couvrent les exigences du produit. Il faut également effectuer des revues de code et des audits de qualité pour s'assurer que les tests automatisés respectent les normes et les critères de qualité définis. • L'automatisation des tests sans analyse ni conception préalables. Il ne suffit pas de reproduire les tests manuels existants en les automatisant. Il faut analyser les besoins et les exigences du produit, ainsi que les risques et les priorités associés. Il faut également concevoir des cas de test adaptés à l'automatisation, en utilisant des techniques comme les user stories, les scénarios ou les modèles. • L'automatisation des tests sans intégration ni collaboration. Il est important de synchroniser les tests automatisés avec le cycle de développement du logiciel, en utilisant des outils comme les systèmes de contrôle de version, les serveurs d'intégration continue ou les plateformes de test. Il faut également favoriser la collaboration entre les testeurs, les développeurs et les autres parties prenantes du projet, en utilisant des méthodes comme l'agilité ou le DevOps. • L'automatisation des tests sans mesure ni amélioration. Il est essentiel de mesurer l'efficacité et l'efficience des tests automatisés, en utilisant des indicateurs comme la couverture des tests, le taux de détection des défauts, le temps d'exécution des tests ou le coût du test. Il faut également chercher à améliorer en permanence la qualité et la performance des tests automatisés, en utilisant des techniques comme les revues de code, les audits de qualité ou les retours d'expérience . • Besoin d'opérateur et d'investigation constante • Ne peut pas être fait sur des environnements instables • Demande un contrôle sur les mocks et les jeux de données de test • Demande une expertise et/ou spécialité dans le domaine • etc. Il n'existe pas de formule universelle pour réussir l'automatisation des tests. Chaque projet a ses propres besoins, contraintes et objectifs. Il faut donc adopter une stratégie d'automatisation qui soit adaptée au contexte, aux ressources et aux attentes du projet.	
Suivre les 3 principes de la gestion de test (requiert de la maturité) • Priorisez les tests les plus importants et les plus critiques. Il n'est pas possible ni nécessaire de tester toutes les fonctionnalités et tous les scénarios possibles. Vous devez déterminer les tests qui ont le plus d'impact sur la qualité du produit, en tenant compte des risques, des coûts et des bénéfices. •	Long
Intégrer les meilleures pratiques de la gestion de test (requiert de la maturité) ex. Formulez des objectifs et des attentes clairs. Il est essentiel de définir le périmètre, le budget, le calendrier et les critères de qualité de votre projet. Vous devez également communiquer régulièrement avec vos clients et vos parties prenantes pour recueillir leurs besoins, leurs feedbacks et leurs validations. ex. mettre en place des processus d'assurance qualité, d'intégration continue et de déploiement continu pour détecter et corriger les erreurs le plus tôt possible. ex. CT continous testing	Long
cycle de vie DevOps	Long

Évaluation

Maturité : 2/5
Priorité : Haute
Effort : Haute

PRG-180801 - Darwin ProgramContinuous improvement / Amélioration continueEnvironnements de test

Share

Environnements de test



Owned by Michel Belanger

Last updated: Nov 07, 2023

2 min read4 people viewed

Need : Définition (Analyse+Schems) des environnements de test pour le cycle de vie du dev

Success Acceptance Criteria :

- Environnements de tests réservé qu'à Drupal pour les promotions de livraison

Recommandation

Le tableau du flux des promotions pour une livraison rapide et sans embûche :

	PREV	**ACC**	**PROD (blue/green)**
Blue/Green	oui (?)	oui	oui
Mock Jeux de données de test	• Les APIs sont mock-és • **Mêmes données de test** pour les 3 environnements Pourra aussi se connecter, avec une switch, à l'API de ACC au besoin spécifique. Il sera important de ne pas utiliser si c'est pour expérimenter avec les APIs, au risque de déstabiliser/compromettre l'environnement ACC.	• Pas mock-é • **Mêmes données de test** pour les 3 environnements	• Pas mock-é • **Mêmes données de test** pour les 3 environnements
Type de test	• Acceptance • Accessibilité • Analytique • Compatibility • Performance • Non-régression (fonctionnalité) • Résilience • Sanitaire • Security • Smoke	• Sanitaire • Intégrité du déploiement • Test contrat • Test API • Test de communication • Test d'interaction • Performance	• Sanitaire • Intégrité du déploiement • Analytiques • Statistiques • Statistique • Analytique

	• Statistique • Usabilité	• Analytique • Concurrence-collision • Non-régression (système) • Résilience • Security • Smoke • Statistique	
DoD	Pour passé en ACC Ici on test la branche (story/bug) On doit packager en une livraison (version)	Pour passé en PROD La non-régression	Pour se dire que c'est Done
Réservé à	L'équipe Drupal et les tests auto Personne d'autre que Drupal peut faire des changements Tout changement doit être avertis	L'équipe Drupal et les tests auto Personne d'autre que Drupal peut faire des changements Tout changement doit être avertis	pas réservé
	Peut aussi servir de bac à sable pour expérimenter		
Validation en amont	UAT (Saliha, Paul, SN…)		
	Automatisation des tests		

Important !

BSI, ETYIA ou autres ne doivent en aucun cas modifier ou altérer nos environnement sans une demande venant de nous.

Donc pas d'update de personne sans la demande ou consentement préalable à Drupal.

Contrôle de version

Il sera primordiale pour nous de s'assurer d'être à jour sur les mêmes versions de code/APIs que le code/APIs de PROD avant de promouvoir le tout en PROD. C'est le même concept que du merge/rebase de branche.

Qu'arrivera t-il avec DEV4, ACC4, PPX, STAGING ou tout autres environnements ?

- Ce sera des environnements bac-à-sable partagés ou de promotion BSI ou ETIYA… surtout externes aux promotions de la livraison Drupal.
- Tous ces environnements **ne seront plus utilisé et/ou testé** par Drupal pour promotion en prod.
- Si il y a des bugs dans ces environnements, voir d'abord si c'est un réel bug (investigation comme un bug de prod) et valider si reproductible en PROD, PREV et ACC… Donc, si il y a un bug, ok, on corrige sur PREV, ACC, PROD (une mise à jour sera disponible pour les autres environnements). Pour bien comprendre, on ne gèrera plus le suivi et la correction des "anomalies" des environnements (partagés).
- Certainement on peut venir en aide, mais ce n'est pas notre responsabilité.
- La stabilité de ces environnements **n'est pas du ressort de Drupal**, mais bien des équipes qui utilisent les environnements ou par les DevOps.
- Faire les mises à jours (tigliv etc.), les backup, les resets, les configurations, etc. de ces

environnements ne feront pas partie des responsabilités/tâches de Drupal, mais bien des équipes qui utilisent les environnements ou par les DevOps.

GRAPHIQUE

https://videotron.sharepoint.com/:u:/r/sites/Web2/Documents%20partages/General/Ci-Ct-Cd.vsdx?d=w4f725b76683249e2a97ed273832c8590&csf=1&web=1&e=0Aiiay

PRG-180801 - Darwin ProgramContinuous improvement / Amélioration continueQuoi faire quand Ali (QA) n'est pas disponible/malade/vacance... ?

Share

Quoi faire quand Ali (QA) n'est pas disponible/malade/vacance... ?



Owned by Michel Belanger

Last updated: yesterday at 4:42 PM

2 min read4 people viewed

Premièrement, ne pas paniquer

Impact

L'impact si le QA n'est pas disponible, malade ou en vacances dans le contexte logiciel et que vous n'avez pas planifié un plan B peut être important pour la qualité et la livraison de votre produit. Voici quelques conséquences possibles :

- Vous risquez de retarder le développement et la mise en production de votre produit, car vous n'aurez pas de feedback rapide et fiable sur les fonctionnalités et les bugs.
- Vous risquez de compromettre la satisfaction et la fidélisation de vos clients, car vous leur livrerez un produit qui ne répond pas à leurs besoins ou qui contient des erreurs.
- Vous risquez de nuire à la collaboration et à la confiance au sein de votre équipe Agile, car vous devrez assumer des tâches qui ne sont pas de votre ressort ou qui dépassent vos capacités.

État actuel

Présentement tout repose sur une personne et si cette personne n'est pas là, c'est la panique général ! il n'y a pas de plan B :

- la livraison est bloquée
- personne pour dispatcher les anomalies
-

Recommandation

Il est donc recommandé de prévoir un plan B pour pallier l'absence du QA, en utilisant ces suggestions.

Vous pouvez essayer de répartir les tâches de QA entre les autres membres de l'équipe, en fonction de leurs compétences et de leur disponibilité. Par exemple, vous pouvez demander à un développeur de tester le code d'un autre développeur, ou à un analyste d'affaires de vérifier les exigences du client. Cela peut vous aider à maintenir la qualité du produit et à respecter les délais.

Vous pouvez également faire appel à un QA externe, soit en embauchant un consultant, soit en sous-traitant à une entreprise spécialisée. Cela peut vous permettre de bénéficier d'une expertise supplémentaire et d'une perspective extérieure. Toutefois, vous devrez vous assurer que le QA externe soit bien intégré à l'équipe et qu'il respecte les principes et les pratiques de la méthode.

Vous pouvez aussi profiter de l'absence du QA pour améliorer vos pratiques de développement, en adoptant des techniques telles que le développement piloté par les tests (TDD), l'intégration continue, le déploiement continu, le code review, Shift-left, etc. Cela peut vous aider à réduire les erreurs, à augmenter la couverture des tests et à accélérer le processus de livraison.

Points d'actions

23. Passer la connaissance de @Ali Habach (Unlicensed) à @Rym Ghariani (Unlicensed) dans le but que Rym puisse s'occuper de la qualité de la prochaine release (.16 sera du shadowing et la .17 sera à RYM)
24. Passer la connaissance de @Ali Habach (Unlicensed) à @Rym Ghariani (Unlicensed) dans le but que Rym puisse s'occuper de la gestion des anomalies
25. …

Évaluation

0/5

Automated test priority

-
- 

Owned by Michel Belanger

Last updated: Nov 07, 2023

1 min read3 people viewed

Rencontre du 16-oct-2023 @Ghariani Rym @Michel Belanger **et Ali**

Recontre du 18-10-2023

AUTOMATISATION :

26. LANDING PAGE (@Rym Ghariani)
27. EN/FR

28. Placeholders
29. Header/Footer
30. PREVIEW (URL Coming from DEV en liant Jenkins avec GoTest) – **DEMANDE AU DEVOPS (@alexandre.rosenblum@alithya.com)**
31. AUT10 (DEV4)
32. AUT11 (ACC4)
33. Client Login
- EN/FR
- Placeholders
- Header/Footer
- PREVIEW (URL Coming from DEV en liant Jenkins avec GoTest) – **DEMANDE AU DEVOPS (@alexandre.rosenblum@alithya.com)**
- AUT10 (DEV4)
- AUT11 (ACC4)

34. Client Login
    - Connexion
    - Section Helix
    - Entrer crédentiels
    - PREVIEW (URL Coming from DEV en liant Jenkins avec GoTest)
    - AUT10 (DEV4)
    - AUT11 (ACC4)
35. API UP (

    @Rym Ghariani + @alexandre.rosenblum@alithya.com + @David Pichot)
    - Les API répondent
    - AUT10 (DEV4)
    - AUT11 (ACC4)
36. CHECK-e (

    @Rym Ghariani + @Michel Belanger)
    - Différents Adresses
    - Configurateur/Simulateur
    - Client/Agent/Anonyme
    - AUT10 (DEV4)
    - AUT11 (ACC4)
37. TMO Fake Tunnel (

    @Rym Ghariani + @Michel Belanger)
    - Achat plan
    - Achat plan+telephone
    - Différents options du formulaire
    - Edit/Supprimer
    - PREVIEW (URL Coming from DEV en liant Jenkins avec GoTest)
    - AUT10 (DEV4)
    - AUT11 (ACC4)

PRG-180801 - Darwin ProgramContinuous improvement / Amélioration continueRendre l'équipe autonome dans l'évolution de la maturité de la pratique de l'assurance qualité

Share

Rendre l'équipe autonome dans l'évolution de la maturité de la pratique de l'assurance qualité

-
- 

Owned by Michel Belanger

Last updated: yesterday at 4:48 PM

2 min read4 people viewed

Objectif principale : Transformation du contrôle vers la pratique de l’assurance qualité.

38. La première chose à faire pour la réussite de la prise en main de la pratique de l'assurance qualité, c'est d'avoir un bon coffre d'outils.
39. Il est important de pratiquer l’**assurance** qualité versus seulement que le **contrôle** de la qualité.
40. Pratiquer la validation au fur et à mesure par les experts dans son domaine.
41. Définir nos tactiques assurance qualité sur le "BIG picture" pas juste sur le bon de commande.
42. Avoir une visibilité sur le retour de l'investissement
43. Finalement, mettre en œuvre intelligemment avec succès la pratique de la qualité, ensemble et guidé par un Coach QA !

Points d’actions

@Michel Belanger (Deactivated) Présenter la feuille de route et les objectifs à @Rym Ghariani (Unlicensed) de toutes les sphères de la maturité (l’ensemble de la pratique). Peut-être même inclure l’aide des QA Vidéotron et les Coach QA Alithya on the side pour venir en aide à RYM.
Transformation @Rym Ghariani (Unlicensed) et @Ali Habach (Unlicensed) en QA moderne (pas contrôleur de la qualité mais bien assureur de la qualité)
Transformation pour que l'équipe soit mature dans la pratique de l'assurance qualité adaptée à ses besoins et visions Agile ci/cd/ct

Objectif secondaire (en simultané) : Augmenter la maturité de la pratique de l’assurance qualité.

todo…

- Maturity / Maturité (gestion de test)

PRG-180801 - Darwin ProgramMaturity / MaturitéTMMI

Share

TMMI

-
- 

Owned by Michel Belanger

Oct 24, 2023

7 min read6 people viewed

Selon TMMi… (**T**est **M**aturity **M**odel **I**ntegration)

Niveau 1 : Processus chaotique

Ce niveau indique que l’organisation ne suit pas toutes les bonnes pratiques de base du test. Pour passer au niveau 2, il faut mettre en place une politique et une stratégie de test, planifier, surveiller et contrôler le test, concevoir et exécuter les tests, et avoir un environnement de test stable.

--->

Niveau 2 : Processus géré

Ce niveau indique que l’organisation suit les bonnes pratiques de base du test et dispose d’un processus de test défini et géré.

- Pour passer au niveau 3, il faut mettre en place une assurance qualité indépendante, une gestion des exigences de test, une conception de test basée sur les risques, des revues techniques, des tests non fonctionnels et des tests d’intégration.

--->

Niveau 3 : Processus défini

Ce niveau indique que l’organisation suit un processus de test standardisé et documenté, adapté aux besoins spécifiques des projets.

- Pour passer au niveau 4, il faut mettre en place une gestion des défauts avancée, une mesure du test, une analyse prédictive, une amélioration continue du processus de test et un test d'acceptation.

  --->

  Niveau 4 : Processus mesuré

  Ce niveau indique que l'organisation mesure et contrôle le processus de test et ses produits, et utilise les données collectées pour gérer le risque et améliorer la qualité.

- Pour passer au niveau 5, il faut mettre en place un test d'optimisation du processus, un test d'amélioration de la qualité, un test d'ingénierie défensive et un test adaptatif.

  --->

  Niveau 5 : Processus en optimisation

  Ce niveau indique que l'organisation optimise le processus de test en fonction des objectifs d'affaires et des retours d'expérience, et utilise des techniques innovantes pour améliorer la performance du test. (exemple : Optimiser la quantité (éliminer le superflu du genre la pyramide))

  Voici les indicateurs de la maturité :

  Capacité fonctionnelle

  c'est la capacité qu'ont les fonctionnalités d'un logiciel à répondre aux exigences et besoins explicites ou implicites des usagers.

- **Aptitude** : c'est la capacité du logiciel à fournir un ensemble approprié de fonctions pour des tâches et des objectifs spécifiés.
- **Exactitude** : c'est la capacité du logiciel à fournir des résultats ou des effets corrects ou conformes aux attentes.
- **Interopérabilité** : c'est la capacité du logiciel à interagir avec un ou plusieurs systèmes spécifiés.
- **Sécurité** : c'est la capacité du logiciel à protéger les informations et les données de sorte que seules les personnes ou les systèmes autorisés puissent y avoir accès.
- **Conformité** : c'est la capacité du logiciel à adhérer aux normes, conventions ou réglementations en vigueur dans le domaine d'application.
  Facilité d'utilisation

  c'est l'effort nécessaire pour apprendre à manipuler le logiciel.

- **Compréhensibilité** : c'est la capacité du logiciel à permettre à l'utilisateur de comprendre s'il convient à ses besoins et comment il peut être utilisé pour atteindre ses objectifs.
- **Apprenabilité** : c'est la capacité du logiciel à permettre à l'utilisateur d'apprendre son application.
- **Exploitabilité** : c'est la capacité du logiciel à permettre à l'utilisateur d'utiliser ses fonctions efficacement et avec satisfaction.
- **Robustesse** : c'est la capacité du logiciel à maintenir un niveau de performance acceptable en cas d'erreurs humaines ou d'utilisation incorrecte.
- **Attractivité** : c'est la capacité du logiciel à être agréable et satisfaisant pour l'utilisateur.
  Fiabilité

  c'est la capacité d'un logiciel de rendre des résultats corrects quelles que soient les conditions d'exploitation.

- **Maturité** : c'est la capacité du logiciel à éviter les défaillances dues aux erreurs dans le logiciel.

- **Tolérance aux fautes** : c'est la capacité du logiciel à maintenir un niveau de performance acceptable en cas de pannes ou d'influences externes indésirables.
- **Récupérabilité** : c'est la capacité du logiciel à rétablir son niveau de performance et à récupérer les données affectées en cas de panne ou d'erreur.
  Performance

  c'est le rapport entre la quantité de ressources utilisées (moyens matériels, temps, personnel), et la quantité de résultats délivrés.

- **Comportement dans le temps** : c'est la capacité du logiciel à fournir des temps de réponse, des temps de traitement et des délais appropriés.
- **Utilisation des ressources** : c'est la capacité du logiciel à utiliser une quantité et un type appropriés de ressources lorsqu'il exécute ses fonctions.
  Maintenabilité

  c'est l'effort nécessaire pour modifier ou corriger le logiciel.

- **Analysabilité** : c'est la facilité avec laquelle il est possible d'analyser le code source ou le comportement du logiciel pour localiser une cause donnée d'une défaillance ou d'une non-conformité.
- **Modifiabilité** : c'est la facilité avec laquelle il est possible de modifier le code source ou le comportement du logiciel sans introduire de défauts ou de dégrader les performances existantes.
- **Stabilité** : c'est la capacité du logiciel à éviter les effets inattendus dus aux modifications apportées au code source ou au comportement du logiciel.
- **Testabilité** : c'est la facilité avec laquelle il est possible de valider le code source ou le comportement modifié du logiciel par rapport aux exigences spécifiées.
  Portabilité

  c'est l'effort nécessaire pour transférer le logiciel d'un environnement à un autre.

- **Adaptabilité** : c'est la capacité du logiciel à s'adapter à des changements de spécifications ou d'environnements opérationnels sans nécessiter d'actions supplémentaires.
- **Installabilité** : c'est la facilité avec laquelle il est possible d'installer le logiciel dans un environnement spécifié.
- **Conformité** : c'est la capacité du logiciel à adhérer aux normes ou conventions relatives à la portabilité.
- **Remplaçabilité** : c'est la capacité du logiciel à être utilisé à la place d'un autre logiciel spécifié dans le même environnement et pour le même but.

Tableau complet

Niveau de maturité TMMi	Objectifs et pratiques du processus de test	Caractéristiques de qualité du produit logiciel	Indicateurs et mesures du produit logiciel	Comment faire pour parvenir au prochain niveau (+exemple)

Niveau de maturité TMMi	Objectifs et pratiques du processus de test	Caractéristiques de qualité du produit logiciel	Indicateurs et mesures du produit logiciel	Comment faire pour parvenir au prochain niveau (+exemple)
Niveau 1 : Processus chaotique	Aucun objectif ni pratique spécifique. Le test est improvisé, non planifié, non contrôlé et non	Aucune caractéristique de qualité n'est définie ni mesurée. Le produit logiciel est de qualité	Aucun indicateur ni mesure n'est utilisé. Le produit logiciel n'est pas évalué	Pour passer au niveau 2, il faut mettre en place une politique et une stratégie de test, planifier, surveiller et contrôler le test, concevoir et exécuter les tests, et avoir un environnement de test stable.

	documenté.	aléatoire et imprévisible.	selon des critères objectifs.	Pour définir une politique et une stratégie de test qui précisent les objectifs, les responsabilités, les méthodes, les outils et les normes à utiliser pour le test. (exemple : utiliser une norme pour la documentation du test, qui spécifie la forme et le contenu des documents à produire lors des différentes phases du test, tels que le plan de test, le cas de test, le rapport de test, etc.)
Niveau 2 : Processus géré	Mettre en place une politique et une stratégie de test, planifier, surveiller et contrôler le test, concevoir et exécuter les tests, et avoir un environnement de test stable.	Définir les caractéristiques de qualité requises pour le produit logiciel, en fonction des besoins des utilisateurs et des parties prenantes.	Utiliser les indicateurs internes pour mesurer la qualité du produit logiciel pendant le processus de test, en fonction des caractéristiques de qualité définies.	Pour passer au niveau 3, il faut mettre en place une assurance qualité indépendante, une gestion des exigences de test, une conception de test basée sur les risques, des revues techniques, des tests non fonctionnels et des tests d'intégration. Pour mettre en place une assurance qualité indépendante qui vérifie que le processus de test est conforme aux normes et aux exigences établies. (exemple : utiliser une norme pour l'accréditation des laboratoires de test, qui établit les exigences générales concernant la compétence technique et le management des laboratoires qui réalisent des activités d'essais ou d'étalonnage.)
Niveau 3 : Processus défini	Mettre en place une assurance qualité indépendante, une gestion des exigences de test, une conception de test basée sur les risques, des revues techniques, des tests non fonctionnels et des tests d'intégration.	Affiner les caractéristiques de qualité du produit logiciel, en tenant compte des risques, des contraintes et des priorités du projet.	Utiliser les indicateurs externes pour évaluer la qualité du produit logiciel à la fin du processus de test, en fonction des caractéristiques de qualité affinées.	Pour passer au niveau 4, il faut mettre en place une gestion des défauts avancée, une mesure du test, une analyse prédictive, une amélioration continue du processus de test et un test d'acceptation en amont. Pour mettre en place une gestion des défauts avancée qui permet de collecter, d'analyser, de résoudre et de prévenir les défauts détectés lors du processus de test. (exemple : utiliser une norme pour la classification des défauts, qui fournit un vocabulaire commun et un processus uniforme pour identifier, documenter, classer et suivre les défauts rencontrés lors du développement ou de l'utilisation d'un logiciel.)
Niveau 4 : Processus mesuré	Mettre en place une gestion des défauts avancée, une mesure du	Optimiser les caractéristiques de qualité du produit	Utiliser les indicateurs internes et externes pour	Pour passer au niveau 5, il faut mettre en place un test d'optimisation du processus, un test d'amélioration de la qualité, un test

	test, une analyse prédictive, une amélioration continue du processus de test et un test d'acceptation.	logiciel, en utilisant les données collectées lors du processus de test pour gérer le risque et améliorer la qualité.	comparer la qualité du produit logiciel avec les exigences initiales et les attentes des utilisateurs.	d'ingénierie défensive et un test adaptatif. Pour mettre en place un test d'optimisation du processus qui permet d'identifier et d'éliminer les activités inutiles ou inefficaces du processus de test. (exemple : utiliser la méthode Lean Six Sigma pour améliorer le processus de test, qui combine les principes du Lean (réduction du gaspillage) et du Six Sigma (réduction de la variabilité) pour optimiser la valeur ajoutée, la qualité et la productivité du processus de test.)
Niveau 5 : Processus en optimisation	Mettre en place un test d'optimisation du processus, un test d'amélioration de la qualité, un test d'ingénierie défensive et un test adaptatif.	Innover dans les caractéristiques de qualité du produit logiciel, en utilisant des techniques avancées pour améliorer la performance du test et la satisfaction des utilisateurs.	Utiliser les indicateurs internes et externes pour mesurer l'impact du test sur la qualité du produit logiciel et sur les objectifs d'affaires.	Mettre en place un test d'ingénierie défensive qui permet de détecter et de corriger les erreurs potentielles avant qu'elles ne deviennent des défauts. (exemple : utiliser la méthode Design by Contract pour spécifier les conditions préalables et postérieures des fonctions du logiciel, qui définissent les obligations et les bénéfices mutuels entre le fournisseur et le client d'un service logiciel, et qui permettent de vérifier que le contrat est respecté lors de l'exécution du logiciel. Ou encore optimiser la quantité en éliminant le superflu du genre la pyramide ou faire des RCA pour les bugs, pour être préventif, pro-actif, alerté et être très réactif déjà prêt à l'éventualité)

Add label

- Product health / Santé du produit
- Satisfaction surveys / Enquêtes de satisfaction

Les **enquêtes de satisfaction** permettent de recueillir le feedback des clients internes ou externes sur la qualité des processus de gestion et leur valeur ajoutée. Elles permettent d'évaluer le niveau de satisfaction, de fidélisation et de confiance des clients.

Par exemple, voici un questionnaire de satisfaction client sur le processus de gestion commerciale d'une entreprise :

todo

- Test environment stability / Stabilité de l'environnement de test
- Process mapping / Cartographie des processus

La **cartographie des processus** permet de représenter graphiquement les processus de gestion, en identifiant les entrées, les sorties, les acteurs, les responsabilités, les indicateurs et les objectifs. Elle facilite la compréhension, la communication et l'analyse des processus de gestion.

Par exemple, voici une cartographie des processus de gestion commerciale d'une entreprise :

- Key performance indicators (KPI) / Indicateurs clés de performance

  Les **indicateurs clés de performance (KPI)** permettent de mesurer quantitativement ou qualitativement la performance des processus de gestion, en comparant les résultats obtenus aux objectifs fixés. Ils peuvent être de différents types, tels que la productivité, la qualité, la satisfaction, le coût, le délai, etc. Ils doivent être pertinents, mesurables, atteignables, réalistes et temporels[3].

  Par exemple, voici quelques indicateurs clés de performance pour le processus de gestion des ressources humaines :

Indicateur	Définition	Objectif
Taux d'absentéisme	Nombre d'heures d'absence / Nombre d'heures théoriques	Réduire l'absentéisme
Taux de turnover	Nombre de départs volontaires / Effectif moyen	Réduire le turnover
Taux de satisfaction des employés	Nombre d'employés satisfaits / Nombre total d'employés	Augmenter la satisfaction des employés
Coût moyen par recrutement	Coût total du recrutement / Nombre de recrutements	Réduire le coût du recrutement
Délai moyen de recrutement	Durée moyenne entre la publication de l'offre et l'embauche du candidat	Réduire le délai de recrutement

- Benchmarks

  Les **benchmarks** permettent de comparer les processus de gestion à ceux d'autres organisations similaires ou concurrentes, afin d'identifier les meilleures pratiques, les points forts et les points faibles. Ils permettent d'évaluer le niveau de compétitivité et d'innovation des processus de gestion.

  Par exemple, voici un tableau comparatif des processus de gestion logistique entre deux entreprises du secteur du e-commerce :

Critère	Entreprise A	Entreprise B
Délai moyen de livraison	3 jours	2 jours
Taux de livraison à temps	95%	98%
Coût moyen par livraison	5€	4€
Taux de satisfaction client	90%	92%
Taux de retour produit	10%	8%

pas certain de l'avoir

- Audit interne
- Terminologie
- Rôles et responsabilités
- Coffre/boîte à outils

- Gestion de test
- Gestion de bug
- Planification
- Qualimétrie

Audit interne

Impact

- **Risques**

L'assurance qualité du logiciel est un processus qui vise à garantir que le logiciel développé répond aux exigences fonctionnelles, techniques et de performance attendues par les utilisateurs et les clients. Elle implique des activités de planification, de conception, de codage, de test, de revue, de mesure et d'amélioration continue du logiciel. Si il n'y avait pas d'audit interne de l'assurance qualité du logiciel, les risques et les impacts possibles seraient les suivants:

- Une diminution de la qualité et de la fiabilité du logiciel, qui pourrait entraîner des dysfonctionnements, des bugs, des failles de sécurité ou des non-conformités aux normes et aux réglementations.
- Une insatisfaction et une perte de confiance des utilisateurs et des clients, qui pourraient se plaindre ou se tourner vers d'autres solutions logicielles plus performantes et plus sûres.
- Une augmentation des coûts et des délais de développement, de maintenance et de correction du logiciel, qui pourrait affecter la rentabilité et la compétitivité du projet ou de l'organisation.
- Une réduction de l'efficacité et de l'innovation du logiciel, qui pourrait limiter sa capacité à répondre aux besoins évolutifs du marché et aux attentes des utilisateurs.
- Une exposition à des risques opérationnels, financiers, juridiques ou stratégiques, qui pourraient compromettre la réussite ou la pérennité du projet ou de l'organisation.

  L'audit interne de l'assurance qualité du logiciel est donc un élément essentiel pour assurer la conformité, la sécurité et la performance du logiciel, ainsi que pour renforcer la valeur ajoutée et la satisfaction des utilisateurs et des clients. Il contribue à la maîtrise des risques, au respect des obligations légales et éthiques, et à l'amélioration continue de la qualité

- **Niveaux de maturité**

  Niveau 1: Il n'y a pas d'audit défini ni documenté pour le projet logiciel. Les activités d'audit sont inexistantes ou inadaptées. L'évaluation et l'amélioration de la conformité ne sont pas réalisées ni prises en compte.

  Niveau 2: Il existe un audit basique, qui est réalisé de manière inconsistante. Les activités d'audit sont planifiées et exécutées selon des règles simples, sans suivi ni vérification systématiques. L'évaluation et l'amélioration de la conformité sont réalisées de manière subjective ou arbitraire.

  Niveau 3: Il existe un audit standardisé, qui est réalisé de manière rigoureuse. Les activités d'audit sont planifiées et exécutées selon des méthodes et des techniques reconnues, avec un suivi et une vérification systématiques. L'évaluation et l'amélioration de la conformité sont réalisées de manière objective et systématique.

  Niveau 4: Il existe un audit optimisé, qui est adapté aux besoins spécifiques du projet. Les activités d'audit sont planifiées et exécutées selon des méthodes et des techniques adaptées au contexte du projet, avec un suivi et une vérification automatisés. L'évaluation et l'amélioration de la conformité sont réalisées de manière optimale et ajustée en fonction de l'avancement du projet.

  Niveau 5: Il existe un audit innovant, qui est constamment amélioré selon les meilleures pratiques du marché. Les activités d'audit sont planifiées et exécutées selon des outils et des méthodes avancées, qui permettent de couvrir tous les aspects du projet. L'évaluation et l'amélioration de la conformité sont réalisées de manière innovante et flexible en fonction des opportunités du projet.

- **Avantages**

Avoir une audit mature de l'assurance qualité du logiciel présente de nombreux avantages pour une organisation, tels que :

- Une amélioration de la qualité et de la fiabilité du logiciel, qui réduit les risques de dysfonctionnements, de bugs, de failles de sécurité ou de non-conformités aux normes et aux réglementations.
- Une augmentation de la satisfaction et de la fidélité des utilisateurs et des clients, qui apprécient un logiciel performant, sécurisé et adapté à leurs besoins.
- Une réduction des coûts et des délais de développement, de maintenance et de correction du logiciel, qui optimise la rentabilité et la compétitivité du projet ou de l'organisation.
- Une stimulation de l'efficacité et de l'innovation du logiciel, qui renforce sa capacité à répondre aux besoins évolutifs du marché et aux attentes des utilisateurs.
- Une maîtrise des risques opérationnels, financiers, juridiques ou stratégiques, qui assure la réussite et la pérennité du projet ou de l'organisation.
  L'audit mature de l'assurance qualité du logiciel implique la mise en place d'un système d'assurance qualité basé sur des normes, des audits, des revues, des tests, des mesures, des politiques, des processus et des procédures clairement définis et efficaces. Il permet également d'évaluer et d'améliorer en continu la qualité du logiciel grâce à une analyse et une collecte des erreurs, une gestion de la configuration logicielle et une gestion du risque.
  État actuel
- Constats et observations
- Il n'y a pas d'audit interne
- Non-conformités
- …
  Recommandation
- Opportunités d'amélioration…
  Évaluation

  Maturité : 1/5
  Priorité : Haute
  Effort : Haute

Terminologie

Impact

- **Risques**
  Ne pas avoir de terminologie de l'assurance qualité du logiciel peut entraîner des risques et des impacts négatifs pour une organisation, tels que :
- Une confusion et une incompréhension entre les parties prenantes du projet logiciel, qui peuvent avoir des attentes, des besoins et des exigences différents ou contradictoires.
- Une difficulté à établir et à respecter les normes, les règles et les procédures de qualité, qui peuvent varier selon les contextes, les domaines ou les réglementations.
- Une perte de temps et d'argent à cause des erreurs, des défauts, des non-conformités ou des litiges liés à la qualité du logiciel.
- Une dégradation de la réputation et de la compétitivité de l'organisation, qui peut perdre la confiance et la fidélisation de ses clients ou de ses partenaires.
  Avoir une terminologie de l'assurance qualité du logiciel est donc un élément essentiel pour assurer la clarté, la cohérence et la communication entre les acteurs impliqués dans le développement, le déploiement et l'utilisation du logiciel. Elle permet également de faciliter l'application et l'évaluation des processus et des pratiques de qualité, ainsi que d'améliorer la performance et la satisfaction du logiciel.
- **Niveaux de maturité**
  Niveau 1: Il n'y a pas de terminologie définie ni documentée pour le projet logiciel. Les

termes utilisés sont ambigus, imprécis ou contradictoires. La communication entre les acteurs du projet est difficile ou incomplète.
Niveau 2: Il existe une terminologie basique, qui est utilisée de manière inconsistante. Les termes utilisés sont clairs, mais pas toujours adaptés ou cohérents. La communication entre les acteurs du projet est possible, mais pas toujours efficace.
Niveau 3: Il existe une terminologie standardisée, qui est utilisée de manière rigoureuse. Les termes utilisés sont pertinents, précis et cohérents. La communication entre les acteurs du projet est fluide et complète.
Niveau 4: Il existe une terminologie optimisée, qui est adaptée aux besoins spécifiques du projet. Les termes utilisés sont spécifiques, complets et harmonisés. La communication entre les acteurs du projet est optimale et sans ambiguïté.
Niveau 5: Il existe une terminologie innovante, qui est constamment améliorée selon les meilleures pratiques du marché. Les termes utilisés sont originaux, exhaustifs et normalisés. La communication entre les acteurs du projet est excellente et sans perte d'information.

- **Avantages**
  Avoir la terminologie en assurance qualité du logiciel présente plusieurs avantages pour une organisation, tels que :
- Une meilleure communication et compréhension entre les parties prenantes du projet logiciel, qui peuvent utiliser un langage commun et précis pour exprimer leurs besoins, leurs exigences et leurs attentes.
- Une facilité à établir et à respecter les normes, les règles et les procédures de qualité, qui peuvent être clairement définies et documentées selon les standards internationaux.
- Une réduction des erreurs, des défauts, des non-conformités ou des litiges liés à la qualité du logiciel, qui peuvent être évités ou détectés plus rapidement grâce à des techniques, des pratiques et des outils appropriés.
- Une amélioration de la réputation et de la compétitivité de l'organisation, qui peut offrir un logiciel de qualité, performant, sécurisé et adapté aux besoins des utilisateurs et des clients. L'assurance qualité du logiciel est donc un processus qui vise à garantir que le logiciel développé répond aux exigences fonctionnelles, techniques et de performance attendues par les utilisateurs et les clients. Elle implique des activités de planification, de conception, de codage, de test, de revue, de mesure et d'amélioration continue du logiciel.
  État actuel
- Constats et observations
- Il n'y a pas de terminologie
- Non-conformités
- …
  Recommandation
- Opportunités d'amélioration…
  Évaluation

  Maturité : 1/5
  Priorité : Haute
  Effort : Haute

Rôles et responsabilités

Impact

- **Risques**
- Une confusion ou une incompréhension entre les parties prenantes du projet logiciel, qui peuvent avoir des attentes, des besoins et des exigences différents ou contradictoires.
- Une difficulté à établir et à respecter les normes, les règles et les procédures de qualité, qui peuvent varier selon les contextes, les domaines ou les réglementations.

- Une perte de temps et d'argent à cause des erreurs, des défauts, des non-conformités ou des litiges liés à la qualité du logiciel.
- Une dégradation de la réputation et de la compétitivité de l'organisation, qui peut perdre la confiance et la fidélisation de ses clients ou de ses partenaires.
- Une exposition à des risques opérationnels, financiers, juridiques ou stratégiques, qui pourraient compromettre la réussite ou la pérennité du projet ou de l'organisation.
- **Niveaux de maturité**
  …Tous les rôles du logiciel et les responsabilités envers l'assurance qualité
  Niveau 1: Il n'y a pas de rôles ni de responsabilités définis ni documentés pour le projet logiciel. Les acteurs du projet sont indéfinis, interchangeables ou redondants. La répartition des tâches est aléatoire ou arbitraire.
  Niveau 2: Il existe des rôles et des responsabilités basiques, qui sont attribués de manière inconsistante. Les acteurs du projet sont identifiés, mais pas toujours qualifiés ou disponibles. La répartition des tâches est basée sur des critères simples, mais pas toujours respectée ou contrôlée.
  Niveau 3: Il existe des rôles et des responsabilités standardisés, qui sont attribués de manière rigoureuse. Les acteurs du projet sont qualifiés, disponibles et impliqués. La répartition des tâches est basée sur des critères objectifs, et est respectée et contrôlée systématiquement.
  Niveau 4: Il existe des rôles et des responsabilités optimisés, qui sont attribués de manière adaptée aux besoins spécifiques du projet. Les acteurs du projet sont compétents, motivés et autonomes. La répartition des tâches est basée sur des méthodes et des techniques reconnues, et est optimisée et ajustée en fonction de l'avancement du projet.
  Niveau 5: Il existe des rôles et des responsabilités innovants, qui sont attribués de manière constamment améliorée selon les meilleures pratiques du marché. Les acteurs du projet sont experts, engagés et coopératifs. La répartition des tâches est basée sur des outils et des méthodes avancées, et est innovante et flexible en fonction des opportunités du projet.
- **Avantages**
- Une meilleure communication et compréhension entre les parties prenantes du projet logiciel, qui peuvent utiliser un langage commun et précis pour exprimer leurs besoins, leurs exigences et leurs attentes.
- Une facilité à établir et à respecter les normes, les règles et les procédures de qualité, qui peuvent être clairement définies et documentées selon les standards internationaux.
- Une réduction des erreurs, des défauts, des non-conformités ou des litiges liés à la qualité du logiciel, qui peuvent être évités ou détectés plus rapidement grâce à des techniques, des pratiques et des outils appropriés.
- Une amélioration de la réputation et de la compétitivité de l'organisation, qui peut offrir un logiciel de qualité, performant, sécurisé et adapté aux besoins des utilisateurs et des clients.
- Une maîtrise des risques opérationnels, financiers, juridiques ou stratégiques, qui assure la réussite et la pérennité du projet ou de l'organisation.
  État actuel
- Constats et observations
- Il n'y a pas de responsabilité envers l'assurance qualité
- Non-conformités
- …
  Recommandation
- Opportunités d'amélioration…
  Évaluation

  Maturité : 1/5
  Priorité : Haute
  Effort : Haute

Coffre/boîte à outils

Impact

- **Risques**

  …

- **Niveaux de maturité**

  Niveau 1: Il n'y a pas de boîte à outils définie ni documentée pour le projet logiciel. Les outils utilisés sont obsolètes, inadaptés ou inexistants. L'utilisation des outils n'est pas maîtrisée ni contrôlée.

  Niveau 2: Il existe une boîte à outils basique, qui est utilisée de manière inconsistante. Les outils utilisés sont courants, mais pas toujours appropriés ou cohérents. L'utilisation des outils n'est pas optimale ni vérifiée.

  Niveau 3: Il existe une boîte à outils standardisée, qui est utilisée de manière rigoureuse. Les outils utilisés sont pertinents, précis et cohérents. L'utilisation des outils n'est pas seulement efficace mais aussi mesurée et contrôlée.

  Niveau 4: Il existe une boîte à outils optimisée, qui est adaptée aux besoins spécifiques du projet. Les outils utilisés sont spécifiques, complets et harmonisés. L'utilisation des outils n'est pas seulement mesurée et contrôlée mais aussi optimisée et ajustée en fonction de l'avancement du projet.

  Niveau 5: Il existe une boîte à outils innovante, qui est constamment améliorée selon les meilleures pratiques du marché. Les outils utilisés sont originaux, exhaustifs et normalisés. L'utilisation des outils n'est pas seulement optimisée et ajustée mais aussi innovante et flexible en fonction des opportunités du projet.

- **Avantages**
- …

État actuel

- Constats et observations
- Il n'y a pas d'outils
- Non-conformités
- …

Recommandation

- Opportunités d'amélioration…

Évaluation

Maturité : 1/5
Priorité : Haute
Effort : Haute

Gestion de test

Impact

- **Risques**

  …

- **Niveaux de maturité**

  Niveau 1: Il n'y a pas de processus de test défini ni documenté. Les tests sont effectués de manière ad hoc, sans planification ni suivi. Les résultats des tests ne sont pas analysés ni communiqués. La qualité du logiciel n'est pas mesurée ni contrôlée.

  Niveau 2: Il existe un processus de test basique, qui est appliqué de manière inconsistante. Les tests sont planifiés et exécutés selon des objectifs et des critères d'acceptation. Les résultats des tests sont enregistrés et rapportés. La qualité du logiciel est évaluée de manière subjective.

Niveau 3: Il existe un processus de test standardisé, qui est suivi de manière rigoureuse. Les tests sont conçus et exécutés selon des méthodes et des techniques reconnues. Les résultats des tests sont analysés et utilisés pour améliorer le processus de test. La qualité du logiciel est mesurée et contrôlée selon des indicateurs objectifs.
Niveau 4: Il existe un processus de test optimisé, qui est adapté aux besoins spécifiques du projet. Les tests sont automatisés et intégrés au processus de développement. Les résultats des tests sont exploités pour optimiser la performance et la fiabilité du logiciel. La qualité du logiciel est prédite et garantie selon des normes élevées.
Niveau 5: Il existe un processus de test innovant, qui est constamment amélioré selon les meilleures pratiques du marché. Les tests sont réalisés avec des outils et des méthodes avancées, qui permettent de couvrir tous les aspects de la qualité du logiciel. Les résultats des tests sont partagés et valorisés au sein de l'organisation. La qualité du logiciel est un facteur clé de succès et de compétitivité.

- **Avantages**
- …

État actuel

- Constats et observations
- Il n'y a pas de gestion de test
- Non-conformités
- …

Recommandation

- Opportunités d'amélioration…

Évaluation

Maturité : 1/5
Priorité : Haute
Effort : Haute

Gestion de bug

Impact

- **Risques**

…

- **Niveaux de maturité**

Niveau 1: Il n'y a pas de processus de gestion de bug défini ni documenté. Les bugs sont détectés et corrigés de manière aléatoire, sans priorisation ni traçabilité. Les causes et les impacts des bugs ne sont pas analysés ni communiqués. La satisfaction du client n'est pas mesurée ni prise en compte.
Niveau 2: Il existe un processus de gestion de bug basique, qui est appliqué de manière inconsistante. Les bugs sont enregistrés et corrigés selon des règles simples, sans suivi ni vérification. Les causes et les impacts des bugs sont évalués de manière subjective. La satisfaction du client est écoutée mais pas systématiquement prise en compte.
Niveau 3: Il existe un processus de gestion de bug standardisé, qui est suivi de manière rigoureuse. Les bugs sont classifiés et corrigés selon des critères objectifs, avec un suivi et une vérification systématiques. Les causes et les impacts des bugs sont analysés et utilisés pour améliorer le processus de développement. La satisfaction du client est mesurée et contrôlée selon des indicateurs objectifs.
Niveau 4: Il existe un processus de gestion de bug optimisé, qui est adapté aux besoins spécifiques du projet. Les bugs sont prévenus et corrigés selon des méthodes et des techniques reconnues, avec un suivi et une vérification automatisés. Les causes et les impacts des bugs sont exploités pour optimiser la performance et la fiabilité du logiciel. La satisfaction du client est prédite et garantie selon des normes élevées.

Niveau 5: Il existe un processus de gestion de bug innovant, qui est constamment amélioré selon les meilleures pratiques du marché. Les bugs sont détectés et corrigés avec des outils et des méthodes avancées, qui permettent de couvrir tous les aspects du logiciel. Les causes et les impacts des bugs sont partagés et valorisés au sein de l'organisation. La satisfaction du client est un facteur clé de succès et de compétitivité.

- **Avantages**
- …

État actuel

- Constats et observations
- Il y a une gestion de bug de base
- Non-conformités
- …

Recommandation

- Opportunités d'amélioration…

Évaluation

Maturité : 1/5
Priorité : Haute
Effort : Haute

Planification

Impact

- **Risques**

…

- **Niveaux de maturité**

…Plan de test, gestion de livraison

Niveau 1: Il n'y a pas de planification définie ni documentée pour le projet logiciel. Les activités de test et de livraison sont effectuées de manière ad hoc, sans objectifs ni critères clairs. Le suivi et le contrôle du projet sont inexistants ou inefficaces.
Niveau 2: Il existe une planification basique, qui est appliquée de manière inconsistante. Les activités de test et de livraison sont planifiées et exécutées selon des règles simples, sans suivi ni vérification systématiques. Le suivi et le contrôle du projet sont possibles, mais pas toujours efficaces.
Niveau 3: Il existe une planification standardisée, qui est suivie de manière rigoureuse. Les activités de test et de livraison sont planifiées et exécutées selon des méthodes et des techniques reconnues, avec un suivi et une vérification systématiques. Le suivi et le contrôle du projet sont efficaces et complets.
Niveau 4: Il existe une planification optimisée, qui est adaptée aux besoins spécifiques du projet. Les activités de test et de livraison sont planifiées et exécutées selon des méthodes et des techniques adaptées au contexte du projet, avec un suivi et une vérification automatisés. Le suivi et le contrôle du projet sont optimisés et ajustés en fonction de l'avancement du projet.
Niveau 5: Il existe une planification innovante, qui est constamment améliorée selon les meilleures pratiques du marché. Les activités de test et de livraison sont planifiées et exécutées selon des outils et des méthodes avancées, qui permettent de couvrir tous les aspects du projet. Le suivi et le contrôle du projet sont innovants et flexibles en fonction des opportunités du projet.

- **Avantages**
- …

État actuel

- Constats et observations

- Il n'y a pas de planification
- Non-conformités
- …

Recommandation

- Opportunités d’amélioration…

Évaluation

Maturité : 1/5
Priorité : Haute
Effort : Haute

Qualimétrie

Impact

- **Risques**

…

- **Niveaux de maturité**

…Se mesurer pour s’améliorer

Niveau 1: Il n’y a pas de qualimétrie définie ni documentée pour le projet logiciel. Les indicateurs de qualité sont inexistants ou inadaptés. La mesure et l’amélioration de la qualité ne sont pas réalisées ni prises en compte.
Niveau 2: Il existe une qualimétrie basique, qui est utilisée de manière inconsistante. Les indicateurs de qualité sont courants, mais pas toujours appropriés ou cohérents. La mesure et l’amélioration de la qualité sont réalisées de manière subjective ou arbitraire.
Niveau 3: Il existe une qualimétrie standardisée, qui est utilisée de manière rigoureuse. Les indicateurs de qualité sont pertinents, précis et cohérents. La mesure et l’amélioration de la qualité sont réalisées de manière objective et systématique.
Niveau 4: Il existe une qualimétrie optimisée, qui est adaptée aux besoins spécifiques du projet. Les indicateurs de qualité sont spécifiques, complets et harmonisés. La mesure et l’amélioration de la qualité sont réalisées de manière optimale et ajustée en fonction de l’avancement du projet.
Niveau 5: Il existe une qualimétrie innovante, qui est constamment améliorée selon les meilleures pratiques du marché. Les indicateurs de qualité sont originaux, exhaustifs et normalisés. La mesure et l’amélioration de la qualité sont réalisées de manière innovante et flexible en fonction des opportunités du projet.

- **Avantages**
- …

État actuel

- Constats et observations
- Il n'y a pas de qualimétrie
- Non-conformités
- …

Recommandation

- Opportunités d’amélioration…

Évaluation

Maturité : 1/5
Priorité : Haute
Effort : Haute

Positionnement d'amélioration du processus assurance qualité (actions, réalisations et projections)

Il est important de souligner que les améliorations sont itératives et que nous devons être fonctionnel dès le premier jour.

Priorités :

44. Amélioration des environnements de test (stable, private for Drupal and test data control)
45. Avoir un coffre d'outils (il n'y a rien de bon chez Vidéotron, il faut se débrouiller par nous même) **! URGENT !**
46. Amélioration, avoir de la documentation de test (avant d'intégrer des nouvelles ressources) → voir pour une solution temporaire.
47. Mettre en place l'audit et le plan d'actions
48. Suivre le plan…

Attention

- Pour éviter les pièges, ne pas automatiser avant d'avoir une stratégie et un plan d'automatisation **(pour faire une stratégie il faut avoir une couverture et pour avoir une couverture il faut documenter les tests et pour documenter il faut un outil professionnel prêt à l'utilisation conçu pour ça et de préférence pensé en vue d'automatiser)**
-

Agenda des jalons

OKR (objectifs et résultats clés)	État	Date de début	Estimation du temps	Date cible (prévue)	Tâches (Jira)
QAP | Audition - Quality assurance process improvement	IN PROGRESS	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	Presentation of the roadmap, creation of the first Jira tasks (to the team)
					Workshop "What does a QA Coach do?"
					Onboarding QA (SDET profile)
					Projection over 3 weeks (by sprint)
					Long-term projection (per PI/Q and year)
					Positioning report
					Roadmaps
					Quality Assurance Roles, Responsibilities, and Expectations
					Terminer la politique AQ
					Première analyse de l'état actuel (Pistes et Propositions d'améliorations) Case départ,niveau 1 de la maturité, il n'y a aucune documentation des tests et manque d'outils fonctionnels QA (**URGENCE !**) L'équipe Alithya se retrouve sans

					outils QA car l'équipe Appnovation était autonome et a quittée avec leurs outils QA. Videotron est en restructuration QA, il n'y a rien de bon à proposer pour nos besoins et contextes présentement. • Trop d'environnements de test et ultra instables. Tous les tests sont dans des environnements E2E (Pas de test unitaire, intégration et système) Une demande d'aide d'Alithya pour l'équipe Drupal pour pallier au manque que nous n'avons plus d'outils chez Videotron : **Solution → GoTest**
					Analyse profonde et complète de l'état (Pistes et Propositions d'améliorations)
QAP | Audition - Quality assurance process improvement - Maturity level 2/5					
QAP | Audition - Quality assurance process improvement - Maturity level 3/5					
QAP | Audition - Quality assurance process improvement - Maturity level 4/5					
QAP | Audition - Quality assurance process improvement - Maturity level 5/5					
QAP | Bug management -		Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	CYCLE DE VIE (GESTION) DES ANOMALIES

Tracking, Root cause and documented by tests					
					Bug tracking
					Bug distribution (dispatch)
QAP | Bug management - Maturity level 2/5					
QAP | Bug management - Maturity level 3/5					
QAP | Bug management - Maturity level 4/5					
QAP | Bug management - Maturity level 5/5					
QAP | Qualimetry - Measurement reference	IN PROGRESS	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	Établir les métriques et les objectifs de la Qualimétrie
					Monitorer les exécutions
					Monitoring
					Traçabilité, journalisation
					Test monitoring (Grafana) - First board + at less 1 notification alert
QAP | Qualimetry - Maturity level 2/5					
QAP | Qualimetry - Maturity level 3/5					
QAP | Qualimetry - Maturity level 4/5					
QAP | Qualimetry - Maturity level 5/5					
QAP | Test management - Maturity level 1/5	IN PROGRESS	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	Establish test management conventions

					Reduce, privatize, stabilize and define test types for all test environments (priority) (gain confidence in what has already been tested and avoid over-testing)
					Write all non-regression tests (manual) with expected results
					Workshop - TMMI
					Workshop - Test plan & documented story/bug by test
					Workshop - Test control
					Workshop - Test monitoring
					Workshop - Test environment
					Workshop - Test management (testing strategy & policy)
					Workshop - Test documentation
					Workshop - Testing terminology
					Renommer les tests de la non-régression avec la nomenclature de test selon la politique. (incluent les résultats attendus) TMO Content Suscription
					Écrire les tests sanitaires prioritaires avec la nomenclature de test selon la politique. (incluent les résultats attendus) Voici les validations que je dois faire avant de commencer les tests “QA” API (ON) Le site et sa navigation (main) est accessible Login(s) (je peux me connecter)
					Automatiser les tests sanitaires (one shot)
					Automatiser progressivement les tests de la non-régression (couverture global)
					Établir la procédure/plan, pour automatiser les tests de la non-régression pendant l’implémentation des stories (automatisation en just in time) cela fait partie du CT (continous testing)
					Atelier du Shift-left → Les bases avec les 3 amigos et les plans de test Raffiner les stories et les bugs avec les trois amigos avant de planifier et s’engager dans les sprints. (Jira+GoTest) → ce avant de pointer

					Planifier la qualité (Plan de test) et faire le suivi qualité des livraisons (Jira+GoTest) → ce durant le planning
					Inclure et découper les règles d'affaires comme résultats attendus dans les tests
					Mock - Implantation Mock - Premier test sanitaire API Mock - Premier test sanitaire UI Mock - Faire la liste de tous les tests sanitaires et les planifier (docu+auto)
					Faire les tests sur ACC
					Contrôle des jeux de donner avoir un client que le mot de passe ne change jamais avoir un csr qui ne change jamais
					Palier au captcha Palier au token API
					Test basé sur le modèle (Model Based Testing)
					Collaboration et partage des tests GoTest avec SN Impliquer le front-end (UI-UX) dans la loop QA (Tester les components Bifrost (manuels et automatisés)) Impliquer DevOps dans la loop QA Impliquer le Dev dans la loop QA Impliquer l'analyste dans la loop QA Impliquer le PO dans la loop QA Impliquer SN dans la loop QA Impliquer ETIYA et BSI dans la loop QA
					Les 3 principes des fondation de gestion de test Le 1er principe : Respect du cycle complet de la vie du développement Le 2eme principe : Optimiser la quantité (éliminer le superflu) Le 3eme principe : Hyper-automatisation
					Contrôle sur les jeux de données
					Écrire une story incluant l'assurance qualité Écrire un bug/anomalie incluant l'assurance qualité (RCA+bug tracking)
					Environnements de test stable et sur demande
					Documentation vivante et Documentation intelligente
					Transparence sur les résultats
					Cycle de vie livraison

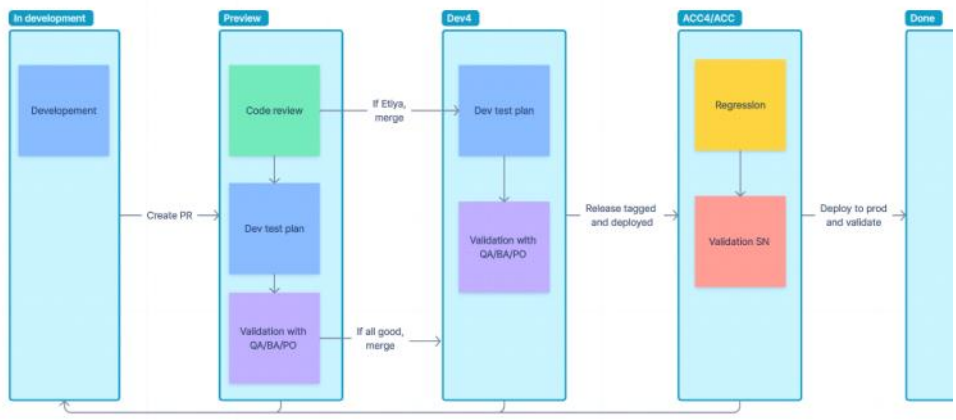
					Post mortem livraison
					Avoir le contrôle de nos data de test / Mock
					Moins de promotions d'environnements de tests
					Conception/documentation de test Gestion des données
					Environnements de test Planification et Organisation (plan de test)
					Exécution de test
					GoTest | Exécuter un test API ETIYA
					GoTest | Exécuter un test API BSI
					GoTest | Écrire tous les tests de non-régression (manuel), documenter les tests manuels est prioritaires (l'automatisation viendra par priorité)
QAP | Test management - Maturity level 2/5	TODO	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	
QAP | Test management - Maturity level 3/5	TODO	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	
QAP | Test management - Maturity level 4/5	TODO	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	

QAP | Test management - Maturity level 5/5	TODO	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	
Terminology - Maturity level 1/5					
Terminology - Maturity level 2/5					
Terminology - Maturity level 3/5					
Terminology - Maturity level 4/5					
Terminology - Maturity level 5/5					
Roles and responsibilities - Maturity level 1/5					
Roles and responsibilities - Maturity level 2/5					
Roles and responsibilities - Maturity level 3/5					
Roles and responsibilities - Maturity level 4/5					
Roles and responsibilities - Maturity level 5/5					
Planification - Maturity level 1/5					
Planification - Maturity level					

2/5					
Planification - Maturity level 3/5					
Planification - Maturity level 4/5					
Planification - Maturity level 5/5					
QAP | Tool box - Basics	IN PROGRESS	Sep 21, 2023	Sep 21, 2023	Sep 21, 2023	GoTest | Configurer les SUT GoTest pour accéder aux environnements de test de Vidéotron
					Demande de GoTest
					Stabiliser les exécutions par les plans de test
					Formations et ateliers GoTest
					Écrire les conventions de structure et de nomenclature et les partager GoTest
					Coffre d'outils
					Correction, Prevention, Réaction
					Gestion de changement Gestion de risque Auto audition/évaluation
					Politiques et Processus
					Rôles et responsabilités
					Terminology
					Formations/ateliers Qu'est-ce qu'un Coach QA ? Shift-left & Shift-Right 3 amigos & ATDD, TDD et BDD Stratégie & Politique RCA Bien documenter un bug ou un récit lié à des tests QA Agile Qualité en continu Type de test Les niveaux de test Qualité en continu Gestion de bug Gestion de test Qualimétrie Planification L'affaire de tous ! (Rôles et responsabilités QA) Les meilleurs pratiques qualité Livrer rapidement de la qualité Les termes de la qualité Qualité Ci-Cd-Ct

					Qualité moderne VS old school
					~~Suivi de la non-régression dans Excel (temporairement)~~ (remplacé par GoTest)
					Prise en charge des tests Cypress dans le pipeline
					Canal de déploiement, suivi des promotions
					Santé global de la satisfactionêtre pro-actif dans nos stratégie QAAnalytics : AppDynamics
					Grafana
					Plan test : Implémenter le concept de Plan de test pour les bugs et le stories
					Donner de la visibilité de l'accomplissement du niveau de la maturité de l'assurance qualité. Le roadmap QA
					Faire un bilan de l'avancement hebdomadaire avec la feuille de route
					Créer un Epic "Implémentation/utilisation GoTest"
					GoTest | Conventions
					GoTest | Demande d'amélioration pour avoir la fonctionnalité Jira au niveau des test plans
					GoTest | Importer les tests existant dans la nouvelle instance et s'assurer qu'ils fonctionnes
					GoTest | Ajouter les utilisateurs de l'équipe (j'ai déjà ajouter Ali et Laura)
					GoTest | Voir avec David Lagacé pour que les SUTs soient sur le VPN Videotron
					GoTest | Voir avec David Lagacé pour exécuter les tests sur Browserstack
					GoTest | Error pour les exécutions des test plans
					GoTest | Valider les exécutions multiples
					GoTest | Ajouter les exécutions de nuit pour les stats de santé/performances
					GoTest | Exécuter des tests et recevoir le résultat par un url pour Appdynamic
					GoTest | Exécuter et recueillir les tests sanitaires + résultats dans les pipelines après le déploiement de l'environnement

QAP | Tool box - Maturity level 2/5					
QAP | Tool box - Maturity level 3/5					
QAP | Tool box - Maturity level 4/5					
QAP | Tool box - Maturity level 5/5					



Quality Assurance (strategy and policy) / Assurance qualité (stratégie & politique)

January 18, 2024 10:46 AM

Here we are in the strategy and the modern quality assurance policy flexible to cutting-edge, “legacy” and/or hybrid contexts. This formula encourages us to measure ourselves and therefore towards continuous improvement

-

Nous sommes ici dans la stratégie et la politique d’assurance qualité moderne et adaptable aux contextes pointus, « hérités » et/ou hybrides. Cette formule nous incite à nous mesurer et donc à nous améliorer continuellement

- Conventions
- Life cycle / Cycle de vie
- Qualimetry / Qualimétrie
- Roles and responsibilities / Rôles et responsabilités
- Toolbox / Boîte à outils

4 Métriques QA

January 18, 2024 10:47 AM

ATTENTION nous travaillons actuellement à une nouvelle version de ces métriques

Les métriques AQ sont complémentaires aux métriques Agile et Produits. Elles sont gérées et maintenues à travers les différents sprint d'un projet.

Rappel - Métriques Agile

1.
-
-
2.
-
-
3.
-

Rappel - Métriques Produit

Les métriques AQ doivent permettre à l'ensemble des parties prenantes, principalement notre client, d'avoir un état clair de l'état de santé de son système.

Table des matières

- Métriques spécifiques à l'AQ
  - Les métriques AQ couvre 4 axes afin d'évaluer la qualité du produit tout au long de son développement, donc "in sprint"
  - Comment et quand calculer ces indices
  - Cartographie des tests
- Comment illustrer la cartographie
- Présenter les métriques
  - À qui les présenter?
  - Qui présente?
  - Quand?
  - Comment?

Métriques spécifiques à l'AQ

Les métriques AQ couvre 4 axes afin d'évaluer la qualité du produit tout au long de son développement, donc "in sprint"

4. L'indice de couverture fonctionnelle illustre la qualité et la traçabilité des récits documentés
5. L'indice d'automatisation illustre le % d'automatisation ainsi que l'indice de dette vers l'automatisation. Cet indice considère également les tests manuels. Il est le reflet de la stratégie d'automatisation, qui englobe ce qui sera automatisés ou non.
6. L'indice de confiance, lui, illustre la fiabilité et la stabilité du produit au fil des déploiements et du temps.
7. La cartographie des tests, telle quelle, n'est pas une "vraie" métrique puisqu'elle n'est pas le résultat d'un calcul mais plutôt une représentation de l'équilibre à atteindre. Elle illustre

l'équilibre des niveaux et types de tests de manière à s’assurer que le volet business d’un produit est autant validé que son volet technique.

Indice de couverture fonctionnelle

Indice d’automatisation

Indice de confiance

Comment et quand calculer ces indices

comment les calculer?

-
-
-
-
-

Cartographie des tests

La cartographie des tests permet d’illustrer si les différents axes de qualité d’un produit sont validés, autant le volet affaires que le besoin technologique doit être validés pour assurer la qualité d’un produit.

Nous utilisons le quadrant de tests Agile pour illustrer cet équilibre recherché (un gabarit est disponible ci-bas)

Les quadrants Q2 et Q3 permettent de couvrir l’aspect business du produit.

Dans le quadrant Q2, nous cherchons à avoir une couverture des récits in sprint par des tests automatisés idéalement. Ce quadrant est habituellement une représentation du principal travail des spécialistes AQ d'une équipe de réalisation.

Le quadrant Q3, quand à lui, illustre les tests fait par le client (ou son représentant) permettant d'accepter la livraison. Ces test sont habituellement des tests manuels. Les spécialistes AQ peuvent accompagner le client s'il a des doutes sur l'approche à prendre ou la couverture de ses tests.

Les quadrants Q1 et Q4, eux, sont presqu'invisibles au client. Ils servent à conforter l'équipe de livraison. Ce seront des tests techniques. Ils seront principalement la responsabilités des développeurs ou de spécialistes de ces tests d'exceptions. Cependant, les spécialistes AQ sont de plus en plus appelés à contribuer à leur réalisation. L'ensemble de ses types de tests sont des tests automatisés et inclus dans le pipeline de développement.

Comment illustrer la cartographie

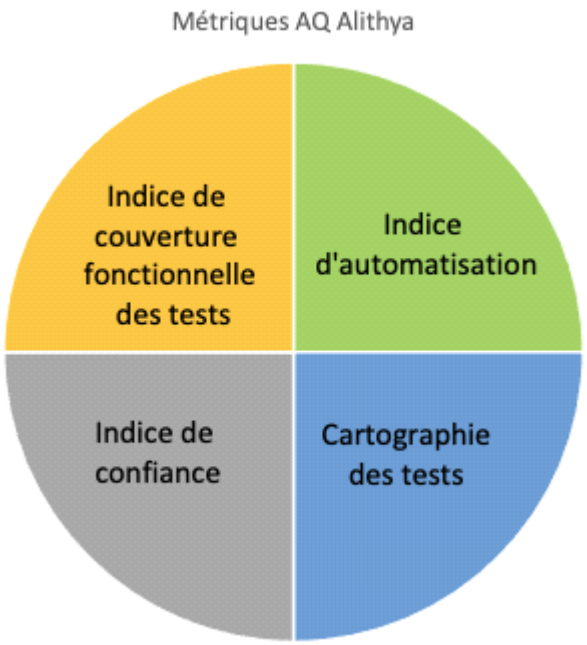
Comment illustrer la cartographie

-
-
-
-

Présenter les métriques

Maintenant, présenter les métriques

-
-
-
-



Gestion de test (la stratégie de test)

January 18, 2024 10:48 AM

Ce document résume tous les éléments à considérer afin de mettre en œuvre les efforts d'essais substantiels selon l'entente sur les conventions engagés, pour assurer efficacement un service qualité à travers les défis et les différents projets, peu importe le profil du client

Cette stratégie comprend le test manuel et automatisé

Ce document s'adresse :

- au client qui doit s'assurer que toutes ses exigences en matière d'essais (propriétés à tester, niveaux de qualité visés, étapes à franchir, etc…) ont été prises en considération et consignées ;
- au chargé de projet qui doit documenter toutes ces exigences (risques, propriétés à tester, niveaux de qualité visés, etc.) et veiller au respect de la stratégie d'essais ;
- à l'équipe de gestion de test (service qualité) et de réalisation qui doit être informée :
- La bibliothèque des tests, des plans de test pour chaque livraisons
- Les principes, approches, tactiques, techniques, le coffre à outils et son utilisation pour valider la qualité des biens livrables.

L'existence et le maintien de ce document pour l'ensemble du développement permettra d'établir la confiance vers la satisfaction.

Notes :

- Ceci n'est pas un plan ! Ce ne sont que les ententes, conventions, règles, nos façons de faire, nos guides, nos ligne directrices ne pas confondre avec un plan de test ou la mise en œuvre de la stratégie.
- La stratégie de test s'applique qu'à l'équipe service qualité et peut servir de modèle par la suite pour l'entreprise.

Contributeur : @Michel Belanger

Sommaire

-
-
    -
    -
    -
-
    -
    -
-
-
    -
    -
    -
    -
    -
    -
    -
    -
    -
    -
-

-
-
-

- Peut-être générer par API cli et/ou Front-End (le but ici est de générer les données de la même façon que l'utilisateur ou presque*)
- Les requêtes direct aux bd peuvent-être utilisées, exceptionnellement, pour modifier des états ou des switch… ex. gestion de reset, timeout, active, delay, on/off…
- Ne pas utiliser les datasets, il faut utiliser directement la source des jeux de données avec une boucle contrôlé

Tactiques et techniques

Tactique ou technique	☑ Portée ❌ Hors portée (selon les ententes du contrat)	Description / Objectif	Utilisation	Responsable	Env. SUTs	Jeu(x) de données
Le "LEGACY"	☑❌	Comment négocier avec le legacy… exemple : Ne pas transférer d'un coup tous les vieux tests dans un nouveau système ! Il faut avant tout, prioriser et procéder de façon itérative le transfère selon les conventions en vigueurs exemple : L'application n'est pas découplable par niveaux et donc pas moquable, l'évaluation de la valeur ajoutée sera différente.				
Test de non-régression	☑					
Test d'acceptation	☑					
Test fonctionnel et non-fonctionnels	❌					
Test continus	❌					
Test A/B	❌					
Test simultanés	❌					
Test de conformité						
Test par comparaison (actual → expected) • **métamorphique** (défini)	❌					

• **par propriété** (généré)						
Boite noir / boite blanche	☑❌	Comment négocier/tester avec les 3rd parties ? Comment assurer la qualité sans l'accès au développement ?				

Planification et gestion de livraison

Plan de test

Plan de test

ARCHIVED

-
- 

Owned by Michel BelangerUpdated on Mar 28, 2023

Un plan de test (livraison/release/delivery test plan) est un document détaillé qui décrit et met en actions la stratégie de test, les objectifs, les ressources nécessaires, le calendrier et les critères de réussite pour assurer la qualité d'une livraison/version d'une fonctionnalité ou une suite ou une librairie ou une epic ou une story ou un logi

Confluence

Open preview

Rédaction d'un plan de test

-
- ❌

Owned by Michel BelangerUpdated on Feb 28, 2023

OBJECTIF, Nous verrons ici comment produire un plan de test, le documenter, l'exécuter, faire la communication du suivi, le compléter, Aide, Procédures, Utilisation, Instructions, Guide, Gabarits, Exemples… Définition : https://alithya.atlassian.net/wiki/spaces/PAQ/pages/154633076739 Comment rédiger un plan de test ? suivez l'exemple/modèle : 0001.

Confluence

Open preview

exemple GoTest : https://training.alithyagotest.com/askida/editor/project/1300

Cycle de vie

Pour utilisateur, design, dev, bug, test, support, DevSecOps… (un seul cycle Agile et commun)

Cycle de vie du développement

-
- 

Owned by Michel BelangerUpdated on Mar 8, 2023

Utilisez les 6 phases et 3 états pour les releases (livraisons), epic/story/bug (ex.Jira), GoTest, plan de test… c'est notre modèle de base pour tous les cycles de développement. https://alithya.atlassian.net/wiki/spaces/PAQ/pages/154320535553

Confluence

Open preview

Post mortem

Pour chaque livraison… fêter les bons coups et entreprendre des résolutions/actions dans le but de s'améliorer

Livraison et Branches/Repo

Version

MUST tous les produits, les fonctions, les api, les services, les components, les librairies, les suites doivent être versionnés pour effectuer un suivi de la qualité

le format minimum standard est [major].[minor], la mineur est à tous les changements (maintenance), la majeur est pour des changements affaire ou UI/UX ou d'année ou ajout d'une nouvelle fonctionnalité…

C'est une itération de changements 1.0, 1.1, 1.2, 1.3, 2.0, 2.1, 2.2…

Attention, il aussi le numéro/version d'un "build" ce type de versionnage n'est pas planifiable c'est un tag !
Il faut aussi comprendre que par exemple en python, nous n'avons pas de "build" donc le numéro de version est le tag du dernier push sur la branche
Il est indispensable de rattacher ces "tags" à la version et le format commun est [major].[minor].[revision].[build]

C'est pourquoi nous avons souvent plusieurs version pour une application

ex. ici nous avons l'application "APPONE"

- APPONE 3.2
- API 1.234
- CATlib 12.3.0.1234
- SettingPortal 2.1

Il est important d'avoir la liste de tous les changements et d'attacher les résultats des tests (release notes)

Catégoriser les tests (quadrant)

Planifier/Mobiliser/Organiser/Responsabilisé les parties tiers du métier

known issue management

Communication

Collaboration, rôles, responsabilités, compétences et communication

Il est important d'avoir un point de contact et d'entrée dans le cycle de vie du développement, un responsable qui va faire la liaison avec le service QA et le développement de l'entreprise… Cela, pour chacune des phases, incluant la satisfaction du client !

Validation à toutes les phases

Ne pas automatiser sur l'incertitude du développement

Ne pas automatiser après le dev… commencer avant le dev pour que le dev puisse utiliser l'automatisation pour valider son dev

Les métriques et qualimétrie

- progress
- defect (rca, bug tracking)
- coverage (code, cases…)

Présentation, rencontre "état de santé"

Préparer une présentation de la stratégie ex, PowerPoint et la présenter à un cycle régulier avec un portrait de la qualimétrie santé global

Types de test

Quels types de tests sont actuellement couverts + 100% et quel est l'objectif de

couverture ?

- Quels outils pour chaque types ?
- Qui, quand, où…?

Il est important de définir le type du test lors de la rédaction du test ! Un test peut avoir plusieurs types



Type		☑ Portée ❌ Hors portée (selon les ententes du contrat)	Description / Objectif	Utilisation	Responsable	Env. SUTs	Jeu(x) de données
Test sanitaire/intégrité (santé)	Test de fumée (smoke)	☑					
	Test de déploiement	☑					
	Test d'installation	☑					
	Test de publication	☑					
	Test de diffusion	☑					
	Test de connexion	☑					
Test data	Test de configuration	☑❌					
	Test unitaire de code	❌					
Test d'analyses de contrôle du développement		☑❌	ex. analyse de code statique, analyse des flux de données, analyse du DOM, analyse des métriques, analyse de couverture de code, traçabilité, journalisation, etc.				
Compatibility		❌	(ex. internationalisation et localisation, navigateurs web (browsers), plateformes,				

			environnements, os, appareils, etc.) Système sous test web mobile ? web tablette ? web desktop ? Edge, Safari, Chrome, Firefox… IOS ? Android ? Mac OSX ? Windows ? Linux ? native mobile ? native tablette ? native desktop ? version ? Localisation tester l'app dans toutes les langues L'accessibilité				
Test destructifs/ choas		✗					
Test d'utilisation		✗					
Test de performance		✗					
Test d'accessibilité		✗					
Test de sécurité		✗					
Test de concurrence/collision		✗	ex. deadlocks, race conditions and problems with shared memory/resource				

Spécifique aux tests automatisés

Modèle opérationnel de test automatisé

-
- 

Owned by Michel BelangerUpdated on Mar 14, 2023

Les objectifs de l'automatisation de la majorité des tests de la nouvelle application sont de : Effectuer l'exécution complète des scénarios de tests à travers CI/CD afin d'assurer de la non-régression, Maximiser le temps d'exécution des tests en évitant la répétition des exécutions manuelles des différents scénarios de tests, Valider le processus

Confluence

Open preview

Spécifique aux tests manuels

Modèle opérationnel de test manuel

-
- 

Owned by Michel BelangerUpdated on Feb 27, 2023

Comment utiliser GoTest/Jira dans le contexte du manuel ?
Confluence

Open preview

Les variables

Utilisation du **Snake-Kebab** pour la nomenclature des variables

le _ pour les espaces
le - pour les jonctions d'un path ou séparateur de sections
Ne pas utiliser le - dans les mot français composés (arc-en-ciel)
MAJUSCULE pour globale
minuscule pour local
ne pas utiliser la première lettre d'un mot en majuscule et le reste en minuscule (~~Minuscule~~)
ex. Globale : PET_CLINIC-PET_OWNER-NEW (pet clinic/pet owner/new)
ex. Local : lignes_arcenciel (Lignes arc-en-ciel)

Rédaction de Test

Rédaction de test

https://training.alithyagotest.com/askida/editor/scenario/17915

todo

Généraux

Gardez-le simple et doux (KISS)

Plus simple est toujours mieux, Réduire la complexité autant que possible. (Keep It Simple and Soft)

Mémo :

ref. Stratégie de tests Alithya

Les meilleurs pratiques et les anti-patterns

chaque élément ici doit être écrit comme une convention et faire partie intégrante de la stratégie

Suivre la formule → Faire la cuisine avec ingrédient/quantité/contenant (inventaire/préalable…), recette (définition/structure…), préparation (planification/utilisation/technique…)

UTILISATION/CONVENTION
→ Le besoin et la solution : Répond aux exigences qui ont guidé sa conception et son développement (En d'autres termes, fait-il ce qu'il est censé faire quand il est censé le faire ?)

*** Le plan c'est la couverture, les résultats, les spécification… etc !

Idée rough | La stratégie de l'assurance qualité

January 18, 2024 10:49 AM

La stratégie est notre guide/vision composée de nos **conventions**, règles, processus, procédures, positionnements, recommandations, pratiques, méthodologies, tactiques, choix du coffre d'outils et la façon d'utiliser les outils, etc.
Mais aussi composée de ***sous-stratégies*** *comme* ***"stratégie de test" par exemple.***

Bref ! C'est la définition commune de l'assurance qualité.

Stratégie Méthodologie/Framework

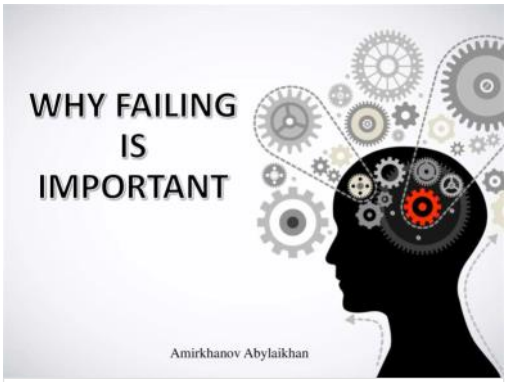
January 18, 2024 10:49 AM

Normalement, il y a une direction/guide Agile dans une entreprise, il faut arrimer l'assurance qualité avec Agile…
Agile et Assurance qualité est une entité inséparable

Exemples :

Amélioration continue (fail fast… success!)













SAFe

Scaled Agile Framework (SAFe) Tutorial | SAFe Agile Framework Tutorial | Introduction to SAFe Agile - YouTube



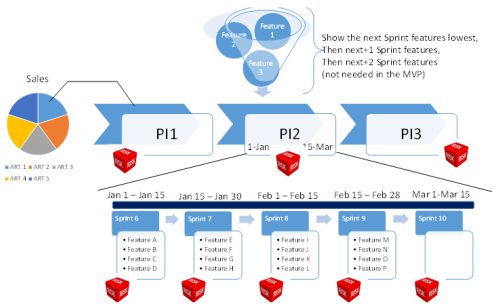
Dans le framework SAFe, ***l'agilité ne doit jamais être obtenue au détriment de la qualité***. SAFe exige des équipes de tous les niveaux qu'elles définissent ce que « terminé » signifie pour chaque tâche ou projet, et qu'elles intègrent des pratiques de développement de qualité dans chaque contrat de travail. Selon SAFe, **il existe cinq dimensions clés de qualité intégrée : le flux, la qualité de l'architecture et du design, la qualité du code, la qualité du système et la qualité de la livraison.**

Agile Release Train (ART)

Ce train-ci est une métaphore. L'ART est un groupe de plusieurs équipes Agile qui se réunissent pour développer et livrer des solutions spécifiques. Le train contient généralement entre 50 et 125 personnes qui travaillent toutes ensemble pour développer et déployer des solutions précises.







Stratégie de test (wip)

January 18, 2024 10:51 AM

IMAGE

UTILISATION

La stratégie de test moderne et Agile qui fonctionne aussi pour le waterfall

Il faut comprendre que ceci est un objectif de la stratégie global de haut niveau et non pas une Stratégie spécifique ou Plan de test d'un **projet.**

La réalité/maturité de chacun n'est pas la même, il faut simplement se faire un plan avec des objectifs en symbiose avec la stratégie, pour y arriver.

Les principes de la stratégie de test

Soyez prudent quand il est écrit "**possible**", cela inclut d'éviter les pièges et les mauvaises pratiques (anti-patterns)

1. Décalage en amont (shift-Left) Valider au plus vite/tôt possible
2. Avoir le moins de tests possible

a. Valeur ajoutée
b. Le risque
c. Affaire ou équipe versus technologie ou produit

1. Automatiser le maximum possible

a. Hyperautomatisation
b. Le coffre à outils
c. Le niveau des compétences/connaissances

Conventions/procedure/processus

Terminologies

Agile ou Waterfall ? La stratégie ce n'est pas un choix selon l'état actuel mais la direction à prendre en compte les défis et et définir le comment y parvenir par la suite

Définition SDLC, les phases, les projets et Jira

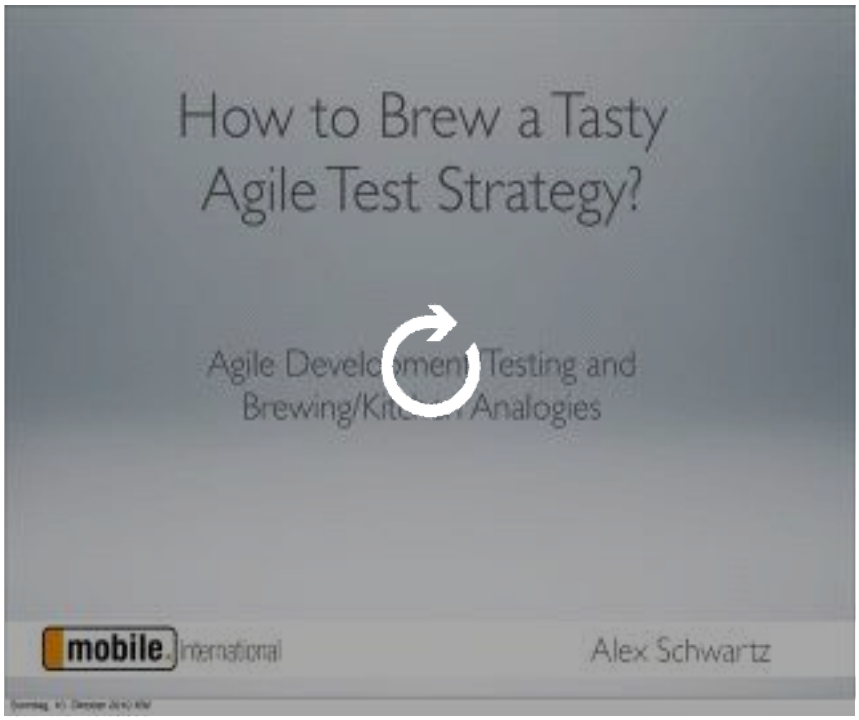
La documentation

Kanban

Le coffre d’outils

Complete Guide to Test Strategy Document [With Template] (artoftesting.com)

How to brew a tasty agile test strategy (slideshare.net)



mon pref.

Avoir le moins de tests possible

January 18, 2024 10:52 AM

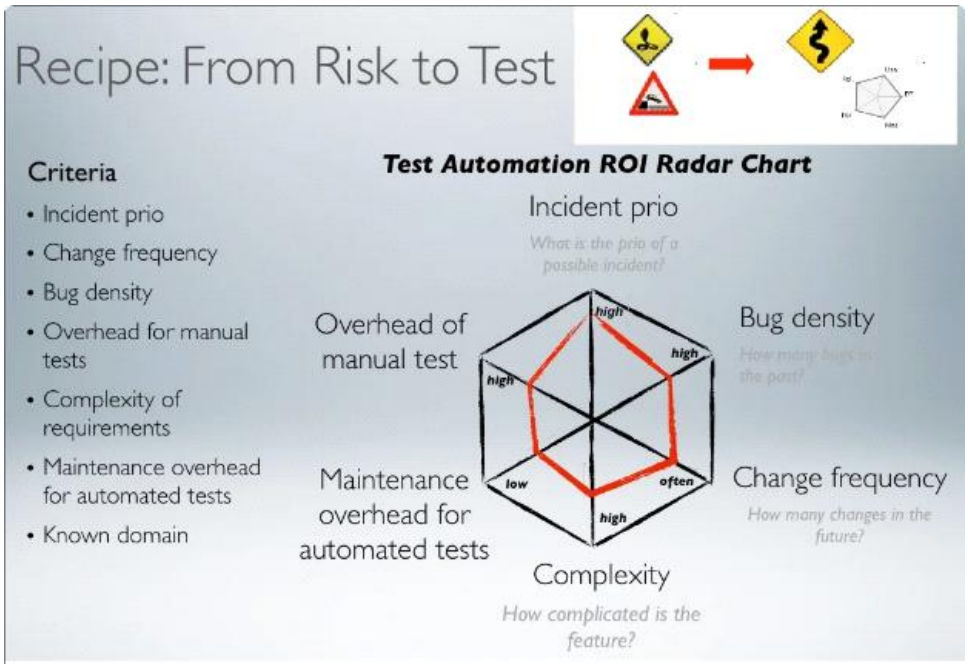
Éviter de tester ne veux pas dire de ne pas tester…

Comme l'art martial, le premier réflexe au combat à avoir est d'éviter le combat.

Nous pouvons éviter de faire des tests dans la façon d'implémenter le code ou dans l'ordre des validations

…

Nous pouvons éviter des tests selon le risque
Viser le découpage de la couverture avec La pyramide de test (niveaux de test) permet aussi de réduire considérablement le nombre de tests



Valider au plus vite/tôt possible

January 18, 2024 10:52 AM

Automatiser le maximum possible

January 18, 2024 10:52 AM

La maturité des tests (TMMI)

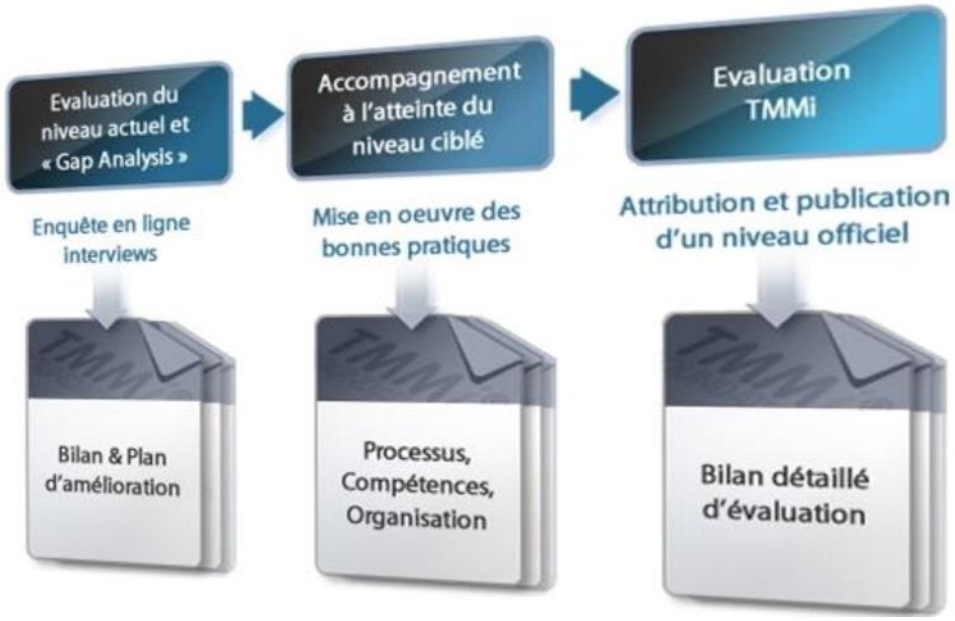
January 18, 2024 10:52 AM

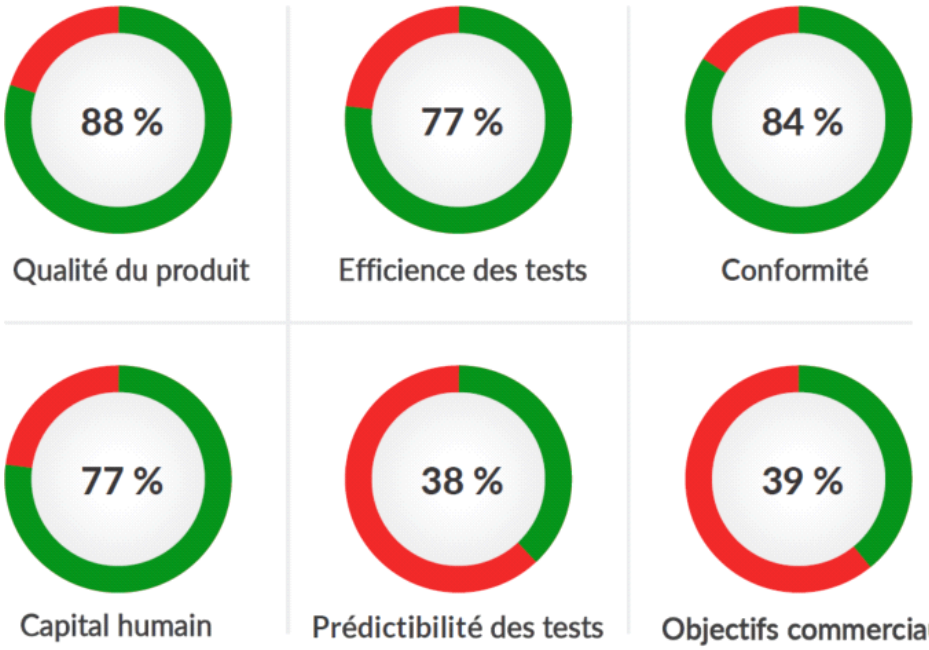
T*esting* **M***aturity* **M***odel* **I***ntegration*

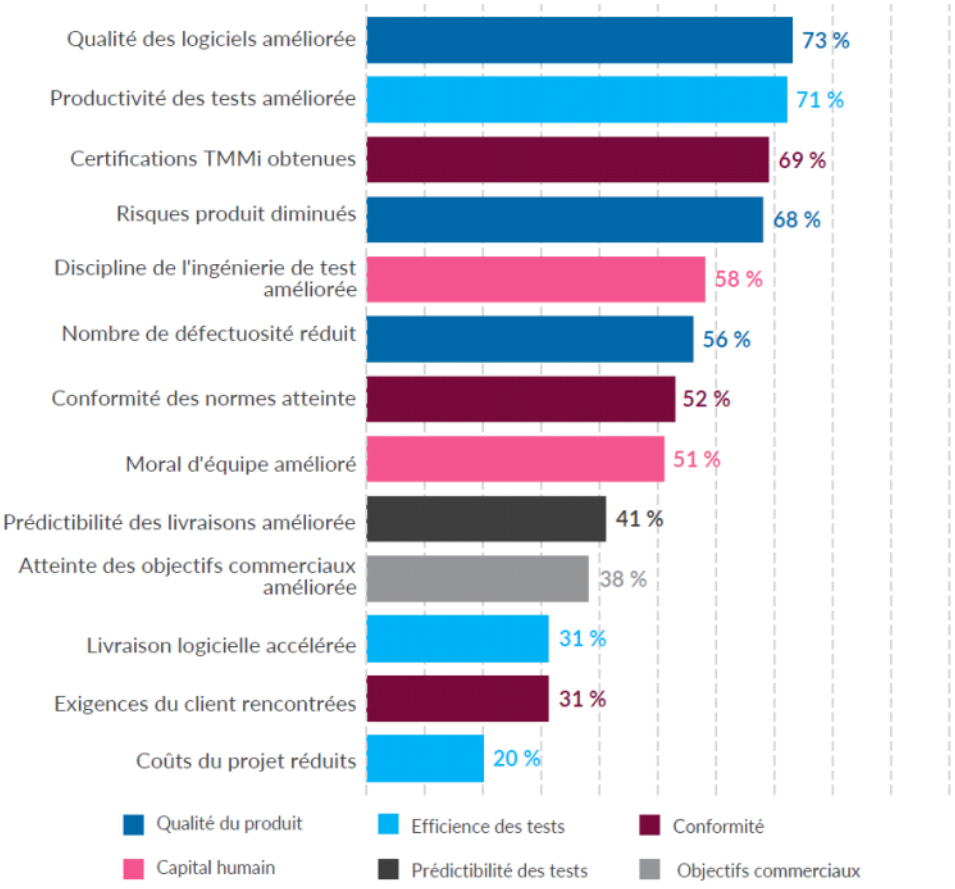
Niveau 1 Initial	• Test Adhoc/Exploratoire/Monkey-testing • Modèle non uniformisé (projet par projet) • Pas de réutilisation des tests • aucun gestionnaire de tests
Niveau 2 Géré	• Politique et Stratégie de test • Gestion des tests et Planifications des tests • Pilotage et Contrôle des tests • Conception et Exécution des tests • Environnement de test (stable et sur demande ex. branch preview)
Niveau 3 Défini	• Organisation de test • Programme de formation au test • Intégration et cycle de vie du test • Test non-fonctionnel • Revues par les pairs
Niveau 4 Mesuré	• Mesure du test • Évaluation de la qualité logicielle • Revues avancées • Qualimétrie • Chasse aux bugs
Niveau 5 En Optimisation	• Prévention des bugs • Optimisation du processus de test • Contrôle de la qualité

TMMi Industry Survey v3 Certilog (surveygizmo.com)

Des stats de l'efficacité du TMMI appliquée (pour les amoureux des stats 🙂)


Evaluation du niveau actuel et « Gap Analysis »
Accompagnement à l'atteinte du niveau ciblé
Evaluation TMMi
Enquête en ligne
interviews
Mise en oeuvre des bonnes pratiques
Attribution et publication d'un niveau officiel
Bilan & Plan d'amélioration
Processus, Compétences, Organisation
Bilan détaillé d'évaluation


88 %
Qualité du produit
77 %
Efficience des tests
84 %
Conformité
77 %
Capital humain
38 %
Prédictibilité des tests
39 %
Objectifs commerciaux


Qualité des logiciels améliorée
73 %
Productivité des tests améliorée
71 %
Certifications TMMi obtenues
69 %
Risques produit diminués
68 %
Discipline de l'ingénierie de test améliorée
58 %
Nombre de défectuosité réduit
56 %
Conformité des normes atteinte
52 %
Moral d'équipe amélioré
51 %
Prédictibilité des livraisons améliorée
41 %
Atteinte des objectifs commerciaux améliorée
38 %
Livraison logicielle accélérée
31 %
Exigences du client rencontrées
31 %
Coûts du projet réduits
20 %
Qualité du produit
Efficience des tests
Conformité
Capital humain
Prédictibilité des tests
Objectifs commerciaux

Stratégie du Delta

January 18, 2024 10:53 AM

La maturité et le suivi de l'amélioration ainsi que le pérennité de l'assurance qualité du projet/produit/fonctionnalité est fondamental…

Ref.

- Qualité Logicielle : Comment l'évaluer et la gérer ? - SoftFluent
- Comment évaluer la maturité digitale de votre entreprise (wevalgo.com)

Voici un exemple d'un tableau du Delta (1-10) pour évaluer la maturité par projet incluant la stratégie de test

Projet	Jira projet X
Équipe produit	Team X
Responsable AQ (QA lead)	QA lead X
Valider au plus vite/tôt possible	0
Avoir le moins de tests possible	0
Automatiser le plus possible	1
Sécurité	0
Compatibilité	1
Performance	0
Utilisabilité	0
Accessibilité	0
Localisation	1
DevOps	0
Ci/Cd	0
Environnements de test	1
Données de test	1
Mock	1
Déploiements	1
Gestion de test	1
Modularité des tests	0
Plan de test	0
Organisation Jira/SDLC	0
Amélioration en continue	0
Story documentation	0

Gestion de livraison	0
Communication/entraide	1
Gestion des anomalies	1
Traçabilité/prévention des anomalies	0
Documentation vivante à source unique	0

Stratégie des responsabilités (assurance qualité)

January 18, 2024 10:54 AM

Souvent lors de l’embauche, il n’est écrit nul part qu’elle sont vos/nos responsabilités en matière d'assurance qualité !
Cela occasion beaucoup de friction ! C’est pourquoi une stratégie des responsabilité existe !

Feuille de route

January 18, 2024 10:54 AM

OBJECTIF

Phases, étapes, Implantations et Ateliers, faire la communication du suivi (feuille de route)

- Ressource
- besoin d'un product manger (PO)
  définition du product manager ?
- inventaire des tests présents etc.
- les ateliers…
- …
- Viser :

a. Doit être **aligné avec les tests** manuels (convention, nomenclature…)
   i. un test auto est aussi un test manuel et vice versa
a. Phase 1 - **Automatisation des sanity tests** (smoke, happy, api comm & conn)
   i. Établir du fait même les fondations et les bases/modèles à suivre
a. Coller les exécutions sur les versions/livraisons

exemple d'un roadmap

Gestion de test (stratégie de test)

January 18, 2024 10:55 AM

INTRODUCTION & MISSION

🎯

Ce document résume tous les éléments à considérer afin de mettre en œuvre les efforts d'essais substantiels selon l'entente sur les conventions engagés, pour assurer efficacement un service qualité à travers les défis et les différents projets, peu importe le profil du client

C’est un résumé de l'ensemble des éléments à considérer afin de planifier des efforts de tests conséquents avec la portée, les enjeux et les différents paramètres.

Ce document vise à :

- présenter la stratégie en fonction des différents types d’essais qui seront exécutés dans le cadre du projet,
- décrire les différents éléments de la stratégie qui permettront de traiter toutes les exigences en matière d'essais.

Ce document s’adresse :

- au client qui doit s'assurer que toutes ses exigences en matière d'essais (propriétés à tester, niveaux de qualité visés, étapes à franchir, etc…) ont été prises en considération et consignées ;
- au chargé de projet qui doit documenter toutes ces exigences (risques, propriétés à tester, niveaux de qualité visés, etc.) et veiller au respect de la stratégie d’essais ;
- à l'équipe d'essais et de réalisation qui doit être informée :
- de la couverture des essais qu’il faudra prendre en considération dans le cadre de ce projet,
- des méthodes, techniques, installations et outils à utiliser pour tester les biens livrables du projet.

L’existence et le maintien de ce document pour l'ensemble du développement permettra d'établir la confiance vers la satisfaction. C'est l'un des documents importants des livrables de test. Comme les autres livrables de test, l'équipe de test le partagera avec les parties prenantes pour une meilleure compréhension de la portée du projet et des approches de test.

Notes :

- Ceci n’est pas un plan ! Ce ne sont que les ententes, conventions, règles, nos façons de faire, nos guides, nos ligne directrices ne pas confondre avec un plan de test ou la mise en œuvre de la stratégie.
- La stratégie de test s’applique qu'à l'équipe service qualité et peut servir de modèle par la suite pour l’entreprise.

OBJECTIFS

Objectifs primaires

Les objectifs primaires de tests sont de valider l'approche de tests imaginés pour assurer que l’application respecte le comportement attendu du système tel que décrit dans les spécifications.

Voici les objectifs concernés :

a) Il s’agit de bien vérifier et valider la non-régression des données/fonctionnalités existantes dans LES PROJETS TODO :

- Lien vers la base des tests auto – TODO

Qu'est-ce que nous testons?	Comment ?
Vérifier plusieurs composants pour chaque module développé ou existant	Pouvoir comparer, ajouter, modifier, visualiser, réviser, enregistrer et ajouter un commentaire ou suivi pour chaque module des projets XX TODO

b) Vérifier et valider que les User Stories du sprint courant sont couverts par des scénarios

c) Exécution des tests fonctionnels pour chaque fonctionnalité prête à tester dans le sprint

d) Vérifier et compléter le plan de test selon besoin

e) Effectuer des tests fonctionnels et/ou de non-régression sur l'env. DEV, QA et UAT.

Objectifs secondaires

Les objectifs secondaires de tests sont d'assurer que les éléments suivants fonctionnent adéquatement

Qu'est-ce que nous testons?	Comment ?
La langue	Valider que l'application réagit adéquatement lorsque l'utilisateur change la langue via l'hyperlien 'Français' ou 'English' et que l'affichage de la langue choisie se reflète au niveau des champs multilingues.
La navigation	Valider que l'ensemble des éléments permettant à l'utilisateur de naviguer au travers de l'application, fonctionne adéquatement : · menus, · fils d'arianes, · boutons, · hyperliens, · etc…

Objectifs spécifiques aux tests manuels

Modèle opérationnel de test manuel

-
- 

Owned by Michel BelangerUpdated on Feb 27, 2023
Comment utiliser GoTest/Jira dans le contexte du manuel ?
Confluence

Open preview

Objectifs spécifiques aux tests automatisés (automatisation)

Modèle opérationnel de test automatisé

-
- 

Owned by Michel BelangerUpdated on Mar 14, 2023
Les objectifs de l'automatisation de la majorité des tests de la nouvelle application sont de : Effectuer l'exécution complète des scénarios de tests à travers CI/CD afin d'assurer de la non-régression, Maximiser le temps d'exécution des tests en évitant la répétition des exécutions manuelles des différents scénarios de tests, Valider le processus
Confluence

Open preview

Analyse et diagnostic des exécutions automatisées

Les résultats des exécutions sont présentés dans le tableau de bord GoTest sous forme

graphique avec des métriques.

Voici les 3 statuts pouvant être représentés au niveau des résultats :

Non scripté	Correspond aux scénarios de tests qui ont été documentés mais dont le scripting n'est pas complété.
Succès	Correspond aux scénarios dont le résultat obtenu par le script d'automatisation correspond au résultat attendu.
Échec	Correspond aux scénarios dont le résultat obtenu par le script d'automatisation ne correspond pas au résultat attendu.
Erreur	Correspond aux scénarios dont l'exécution n'a pu être complétée à cause de problèmes critiques (ex : réseau non disponible).

PRINCIPES

Les fondations, les bases qui régissent le fonctionnement de la stratégie de test

Le 1er principe : Respect du cycle de vie (cycle complet de la vie du développement)ARCHIVED

Analyse et validation pour chaque phase du cycle en décalant vers l'amont et en accomplissant vers l'aval (shift-Left et shift-right)

Le 2eme principe : Le principe d'optimiser la quantité (éliminer le superflu)

Niveaux de test

Dans cette section, nous présentons les différents niveaux moqués pour l'ensemble des systèmes, dans le but de répartir les tests et donc de tester par niveaux.

Nous devrons soustraire les exceptions qui ne se décomposent pas en niveaux, telles que le développement non interne (tierce partie) ou l'héritage monolithe (legacy).
Ainsi, nous opterons plutôt, à ce moment-là, pour tester en moque "in/out" et/ou test de boîte noire en fonction des besoins et de l'évaluation pondérée.

L'ensemble de ces tests est à décliner pour chaque Sprint des projets.

Les niveaux de tests sont classiquement définis selon la **pyramide** de tests Agile telle que présentée ci-après :

Niveaux de test

-
- 

Owned by Michel BelangerUpdated on May 18, 2023
Niveaux de test (sharepoint.com)
Confluence

Open preview

La pyramide de test (niveaux de test)

-
- 

Owned by Michel BelangerUpdated on May 5, 2023
Définition ici : Pyramide de test (niveaux de test) (sharepoint.com)
https://alithya.atlassian.net/l/cp/nUoYPhBU
https://alithya.atlassian.net/wiki/spaces/PAQ/pages/154509377571
https://alithya.atlassian.net/wiki/spaces/PAQ/pages/154318471719
https://alithya.atlassian.net/wiki/spaces/PAQ/pages/154507313179
Confluence

Risques et inconnues

Cette section détaille les risques éventuels associés aux tests, ainsi que l'impact pouvant affecter la qualité des tests ainsi que les éléments de mitigation.

exemples :

- priorité des incidents possibles
- fréquence de changement (est-ce que l'application va changer beaucoup prochainement?)
- densité de bugs (la quantité de bugs / stabilité présente de l'application)
- surcharge des tests manuels
- complexité des exigences/besoins (est-ce que la fonction est complexe?)
- domaine connu/inconnu
- niveau d'accomplissement de l'architecture
- impacte du non fonctionnel
- valeur ajoutée
- …

No	Type de risque	Description	Impact	Mitigation
1	Documentation incomplète ou manquante	Documentation et expertises TI limitées	Moyen	Écriture des critères d'acceptances au fur et à mesure de la définition des user stories. Description fine et compréhensive des story
2	Ressources	Une seule experte au niveau du domaine d'affaire qui est en congé, maladie ou indisponibilité	Moyen	Prévoir un backup dans le domaine d'expertise de la ressource.
3	Environnement(s)	La non-disponibilité d'une plateforme de test	Élevé	Meilleure coopération entre l'équipe projet et l'équipe IT
4	Délais	Détection tardive des bugs	Moyen	-Augmenter les efforts des tests aux niveaux inférieurs (Unitaire, intégration) -Interaction entre DEV/QA pour une compréhension commune des récits -les tests de non-régression via des tests automatisés pour trouver les problèmes au plus tôt
5	Technologie	Technique non maitrisée, complexité de l'intégration de différents composants, problèmes de configuration, …	Moyen	Prévoir des formations inter-équipe, des documentations bien détaillés sur un lien partageable, prévoir des formations avec des certifications.
6	Méthode	Non résolution rapide de problèmes mis en évidence par les tests	Élevé	Intégrer la correction des bugs bloquants dans le sprint en pesant leur impact.

L*égende « Impact »:*

Faible	*Le risque est considéré négligeable et a peu d'impact sur la qualité.*
Moyen	*Le risque est considéré comme gérable et a un certain impact maîtrisable sur la qualité.*
Élevé	*Le risque est considéré comme dangereux et peut avoir un impact majeur qui nuira gravement à la qualité.*

Le 3eme principe : Hyper-automatisationARCHIVED

- Automatiser le plus possible (à valeur ajoutée)
- Automatiser les exécutions des tests automatisés
- Automatiser la documentation
- Automatiser la qualité

APPROCHES

Processus de tests

- EXPLIQUER LE PROCESSUS
- …….
- TODO

Processus de tests automatisés :

Infrastructure et Environnements de tests

Cette section va définir les environnements dont nous disposeront

Il existe plusieurs types d'environnements de test. Les environnements de test peuvent être classés en fonction de leur emplacement, de leur complexité et de leur objectif.

Il y aura 4 types d'environnement :

Environnement	Web app URL interne	Description
DEV	[url] TODO	Environnement réservé pour les nouvelles fonctionnalités et celles récemment développés, le développeur déploie lorsqu'il estime que sa version répond bien à la fonctionnalité demandée par le client.
QA	[url] TODO	Environnement réservé uniquement aux tests de régression et au test avant le passage aux tests d'acceptations. Il permet de valider que le développement du nouvel incrément ne vient pas briser les fonctionnalités déjà en place. Le déploiement pourrait être automatique ou effectué manuellement sur cet environnement
UAT	[url] TODO	Environnement qui doit être le plus proche possible de l'environnement de production. Il reproduit à l'identique l'environnement de production (nombre de machines, processeurs, mémoires, versions des applications, Base de données, …). Il permet de valider la bonne exécution de l'application avant le passage en production.
PROD	N/A	Environnement qui doit être très stable et fonctionnel. Il contiendra les fonctionnalités validés et acceptés par le client.

Environnements de test stable et sur demande

1. **Environnement de déploiement moquable**/configurable sur demande **stable** (docker) | ouvert aux branch previews
2. Pas d'environnements traditionnels dev-qa-uat-preprod…

Jeux de données

Créer des données de test de haute qualité

Les données de test de haute qualité sont plus lisibles pour les outils de test d'automatisation. Assurez-vous de le formater correctement dans un type de fichier compatible. Lorsque vous disposez de données externes, vous pouvez facilement réutiliser et maintenir vos tests. De plus, l'ajout de nouvelles données n'affectera pas le test. Bien que la création de données de test prenne du temps, il est nécessaire de consacrer du temps et des efforts à sa structure. Essayez de créer les informations au début du processus de développement afin de pouvoir les étendre au besoin pendant les tests.

- 100% Contrôle et génération des **jeux de données** pour tous les tests
- Pas d'injections direct dans les bases de données de l'app, ni des données de production et/ou anonymisées.
- Peut-être générer par API cli et/ou Front-End (le but ici est de générer les données de la même façon que l'utilisateur ou presque*)
- Les requêtes direct aux bd peuvent-être utilisées, exceptionnellement, pour modifier des états ou des switch… ex. gestion de reset, timeout, active, delay, on/off…
- Ne pas utiliser les datasets, il faut utiliser directement la source des jeux de données avec une boucle contrôlé

Outils de tests ou de support

Cette section présente le récapitulatif des outils à utiliser pour réaliser les différents niveaux de tests.

Outil	Description
GoTEST / ZEPHIR	Gestion des cas de tests
JIRA	Suivi et gestion des anomalies
GoTEST	Outil d'Automatisation des tests
Jest - TODO	Outil de tests unitaires
Gitlab - TODO	Gestion des tests et processus d'intégrations continue
Sonar - TODO	Audits de code

Test basé sur le modèle (Model Based Testing)

Le point d'entrée le digital twins de l'application (C'est ce que l'application est et fait)
On attache tous nos tests sur le modèle

Documentation vivante et Documentation intelligente

3. La documentation commence par simplifier/clarifier la source pour éviter de créer de la sur-documentation en ajoutant une explication/commentaire/description/sommaire
4. La documentation n'est pas statique elle évolue en continue et est "versionnable"
5. Pas de doublons, que des références et une seule source pour chaque documentation

Transparence sur les résultats

En aucun cas les résultats doivent être trafiqués pour plaire à l'audience, par contre voici la façon de communiquer les résultats de façons passives et positives…

Planification et gestion de livraison

Plan de test
Plan de test

ARCHIVED

-
- 

Owned by Michel BelangerUpdated on Mar 28, 2023
Un plan de test (livraison/release/delivery test plan) est un document détaillé qui décrit et

met en actions la stratégie de test, les objectifs, les ressources nécessaires, le calendrier et les critères de réussite pour assurer la qualité d'une livraison/version d’une fonctionnalité ou une suite ou une librairie ou une epic ou une story ou un logi
Confluence

Open preview

Rédaction d'un plan de test

-
- 

Owned by Michel BelangerUpdated on Feb 28, 2023
OBJECTIF, Nous verrons ici comment produire un plan de test, le documenter, l'exécuter, faire la communication du suivi, le compléter, Aide, Procédures, Utilisation, Instructions, Guide, Gabarits, Exemples… Définition : https://alithya.atlassian.net/wiki/spaces/PAQ/pages/154633076739 Comment rédiger un plan de test ? suivez l'exemple/modèle : 0001.
Confluence

Open preview

exemple GoTest :

Alithya GoTest

Cycle de vie

Pour utilisateur, design, dev, bug, test, support, DevSecOps… (un seul cycle Agile et commun)

Cycle de vie du développement

-
- 

Owned by Michel BelangerUpdated on Mar 8, 2023
Utilisez les 6 phases et 3 états pour les releases (livraisons), epic/story/bug (ex.Jira), GoTest, plan de test… c'est notre modèle de base pour tous les cycles de développement. https://alithya.atlassian.net/wiki/spaces/PAQ/pages/154320535553
Confluence

Open preview

Post mortem

Pour chaque livraison… fêter les bons coups et entreprendre des résolutions/actions dans le but de s’améliorer

Livraison et Branches/Repo

Version

MUST tous les produits, les fonctions, les api, les services, les components, les librairies, les suites doivent être versionnés pour effectuer un suivi de la qualité

le format minimum standard est [major].[minor], la mineur est à tous les changements (maintenance), la majeur est pour des changements affaire ou UI/UX ou d’année ou ajout d’une nouvelle fonctionnalité…

C’est une itération de changements 1.0, 1.1, 1.2, 1.3, 2.0, 2.1, 2.2…

Attention, il aussi le numéro/version d’un “build” ce type de versionnage n’est pas planifiable c’est un tag !
Il faut aussi comprendre que par exemple en python, nous n’avons pas de “build” donc le numéro de version est le tag du dernier push sur la branche

Il est indispensable de rattacher ces “tags” à la version et le format commun est [major].[minor].[revision].[build]

C’est pourquoi nous avons souvent plusieurs version pour une application

ex. ici nous avons l’application “APPONE“

- APPONE 3.2
- API 1.234
- CATlib 12.3.0.1234
- SettingPortal 2.1

Il est important d’avoir la liste de tous les changements et d’attacher les résultats des tests (release notes)

Catégoriser les tests (quadrant)

Planifier/Mobiliser/Organiser/Responsabilisé les parties tiers du métier

known issue management

Exécution des plans de tests

L’exécution des plans de tests est réalisée manuellement et en intégration continue lors de déploiement sur les environnements de tests

Une fois Le récit est prêt à tester, la ressource disponible (autre que celle qui a effectué la rédaction du plan de test) va prendre en charge l’exécution des cas de tests du récit.

Exécution du plan de test automatisée serait lancée à travers CI/CD

Communication

Collaboration, rôles, responsabilités, compétences et communication

Il est important d'avoir un point de contact et d’entrée dans le cycle de vie du développement, un responsable qui va faire la liaison avec le service QA et le développement de l’entreprise… Cela, pour chacune des phases, incluant la satisfaction du client !

Validation à toutes les phases

Ne pas automatiser sur l’incertitude du développement

Ne pas automatiser après le dev… commencer avant le dev pour que le dev puisse utiliser l’automatisation pour valider son dev

Les métriques et qualimétrie

- progress
- defect (rca, bug tracking)
- coverage (code, cases…)

Présentation, rencontre “état de santé“

Préparer une présentation de la stratégie ex, PowerPoint et la présenter à un cycle régulier avec un portrait de la qualimétrie santé global

TACTIQUES ET TECHNIQUES

Inclusion(s)

Types de testARCHIVED	Objectif des tests	Méthode de test	Critères d'entrée Des scénarios de test	Critères de sortie (d'Acceptation)	Responsables/ Propriétaires	Env. De tests	Jeu(x) de données

Validation unitaire de l'intégrité du code (unit test)	• Vérifier le composant, module ou unité de code et les erreurs de programmation ; • Vérifier que le code est conforme aux spécifications fonctionnelles ;	• Executer les tests unitaires sur le poste local • Automatisés (exécuté à travers le CI/CD à chaque déploiement) et c'est une condition bloquante du CI/CD	Récits fonctionnels clairement définis	• Taux de couverture de code par les tests unitaires. • Tous les tests prévus ont été exécutés. • Revue de code approuvée • La vérification des indicateurs des tests statiques (via SONAR) qui contrôle la qualité du code et le respect des standards (Respect de 80% de couverture de code)	Développeurs	DEV	...
Test fonctionnel / intégré	• Vérifier que les spécifications fonctionnelles correspondant à ce qui a été établi dans l'analyse fonctionnelle; • Intégration entre les systèmes.	• Manuels pour tester l'interaction entre les composants et révéler des régressions • Ces tests font partis du pipeline de déploiement et sont exécutés à chaque déploiement. Ils font office d'une première série de tests de régression	Tests unitaires réussis	Confirmation par l'équipe de développement que les tests fonctionnels/intégrés sont complétés avec succès.	Développeurs & Testeurs	DEV et QA	...
Test de régression du développement de la	Vérifie que le système n'est pas impacté par l'ajout ou la modification	• Manuels : à chaque fin de sprint il faut exécuter des tests de non-	Les tests fonctionnels validés avec succès	Ne pas avoir des régressions sur les fonctionnalités existants	Testeurs Manuels et automatisés	DEV et QA	...

plateforme	d'une fonctionnalité	régression sur les env. de développement. • Automatisés : amplifier la suite de non-régression automatisés avec les nouvelles fonctionnalités du sprint N-1.		(SPRINTS-1)			
Test d'acceptation	• Valider que les besoins d'affaires ont été satisfaits ;	• Manuels en exécutant les cas d'utilisation en utilisant des données valides et non valides tout en vérifiant : · Les résultats attendus avec des données valides. · Les messages d'erreur lorsque des données non valides sont utilisées.	Les résultats de l'exécution des fonctionnel /intégré sont les Résultats attendus des cas de tests	Acceptation de la livraison validée. · Tous les tests prévus ont été exécutés. · Toutes les anomalies identifiées ont été enregistrées. · Au minimum, toutes les anomalies Bloquantes et Majeures ont été corrigée d'abord par les développeurs, validée ensuite par les testeurs et enfin clôturée	• PO • Le client	DEV UAT	...
Test API	Pouvoir tester la logique métier de l'application de manière stable, indépendamment de l'interface utilisateur	• Manuel : sur le poste DEV ou QA (avec l'outil Postman) • Automatisé : Devrait être inclus comme étape sur le CI/CD et exécuté à chaque déploiement, devront couvrir les	Respect de la couverture de code	Les validations api sont concluants (les apis sont up et fonctionnels)	Dev / QA	Dev	

		routes API relatives à saraga 2.0 Doivent être les inputs des tests UI automatisés (le pre-requis est un appel des routes API)					
Test Web service							
…							

Autre(s)

Voici les autres types de tests:

Type de tests	Justificatif(s)
Test de sécurité	
Test de performance	Des tests qui peuvent être inclus pour vérifier la performance de l'application
…	

Exclusion(s)

Les types de tests suivants sont exclus :

Type de tests	Justificatif(s)
Test sur mobile et /ou tablette	Que la version web qui est disponible
…	
…	

Hypothèses (ou Facteurs de succès)

Les hypothèses sur lesquelles les efforts de tests sont basés

- Les tests fonctionnels seront exécutés sur le dev
- Les tests seront exécutés sur plusieurs navigateurs web **(Chrome,FifeFox et Edge)**
- Les tests seront exécutés dans les langues configurées ex. FR et EN
- Les tests de non-régression et d'intégrité seront exécutés dans les différents environnements

Contraintes

Les contraintes qui sont connues:

- Le déploiement vers l'env. QA nécessite un déploiement manuel depuis le pipeline de l'outil CI – TODO

Périmètre de test

Un diagramme d'architecture haut niveau : TODO

La portée de la stratégie de test comprend le test de la solution complète des systèmes afin de garantir que toutes les exigences sont remplies.

Voici les fonctionnalités à considérer :

- ….

Rôles et responsabilités

Rôles et responsabilités (besoins et attentes)

-
- 

Owned by Michel BelangerUpdated on Aug 30, 2023
Souvent lors de l'embauche, il n'est écrit nul part qu'elle sont vos/nos responsabilités en matière d'assurance qualité ! Cela occasion beaucoup de friction ! C'est pourquoi une stratégie des responsabilité existe ! Voir les définitions des rôles https://alithya.atlassian.net/wiki/spaces/PAQ/pages/1666285599 Exemple de cartographie relationnel entr
Confluence

Open preview

CYCLE DE VIE (GESTION) DES ANOMALIES

Dans le cas où une anomalie est découverte lors de l'exécution des essais d'Assurance qualité (AQ), les testeurs créent les anomalies et les assignent au DEV , après que le fix soit effectué , ils doivent faire une deuxième repasse pour revalider les defects et faire un test de regression sur le récit concerné. Si tout est ok, les testeurs doivent attendre la validation UX/UI, qui a son tour peut détecter ou non des defects sur le récit , après une revalidation passée avec succès, le récit passe en statut validation PO pour une approbation PO, si le PO ne remonte pas des anomalies , le récit est ensuite en statut terminé.

Les anomalies seront documentées dans l'outil JIRA qui permet la documentation et la gestion des billets d'anomalies dans le flow assurant le suivi de l'évolution de l'anomalie ci-dessus – TODO

Si les testeurs découvrent des anomalies, plusieurs étapes doivent être effectuées pour que cette anomalie soit corrigée.

L'analyste QA crée autant d'anomalies qu'il y a de bugs découverts

6. Documente en détail toutes les étapes menant à l'anomalie,
7. Indique la regle de gestion liée a l'anomalie,
8. Ajoute des preuves à l'anomalie reportée (capture d'ecran, …)

Chaque bug a une sévérité et une priorité

Sévérité

Sévérité	Description de la sévérité
1- Critique	Problème sérieux qui rend impossible tout test / travail supplémentaire jusqu'à ce qu'il soit résolu (« Showstopper »). § Le système est inutilisable et/ou les coûts engendrés peuvent être importants et/ou l'intégrité des données est dégradée; § La fonction échouée est inutilisable et il n'y a pas de méthode alternative acceptable pour atteindre les résultats requis. Ces anomalies doivent être corrigées en premier (1er).
2- Sévère	Une erreur significative qui empêche le système d'être opérationnel, mais d'autres tests peuvent continuer. Le système n'est pas complètement exploitable ou est instable. Il existe une solution de contournement (« Workaround »). Ces anomalies doivent être corrigées en deuxième (2e).

3- Pénalisante	Une erreur qui n'est pas bloquante (autre que sévérité 1 et 2), mais assez importante pour mener à bien le test du scénario. Une solution de contournement a été fournie pour un problème et l'erreur est toujours persistante. Le résultat attendu n'a pas été obtenu (résultat incomplet, incorrect ou inconsistant). Ces anomalies doivent être corrigées en troisième (3e).
4- Mineure	Un problème mineur est survenu et n'empêche pas le système d'être opérationnel, mais il peut gêner l'utilisateur (ex. : problème d'affichage). Les résultats attendus ne sont pas respectés, mais n'entravent pas l'utilisation de la fonctionnalité. Ces anomalies doivent être corrigées en dernier lieu.

Priorité d'une anomalie

La priorité d'une anomalie est utilisée pour refléter le niveau d'urgence à laquelle l'anomalie doit être corrigée. Les anomalies de priorité 1 et 2 doivent être résolues avant la mise en production. Quant aux anomalies avec la priorité 3, une décision doit être prise, afin de déterminer si le projet procède ou non à la mise en production, ou si elle doit être traitée en tant que problème post-implantation.

Priorité	Description de la priorité
1- Élevé	L'anomalie survenue est considérée comme une priorité absolue, elle doit être résolue pour la prochaine version de code.
2- Moyen	L'anomalie survenue est considérée comme une priorité moindre que la priorité 1, celle-ci nécessite une correction d'ici la fin du niveau de test en cours ou d'ici la prochaine livraison du code.
3- Bas	L'anomalie survenue est considérée comme acceptable pour procéder à la mise en production ou est traitée en tant que problème post-implantation.

LIVRABLE DE TEST

Les tests seront livrés via documents Word et une extraction HTML du tableau de bord de GoTest. Celui-ci contiendra l'ensemble des tests exécutés et leurs résultats.

Livrable	Format	Détail(s)	Moment de production	Fréquence	Propriétaire	Contributeur(s)	Approbateur	Destinataire(s)	Dépôt
Stratégie de tests	Ce document	Présente l'approche de tests qui a été établie pour la ou l'ensemble des livraisons	Début du projet	Document vivant; doit être bonifié et/ou ajusté au cours du projet					SharePoint
Plan de tests	Fichier Excel Exporté depuis GoTest Qui	Documentation des cas d'essais fonctionnels	Début du sprint	Doit être bonifié et/ou ajusté pour chaque	Analyste QA	Analyste QA	Analyste QA	Équipe de réalisation du projet	SharePoint

	comprend les fonctionnalités du sprint			mise à jour du RG...					
Rapport d'exécution des tests par sprint	Document partagé incluant les fonctionnalités et anomalies traitées durant un sprint.	Document incluant les fonctionnalités prévues en test, fonctionnalités testées, fonctionnalités non livrées, anomalies corrigées au sein du sprint	Début de sprint	Doit être bonifié et/ou ajusté pour chaque livraison du projet	Analyste QA	Analyste QA	Analyste QA	Équipe de réalisation du projet	Sharepoint
Rapport des tests automatisés	Document généré après l'exécution automatique des scenarios de test	Document incluant le statut des scenarios de test avec les détails des étapes de test, il permet de debuger aussi car il contient des captures et les etapes echouées.	Fin de sprint	A chaque lancement des scenarios automatisés	Analyste QA	Analyste QA	Analyste QA	Équipe de réalisation du projet	Sharepoint

Rédaction d'un plan de test

January 18, 2024 10:55 AM

OBJECTIF, Nous verrons ici comment produire un plan de test, le documenter, l'exécuter, faire la communication du suivi, le compléter, Aide, Procédures, Utilisation, Instructions, Guide, Gabarits, Exemples…

Définition : Plan de testARCHIVED

Comment rédiger un plan de test ?

suivez l'exemple/modèle : 0001.0000 (exemple)

Voici le plan de test communément dit -> la définition de terminer (DoD) :

(Ne pas oublier la référence à la stratégie de test)

Un plan de test ?

Qu'est-ce qu'un plan de test de livraison (release/delivery test plan) et pourquoi en avons-nous besoin ?

Un plan de test est un document détaillé qui décrit et met en actions la stratégie de test, les objectifs, les ressources nécessaires, le calendrier et les critères de réussite pour assurer la qualité d'une livraison/version d'une fonctionnalité ou une suite ou une librairie ou une epic ou une story ou un logiciel spécifique, etc…

L'objectif principal, bien sûr, est d'être pro-actif, prévenir, découvrir les défauts, les erreurs et toute autre lacune qui pourrait empêcher le logiciel d'agir comme prévu ou de fournir une mauvaise expérience à vos utilisateurs. Plus précisément, un plan de test garantit à votre logiciel une qualité satisfaction client

Portée (portée) et Hors de portée

Quelle est la portée de la livraison ?

Qu'est-ce qui n'est pas inclus ?

Objectif de test

quoi et pourquoi ce sera testé ?

Quel seront les tests ?

Environnement

où que ce sera testé ?

Documents de référence

Politique de la qualité

Stratégie de test

Stratégie de bug

...

Approche de test détaillée

comment ce sera testé (jeux de données)

Analyse des besoins (fonctionnel et affaire)

Analyse des solutions (technique et technologique)

Type des tests couverts

Tester tous les liens ?

Test des formulaires ?

Tests de cookies ?

Validation (HTML/CSS) ?

Test de base de données ?

Test API ?

Utilisabilité/convivialité ?

Tests de compatibilité ?

Test de performance, load, stress, sécurité ?

Smoke ?

Beta, A-B ?

Switch ?

Test de conception

Horaire des exécutions et suivi de la feuille de route

Quand ce sera testé (suivi, roadmap, périodes et dates)

Suivi des bugs

Les ressources

par qui ce sera testé ?

qui écrit les tests ?

qui fait les révisions ?

qui exécute les tests ?

qui valide les phases (définir, planifier, intégrer, diffuser, opérer, veiller) ?

qui valide le risque (bug) ?

qui valide GO-No-GO ?

qui fait le suivi des bugs ?

qui définit la portée ?

qui définit l'objectif ?

qui fait le lien de tout la documentation ?

qui fait les analyses besoins, solutions ?

qui fait le

qui s'occupe des environnements, déploiement, etc...

qui s'occupe des jeux de données

qui s'occupe des modèles de test

qui s'occupe d'automatiser

qui s'occupe d'implémenter l'automatisation dans l'infrastructure

qui s'occupe de l'investigation des exécutions automatisés qui fails

qui s'occupe de la maintenance des tests automatisés

Pass/Fail Criteria

Test Cases and Test Scenarios

Rapport final (release notes)

Rédaction de test

January 18, 2024 10:56 AM

Comment rédiger un test ?

Préalable :

Le modèle doit être terminé avant de créer, modifier un test

Gestion de test (la stratégie de test) | Principes

Définition d'un test

1. Clarté
2. Maintenabilité
3. Réutilisabilité
4. Simultanéité/autonome (unitaire)
   Gardez à l'esprit le principe de cohérence élevée

   La cohérence devrait être le principe principal de la création de test.

   Règle des scouts

   Laissez le terrain de camping plus propre que vous ne l'avez trouvé après l'utilisation.

   Définir la définition de « complété » pour chaque scénario

   Partager avec l'équipe ce qui faits qu'un scénario de test est reconnu comme « complété ». (Definition of Done)

Langue

GoTest peut être utiliser en français, donc respect les normes de la langue française du Québec.

Pour ce qui est du contenu… Se sera en Anglais seulement pour la conception/développement (+ voir comment fournir le service des rapports en français ou autres langues, au besoin)

Pourquoi en Anglais ? rep. c'est la langue officiel du développement logiciel. Et que c'est une convention+bonne pratique que de travailler dans une langue seulement pour éviter les anti-patterns

Utilisation du **Gherkin** pour le titre des scenarios

Pour aider l'uniformité nous utilisons les mots clés du Gherkin (GIVEN, WHEN, THEN) pour rédiger les scénarios

- Il est important de **dépersonnaliser** et passer par **l'évènement** en vue de définir par le **comportement** de la fonctionnalité (il faut être du point du vue de la fonctionnalité non pas du point de vue de l'utilisateur)
- **GIVEN** (**a**rrange/context) **WHEN** (**a**ct/event) **THEN** (**a**ssert/expect)
- **BAD** ex. ~~**GIVEN** I'm on login page **WHEN** I click on sign in button **THEN** I'm on the home page~~
- **GOOD** ex. **GIVEN** authentification **WHEN** authenticate **THEN** authentificated
  opposite : **GIVEN** authentification **WHEN** authenticate **THEN un**authenticated

  ***Si vous vous dites que le titre est trop évident, c'est que vous êtes dans la bonne voie

Cycle de vie

ouvert, rédaction, révision, prêt

Modèle opérationnel de test automatisé

January 18, 2024 10:56 AM

Les objectifs de l'automatisation de la majorité des tests de la nouvelle application sont de :

- Effectuer l'exécution complète des scénarios de tests à travers CI/CD afin d'assurer de la non-régression,
- Maximiser le temps d'exécution des tests en évitant la répétition des exécutions manuelles des différents scénarios de tests,
- Valider le processus d'affaire complet de manière systématique et récurrente,
- Inclure la gestion des données dans les scénarios de tests de manière automatisée.
-

……..

🎯

Kit de base, coffre à outils, etc… ready to go !

Le modèle opérationnel de test automatisé définit l'automatisation des tests Agile par l'interne du service qualité qui est branché dans toutes la phases du développement pour assurer une fluidité et une efficacité complète de l'assurance qualité chez le client.

Ce modèle d'automatisation est complet, fonctionnel, flexible et 100% compatible, agnostique du niveau de maturité ou des méthodologies de l'entreprise (Waterfall, hybrid, Agile, etc.).

- Peut-être présenté/expliqué dès maintenant
- Peut-être en vigueur le jour 1

C'est le guide complet de l'utilisation et des conventions GoTest/Jira dans le contexte de l'automatisation des tests.

Critères de succès

Due date :

Présentation de ces éléments jeudi prochain le **2 février 2023**

Estimation d'avoir terminé à 95% pour le **6 mars 2023**.

Présentation au client le **8 mars 2023**.

- ☑ Dans un premier temps, je veux avoir, pour le volet automatisation
- ☑ Kit de base qui pourrait être expliqué et utilisé dès lundi matin prochain, si on démarrait lundi.
- ☑ On verra à améliorer le kit de base demandé ici dans le futur et selon nos observations.
- ☑ C'est dans le cadre d'un nouveau service qu'on veut déployer dans plusieurs entreprises
- ☑ Je m'attends à un kit en mode « bullet point ».
- ☑ Public cible : les utilisateurs seront nos QA assignés à l'automatisation
- ☑ Nous avons déjà plusieurs de ces informations, il s'agit de les regrouper, les raffiner et les rassembler dans ce kit d'automatisation QAAS Alithya.
- En dedans d'une à 2 heures, ils devraient avoir en main tous les outils pour prendre une story ou un cas de tests et débuter l'automatisation.
- Artéfacts **très pragmatiques,** pas en mode exploratoire, pas dans du coaching, ni de la R&D… contexte qui n'est pas exploratoire, ni de coaching.

Table des matières

-

POSITIONNEMENT

Pourquoi automatiser, les raisons et les vertus

-
-
-
-

1.
2.
3.

CONVENTIONS PRINCIPALES

Principales

-
-

-
-

-
-
-
-
-
-
-
-
-
-

4.
a.
1.
2.
a.
1.
2.
3.
a.
1.
2.
3.
4. 😉

-
-
-
-
-
-

5.

6.

7.
8.
9.
10.
11.

CONVENTIONS GOTEST

GoTest

-
-
-
-
-
-
-
-

-
-
-
-

-
-
-
-

-
-
-

-
-
-
-
-

-
-
-
-
-
-
-
-
- 😉
-
-

??? Je ne comprends pas ça

Gardez les responsabilités des éléments spécifiques

Chaque composant devrait avoir une responsabilité liée à sa portée.

Principe de responsabilité unique (SRP)

Un objet doit avoir une, et une seule, raison de changer. (Single Responsibility Principle)

nommage

- Découpage par niveau pour éviter de retester les intégrations et les systèmes à chaque exécutions des tests
- Qualitatif plutôt que quantitatif
- Une tolérance zéro pour les tests qui échouent est le seul moyen de rester au top
- basculer vers la **gauche** pour automatiser, intégrer et collaborer vers le besoin du client
- basculer vers la **droite** pour automatiser, intégrer et collaborer vers la satisfaction du client
-

Pièges à éviter :

- dfg

répercussions transversales

ramifications à plusieurs niveaux

Mission / ScopeRisques & Hypothèses **Rôles / responsabilités** Processus & Cadre OutillageMéthodesLivrables

Infrastructure de testRecettesTypes de testsOutillage

DocumentationScriptingGestion des données & envir.Montée en chargeFormation

Le service n’est pas que contrôler la qualité…
Définir et positionner le contrôle, analyse et gestion de la qualité

Les ateliers, rencontres et cérémonies

La portée de l’automatisation

12. Définir la mission et la portée
13. Déterminer l’approche qui sera préconisée (approches, tactiques, techniques, méthodologies, framework)

a. Processus & Cadre
b. Déterminer notre tolérance aux risques (Risques & Hypothèses)
c. Déterminer ce qui sera automatisé versus ce qui sera testé manuellement
d. Plan de tests / Livrables

1. Valider si nous avons tous les pré-requis?
2. Définir et déterminer les outils, les environnements et les sources de données qui serviront l’automatisation
3. Qui fera quoi, quand (rôles et responsabilités dans le cycle du développement)
4. Finalement définir des outils (Outillage)

Ci/Cd and DevOps

Oui valider le plus tôt possible et souvent, mais aussi rédiger et automatiser le plutôt possible.

Un script pour tester le mobile et le web

Mise en oeuvre

5. Définition, Pré-requis

a. Les end-points du service sont opérationnels dans le cycle de vie du développement
b. SUTs
c. Jeux de données

1. Préparation/Planification

a. automatiser les sanity tests avant de commencer l’inventaire des tests manuels
b. génération de jeux de données (par api)
c. faire les happy path fonctionnel E2E
d. …

1. Réalisation/Kick-start (FAIRE, GÉRER, FORMALISER)
2. Évaluation/Audit/Maturité et Ajustement/Amélioration continue (QUANTIFIER, OPTIMISER)

Outillage

Coffre à outils, GoTest…

Contributeurs pour l'assurance qualité

January 18, 2024 2:56 PM

Rôle	Contribution pour l'assurance qualité
PO (Product Owner)	Le PO définit les critères d'acceptation pour chaque fonctionnalité et s'assure que ces critères sont respectés. Il travaille en étroite collaboration avec l'équipe de développement et les parties prenantes pour comprendre les besoins des utilisateurs et traduire ces besoins en exigences de produit.
SM (Scrum Master)	Le SM facilite la communication entre l'équipe de développement et le PO. Il aide à résoudre les obstacles qui pourraient empêcher l'équipe de respecter les critères d'acceptation et les normes de qualité.
Architecte	L'architecte conçoit l'architecture du logiciel pour s'assurer qu'il est bien construit et fonctionne correctement. Il est également responsable de la cohérence entre la mise en œuvre effective et le modèle d'architecture préconisé.
Tech Lead	Le Tech Lead est responsable de la qualité du code et de la conception technique des produits. Il travaille en étroite collaboration avec l'équipe de développement pour s'assurer que les normes de codage et les meilleures pratiques sont respectées.
Automaticien	L'Automaticien est responsable de la mise en place et de l'exécution des tests automatisés pour s'assurer que le logiciel fonctionne comme prévu.
Testeur	Le testeur exécute des scénarios de test et analyse les résultats pour identifier les problèmes et les anomalies. Il travaille en étroite collaboration avec l'équipe de développement pour corriger les problèmes identifiés.
Validateur	Le validateur vérifie que le produit ou le service répond aux normes de qualité établies. Il est responsable de la mise en œuvre de la politique d'assurance qualité et de l'organisation des démarches qualité.
Vérificateur	Le vérificateur est responsable de la conception de la stratégie de tests, de l'exécution des tests et de l'analyse des résultats. Il suit les anomalies de la qualification jusqu'à la validation.
SME (Subject Matter Expert)	Le SME est un expert dans un domaine spécifique. Il peut aider à définir les critères de qualité et à tester les fonctionnalités spécifiques du logiciel.
BA (Business Analyst)	Le BA aide les entreprises à analyser leurs processus, produits, services et systèmes pour améliorer les processus actuels et prendre des décisions rentables grâce à des informations et à l'analyse de données.
Analyste fonctionnel	L'analyste fonctionnel analyse les besoins des utilisateurs, conçoit des solutions pour répondre à ces besoins et guide les développeurs dans la réalisation de ces solutions.
Coach Agile	Le Coach Agile guide l'équipe dans l'adoption des principes et pratiques agiles, aide à résoudre les problèmes et favorise l'amélioration continue.
Coach QA	Le Coach QA guide la culture et les réflexes de l'entreprise, tous les rôles et les produits, ce dans l'ensemble des activités de la pratique assurance qualité.
Contrôleur QA	Le contrôleur QA supervise les mesures de performance de la qualité, détecte et traite les non-conformités.
Gestionnaire QA	Le gestionnaire QA élabore des procédures pour inspecter et signaler les problèmes de qualité, surveille toutes les opérations qui affectent la qualité et guide les inspecteurs et les techniciens.

Lead QA	Le Lead QA met en place tous les outils et processus pour tester les applications ou les sprints qui sont développés.
Chef de produit	Le chef de produit porte la vision des évolutions de la solution et s'assure qu'elle apporte le maximum de valeur pour les clients/utilisateurs et le métier.
PM (Project Manager)	Le PM gère le projet, y compris la planification, l'exécution, le suivi et le contrôle du projet.
CTO (Chief Technology Officer)	Le CTO est responsable du bon fonctionnement de la technologie au service de l'entreprise. Cela comprend les systèmes d'information, les serveurs de stockage de données, les applications internes, les logiciels, le site web, etc. Le CTO est responsable de la stratégie technologique globale, y compris la sélection des outils et des processus pour l'assurance qualité.
Directeurs	Les directeurs supervisent les opérations quotidiennes, établissent des politiques et des objectifs, coordonnent les activités entre les départements et guident les décisions stratégiques.
VP (Vice President)	Le VP est un dirigeant de haut niveau qui est responsable d'un département spécifique ou d'une division au sein de l'entreprise.
Client	Le client est la partie prenante qui a commandé le développement du logiciel et qui bénéficiera de son utilisation.
Utilisateur	L'utilisateur est la personne qui utilise le logiciel dans son travail ou sa vie quotidienne.
Marketing	L'équipe marketing promeut le logiciel auprès des clients potentiels et existants, et recueille des informations sur le marché et les clients pour aider à orienter le développement du produit.
DBA (Database Administrator)	Le DBA gère et maintient la base de données du logiciel, y compris la performance, la sécurité et la récupération des données.
Experts en analytics	Les experts en analytics analysent les données de l'utilisation du logiciel pour obtenir des informations précieuses qui peuvent aider à améliorer le produit.
UX (User Experience Designer)	Le designer UX se concentre sur l'expérience globale de l'utilisateur avec le logiciel, y compris la facilité d'utilisation, l'efficacité et la satisfaction de l'utilisateur. (incluant l'accessibilité)
UI (User Interface Designer)	Le designer UI se concentre sur l'apparence du logiciel, y compris la disposition, les couleurs, les polices et les images.
Rédaction	L'équipe de rédaction crée et édite le contenu écrit pour le logiciel, y compris les manuels d'utilisation, l'aide en ligne et les messages d'erreur.
Expert en langue	L'expert en langue aide à traduire le logiciel et la documentation dans différentes langues, et à s'assurer que le contenu est culturellement approprié.
PDG	Le PDG définit la vision et la stratégie globale de l'entreprise, y compris l'importance de la qualité du logiciel.
CIO/Chief Digital Officer/Chief Innovation Officer	Ces rôles sont responsables de la mise en œuvre de la stratégie numérique et de l'innovation, y compris l'adoption de nouvelles technologies pour améliorer la qualité du logiciel.
VP de la gestion des produits/Responsable des produits	Ces rôles sont responsables de la définition de la vision du produit et de la garantie que le produit répond aux attentes des clients en termes de qualité.
Gestionnaire de produit	Le gestionnaire de produit travaille en étroite collaboration avec l'équipe de développement pour définir les exigences du produit et s'assurer que le

	produit final répond à ces exigences.
Architecte en chef	L'architecte en chef est responsable de la conception de l'architecture du système pour s'assurer qu'elle est bien construite et fonctionne correctement.
Architecte logiciel	L'architecte logiciel conçoit l'architecture du logiciel pour s'assurer qu'il est bien construit et fonctionne correctement.
Ingénieur logiciel principal	L'ingénieur logiciel principal est responsable de la conception et du développement de logiciels de haute qualité.
Ingénieur logiciel senior/Développeur logiciel senior	Ces rôles sont responsables de la conception et du développement de logiciels de haute qualité.
Ingénieur logiciel	L'ingénieur logiciel est responsable du développement de logiciels de haute qualité.
Développeur logiciel	Le développeur logiciel est responsable du développement de logiciels de haute qualité.
Développeur logiciel junior	Le développeur logiciel junior est responsable du développement de logiciels de haute qualité sous la supervision d'un ingénieur logiciel senior ou principal.
Stagiaire développeur logiciel	Le stagiaire développeur logiciel est responsable du développement de logiciels de haute qualité sous la supervision d'un ingénieur logiciel senior ou principal.
Directeur principal IT	Le directeur principal IT est responsable de la gestion de l'équipe IT et de la garantie que les normes de qualité sont respectées tout au long du processus de développement.
Gestionnaire IT Infra	Le gestionnaire IT Infra est responsable de la gestion de l'infrastructure IT et de la garantie que les normes de qualité sont respectées tout au long du processus de développement.
Gestionnaire des achats	Le gestionnaire des achats est responsable de l'achat de matériel et de logiciels de haute qualité pour soutenir le processus de développement.
Gestionnaire de l'apprentissage et du développement	Le gestionnaire de l'apprentissage et du développement est responsable de la formation et du développement des employés pour s'assurer qu'ils ont les compétences nécessaires pour produire des logiciels de haute qualité.
Coordinateurs de l'apprentissage et du développement	Les coordinateurs de l'apprentissage et du développement sont responsables de la coordination des activités de formation et de développement pour s'assurer que les employés ont les compétences nécessaires pour produire des logiciels de haute qualité.
VP du marketing	Le VP du marketing est responsable de la communication de la valeur du produit aux clients, y compris la qualité du logiciel.
Responsable technique/Responsable d'ingénierie/Responsable d'équipe	Ces rôles sont responsables de la gestion de l'équipe de développement et de la garantie que les normes de qualité sont respectées tout au long du processus de développement.

Autres

- VP de l'ingénierie/Directeur de l'ingénierie : Ces rôles sont responsables de la gestion de l'équipe d'ingénierie et de la garantie que les normes de qualité sont respectées tout au long du processus de développement.
- Chef de projet d'ingénierie/Gestionnaire d'ingénierie : Ces rôles sont responsables de la

gestion des projets d'ingénierie et de la garantie que les normes de qualité sont respectées tout au long du processus de développement.

- Directeur de l'ingénierie : Le directeur de l'ingénierie est responsable de la gestion de l'équipe d'ingénierie et de la garantie que les normes de qualité sont respectées tout au long du processus de développement.
- Gestionnaire de projet technique : Le gestionnaire de projet technique est responsable de la gestion des projets techniques et de la garantie que les normes de qualité sont respectées tout au long du processus de développement.
- Gestionnaire de projet : Le gestionnaire de projet est responsable de la gestion des projets et de la garantie que les normes de qualité sont respectées tout au long du processus de développement.
- Gestionnaire de compte d'affaires : Le gestionnaire de compte d'affaires est responsable de la gestion des relations avec les clients et de la garantie que les produits répondent aux attentes des clients en termes de qualité.
- Coordinateur RH : Le coordinateur RH est responsable de la coordination des activités des ressources humaines, y compris le recrutement et la formation des employés pour s'assurer qu'ils ont les compétences nécessaires pour produire des logiciels de haute qualité.
- Coordinateur de la paie : Le coordinateur de la paie est responsable de la gestion de la paie des employés pour s'assurer qu'ils sont correctement rémunérés pour leur travail.
- Coordinateur du recrutement : Le coordinateur du recrutement est responsable de la coordination des activités de recrutement pour s'assurer que l'entreprise attire et retient les meilleurs talents.
- Spécialiste RH : Le spécialiste RH est responsable de la gestion des activités des ressources humaines, y compris le recrutement et la formation des employés pour s'assurer qu'ils ont les compétences nécessaires pour produire des logiciels de haute qualité.
- Généraliste RH : Le généraliste RH est responsable de la gestion des activités des ressources humaines, y compris le recrutement et la formation des employés pour s'assurer qu'ils ont les compétences nécessaires pour produire des logiciels de haute qualité.
- Recruteur : Le recruteur est responsable de l'attraction et du recrutement des meilleurs talents pour s'assurer que l'entreprise dispose des compétences nécessaires pour produire des logiciels de haute qualité.
- Spécialiste de l'information sur les ressources humaines : Le spécialiste de l'information sur les ressources humaines est responsable de la gestion des informations sur les ressources humaines pour s'assurer que l'entreprise dispose des informations nécessaires pour prendre des décisions éclairées.
- Gestionnaire RH : Le gestionnaire RH est responsable de la gestion des activités des ressources humaines, y compris le recrutement et la formation des employés pour s'assurer qu'ils ont les compétences nécessaires pour produire des logiciels de haute qualité.
- Gestionnaire du recrutement : Le gestionnaire du recrutement est responsable de la gestion des activités de recrutement pour s'assurer que l'entreprise attire et retient les meilleurs talents.
- Partenaire d'affaires RH : Le partenaire d'affaires RH est responsable de la gestion des relations avec les partenaires d'affaires pour s'assurer que l'entreprise dispose des ressources nécessaires pour produire des logiciels de haute qualité.
- Directeur du recrutement : Un directeur du recrutement en assurance qualité logicielle serait responsable de la compréhension des besoins de l'entreprise et de la mise en place d'un plan de recrutement. Ils développeraient des méthodes de recrutement et des bases de données, évalueraient les résultats des campagnes de recrutement et rendraient compte de leurs progrès à leurs supérieurs et à leurs opérationnels
- VP de RH (VP of HR) : Un VP des RH en assurance qualité logicielle collaborerait avec l'équipe de direction pour développer la vision et la mission de l'entreprise. Ils élaboreraient des plans stratégiques en matière de RH (recrutement, formation, rémunération, etc.) et

recruteraient, guideraient et évalueraient le travail des gestionnaires des RH

- Directeur des ressources humaines : Un directeur des ressources humaines en assurance qualité logicielle gérerait l'ensemble des services RH de l'entreprise, en tenant compte des objectifs de sa direction générale et en supervisant ses équipes. Leurs principales missions seraient de définir et piloter les stratégies et politiques RH de leur entreprise en accord avec leur direction
- Consultant en carrière : Un consultant en assurance qualité logicielle participera à l'élaboration, à la mise en œuvre et à la réalisation de stratégies d'assurance qualité et de gestion des risques. Ils conseilleraient les responsables des systèmes d'information et formuleraient les recommandations appropriées. Ils développeraient et appliqueraient également des solutions aux défaillances informatiques
- Coordinateur des affectations : Un coordonnateur des affectations en assurance qualité logicielle serait responsable de coordonner les activités liées à la qualité, de mettre en place des systèmes de gestion de la qualité et de s'assurer de leur conformité aux normes et règlements en vigueur
- Coordinateur des placements : Un coordonnateur des placements en assurance qualité logicielle définira et assurera le déploiement opérationnel de la gestion des risques et du programme de sécurité. Ainsi, le coordonnateur de la qualité piloterait et contrôlerait la mise en œuvre de la politique d'assurance qualité et du plan d'action de l'établissement
- Stratège du développement de carrière : Un stratège en développement de carrière en assurance de la qualité des logiciels serait responsable de l'élaboration et de la mise en œuvre de stratégies et de programmes de développement de carrière au sein du service d'assurance de la qualité. Ils travailleraient en étroite collaboration avec les RH et la direction pour identifier les cheminements de carrière et fournir aux employés les ressources et les possibilités nécessaires à l'avancement professionnel.
- Agent du personnel : Un agent du personnel en assurance qualité logicielle serait responsable de prendre en compte l'état de motivation du personnel, d'être capable de percevoir le niveau de sensibilisation des employés à la qualité du travail effectué et de mener des actions pour améliorer le rendement et la productivité
- Officier des ressources humaines : Un responsable des ressources humaines en assurance qualité logicielle serait responsable d'animer, de motiver et de mobiliser une équipe de collaborateurs, de concevoir et d'animer des projets et/ou des programmes de qualité, d'évaluer, de développer et de reconnaître les collaborateurs de leur organisation, de développer et d'entretenir des relations professionnelles avec les autorités administratives et la supervision, et de définir des normes de qualité. normes et indicateurs.

...

Add label

Cycle de vie du développement

January 18, 2024 10:56 AM

Utilisez les 6 phases et 3 états pour les releases (livraisons), epic/story/bug (ex.Jira), GoTest, plan de test… c'est notre modèle de base pour tous les cycles de développement.

Cycle de vie du développement logiciel (SDLC)

Un cycle est une boucle itérative…

Les systèmes d'information et l'ingénierie logicielle, le cycle de vie du développement des systèmes (SDLC), également appelé cycle de vie du développement d'applications, est un processus de planification, de création, de test et de déploiement, donc, cycle de livraison d'un système d'information. Le concept SDLC s'applique à une gamme de configurations matérielles et logicielles, car un système peut être composé uniquement de matériel, de logiciel uniquement ou d'une combinaison des deux.

Les phases

Il y a généralement six phases dans ce cycle :

1. **DÉFINIR** (analyse des exigences/spécifications/besoins/solutions),
2. **PLANIFIER** (conception, plan de test…)
3. **INTÉGRER** (développement)
4. **DIFFUSER** (mise en œuvre)
5. **OPÉRER**
6. **VEILLER** (Satisfaction, alertes et statistiques)

Les états

Il y a généralement 3 états par phase :

7. **Analyser**
8. **Faire**
9. **Valider** (tester)

Notez que “**veiller**“, à la satisfaction, focus plus sur le positif → “assurer“, “protéger”, “prendre soin de”…
plutôt que “mesurer”, “évaluer“, “contrôler“, “surveiller“, “punir ou avoir des conséquences“…

Le "flow"

La documentation/métriques scientifique

Si vous suivez la méthode du chercheur scientifique en amélioration continue, lors de chaque phase, vous obtiendrez donc le succès, la confiance et la satisfaction !

Prendre des notes (comme un chercheur scientifique) est important et sous-estimé. Les traces est le levier de l’amélioration.

Il est important de recommencer toutes les phases lorsque une phase échoue !
Cela peut paraître intense, mais ce ne l’est pas, et surtout que c’est très payant

🙂

Le principe est de repositionner rapidement le besoin en continue si il y a lieu, plutôt que d’attendre à la fin pour le réaliser et trop tarder pour réviser. (ex. de questions après

l'évolution du contexte : la priorisation, la valeur ajouté, le RoI, mise-à-jour de la documentation, etc.)

Vous remarquerez qu'en général nous passons instantanément les phases précédentes pour continuer là où nous devions continuer

Dans un cadre d'un sprint Agile, un récit (story) ou un bug devra avoir passé par toutes les phases avant de se terminer

Voici le "flow" du retour au début :

Voici différentes façons de représenter le "FLOW"
Linéaire
"SHELL" or "STAIR"

il y a aussi l'escalier montant, ce n'est qu'aller vers le haut plutôt que le bas)

"WHEEL" or "DONUT"
"INFINI" or "8" or "DevOps"
"V" or "BOL" or "Shift-Left"

D'autres profils SDLC

Il y a d'autres profils des phases et états comme le "Waterfall" par exemple, qui fait les validations/tests en tant qu'une des phases à la fin, souvent dit "UAT" (**important : ce n'est pas la définition exacte de UAT**)

ref.: Systems development life cycle - Wikipedia

Gestion de bug

January 18, 2024 10:57 AM

OBJECTIF

inclus les bugs et incidents

La gestion des anomalies (bugs) en assurance qualité du logiciel est un processus qui consiste à identifier, suivre et résoudre les anomalies dans les logiciels. La gestion des anomalies peut inclure la collecte d'informations sur les anomalies, la création de rapports d'anomalies et la mise en place de processus pour résoudre les anomalies.

- Filtrer les bugs au support à la satisfaction clientèle et en faire un suivi exemplaire
- Bug tracking
- À toutes les phases du cycle de vie

RCA

Qualimétrie (santé qualité centralisé)

January 18, 2024 10:58 AM

La qualimétrie est une méthode d’analyse de l’assurance qualité du logiciel. Elle permet d’évaluer la qualité en utilisant des mesures quantitatives et qualitatives. La santé qualité centralisé est un système de gestion de la qualité qui permet de centraliser les informations sur la qualité et d'utiliser les statistiques pour l’amélioration.

- Audition (audit)
  - Maturité (le delta en assurance qualité logicielle)
    - Critères d'évaluation de maturité des professionnels QA
  - Définition maturité de test
  - Calculatrice de maturité de test

Calculatrice de maturité de test

January 18, 2024 10:58 AM

Réaliser un diagnostic de maturité sur les activités de tests de son organisation est une chose relativement aisée ; dont l'intérêt prend toute sa dimension lorsque les coûts sont évités ou même le ROI sont affichés.

« Aisée » lorsque l'on dispose des outils nécessaires. Bien sûr. Les modèles que je choisie permet pour une analyse pertinente et complète de tous les éléments liés aux tests.

La « mise en musique » ainsi que les outils qui permettent l'analyse des données restent des compétences qui s'étayent avec les années d'expériences.

Il existe quelques méthodes de référence, j'utilise pour ma part celle de **TPI**.

Le modèle **TPI** (ou amélioration des processus de tests) a été construit à partir du savoir-faire et de l'expérience d'un grand nombre de professionnels du test.

Il sert de support à toute démarche d'amélioration des processus de tests.

Ce modèle permet de connaître la maturité des processus de tests d'une organisation à un moment donné, pour définir par la suite des étapes d'améliorations progressives et contrôlables de manière factuelle.

Les principes:

- Ils couvrent les activites de test selon plusieurs axes:
- Organisation
- Mise en oeuvre
- Outils et methodologie
- RH
- Pour trois niveaux de maturite:
- Controle
- Efficient
- Optimise

Les 16 secteurs cles:

Une matrice avec plusieurs points de controle:

fichier de maturite en PJ

Pendant l'évaluation des preuves de qualité suffisantes sont validées pour déterminer si elles sont correctes ou pas. Toute les notes doivent permettre une traçabilité avec les preuves et être justifiables. Les conclusions et commentaires doivent être rapportés et les notes sur les évaluations formelles doivent être enregistrées comme consensus entre les évaluateurs. Les notes vont automatiquement au dénominateur commun le plus bas.

1 - Notation des pratiques spécifiques et Génériques:

Complètement (Fully F) : > 85% - Preuves cohérentes convaincantes de la conformité. Le processus est systématique et généralisé. Pas de faiblesses évidentes dans la distribution, l'application ou les résultats de ce processus.

En grande partie (Largely L): > 50 % - preuves significatives de la conformité. Le processus est susceptible d'être systématique et généralisé. Il peut y avoir quelques faiblesses mineures dans la distribution, l'application ou les résultats de ce processus.

Partiellement P: >15% - quelques preuves significatives de la conformité. Le processus peut présenter des faiblesses importantes, être incomplet, non généralisé ou incompatible avec l'application ou les résultats.

Non Atteint N: <=15% - en lien avec un attribut d'un processus particulier, peu ou pas de preuves de la conformité.

NA: Non Applicable - utilisé si l'attribut du processus n'est pas applicable à l'organisation en cours d'évaluation et est donc exclu des résultats.

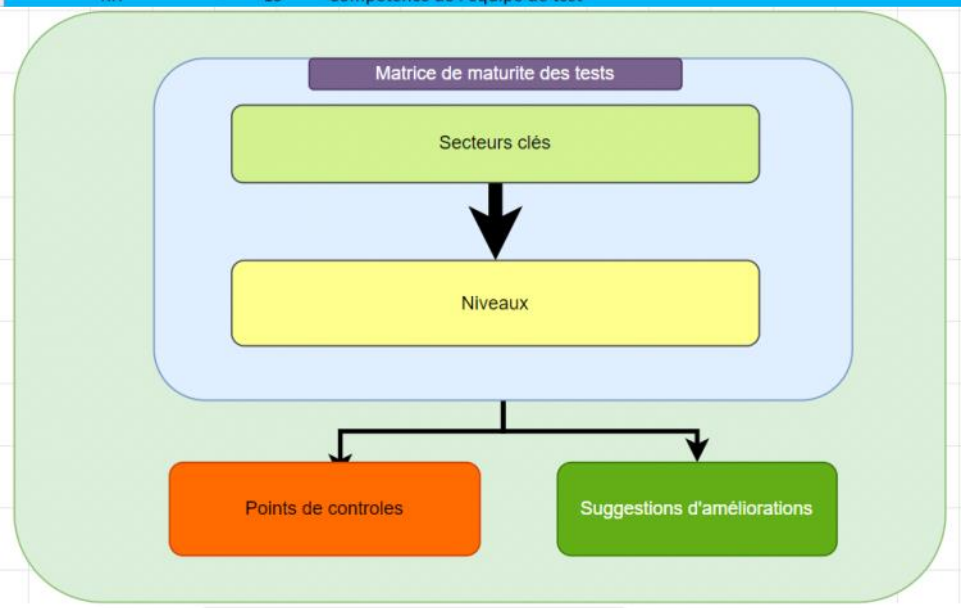
Non Evalué - utilisé si l'attribut du processus n'est pas évaluable en raison des preuves insuffisantes ou incompatibles.

Calculatrice de maturite

Audit AQ - grille de maturitéFR.xlsx
04 Apr 2023, 04:50 PM

Audit AQ - grille de maturitéFR.xlsx
04 Apr 2023, 04:50 PM

Categorie	#	Les 16 Secteur Cles
Mise en oeuve	3	Strategie de Test
	5	Communication
	6	Reporting
	7	Gestion des processus de Test
	8	Estimation et Planification
	9	Metrique
	10	Gestions des anomalies
	11	Gestion du patrimoine de test (Architecture)
	14	Conception des cas de test
Outils et Methodologie	12	Methodologie
	15	Outils de test
	16	Environnement de Test
Organisation	1	Degre d'implication des acteurs projet dans le processus de test
	2	Degre de particiaption de l'equipe de tests dans la gestion globale du projet
	4	Organisation des Tests: Equipes et procedures
RH	13	Competence de l'equipe de test



(done) GERKIN, tout un casse-tête !

January 18, 2024 1:31 PM

Gherkin : Tout Un Casse-Tête !

Introduction

Gherkin est un langage de spécification utilisé pour décrire le comportement des logiciels. Il est souvent utilisé dans les tests automatisés et est conçu pour être facile à lire et à comprendre pour les non-programmeurs. Cependant, sa mise en œuvre peut s'avérer complexe, surtout lorsqu'elle implique la collaboration entre un représentant affaire/utilisateur, un représentent assurance qualité, un représentant du développement et/ou automaticien des tests.

La Simplicité De Gherkin : Une Illusion ?

Bien que Gherkin puisse sembler simple à expliquer, sa mise en œuvre peut être un véritable casse-tête. En effet, écrire des tests en Gherkin nécessite non seulement une compréhension claire des exigences du logiciel, mais aussi une communication efficace entre toutes les parties prenantes.

L'Importance De La Collaboration

Une mise en œuvre réussie de Gherkin nécessite une collaboration étroite entre le PO, le QA et le Dev. Ensemble, ils peuvent travailler pour décrire le comportement attendu du logiciel et s'assurer que toutes les parties prenantes ont une compréhension commune des exigences.

Comment Écrire Des Tests En Gherkin

Voici quelques conseils pour écrire des tests en Gherkin :

- Utilisez des phrases simples et claires pour décrire le comportement attendu du logiciel.
- Évitez les détails techniques inutiles et concentrez-vous sur les résultats attendus.
- Utilisez des mots clés tels que Given, When et Then pour organiser vos scénarios de test.
- Évitez les scénarios de test trop longs ou trop complexes.
- Assurez-vous que chaque scénario de test est indépendant et peut être exécuté séparément.

Dépersonnaliser Les Scénarios De Test

Il est important de dépersonnaliser les scénarios de test et de se concentrer sur le comportement de la fonctionnalité plutôt que sur le point de vue de l'utilisateur. Par exemple, au lieu de dire "GIVEN I'm on login page WHEN I click on sign in button THEN I'm on the home page", il serait préférable de dire "GIVEN authentification WHEN authenticate THEN authentificated".

Pourquoi Suivre Cette Façon D'Écrire Un Scénario

Un scénario n'est pas un ensemble d'étapes, mais plutôt une description du comportement attendu d'une fonctionnalité. L'utilisation d'un seul scénario pour différents environnements de données et plusieurs validations est l'approche préconisée. De plus, il est important d'extraire les préconditions pour éviter d'avoir un GIVEN avec des étapes de reproduction. Cela permet d'isoler le test du contexte et de pouvoir exécuter le test dans différents contextes.

Conclusion

Gherkin est un outil puissant pour améliorer la communication et la collaboration au sein

des équipes de développement et de test. Cependant, sa mise en œuvre peut être un défi. En gardant à l'esprit les conseils ci-dessus, vous pouvez surmonter ce casse-tête et tirer pleinement parti de Gherkin.

***Comment écrire un récit de l'utilisateur (user story) en symbiose avec l'assurance qualité et de façon efficace ?

January 18, 2024 1:36 PM

Définition maturité de test

January 18, 2024 2:53 PM

WIP [CSNA031-88] Grille de maturité AQ - ASKIDA Jira

🎯

OBJECTIF

Le but, l'auditoire cible, prérequis…

La maturité des tests est un sujet complexe dans le monde du test logiciel. Il n'est d'ailleurs pas étonnant que pour mesurer cette maturité ou même élaborer des moyens d'industrialiser l'évaluation de cette maturité, on passe par des « experts ».

- 1 - Les modèles d'évaluation de la maturité des tests
    - Pourquoi des modèles de maturité ?
  - A - Le modèle TMMI
    - NIVEAU 1: "FAIRE"
    - NIVEAU 2: "GERER"
    - NIVEAU 3: "FORMALISER"
    - NIVEAU 4: "QUANTIFIER - MESURER"
    - NIVEAU 5: "OPTIMISER"
  - B - Le modèle TPI
- 2 - Comment réaliser une evaluation TMMI
  - A - Definition du périmètre a évaluer
  - B - Mesurer: Pratiquer le scoring

1 - Les modèles d'évaluation de la maturité des tests

Il existe actuellement 2 modèles pré-dominant pour mesurer une maturité des tests. Ces modèles sont le modèle TMMI (inspiré de CMMI) et le modèle TPI.

Pourquoi des modèles de maturité ?

Pourquoi ces modèles ont vu le jour? Les raisons sont évidemment nombreuses mais s'il faut en retenir 2, ce sont celles-ci:

- Il est important de savoir où l'on se situe si l'on veut s'améliorer

  Toute démarche d'amélioration part de constats et de mesure. Les modèles d'évaluation TMMI et TPI permettent de déterminer son niveau de maturité selon certains axes mais aussi de prioriser des axes d'amélioration.

- Le test étant très dépendant du contexte, il est nécessaire d'avoir un outil « impartial » pour pouvoir comparer diverses situations

  Comparer 2 logiciels en terme de qualité des processus de test, est aussi complexe que comparer 2 équipes Agiles. Afin d'être le plus « neutre » possible, il est nécessaire de simplifier, de proposer des modèles pouvant permettre une comparaison.

A - Le modèle TMMI

Bien que ce modèle soit appliqué au processus de développement logiciel (CMMI), il est finalement utilisé pour d'autres processus tels que l'assurance qualité. Le modèle TMMI est un modèle inspiré de CMMI.

Comme CMMI, il a 5 niveaux de maturité différents de 1 à 5 (dans les faits 4 car le niveau 1 est la non atteinte du niveau 2). Au fur et à mesure que nous progressons vers le niveau 5 à partir de 1, la variabilité et l'incohérence se réduisent. Ci-dessous les détails de 5 niveaux.

Ici, nous passerons en revue les 5 niveaux CMMI en ce qui concerne le processus d'assurance qualité et ce que tous les entrants/résultats sont attendus pour chaque niveau pour faire mûrir un processus d'assurance qualité et atteindre le niveau 5.

De même, si l'on veut se représenter les éléments obligatoires à atteindre, Chaque niveau contenant les niveaux précédents mais demandant certaines spécificités comme suit:

NIVEAU 1: "FAIRE"

A ce niveau, l'importance n'est accordée ni à la planification, ni au respect des processus, des directives et des normes.

Il n'y a pas de manière standardisée et cohérente d'effectuer une tâche. La seule chose importante à ce niveau est le respect des délais, quelle que soit la qualité du produit final et des livrables. Comme il n'y a pas de normes et de processus prédéfinis, la même tâche est effectuée de différentes manières par différentes personnes.

Cela devient encore plus non systématique et incohérent si la même tâche est effectuée différemment la prochaine fois, car aucun document disponible dans le processus ne permet de reproduire le processus. Ainsi, à ce niveau, le processus est mal maîtrisé, imprévisible et réactif.

Exemple QA

L'exemple serait que dans une organisation, bien que l'AQ soit l'une des phases du cycle de vie d'un produit, il n'y a pas de normes ni de processus définis, pas de modèles pour les livrables d'AQ - plan de test, stratégie de test, scénarios de test et les cas de tests ne sont pas standardisés.

Même si ces choses sont définies et documentées, tous les membres de l'équipe ont leur propre façon de faire les tâches et le processus n'est pas du tout cohérent. Donc, fondamentalement, il n'y a aucun contrôle sur l'AQ et c'est une phase chaotique.

NIVEAU 2: "GERER"

Dans cette phase, nous obtenons une solution au problème concernant l'indisponibilité des processus, de la méthodologie et des normes d'AQ que nous avons vu au niveau 1. On assure la mise en place des éléments « nécessaires » pour tester dans de bonnes conditions. Ces éléments sont par exemple la mise en place d'une stratégie de test, la mise à disposition d'un environnement de test, des tests conçus… Nous avons des processus, une méthodologie et des normes en place.

Les normes et les processus sont non seulement finalisés mais également bien documentés afin qu'ils puissent être réutilisés par tâches similaires qui ont été effectuées précédemment. C'est pourquoi ce niveau est dit "répétable" - car nous pouvons répéter les étapes pour effectuer le même type de travail. Ainsi, l'accent est mis sur la gestion de projet de base à ce niveau.

Exemple QA

Définir le processus et la méthodologie d'AQ globale pour différents types de tests tels que les tests fonctionnels, de données, de performance, etc. Les plans de test, les stratégies de test, les scénarios de test et les cas de test doivent tous être en place

NIVEAU 3: “FORMALISER”

Au niveau 3, les personnes sont motivées à suivre les standards et les processus définis au niveau 2. Pour cela, il faut avant tout, que les processus soient transmis à toutes les personnes impliquées. Il est necessaire d'identifié que toutes les compétences sont nécessaires pour utiliser les processus et les normes de manière efficace et efficiente et aussi si une formation est nécessaire pour cela.

Ensuite, motivez et soutenez les ressources pour suivre ces normes et processus. Ici, les personnes plus expérimentées ont besoin de partager leurs connaissances avec les autres.

L'accent est mis sur la documentation, la normalisation des processus et l'intégration.

Exemple QA

Organisez des webinaires et des sessions de formation pour permettre aux gens de se familiariser avec le processus et les normes d'AQ nouvellement définis et les motiver à les utiliser au cours de leur travail quotidien sur le projet.

NIVEAU 4: “QUANTIFIER - MESURER”

A ce niveau, les processus définis au niveau 3 sont mesurés quantitativement. Ceci est fait pour contrôler l'effort requis sur n'importe quelle tâche. Sur la base de cette analyse quantitative, les processus peuvent être ajustés si nécessaire, et cela sans dégrader la qualité du produit final.

Une analyse est effectuée en divisant le processus complet en sous-processus plus petits, puis des techniques quantitatives sont appliquées à ces sous-processus. Selon les résultats, les sous-processus peuvent être ajustés si nécessaire.

Ce niveau est appelé prévisible car, sur la base d'une expérience antérieure, nous pouvons prédire quantitativement le processus et utiliser cette prédiction pour les processus à venir.

Les domaines de processus clés du niveau 4 de CMM sont la gestion de projet quantitative et la performance des processus organisationnels.

En resumé, le processus est mesuré et contrôlé à ce niveau.

Exemple QA

Effectuer des audits réguliers serait une bonne idée ici. Cela peut inclure la vérification si les équipes suivent les processus définis, en utilisant des modèles standard, adhèrent à la méthodologie ou non.

Si vous êtes dans les tests d'automatisation, alors faire des revues de code périodiques des scripts de test d'automatisation serait un bon exemple ici.

NIVEAU 5: “OPTIMISER”

À ce niveau, des moyens innovants sont identifiés pour améliorer encore les processus et les normes prédéfinis. Il s'agit d'un processus continu.

Pour cela, nos propres processus sont surveillés et repensés en permanence en ajoutant de nouveaux outils et technologies, par des recherches et études continues et en nous tenant au courant des nouvelles informations sur le marché.

Cela peut également être réalisé en comparant d'autres organisations et en apprenant d'elles et en essayant d'améliorer notre processus en y ajoutant de nouvelles innovations. Ainsi, l'accent est mis sur l'amélioration continue des processus à ce niveau. Les principaux domaines de processus sont la gestion de la performance organisationnelle et la gestion de projet quantitative.

Exemple QA

Continuer à améliorer la méthodologie et les processus définis sur la base des résultats des audits précédents.

Sur la base de certaines études, il a été conclu que les organisations au niveau 1 peuvent dépenser 1 000 $ pour une tâche particulière, puis pour la même tâche, l'organisation au niveau 5 doit dépenser 10 $.

Récemment, dans mon organisation, il a été identifié que nous effectuons manuellement les tests de régression, ce qui nécessite la répétition manuelle du même type d'effort et consomme beaucoup d'heures qui peuvent être économisées et consacrées à d'autres travaux productifs.

Nous avons ensuite fait une preuve de concept pour automatiser le processus de test de régression à l'aide d'un outil de test d'automatisation. Le POC s'est bien passé et finalement, nous avons réussi à faire les tests de régression grâce à des scripts de test d'automatisation. Cela a permis d'économiser beaucoup d'efforts et de temps et a contribué à l'amélioration globale du processus.

Après avoir parcouru les 5 niveaux mentionnés ci-dessus, il semble difficile d'atteindre le niveau 3. Une fois qu'il est atteint, les prochains niveaux ne sont pas trop loin et difficiles à atteindre :)

Des stats de l'efficacité du TMMI appliquée (pour les amoureux des stats



)

B - Le modèle TPI

Le modèle TPI se différencie de TMMI principalement par sa forme car, au final, beaucoup de critères évalués sont communs. Dans TPI, plutôt que de proposer 5 niveaux globaux l'évaluation se fait par rapport à 16 axes. Chaque axe ayant une note allant de 0 (initial) à 4 (optimisé), le niveau 1 étant contrôlé et le niveau 2 efficient. Il est bon de noter qu'une note de 1 n'est donc pas mauvaise même si au final cela revient à 1/4. Voici ce à quoi peut ressembler un radar issue de TPI:

L'intérêt d'une telle représentation est d'être plus « fin » dans ces évaluations: on peut être très bon sur certains axes et moins sur d'autres. On arrive donc à une évaluation sous forme de radar plutôt qu'un simple « niveau » (qui lui a pour avantage d'être plus simple à appréhender).

Les axes de mesures sont variés et regroupés en 3 « familles »:

2 - Comment réaliser une evaluation TMMI

TOUT N'EST PAS OBLIGATOIRE!

Composants requis:

Ce qu'une organisation doit accomplir pour satisfaire à un domaine de processus. Doit être clairement mis en place dans les processus d'une organisation (i.e. Les objectifs spécifiques et génériques).

Composants attendus:

Ce qui va typiquement être mis en place pour satisfaire à un composant requis. Ils guident ceux qui mettent en œuvre des améliorations. (i.e. Les pratiques spécifiques et génériques)

Composants informatifs:

Fournissent des détails qui aident les organisations à comprendre comment aborder les composants ci-dessus (p. ex. des sous-pratiques, des exemples de produits d'activité, des notes, des exemples…

A - Definition du périmètre a évaluer

On doit préciser les caractéristiques de votre organisation de test et en déduire un périmètre représentatif, comme par exemple:

- Comment les tests sont ils organises ? (niveaux de tests, activités transverses, activités sous-traitees)
- Quelles sont les interactions avec les MOA, MOE, l'equipe exploitation?
- Quelles sont les differentes modes de developpement?
- Quelles sont les principales categories de projet?
- Quelles sont ls principales technologies utilisées?

Identifier les personnes a rencontrer au niveau de 2 et 3 comme suit:

B - Mesurer: Pratiquer le scoring

Pendant l'évaluation des preuves de qualité suffisantes sont validées pour déterminer si elles sont correctes ou pas. Toute les notes doivent permettre une traçabilité avec les preuves et être justifiables. Les conclusions et commentaires doivent être rapportés et les notes sur les évaluations formelles doivent être enregistrées comme consensus entre les évaluateurs. Les notes vont automatiquement au dénominateur commun le plus bas.

1 - Notation des pratiques spécifiques et Génériques:

Complètement (Fully F) : > 85% - Preuves cohérentes convaincantes de la conformité. Le processus est systématique et généralisé. Pas de faiblesses évidentes dans la distribution, l'application ou les résultats de ce processus.

En grande partie (Largely L): > 50 % - preuves significatives de la conformité. Le processus est susceptible d'être systématique et généralisé. Il peut y avoir quelques faiblesses mineures dans la distribution, l'application ou les résultats de ce processus.

Partiellement P: >15% - quelques preuves significatives de la conformité. Le processus peut présenter des faiblesses importantes, être incomplet, non généralisé ou incompatible avec l'application ou les résultats.

Non Atteint N: <=15% - en lien avec un attribut d'un processus particulier, peu ou pas de preuves de la conformité.

NA: Non Applicable - utilisé si l'attribut du processus n'est pas applicable à l'organisation en cours d'évaluation et est donc exclu des résultats.

Non Evalué - utilisé si l'attribut du processus n'est pas évaluable en raison des preuves insuffisantes ou incompatibles.

- **Exemple de notation 1: "**Surveillance et Controle du test**" = F**

  OS : Objectif specifique

  PS : Pratique specifique

  OS1 : Surveiller l'Avancement du Test par rapport au Planning = **F**

OS2 : Surveiller la Qualité du Produit par rapport au Plan et aux Attentes = **F**

OS3 : Gérer les Actions Correctives jusqu'à leur Terme = **F**

PS 3.1 : Analyser les problèmes **= F**

Sous-P 1 : Recueillir les problèmes pour analyse = F

Sous-P 2 : Analyser les problèmes pour déterminer la nécessité = F

PS 3.2 : Prendre des mesures correctives = **F**

Sous-P 1 : Déterminer et documenter les mesures appropriées... = F

Sous-P 2 : Revoir les mesures a prendre et obtenir un accord.. = F

Sous-P 3 : Renegocier les engagements avec les parties prenantes... = F

PS 3.3 : Gerer les mesures correctives = **F**

Sous-P 1 : Surveiller les actions correctives jusqu'a leur terme = F

Sous-P 2 : Analyser les resultats des actions correctives afin de determiner leur efficacite= F

- **Exemple de notation 2: "**Surveillance et Controle du test**" = P**

  OS : Objectif specifique

  PS : Pratique specifique

  OS1 : Surveiller l'Avancement du Test par rapport au Planning = **F**

  OS2 : Surveiller la Qualité du Produit par rapport au Plan et aux Attentes = **F**

  OS3 : Gérer les Actions Correctives jusqu'à leur Terme = **P**

  PS 3.1 : Analyser les problèmes **= F**

  Sous-P 1 : Recueillir les problèmes pour analyse = F

  Sous-P 2 : Analyser les problèmes pour déterminer la nécessité = F

  PS 3.2 : Prendre des mesures correctives = **P**

  Sous-P 1 : Déterminer et documenter les mesures appropriées... = F

  Sous-P 2 : Revoir les mesures a prendre et obtenir un accord.. = P

  Sous-P 3 : Renegocier les engagements avec les parties prenantes... = F

  PS 3.3 : Gerer les mesures correctives = **L**

  Sous-P 1 : Surveiller les actions correctives jusqu'a leur terme = F

  Sous-P 2 : Analyser les resultats des actions correctives afin de determiner leur efficacite= L

1 - Notation des domaines de processus

Le resultat du domaine de processus correspond a la note la plus basse mise a un objectif.

La maturité des tests (TMMI)

@Maria-Luisa Quintana (Unlicensed)

Presentation générale - Niveaux maturité 2018 -FR.pptx
26 Mar 2022, 10:47 AM

Add label

XPaths

January 18, 2024 10:05 AM

- //*[@id="vCompaniesForm"]/table/tbody/tr[5]/td[1]/input
- //*[@id="vCompaniesForm"]/table/tbody/tr[5]/td[3]/span
- /html/body/div[3]/div[2]/div[2]/div/div[2]/div[29]/div/div[1]/div[2]/a

ChroPath : obtenir les chemins xpath, xpath absolus et CSS. Modifier, inspecter et vérifier les sélecteurs XPath et CSS dans le panneau devtools lui-même.
XPath Helper : makes it easy to extract, edit, and evaluate XPath queries on any webpage.
FirePath : Extension Firebug qui ajoute un outil de développement pour éditer, inspecter et générer des XPath, CSS et JQuery. N'est pas compatible avec la version Quantum de Firefox
www.freeformatter.com : Permet de tester des XPath par rapport à un fichier XML.

I. Définition : Une expression xpath est un chemin de localistaion constituée de pas de localisation.

II. Bonnes pratiques et celles à éviter

Bonnes Pratiques	Pratiques À ÉVITER	Notes
• Écrire toujours son xpath • Utiliser un **xpath relatif**. C'est un xpath qui fait sa recherche à partir du milieu de la structure DOM HTML. Il commence par la double barre oblique (//), ce qui signifie qu'il peut rechercher l'élément n'importe où sur la page Web. • Utiliser plusieurs propriétés ou attribues pour être sûr de trouver l'élément souhaité	• Éviter l'utilisation des xpaths trouvés avec Firepath/ChroPath • Éviter d'utiliser les xpaths en faisant: Clique droit sur l'élément > Inspecter > Clique droit > Copier > Copier Xpath. • **Xpath absolu ou xpath dynamique** : utilise le chemin complet de l'élément c'est-à-dire de la racine à l'élément désiré. • (Exemple: **/html/body/div[3]/div[2]/div[2]/div/div[2]/div[3]/div/div[1]/div[2]/a**)	• Avec ces xpaths, dès qu'un niveau est ajouté ils ne fonctionneraient plus. Exemple: //html//div[4]/div[1]/span[2]/button[1]. Ce xpath = trouve le premier bouton du deuxième span de la première div se trouvant dans la quatrième div de la page html. • **Pourquoi utiliser le xpath relatif et non le xpath absolu? Avec le xpath absolu, le test échouera dès qu'un changement est fait dans le chemin de l'élément tandis que le xpath relatif peut rechercher l'élément n'importe où sur la page web.**

III. Où puis-je écrire un xpath

1. Ouvrer l'application avec chrome
2. Cliquer sur F12
3. Faire Crtl + F pour avoir le champ "Find by string, selector, or XPath"
4. Utiliser le bouton d'inspection afin de connaitre les attribues et autres informations sur l'élément à inspecter

5. Écrire votre xpath dans le champ "Find by string, selector, or XPath"

IV. Écriture des xpaths

ÉLÉMENT à Trouver	XPATH Conçu	Xpath de chropath (À éviter)	NOTES
ÉLÉMENT à Trouver	XPATH Conçu	Xpath de chropath (À éviter)	NOTES
Bouton: "Menu" Preview unavailable	//*[@class='dl-trigger js-dl-trigger hidden-md hidden-lg']//*[text()='Menu']	//button[@class='dl-trigger js-dl-trigger hidden-md hidden-lg']	SiteWeb: https://wfecd.stg.dollarama.orckestra.cloud/en-CA/
Champ "Username" en utilisant multiple propriétés (**and**)	//input[@type='email' and @id='username']	//input[@id='username']	SiteWeb: https://wfecd.stg.dollarama.orckestra.cloud/en-CA/ Ici: Nous avons utilisé deux attributs: type et id
Champ "Username" en utilisant multiple propriétés (**or**)	//*[@type='email' or @id='username']		
Bouton Sign in en utilisant la fonction **text()**	//button[@type='submit' and text()='Sign in ']	//button[@data-qa='login-submit']	SiteWeb: https://wfecd.stg.dollarama.orckestra.cloud/en-CA/ Ici: Nous avons utilisé l'attribut (type) et la fonction "text()"
Trouver le bouton "Sign in" en utilisant "**contains**"	//button[@type='submit'] **[contains(text(),'Sign ')]** ou **//button[@type='submit'] [contains(text(),'Sign in ')]**		Avec contains, vous pouvez utiliser une partie du texte ou la totalité
Bouton "Add case to cart" en utilisant **parent::** Preview unavailable	//button[div[text()='Add case' and 'to cart']]	//html//div[2]/div[1]/span[2]/button[1]	SiteWeb: https://wfecd.stg.dollarama.orckestra.cloud/en-CA/ Avec le xpath conçu en utilisant **Immediate parent : []**, on est certain que c'est le bouton "Add case to cart'.
lien " Favorites " Preview unavailable	//a[@href='/en-CA/my-account/favourites']		SiteWeb: https://wfecd.stg.dollarama.orckestra.cloud/en-CA/ Ici: Nous avons utilisé le lien
Cocher un bouton radio	//input[@type='radio' and @name='PaymentMethod' and @data-oc-change='selectCreditCardPaymentMethod']		SiteWeb: https://wfecd.stg.dollarama.orckestra.cloud/en-CA/ > Signed in > Catering, Restaurants & Hotels > Add case to cart > Cart > Checkout now. Sur cette page il y a six(06) boutons

			radio d'ou l'utilisation des trois attribues pour trouver le bouton voulu. Ici, nous avons utilisé trois attributs(type, name et "data-oc-change") pour trouver le bouton radio payer par carte de crédit car il y a six (06) boutons radios sur la page
Cocher un "checkbox" en utilisant "ancestor::" Preview unavailable	//*[input[@value='1.00$ to 1.50$' and @name='UnitPriceRefinerText[]']]/ancestor::div[@class='checkbox']		
Trouver un élément en utilisant "**preceding-sibling::**" Preview unavailable	//a[contains(text(),'compagnie1')]//parent::td[@class='datalistrow'] //preceding-sibling::td[@class='datalistrow']	//*[@id="vCompaniesForm"]/table/tbody/tr[5]/td[1]/input	Utiliser cette methode (xpath conçu) pour être certain que vous selectionnez le bon élément pour supprimer ou éditer. Note: Avec le xpath (À éviter), vous n'êtes pas sûr de supprimer la compagnie "compagnie1" mais la première compagnie de la liste.
Trouver un élément en utilisant "**following-sibling::**" Preview unavailable	//a[contains(text(),'compagnie1')]//parent::td[@class='datalistrow'] //following-sibling::td//span[text()='123-123-1234']	//*[@id="vCompaniesForm"]/table/tbody/tr[5]/td[3]/span	
utiliser "**descendant::**" Preview unavailable	//div[@class='row']/descendant::input		Ce xpath ressortira tous les champs input dont le parent est "div[@class='row']
Trouver item précis avec son prix et son nom Preview unavailable	//*[div[text()='Add case' and 'to cart ']]//parent::span//preceding-sibling::div[div/span[text()='$240.00']]//preceding-sibling::div//*[@title='Large Print Coated Playing Cards']	/html/body/div[3]/div[2]/div[2]/div/div[2]/div[29]/div/div[1]/div[2]/a	
Trouver le premier item "Add case to cart"	(//button[div[text()='Add case' and 'to cart']])[1]	//button[@class='btn btn-primary text-uppercase']	
Trouver le dernier élément avec la fonction **last()**	(//button[div[text()='Add case' and 'to cart']])**[last()]**	//html//div[34]/div[1]/span[2]/button[1]	
Trouver l'avant-dernier	(//button[div[text()='Add case' and 'to cart']])	//html//div[33]/div[1]/span[2	

élément avec la fonction **last()-1**	**[last()-1]**	]/button[1]	
Trouver le nombre de boutons "Add case to cart" avec la fonction **count()**	**count(**//button[div[text()=' Add case' and 'to cart']]**)**		Utilisez ChroPath pour voir le résultat en entrant le xpath dans le champ "QUERY" Preview unavailable
Trouver le dernier élément avec la fonction **position()**	//div[position()='10']		ce xpath pointera le dixième élement Preview unavailable

Add label

Related pages

Conventions Alithya GoTest - CONVENTIONS Alithya - Scripting Best Practices / Standards (automation)

Conventions Alithya GoTest - CONVENTIONS Alithya - Scripting Best Practices / Standards (automation)

Pratique Assurance Qualité

Often read together

XPaths - Meilleures pratiques

XPaths - Meilleures pratiques

Alithya GoTest - Fr

Often read together

Les postes (positions) et rôles

January 18, 2024 10:05 AM

Pratique Assurance Qualité Travaux en cours (WiP)Les postes (positions) et rôles

- In progress

Summarize
Share

Les postes (positions) et rôles



Owned by Michel Belanger
Last updated: 2023-03-14 16:01
1 min read15 people viewed

Définition

Poste	Rôle
Est le statut ou la position ou le titre selon un échelon	Est l'état de joué/acté par un individu

Un individu qui a une promotion de son poste aura un rôle différent avec des responsabilités différentes

exemple de postes/positions :

Niveau 1	Niveau 2	Niveau 3	Niveau 4	Niveau 5	Niveau 6	Niveau 7	Niveau 8	Niveau 9

Niveau 1	Niveau 2	Niveau 3	Niveau 4	Niveau 5	Niveau 6	Niveau 7	Niveau 8	Niveau 9
Contrôleur qualité (technicien)	Analyste qualité (architect)	Lead qualité (ninja)	Coach qualité (maître)	Directeur qualité	Vice-président qualité	Président qualité	King qualité	Oracle qualité (dieu)

Voyez les différents rôles et postes du logiciel et leur lien avec la qualité :

Rôles et responsabilités de l'assurance qualité
Postes d'assurance qualité Alithya

Rôles et responsabilités de l’assurance qualité

January 18, 2024 10:06 AM

1.4 IV. Définition des rôles d’assurance qualité

Rôles	Responsabilités au niveau des tests
Lead AQ (Test lead)	Le Lead AQ a le rôle de rédiger la stratégie de test, d’identifier les objectifs et les risques reliés aux tests, d’effectuer une analyse de l’approche des tests, d’évaluation les efforts des tests, d’identifier les ressources physiques de tests ainsi que les niveaux de tests. Il met en œuvre, prépare, planifie et contrôle l’exécution des tests et la correction des anomalies, en plus d’évaluer les critères d’entrée et de sortie de chacun de types de test. Il est responsable de générer les rapports de suivi des tests et de suivi des anomalies. Il participe aux activités de fermetures d’assurance qualité d’un projet.
Analyste de test (automatisés)	L'analyste de test identifie et définit les différents scénarios de tests. Il décrit les validations à effectuer et précise les résultats attendus.
Lead automatisation	Le lead automatisation est responsable du choix et de la gestion des outils utilisés pour fins d’automatisation. Il est également responsable de l’identification et la conception des recettes d’automatisation. Il agit également à titre de référence et de facilitateur lorsque les scripteurs rencontre des défis techniques ou technologiques liés à l’automatisation.
Scripteur	Le scripteur est un développeur de scripts automatisés
Analyste fonctionnel (Lead fonctionnel)	L’analyste fonctionnel s’assure de la compréhension fonctionnelle de l’application ciblée par les tests. Il analyse les besoins des utilisateurs et les formalise avant de proposer les solutions les mieux adaptées. Dans le projet, il lui arrive d’effectuer des tests intégrés pour valider l’intégrité des fonctionnalités.
Analyste-programmeur / Développeur	L’analyste-programmeur développe, crée, modifie et vérifie les programmes informatiques utilisés dans son entreprise. Il analyse les besoins des utilisateurs et développe des solutions informatiques, crée des logiciels ou redéfinit leurs fonctions pour optimiser leur efficacité. • En plus de programmer, il est responsable de s’assurer de la qualité du code qu’il livre
Analyste Infra (Expertise réseau) (Expertise exploitation)	L’analyste infra gère notamment la définition de la stratégie technique et des architectures, le déploiement et la gestion des matériels, des télécommunications et des outils bureautiques, la mise en oeuvre des applications et le pilotage des systèmes. • Il est impliqué dans les tests lorsque des changements sont applicables à l’infrastructure
Analyste sécurité (Expertise Cyber-sécurité)	L’analyste sécurité intervient dans la coordination de la mise en oeuvre des politiques de sécurité et le suivi du plan de contrôle de la sécurité informatique. Il participe également aux activités de veille et de conseil aux équipes métier et informatiques dans le domaine de la sécurité informatique et de la cybercriminalité, en plus de faire de la surveillance de la sécurité informatique. • Il est impliqué dans les tests lorsque des changements sont applicables et touchent les aspects de sécurité
PO (Product Owner) / représentant de l’unité	Le représentant de l’unité d’affaires est la ressource principale qui initie et qui est mandataire d’un projet. Il a pour rôle d’identifier les besoins et les requis d’affaires relatifs à la solution qui sera mise en place dans son unité. Il est responsable de valider que la solution répond à ses besoins et doit en approuver la mise en

d’affaires	production.
Fournisseur externe	Le représentant du fournisseur externe a pour rôle d’analyser la faisabilité de mettre en œuvre les besoins et exigences identifiés par le client. Pour ce faire, il développe des solutions informatiques, crée des logiciels ou redéfinit leurs fonctions pour optimiser leur efficacité. Il est responsable de livrer le code, d’assurer la fonctionnalité de la solution livrée et de respecter les ententes reliées aux tests.

(Lead ou responsable en assurance qualité du logiciel)

Le responsable de l’assurance qualité (lead) est un professionnel qui définit et coordonne la mise en œuvre de la politique d’assurance qualité (méthodes / organisation / process / audit) en y associant des indicateurs de contrôle. Il/elle veille au respect des normes et règles d’hygiène et de sécurité et en garantit l’application. Le responsable de l’assurance qualité travaille en étroite collaboration avec les équipes de développement pour s’assurer que les normes de qualité sont respectées tout au long du cycle de vie du logiciel.

Le Lead qualité a le rôle de rédiger la stratégie de test, d’identifier les objectifs et les risques reliés aux tests, d’effectuer une analyse de l’approche des tests, d’évaluation les efforts des tests, d’identifier les ressources physiques de tests ainsi que les niveaux de tests. Il met en œuvre, prépare, planifie et contrôle l’exécution des tests et la correction des anomalies, en plus d’évaluer les critères d’entrée et de sortie de chacun de types de test. Il est responsable de générer les rapports de suivi des tests et de suivi des anomalies. Il participe aux activités de fermetures d’assurance qualité d’un projet.

-

Est l'équivalent de PO (AGILE)

Un analyste d’affaires est chargé d’évaluer les besoins d’une organisation ou d’une unité d’affaires et de documenter ses processus et systèmes et d’analyser son modèle d’affaires. Il agit généralement comme un agent de liaison entre les différentes parties prenantes de l’organisation. Les analystes d’affaires sont chargés d’analyser les besoins de leurs clients d’affaires. Ils occupent généralement une fonction de liaison entre le côté affaires de l’entreprise et les prestataires de services internes et externes.

-

Le scripteur est un développeur de scripts automatisés

-

Un analyste fonctionnel est un professionnel qui travaille dans le domaine de l’informatique et qui est chargé d’analyser les besoins des utilisateurs finaux et de traduire ces besoins en spécifications fonctionnelles pour les développeurs. Les analystes fonctionnels travaillent généralement en étroite collaboration avec les développeurs pour s’assurer que les spécifications sont correctement mises en œuvre et que le produit final répond aux besoins des utilisateurs.

L’analyste fonctionnel s’assure de la compréhension fonctionnelle de l’application ciblée par les tests. Il analyse les besoins des utilisateurs et les formalise avant de proposer les solutions les mieux adaptées. Dans le projet, il lui arrive d’effectuer des tests intégrés pour valider l’intégrité des fonctionnalités.

Add label

-

L’analyste-programmeur développe, crée, modifie et vérifie les programmes informatiques utilisés dans son entreprise. Il analyse les besoins des utilisateurs et développe des solutions

informatiques, crée des logiciels ou redéfinit leurs fonctions pour optimiser leur efficacité.

§ En plus de programmer, il est responsable de s'assurer de la qualité du code qu'il livre.

Un analyste-programmeur est un professionnel qui conçoit et développe des programmes informatiques. Le travail de l'analyste-programmeur consiste à analyser les besoins des utilisateurs, à concevoir des solutions informatiques pour répondre à ces besoins et à développer des programmes informatiques pour mettre en œuvre ces solutions.

-

L'analyste sécurité intervient dans la coordination de la mise en oeuvre des politiques de sécurité et le suivi du plan de contrôle de la sécurité informatique. Il participe également aux activités de veille et de conseil aux équipes métier et informatiques dans le domaine de la sécurité informatique et de la cybercriminalité, en plus de faire de la surveillance de la sécurité informatique.

§ Il est impliqué dans les tests lorsque des changements sont applicables et touchent les aspects de sécurité.

-

L'analyste sécurité intervient dans la coordination de la mise en oeuvre des politiques de sécurité et le suivi du plan de contrôle de la sécurité informatique. Il participe également aux activités de veille et de conseil aux équipes métier et informatiques dans le domaine de la sécurité informatique et de la cybercriminalité, en plus de faire de la surveillance de la sécurité informatique.

§ Il est impliqué dans les tests lorsque des changements sont applicables et touchent les aspects de sécurité.

-

L'analyste de test identifie et définit les différents scénarios de tests. Il décrit les validations à effectuer et précise les résultats attendus.

-

Le lead automatisation est responsable du choix et de la gestion des outils utilisés pour fins d'automatisation. Il est également responsable de l'identification et la conception des recettes d'automatisation. Il agit également à titre de référence et de facilitateur lorsque les scripteurs rencontre des défis techniques ou technologiques liés à l'automatisation.

-

Le représentant de l'unité d'affaires est la ressource principale qui initie et qui est mandataire d'un projet. Il a pour rôle d'identifier les besoins et les requis d'affaires relatifs à la solution qui sera mise en place dans son unité. Il est responsable de valider que la solution répond à ses besoins et doit en approuver la mise en production.

-

Le représentant du fournisseur externe a pour rôle d'analyser la faisabilité de mettre en œuvre les besoins et exigences identifiés par le client. Pour ce faire, il développe des solutions informatiques, crée des logiciels ou redéfinit leurs fonctions pour optimiser leur efficacité. Il est responsable de livrer le code, d'assurer la fonctionnalité de la solution livrée et de respecter les ententes reliées aux tests.

-

(spécialiste en assurance qualité du logiciel)

Un spécialiste qualité est un professionnel qui travaille dans le domaine de l'informatique et qui est responsable de la mise en œuvre de la méthodologie de gestion de la qualité pour les projets informatiques. Le rôle du spécialiste qualité est de s'assurer que les produits et services informatiques répondent aux normes de qualité établies et aux besoins des utilisateurs. Le spécialiste qualité travaille en étroite collaboration avec les membres de

l’équipe du projet pour s’assurer que les objectifs du projet sont atteints et que le produit final répond aux besoins des utilisateurs.

Est un ensemble de rôles :

- Analyste en assurance qualité du logiciel
- Technicien, testeur ou contrôleur en assurance qualité du logiciel
- Automaticien en assurance qualité du logiciel

Les spécialistes qualité sont responsables de la surveillance, de l'inspection et de proposer des mesures de correction ou d'amélioration des produits finaux d'une entreprise et de ses procédés pour respecter les normes de qualités établies. Ils préparent et implémentent les politiques et les procédures d'assurance qualité et réalisent des inspections de routine et des tests de qualité. Ils identifient et résolvent les flux de travail et les problèmes de production et s'assurent que les normes et les réglementations de sécurité de l'entreprise sont observées. Il s'occupent et discutent des problèmes et proposent des solutions aux supérieurs et documentent les activités en créant des rapports d'audit. Ils font des recommandations pour améliorer et pour créer du matériel de formation et des manuels d'utilisation.

Les spécialistes de qualité élaborent également les procédures d'échantillonnage et les lignes directrices pour la collecte et le rapport des données de qualité et supervisent l'implémentation pour garantir l'efficacité des inspections. Ils analysent les réclamations des clients et les autres problèmes de non-respect et font leurs recommandations sur base de leurs découvertes. Ils garantissent que l'entreprise continue à respecter les réglementations et les exigences de qualité de l'industrie, ils analysent les résultats des audits, ils réalisent les procédures de correction appropriées et ils supervisent les procédures de gestion des risques. Les spécialistes de qualité doivent avoir au minimum un diplôme d'études secondaires ou équivalent et une certification en assurance qualité.

-

Un coach AGILE aide les organisations, les équipes et les individus le passage à Agile et, en fin de compte, augmenter la productivité.

Un Coach Agile est un professionnel qui travaille dans le domaine de l’informatique et qui est responsable de la mise en œuvre de la méthodologie Agile pour les projets informatiques. Le rôle du Coach Agile est de s’assurer que les membres de l’équipe du projet comprennent les principes de l’Agilité et qu’ils sont en mesure de les appliquer dans leur travail quotidien. Le Coach Agile travaille en étroite collaboration avec les membres de l’équipe du projet pour s’assurer que les objectifs du projet sont atteints et que le produit final répond aux besoins des utilisateurs.

Adopter les pratiques et méthodes AGILE tout en intégrant les valeurs et les mentalités AGILE est un défi sans fin. C'est là qu'un coach agile entre en jeu, il inspire à être plus efficaces, transparentes, cohérentes, permettant de meilleurs résultats pour les solutions, produits et services aux clients. C’est un excellent moyen de rationaliser, apporter des changements organisationnels importants et motiver les employés. C’est un changement de culture basé sur des principes, des pratiques et des valeurs agiles centrés sur l'humain éprouvés. Ce changement de culture aide les personnes et les organisations à continuer de prospérer dans un monde du travail en constante évolution.

Il aide aussi à faire évoluer les pratiques AGILE, être plus flexibles, transparents et efficaces. Les coachs Agiles le font à la fois en introduisant des méthodes Agiles et en encourageant un changement de culture et de mentalité sur le lieu de travail.

Lorsque l’on associe le terme "Coach" avec l’adjectif “***agile“***, le rôle du Coach s’enrichit de la culture, des valeurs, des principes, des pratiques et des outils agiles. De fait, il est un **agent**

de la gestion du changement.

C'est d'ailleurs sur ce **champs organisationnel** que les entreprises sont en demande, bien que le coach AGILE puisse également apporter des compétences additionnelles utiles et appréciées comme : **la psychologie, le management d'équipes, la facilitation, la technique ou le fonctionnel**.

« Un coach AGILE aide une équipe ou un individu à adopter, à améliorer les méthodes et les pratiques AGILE. Un coach aide les gens à repenser et à changer la manière dont ils conçoivent le développement. » *Allan KELLY*

Selon une étude de Yahoo!, les coachs agiles peuvent contribuer de manière significative à améliorer les organisations humaines. Dans cette étude, les équipes Scrum ne bénéficiant pas du soutien d'un coach augmenteraient leur productivité de seulement 35 % alors que, pour celles en bénéficiant, la productivité augmenterait de 300% ou plus.

(source :



The Role of the Agile Coach)

Un coach AGILE est un poste spécial car c'est un poste de cadre/direction actifs et présents au sein des équipes comme les coachs au hockey, il est présent sur le banc, dans les session d'entraînement, dans les vestiaires, il fait partie de l'équipe ou des équipes produits.

Cet agent du changement (amélioration) est attaché au **respect des personnes**, au **cadre sécurisant** et à la **transformation** AGILE induite par l'arrivée d'une nouvelle culture et de nouvelles pratiques.

Les **enjeux** adressés par le coach AGILE sont de **plusieurs natures**, selon Véronique MESSAGER : :

- **Systémique**, provenant de la pression exercée par l'entreprise (= le système)
- **Psychologique**, liée à la constitution du matériel psychique : le cerveau a des particularités de fonctionnement cognitif créant des perturbations émotionnelles, qui s'exacerbent dans les situations où la créativité est sollicitée (= l'humain)
- **Technique**, liée au manque de compétences et d'outils (= la technologie)

Le coach AGILE est, en somme, un **catalyseur de la transition** AGILE pour guider les équipes et l'organisation dans leur prise d'**autonomie**, de **responsabilisation** et d'**auto-organisation**.

Que fait un coach AGILE ?

Voici les tâches quotidiennes qu'un coach AGILE peut être amené à effectuer :

- Planifier et concevoir l'adoption d'AGILE dans plusieurs équipes
- Fournir des sessions de formation sur les frameworks Agiles, comme Scrum, Kanban et SAFe
- Favoriser une culture d'ouverture et de sécurité psychologique
- Coacher les leaders (comme les Scrum Masters, les Product Owners et les cadres) sur les pratiques de leadership Agile
- Développer un modèle opérationnel ou une feuille de route pour les futures pratiques Agiles
- Diriger en tant que modèle pour les valeurs Agiles
- Intégrer des équipes ou des départements Agiles dans les processus plus larges d'une entreprise principalement non Agile.
- Encourager la prolifération des meilleures pratiques Agiles entre les différentes équipes.
- Mentorat et coaching des animateurs de l'équipe Agile.
- Surmonter la résistance de la haute direction en raison d'une réduction perçue du contrôle.
- Mesurer les résultats de la transition vers Agile.
- Prépare, anime et capitalise sur des ateliers de travail collaboratifs;

- Soutien et guide par des gabarits et autres outils praticopratiques
- Organise des rencontres caractérisées par des objectifs clairs et la présence des bonnes personnes pour les atteindre
- Facilite et guide les discussions pour qu'elles soient productives.
- Concentre son énergie et son attention sur l'intégration et l'alignement des participants
- Assure une vision globale et une organisation de l'information pour qu'elle soit signifiante
- Écoute attentivement, questionne pertinemment et résume le fruit des discussions
- Cible les interventions hors sujet et les évacues respectueusement des discussions
- Identifie et adresse les bloquants
- Donne du feedback et soutien l'amélioration continue de la culture organisationnelle
- Procure un environnement sécurisant aux divers acteurs et intervient en facilitateur dans la résolution des conflits
- Agit de sorte de soutenir et bonifier l'engagement des individus, leur motivation et leur plaisir
- Assure l'adoption d'un état d'esprit agile et sa mise en œuvre
- Soutien l'identification des actions à tenir et l'élaboration du plan de réalisation
- Assure un suivi des progrès et des résultats livrés
- Plan d'accompagnement individualisé (affaires et RI)
- Activités de Team building
- Soutenir un alignement commun de la définition de la vision du produit, de la portée et de la solution minimum livrable à utiliser lors des ateliers de travail collaboratif

Quel sont les responsabilités du coach Agile face à l'assurance qualité

Peut-on être Agile sans faire de l'assurance qualité ?
Peut-on faire de l'assurance qualité sans être Agile ?

Deux visions s'opposent souvent lors des débats entre Agilistes et Qualiticiens : celle, classique, d'une Assurance Qualité indépendante, et celle, correspondant aux principes Agiles, d'une Assurance Qualité totalement intégrée au développement produit.

Comment une entreprise Agile, ou en cours de transformation Agile, peut-elle se positionner sur l'une ou l'autre de ces visions, ou encore adopter une position intermédiaire qui tire parti de la complémentarité de ces deux visions ? Et comment mettre en œuvre puis faire évoluer cette organisation de l'Assurance Qualité ?

Les enjeux qualité en contexte Agile

Pour une organisation Agile, la qualité des produits est essentielle et fait partie de la valeur délivrée. Elle passe par la responsabilisation des équipes de développement, par le respect de règles de programmation, par la mise en œuvre outillée de tests automatisés à chaque cycle de développement et par la correction systématique et prioritaire des anomalies.

Cependant, l'agilité n'est pas forcément seule, certains grands projets ont des interfaces multi-métiers dont certains ne sont pas agiles : intégration de logiciel dans du matériel.

Les tests ne font pas tout : ils ne donnent qu'un indice de confiance sur la qualité produit, et doivent être complétés d'inspections et revues de la documentation. De plus, dans certains cas, les tests sont possibles seulement en environnement réel (tir de fusée), voire tout simplement impossibles.

L'agilité n'est généralement pas non plus totalement stable et mature lorsque la transformation de l'organisation est en cours, et sa mise en œuvre nécessite un accompagnement, voire une surveillance perpétuel et encore moins la prise en considération de la qualité dans les conventions.

Enfin, le contexte peut faire que l'on doive continuer de fournir des preuves de conformité par rapport à une norme ou à un framework.

Lorsqu'on pose à un panel de personnes la question suivante : « si je vous dis Qualité, dites-moi tout ce que cela évoque pour vous », on obtient systématiquement plusieurs types de réponses :

La qualité, c'est d'avoir des produits fonctionnels, sans défaut et qui satisfont les clients et utilisateurs.

La qualité, c'est d'effectuer des tests, des vérifications et des revues pour éliminer les défauts et améliorer le fonctionnement.

La qualité, ils nous ralentissent et nous fatiguent à chercher la petite bête, à intervenir quand on n'a pas besoin d'eux et à nous imposer des règles inutiles.

Cela traduit d'une part une frustration en termes de collaboration inefficace avec des équipes dédiées, d'autre part une mauvaise compréhension.

C'est pourquoi il est utile de proposer des définitions :

- La Qualité, c'est le **QUOI**, l'objectif, le résultat recherché en termes de satisfaction du client, de fonctionnalité du produit et d'absence de défaut.
- L'Assurance Qualité, c'est le **COMMENT**, l'ensemble des activités menées tout au long du cycle de vie pour atteindre l'objectif de Qualité ci-dessus : tests, vérifications, revues, surveillance, résolution de problèmes, amélioration continue.
- L'Organisation Qualité, c'est le **QUI** : l'ensemble des acteurs qui contribuent aux activités d'Assurance Qualité ci-dessus, parfois des qualiticiens appartenant à une Direction Qualité, mais aussi des rôles Agiles, des coachs, des managers, des experts et les équipes.

On peut distinguer la Qualité **Produit** et la Qualité **Processus**, la combinaison des deux résultant en Qualité pour le client et en valeur pour l'organisation : elle maîtrisera les produits, services et processus, sera reconnue sur le marché et améliorera la compétence et la motivation de son personnel.

La problématique à résoudre, dans tous les contextes mais plus particulièrement en contexte Agile est de déterminer QUI ASSURE QUOI, en d'autres termes, quelle organisation mettre en place pour pratiquer les activités d'Assurance Qualité, au service des objectifs Qualité.

Si nous cherchons à établir une topologie des organisations Qualité en contexte Agile, nous observons une combinaison entre l'organisation de l'Assurance Qualité Produit et celle de l'Assurance Qualité Processus, selon qu'elles sont respectivement centralisées ou décentralisées.

Cela dépend à la fois des contraintes internes et externes (multi-métiers, nécessité de prouver la conformité…) et de la maturité de l'organisation et des pratiques Agiles : tests intégrés et automatisés, intégration de la dimension qualité dans la « definition of done » et les « user stories », amélioration continue portée par les rétrospectives, responsabilité des équipes et engagement des rôles Agiles envers la qualité processus.

Nous considérons donc qu'une entreprise en contexte Agile doit déterminer l'organisation et les pratiques d'assurance qualité les mieux adaptées à son contexte industriel / logiciel. Il s'agit de positionner ce fameux curseur pour la qualité produit et pour la qualité processus.

Une entreprise qui opère une transformation Agile partira souvent d'une organisation Qualité centralisée, les qualiticiens étant relativement indépendants des projets/produits.

Agilifier l'assurance qualité consistera dans un premier temps à synchroniser ses activités avec la cadence des développements, puis à progressivement transférer la responsabilité de certaines de ces activités au sein des équipes Agiles, avec une autonomie croissante de ces équipes sur la Qualité.

Le succès de l'agilification de l'assurance qualité ne consiste pas à l'éliminer purement et

simplement, bien au contraire, mais à procéder à ce transfert en mode Act-Assist-Answer, des qualiticiens vers les rôles Agiles (SM, RTE, STE) et le management, puis de ceux-ci vers les équipes.

On le voit, le positionnement du curseur est dynamique et évolue selon une stratégie d'amélioration continue.

On peut alors observer les phénomènes suivants :

Les objectifs Qualité sont clairement représentés dans les supports-clés, qui comportent un volet Qualité : Definition of done, Definition of Ready, Acceptance, User Stories, Roadmap.

La dimension Qualité est abordée dans toutes les cérémonies : sprint planning, PI planning, release review, rétrospective, et avec les interfaces clients/métiers et Ops.

Les rétrospectives sont efficaces et supportent l'amélioration continue des pratiques Qualité.

Les Quality Gates (contrôle) correspondent aux phases du cycle de développement.

Le « processus » Agile est maîtrisé, adapté et fluide et sert la qualité, pas l'inverse.

La Qualité est alors vraiment au cœur des préoccupations des équipes Agiles, vraiment l'affaire de tous, et la partie centralisée de l'Assurance Qualité répond au juste nécessaire.

L'odyssée de l'assurance qualité est immense et transversal… est-ce une tâche impossible que AGILE et QUALITÉ puisse cohabiter ensemble ?

Seul, un coach AGILE pourrait y arriver de la même façon qu'un entreprise sans coach AGILE peut arriver par elle-même à être AGILE !

Comment y arriver ?… c'est là, l'importance, qu'un Coach qualité entre en jeu



Aujourd'hui dans l'industrie du logiciel, le client est toujours à la recherche de déploiements fréquents et itératifs Ci-Cd dans des contextes DevSecOps, Shift-Left...

Si vous êtes une personne chargée de l'assurance qualité, vous devez tester un système dont les exigences changent fréquemment ou que de nouvelles exigences arrivent rapidement, ce qui entraîne des changements fréquents de code. En tant que ressource d'assurance qualité idéale, vous devez intervenir du premier au dernier jour du cycle de vie du développement (livraison), en vous assurant que tous les membres de l'équipe et les clients sont sur la même longueur d'onde (analyses et validations) tout au long du développement.

À même titre que le coach Agile, un coach qualité aide/accompagne l'organisation, les équipes, l'équipe, l'individue à assurer la qualité dans différents contextes (Agile, hybride, Waterfall, classique, moderne, ou autres), proposer des pistes de solutions, s'entraîner, etc.

De pair avec le coach AGILE, le coach QUALITÉ s'occupera de coch-er la partie/prolongement de la qualité de AGILE et de consolider la qualité dans l'ensemble de l'œuvre.

Les coachs Agiles sont parfois appelés :

- facilitateurs AGILE,
- consultants AGILE,
- conseillé AGILE
- Scrum master, flow master
- coachs Scrum,
- iteration managers,
- coach Kanban

- coach Agile d'entreprise

  Séparons cela en 3 niveaux de coach AGILE

- **Facilitateur AGILE (scrum master ou flow master)-** Travaille avec une ou plusieurs équipes à un niveau individuel et est responsable de l'implantation et du maintien des pratiques et cérémonies Agile au sein de ces équipes. (permanant)
- **Coach AGILE (project manager)-** Travaille à plusieurs niveaux d'équipe et d'organisation. Engage la haute direction, facilite le dialogue entre les départements et fait progresser l'adoption AGILE dans toute l'entreprise.
- **Coach AGILE d'entreprise (Spécialiste en amélioration/innovation relatif à la gestion du changement)-** Travaille au niveau de l'entreprise, gère le changement organisationnel et culturel, introduit des méthodologies Agile au niveau de l'entreprise et coache le leadership exécutif.

  -

  (Coach en assurance qualité du logiciel)

  Le coach en assurance qualité logiciel est un professionnel qui accompagne les équipes de développement de logiciels dans la mise en place de bonnes pratiques d'assurance qualité. Il est responsable de la formation des équipes sur les méthodologies et les outils d'assurance qualité. Il travaille en étroite collaboration avec les analystes en assurance qualité et les développeurs pour s'assurer que les normes de qualité sont respectées tout au long du cycle de vie du logiciel.

  Lorsque l'on associe le terme "Coach " avec l'adjectif "***qualité"***, le rôle du Coach s'enrichit de la culture, des valeurs, des principes, des pratiques et des outils qualité. De fait, il est un **agent de l'amélioration organisationnel**.

  Il va venir accompagner une entreprise dans son changement ou amélioration de la qualité dans la stratégie de gestion de projet. Les entreprises, les équipes et les individus qui comprennent et souhaitent intégrer cet état d'esprit ne savent souvent pas par où commencer et comment le traduire en action au quotidien, surtout dans un contexte AGILE. Être accompagné par un coach qualité sur le terrain permet un soutien au jour le jour et assure une progression constante vers l'objectif visé : assurer la qualité.

  Former les équipes aux méthodes qualités

  Nous l'avons dit, le coach qualité, munie de ses connaissances, vient former les équipes à ces méthodes de gestion de projet.

  Il va **aider à l'instauration de la qualité dans les équipes**, au **découpage des différents rôles et responsabilités de la qualité** (notamment scrum master, product owner, qa, arch, designer, analyste, dev…), et va **suivre individuellement** chacun des membres de l'équipe.

  Son rôle sera d'expliquer à chacun les principes de la qualité en fonction de son titre, et de veiller à ce que l'équipe travaille en harmonie, en suivant les préceptes définis par cette stratégie qualité.

  Accompagner une entreprise dans son changement de stratégie qualité

  Il y a 2 rôles d'un coach qualité :

- implantation (court terme)
- pérennité (permanant, à renouvellement ou long terme)
  Implantation

  Au-delà de son travail de formation d'équipe, le coach qualité s'engage dans un chantier souvent plus important : celui **d'accompagnement à l'amélioration et du changement**.

  Ce changement interne à une entreprise vient avec un but souvent plus profond. Cela peut

être **augmenter la qualité…**

Quel que soit l'objectif, le coach qualité vient accompagner les dirigeants et cadres de la société dans ce changement, car il impacte l'entreprise à tous les niveaux.

Il n'a donc, en théorie, pas vocation à travailler avec une compagnie sur le très long terme. Une fois sa mission de coaching terminée, il va en général accompagner une autre équipe ou une autre entreprise.

Pérennité

Cependant, nous préférerons garder un coach qualité (spécialiste qualité) par équipe comme pour le coach AGILE (facilitateur agile), pour assurer la pérennité causé par de la maturité, l'évolution, le changement constant et fréquent des ressources et/ou technologies.

Le coach assurance qualité est chargé de surveiller les appels et les tâches terminés dans le cadre de l'effort continu pour coach-er et encadrer et assurer une expérience d'équipe. Travaillant en étroite collaboration avec la direction et utilisant des processus d'assurance qualité standardisés, le coach fournit un soutien individuel par le biais de séances de coaching, d'analyse de maturité/audit, d'un soutien côte à côte et de formations.

Le coach assurance qualité est également chargé d'identifier les opportunités d'amélioration, de la formation, de la documentation, des processus, des rapports au niveau de l'entreprise et de l'automatisation de la qualité pour avoir un impact positif sur la satisfaction. clients, employés et produit.

Les compétences et qualités requises pour être coach qualité

Au vu de son rôle de formateur et des responsabilités qui lui incombent, le coach qualité doit faire preuve de certaines compétences bien spécifiques. Ce professionnel doit en effet être :

- **expert en communication**, pour transmettre les préceptes qualité ;
- **patient**, car les méthodes qualité ne sont pas toujours simples à comprendre et à maîtriser ;
- réactif et **capable de s'adapter** à toute situation : conflits, refus de se plier aux règles/conventions, etc.

En dehors de ces soft-skills, le coach qualité doit aussi **avoir de solides compétences techniques, être spécialiste dans le domaine**. Pour pouvoir enseigner comment s'organiser, il doit comprendre les réalités du métier.

Vous recherchez de

Vous souhaitez vous former, être entraîné et guidé pour exceller dans vos responsabilités d'assurer la qualité du logiciel dans le contexte de votre travail ?

Peu importe votre rôle et domaine d'expertise dans le domaine du logiciel !

- Oui, la maîtrise de l'assurance qualité est une culture qui demande l'éveil des réflexes passant par la discipline !
- La vue d'ensemble du cycle de vie du développement incluant définition, planification des livraisons, déploiements/opérations et la satisfaction en nous et le client en tout temps. Les différentes façons de s'améliorer pour un but commun

Le coach est là pour vous guider, même si les propositions n'ont pas toujours de sens pour vous, il vous propose des exercices de conditionnement/entraînement selon votre niveau/profil pour dépasser vos limites dans le but d'atteindre la confiance en vous.

Il révise les bases avec vous, pour uniformiser la compréhension dans le but de mieux communiquer et être efficace ensemble.

C’est quoi un coach et quel est la différence avec un coach Agile

Coach	Coach Agile

Agile versus Qualité, même chose, compétition ou collaboration ?

Peut-on être Agile sans faire de l'assurance qualité ?
Peut-on faire de l’assurance qualité sans être Agile ?

Depuis plusieurs années nous comprenons l’importance crucial d’un coach, aujourd’hui il n’est pas rare de voir un coach en Amélioration, Agile et/ou Qualité.

Le coach Agile a souvent plusieurs chapeaux, rôles et responsabilités :

- Scrum master, Flow master, Coach Flow
- Project manager,
- Chargé des livraisons,
- DevSecOps,
- Ci-Cd…
- parfois même PO, QA et plus encore…

Attention ! *ici n’est pas la définition d’un coach Agile* **!**

Le coach Qualité se retrouve souvent aussi dans la situation de “bouche-trous” ! C’est le piège le plus fréquent car le rôle de coach est très transversal et omni présent. Éliminons d’abord ce bruit pour mieux différencier Agile de Qualité !

Voyons ensemble les différences

	Qualité	Agile
Assure que la stratégie de la qualité est respecté	☑	✗
Assure que le manifeste Agile est respecté	✗	☑
Facilite et guide les discussions pour qu’elles soient productives	✗	☑
L’**autonomie** de l’équipe sur son cadre de travail agile		
Une **progression significative** vers les objectifs spécifiques sur lesquels on s’est mis d’accord		
Enseigne l’autonomie des communications et rencontres	✗	☑
Prioriser : viser d’abord les adaptations qui vont apporter des résultats en lien avec les objectifs spécifiques		
Aider l’équipe à trouver ses propres solutions : le coach ne trouve pas des solutions à la place de l’équipe, il apporte de la matière (principes, rôles et responsabilités, réunions types, affichages visuels, bonnes pratiques, …) mais sous forme de propositions plutôt que préconisations.		
Un alignement perpétuel de tous sur une même vision de ce qu’est l’agilité	✗	☑
Un alignement perpétuel de tous sur une même vision de ce qu’est l’assurance qualité	☑	✗
Propose des méthodologies et meilleurs pratiques de l'assurance qualité	☑	☑
Écrit, automatise et exécute un test	✗	✗
Un coach de la qualité peut très bien fonctionner dans les contextes hybride ou absence Agile	☑	✗
Coordonne les cérémonies/mêlées Kanban/Scrum, anime certaines réunions	✗	☑

Aide au déroulement et l'organisation de la cérémonie de la planification (flow master)	✗	☑
Aide à la conception du plan de test d'une livraison pendant la cérémonie de la planification (QA Specialist)	☑	✗
Démontre la façon proposer de faire les tests	☑	✗
Activités de Team building	✗	☑
améliorer la collaboration		
l'autonomie et l'amélioration continue des personnes et des équipes tout en conservant un cadre motivant	✗	☑
faire émerger les objectifs attendus de la transformation et les **critères de succès**		
Assurer l'autonomie du contrôle de la qualité	☑	✗
Assurer l'autonomie de l'analyse de la qualité	☑	✗
Assurer l'autonomie de la gestion de la qualité	☑	✗
Faciliter les échanges entre les différentes parties-prenantes	✗	☑
améliorer ses capacités sur un développement ou sur missions	✗	☑
motiver ses équipes et les aider à réaliser leurs projets		☑
améliorer les performances, communication ou encore leur management		
ne change pas les méthodes de travail		
effectuer des changements ou appliquer de nouvelles méthodes		
améliorer l'autonomie de l'équipe		
optimisant la communication ou les méthodes de travail		
il se doit de trouver des méthodes pour faciliter la vie de ses équipes		
l'environnement de travail soit propice à l'émergence des bonnes idées		
Préparer et donner de la formation et des ateliers de perfectionnement aux équipes de travail et à leurs dirigeants. Participer à la mise en place des principes et pratiques Agiles au sein de l'organisation. Établir des plans d'accompagnement auprès des équipes et des individus en fonction des objectifs de performance à atteindre, du contexte d'intervention et du niveau de maturité Agile actuelle. Savoir cerner la vision globale des besoins, problèmes, opportunités et enjeux d'un projet agile. Agiliser les processus de gestion de projet (planification, suivi, reddition de comptes, coût, etc.). Intégrer la synchronisation multi équipes ou multi projets agiles; Assurer la durabilité de la transformation organisationnelle agile;		
Facilitateur de la communication et dans la résolution des conflits	?	?
Assure un suivi des progrès et des résultats livrés	?	?

Maintenant que nous connaissons ce qu'est un coach et un coach Agile… Voyons maintenant, le COACH en **assurance qualité** ?

Même si le métier se résume à un seul poste, le coach qualité se doit d'avoir plusieurs atouts, plusieurs postures **afin de faire face à tous types de problèmes**. Il ne peut pas se contenter d'avoir une seule corde à son arc, **il se doit d'être polyvalent avec de nombreuses ressources**, voici quel rôle un coach qualité doit savoir incarner :

- **Facilitateur,** ce rôle est peut-être le plus important pour ce coaching. En effet, **il se doit de trouver des méthodes pour faciliter la vie de ses équipes**. Trouver des solutions, prendre les bonnes solutions, faciliter les échanges sont des actions primordiales. En plus d'aider, il doit faire en sorte que **l'environnement de travail soit propice à l'émergence des bonnes idées** en créant un cadre de vie sain et bienveillant pour ses salariés. Pour finir, il a aussi le rôle d'assurer des présentations lors de conférences ou d'ateliers.
- **Coach professionnel,** en effet, il doit inspirer et faire en sorte d'aller chercher les meilleurs de ses équipes et de ses salariés. Que ce soit en étant à l'écoute, en accompagnant, ou même en inspirant, **le coach qualité intègre tout son milieu dans parcours et son développement**. Il **renforce le sentiment de sécurité auprès de ses salariés en créant un cadre propice à la transformation** de ces derniers. En effet, dès son arrivée le changement est à venir, il est donc important de faire les choses en accompagnant au maximum les équipes.
- **Mentor,** le but étant de **partager ses connaissances et ses compétences**. En effet, le mentoring est très majeur dans ce genre de métier. Les employés doivent voir ce rôle comme celui d'un **réel guide à qui l'on peut faire confiance pour évoluer et améliorer ses capacités.**
- **Formateur,** le coach qualité devra faire comprendre à ses équipes le but. **La qualité dans le management reste nichée**, il n'est de ce fait pas rare que certains employés ne soient pas habitués à ces méthodes. Aucun problème, le but n'est pas de les stigmatiser, **mais bien au contraire de les former.** Le coach devra faire en sorte que toutes les équipes comprennent et acceptent ce type de management **qui tend à renforcer les liens et de créer des groupes plus soudés et plus efficaces.**
- **Consultant,** même si ce rôle est un hybride de directeur et d'un responsable, le coach qualité se doit d'avoir une figure d'autorité et par conséquent d'être l'initiateur des actions de ses équipes. **Ce dernier doit enclencher la dynamique du changement et faire en sorte que les choses bougent autour de lui tout en respectant le rythme.** Comme énoncé auparavant, le fait de **prendre des décisions, d'échanger, et de conseiller contribue à cela e**t aident les salariés à gagner en confiance vis-à-vis de leur coaching.
- **Ambassadeur** de la qualité, ne personne qui respire et qui vit selon un état d'esprit qualité; qui permets aux autres d'en comprendre et d'en apprécier les bénéfices; qui guide les gens de tous les niveaux à cet effet et qui le mets en pratique.

Toutes ces facettes représentent parfaitement ce à quoi un coach qualité doit faire face pour réussir sa mission. C'est un profil et un métier très spécifique qui, lorsqu'il est bien fait, **crée un réel avantage concurrentiel pour l'entreprise qui l'embauche.**

Assurance qualité versus contrôle de la qualité ?

Le contrôle de qualité existe même si tu ne fais pas d'assurance qualité !

C'est le concept de cause et conséquence…

cause => L'utilisateur utilise une application de qualité médiocre

conséquence => L’utilisateur n’est pas satisfait, il cherche maintenant une autre solution

L'assurance qualité consiste à prévenir, à garantir la confiance et à viser la satisfaction vers le succès de l’utilisation

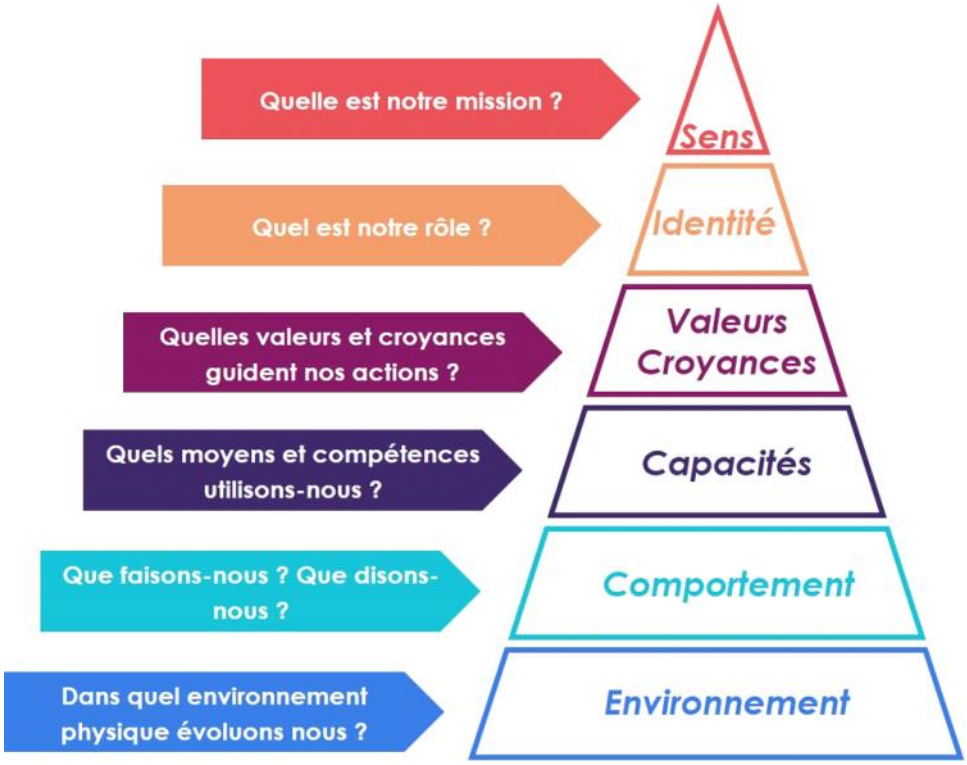
-

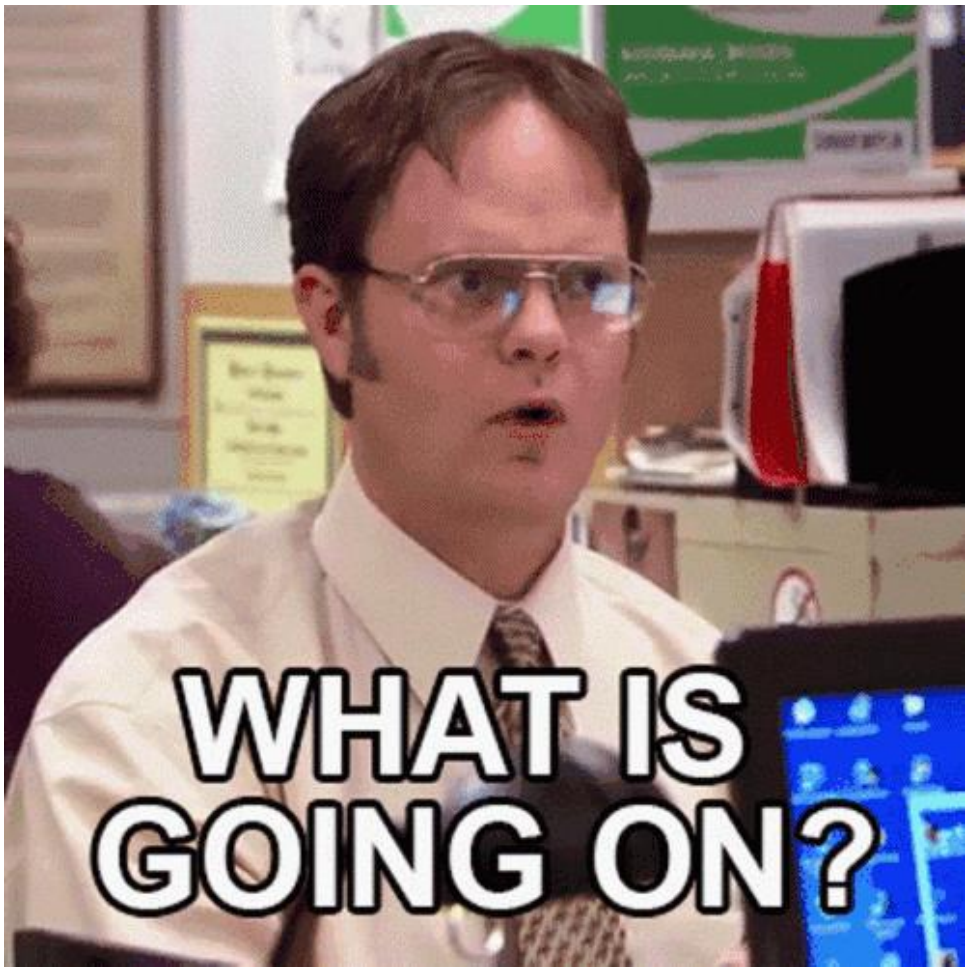
(done) E2. Faire rayonner le côté sombre de la pyramide - Les racines

January 18, 2024 10:14 AM






Quelle est notre mission ?
Sens
Quel est notre rôle ?
Identité
Quelles valeurs et croyances guident nos actions ?
Valeurs Croyances
Quels moyens et compétences utilisons-nous ?
Capacités
Que faisons-nous ? Que disons-nous ?
Comportement
Dans quel environnement physique évoluons nous ?
Environnement


WHAT IS GOING ON?

Mais c’est quoi le concept d’une pyramide au juste ?

Oublions un peu la qualité et les tests. Voici un exemple par assemblage, fondations et échelons… Cette pyramide nous démontre les étapes d’une **fondation** pour atteindre le but, le “**sens**” de la pyramide. On voit qu'il est difficile d'atteindre le "sens" si on n'a pas les autres niveaux... vous remarquerez, aussi, que tous les niveaux ont la même importance, sinon le “sens“ de la pyramide s'écroulerait (n'existerait pas) sans eux !

Si on voulait une pyramide de test avec les 3 niveaux, il faudrait dire que "SYSTEM" c'est comme "SENS" et qu'il faut passer par les étapes (ou marches d'escalier) de la fondation jusqu'à arriver à "SYSTEM" pour que la pyramide prenne tout son "SENS". Par exemple, on ne pourrait pas commencer par assembler le toit (système) avant le plancher (unitaire) et les mures (intégration) par exemple… Posons-nous la question, est-ce le cas avec la pyramide des niveaux de test ? 😉

Pourquoi alors la pyramide des niveaux de test est triangle ?

Il y a plusieurs raison de vouloir un triangle ?

1. priorisation
2. hiérarchie, position
3. niveau
4. économie
5. croissance et décroissance
6. ordre
7. …

Bien oui, pour la pyramide des niveaux de test, la seule raison est d'évidence **quantitative** ! Donc à titre indicateur, si à chaque fois que vous devez tester unitairement vous devez aussi tester son intégration et dans les systèmes, c’est que vous devriez revoir votre découpage



Nous voyons souvent la pyramide avec un indicateur que c’est mieux de faire des tests unitaires :

- Foudroyant (Fast/Rapid),
- Isolé (Isolated/Independent),
- Répétable (Repeatable),
- Souverain (Self-Validated/Verification),
- Temps opportun (Timely/Carefully),

Mais c’est le découpage de l’**ensemble de la pyramide** qui fait que c’est F.I.R.S.T. et non pas parce qu'unitaire c’est mieux que intégration ou système.

Maintenant, si on parlait de l’ordre de conception/rédaction/exécution… et oui, si je vous disais que la meilleur pratique est par l'ordre suivant :

8. exécution de la validation **système**
9. ensuite exécution de la validation **intégration**
10. et finalement exécution de la validation **unitaire**

J’entent déjà le dev dire l’inverse



Si je vous pose la question de que valideriez-vous en premier : La centrale électrique, la distribution, le système électrique, la prise, l’ampoule, ou la résistance (lumière) ? Oui on

peut créer une résistance et la tester sans le reste et même en parallèle… Mais sans contexte fonctionnel (intégration et système) je me demande quel est le sens de tester ! Dans la pyramide, l’ordre de conception/rédaction/exécution sera inverse, donc, de haut en bas.

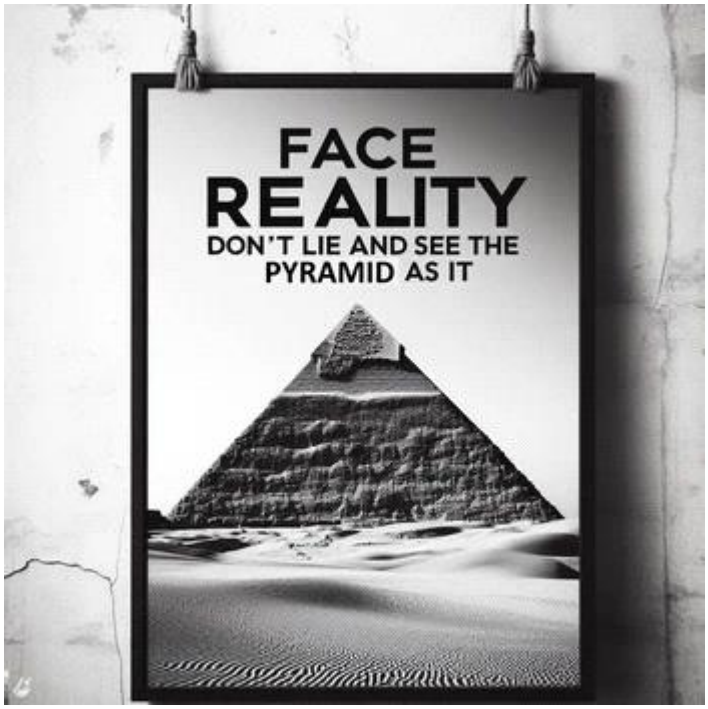
Bref, ce n'est pas par priorisation, ni fondement, ni objectif quantitatif... c'est seulement représentatif du fait que **quand on découpe un test par niveaux** il y aura, inévitablement, par exemple une validation système pour plusieurs validations d'intégration pour de nombreuses validations unitaires.

à suivre dans la prochaine épisode…

Add label

(done) E3. Faire rayonner le côté sombre de la pyramide - La réalité

January 18, 2024 10:15 AM



Est-ce pour tous les contextes/réalités de développement ?

La confrontation du legacy, les 3rd parties et les applications en boîte noire ! Comment pouvons-nous appliquer la pyramide dans un contexte où le développement est fermé, monolithe ou que ce n'est que de la configuration, comme des CRMs ? Nous pouvons seulement valider les spécifications/requis/exigences à la fin, une fois livrés. Voilà, si l'idée de la pyramide ne repose que sur les niveaux… Donc, pas de niveaux = pas de pyramide.

Faire de la fuite

Parfois, on décide de tester, on arrive à convaincre les "hautes sphères" du bénéfice et qu'il faut y consacrer du temps (et donc de l'argent), mais :

- *"Les test unitaires, on n'y arrivera pas, notre code est trop complexe"* (trop couplé)
- *"Les tests d'intégration, c'est compliqué à mettre en place, autant vouloir déployer l'application au complet."*
- *"Le mieux, ça serait quand même d'avoir des tests proches de ce que fait notre utilisateur ! Et ça sera plus simple à tester."*

Et nous voilà partis pour déployer l'artillerie lourde : les tests de bout en bout (généralement des tests d'interface graphique type *Selenium*):

- *"Ça va être génial ! On va valider toute l'application en quelques tests, on pourra même montrer les rapports au métier et aux testeurs (pour peu qu'on mette du Cucumber ou du Serenity et faire du BDD). Ils vont nous adorer* ????"

Après quelques semaines / mois, on se rend compte que ce n'était peut-être pas la meilleure idée qu'on ait eue :

- les tests sont lents: *“4h pour 150 tests”*,
- échouent par intermittence : *“Pourquoi c’est rouge ? Probablement Selenium, relance pour voir, met un “retry” de 10 pour être certain“…”*,
- Mais pire que tout, avoir un environnement opérationnel complet et des jeux de données stables relève du casse-tête, voire de la mission impossible ????
- Nous (DEV ou QA) ne sommes pas des experts en automatisation, les QA ne savent pas coder, nous aurions besoin d’un bon rare SDET (software development engineer in test) , au pire un Spécialiste Qualité.

  On investit d'autant plus qu'on se dit qu'il ne devrait pas manquer grand-chose et qu'il serait dommage de tout jeter maintenant. Effectivement, on peut améliorer les choses, mais on a l'impression à ce moment, que le retour sur investissement n'en vaut plus la chandelle et qu'on a l'impression d'être piégé de partout. Maintenant, **la pyramide** arrive comme un sauveuse, une **super héroïnes** !!! … qui se transformera sneaky, rapidement en une super vilaines 😳...souvent après avoir entendu ceci :

  “On va être Agile en commençant par respecter le plus important de la pyramide, concentrons-nous sur les tests unitaires de code qui sont plus faciles à automatiser finalement pour nous les Dev. avec un 70% dans SonarQube, voilà… ah oui, et un peu d’intégration avec Postman en semi-automatique pour la galerie ! …Oublions les tests Système, UI/UX, Intégration, la compatibilité, sécurité, déploiement, accessibilité, UAT et tous les autres tests quoi ! On va le faire faire manuellement par les analystes fonctionnels, business ou QA, ils s'organiseront avec ça, ce sera moins compliqué et ils ont l'habitude de faire de la magie. On doit livrer hier de toute façon, au pire on embauchera plus de QA… Comme on dit, la qualité c'est l'affaire de tout le monde, l’architecte, le PO et le SM pourraient aussi faire des tests, non ?!“

  J’imagine plus d'un sourire "jaune" en lisant ceci, mais rassurez-vous : vous n'êtes pas seul !

  Face à la réalité du sur-testage malgré la bonne volonté de suivre l’idée de la pyramide.

  Même si on veut faire plus de tests unitaires on se retrouve souvent à re-tester tout en intégration. Surtout que l’on ne fait pas de tests système ! On fait des validations Dev, QA, Métier (acceptation) toujours en intégration 2, 3, 4, 5 fois les mêmes tests dans différents environnements quand bien même que ce n’est pas un test d’environnement ! Non seulement refaire les mêmes validations mais aussi refaire en une tonne de copies → l'écriture/couverture. Ce, même si nous sommes en tests automatisés, on veut revalider manuellement pour être certain. Donc même si les équipes disent qu’elles respectent la pyramide, on reste quand même en mode cornet avec juste plus de validations et plus de ralentissement et de “gates“. Bâtir la confiance ce n’est pas facile si même en Agile on reste en silos !

  Attention, ce n'est pas la pyramide qui résoudra tous vos problèmes

  Maturité, qualité, efficacité, stabilité des infrastructures, environnements, déploiements, choix/utilisation de vos outils, DevOps, Ci-Cd, sécurité, mock, création/test data management , ou les bonnes pratiques/anti-patterns ou encore la stratégie… Qui sont, soi-disant, indispensables/prérequis avant de penser Pyramide !

  Commencer par faire du bien à ce qui fait mal…

  Comprendre la raison et de l'importance de chaque niveau en vue de trouver des solutions plutôt que faire de la fuite ! Vous allez vite constater qu’il y a beaucoup de solutions et que les niveaux de test “SYSÈME“ sont aussi importants que les “UNITAIRES“ ou les “INTÉGRATIONS“. Gardez à l'esprit qu'il s'agit d'un indice de très haut niveau, dans un contexte “MOCK“ et très pauvre en liens avec les types de tests.

  Posons-nous, plutôt, ces questions pour trouver des solutions :

- *Que pouvons-nous faire pour que ce ne soit pas **Coûteux** ?*
- *Que pouvons-nous faire pour que ce ne soit pas **Dépendant** ?*
- *Que pouvons-nous faire pour que ce ne soit pas **Lent** ?*
- *Que pouvons-nous faire pour que ce ne soit pas **Fragile** ?*
- Que pouvons-nous faire pour que ce soit **Répétable** ?
- Que pouvons-nous faire pour que les tests soient **Réalisés/Exécutés plus tôt** ?
- Que pouvons-nous faire pour que ce soit **Souverain** ?
- Que pouvons-nous faire pour que ce soit **Efficace** ?

Nous pouvons considérer l'amélioration continue. Cependant, Agile ou pas… La première chose à faire est de s'occuper de ce qui fait le plus mal, avant de commencer à valider ou faire des tests, oui, avant même de commencer à designer, développer et livrer !

ex. communs

- Bien moquer à tous les niveaux avec du data de test contrôlable
- Avoir des environnements/déploiements de test stables, indépendants et sur demande
- Ne pas réinventer la roue, éviter les anti-patterns et ne pas avoir peur d'investir dans des outils précieux pour l'automatisation ex. Eggplant

Peu importe de quel côté, il faut faire face à la chanson, nous pouvons cependant choisir de le faire de façon Rock-On! 🤘

à suivre dans la prochaine épisode…

(done) E4. Faire rayonner le côté sombre de la pyramide - Est-ce possible ?

January 18, 2024 10:15 AM



Est-ce possible de faire rayonner le côté sombre de la pyramide ?

Si on vous pose la question "Quelle pyramide utilisez-vous ?"

En mandat ou lors d'une rencontre, vous aurez sûrement à subir l'affrontement de la pyramide... Nous pouvons toujours répondre que l'utilisation de la pyramide est selon le besoin de la qualité ! L'important c'est de mettre le focus sur les "meilleurs pratiques de la qualité" qui peuvent aider à atteindre les objectifs (besoins).

Avantages versus satisfaction des besoins

Est-ce un bon retour sur l'investissement, une valeur ajoutée ?

Grâce à la pyramide, on découvre les 3 niveaux de test et les développeurs font, de nos jours, pratiquement, tous des tests unitaires dans leur code. Mais pour le reste, une question comme **"quels sont les tests qui sont payants à exécuter ou qui ont de la valeur ajoutée de la valeur ajoutée ou quelles exécutions/sections pouvons-nous éviter de retester à chaque fois ?"**, c'est bien plus efficace qu'une pyramide mal comprise. Prioriser les tests par niveau de risque, shift-left et shift-right, partager la couverture entre les niveaux sont des concepts beaucoup plus louables, concrets et réalistes.

Avant de penser à une pyramide

Implémenter les préalables à l'automatisation avant même de penser à la pyramide est la première chose à faire, ensuite préparer le passage (le switch et la cohabitation du manuel et l'automatisation)

Voici les préalables :

1. Mock d'environnements/déploiements stables
2. Contrôle du data des tests (jeux de données)
3. SUTs/HUBs d'images d'environnements utilisateur
4. Réutilisation des tests
5. Le coffre à outils
6. La couverture, la Qualimétrie, la documentation
7. etc…

CONCLUSION

Selon moi, la seule façon de faire rayonner la pyramide est :

8. **Éviter les pièges du côté sombre car cela peut rapidement tourner à un véritable cauchemar !**

- Ne pas sur interpréter/sur utiliser la pyramide.
- Attention aux fausses pyramides ou l'altération de la pyramide pour essayer de tout solutionner les besoins.
- La pyramide n'est pas la stratégie de test et encore moins universellement applicable dans tous les contextes.
- Il faut éviter l'imposition et l'absolue sans en connaitre les raisons la compréhension de l'utilisation.
- 70 % de tests unitaires, 20 % de tests d'intégration et 10 % de tests système… Tout comme l'indépendance des tests, il ne faut pas rechercher à tout prix obtenir une belle pyramide des tests… Être parfait n'est pas forcément un objectif en soi, voir plutôt à atteindre de réelles objectifs comme la confiance et la satisfaction.
- La pyramide est un "BUZZWORD" très puissant, on pourrait vous décrédibiliser ou même vous bannir, si vous tentiez de dire que la pyramide n'apporte aucune valeur ajoutée ! Surtout quand tout l'univers du développement repose sur la pyramide ! Il faut y aller avec précaution, prudence et diplomatie pour éviter les cataclysmes… C'est peut être une bonne indication dans certains contextes. Il est bon de le garder à l'esprit qu'à tout moment nous pouvons décider de l'ignorer, tout en s'assurant de comprendre l'impact et le pourquoi.
- Est-ce que la pyramide est toujours d'actualité ou pertinente ?
- Rappelons-nous qu'il est facile de s'y perdre !

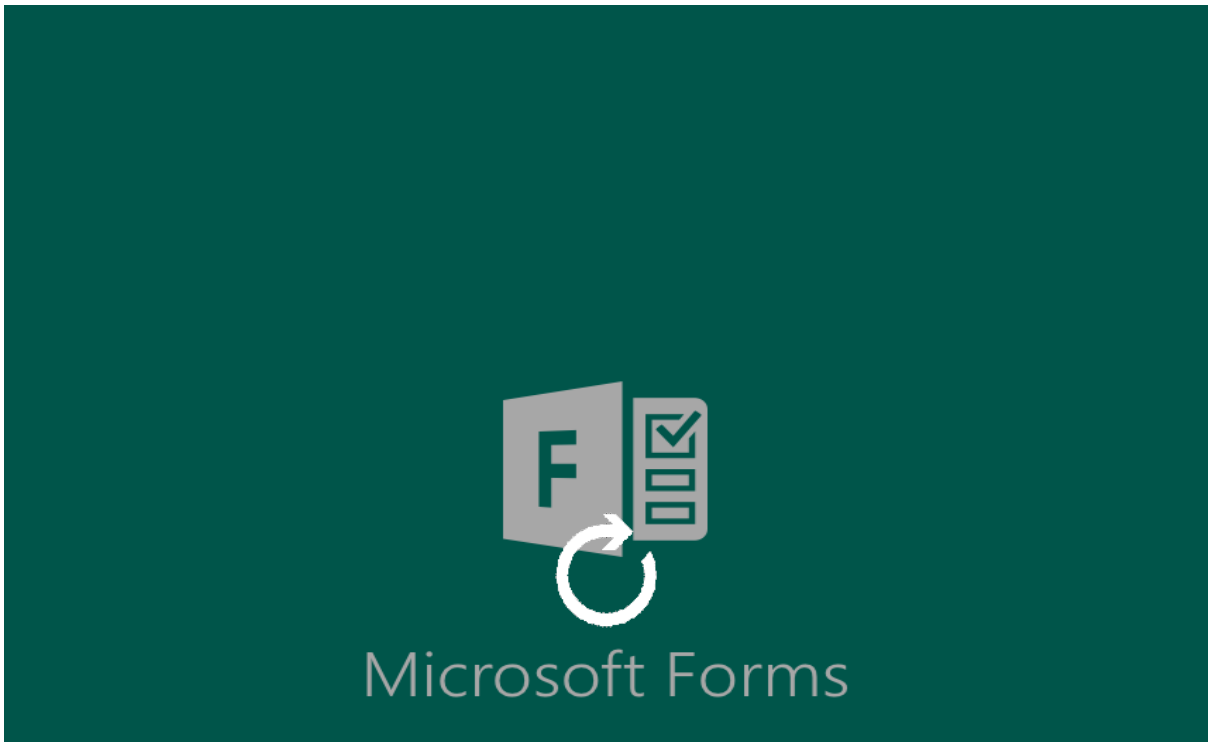
9. **Respecter sa vrai nature**
10. ✕

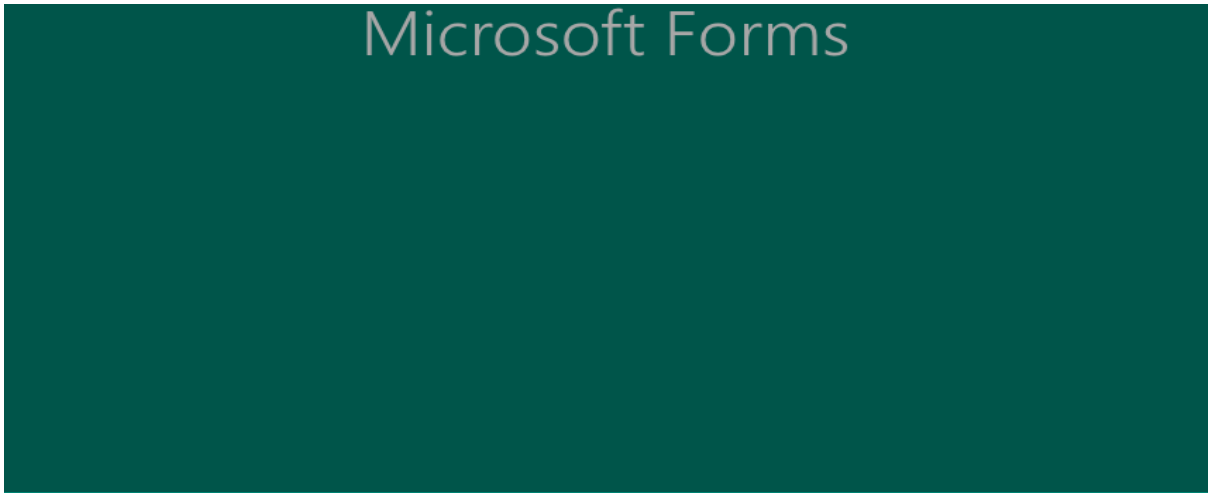
car la qualité ne se résume pas qu'à la pyramide !

- Maîtrisons la compréhension des niveaux de test.
- Gardons le focus sur la définition de La pyramide de test (niveaux de test)
- Continuons de promouvoir la qualité dans l'amélioration continue, tout en restant simple et vrai car il n'y a pas qu'un seul chemin pour y parvenir.

Même avec les solutions/astuces, rien n'est garanti ou absolu !

N'hésitez pas, si vous avez le goût de mesurer vos connaissances avec le Quiz éclaire sur la pyramide




Microsoft Forms

(done) Un test de niveau unitaire n'est pas juste du code, c'est fonctionnel et ça peut même être une validation UI !

January 18, 2024 10:16 AM

Un test de niveau unitaire : c’est bien plus que tester du code

Les tests unitaires sont souvent perçus comme une vérification du bon fonctionnement du code. Cependant, ils vont bien au-delà de cette simple définition. En réalité, un test unitaire peut être fonctionnel et même servir à valider une interface utilisateur (UI).

Les tests unitaires et la fonctionnalité

Les tests unitaires permettent de vérifier si une fonction ou une méthode spécifique produit le résultat attendu dans différentes conditions. Ils peuvent également être utilisés pour tester des fonctionnalités spécifiques d’une application. Par exemple, si vous développez une application de commerce électronique, vous pouvez écrire des tests unitaires pour vérifier si le panier d’achat fonctionne correctement lorsque l’utilisateur ajoute ou supprime des articles.

Les tests unitaires et l’interface utilisateur

Bien que les tests unitaires soient généralement associés au code, ils peuvent également être utilisés pour tester des éléments de l’interface utilisateur. Par exemple, vous pouvez écrire des tests unitaires pour vérifier si un bouton est correctement activé ou désactivé en fonction de certaines conditions. De plus, des outils comme Selenium permettent d’automatiser les tests d’interface utilisateur, ce qui peut grandement faciliter le processus de test.

Les tests unitaires dans l’intégration continue

Les tests unitaires sont souvent utilisés dans le cadre d’un pipeline d’intégration continue. Chaque fois qu’un changement est apporté au code, les tests unitaires sont exécutés automatiquement pour s’assurer que le changement n’a pas introduit de nouveaux bugs. Cela permet de détecter rapidement les problèmes et de les corriger avant qu’ils n’affectent les utilisateurs.

Les tests unitaires et Storybook

Un autre excellent exemple de tests unitaires dans le contexte de l’interface utilisateur est l’utilisation de Storybook. Storybook est un outil open source pour développer des composants d’interface utilisateur de manière isolée. Il permet aux développeurs de créer des composants individuellement et de visualiser toutes les différentes états de ces composants.

En utilisant Storybook, vous pouvez écrire des tests unitaires pour vos composants d’interface utilisateur. Par exemple, si vous avez un composant de bouton, vous pouvez écrire des tests pour vérifier si le bouton est rendu correctement, si les clics sur le bouton déclenchent les actions appropriées, et si le bouton est correctement désactivé ou activé en fonction de certaines conditions.

De plus, avec des addons comme Storyshots, vous pouvez même ajouter des tests de capture instantanée automatisés à votre Storybook. Cela vous permet de vérifier que vos composants ne changent pas de manière inattendue entre les différentes versions de votre application.

En résumé, Storybook offre une excellente façon de réaliser des tests unitaires sur vos composants d'interface utilisateur, vous aidant à garantir que votre UI fonctionne comme prévu.

L'automatisation des tests unitaires

Il est important de noter que bien que l'on pense que les tests unitaires sont inévitablement automatisés, pourtant ils ne le sont pas systématiquement. La documentation des tests unitaires passe par le même flux que tous les tests, soit d'abord par la documentation et l'exécution manuel.

L'automatisation des tests unitaires dépend de plusieurs facteurs, notamment la complexité du code, les ressources disponibles et les exigences du projet.

L'automatisation des tests unitaires peut offrir plusieurs avantages. Elle permet d'exécuter rapidement un grand nombre de tests, ce qui peut améliorer l'efficacité du processus de développement. De plus, une fois que les tests sont automatisés, ils peuvent être exécutés à tout moment, ce qui permet de détecter rapidement les régressions et si automatisé TDD nous pourrons valider au fur et à mesure de façon automatique.

Cependant, l'automatisation nécessite un investissement initial en termes de temps et de ressources. De plus, tous les tests ne sont pas adaptés à l'automatisation.

Je souligne que les tests unitaires UI peuvent très bien être automatiser et ce même avant le développement, l'architecture et même le design comme nous avons vu dans l'exemple des components.

En fin de compte, la décision d'automatiser les tests unitaires doit être prise en fonction des besoins spécifiques du projet et des outils disponibles.

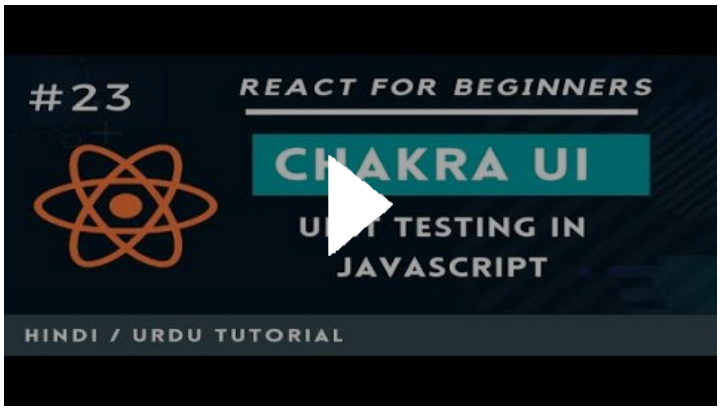
Conclusion

Les tests unitaires sont un outil puissant pour assurer la qualité du code et des fonctionnalités d'une application. Ils ne se limitent pas à vérifier le bon fonctionnement du code, mais peuvent également être utilisés pour valider les fonctionnalités et l'interface utilisateur. En intégrant les tests unitaires dans votre processus de développement, vous pouvez améliorer la qualité de votre application et offrir une meilleure expérience à vos utilisateurs.

Un peu plus loin…

UI Unit Testing – Think Like a Tester (thinkingtester.com)

[#23] Chakra UI Unit Testing with React Testing Library and Jest in Hindi and Urdu



Introducing UI Unit Testing with TestBench | Vaadin

How to Unit Test Angular UI Components – andrewhalil.com

Functional Unit Test (FUT)

Le test d'unité fonctionnelle est un processus d'assurance qualité et un type de test de boîte noire qui fonde ses cas de test sur les spécifications du composant logiciel testé.

Un test de fonctionnalité d'application (ou test unitaire fonctionnel) est l'acte de valider les fonctions pour la norme parmi les configurations inférieures "ou comme prévu" avec la configuration mise en place a été appliquée. Le test unitaire fonctionnel est effectué par l'équipe de projet fonctionnel une fois que toutes les configurations/configurations ont été terminées pour valider la fonctionnalité de l'application. La configuration et la documentation de la configuration seront ajustées en fonction des commentaires reçus pendant l'activité de test UT. Le test de sécurité de l'application (c'est-à-dire le test des profils utilisateur de sécurité définis) est également utilisé aux niveaux de test unitaire + en utilisant les identifiants utilisateur "mock" sélectionnés. Tous les tests unitaires se fait avec des données moques dans l'environnement de développement et est sous la responsabilité de l'équipe projet ou produit (développement et flux fonctionnels)

Technical Unit Test

Les tests unitaires techniques (TUT) se concentrent sur la fonctionnalité d'objets modifiés/ajoutés personnalisés développés individuellement. En tant que première "ligne de défense" dans le processus de test, les tests unitaires visent à découvrir les problèmes tôt dans le processus de développement. Assurez-vous que les objets ensuite testés individuellement sont sans erreur avant d'entrer dans test de niveau système. Voir également pour déterminer que l'objet a répondu à l'intention de conception pour chaque objet, un test unitaire #cript est créé pour valider que le test de l'objet répond aux exigences de la spécification fonctionnelle/ technique. Ainsi, les tests unitaires techniques appellent des objets ou des unités de développement (par exemple, des interfaces de personnalisation, des programmes de conversion de données, des rapports) selon leurs spécifications fonctionnelles/techniques approuvées.

…attention ce n’est pas tout !
Les tests fonctionnels font référence aux activités qui vérifient une action ou une fonction spécifique du code. (c'est la définition officielle du logiciel de test)

Encore faut-il comprendre que le code est basé et implémenté à partir de spécification fonctionnel et ce tout à fait naturel que la structure de l’architecture du code suive ces spécifications.
Non seulement nous avons des fonctions dans les classes mais la logique et les objets suivent aussi le fonctionnel pour former un contexte, des actions, et des résultats attendu !
Donc, si tout ce que l’on développe se résume à du fonctionnel ? c’est quoi du non-fonctionnel

Functional Unit Testing Methods for Practitioners: Short Guideline (

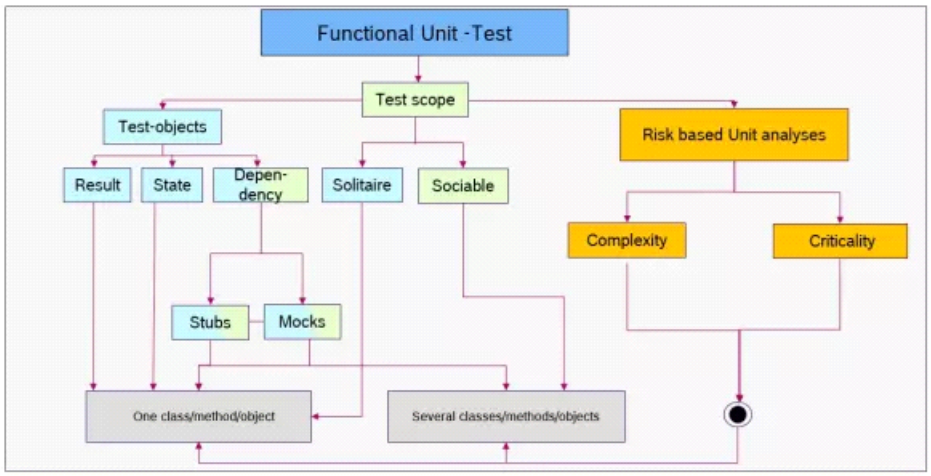
Sogeti)

Ajoutons zen encore plus avec du functional programing (FP)

Making unit-test fun again with Functional Programming - DEV Community

Un Framework Agile c'est inspiré du développement piloté par le fonctionnel : Feature-driven development - Wikipedia en lien avec Agile architecture - Wikipedia… mais il y en a plusieurs autre (functional and model-base). Cela rend non seulement le code toujours solide, facile à comprendre, claire et bien structuré, même chose pour tout le reste du développement jusqu'au support et la satisfaction clientèle (bref, ça simplifie tout)

Donc oui, il est tout à fait sensé de réutiliser les mêmes spécifications fonctionnel pour piloter l'architecture, le développement, peu importe le niveau de test automatisé ou manuel !!!




HUH?

***Comment réussir, efficacement, le décalage en amont (shift-left) ?

January 18, 2024 10:17 AM

Le décalage en amont, ou “shift-left”, est une approche qui vise à intégrer les tests et le suivi dès le début du cycle de développement. Cette méthode permet de transmettre rapidement et facilement les connaissances de votre service d’assistance à tous les employés de votre entreprise. Dans cet article, nous allons explorer comment réussir efficacement le décalage en amont en suivant le cycle : définir, planifier, intégrer, diffuser, opérer, veiller.

Définir

La première étape du cycle est la définition. C’est ici que les exigences du projet sont définies. L’analyse des exigences est cruciale pour comprendre ce que le projet doit accomplir. Une fois les exigences définies, elles doivent être validées pour s’assurer qu’elles sont réalisables et qu’elles répondent aux besoins du projet.

Planifier

La planification est l’étape où les tâches sont organisées et programmées. C’est ici que les ressources sont allouées et que les délais sont fixés. Comme pour la définition, l’analyse est essentielle pour une planification efficace. Une fois la planification terminée, elle doit être validée pour s’assurer que les tâches sont réalisables dans les délais impartis.

Intégrer

L’intégration est l’étape où les différentes parties du projet sont combinées pour former un tout cohérent. L’analyse est utilisée pour identifier les problèmes potentiels qui pourraient survenir lors de l’intégration. Une fois l’intégration terminée, elle doit être validée pour s’assurer que toutes les parties fonctionnent correctement ensemble.

Diffuser

La diffusion est l’étape où le produit est rendu disponible pour les utilisateurs. L’analyse est utilisée pour déterminer la meilleure façon de diffuser le produit. Une fois la diffusion terminée, elle doit être validée pour s’assurer que le produit est accessible et fonctionne correctement pour les utilisateurs.

Opérer

L’opération est l’étape où le produit est en cours d’utilisation. L’analyse est utilisée pour surveiller le fonctionnement du produit et identifier les problèmes potentiels. Une fois l’opération terminée, elle doit être validée pour s’assurer que le produit fonctionne correctement.

Veiller

La veille est l’étape où le produit est surveillé pour s’assurer qu’il continue de fonctionner correctement. L’analyse est utilisée pour identifier les problèmes potentiels et les tendances. Une fois la veille terminée, elle doit être validée pour s’assurer que le produit continue de répondre aux exigences.

En conclusion, le décalage en amont est une approche efficace pour améliorer la qualité et l’efficacité du développement. En intégrant l’analyse et la validation à chaque étape du cycle, il est possible de détecter et de résoudre les problèmes plus tôt, ce qui peut conduire à un produit de meilleure qualité et à une plus grande satisfaction des utilisateurs.

Notes…

Comment réussir, efficacement, le décalage en amont dans le cycle de vie du développement logiciel piloter par les tests en continus (Ct) ?

L'optimisation de la qualité est une odyssée en soi !

Faire de la qualité sans assurance qualité, est-ce possible… mais oui la qualité existe quand même, peut importe comment, car il y aura toujours le retour du client.

Nous pouvons cependant :

1. être **rapide** : valider avec le client le plus rapidement possible avec les plus petits détails dans un context réel
2. être **efficace** : connaissance de la structure, du milieu, cause profonde, proactif, alertes/indicateurs/notifications…
3. être **confiant** : veiller sur la sécurité, valeurs ajoutées, donc la satisfaction du client, mais aussi de nous

Pour atteindre le **succès** !
Le Shift-Left est une excellente pratique, pour ne pas dire la première/meilleure pratique/réflexe que nous devrions toutes et tous d'adopter pour arriver à survivre et se démarquer.

C’est ici que nous allons "challenger" tout nos concepts de la qualité !

4. Premièrement le Cycle de vie du développement logiciel (SDLC)ARCHIVED
5. Le passé présent futur du développeur

a. le "right" (futur)
b. le "just in time" (nombreux sont les gens qui pensent que le shift-left c’est jusqu’ici)
c. "left" (passé)

1. Passons à travers les phases pour voir comment on peut faire, concrètement, du "shift-left" en Ct dans du Ci-Cd

a. Définir (atdd, fdd, bdd…) → valider le pourquoi, quoi, qui, comment… Définir la couverture total des tests, “first“ ! Et de façon claire et précis pour le logiciel et tout le monde (UML, AGILE, TDD, ATDD, FDD, DDD, BDD)
b. Planifier → valider le quoi-quand-qui (***soulignons que jusqu’ici nous n’avons pas encore code-é***)
c. Intégrer (tdd) → valider la mise en place (petites itérations de test → code → test → code…)
d. Diffuser → valider le déploiement
e. Opérer → valider le contexte
f. Veiller → valider la satisfaction

1. Comment parvenir à faire du "shift-left", même dans un contexte legacy/waterfall ou hybrid/évolutif en continu ?

a. Voilà l’importance et le pourquoi d’une bonne STRTÉGIE
b. ✕

Si je vous pose maintenant la question, qui est responsable du Shift-Left, bon nous allons dire tout le monde !

- Qui est responsable de la modélisation/l’architecture ?
- Qui est responsable de la qualité ?
- Qui est responsable des tests ?
- Qui est responsable de automatisation ?
- Qui est responsable des analyses ?
- Qui est responsable du code ?
- Qui est responsable du ui/ux ?
- Qui est responsable des produits ?

- Qui est responsable des fonctionnalités ?
- Qui est responsable des systèmes ?
- Qui est responsable des intégrations ?
- Qui est responsable des déploiements ?
- Qui est responsable des environnements ?
- Qui est responsable de la documentation ?
- Qui est responsable des livraisons ?
- Qui est responsable du support, satisfaction à la clientèle ?
- Qui est responsable des validations ?
- Qui est responsable des bugs ?
- ……

  Commençons donc par qui est responsable de la stratégie ?
  rep. Que dite-vous d'un positionnement du département R&D (TI), "first"… Nous parlons ici d'une **décision/direction/responsabilité** corporative/entreprise.

  Le shift-left impacte réellement non seulement sur les logiciels, les départements, les procédures, les processus,les conventions, les attentes mais l'entreprise entière et surtout le client.

  ## C'est plus facile à dire qu'à faire !

  Comment faire la transition dans un vieux gros bateau waterfall, legacy ou un mélange de tout ça ?
  Comment faire du Shift-Left dans une équipe réfractaire ?

  ### Voici des guides, des astuces, des propositions de solutions

  dsfh

  Les sujets abordé

- TDD (ATDD, FDD, DDD, BDD)
- Gherkin est une langage humainement compréhensible
- Shift-Left
- Itération TDD (code)
- zero bug
- zero debug
- Les 3 niveaux, pyramide, test unitaire
- Intégrité vs intégré vs intégration
- suite/scénario versus case/unitaire
- DevOps
- Sécurité et DevSecOps
- Gestionnaire ou pas
- Jira ou pas
- Bug
- Story versus Bug versus Feature request versus Ticket, Issue… ?
- Les causes de la non réussite :
- Déformation de l'architecture, du besoin, de la solution, du comportement…
- Ambiguïté, incompréhension de l'architecture, du besoin, de la solution, du comportement… mais aussi des termes, procédures, processus, pratiques, méthodologies, conventions, attentes, règles…

- Pas de ligne directrice (stratégie)
- Qualimétrie
- le contrôle statique
- pipeline impacte
- data (not from prod)
- mock
- sur-documentation, documentation vivante et auto-documentation (self-doc)
- sur-tester
- ramener tout au code !?
- Valider par le spécialiste et le client
- feedback
- Le SDLC et Ci-Cd

Si trop long, partitioner en capsule

Ébauche (example) de l'ordre de la présentation

1. L'idée d'une application
2. Création d'un projet+story

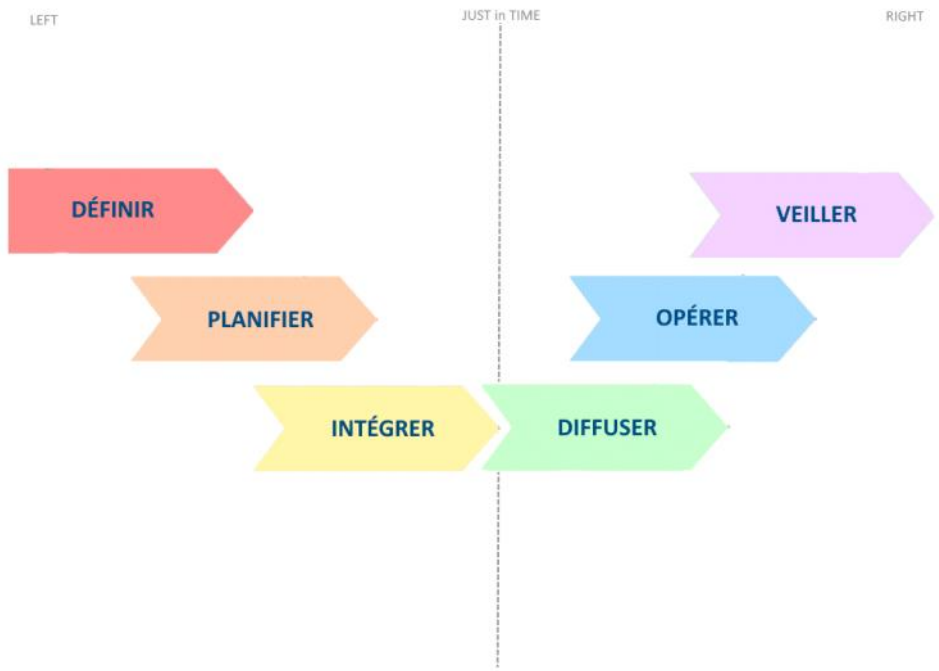
a. Shift-Left, commençons un plus en amont

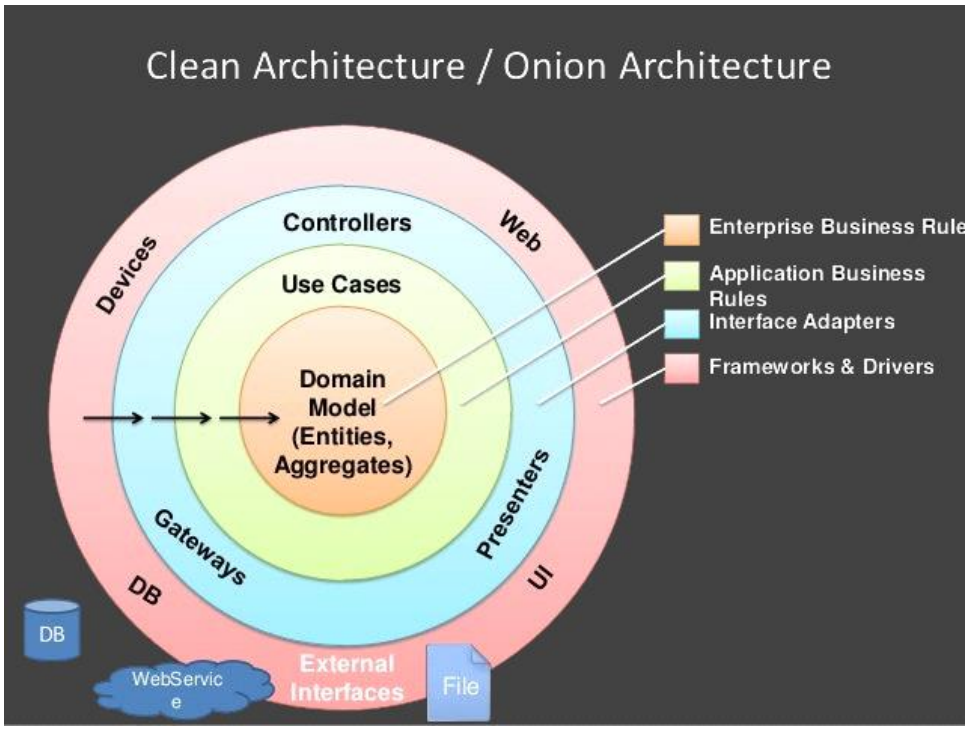
b. ×

1. **Épopée** Définition de la stratégie (sécurité, etc.)
2. **Épopée** Définition de la modélisation/Architecture
   Définition de la (**ici est la clé du succès**… ATDD(FDD/DDD/BDD))

a. Comment automatiser le modèle (unitaire, intégration et système)

1. Revenons à la **Story**
2. Sous-tâche
3. Création d'un requis, notre premier bon de commande
4.




Clean Architecture / Onion Architecture
Controllers
Use Cases
Domain Model (Entities, Aggregates)
Devices
Web
Gateways
Presenters
DB
UI
External Interfaces
DB
WebServic e
File
Enterprise Business Rule
Application Business Rules
Interface Adapters
Frameworks & Drivers

***Quels est votre rôle et vos responsabilités dans l'assurance qualité ?

January 18, 2024 10:18 AM

Commençons par les ambassadeurs de la qualité

Les responsables de l'assurance qualité

Il ne faut surtout pas le voir comme le responsable d'équipe de l'assurance qualité traditionnelle ou un "QA" (analyste/contrôleur/testeur) dans une équipe Agile, ici nous parlons d'un responsable de l'assurance qualité **dans un contexte Agile**.
Nous pourrions aussi dire Coach QA ou Facilitateur QA pour le différencier du directeur/contrôleur/testeur ("le QA") à même titre que Coach/Facilitateur Agile qui est différent du Flow Master ("le Scrum Master" ou "le Kanban Master")



Information connexe : Les leads de développement, les leads d'assurance qualité et les leads de l'affaire doivent également être alignées, en symbiose avec la stratégie Agile

1. Établir les normes à suivre

Un bon **responsable de l'assurance qualité** ne contrôle pas ; il ou elle donne aux **membres de l'équipe agile** une certaine responsabilité et indépendance pour gérer le travail de qualité par eux-mêmes. Cependant, cela ne signifie pas qu'il ne devrait pas y avoir des **normes/conventions établies** en place, que toute l'équipe doit suivre.

L'organisation dans son ensemble doit fonctionner comme une unité cohérente !

Il devrait avoir quelques "normes prédéfinies" => "stratégie" telles que :

- Quels outils de test de logiciels utiliser et comment les utiliser
- Quelles méthodologies de test appliquer
- Comment déployer une gestion efficace des défauts, par exemple en définissant les statuts des défauts
- Tactiques, méthodologies et meilleurs pratiques pour reconnaître l'état d'une construction et travailler en conséquence.
- etc.

Une stratégie Agile de l'assurance qualité est aussi composé de plusieurs sous-stratégies comme la "stratégie de test" ou encore la "stratégie d'automatisation des tests", etc.

Attention

À ne pas confondre avec Cycle de test ou Plan de test, qui est cependant censé suivre la stratégie.

Un **responsable de l'assurance qualité** travaille avec les chefs de projet pour décider des normes à utiliser. Cela garantit que tous les membres des équipes suivent des techniques similaires et sont formés selon les mêmes ensembles de règles. Les responsables veillent ensuite les **progrès** et la maturité/pérennité/constance de la qualité au cours du processus de développement. Ces lignes directrices sont utilisées par l'équipe R&D pour intégrer **les processus de l'assurance qualité au** sein de leurs équipes de développement.

L'alignement de la stratégie ce fait de façon itérative de petites parties en amélioration continue.

2. Constituez les équipes judicieusement

Toutes les équipes de développement travaillent selon leur propre domaine d'expertise. Par

exemple, une équipe s'occupe de l'interface utilisateur et d'autres se concentrent sur les services backend. Divers domaines de développement nécessitent des connaissances spécialisées telles que les tests automatisés et manuels. L'automatisation des tests est différente car elle dépend de l'interface utilisateur, de l'implémentation des feuilles de style en cascade ou de l'API REST.

Un **responsable de l'assurance qualité** construit soigneusement les équipes, composées de membres d'équipe avec différentes compétences, comme un **architecte de l'assurance qualité, des** testeurs d'interface utilisateur, des testeurs fonctionnels, des testeurs d'automatisation et des testeurs de performance. En plus de cela, ils définissent les responsabilités et les rôles des membres de l'équipe, et définissent comment accomplir les tâches. Ils doivent garantir que les testeurs de développement sont correctement équipés pour tester le travail de développement spécifique de leur équipe.

3. Veiller à ce que les mesures de qualité (Qualimétrie) soient comprises, bien utilisées et respecté selon la stratégie de l'assurance qualité
Delta de maturité de l'assurance qualité : Mesurer si les processus sont bien respectés en vu de proposer des solutions d'amélioration

Il existe de nombreuses raisons pour lesquelles les équipes pourraient ne pas fonctionner conformément aux attentes. Ils peuvent ne pas suivre les techniques de test définies, ce qui pourrait éventuellement augmenter le nombre de défauts. Si le **responsable de l'assurance qualité** a mis en place les bonnes mesures, il peut facilement aller à la racine du problème, approfondir soigneusement les pratiques de travail de l'équipe et les mettre à niveau.

Traçabilité/documentation des tests/bogues : Mesurer/analyser les traces fourni par la documentation des tests

Un responsable de l'assurance qualité doit adopter une vision macroscopique de la qualité en observant les tendances des métriques dans toutes les équipes. Cela fournit une vue complète du code de qualité de l'organisation et reconnaît les équipes qui ne travaillent peut-être pas à la hauteur.

Observez la qualité du système complet. C'est le choix **de l'architecte de l'assurance qualité** d'effectuer ou non des tests interfonctionnels pour garantir que tout fonctionne bien ensemble. Ils testent la connexion entre les modules dans diverses configurations et environnements pour garantir que le système fonctionne correctement, sans aucun problème de performances, de sécurité ou d'intégration.

La principale responsabilité d'un **responsable de l'assurance qualité est** de réfléchir à la qualité **globale du système incluant la satisfaction clientèle**. Lorsqu'ils examinent les problèmes au niveau du système, ils peuvent se considérer comme "le gardien" entre les **équipes de développement, de test et d'assurance qualité.**

4. Gestion autonome de s'organiser, des demandes des ressources et demandes d'aide

Les responsables de l'assurance qualité doivent veiller que les équipes soient capables de gérer les demandes et organiser les ressources par eux-mêmes, plutôt que de compter sur une aide externe telle que davantage de ressources ou d'outils de l'assurance qualité ou d'argent. En termes simples, ils devraient être en mesure de réorganiser les ressources existantes pour répondre aux demandes en tant qu'unité Agile. Idéalement, tous les membres de l'équipe devraient avoir la compétence pour exécuter divers tests, car tous les membres de l'équipe ont de multiples compétences et n'importe qui peut se montrer à la hauteur en relation avec les experts/spécialistes en la matière. Dans certaines situations prévues/préplanifier, le **responsable de l'assurance qualité** peut déplacer des testeurs entre les équipes pour répondre aux exigences de tests supplémentaires. Cette méthode est appelée "essaimage"

Exemple : Les attentes du suivi du développement "scrum" (incluant la qualité) repose sur le "Scrum-Master", donc la ressource experte/spécialiste lead QA est là pour suggérer des solutions, la mise en place et veiller sa pérennité au sein l'équipe autonome Agile.

Attention

Le responsable AQ devra soulever les besoins de nouvelles ressources, seulement, si l'équipe Agile n'est pas complète, sinon il devrait planifier des formations et ateliers pour éventuellement arriver à ce que l'équipe soit autonome pour fournir un développement de qualité.

La surcharge de capacité/vélocité versus délais prévues des livraisons par une équipe Agile complète doit être aussi considéré/faisant partie intégrante de la stratégie de test et des plans de test...
Non pas ajouter des ressources au moment de surprise urgente en phase terminal des tests avant la livraison !

Analyser, Expérimenter, Réfléchir, Concevoir, Réaliser, Valider, Déployer, Opérer, UAT... ne sont pas des entités séparées, elles sont toutes un tout avec le "développer" par l'équipe Agile

La direction et les gestionnaires de l’assurance qualité

sdfg

Les analystes de l’assurance qualité

sfdg

Les spécialistes, experts de l’assurance qualité

sdfg

Les contrôleurs de l’assurance qualité

sdfg

Les ingénieurs en automatisation de l’assurance qualité (SDET)

sdfg

etc…

Examinons maintenant d'autres rôles qui ne relèvent pas de l'assurance qualité, mais qui sont essentiels à leurs responsabilités d'assurance qualité.

Les coach Agile

dsf

Les flow master, scrum master

sdf

Les product owner

sdf

Les project manager

xf

Les architectes

Les analystes programmeurs

Les analystes affaires
Les analystes fonctionnels
Les designer, UI, UX, accessibilité
Les rédacteurs, traducteurs
Les techniciens au support et à la satisfaction
Les devops
Les hackers éthiques
Les SDET
La sécurité, performance, etc…
Les dba
ETC.

(done) Intégrité vs Intégré vs Intégration vs Intégralité

January 18, 2024 10:18 AM

Introduction

Dans le domaine du logiciel, les termes “intégrité”, “intégré”, “intégration” et “intégralité” sont souvent utilisés. Bien qu’ils semblent similaires, ils ont des significations distinctes et jouent des rôles différents dans le développement logiciel.

Intégrité

L’intégrité fait référence à la cohérence et à la fiabilité d’un système logiciel. Un système d’intégrité élevée est un système qui fonctionne comme prévu, sans failles ni erreurs.

Intégré

Un logiciel intégré est un logiciel qui a été combiné avec d’autres logiciels ou systèmes pour fonctionner comme une unité cohérente. Cela peut impliquer l’intégration de différentes fonctionnalités, modules ou systèmes.

Intégration

L’intégration est le processus d’assemblage de différents composants logiciels en un système unifié. Cela peut impliquer l’intégration de différents modules, l’intégration de logiciels avec du matériel, ou l’intégration de systèmes logiciels avec d’autres systèmes.

Intégralité

L’intégralité fait référence à la complétude d’un système logiciel. Un système logiciel intégral est un système qui a toutes les fonctionnalités et capacités requises pour répondre aux besoins des utilisateurs.

Conclusion

Comprendre la différence entre l’intégrité, l’intégré, l’intégration et l’intégralité est essentiel pour naviguer efficacement dans le domaine du logiciel. En gardant à l’esprit ces distinctions, les équipes de développement peuvent travailler de manière plus efficace et produire des logiciels de meilleure qualité.

(done) Quoi?! Nous pouvons assurer la qualité automatisée de l'architecture et du design (ui/ux) avant le développement !

January 18, 2024 10:18 AM

Introduction

Dans le monde du développement logiciel, l'assurance qualité est souvent associée à la validation du code et des fonctionnalités. Cependant, saviez-vous que nous pouvons également assurer la qualité automatisée de l'architecture et du design (UI/UX) avant même le développement ? Oui, c'est possible !

L'Importance De L'Architecture Et Du Design

L'architecture et le design sont des éléments cruciaux de tout produit logiciel. Une architecture solide assure la stabilité, la performance et la maintenabilité du logiciel. Un bon design, quant à lui, garantit une expérience utilisateur agréable et intuitive.

Assurance Qualité Automatisée

Grâce à divers outils et technologies, nous pouvons automatiser l'assurance qualité de l'architecture et du design. Par exemple, nous pouvons utiliser des outils d'analyse statique pour vérifier la conformité de l'architecture aux meilleures pratiques et aux normes de l'industrie. Pour le design, nous pouvons utiliser des outils de prototypage et de test utilisateur pour valider l'ergonomie et l'accessibilité du design.

Avantages De L'Assurance Qualité Pré-Développement

L'assurance qualité de l'architecture et du design avant le développement présente plusieurs avantages. Elle permet de détecter et de corriger les problèmes potentiels avant qu'ils ne se transforment en bugs coûteux à réparer. Elle permet également de s'assurer que le produit final répondra aux attentes des utilisateurs en termes de performance et d'expérience utilisateur.

Conclusion

L'assurance qualité ne se limite pas au code et aux fonctionnalités. En assurant la qualité de l'architecture et du design avant le développement, nous pouvons créer des produits logiciels de meilleure qualité et offrir une meilleure expérience à nos utilisateurs.

Ici nous parlerons de MBT Model-based testing mais automatisé par le digital-twins

Intéressant

3-Minute Demo: Enterprise



Cete approche nous permet de développer solidement notre business logic et baser le développement sur du solide déjà validé !

on peut aussi faire du reverse generated automation pour u legacy par exemple ou du développement qui n'a pas encore d’architecture (modèle)

How to automatically generate models and snippets with Eggplant.

***Quoi?! il y a plusieurs DoR et plusieurs DoD !

January 18, 2024 10:19 AM

Qu’est-ce qui fait que chaque phase est prête ou terminée ?

C’est seulement après avoir terminé le **DoR** que le **DoD** est défini



(done) Le succès est comme un iceberg

January 18, 2024 10:20 AM



La partie sous l'eau de l'assurance qualité du logiciel

Introduction

Dans le monde fascinant de l'assurance qualité du logiciel, le succès est souvent perçu comme l'iceberg que l'on voit flotter à la surface de l'eau. Cependant, tout comme un iceberg, une grande partie de ce qui contribue au succès se trouve sous la surface, invisible à l'œil nu. Dans cet article, nous allons explorer ces aspects moins visibles mais tout aussi cruciaux du succès dans l'assurance qualité du logiciel.

Partie 1 : L'organisation

L'organisation d'une équipe d'assurance qualité est un élément crucial du succès qui est souvent négligé. Cela comprend la définition des rôles et responsabilités, la mise en place de processus efficaces, et la promotion d'une culture de qualité au sein de l'équipe et de l'organisation dans son ensemble. Bien que ces efforts ne soient pas toujours visibles de l'extérieur, ils sont essentiels pour assurer la qualité du logiciel.

Définition des rôles et responsabilités

Chaque membre de l'équipe a un rôle spécifique à jouer dans l'assurance qualité. Que vous soyez un testeur, un développeur, un chef de projet ou un analyste d'affaires, comprendre votre rôle et vos responsabilités est essentiel pour contribuer efficacement à l'assurance qualité.

Mise en place de processus efficaces

Un processus efficace est la colonne vertébrale de toute équipe d'assurance qualité. Cela comprend des processus pour la gestion des tests, la gestion des défauts, la gestion des exigences, et bien d'autres. Un processus efficace permet de s'assurer que toutes les tâches sont effectuées de manière cohérente et efficace.

Promotion d'une culture de qualité

Une culture de qualité est une culture où la qualité est valorisée et promue à tous les niveaux de l'organisation. Cela signifie que chaque membre de l'équipe comprend l'importance de la qualité et s'efforce constamment de l'améliorer.

Partie 2 : Les efforts invisibles

De nombreux efforts qui contribuent au succès de l'assurance qualité du logiciel sont invisibles pour ceux qui ne sont pas directement impliqués dans le processus. Cela peut inclure la formation continue pour rester à jour avec les dernières technologies et méthodologies, la recherche et l'implémentation de nouveaux outils et techniques, et le travail de collaboration avec d'autres équipes pour comprendre leurs besoins et attentes. Ces efforts, bien que souvent invisibles, sont essentiels pour maintenir et améliorer la qualité du logiciel.

Formation continue

Dans le domaine en constante évolution de l'assurance qualité du logiciel, la formation continue est essentielle. Que ce soit par le biais de cours en ligne, de webinaires, de conférences ou de lectures, rester à jour avec les dernières technologies et méthodologies est crucial pour réussir.

Recherche et implémentation de nouveaux outils et techniques

Avec l'évolution rapide des technologies, de nouveaux outils et techniques sont constamment développés pour aider à l'assurance qualité. La recherche et l'implémentation de ces nouveaux outils et techniques peuvent grandement améliorer l'efficacité et l'efficience de l'équipe d'assurance qualité.

Collaboration avec d'autres équipes

L'assurance qualité ne se fait pas en vase clos. Il est important de collaborer avec d'autres équipes, comme l'équipe de développement, l'équipe des opérations, et même l'équipe des ventes et du marketing, pour comprendre leurs besoins et attentes. Cette collaboration peut aider à identifier les problèmes potentiels plus tôt et à améliorer la qualité du logiciel.

Partie 3 : La partie immergée de l'iceberg

Tout comme la partie immergée d'un iceberg, ces efforts invisibles et l'organisation de l'équipe sont une partie intégrante du succès de l'assurance qualité du logiciel. Sans eux, la partie visible du succès ne serait pas possible. En reconnaissant et en valorisant ces efforts, nous pouvons non seulement améliorer la qualité de notre logiciel, mais aussi renforcer notre culture de qualité et notre capacité à réussir dans un environnement en constante évolution.

Des aspects souvent invisibles mais essentiels qui contribuent à la culture de la pratique de l'assurance qualité du logiciel. Voici une liste de 100 éléments qui pourraient être "sous l'eau" :

1. **Leadership éclairé** : Les leaders qui comprennent l'importance de l'assurance qualité et la soutiennent activement.
2. **Communication ouverte** : Une culture où les questions, les préoccupations et les idées peuvent être librement partagées.

3. **Formation continue** : L'investissement dans l'éducation et la formation pour rester à jour avec les dernières pratiques d'assurance qualité.
4. **Mentorat** : Les mentors expérimentés qui guident et soutiennent les membres moins expérimentés de l'équipe.
5. **Apprentissage à partir des erreurs** : Une culture qui voit les erreurs comme des opportunités d'apprentissage plutôt que comme des échecs.
6. **Collaboration interfonctionnelle** : La collaboration étroite avec d'autres équipes, comme le développement, les opérations, le support client, etc.
7. **Respect des normes et réglementations** : La connaissance et le respect des normes et réglementations pertinentes dans l'industrie.
8. **Gestion efficace des défauts** : La capacité à suivre, gérer et résoudre efficacement les défauts.
9. **Automatisation des tests** : L'utilisation d'outils et de techniques d'automatisation pour rendre les tests plus efficaces et précis.
10. **Pratiques Agile** : L'adoption de méthodologies Agile pour une livraison plus rapide et plus réactive.
11. **Intégration continue/déploiement continu (CI/CD)** : L'utilisation de CI/CD pour intégrer et tester régulièrement le code.
12. **Infrastructure en tant que code (IaC)** : L'utilisation de l'IaC pour gérer et provisionner l'infrastructure informatique.
13. **Tests basés sur le comportement** : L'adoption de techniques de tests basés sur le comportement pour tester le logiciel à partir de la perspective de l'utilisateur.
14. **Gestion des environnements de test** : La capacité à gérer et à maintenir des environnements de test stables.
15. **Gestion des données de test** : La capacité à gérer et à maintenir des données de test précises et à jour.
16. **Revues de code** : La pratique des revues de code pour améliorer la qualité du code et partager les connaissances.
17. **Tests exploratoires** : L'utilisation de tests exploratoires pour découvrir des problèmes qui ne sont pas couverts par les tests automatisés.
18. **Veille technologique** : La veille technologique pour rester à jour avec les dernières technologies et tendances.
19. **Gestion des risques** : La capacité à identifier, évaluer et atténuer les risques.
20. **Planification des tests** : La capacité à planifier efficacement les tests pour maximiser la couverture et l'efficacité.
21. **Documentation** : La création et la maintenance de la documentation de test pour assurer la transparence et la reproductibilité.
22. **Rétrospectives** : La tenue de rétrospectives pour réfléchir aux succès et aux échecs et s'améliorer continuellement.
23. **Culture de l'amélioration continue** : Une culture qui valorise et encourage l'amélioration continue.
24. **Flexibilité** : La capacité à s'adapter aux changements et à être flexible dans les méthodes de travail.
25. **Responsabilité** : Un sentiment de responsabilité pour la qualité du logiciel.
26. **Soutien organisationnel** : Le soutien de l'organisation dans son ensemble à l'assurance qualité.
27. **Budget pour les outils et la formation** : Un budget suffisant pour les outils d'assurance qualité et la formation.
28. **Reconnaissance et récompense** : La reconnaissance et la récompense des efforts d'assurance qualité.
29. **Équilibre entre le travail et la vie privée** : Un bon équilibre entre le travail et la vie privée pour éviter le burnout et maintenir la productivité.

30. **Diversité et inclusion** : Une équipe diversifiée et inclusive qui apporte une variété de perspectives.
31. **Gestion du stress** : Des stratégies efficaces de gestion du stress pour maintenir la santé mentale et la productivité.
32. **Soutien des pairs** : Un soutien solide et une camaraderie entre les membres de l'équipe.
33. **Feedback constructif** : Une culture de feedback constructif pour l'amélioration continue.
34. **Vision et objectifs clairs** : Une vision et des objectifs clairs pour guider le travail d'assurance qualité.
35. **Tests basés sur les risques** : L'utilisation de tests basés sur les risques pour prioriser les efforts de test.
36. **Gestion des attentes** : La gestion efficace des attentes des parties prenantes.
37. **Soutien à la carrière** : Le soutien à la croissance et au développement de carrière des professionnels de l'assurance qualité.
38. **Pratiques éthiques** : L'engagement envers des pratiques éthiques dans l'assurance qualité.
39. **Transparence** : La transparence dans les processus et les décisions d'assurance qualité.
40. **Sensibilisation à la qualité** : La sensibilisation à l'importance de la qualité dans toute l'organisation.
41. **Compétences en résolution de problèmes** : Des compétences solides en résolution de problèmes pour identifier et résoudre les problèmes de qualité.
42. **Compétences en pensée critique** : Des compétences en pensée critique pour évaluer les problèmes de qualité et prendre des décisions éclairées.
43. **Compétences en communication** : Des compétences en communication pour partager efficacement les informations sur la qualité.
44. **Compétences en collaboration** : Des compétences en collaboration pour travailler efficacement avec d'autres équipes et parties prenantes.
45. **Compétences techniques** : Des compétences techniques pour comprendre le logiciel et utiliser efficacement les outils d'assurance qualité.
46. **Compétences en gestion de projet** : Des compétences en gestion de projet pour planifier et gérer efficacement les efforts d'assurance qualité.
47. **Compétences en leadership** : Des compétences en leadership pour diriger les efforts d'assurance qualité et influencer les autres en faveur de la qualité.
48. **Compétences en négociation** : Des compétences en négociation pour gérer les conflits et obtenir les ressources nécessaires pour l'assurance qualité.
49. **Compétences en analyse de données** : Des compétences en analyse de données pour comprendre les tendances de qualité et prendre des décisions éclairées.
50. **Compétences en gestion du temps** : Des compétences en gestion du temps pour gérer efficacement le temps et les ressources dans les efforts d'assurance qualité.
51. **Compétences en gestion du changement** : Des compétences en gestion du changement pour gérer efficacement les changements dans le logiciel et les processus d'assurance qualité.
52. **Compétences en apprentissage autonome** : Des compétences en apprentissage autonome pour apprendre de manière autonome et rester à jour avec les dernières technologies et méthodologies.
53. **Compétences en gestion des risques** : Des compétences en gestion des risques pour identifier et atténuer les risques liés à la qualité.
54. **Compétences en gestion des conflits** : Des compétences en gestion des conflits pour gérer efficacement les conflits et les désaccords dans les efforts d'assurance qualité.
55. **Compétences en prise de décision** : Des compétences en prise de décision pour prendre des décisions éclairées et efficaces dans les efforts d'assurance qualité.
56. **Compétences en gestion des ressources** : Des compétences en gestion des ressources pour gérer efficacement les ressources disponibles pour l'assurance qualité.
57. **Compétences en gestion des parties prenantes** : Des compétences en gestion des parties

prenantes pour gérer efficacement les attentes et les besoins des parties prenantes.

58. **Compétences en gestion de la qualité** : Des compétences en gestion de la qualité pour gérer efficacement les efforts d'assurance qualité.
59. **Compétences en gestion des fournisseurs** : Des compétences en gestion des fournisseurs pour gérer efficacement les relations avec les fournisseurs d'outils et de services d'assurance qualité.
60. **Compétences en résolution de problèmes** : La capacité à résoudre efficacement les problèmes qui se posent lors des tests.
61. **Compétences en gestion du stress** : La capacité à gérer le stress qui peut survenir lors de la gestion de projets d'assurance qualité complexes.
62. **Compétences en gestion du temps** : La capacité à gérer efficacement le temps pour s'assurer que tous les aspects de l'assurance qualité sont correctement traités.
63. **Compétences en gestion de projet** : La capacité à gérer efficacement les projets d'assurance qualité, y compris la planification, la budgétisation, et la gestion des ressources.
64. **Compétences en leadership** : La capacité à diriger une équipe d'assurance qualité, y compris la motivation des membres de l'équipe, la résolution des conflits, et la promotion d'une culture de qualité.
65. **Compétences en négociation** : La capacité à négocier efficacement avec les autres parties prenantes, y compris les développeurs, les gestionnaires, et les clients.
66. **Compétences en analyse de données** : La capacité à analyser les données de test pour identifier les tendances, les anomalies, et les domaines d'amélioration.
67. **Compétences en pensée critique** : La capacité à évaluer de manière critique les processus de test, les outils, et les résultats pour améliorer constamment l'assurance qualité.
68. **Compétences en apprentissage autonome** : La capacité à apprendre de manière autonome et à rester à jour avec les dernières technologies et méthodologies d'assurance qualité.
69. **Compétences en gestion des risques** : La capacité à identifier, évaluer, et atténuer les risques associés à l'assurance qualité.
70. **Compétences en gestion des conflits** : La capacité à gérer efficacement les conflits au sein de l'équipe d'assurance qualité et avec d'autres parties prenantes.
71. **Compétences en prise de décision** : La capacité à prendre des décisions éclairées et efficaces en matière d'assurance qualité.
72. **Compétences en gestion des ressources** : La capacité à gérer efficacement les ressources disponibles pour l'assurance qualité, y compris les outils, le temps, et le personnel.
73. **Compétences en gestion des parties prenantes** : La capacité à gérer efficacement les attentes et les besoins des parties prenantes en matière d'assurance qualité.
74. **Compétences en gestion de la qualité** : La capacité à gérer efficacement les efforts d'assurance qualité, y compris la planification, l'exécution, et le suivi des tests.
75. **Compétences en gestion des fournisseurs** : La capacité à gérer efficacement les relations avec les fournisseurs d'outils et de services d'assurance qualité.
76. **Compétences en gestion du changement** : La capacité à gérer efficacement le changement dans les processus de test, les outils, et le logiciel lui-même.
77. **Compétences en gestion de la performance** : La capacité à gérer efficacement la performance de l'équipe d'assurance qualité, y compris la définition des objectifs, le suivi des progrès, et la fourniture de feedback.
78. **Compétences en gestion de la diversité** : La capacité à gérer efficacement la diversité au sein de l'équipe d'assurance qualité, y compris la promotion de l'inclusion et le respect des différences.
79. **Compétences en gestion de l'innovation** : La capacité à gérer efficacement l'innovation en matière d'assurance qualité, y compris l'adoption de nouvelles technologies et méthodologies.
80. **Compétences en gestion de la connaissance** : La capacité à gérer efficacement la connaissance au sein de l'équipe d'assurance qualité, y compris le partage des

connaissances, la documentation, et la formation.

81. **Compétences en gestion de l'éthique** : La capacité à gérer efficacement l'éthique en matière d'assurance qualité, y compris le respect des normes éthiques et la promotion d'une culture d'intégrité.
82. **Compétences en gestion de la conformité** : La capacité à gérer efficacement la conformité en matière d'assurance qualité, y compris le respect des normes et réglementations pertinentes.
83. **Compétences en gestion de la satisfaction des clients** : La capacité à gérer efficacement la satisfaction des clients en matière d'assurance qualité, y compris la collecte de feedback, la résolution des problèmes, et l'amélioration continue.
84. **Compétences en gestion de la motivation** : La capacité à gérer efficacement la motivation au sein de l'équipe d'assurance qualité, y compris la reconnaissance des efforts, la fourniture de récompenses, et la promotion d'un environnement de travail positif.
85. **Compétences en gestion de l'équilibre travail-vie personnelle** : La capacité à gérer efficacement l'équilibre travail-vie personnelle au sein de l'équipe d'assurance qualité, y compris la promotion de la flexibilité, la prévention du burnout, et la prise en compte des besoins personnels.
86. **Compétences en gestion de la résilience** : La capacité à gérer efficacement la résilience au sein de l'équipe d'assurance qualité, y compris la promotion de la résilience face aux défis, la gestion du stress, et la fourniture de soutien en cas de difficultés.
87. **Compétences en gestion de l'engagement** : La capacité à gérer efficacement l'engagement au sein de l'équipe d'assurance qualité, y compris la promotion de l'engagement envers les objectifs de l'équipe, la fourniture d'opportunités de développement, et la reconnaissance des contributions.
88. **Compétences en gestion de la collaboration** : La capacité à gérer efficacement la collaboration au sein de l'équipe d'assurance qualité, y compris la promotion de la collaboration, la résolution des conflits, et la facilitation de la communication.
89. **Compétences en gestion de l'efficacité** : La capacité à gérer efficacement l'efficacité de l'équipe d'assurance qualité, y compris la promotion de l'efficacité, la mise en place de processus efficaces, et la recherche d'améliorations continues.
90. **Compétences en gestion de l'efficience** : La capacité à gérer efficacement l'efficience de l'équipe d'assurance qualité, y compris la promotion de l'efficience, la minimisation des déchets, et la maximisation de l'utilisation des ressources.
91. **Compétences en gestion de l'excellence** : La capacité à gérer efficacement l'excellence au sein de l'équipe d'assurance qualité, y compris la promotion de l'excellence, la définition de normes élevées, et la recherche de l'amélioration continue.
92. **Compétences en gestion de l'adaptabilité** : La capacité à gérer efficacement l'adaptabilité au sein de l'équipe d'assurance qualité, y compris la promotion de l'adaptabilité, la gestion du changement, et la préparation à l'avenir.
93. **Compétences en gestion de l'agilité** : La capacité à gérer efficacement l'agilité au sein de l'équipe d'assurance qualité, y compris la promotion de l'agilité, l'adoption de méthodologies agiles, et la réponse rapide aux changements.
94. **Compétences en gestion de l'alignement** : La capacité à gérer efficacement l'alignement au sein de l'équipe d'assurance qualité, y compris l'alignement sur les objectifs de l'organisation, l'alignement sur les besoins des clients, et l'alignement sur les normes et réglementations pertinentes.
95. **Compétences en gestion de l'autonomie** : La capacité à gérer efficacement l'autonomie au sein de l'équipe d'assurance qualité, y compris la promotion de l'autonomie, la fourniture de responsabilités, et la confiance en la capacité des membres de l'équipe à prendre des décisions.
96. **Compétences en gestion de l'ambition** : La capacité à gérer efficacement l'ambition au sein de l'équipe d'assurance qualité, y compris la promotion de l'ambition, la définition

d’objectifs ambitieux, et le soutien à l’atteinte de ces objectifs.

97. **Compétences en gestion de l’appréciation** : La capacité à gérer efficacement l’appréciation au sein de l’équipe d’assurance qualité, y compris la reconnaissance des efforts, la célébration des succès, et la valorisation des contributions de chaque membre de l’équipe.
98. **Compétences en gestion de l’adaptabilité** : La capacité à gérer efficacement l’adaptabilité au sein de l’équipe d’assurance qualité, y compris la promotion de l’adaptabilité, la gestion du changement, et la préparation à l’avenir.
99. **Compétences en gestion de l’agilité** : La capacité à gérer efficacement l’agilité au sein de l’équipe d’assurance qualité, y compris la promotion de l’agilité, l’adoption de méthodologies agiles, et la réponse rapide aux changements.
100. **Compétences en gestion de l’alignement** : La capacité à gérer efficacement l’alignement au sein de l’équipe d’assurance qualité, y compris l’alignement sur les objectifs de l’organisation, l’alignement sur les besoins des clients, et l’alignement sur les normes et réglementations pertinentes.
101. ETC.

Conclusion

En conclusion, le succès dans l’assurance qualité du logiciel est comme un iceberg. Bien que nous ne voyions souvent que les résultats visibles, il est important de reconnaître et de valoriser les efforts et l’organisation qui se trouvent sous la surface. En faisant cela, nous pouvons non seulement améliorer la qualité de notre logiciel, mais aussi renforcer notre culture de qualité et notre capacité à réussir dans un environnement en constante évolution.

***Comment écrire un récit de l'utilisateur (user story) en symbiose avec l'assurance qualité et de façon efficace ?

January 18, 2024 10:20 AM

Les user stories sont une technique de description des besoins d'un utilisateur dans un langage simple et compréhensible pour tous les membres de l'équipe de développement. Les user stories ne sont pas des exigences détaillées, mais plutôt une description du résultat souhaité1. Les user stories s'intègrent parfaitement aux frameworks Agile comme Scrum et Kanban.

Pour écrire une user story efficace, il est important de bien comprendre les besoins des utilisateurs et d'identifier le profil de l'utilisateur final en cernant votre public cible. Des profils "Persona" peuvent vous aider à comprendre le problème que le personnage veut voir résolu en utilisant le produit, et ce, à l'aide d'une conception visuelle.

En ce qui concerne l'assurance qualité, il est important de s'assurer que les user stories sont claires et précises pour éviter toute confusion ou malentendu entre les membres de l'équipe1. Il est également important de s'assurer que les user stories sont testables et mesurables pour garantir leur efficacité.

Dans "User Story" nous oublions souvent le mot "utilisateur" (user) et pourtant ce qui est le plus important du "récit de l'***utilisateur***" !

Pourquoi nous avons la convention d'une STORY dans la stratégie ?... Simplement parce que la documentation/couverture/validation est l'essence même de la définition du besoin et de la solution (le DoD/ l'Acceptation)

Comment écrire un récit de l'utilisateur en symbiose avec la qualité et de façon efficace ?

Les "user stories" font partie d'une approche agile qui permet de passer de l'écriture des exigences à la discussion. Toutes les user stories agiles incluent une ou deux phrases écrites et, plus important encore, une série de conversations sur la fonctionnalité souhaitée.

Une description courte et simple d'une fonctionnalité racontée du point de vue de la personne qui souhaite la nouvelle fonctionnalité, généralement un utilisateur ou un client du système. Les "user stories" suivent généralement un modèle simple :

En tant que < utilisateur (type) >, je veux < objectif > pour que < raison >.

C'est quoi la dif. entre AsISo et Gerkins ?

Certains seront surpris, mais le but est le même

×

c'est juste une façon différente de dire le besoin

	AsISo	Gerkins
	AS who (include the context) **I WANT** objective **SO** reason	**GIVEN** context (include the who) **WHEN** action **THEN** result **EXIST**
exemple	**AS** an utilisateur **I WANT** convertir l'unité de température à partir de Fahrenheit	**GIVEN** fahrenheit **WHEN** convertir **THEN** Celsius **EXIST**

	SO que je puisse lire la température en Celsius	
Point de vu	Le besoin utilisateur	Le besoin fonctionnel

Par contre à NOTER !

AsISo est pour une fonctionnalité (feature), beaucoup vont penser que c'est pour une story !!! l'important est d'avoir **Qui, Quoi, Pourquoi** **"As a <user or stakeholder type>, I want <some software feature**>, So that <some business value>" or "In order to <achieve some business value>, As a <stakeholder type>, I want <some new system **feature**>"
Gerkin est pour un scenario l'important est d'avoir **Contexte, Action, Attente(s)** **Arrange, Act, Assert** **Contexte peut être Setup, Precondition, Prerequisite, etc.**

Mais alors comment cela peut nous aider à écrire une story ?

La réponse est par la définition du besoin, définition de la solution, ainsi les spécifications et sa couverture testable.

…….

Bon ajoutons la relation avec la qualité

Si nous voulons ajouter un validation d'un comportement inverse comme dans l'exemple... Qu'arrivera t-il si l'utilisateur est déjà en celsius, par exemple ?

Option A (avec une épopée)

Est-ce qu'il est bon de faire un autre story, donc un test/une validation par story et partir d'une épopée ?

Certains seront surpris, mais oui nous pourrions livrer seulement la conversion fahrenheit à celsius "happy path" et une autre story pour l'éligibilité avant la conversion.

Même que découper une story de cette façon nous simplifie la vie de beaucoup, c'est le concept INVEST et donc le plus petit format (unitaire) testable.

- ici on ne se pose pas la question de la couverture, car une story = un test
- Plus, facile a estimer, car c'est très claire
- etc.

De cette façon, nous utilisons les "epic" en tant qu'exigence, ce qui prends tout son sens du pourquoi une "epic" !

Maintenant, comment écrit-on une "epic" ?

Et bien, c'est simple, les stories sont la définition de l'épopée, pas besoin d'un roman, juste un titre est généralement suffisant.

Si on part avec le "**SO** convertir en Celsius" ou le " **WHEN TO** convertir en celsius", nous pouvons, par exemple, nommer l'épopée : "Convertir en celsius"

Option B (sans épopée)

Bon, dans la réalité, si on fait l'option A, un dev sufi pour réaliser une epic complète !

Nous voulons faire ça moins unitairement et plus au niveau de la "feature", donc plusieurs scenarios pour une fonctionnalité.
Nous nous retrouvons ici dans les situations les plus communes... avoir moins de stories... souvent le centre d'une exigence est la story et un découpe par la suite la story au besoin selon l'estimé trop grande

Donc on voudrait être moins granulaire...

Cela augmente le risque de ne pas pouvoir tester complètement la story ou...

Nous devons nous assurer de couvrir tout les scenarios possibles dans la story

todo

Quelle est la différence entre :

- Définition du besoin et de la solution
- Critère d'acceptation
- UAT
- Définition de terminer (DoD)
- Plan
- Couverture
- Test
- Conditions de succès
- Description/Sommaire

Certains seront surpris, mais le but est le même



Ce sont tous des façons différentes d'exprimer le même besoin/solution

Une équipes mature pourrait utiliser juste des récits (sans épopée) avec un simple titre par exemple "convertir celcius", rien d'autre sauf un lien d'un plan de test. Pourtant, tout est complet, pas de confusion et on peut faire le développement zero bug piloté par les tests.

Voyons ensemble le SDLC en lien avec une story

todo

(done) L'art d'écrire un bug à la "perfection"

January 18, 2024 10:21 AM

Un kata pour maitriser l’art d'écrire un bug

Dans le monde complexe de l’assurance qualité du logiciel, la capacité à rédiger un rapport de bug efficace est une compétence inestimable. Un rapport de bug bien rédigé peut faire la différence entre un bug qui est rapidement résolu et un bug qui reste non résolu pendant une période prolongée. Dans ce blog, nous allons explorer les éléments clés d’un rapport de bug “parfait”.

L’importance de la documentation

Un bon rapport de bug commence par une documentation solide. Cela comprend l’attribution d’un numéro unique à chaque bug, la fourniture d’une description détaillée du problème, et l’inclusion de preuves telles que des captures d’écran ou des vidéos. De plus, il est essentiel de documenter les étapes pour reproduire le bug, ainsi que toute autre information pertinente telle que la version du logiciel et l’impact sur l’utilisateur.

La classification du bug

Il est également important de classer correctement le bug. Cela comprend la détermination du type de bug, son emplacement dans le cycle de vie du développement, la couche de l’application affectée, et l’équipe ou la personne responsable. Cette classification aide à prioriser le bug et à planifier sa résolution.

L’évaluation du bug

L’évaluation du bug est une autre étape cruciale dans la rédaction d’un rapport de bug. Cela comprend l’évaluation de la criticité du bug, de sa priorité, de son impact sur les tests manuels et automatiques, de son impact sur l’utilisateur, et de son impact sur la vision de l’organisation. Cette évaluation aide à comprendre l’importance du bug et à allouer les ressources appropriées pour sa résolution.

L’importance de l’outil de gestion de test

Un outil de gestion de test est un atout précieux lors de la rédaction d’un rapport de bug. Il permet de suivre le bug et les tests associés, évitant ainsi de sur-documenter ou de réécrire les informations dans le rapport de bug.

La conclusion

En conclusion, la rédaction d’un rapport de bug “parfait” est un art qui nécessite de la précision, de la clarté, et une attention aux détails. En suivant ces bonnes pratiques, vous pouvez rédiger des rapports de bugs qui non seulement contribuent à améliorer la qualité du logiciel, mais aussi facilitent le travail de votre équipe et de l’organisation dans son ensemble.

(done) Agile VS Quality

January 18, 2024 10:21 AM

Agile vs Quality : Une Relation Complexe

Introduction

Il est courant de penser que l'Agile se moque complètement de la qualité. Cependant, c'est une idée fausse. L'assurance qualité est une partie intégrante de la méthodologie Agile.

Agile et Qualité : Un Lien Inextricable

L'approche Agile se concentre sur le déplacement du rôle de la qualité plus tôt et de manière continue tout au long du cycle de développement. Contrairement à des méthodes comme Waterfall, qui valident la qualité vers la fin d'un projet, Agile permet une pivotement et un changement de direction continus, même tard dans le jeu.

User Story et Qualité

Les User Stories jouent un rôle crucial dans la qualité en Agile. Elles aident à définir clairement les attentes et les critères de réussite pour chaque fonctionnalité, ce qui facilite le processus de test et d'assurance qualité.

Bonnes Pratiques

Une bonne pratique en Agile est la méthode "Product Backlog Building" (PBB). Elle aide à organiser et à prioriser les tâches, ce qui est essentiel pour maintenir la qualité tout en respectant les délais.

S'organiser Efficacement

Que vous utilisiez Kanban ou Scrum, une organisation efficace est la clé du succès en Agile. Cela permet de s'assurer que toutes les tâches sont suivies et que rien n'est négligé, ce qui contribue à maintenir la qualité.

Un Dernier Mot : Agile ou Waterfall ?

Si vous croyez que vous êtes Agile dans Jira mais que vous validez quand même la qualité vers la fin d'un projet ou de vos sprints ou de vos releases, je suis désolé de dire ça, mais vous n'êtes pas vraiment Agile, vous êtes en mode Waterfall. En effet, l'essence de l'Agile est l'intégration continue de la qualité tout au long du cycle de développement, et non pas une validation de la qualité à la fin. Êtes-vous Agile ?!

Conclusion

On ne peut pas faire de l'Agile sans qualité, mais on peut faire de la qualité sans l'Agile. Cependant, lorsque les deux sont combinés, ils peuvent conduire à des produits logiciels exceptionnels qui répondent aux besoins des utilisateurs tout en maintenant des normes de qualité élevées. N'oubliez pas, être Agile, c'est plus qu'un simple mot à la mode, c'est un engagement envers une approche de développement qui valorise la qualité à chaque étape.

C'est un challenge aujourd'hui d'avoir à la fois qualité et agilité.

C'est déjà difficile d'être Agile, si il faut faire de la qualité en plus, ah bin là !


On écrit les specs dans un Backlog et on découpe ça en sprints
Le Chef de Projet devient Scrum Master
On installe Jira
Et voilà, on est Agile !

(done) L'avenir tend vers l'ère du "BLACK-BOX testing" !

January 18, 2024 10:21 AM

L’avenir tend vers l’ère du “Black-Box Testing”

Introduction

Dans le monde en constante évolution du développement logiciel, les méthodologies de test doivent s’adapter et évoluer pour répondre aux nouvelles exigences et technologies. Une tendance émergente est l’adoption croissante du “Black-Box Testing”, en particulier avec l’arrivée de solutions comme Microsoft Dynamics.

Qu’est-ce que le “Black-Box Testing” ?

Le “Black-Box Testing” est une méthode de test où le système est considéré comme une “boîte noire”. Les testeurs n’ont pas besoin de connaître la structure interne ou la mise en œuvre du système. Au lieu de cela, ils se concentrent sur l’entrée et la sortie - les données d’entrée sont fournies et la sortie est vérifiée par rapport aux résultats attendus.

Pourquoi le “Black-Box Testing” ?

Avec des solutions comme Microsoft Dynamics, le “Black-Box Testing” devient de plus en plus pertinent. Ces solutions sont souvent complexes et personnalisées pour répondre aux besoins spécifiques de l’entreprise. Par conséquent, il peut être plus efficace de se concentrer sur les fonctionnalités et les performances du système du point de vue de l’utilisateur, plutôt que de se perdre dans les détails de la mise en œuvre.

L’Accès Limité Au Code Source

De plus en plus, les équipes de test n’ont pas accès au code source des applications qu’elles testent. Cela rend les approches de test traditionnelles comme le “White-Box Testing” ou le “Gray-Box Testing” impossibles à mettre en œuvre. Dans ces cas, le “Black-Box Testing” devient la seule option viable. Cela oblige les équipes à repenser leur stratégie de qualité et la gestion de leurs tests pour s’adapter à cette réalité.

Conclusion

L’avenir du test logiciel semble se diriger vers l’ère du “Black-Box Testing”. Cette approche offre de nombreux avantages, notamment une meilleure concentration sur l’expérience utilisateur et une plus grande efficacité des tests. Cependant, comme pour toute méthodologie, il est important de l’appliquer judicieusement et en conjonction avec d’autres techniques de test pour garantir la qualité du logiciel.

(done) GERKIN, tout un casse-tête !

January 18, 2024 10:21 AM

Gherkin : Tout Un Casse-Tête !

Introduction

Gherkin est un langage de spécification utilisé pour décrire le comportement des logiciels. Il est souvent utilisé dans les tests automatisés et est conçu pour être facile à lire et à comprendre pour les non-programmeurs. Cependant, sa mise en œuvre peut s'avérer complexe, surtout lorsqu'elle implique la collaboration entre un Product Owner (PO), un Quality Assurance (QA) et un développeur (Dev).

La Simplicité De Gherkin : Une Illusion ?

Bien que Gherkin puisse sembler simple à expliquer, sa mise en œuvre peut être un véritable casse-tête. En effet, écrire des tests en Gherkin nécessite non seulement une compréhension claire des exigences du logiciel, mais aussi une communication efficace entre toutes les parties prenantes.

L'Importance De La Collaboration

Une mise en œuvre réussie de Gherkin nécessite une collaboration étroite entre le PO, le QA et le Dev. Ensemble, ils peuvent travailler pour décrire le comportement attendu du logiciel et s'assurer que toutes les parties prenantes ont une compréhension commune des exigences.

Comment Écrire Des Tests En Gherkin

Voici quelques conseils pour écrire des tests en Gherkin :

- Utilisez des phrases simples et claires pour décrire le comportement attendu du logiciel.
- Évitez les détails techniques inutiles et concentrez-vous sur les résultats attendus.
- Utilisez des mots clés tels que Given, When et Then pour organiser vos scénarios de test.
- Évitez les scénarios de test trop longs ou trop complexes.
- Assurez-vous que chaque scénario de test est indépendant et peut être exécuté séparément.

Dépersonnaliser Les Scénarios De Test

Il est important de dépersonnaliser les scénarios de test et de se concentrer sur le comportement de la fonctionnalité plutôt que sur le point de vue de l'utilisateur. Par exemple, au lieu de dire "GIVEN I'm on login page WHEN I click on sign in button THEN I'm on the home page", il serait préférable de dire "GIVEN authentification WHEN authenticate THEN authentificated".

Pourquoi Suivre Cette Façon D'Écrire Un Scénario

Un scénario n'est pas un ensemble d'étapes, mais plutôt une description du comportement attendu d'une fonctionnalité. L'utilisation d'un seul scénario pour différents environnements de données et plusieurs validations est l'approche préconisée. De plus, il est important d'extraire les préconditions pour éviter d'avoir un GIVEN avec des étapes de reproduction. Cela permet d'isoler le test du contexte et de pouvoir exécuter le test dans différents contextes.

Conclusion

Gherkin est un outil puissant pour améliorer la communication et la collaboration au sein des équipes de développement et de test. Cependant, sa mise en œuvre peut être un défi.

En gardant à l’esprit les conseils ci-dessus, vous pouvez surmonter ce casse-tête et tirer pleinement parti de Gherkin.

(done) Quoi?! Le TDD ce n'est pas du code de "unit test" ?

January 18, 2024 10:22 AM

Quoi?! Le TDD ce n’est pas du code de ‘unit test’

Introduction

Le Test Driven Development (TDD) est une approche de développement logiciel qui consiste à écrire des tests avant d’écrire le code de production. Cette approche permet aux développeurs de se concentrer sur les exigences du logiciel et de s’assurer que le code est testé dès le début du processus de développement.

Le TDD : Une Approche Pré-Développement

Il est important de préciser que le TDD commence avant même le développement. Il débute lors de l’élaboration de ce qui doit être testé, comment cela doit être testé, etc. En fait, la création des plans de test, qui définissent ce qui va être testé et comment, fait partie intégrante du TDD. Et pourtant, à ce stade, nous n’avons pas encore commencé le développement.

Le TDD et les différents types de tests

Bien que le TDD soit souvent associé aux tests unitaires, il ne se limite pas à cette seule forme de test. Le TDD peut être utilisé pour écrire des tests d’intégration, des tests fonctionnels et même des tests d’acceptation. Il est important de noter que ces tests peuvent être automatisés ou non.

L’objectif principal du TDD

L’objectif principal du TDD est de s’assurer que le code est testé dès le début du processus de développement et que les développeurs se concentrent sur les exigences du logiciel. En écrivant des tests avant d’écrire le code de production, les développeurs peuvent s’assurer que le code est testable et qu’il répond aux exigences du logiciel.

Conclusion

Le TDD est une approche puissante qui va bien au-delà de l’écriture de tests unitaires. En mettant l’accent sur les tests dès le début du processus de développement, le TDD aide à garantir que le code est de haute qualité, testable et qu’il répond aux exigences du logiciel.

(done) Quoi?! Blog | Le BDD ce n'est pas de l'automatisation ni du code !

January 18, 2024 10:22 AM

Le BDD ce n’est pas de l’automatisation ni du code

Introduction

Le Behavior Driven Development (BDD) est une approche de développement logiciel qui se concentre sur le comportement attendu d’un logiciel plutôt que sur les détails techniques de sa mise en œuvre. Bien que le BDD implique souvent l’automatisation et le code, il ne se limite pas à ces aspects du développement logiciel.

Qu’est-ce que le BDD ?

Le BDD est avant tout une approche centrée sur le comportement qui vise à s’assurer que le logiciel répond aux exigences du client. Il est souvent utilisé pour écrire des tests qui sont compréhensibles pour les non-programmeurs, ce qui facilite la communication entre les développeurs, les testeurs et les parties prenantes non techniques.

Au-delà de l’automatisation et du code

Bien que le BDD puisse être utilisé pour écrire des tests automatisés, son objectif principal n’est pas l’automatisation ou le code. Au lieu de cela, le BDD se concentre sur la définition claire du comportement attendu du logiciel et sur la garantie que toutes les parties prenantes ont une compréhension commune de ce comportement.

Conclusion

En utilisant le BDD, les équipes de développement peuvent travailler ensemble pour décrire le comportement attendu d’un logiciel, indépendamment des détails techniques de sa mise en œuvre. Les tests écrits en utilisant le BDD peuvent être utilisés pour valider que le logiciel répond aux exigences du client, qu’ils soient automatisés ou non.

(done) Stratégie VS Plan ?

January 18, 2024 10:22 AM

Stratégie vs Plan de Test : Une Vue Globale De L'Assurance Qualité Du Logiciel

Introduction

Dans le domaine de l'assurance qualité du logiciel, il est essentiel de distinguer entre la stratégie globale de l'assurance qualité et les plans de test spécifiques. Bien qu'ils soient tous deux cruciaux pour assurer la qualité du logiciel, ils servent des objectifs différents et ont des implications différentes.

Stratégie d'Assurance Qualité : La Vue d'Ensemble

La stratégie d'assurance qualité est un ensemble de **principes**, **approches**, **techniques**, **tactiques** et **méthodologies** qui définissent la culture de la pratique de l'assurance qualité au sein d'une organisation. Elle est à un niveau élevé et évolue rarement. La stratégie d'assurance qualité fournit une vision globale de l'approche de l'assurance qualité, englobant non seulement les tests manuels et automatisés, mais aussi la gestion des bugs, la sécurité et d'autres aspects de l'assurance qualité.

Plan de Test : La Mise en Œuvre

Le plan de test, en revanche, est une **planification détaillée** de comment on va assurer la qualité d'une release. Il peut être attaché à une version spécifique et peut être découpé en sous-plans de test pour chaque bug et story dans le plan de test de la release. Le plan de test a une durée de vie définie et est constamment mis à jour pour refléter les changements dans les exigences ou les priorités.

Pourquoi Les Distinguer ?

La distinction entre la stratégie d'assurance qualité et le plan de test est cruciale pour une mise en œuvre efficace de l'assurance qualité. La stratégie d'assurance qualité fournit le cadre et les directives, tandis que le plan de test met en œuvre ces directives de manière spécifique et détaillée.

Conclusion

Dans l'assurance qualité du logiciel, il est important de comprendre la différence entre la stratégie d'assurance qualité et le plan de test. En gardant à l'esprit que la stratégie d'assurance qualité est le guide global et que le plan de test est la mise en œuvre spécifique de ce guide, les équipes peuvent travailler de manière plus efficace et produire un travail de meilleure qualité.

(done) Les chapeaux/Les habits

January 18, 2024 10:22 AM

L’habit ne fait pas le moine



L’Habit Ne Fait Pas Le Moine : L’Impact De L’Assurance Qualité Dans Les Rôles Et Le Multitâche Agile

Introduction

Dans le monde du développement logiciel Agile, l’assurance qualité joue un rôle crucial. Les individus endossent souvent plusieurs rôles, y compris celui de l’assurance qualité. Cependant, comme le dit le proverbe, “l’habit ne fait pas le moine”. Un titre ou un rôle ne définit pas nécessairement les compétences ou les contributions d’un individu.

L’Assurance Qualité Dans Agile

L’assurance qualité dans Agile est essentielle pour garantir la livraison de produits logiciels de haute qualité. Elle implique une série de tests et de vérifications pour s’assurer que le logiciel fonctionne comme prévu. Cependant, l’assurance qualité ne se limite pas à la détection et à la correction des bugs. Elle englobe également la prévention des bugs, l’amélioration continue des processus et la promotion d’une culture de qualité.

Les Rôles Dans Agile

Dans une équipe Agile, les rôles sont souvent fluides. Un développeur peut endosser le rôle de testeur, un Scrum Master peut contribuer au développement, et ainsi de suite. C’est ce qu’on appelle le **multitâche**. Cependant, il est important de noter que le fait d’endosser un rôle ne signifie pas nécessairement que l’individu possède toutes les compétences nécessaires pour ce rôle. C’est là que l’assurance qualité entre en jeu, en veillant à ce que chaque rôle soit exécuté correctement et efficacement.

Le Multitâche Dans Agile

Le multitâche est une compétence précieuse dans une équipe Agile. Il permet à l’équipe de rester flexible et de s’adapter rapidement aux changements. Cependant, le multitâche peut aussi être un défi. Il peut mener à l’éparpillement et à une diminution de la qualité du travail. Encore une fois, l’assurance qualité joue un rôle crucial ici, en veillant à ce que le multitâche n’entraîne pas une baisse de la qualité.

L’Habit Ne Fait Pas Le Moine

C’est là que le proverbe “l’habit ne fait pas le moine” entre en jeu. Juste parce qu’un individu endosse un certain rôle, cela ne signifie pas qu’il est le plus qualifié ou le plus compétent pour ce rôle. Il est important de reconnaître les forces et les faiblesses de chaque membre de l’équipe et de les utiliser de manière optimale. L’assurance qualité aide à identifier ces forces et faiblesses et à s’assurer que chaque individu est le mieux placé pour réussir.

Conclusion

Dans le développement logiciel Agile, l’assurance qualité est essentielle pour garantir la livraison de produits de haute qualité. Elle joue un rôle crucial dans la gestion des rôles et du multitâche au sein de l’équipe. En reconnaissant cela, les équipes peuvent travailler de manière plus efficace et produire un travail de meilleure qualité.

(done) Lean vs Six Sigma

January 18, 2024 10:23 AM

Lean vs Six Sigma : Une Comparaison

Introduction

Dans le domaine de l'amélioration des processus et de la gestion de la qualité, deux méthodologies se distinguent : Lean et Six Sigma. Bien qu'elles aient des objectifs similaires, elles diffèrent par leur approche et leur mise en œuvre.

Lean : Réduire Les Déchets

La méthodologie Lean vise à **réduire les déchets** et à améliorer l'efficacité en identifiant la valeur et le flux de valeur. Elle se concentre sur l'élimination des déchets pour améliorer les performances. L'hypothèse sous-jacente est que de nombreuses petites améliorations sont préférables à une analyse systémique. L'effet principal est une réduction du temps de flux, avec comme effet secondaire une diminution de la variation.

Six Sigma : Réduire La Variation

La méthodologie Six Sigma, quant à elle, vise à **réduire la variation**. Elle suit une approche DMAIC (Définir, Mesurer, Analyser, Améliorer, Contrôler) et part du principe qu'un problème existe. L'hypothèse sous-jacente est que les chiffres et les nombres sont valorisés. L'effet principal est une sortie de processus uniforme, avec comme effet secondaire une diminution des déchets.

Critiques

Chacune de ces méthodologies a ses critiques. Pour Lean, on critique souvent le fait que l'analyse statistique ou systémique n'est pas valorisée. Pour Six Sigma, la critique est que l'interaction du système n'est pas prise en compte et que les processus sont améliorés indépendamment.

Conclusion

Lean et Six Sigma sont deux méthodologies puissantes pour l'amélioration des processus et la gestion de la qualité. Comprendre leurs différences et leurs similitudes peut aider à choisir la bonne approche pour votre organisation.

(done) Test sanitaire (intégrité, santé, salubrité...)

January 18, 2024 10:23 AM

Les Tests Sanitaires Dans Le Domaine Du Logiciel : Intégrité, Santé, Salubrité Et Plus

Introduction

Dans le domaine des tests logiciels, la terminologie peut souvent être déroutante. Les termes “sanity”, “intégrité”, “santé”, “salubrité” et d’autres sont couramment utilisés, mais que signifient-ils exactement ? Et comment s’inscrivent-ils dans le processus de test logiciel ?

Les Tests Sanitaires : Un Pré-Diagnostic

Les tests sanitaires sont comme un **pré-diagnostic** ou un **check-up**. Ils sont généralement effectués **après un build**, un package, une installation, une publication, etc., mais **avant de procéder aux tests de laboratoire plus profonds et plus poussés**.

C’est comme chez le docteur : on n’arrive pas chez le docteur et dès le début on passe un scan, les radios, et tout le tralala. De même, lorsqu’on va au garage, on fait un test de démarrage, on vérifie s’il n’y a rien qui chauffe (smoke), ensuite un court test drive (opération), mais pas une inspection complète.

Les Sous-Types De Tests Sanitaires

Il existe plusieurs sous-types de tests sanitaires, parmi lesquels :

- Test de fumée (smoke)
- Test de déploiement
- Test d’installation
- Test d’intégrité des données
- Test de configuration
- Test d’intégrité de la navigation
- Test d’opération
- Test de démarrage (Boot test)
- Test de connexion
- etc…

Les Tests Sanitaires : Quand Et Comment ?

Nous pouvons effectuer des tests sanitaires **par système**, **par intégration** et **unitaire**. Nous pouvons effectuer des tests sanitaires **après une livraison**, aussi petite soit-elle (in&out). Nous pouvons effectuer des tests sanitaires **à toutes les phases du cycle de développement**.

Il est important de noter que les tests sanitaires **ne sont pas exhaustifs** et ne doivent pas être confondus avec les tests d’acceptation utilisateur (UAT). Ce sont des tests de basculement très rapides, qui deviendront souvent les premiers tests automatisés de non-régression pour maintenir une stabilisation saine de chacune des livraisons.

Le Test De Fumée : Un Sous-Type De Tests Sanitaires

Le test de fumée (“smoke”) est un sous-type de tests sanitaires. Il valide que **tous les morceaux sont là**, mais surtout qu’il **n’y a pas de fuite** (que tout est étanche).

Conclusion

Comprendre la terminologie des tests logiciels est essentiel pour naviguer dans ce domaine

complexe. En définissant clairement des termes tels que “sanity”, “intégrité”, “santé”, “salubrité” et d’autres, nous pouvons mieux comprendre et évaluer la qualité de nos systèmes logiciels.

(done) Pourquoi tant d'entreprises échouent-elles si souvent dans l'assurance qualité, même dans les tests manuels et automatisés.

January 18, 2024 10:23 AM

❌ attentes de temps irréalistes ?
❌ désalignement commercial ?
❌ complexité des infrastructures ?
❌ manque de ressources et d'expertise ?
❌ manque de bons outils et/ou mal utilisés ?

Il y a plusieurs raisons pour lesquelles les entreprises peuvent échouer dans l'assurance qualité logicielle, même lorsqu'elles utilisent des tests manuels et automatisés. Voici quelques raisons courantes :

Manque de planification : Les entreprises peuvent ne pas avoir de plan clair pour l'assurance qualité logicielle, ce qui peut entraîner des tests inefficaces ou inadéquats.

Manque de ressources : Les entreprises peuvent ne pas avoir suffisamment de ressources pour effectuer des tests adéquats, ce qui peut entraîner des erreurs et des défauts dans le logiciel.

Manque de compétences : Les entreprises peuvent ne pas avoir les compétences nécessaires pour effectuer des tests efficaces, ce qui peut entraîner des erreurs et des défauts dans le logiciel.

Manque de collaboration : Les entreprises peuvent ne pas encourager la collaboration entre les équipes de développement et d'assurance qualité, ce qui peut entraîner des erreurs et des défauts dans le logiciel.

Manque d'automatisation : Les entreprises peuvent ne pas automatiser suffisamment leurs tests, ce qui peut entraîner des tests inefficaces ou inadéquats.

En utilisant une approche d'assurance qualité logicielle bien planifiée et en s'assurant que les ressources, les compétences et la collaboration sont en place, les entreprises peuvent améliorer leur taux de réussite en matière d'assurance qualité logicielle.

Les Défis De L'Assurance Qualité : Pourquoi Tant D'Entreprises Échouent-Elles ?

Introduction

L'assurance qualité (AQ) est un élément crucial du développement logiciel. Cependant, de nombreuses entreprises rencontrent des difficultés dans ce domaine, même avec les tests manuels et automatisés. Pourquoi cela se produit-il ?

Manque De Ressources

L'un des défis majeurs est le manque de ressources. Les tests nécessitent du temps, des compétences et des outils appropriés. Souvent, les entreprises sous-estiment ces besoins, ce qui conduit à des échecs en AQ.

Complexité Du Logiciel

La complexité croissante des logiciels rend les tests plus difficiles. Avec des architectures de plus en plus complexes et des intégrations multiples, les tests deviennent une tâche ardue.

Absence De Culture De Test

Dans certaines entreprises, il n'y a pas de culture de test solide. Cela signifie que les tests sont souvent négligés ou réalisés à la hâte, ce qui conduit à des échecs en AQ.

Dépendance Excessive Sur L'Automatisation

L'automatisation des tests est essentielle, mais une dépendance excessive à son égard peut conduire à des problèmes. Les tests automatisés ne peuvent pas remplacer complètement les tests manuels, en particulier pour les aspects liés à l'expérience utilisateur.

Conclusion

Pour réussir en AQ, les entreprises doivent investir suffisamment de ressources, gérer la complexité du logiciel, instaurer une culture de test solide et trouver le bon équilibre entre les tests manuels et automatisés. C'est un défi, mais c'est essentiel pour la livraison de logiciels de haute qualité.

***Quoi?! Nous pouvons automatiser les validations avant le développement !

January 18, 2024 10:24 AM

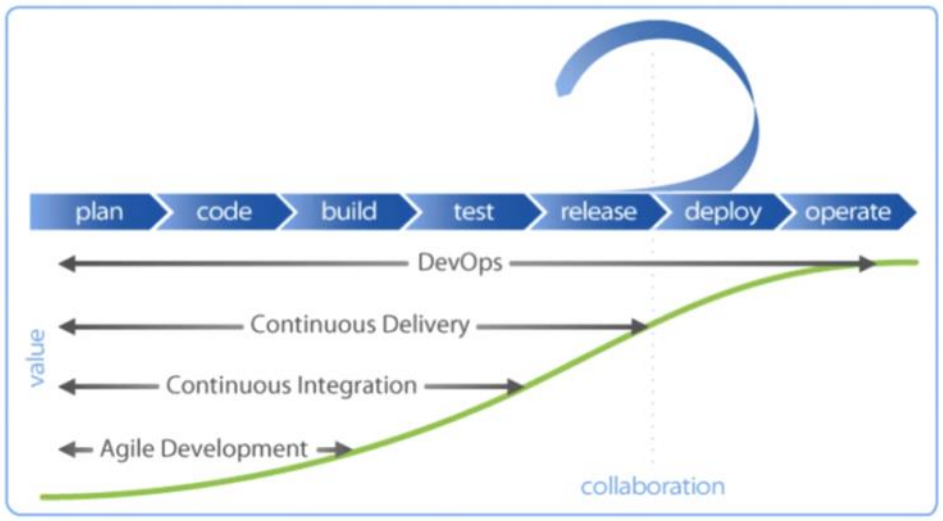
https://youtu.be/yU2psZzIdRI

***Développement en continu Agile

January 18, 2024 10:24 AM

J’aime cette image mais il y aurais quelques modification a faire

Add label



(done) Quality Over Quantity

January 18, 2024 10:24 AM

La Qualité Plutôt Que La Quantité : Un Impératif Dans Le Développement Logiciel

Introduction

Dans le domaine du développement logiciel, l'adage "la qualité plutôt que la quantité" prend une signification particulière. Il est essentiel de privilégier la qualité du code et des fonctionnalités plutôt que leur nombre.

La Qualité Du Code

Un code de qualité est un code **propre**, **efficace** et **maintenable**. Il est préférable d'avoir un code plus court et plus clair, qui fait exactement ce qu'il est censé faire, plutôt que d'avoir un code volumineux et complexe.

La Qualité Des Fonctionnalités

Dans le développement de logiciels, il est préférable de se concentrer sur la création de quelques fonctionnalités de haute qualité plutôt que de nombreuses fonctionnalités médiocres. Les fonctionnalités de haute qualité sont celles qui répondent efficacement aux besoins des utilisateurs et améliorent leur expérience.

La Qualité Dans La Gestion De Projet

Dans la gestion de projet logiciel, privilégier la qualité signifie aussi adopter des méthodologies agiles, favoriser la communication et la collaboration, et mettre l'accent sur l'amélioration continue.

Conclusion

Dans le domaine du logiciel, la qualité plutôt que la quantité est un choix stratégique qui peut conduire à un meilleur code, de meilleures fonctionnalités et une meilleure gestion de projet. C'est un choix qui nécessite de la discipline, mais les bénéfices en termes de satisfaction des utilisateurs et de réussite du projet en valent la peine.

***Scrum, Scrumban ou Kanban ?

January 18, 2024 10:24 AM

SCRUM versus KANBAN

Scrum et Kanban sont deux méthodologies Agile populaires pour la gestion de projet. Bien qu'ils partagent des similitudes, ils ont des différences clés qui peuvent les rendre plus ou moins adaptés à différents projets.

Scrum est une méthodologie Agile qui se concentre sur la livraison de fonctionnalités en itérations courtes appelées sprints. Les équipes Scrum travaillent ensemble pour atteindre un objectif commun et se réunissent régulièrement pour discuter de leur progression et des obstacles éventuels.

Kanban est une méthodologie Agile qui se concentre sur la visualisation du flux de travail et la limitation du travail en cours. Les équipes Kanban travaillent ensemble pour améliorer continuellement leur processus de travail en identifiant les goulots d'étranglement et les inefficacités.

Le choix entre Scrum et Kanban dépendra des besoins spécifiques de votre projet. Si votre projet nécessite une livraison rapide de fonctionnalités et une collaboration étroite entre les membres de l'équipe, Scrum peut être plus adapté. Si votre projet nécessite une amélioration continue du processus de travail et une visualisation claire du flux de travail, Kanban peut être plus adapté.

C'est simple !

Se sont tout les deux un cadre AGILE pour développer, livrer et maintenir des produits dans le cycle de vie du développement logiciel.

	SCRUM	KANBAN
Fonction	Boucle dont le but est de livrer le plus de cartes possible dans une fréquence (période fixe, constante et de même durée)	Boucle dont le but est de livrer à la carte le plus rapidement possible sur ce qu'il y a à livrer (donc aucune fréquence, nous sommes en continu)
Définition	Est un cadre de sprint (itération), c'est un système de planification de production par prévision à court terme. Un sprint n'est pas un marathon ni une course à relais ! nous voyons souvent… • UI/UX, architecture, analyse dans le **sprint-1** • Dev dans **sprint-current** • QA **sprint+1** De plus, les cartes d'un sprint doivent avoir entièrement passées à travers le cycle de vie du développement pour en faire une rétrospective et dire qu'un sprint est terminé 😳	Est un système de planification de production Lean et « juste à temps » à la pièce et en continu… LEAN is… Tiny fast fails → quick learning → big success ! Le Kanban utilise des cartes comme signalisation.
Équipe (constitution)	Équipe **produit** • PO • SM	Équipe **projet** (peut toucher un ou plusieurs produits) • PM

	• ANALYSTE • DEV	• ARCHITECTE • UI • UX • Redac • Coach AGILE • ANALYSTE • DEV • FM
Les rencontres (**Cérémonies Agile**)	Scrum (**MÊLÉE**) Dans le jargon SPRINT(Scrum) on va parler d'une "mêlée" Des rencontres récurrentes et prévues à chaque sprint • Quotidienne Sprint • Affinage (cadrage) • Planification (démarrage) • Révision (revue, démo) • Rétrospective	Kanban (**CAUCUS**) Kanban veut dire "enseigne" (signboard) Les rencontres en Kanban sont pilotées par les phases du cycle de vie du développement et nous allons plutôt dire un **CAUCUS**, contrairement au Scrum qui est une **mêlées**. Tout fonctionne à la carte par les étapes qu'elle devra franchir (DoD) Ce n'est pas dans le cadre de la couche supplémentaire de sprint mais plutôt directement dans la documentation et dans le cadre du SDLC Cycle de vie du développement logiciel (SDLC)ARCHIVED Donc, nous pouvons demander, au besoin, à tout moment pour peut importe la raison (car le but c'est de livrer le plus rapidement possible) Tout est en continu… tester, valider, approver, améliorer, innover, le feedback, la planification, l'intégration, la livraison, la révision, la diffusion/publication, l'opération, ou mesurer, etc…
Comment être Agile et fournir des prévisions à long terme (Roadmaps) Même si Roadmaps est l'antipode de la méthodologie Agile, nous pouvons très bien fournir des estimées	Nous pouvons fournir un estimé à n'importe quelle phases du développement. Cependant, il y une subtilité en SCRUM (sprint), nous pouvons nous engager sur une soumission (story point) seulement au démarrage d'un sprint. Et par la suite raffiner notre évaluation pour les prochains estimés, jusqu'à trouver une moyenne pour l'estimation de la taille. Important, cela reste qu'une estimation… Et une estimation juste ne peut que se faire par la comparaison d'une réalisation/livraison	Nous pouvons fournir un estimé à n'importe quelle phases du développement. En Kanban tout se passe dans les statistiques ! Plus nous documentons la traçabilité et plus nous livrons de fois, plus nous aurons l'heure juste sur l'estimation. Il n'a pas d'autres engagements que de livrer le plus vite possible Important, cela reste qu'une estimation… Et une estimation juste ne peut que se faire par la comparaison d'une réalisation/livraison
La fréquence (le plus	Découper pour obtenir la plus petit	Découper pour obtenir la plus petit

souvent) et le découpage des livraisons	pièce, testable et livrable, possible	pièce, testable et livrable, possible Ici c'est non seulement le plus souvent mais sur le plus rapide
Pour qui ?	Pour les équipes Agile	Pour les équipes Agile
Encourage		• Briser les silos • Briser les goulots (force à régler les problèmes d'infra. ou qu'on ne veut pas voir) • Prise de conscience rapide • Évite le ping-pong •
Piège	• Facilite la création des silos • Désorganise/brise la gestion de projet et d'une entreprise • Facilité la confusion et la perte de confiance • Augment la sur-documentation	
Quelle choisir ?	Si nous avons le besoin de se compliquer la vie ou s'ajouter du faut travail, sans aucune réelle valeur ajoutée tout en voulant se la péter avec le prestige de dire le buzz word "SCRUM" dans nos phrases… Si oui, c'est parfait "Welcome SCRUM"	Si nous voulons être simple, rapide et efficace de haut en bas et de gauche à droite peut importe la maturité des technos, des méthodologies, des pratiques… Et que vous voulez vivre le vrai potentiel de l'Agile. Si oui, c'est donc que le KANBAN est pour vous !

- Il est faut de dire que c'est mieux de commencer avec le SCRUM si on veut bien faire du KANBAN
- Il faut de dire que SCRUM c'est plus Agile, fourni de la meilleure qualité, mieux structuré, mieux organisé, plus transparent et plus claire de ce qui a à faire, d'avoir de meilleures "stories", de mieux estimer, de mieux livrer, de mieux tester, de mieux s'améliorer, de mieux communiquer, d'être plus engagé, plus d'équipe…
  En gros, SCRUM ce n'est, définitivement, pas mieux que KANBAN !
- Il faut de dire qu'une équipe SCRUM est plus soudée, dure plus longtemps, s'entraide plus, se chicane moins…
-
- Il est faut de dire que le Kanban c'est plus pour la maintenance et les bugs… quoi que c'est vrai que c'est mieux finalement lol !

Voilà que maintenant nous pouvons démêler les mêlées (scrums) dans la méthodologie Agile by SCRUM(s)
*Ce n'est pas parce que vous utiliser le mot SCRUM que vous êtes AGILE

×

Mauvaise habitude	**fr**	**en**

Mauvaise habitude	**fr**	**en**
Le scrum	Mêlée quotidienne	Daily scrum
La rétro	Mêlée rétrospective du sprint	Sprint retrospective

		scrum
La revue	Mêlée de révision de sprint	Sprint review scrum
Grooming (*mot à connotation criminel, c'est à proscrire/bannir*)	Affinage ou Raffinement du carnet de commandes	Backlog Refinement
Le scrum de scrum (*quel est lu but d'une rencontre d'une rencontre ?*)	Faudrait être plus claire sur la définition, raison le besoin et le but… Voici des exemples	

(done) Les malentendus courants sur le rôle du QA

January 18, 2024 10:25 AM

Introduction

Dans le monde du développement de logiciels, le rôle et les compétences des professionnels de l'assurance qualité (QA) sont souvent mal compris. Cela peut créer de nombreuses frustrations et inefficacités au sein des équipes de développement.

Les attentes et les compétences des professionnels du QA

Les professionnels du QA ont une gamme de compétences et de responsabilités qui vont bien au-delà des tests simples. Par exemple, un Coach QA peut se concentrer sur l'amélioration des processus de QA et la formation des membres de l'équipe, tandis qu'un analyste QA peut se concentrer sur la planification et l'exécution des tests.

Cependant, ces rôles sont souvent mal compris. Par exemple, un Coach QA ou un analyste QA n'est pas nécessairement un testeur ou un automaticien. Leur rôle principal est d'assurer la qualité à tous les niveaux du processus de développement, et non de réaliser des tests ou de coder des scripts de test.

Les frustrations causées par les malentendus

Ces malentendus peuvent causer de nombreuses frustrations. Par exemple, si les membres de l'équipe s'attendent à ce que le QA effectue tous les tests, ils peuvent négliger leur propre responsabilité en matière de qualité. De même, si les membres de l'équipe s'attendent à ce que le QA écrive tous les scripts de test, ils peuvent négliger l'importance de comprendre et de participer au processus de test.

Conclusion

Il est essentiel de comprendre et de respecter le rôle et les compétences des professionnels du QA. En clarifiant les attentes et en valorisant l'expertise du QA, les équipes de développement peuvent travailler plus efficacement et produire des logiciels de meilleure qualité.

(done) Les pièges de l’automatisation et de l’ajout de QA : Une fausse solution ?

January 18, 2024 10:25 AM

Introduction

Dans le monde de l’assurance qualité (QA), l’automatisation des tests et l’ajout de ressources QA sont souvent considérés comme des solutions pour améliorer la qualité et l’efficacité. Cependant, ces approches peuvent parfois avoir l’effet inverse et créer plus de problèmes qu’elles n’en résolvent.

L’automatisation des tests : Une solution miracle ?

L’automatisation des tests peut sembler être une solution miracle pour améliorer l’efficacité des tests. Cependant, si elle n’est pas correctement mise en œuvre, l’automatisation peut rapidement devenir un cauchemar. Par exemple, si les tests automatisés ne sont pas correctement conçus ou maintenus, ils peuvent devenir obsolètes ou produire des résultats incorrects. De plus, l’automatisation peut parfois donner un faux sentiment de sécurité, conduisant à négliger d’autres aspects importants des tests.

Ajouter plus de QA : La solution à tous les problèmes ?

De même, l’ajout de plus de ressources QA peut sembler être une solution pour améliorer la qualité. Cependant, cela peut parfois conduire à une communication inefficace, à une duplication du travail et à une augmentation des coûts. De plus, cela ne résout pas le problème fondamental si les processus de QA existants sont défectueux.

Résoudre le problème à la source

Plutôt que de se tourner vers ces “fausses solutions”, il est important de résoudre le problème à la source. Cela peut impliquer de revoir les processus de QA existants, de former l’équipe sur les meilleures pratiques de QA, et d’adopter une approche de qualité intégrée où la qualité est prise en compte à chaque étape du processus de développement.

Conclusion

Bien que l’automatisation des tests et l’ajout de ressources QA puissent sembler être des solutions attrayantes, il est important de comprendre qu’ils ne sont pas toujours la réponse. Il est essentiel de résoudre les problèmes à la source et de s’assurer que les processus de QA sont solides et efficaces.

(done) Les tests statiques versus dynamiques

January 18, 2024 10:26 AM

Introduction

Dans le domaine du développement logiciel, les tests jouent un rôle crucial pour assurer la qualité du produit final. Les tests peuvent être classés en deux grandes catégories : les tests statiques et les tests dynamiques.

Les tests statiques

Les tests statiques sont des tests qui sont effectués sans exécuter le code. Ils impliquent l'examen du code, de la documentation et des exigences du logiciel pour trouver les erreurs. Les tests statiques peuvent inclure des activités telles que la revue de code, l'analyse statique du code et l'inspection des exigences.

Les tests dynamiques

Les tests dynamiques, en revanche, impliquent l'exécution du code. Ils vérifient le comportement du logiciel en réponse à certaines entrées ou conditions. Les tests dynamiques peuvent inclure des tests unitaires, des tests d'intégration, des tests de système et des tests d'acceptation.

Comparaison entre les tests statiques et dynamiques

Alors, comment choisir entre les tests statiques et dynamiques ? En réalité, les deux types de tests sont complémentaires et devraient être utilisés ensemble pour assurer la qualité du logiciel. Les tests statiques sont utiles pour détecter et corriger les erreurs tôt dans le cycle de développement, ce qui peut aider à réduire les coûts de correction des bugs. Les tests dynamiques, quant à eux, sont essentiels pour vérifier que le logiciel fonctionne comme prévu dans des conditions réelles.

Conclusion

En somme, les tests statiques et dynamiques sont deux aspects essentiels de l'assurance qualité logicielle. En utilisant ces deux types de tests de manière appropriée et efficace, vous pouvez grandement améliorer la qualité de vos produits logiciels.
Différence entre les tests statiques et dynamiques (avec tableau de comparaison, méthodes et outils) - Technologie - 2023 (suzuran-law-office.com)

(done) Outil de gestion de test versus Référentiel

January 18, 2024 10:26 AM

Introduction

Dans le monde du développement logiciel, la gestion des tests est une étape cruciale. Cela implique l'utilisation d'outils de gestion de tests et de référentiels de tests. Ces deux types d'outils jouent des rôles différents mais complémentaires dans le processus de test.

Outil de gestion de test

Un outil de gestion de test, aussi connu sous le nom de test manager, est un outil qui permet de gérer les tests pour les projets informatiques. Il peut être utilisé pour planifier les tests, suivre l'état des tests et générer des rapports sur les résultats des tests. De plus, l'outil de gestion de test peut également être utilisé pour stocker des artefacts liés aux tests tels que des plans de test, des cas de test et des scripts de test.

Voici quelques exemples d'outils de gestion de test :

- Aqua ALM
- GoTest
- IBM Rational Quality Manager
- Kualitee
- PractiTest
- QA Wolf
- qTest
- Test Collab
- TestLink
- TestRail
- Tuskr
- Xray
- Zephyr

Référentiel de test

Un référentiel de test est un outil qui stocke et gère les artefacts de test pour les projets informatiques. Le référentiel de test peut stocker des plans de test, des cas de test, des scripts de test et d'autres artefacts liés aux tests. Le référentiel de test peut également être utilisé pour suivre l'état des tests et pour générer des rapports sur les résultats des tests.

1. HPE Quality Center (QC)
2. Microsoft Test Manager (MTM) / Team Foundation Server (TFS)
3. Squash Test Management ™
4. TestLink
5. Xray for Jira
6. GoTest

Il y a des outils qui peut être à la fois gestion de test et référentiel

Conclusion

En résumé, l'outil de gestion de test est utilisé pour gérer les tests tandis que le référentiel de test est utilisé pour stocker et gérer les artefacts liés aux tests. Bien que ces deux types d'outils aient des fonctions différentes, ils sont tous deux essentiels pour une gestion efficace des tests dans les projets informatiques.

(done) Réussir en tant qu’employé de l’assurance qualité

January 18, 2024 10:26 AM

Introduction

L’assurance qualité est un élément crucial de tout processus de développement de logiciel. Pour réussir en tant qu’employé de l’assurance qualité, il y a plusieurs compétences et connaissances que vous devez maîtriser.

Connaissance des processus d’assurance qualité et des normes de l’industrie

Il est important d’avoir une connaissance approfondie des processus d’assurance qualité et des normes de l’industrie. Cela comprend la compréhension des différentes méthodologies d’assurance qualité, des normes de codage, des processus de test, et plus encore.

Compréhension des outils et des technologies

Avec l’évolution rapide de la technologie, il est également important d’avoir une bonne compréhension des outils et des technologies utilisés dans l’assurance qualité. Cela peut inclure des outils de suivi des bugs, des plateformes de tests automatisés, des outils de gestion de projet, et plus encore.

Communication efficace

Une bonne communication avec les membres de l’équipe et les parties prenantes est essentielle pour s’assurer que les projets sont livrés à temps et dans les limites du budget. Cela comprend la capacité à communiquer clairement les problèmes, les progrès et les résultats des tests à toutes les parties concernées.

Travail d’équipe et résolution de problèmes

Enfin, il est important d’être capable de travailler en équipe et de résoudre les problèmes rapidement et efficacement. L’assurance qualité est souvent un effort d’équipe, et être capable de collaborer efficacement avec les autres est crucial.

Conclusion

En somme, réussir en tant qu’employé de l’assurance qualité nécessite une combinaison de connaissances techniques, de compétences en communication, et de capacités de travail en équipe et de résolution de problèmes. Avec ces compétences, vous serez bien équipé pour réussir dans le domaine de l’assurance qualité.

(done) Quel est le lien entre cycle de vie du développement devops ou traditionnel et Jira ?

January 18, 2024 10:26 AM

Introduction

Jira est un outil de gestion de projet largement utilisé dans le monde du développement logiciel. Il est conçu pour aider les équipes à planifier, suivre et gérer leurs projets. Cependant, comme tout outil, son efficacité dépend de la façon dont il est utilisé.

Le lien entre le cycle de vie du développement et Jira

Que vous suiviez une approche de développement traditionnelle ou DevOps, Jira peut être un outil précieux. Il permet de créer des tickets pour chaque tâche, de les attribuer à des membres de l'équipe et de suivre leur progression. Cependant, le lien entre le cycle de vie du développement et Jira va au-delà de la simple gestion des tâches.

Dans un cycle de vie de développement DevOps, par exemple, Jira peut être utilisé pour gérer le backlog de produits, planifier les sprints, suivre les bugs, et même intégrer avec des outils de CI/CD pour un suivi en temps réel de la livraison du code.

L'absurdité de Jira

Cependant, malgré ses fonctionnalités robustes, Jira peut souvent sembler absurde et frustrant. Un des problèmes majeurs est que beaucoup de gens l'utilisent mal. Souvent, les administrateurs trouvent trop complexe de changer les états, les colonnes et les configurations. Cela peut conduire à une utilisation inefficace de l'outil, où les équipes se retrouvent mal équipées pour gérer leurs projets.

L'idée derrière Jira est de représenter le plus possible toutes les tâches à faire dans le cycle complet du développement. Cependant, chaque équipe veut des configurations personnalisées, souvent parce qu'elle ne comprend pas comment l'outil est censé fonctionner. Cela a pour conséquence de créer un désordre monumental. Finalement, Jira devient contre-productif, créant une boucle sans fin de confusion et de frustration au détriment des équipes de développement.

Conclusion

C'est une triste réalité que Jira, malgré son potentiel, peut finalement nuire plus qu'aider si mal utilisé. Il est donc crucial de comprendre non seulement comment utiliser Jira, mais aussi comment l'adapter aux besoins spécifiques de votre équipe et de votre projet.

(done) Est-ce que SAFe c’est si Agile, est-ce que c’est “safe” !?

January 18, 2024 10:26 AM

Introduction

SAFe (Scaled Agile Framework) est une méthode de développement Agile qui permet de gérer les projets à grande échelle. Bien que SAFe soit considéré comme une méthode Agile, il y a des débats sur la question de savoir si SAFe est vraiment Agile. Cependant, SAFe est conçu pour aider les entreprises à adopter l’Agilité à grande échelle.

Diagnostic Agile

Un diagnostic Agile peut aider à évaluer la maturité de l’organisation, sa trajectoire de progression et les cibles à atteindre sur les 3 leviers de l’agilité : le capital humain, le modèle opérationnel et le patrimoine technologique. En fin de compte, la transition vers SAFe doit être une décision stratégique basée sur les besoins spécifiques de l’organisation et sa capacité à adopter et à maintenir les principes agiles à grande échelle.

Scrum vs Kanban

Scrum et Kanban sont deux méthodes agiles de gestion de projet. Chacune a ses avantages et peut être plus appropriée en fonction du contexte de l’équipe et du projet. Dans le contexte de SAFe, Kanban peut être préféré à Scrum pour plusieurs raisons :

- **Flux de travail continu** : Contrairement à Scrum qui fonctionne par sprints, Kanban est basé sur un flux de travail continu. Cela peut être bénéfique dans un environnement SAFe où plusieurs équipes travaillent simultanément sur différents aspects d’un projet.
- **Limitation du travail en cours** : Kanban limite le nombre de tâches pouvant être placées dans chaque colonne. Cela évite une liste interminable de tâches et favorise leur réalisation.
- **Adaptabilité** : Kanban est moins cadré que Scrum. Il permet une plus grande flexibilité pour s’adapter aux changements, ce qui est crucial dans un environnement agile à grande échelle comme SAFe.
- **Visualisation du travail** : Le tableau Kanban donne une vue d’ensemble du travail en cours et à venir. Cela peut aider les équipes à mieux comprendre le flux de travail et à identifier les goulots d’étranglement.

Présentation de CARE

CARE (Collaborative Agile Reliable Efficacy) est une méthodologie de gestion qui intègre SAFe, Kanban, XP, tous pilotés par Lean. CARE est une approche qui met l’accent sur la collaboration, l’agilité, la fiabilité et l’efficacité, tout en plaçant la qualité au centre de tout.

- **Collaborative** : CARE favorise la collaboration grâce à SAFe. Elle brise les silos organisationnels, favorisant une communication ouverte entre toutes les parties prenantes du projet.
- **Agile** : L’agilité est assurée par Kanban pour visualiser le flux de travail. CARE adopte également DevOps, CI/CD, les tests continus et le shift-left.
- **Reliable** : La fiabilité est maintenue par XP, qui assure la qualité grâce au cœur de CARE où se trouve le développement piloté par la qualité (Quality-Driven Development, QDD).
- **Efficacy** : L’efficacité est obtenue grâce à Lean, qui guide le processus avec une mentalité d’amélioration continue.

Organigramme CARE proposé

Voici un organigramme CARE proposé qui illustre comment chaque rôle contribue à la qualité :

- **VP QA (TQM)** : S’assure que l’organisation dans son ensemble cultive bien la pratique de la

qualité.

- **Directeur QA** : S’assure que le département spécifique cultive bien la pratique de la qualité.
- **Manager QA** : S’assure que l’équipe SAFe cultive bien la pratique de la qualité.
- **Superviseur QA** : S’assure que l’escouade Kanban cultive bien la pratique de la qualité.
- **Expert QA** : S’assure que l’unité XP3A cultive bien la pratique de la qualité.
- **Coach QA** : Est là pour guider et entraîner tous les rôles et contributeurs QA. Il doit assurer la pérennité de la transformation QA. Ce n’est pas un rôle éphémère. Le coach est le joueur indispensable, le joueur clé, pour atteindre la satisfaction.

Conclusion

En somme, que ce soit SAFe, Scrum, Kanban ou CARE, le choix de la méthodologie dépend des besoins spécifiques de l’organisation. Il est important de noter que 98% des équipes Agile ne sont pas suffisamment matures pour faire du SAFe. Par conséquent, une évaluation de la maturité Agile peut être nécessaire avant de faire un choix. Sans CARE, SAFe pourrait ne pas être aussi “safe” qu’on pourrait le penser. C’est pourquoi il est crucial de choisir la bonne approche pour votre organisation avant de plonger les yeux fermés.

(done) Comment bien choisir et implémenter nos outils d’automatisation ?

January 18, 2024 10:27 AM

Introduction

Le choix et l’implémentation des outils d’automatisation sont des étapes cruciales dans tout projet de développement logiciel. Ces décisions dépendent de plusieurs facteurs, tels que la complexité du projet, les compétences de l’équipe et les besoins spécifiques. Il est important de prendre en compte les avantages et les inconvénients de chaque outil et de choisir celui qui convient le mieux à votre projet.

Le secret du choix des outils d’automatisation

Le secret pour bien choisir et implémenter vos outils d’automatisation réside dans une compréhension approfondie de vos besoins et objectifs. Voici quelques points à considérer :

- **Comprendre vos besoins** : Avant de choisir un outil, il est essentiel de comprendre clairement vos besoins. Quels sont les processus que vous souhaitez automatiser ? Quels sont les résultats que vous espérez obtenir de l’automatisation ?
- **Évaluer les compétences de votre équipe** : Les compétences et l’expérience de votre équipe peuvent grandement influencer le choix de l’outil. Il est important de choisir un outil qui correspond aux compétences de votre équipe.
- **Considérer la complexité du projet** : La complexité du projet peut également influencer le choix de l’outil. Un projet plus complexe peut nécessiter un outil plus sophistiqué.
- **Peser les avantages et les inconvénients** : Chaque outil a ses avantages et ses inconvénients. Il est important de les évaluer en fonction de vos besoins spécifiques.

Implémentation des outils d’automatisation

Une fois que vous avez choisi vos outils, l’étape suivante est l’implémentation. Voici quelques conseils pour une implémentation réussie :

- **Planifiez soigneusement** : Une bonne planification est la clé d’une implémentation réussie. Définissez clairement les étapes de l’implémentation et assurez-vous que toute l’équipe comprend le plan.
- **Testez régulièrement** : Il est important de tester régulièrement l’outil pendant l’implémentation pour s’assurer qu’il fonctionne comme prévu.
- **Formez votre équipe** : Assurez-vous que votre équipe est bien formée à l’utilisation de l’outil. Cela peut inclure des formations formelles, des ateliers ou des sessions de questions-réponses.

Conclusion

Le choix et l’implémentation des outils d’automatisation sont des étapes cruciales dans tout projet de développement logiciel. En comprenant clairement vos besoins, en évaluant les compétences de votre équipe, en tenant compte de la complexité du projet et en pesant soigneusement les avantages et les inconvénients de chaque outil, vous pouvez faire un choix éclairé qui contribuera au succès de votre projet.

(done) Quoi?! Tester en production, c’est dans les meilleures pratiques ? (Shift-Right Feedback Analytics)

January 18, 2024 10:27 AM

Introduction

Dans le monde du développement de logiciels, le cycle de DevOps a révolutionné la façon dont nous pensons le développement et le déploiement. Une pratique qui gagne en popularité est le “testing en production”, ou “shift-right testing”.

Qu’est-ce que le Shift-Right Testing ?

Le shift-right testing est une approche qui étend les tests au-delà de l’environnement de pré-production pour inclure l’environnement de production. Cela signifie que les tests sont effectués pendant et après le déploiement en production.

Pourquoi tester en production ?

Il y a plusieurs raisons pour lesquelles le testing en production est considéré comme une bonne pratique :

- **Données réelles** : Les tests en production utilisent des données réelles, ce qui peut aider à identifier des problèmes qui n’auraient pas été détectés dans un environnement de test.
- **Performance** : Les tests en production peuvent aider à identifier les problèmes de performance qui ne se manifestent que dans un environnement de production.
- **Fiabilité** : Les tests en production peuvent augmenter la confiance dans la fiabilité du logiciel.

Le rôle du monitoring et du feedback dans le cycle de DevOps

Le monitoring et le feedback jouent un rôle crucial dans le cycle de DevOps. Ils alimentent la définition des besoins et de la solution, permettant une amélioration continue du produit. Le monitoring en temps réel permet de détecter rapidement les problèmes, tandis que le feedback des utilisateurs aide à comprendre comment le produit est utilisé et comment il peut être amélioré.

Conclusion

Le testing en production, ou shift-right testing, est une pratique de plus en plus courante dans le cycle de DevOps. En combinant cette approche avec un monitoring efficace et un feedback constant, nous pouvons non seulement assurer la qualité de nos produits, mais aussi continuer à les améliorer en fonction des besoins réels des utilisateurs.

(done) Quoi?! Tous les environnements de développement sont mock-és !

January 18, 2024 10:27 AM

Introduction

Dans le monde du développement de logiciels, il est courant de penser que les environnements de développement sont des répliques exactes des environnements de production. Cependant, en réalité, tous les environnements de développement sont en quelque sorte “mock-és” ou simulés.

Qu’est-ce qu’un environnement mocké ?

Un environnement mocké est une simulation d’un environnement réel qui imite son comportement. Dans le contexte du développement de logiciels, cela signifie que l’environnement de développement est configuré pour simuler l’environnement de production.

Les bases de données dans les environnements de développement

Un aspect souvent négligé lors de la discussion sur les environnements de développement mock-és est la base de données. Même si l’environnement de développement semble complet, il est important de noter que la base de données utilisée n’est pas la même que celle de la production.

En effet, pour des raisons de sécurité et de confidentialité, les données réelles ne sont généralement pas utilisées dans les environnements de développement. Ainsi, même si le reste de l’environnement est configuré pour imiter la production aussi fidèlement que possible, la base de données est généralement une version mockée. Cela signifie que toutes les interactions avec la base de données dans un environnement de développement sont, dans une certaine mesure, des simulations.

Pourquoi les environnements de développement sont-ils mock-és ?

Il y a plusieurs raisons pour lesquelles les environnements de développement sont mock-és :

- **Sécurité** : Les environnements de développement sont souvent isolés des environnements de production pour des raisons de sécurité. Cela permet de protéger les données de production contre les erreurs ou les problèmes qui pourraient survenir lors du développement.
- **Contrôle** : Les environnements de développement permettent un contrôle total sur l’environnement, ce qui peut aider à produire des résultats de test plus cohérents.
- **Efficacité** : Les environnements de développement peuvent souvent être configurés plus rapidement et plus facilement que les environnements de production, ce qui peut accélérer le processus de développement.

Conclusion

Bien que les environnements de développement soient souvent considérés comme des répliques des environnements de production, ils sont en réalité des environnements mock-és conçus pour simuler le comportement de l’environnement de production tout en offrant un niveau de sécurité et de contrôle supplémentaire. Cette pratique est essentielle pour la sécurité, le contrôle et l’efficacité du processus de développement.

Add label

(done) Quoi?! Tester avec un jeu de données anonymisées est l'une des pires mauvaises pratiques !

January 18, 2024 10:28 AM

Introduction

Dans le monde du développement de logiciels, l'anonymisation des données est une pratique courante pour protéger la vie privée des utilisateurs. Cependant, lorsqu'il s'agit de tests, l'utilisation de données anonymisées peut souvent être contre-productive et même nuisible.

Pourquoi tester avec des données anonymisées est une mauvaise pratique

Tester avec des données anonymisées peut sembler une bonne idée pour protéger la vie privée des utilisateurs, mais cela peut en réalité altérer les résultats des tests. Voici pourquoi :

1. **Données non représentatives** : Les données anonymisées peuvent ne pas refléter fidèlement les données réelles que l'application devra traiter. Cela peut conduire à des tests qui ne reflètent pas précisément le comportement réel de l'application.
2. **Risques de sécurité** : Même si les données sont anonymisées, il peut rester des informations sensibles qui peuvent être exploitées. De plus, si l'anonymisation n'est pas réalisée correctement, elle peut être inversée, exposant ainsi les données sensibles.
3. **Complexité supplémentaire** : L'anonymisation des données ajoute une couche de complexité au processus de test, ce qui peut entraîner des erreurs et des inefficacités.

Ne jamais utiliser les données de production pour les tests

Une règle d'or dans le développement de logiciels est de **ne jamais utiliser les données de production pour les tests**. Il y a plusieurs raisons à cela :

4. **Sécurité des données** : Les données de production peuvent contenir des informations sensibles. Utiliser ces données pour les tests peut exposer ces informations à des risques inutiles.
5. **Protection de la vie privée** : Les données de production peuvent contenir des informations personnelles. Leur utilisation dans les tests peut enfreindre les lois sur la protection de la vie privée et les réglementations sur les données.
6. **Fiabilité des tests** : Les données de production peuvent changer constamment, ce qui peut rendre les tests inconsistants et peu fiables.

Au lieu d'utiliser les données de production, il est préférable de générer ses propres données de test ou d'utiliser des données anonymisées ou fictives. Cela permet de s'assurer que les tests sont à la fois sûrs et fiables.

Une meilleure approche : Générer ses propres données

Plutôt que de tester avec des données anonymisées, il est souvent préférable de générer ses propres données de test. Cela permet de contrôler exactement quelles données sont utilisées pour les tests, d'éliminer les risques de sécurité associés à l'utilisation de données réelles et de s'assurer que les données de test sont représentatives des données que l'application devra traiter.

Conclusion

Bien que l'anonymisation des données soit une pratique importante pour la protection de la vie privée, elle n'est pas toujours la meilleure approche pour les tests de logiciels. En générant nos propres données de test, nous pouvons réaliser des tests plus précis et plus sûrs, tout en évitant les pièges associés à l'utilisation de données anonymisées.

(done) Quoi?! La validation des logs/events est la moins pratiquée mais la plus payante !

January 18, 2024 10:28 AM

Introduction

Dans le monde du développement de logiciels, les tests sont une étape cruciale pour assurer la qualité et la fiabilité d'une application. Cependant, un aspect souvent négligé des tests est la validation des logs/events. Bien que moins pratiquée, cette forme de validation peut s'avérer extrêmement bénéfique.

Pourquoi la validation des logs/events est-elle négligée ?

La validation des logs/events est souvent négligée car elle peut sembler moins directement liée à la fonctionnalité de l'application. De plus, la validation des logs peut être techniquement complexe, nécessitant une compréhension approfondie des systèmes de logging et des événements spécifiques à surveiller.

Pourquoi la validation des logs/events est-elle importante ?

La validation des logs/events est importante pour plusieurs raisons :

1. **Détection précoce des problèmes** : Les logs peuvent souvent indiquer des problèmes avant qu'ils ne deviennent apparents dans l'application elle-même.
2. **Compréhension du comportement de l'application** : Les logs peuvent aider à comprendre comment l'application se comporte dans différents scénarios, ce qui peut être utile pour l'optimisation et le débogage.
3. **Sécurité** : La validation des logs peut aider à détecter les activités suspectes ou malveillantes, contribuant ainsi à la sécurité de l'application.

Conclusion

Bien que la validation des logs/events soit souvent négligée dans les tests, elle peut offrir des avantages significatifs. En intégrant la validation des logs dans votre processus de test, vous pouvez améliorer la détection des problèmes, la compréhension du comportement de l'application et la sécurité.

(done) Quoi?! Nous n’avons pas besoin de tester l’intégrité du data par l’interface utilisateur ?

January 18, 2024 10:28 AM

Introduction

Dans le monde du développement de logiciels, il est courant de penser que tester l’intégrité des données à travers l’interface utilisateur est une étape nécessaire. Cependant, cette approche peut souvent conduire à des tests inefficaces et à une mauvaise utilisation des ressources.

Pourquoi tester l’intégrité des données à travers l’interface utilisateur est une erreur courante

Lorsque nous commençons à tester, il est tentant de vouloir vérifier l’intégrité des données à travers l’interface utilisateur. Après tout, c’est l’interface que les utilisateurs vont interagir avec, n’est-ce pas ? Cependant, cette approche présente plusieurs problèmes :

1. **C’est inefficace** : Tester l’intégrité des données à travers l’interface utilisateur peut être un processus lent et fastidieux. Il est souvent beaucoup plus efficace de tester l’intégrité des données à un niveau plus bas, comme la base de données ou l’API.
2. **Cela peut conduire à des tests redondants** : Si vous testez l’intégrité des données à travers l’interface utilisateur, vous risquez de répéter les mêmes tests que ceux qui sont déjà effectués à un niveau plus bas.
3. **Cela peut masquer les vrais problèmes** : Si vous vous concentrez sur le test de l’intégrité des données à travers l’interface utilisateur, vous pouvez manquer d’autres problèmes importants, comme les problèmes d’interface utilisateur ou de performance.

Une meilleure approche

Au lieu de tester l’intégrité des données à travers l’interface utilisateur, il est souvent préférable de tester l’intégrité des données à un niveau plus bas, comme la base de données ou l’API. Cela permet de s’assurer que les données sont correctes, indépendamment de la façon dont elles sont présentées à l’utilisateur.

De plus, il est important de se rappeler que le but des tests n’est pas seulement de vérifier l’intégrité des données, mais aussi de s’assurer que l’application fonctionne correctement et offre une bonne expérience utilisateur. Cela signifie qu’il est tout aussi important de tester d’autres aspects de l’application, comme l’interface utilisateur, la performance, et la sécurité.

Conclusion

Tester l’intégrité des données à travers l’interface utilisateur peut sembler une bonne idée au début, mais c’est souvent une erreur. En adoptant une approche plus efficace et en se concentrant sur les bons aspects des tests, nous pouvons créer des applications de meilleure qualité et offrir une meilleure expérience à nos utilisateurs.

(done) Quoi?! On peut faire du UAT avant et pendant le développement !

January 18, 2024 10:28 AM

Introduction

Dans le monde du développement de logiciels, les tests d'acceptation utilisateur (UAT) sont souvent considérés comme la dernière étape avant le déploiement d'une application. Cependant, il est tout à fait possible - et souvent bénéfique - de réaliser des UAT avant et pendant le développement.

UAT avant le développement (shift-left testing)

Réaliser des UAT avant le développement peut sembler contre-intuitif, mais c'est en fait une excellente façon de comprendre les besoins et les attentes des utilisateurs avant de commencer à coder. Cela peut impliquer des entretiens avec les utilisateurs, des ateliers de conception participative, ou même des prototypes interactifs pour recueillir des commentaires.

UAT pendant le développement

Réaliser des UAT pendant le développement permet de recueillir des commentaires en temps réel et d'ajuster le produit en conséquence. Cela peut aider à identifier les problèmes plus tôt dans le processus de développement, ce qui peut économiser du temps et des ressources.

Avantages des UAT avant et pendant le développement

Il y a plusieurs avantages à réaliser des UAT avant et pendant le développement :

1. **Comprendre les besoins des utilisateurs** : Les UAT peuvent aider à comprendre les besoins et les attentes des utilisateurs, ce qui peut guider le développement du produit.
2. **Identifier les problèmes plus tôt** : Les UAT peuvent aider à identifier les problèmes plus tôt dans le processus de développement, ce qui peut économiser du temps et des ressources.
3. **Améliorer la qualité du produit** : Les UAT peuvent aider à améliorer la qualité du produit en permettant aux développeurs de recueillir des commentaires en temps réel et d'ajuster le produit en conséquence.

Conclusion

Les UAT ne sont pas seulement pour la fin du processus de développement. En réalisant des UAT avant et pendant le développement, vous pouvez améliorer la qualité de votre produit et mieux répondre aux besoins de vos utilisateurs.

(done) Quoi?! E2E, End to End ou bout en bout c’est jamais dans l’environnement complet !

January 18, 2024 10:28 AM

Introduction

Dans le monde du développement de logiciels, les tests de bout en bout (E2E) sont souvent associés à des tests effectués dans un environnement complet. Cependant, ce n’est pas toujours le cas. En réalité, les tests E2E peuvent être effectués à différents niveaux, même dans les tests unitaires de code, et même dans des environnements mockés.

Qu’est-ce qu’un test E2E ?

Un test E2E est un type de test qui vérifie qu’un flux de travail dans une application fonctionne de bout en bout - du point de départ (point A) au point d’arrivée (point B). Il s’agit de s’assurer que l’ensemble du système, y compris toutes ses composantes interdépendantes, fonctionne comme prévu.

Les tests E2E dans les tests unitaires de code

Les tests E2E peuvent même être intégrés dans les tests unitaires de code. Cela signifie que vous pouvez vérifier le flux de travail complet d’une fonctionnalité au niveau du code, sans avoir besoin d’un environnement complet. Cela peut rendre les tests plus rapides, plus fiables et plus faciles à maintenir.

Les tests E2E dans des environnements mockés

Un environnement mocké est une simulation d’un environnement réel qui imite son comportement. Les tests E2E peuvent être effectués dans des environnements mockés pour plusieurs raisons :

1. **Contrôle** : Les environnements mockés permettent un contrôle total sur l’environnement de test, ce qui peut aider à produire des résultats de test plus cohérents.
2. **Rapidité** : Les tests dans des environnements mockés peuvent souvent être exécutés plus rapidement que dans des environnements complets.
3. **Sécurité** : Les tests dans des environnements mockés peuvent réduire les risques associés à l’exécution de tests dans des environnements de production.

Conclusion

Les tests E2E sont un outil précieux pour garantir que les applications fonctionnent comme prévu. Cependant, il est important de comprendre qu’ils ne sont pas toujours effectués dans un environnement complet. En fait, les tests E2E peuvent être effectués à différents niveaux, même dans les tests unitaires de code, et même dans des environnements mockés, offrant ainsi une plus grande flexibilité et efficacité.

(done) Quoi?! Nous développons du code fonctionnel ?

January 18, 2024 10:29 AM

Introduction

Dans le monde du développement de logiciels, il est courant de penser au code en termes de syntaxe, de structures de données et d'algorithmes. Cependant, une perspective souvent négligée est que le code que nous écrivons est en réalité une collection de fonctionnalités destinées à résoudre des problèmes spécifiques pour les utilisateurs.

Le code comme fonctionnalité

Chaque ligne de code que nous écrivons a un but : elle contribue à une fonctionnalité qui apporte de la valeur à l'utilisateur. Qu'il s'agisse d'une fonction qui calcule une somme, d'une routine qui traite des données ou d'une interface utilisateur qui permet aux utilisateurs d'interagir avec notre application, chaque partie du code fait partie d'une fonctionnalité plus grande.

Pourquoi cette perspective est importante

Cette perspective est importante car elle nous aide à nous concentrer sur ce qui est vraiment important : la valeur que nous apportons à nos utilisateurs. Au lieu de nous concentrer uniquement sur l'écriture de code "parfait", nous devrions nous concentrer sur la création de fonctionnalités utiles et significatives pour nos utilisateurs.

Conclusion

En fin de compte, le développement de logiciels ne consiste pas seulement à écrire du code, mais à créer des fonctionnalités qui résolvent des problèmes réels pour les utilisateurs. En adoptant cette perspective, nous pouvons devenir de meilleurs développeurs et créer des produits plus utiles et plus significatifs.

(done) Pourquoi le QA finit-il souvent par tout faire ?

January 18, 2024 10:29 AM

Introduction

Dans le monde du développement de logiciels, l'assurance qualité (QA) joue un rôle crucial pour garantir la qualité des produits ou services. Cependant, il est courant que les professionnels du QA finissent par effectuer des tâches qui ne font pas partie de leur mandat initial.

Le rôle du QA

L'analyste en assurance qualité est responsable de la planification et de l'exécution des tests pour s'assurer que les applications sont exemptes d'erreurs et fonctionnent correctement. Il travaille en étroite collaboration avec les développeurs pour s'assurer que les normes de qualité sont respectées et que les applications sont livrées dans les délais impartis.

Pourquoi le QA finit-il par tout faire ?

Il existe plusieurs raisons pour lesquelles un analyste QA peut finir par effectuer des tâches qui ne font pas partie de son mandat initial :

1. **Charge de travail accrue** : Avec l'augmentation de la complexité des projets et des applications, la charge de travail des analystes QA a également augmenté. Ils peuvent se retrouver à gérer des tâches supplémentaires qui ne font pas partie de leur rôle initial.
2. **Réduction du personnel dans d'autres rôles** : Dans certaines situations, en raison de la réduction du personnel dans d'autres rôles, les analystes QA peuvent se retrouver à assumer des responsabilités supplémentaires.
3. **Évolution des exigences du projet** : Les exigences du projet peuvent changer au fil du temps, ce qui peut amener les analystes QA à assumer de nouvelles responsabilités.

Comment gérer cette situation ?

Dans tous les cas, il est important que l'analyste en assurance qualité communique avec son superviseur pour clarifier son mandat et s'assurer qu'il ne dépasse pas ses limites. Il est également crucial de mettre en place des processus efficaces pour gérer la charge de travail et les responsabilités de chaque membre de l'équipe.

Conclusion

Bien que les professionnels du QA puissent souvent se retrouver à faire plus que ce qui est prévu, il est important de reconnaître l'importance de leur rôle et de s'assurer qu'ils ont le soutien et les ressources nécessaires pour effectuer leur travail efficacement.

(done) Fonctionnel VS non-Fonctionnel : Pourquoi le codage et les tests unitaires sont-ils fonctionnels ?

January 18, 2024 10:29 AM

Introduction

Dans le monde du développement logiciel, il existe une distinction entre les exigences fonctionnelles et non fonctionnelles. Cependant, il y a souvent une confusion sur la place du codage et des tests unitaires dans cette distinction.

Qu'est-ce que les exigences fonctionnelles ?

Les exigences fonctionnelles décrivent ce qu'un système doit faire. Elles se concentrent sur les fonctionnalités du système, comme la capacité d'un logiciel à effectuer une tâche spécifique.

Qu'est-ce que les exigences non fonctionnelles ?

Les exigences non fonctionnelles, en revanche, décrivent comment un système doit fonctionner. Elles se concentrent sur des aspects tels que la performance, la sécurité, l'accessibilité, etc.

Le codage et les tests unitaires sont-ils fonctionnels ?

Il y a une idée fausse commune que le codage et les tests unitaires ne sont pas fonctionnels parce qu'ils ne sont pas directement liés à une fonctionnalité spécifique du système. Cependant, c'est une idée fausse.

Le codage est l'acte de traduire les exigences fonctionnelles en instructions que l'ordinateur peut comprendre. Sans codage, les exigences fonctionnelles ne peuvent pas être mises en œuvre. Par conséquent, le codage est une partie essentielle des exigences fonctionnelles.

De même, les tests unitaires sont utilisés pour vérifier que chaque unité de code fonctionne comme prévu. Ils aident à garantir que les exigences fonctionnelles sont correctement mises en œuvre. Par conséquent, les tests unitaires sont également une partie essentielle des exigences fonctionnelles.

Conclusion

En somme, le codage et les tests unitaires sont des aspects fonctionnels du développement logiciel. Ils sont essentiels pour la mise en œuvre et la vérification des exigences fonctionnelles. Il est important de comprendre cette distinction pour éviter toute confusion ou malentendu.

(done) Hyperautomation ?!

January 18, 2024 10:29 AM

Le principe d'Hyperautomation : Pourquoi automatiser l'ensemble nous rend plus humain

Introduction

L'Hyperautomation est un concept qui vise à automatiser le plus possible de manière intelligente tout en respectant les bonnes pratiques. C'est un principe fondamental de la pratique de l'assurance qualité qui a des vertus importantes.

Qu'est-ce que l'Hyperautomation ?

L'Hyperautomation va au-delà de l'automatisation traditionnelle pour inclure l'automatisation intelligente. Cela signifie que non seulement les tâches sont automatisées, mais elles sont également optimisées grâce à l'apprentissage machine et à l'intelligence artificielle. L'Hyperautomation cherche à automatiser non seulement les tests ou le déploiement, mais l'ensemble du processus.

Les vertus de l'Hyperautomation

L'Hyperautomation a plusieurs vertus. Premièrement, elle permet d'augmenter l'efficacité et la productivité en réduisant le temps et les efforts nécessaires pour accomplir les tâches. Deuxièmement, elle permet d'améliorer la qualité en réduisant les erreurs humaines. Troisièmement, elle permet d'améliorer la satisfaction des clients en fournissant des services plus rapides et plus précis.

Pourquoi l'Hyperautomation nous rend plus humain ?

L'Hyperautomation, en automatisant les tâches répétitives et monotones, libère du temps pour les humains pour se concentrer sur des tâches plus complexes et créatives. Cela permet aux humains de faire ce qu'ils font de mieux : penser de manière créative, résoudre des problèmes complexes et établir des relations humaines. En d'autres termes, l'Hyperautomation nous permet de passer moins de temps sur les tâches routinières et plus de temps sur ce qui nous rend vraiment humains.

Conclusion

En somme, l'Hyperautomation est un principe puissant qui peut transformer la façon dont nous travaillons. En automatisant intelligemment l'ensemble du processus, non seulement nous augmentons l'efficacité et la qualité, mais nous nous permettons également de nous concentrer sur ce qui nous rend vraiment humains.

(done) On s'en moque du "MOCK" !

January 18, 2024 10:30 AM

On s’en moque du “MOCK” !

Introduction

Dans le monde du développement de logiciels, les “mocks” sont souvent mal compris ou mal utilisés. Cependant, ils sont un outil essentiel pour garantir la qualité du code et faciliter les tests. Dans ce blog, nous allons démystifier les mocks et comprendre leur importance.

Qu’est-ce qu’un mock ?

Un mock est une simulation d’un objet réel qui imite le comportement de l’objet réel. Dans le contexte des tests de logiciels, les mocks sont utilisés pour simuler le comportement d’objets réels afin de tester le comportement d’autres objets.

Pourquoi utiliser des mocks ?

Les mocks sont particulièrement utiles pour les tests unitaires, où l’objectif est de tester une partie spécifique du code en isolation. Ils permettent de contrôler l’environnement de test et de s’assurer que le code est testé de manière cohérente et fiable.

Voici quelques raisons d’utiliser des mocks :

1. **Tester par niveaux de test** : Les mocks permettent de tester chaque partie du code en isolation, ce qui facilite l’identification des erreurs.
2. **Contrôler les jeux de données de tests** : Avec les mocks, vous pouvez contrôler précisément les données utilisées pour les tests, ce qui permet d’obtenir des résultats de tests plus cohérents.
3. **Tester dans des environnements ultra stables et contrôlés** : Les mocks permettent de créer un environnement de test stable et contrôlé, ce qui facilite la reproduction des tests et l’identification des erreurs.

Pourquoi les mocks sont plus fiables que les tests dans un environnement complet

Retester les mêmes tests : Un problème courant

Dans de nombreux projets de développement de logiciels, il est courant de retester les mêmes tests dans différents environnements, tels que les environnements d’intégration ou complets. Cependant, cela peut souvent conduire à des efforts redondants et à une utilisation inefficace des ressources.

L’importance des mocks

C’est là qu’interviennent les mocks. L’un des principaux objectifs des mocks est précisément d’éviter de retester les mêmes tests. En utilisant des mocks, vous pouvez simuler le comportement de certaines parties de votre système et vous concentrer sur les tests de la partie du système qui est actuellement en cours de développement ou de modification.

Pourquoi les mocks sont plus fiables

Il est important d’avoir confiance dans les mocks. En fait, il peut être plus fiable de tester avec des mocks bien faits que de tester dans un environnement complet pour les raisons suivantes :

4. **Contrôle total sur les données de test** : Avec les mocks, vous avez un contrôle total sur les données de test, ce qui signifie que vous pouvez tester exactement ce que vous voulez

tester sans vous soucier des données indésirables ou inattendues.

5. **Tests plus rapides** : Les tests avec des mocks sont généralement beaucoup plus rapides que les tests dans un environnement complet, car vous n'avez pas besoin d'initialiser tout l'environnement de test.
6. **Isolation** : Les mocks vous permettent de tester des parties spécifiques de votre système en isolation, ce qui signifie que vous pouvez vous concentrer sur une seule chose à la fois et être sûr que les erreurs ne sont pas causées par d'autres parties du système.
7. **Moins de dépendances** : Avec les mocks, vos tests ont moins de dépendances, ce qui signifie qu'ils sont moins susceptibles d'échouer en raison de problèmes avec les dépendances.

Conclusion

En somme, les mocks sont un outil précieux pour tout développeur ou testeur. Ils permettent d'éviter de retester les mêmes tests, de gagner du temps et d'obtenir des résultats de tests plus fiables. Il est donc essentiel de comprendre et d'apprécier l'importance des mocks dans le processus de test.

Il ne faut pas avoir peur des mocks. Au contraire, ils sont un outil précieux pour tout développeur ou testeur. En permettant de tester le code de manière cohérente et fiable, les mocks contribuent à garantir la qualité du code et à faciliter le processus de test.

(done) Blog | Pourquoi, il n’y a jamais de temps pour les tests !?

January 18, 2024 10:30 AM

Introduction

Dans le monde du développement de logiciels, il est courant d’entendre dire qu’il n’y a jamais assez de temps pour les tests. Cependant, les tests sont une partie essentielle du processus de développement de logiciels. Alors, pourquoi manque-t-on souvent de temps pour les tests et quelles solutions pouvons-nous adopter pour surmonter ce défi ?

Pourquoi manque-t-on de temps pour les tests ?

Il peut y avoir plusieurs raisons pour lesquelles il n’y a jamais assez de temps pour les tests. Voici quelques raisons courantes :

1. **Les délais de livraison sont trop serrés** : Souvent, les délais de livraison sont si serrés qu’il n’y a pas suffisamment de temps pour effectuer des tests approfondis.
2. **Les tests sont considérés comme une tâche secondaire** : Dans certains cas, les tests sont considérés comme une tâche secondaire et ne sont donc pas prioritaires.
3. **Les testeurs ne sont pas impliqués dès le début** : Si les testeurs ne sont pas impliqués dans le processus de développement dès le début, cela peut entraîner des retards dans les tests.
4. **Les tests manuels peuvent être longs et fastidieux** : Les tests manuels peuvent prendre beaucoup de temps et être fastidieux, ce qui peut décourager les testeurs.
5. **La sous-estimation des tests** : La plus fréquente raison est que l’on sous estime toujours le temps pour tester.

Solutions pour surmonter le manque de temps pour les tests

Malgré ces défis, il existe des solutions pour surmonter le manque de temps pour les tests :

6. **Planification des tests dès le début** : Impliquer les testeurs dès le début du processus de développement peut aider à garantir que suffisamment de temps est alloué aux tests. Définir les plans de tests riche et complet avant de commencer à développer.
7. **Automatisation des tests** : L’automatisation des tests peut aider à accélérer le processus de test et à libérer du temps pour des tests plus approfondis. (attention si cela n’est pas bien planifié ou pressé cela va se transformer en cauchemar)
8. **Priorisation des tests** : En priorisant les tests en fonction de l’importance des fonctionnalités ou des composants à tester, on peut s’assurer que les éléments les plus importants sont testés en premier.
9. **Formation continue des testeurs** : En formant continuellement les testeurs sur les dernières méthodologies et outils de test, on peut augmenter leur efficacité et réduire le temps nécessaire pour les tests.
10. **Augmenter l’estimation du temps des tests** : N’ayez pas peur de sur estimer le temps de tester, même le double du temps de l’implémentation et du développement incluant le raffinement le découpage et le planning. Par la suite voir à réajuster à la baisse, mais vous remarquerez qu’il n’y aura pas souvent de réajustement
11. ✕
12. **Les testeurs ne sont pas les QA** : Les testeurs sont les experts dans le domaine, il faut arrêter dès maintenant de faire définir et valider par les QA. Les QA sont là pour facilité l'équipe à l’analyse, aide à la planification, suivi de la couverture des releases… etc. Et bien sur promouvoir et encourager la culture de la pratique assurance qualité dans l’organisation.

Conclusion

Bien qu’il puisse souvent sembler qu’il n’y a jamais assez de temps pour les tests, en adoptant les bonnes stratégies et en utilisant les bons outils, il est possible de surmonter ce défi. Après tout, les tests sont une partie essentielle du processus de développement de logiciels et ne doivent pas être négligés.

(done) Le magicien d'AQ

January 18, 2024 10:30 AM

Introduction

Dans le monde du développement de logiciels, l'assurance qualité (QA) joue un rôle crucial pour garantir la qualité des produits ou services d'une entreprise. Les professionnels du QA, souvent appelés les "magiciens d'AQ", travaillent à différents niveaux pour s'assurer que les produits ou services répondent aux normes de qualité établies.

Le travail des professionnels du QA : Une magie en soi

Le travail des professionnels du QA peut sembler magique car ils sont souvent en mesure de trouver des erreurs ou des problèmes qui ont été manqués par d'autres membres de l'équipe. Cela peut être dû à leur expérience, à leur connaissance des normes de qualité et à leur capacité à penser de manière critique. Cependant, il est important de noter que le travail des professionnels du QA n'est pas magique. Il repose sur une solide connaissance des normes de qualité et sur une méthodologie rigoureuse pour identifier les erreurs et les problèmes.

L'automagicien : Un nouveau rôle dans le monde de l'AQ

L'automagicien est un nouveau rôle dans le monde de l'AQ. Il s'agit d'un professionnel du QA qui utilise l'automatisation pour améliorer l'efficacité et l'efficience des tests. L'automagicien utilise des outils et des technologies pour automatiser les tâches répétitives, ce qui lui permet de se concentrer sur des tâches plus complexes et de résoudre des problèmes plus difficiles.

Le lien avec le Magicien d'Oz

Tout comme le Magicien d'Oz dans le célèbre roman de L. Frank Baum, les professionnels du QA sont souvent perçus comme des figures mystérieuses qui travaillent en coulisses pour assurer la qualité des produits ou services. Cependant, tout comme le Magicien d'Oz, leur "magie" repose sur une solide connaissance et une méthodologie rigoureuse.

Conclusion

Dans le monde du développement de logiciels, l'assurance qualité est essentielle pour garantir la qualité des produits ou services. Que ce soit à travers le travail méticuleux des professionnels du QA ou l'automatisation innovante des automagiciens, l'AQ continue d'être un domaine crucial dans le développement de logiciels de haute qualité.


Le MAGICIEN d'OZ

(done) Quoi?! AUTOMATISER sans CODE c’est une meilleure pratique !

January 18, 2024 10:30 AM

Introduction

L’automatisation des tests est une pratique courante dans le développement de logiciels pour assurer la qualité et la fiabilité du produit. Cependant, l’automatisation des tests sans coder est maintenant considérée comme une meilleure pratique. Cette approche permet aux testeurs de se concentrer sur la création de cas de test et sur l’exécution des tests plutôt que sur la création de scripts de test.

Automatisation des tests sans coder

Les outils d’automatisation des tests sans coder sont conçus pour être utilisés par des personnes qui n’ont pas nécessairement une expérience en programmation. Ils permettent aux testeurs de créer des cas de test en utilisant une interface graphique conviviale plutôt qu’en écrivant du code. Cela permet aux testeurs de créer des cas de test plus rapidement et plus facilement.

Avantages de l’automatisation des tests sans coder

L’automatisation des tests sans coder présente plusieurs avantages :

1. **Gain de temps** : Les testeurs peuvent créer des cas de test plus rapidement car ils n’ont pas à écrire de code.
2. **Accessibilité** : Les testeurs n’ont pas besoin d’avoir des compétences en programmation pour utiliser ces outils.
3. **Réduction des coûts** : Les outils d’automatisation des tests sans coder sont souvent moins chers que les outils d’automatisation des tests qui nécessitent une programmation.
4. **Flexibilité** : Ces outils permettent aux testeurs de tester une variété de fonctionnalités sans avoir à écrire de code spécifique pour chaque fonctionnalité.
5. **Maintenance réduite** : Comme les tests sont créés sans code, ils sont moins susceptibles d’être affectés par les changements dans le code de l’application.

Conclusion

L’automatisation des tests sans coder est une pratique qui gagne en popularité en raison de ses nombreux avantages. Elle permet aux testeurs de se concentrer sur ce qu’ils font de mieux : tester la qualité et la fiabilité du logiciel, sans avoir à se soucier de la programmation.

(done) Agnostique avec la reconnaissance d'image VS dépendance du code

January 18, 2024 10:31 AM

Introduction

Dans le monde de l'automatisation des tests, deux approches se distinguent : l'approche basée sur le code, qui nécessite une connaissance approfondie du code de l'application et de ses dépendances, et l'approche agnostique, qui utilise la reconnaissance d'image pour interagir avec l'application de manière plus générale.

L'approche basée sur le code

L'approche basée sur le code nécessite une connaissance approfondie du code de l'application et de ses dépendances. Cela signifie que les tests doivent être constamment mis à jour pour suivre les changements dans le code de l'application. De plus, cette approche peut nécessiter une configuration et une installation complexes.

L'approche agnostique avec la reconnaissance d'image

L'approche agnostique avec la reconnaissance d'image, en revanche, permet d'automatiser les tests sans avoir besoin de comprendre le code de l'application. Cette approche utilise la reconnaissance d'image pour interagir avec l'application, ce qui permet de tester plusieurs appareils, systèmes d'exploitation et navigateurs sans changer le test. De plus, cette approche peut réduire considérablement le temps nécessaire à l'exécution des tests.

Les outils d'automatisation avec reconnaissance d'image

Il existe plusieurs outils d'automatisation qui utilisent la reconnaissance d'image, parmi lesquels :

1. **Eggplant** : Eggplant est un outil d'automatisation des tests qui utilise l'intelligence artificielle pour améliorer l'efficacité et la précision des tests. Il permet de tester n'importe quelle application ou interface utilisateur sans injecter de code.
2. **Zaptest** : Zaptest est un outil d'automatisation des tests qui permet d'augmenter la productivité de l'entreprise, de réduire les coûts et d'obtenir un retour sur investissement jusqu'à 10 fois supérieur. Il utilise l'intelligence artificielle et la vision par ordinateur pour améliorer l'efficacité des tests.
3. **Airtest Project** : Airtest Project est un cadre d'automatisation des tests développé et open-sourcé par NetEase. Il permet d'interpréter et d'interagir avec l'application comme un véritable utilisateur sans avoir besoin d'accéder au code source.
4. **Beastest** : Beastest est un outil d'automatisation des tests qui permet de tester tout et de façon ultra rapide et modulaire, c'est le best.

L'efficacité de la reconnaissance d'image par OCR

La reconnaissance optique de caractères (OCR) est une technologie qui aide les utilisateurs à extraire du texte d'images ou de documents numérisés et à transformer ce texte dans un format lisible par l'ordinateur. C'est pratique lorsque des données sont nécessaires pour un traitement ultérieur, par exemple pour la comptabilité, la gestion des dépenses, les campagnes de marketing de fidélisation ou la vérification d'identité. En pratique, cela signifie que si vous avez besoin de convertir un document, quel que soit son format (il peut s'agir d'une image contenant du texte, d'un document texte en lui-même, ou d'un PDF par exemple), l'OCR permet cette conversion en reconnaissant les lettres au sein des images afin de reconstituer les phrases ou le contenu textuel d'un fichier donné.

Être agnostique du code

Être agnostique du code signifie que les tests automatisés ne dépendent pas du code sous-jacent de l'application. Cela signifie que les tests continueront à fonctionner même si le code de l'application change ou si la technologie sous-jacente évolue. Cela réduit considérablement la maintenance nécessaire pour les tests automatisés, car ils n'ont pas besoin d'être mis à jour chaque fois que le code change.

Avantages d'utiliser des outils d'automatisation des tests par reconnaissance d'image

Voici une liste de certains des avantages d'utiliser des outils d'automatisation des tests par reconnaissance d'image :

5. **Réduction du temps nécessaire à l'exécution des tests** : Les tests automatisés peuvent être exécutés rapidement, ce qui permet de gagner du temps et d'augmenter l'efficacité.
6. **Réduction des erreurs humaines** : Les tests automatisés éliminent le risque d'erreurs humaines qui peuvent survenir lors de l'exécution manuelle des tests.
7. **Amélioration de la qualité globale du produit** : Les tests automatisés permettent de détecter et de corriger les défauts plus tôt dans le cycle de développement, ce qui améliore la qualité globale du produit.
8. **Réduction des coûts** : Les tests automatisés peuvent réduire les coûts en minimisant le temps nécessaire pour tester les logiciels ou les applications et en minimisant les erreurs qui pourraient être coûteuses à corriger une fois qu'un produit est mis en production.
9. **Accessibilité accrue** : Les tests automatisés basés sur la reconnaissance d'image sont accessibles à un public beaucoup plus large, car ils ne nécessitent pas de compétences en programmation.
10. **Rapidité de développement** : Grâce à des interfaces visuelles conviviales, les testeurs peuvent créer des cas de test automatisés en un temps record.
11. **Moins de dépendance technique** : Traditionnellement, la création de tests automatisés était principalement entre les mains des développeurs et des experts en automatisation. Avec la reconnaissance d'image, cela change.
12. **Amélioration de la confiance** : Les tests automatisés peuvent améliorer la confiance des parties prenantes dans le système.
13. **Flexibilité** : Les tests automatisés basés sur la reconnaissance d'image peuvent être utilisés sur une variété de plateformes et de dispositifs.
14. **Précision** : Les tests automatisés basés sur la reconnaissance d'image peuvent être très précis, ce qui permet de détecter des défauts qui pourraient être manqués par des tests manuels.

Avantages concrets de l'utilisation de technologies basées sur la reconnaissance d'image

Automatisation avant le développement

L'utilisation de technologies basées sur la reconnaissance d'image permet une automatisation avant même le début du développement. Cela signifie que vous pouvez commencer à tester et à valider vos idées dès les premières étapes du cycle de vie du logiciel.

15. **Définition des tests automatisés** : Avec la reconnaissance d'image, vous pouvez définir des tests basés sur des maquettes ou des prototypes visuels, sans avoir besoin d'accéder au code.
16. **Tests d'architecture automatisés** : La reconnaissance d'image peut être utilisée pour tester l'architecture de votre application en comparant les maquettes visuelles avec l'application réelle.
17. **Développement piloté par les tests (TDD) automatisé** : Avec la reconnaissance d'image, vous pouvez écrire des tests qui vérifient l'apparence et le comportement de votre application, plutôt que son code sous-jacent.

Les tests échouent d’abord

Dans une approche TDD, les tests échouent d’abord. C’est-à-dire que les tests sont écrits avant le code, de sorte qu’ils échouent initialement. Ensuite, le code est écrit pour passer ces tests. Cela garantit que le code répond aux exigences définies par les tests.

Moins de maintenance

L’un des plus grands avantages de l’utilisation de technologies basées sur la reconnaissance d’image est qu’elles nécessitent moins de maintenance que les tests basés sur le code. Comme les tests sont basés sur ce que voit l’utilisateur, plutôt que sur le code sous-jacent, ils sont moins susceptibles d’être affectés par les changements dans le code.

Conclusion

L’utilisation de technologies basées sur la reconnaissance d’image pour l’automatisation des tests offre de nombreux avantages. Non seulement elles permettent une automatisation plus précoce dans le cycle de vie du développement de logiciels, mais elles peuvent également réduire la maintenance nécessaire pour les tests automatisés. Cela fait de la reconnaissance d’image une option attrayante pour de nombreuses équipes de développement de logiciels.

Il est important de noter que bien que ces avantages soient significatifs, la mise en œuvre réussie des tests automatisés par reconnaissance d’image nécessite une planification et une exécution soigneuses. Il est également essentiel de choisir le bon outil d’automatisation pour répondre aux besoins spécifiques de votre projet.

L’automatisation des tests avec la reconnaissance d’image offre de nombreux avantages par rapport à l’approche basée sur le code. Non seulement elle permet de tester plusieurs appareils, systèmes d’exploitation et navigateurs sans changer le test, mais elle peut également réduire considérablement le temps nécessaire à l’exécution des tests. De plus, avec des outils comme Eggplant, Zaptest, Airtest Project et Beastest, il est plus facile que jamais de mettre en œuvre cette approche.

(done) DoD vs Condition de succès vs Critères d'acceptation vs Couverture vs DoR vs Gherkin

January 18, 2024 10:31 AM

Introduction

Dans le monde du développement logiciel, il existe plusieurs termes et concepts clés qui sont souvent mal compris ou mal utilisés. Dans ce blog, nous allons démystifier ces termes et comprendre leur véritable signification et leur utilisation.

Définition de finition (DoD)

La définition de finition (DoD) est un ensemble de critères qui doivent être remplis avant qu'un élément de travail puisse être considéré comme terminé. La DoD peut inclure des critères tels que des tests unitaires réussis, une documentation complète et une revue par les pairs. Il est important de noter qu'il peut y avoir plusieurs DoD pour chaque phase du cycle de développement, et le véritable DoD est sur le cycle complet du développement, pas seulement sur une phase spécifique.

Condition de succès

Les conditions de succès sont des critères qui doivent être remplis pour qu'un projet soit considéré comme réussi. Les conditions de succès peuvent inclure des critères tels que le respect des délais, le respect du budget et la satisfaction du client.

Critères d'acceptation

Les critères d'acceptation sont des critères qui doivent être remplis pour qu'un élément de travail soit considéré comme accepté. Les critères d'acceptation peuvent inclure des critères tels que des tests réussis et une revue par les pairs.

Couverture

La couverture est un indicateur de la quantité de code ou de fonctionnalités qui ont été testées. La couverture peut être mesurée en termes de pourcentage du code ou des fonctionnalités testées.

Définition de prêt (DoR)

La définition de prêt (DoR) est un ensemble de critères qui doivent être remplis avant qu'un élément de travail puisse être commencé. Cependant, si une DoD est bien définie et utilisée correctement, une DoR peut ne pas être nécessaire.

Gherkin

Gherkin est un langage utilisé pour écrire des scénarios de test dans le développement logiciel basé sur le comportement. Il utilise une syntaxe simple, en langage naturel, pour décrire le comportement souhaité d'une fonctionnalité. Cependant, il est important de noter que Gherkin n'est pas censé remplacer les histoires utilisateurs, mais plutôt à les compléter en fournissant des détails sur le comportement attendu.

Conclusion

En somme, comprendre et utiliser correctement ces termes et concepts peut grandement améliorer l'efficacité et la qualité du développement logiciel. Il est important de toujours garder à l'esprit le contexte et les objectifs spécifiques de votre projet lors de l'utilisation de ces outils.

(done) Innovation & Amélioration continue

January 18, 2024 10:31 AM

Innovation & Amélioration continue

Introduction

Dans le monde de l'assurance qualité, l'innovation et l'amélioration continue sont essentielles pour rester compétitif et répondre aux besoins changeants des clients. Pour gérer efficacement le changement et la transformation, il est important de prendre en considération des modèles tels que la courbe de diffusion de l'innovation de Rogers et la philosophie Lean.

La courbe de diffusion de l'innovation de Rogers

La courbe de diffusion de l'innovation de Rogers est un modèle qui décrit comment une nouvelle idée ou un nouveau produit se propage à travers différentes catégories de personnes. Selon Rogers, les individus adoptent l'innovation à différents rythmes, généralement représentés par une courbe en forme de cloche divisée en cinq catégories : les innovateurs, les adoptants précoces, la majorité précoce, la majorité tardive et les retardataires. Comprendre où se situent vos équipes sur cette courbe peut aider à élaborer des stratégies de changement efficaces.

Lean

Lean est une philosophie de gestion qui vise à minimiser les déchets tout en maximisant la productivité. Dans le contexte de l'assurance qualité, Lean peut aider à identifier et à éliminer les processus inefficaces, à améliorer la qualité et à accélérer la livraison. Les principes Lean encouragent l'amélioration continue, ce qui peut conduire à une innovation constante.

Des idées

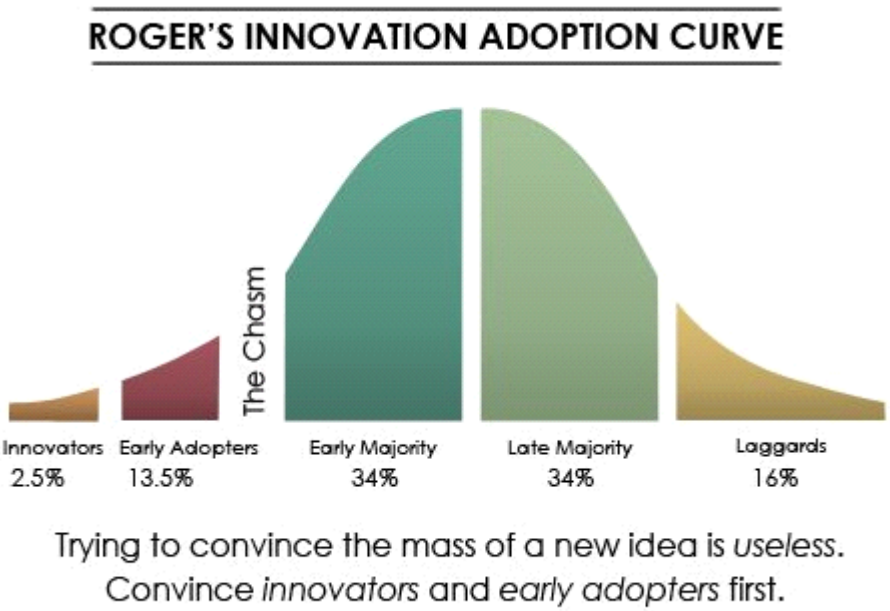
1. **Élaborez une stratégie d'innovation** : Pensez en termes d'amélioration continue dans toutes les sphères de vos activités : production, prestation de services, stratégie marketing et même modèle d'affaires.
2. **Adoptez un mode de gestion horizontale et communicatif** : Favorisez la communication ouverte et la collaboration entre tous les membres de l'équipe.
3. **Prêchez par l'exemple** : Montrez l'importance de l'innovation et de l'amélioration continue par votre propre comportement.
4. **Créez un environnement innovant** : Fournissez les ressources et l'espace nécessaires pour que les employés puissent expérimenter et innover.
5. **Récompensez les bons et les moins bons coups** : Encouragez la prise de risques et l'expérimentation en célébrant aussi bien les succès que les échecs constructifs.
6. **Méthodologies d'amélioration continue** : Explorez différentes méthodologies d'amélioration continue comme la méthode 5S, les cercles de qualité, la méthode Six Sigma, etc.
7. **Outils numériques pour l'amélioration continue** : Utilisez des outils numériques pour rationaliser les différentes étapes de l'amélioration continue : auditer, planifier, tester, mesurer l'efficacité, adopter ou révoquer.
8. **L'innovation au cœur des transformations à succès** : Soulignez comment l'innovation et l'amélioration continue sont au cœur des transformations d'entreprise réussies.
9. **Comprendre la résistance** : La résistance au changement est un instinct naturel chez les humains. Il est important de comprendre pourquoi les gens résistent au changement pour

pouvoir y répondre efficacement.

10. **Communication transparente et écoute** : Pour vaincre la résistance au changement, la transparence et l'écoute sont primordiales. Il faut bien définir la problématique à l'origine du changement et agir de façon objective et rationnelle durant tout le processus.
11. **Fournir une vision claire** : Assurez-vous que le sens du changement ait été expliqué. Les gens sont plus susceptibles d'accepter le changement s'ils comprennent pourquoi il est nécessaire et comment il les affectera personnellement.
12. **Pratiquer l'écoute active** : Écoutez attentivement les préoccupations de vos collaborateurs et répondez à leurs objections. Cela peut aider à dissiper les craintes et à renforcer la confiance.
13. **Ne pas céder face à des oppositions** : Il est important de rester ferme et de ne pas céder face à des oppositions. Cela ne signifie pas ignorer les préoccupations des gens, mais plutôt travailler avec eux pour trouver des solutions.
14. **Récompenser l'adoption du changement** : Encouragez l'adoption du changement en reconnaissant et en récompensant ceux qui l'adoptent. Cela peut aider à motiver les autres à suivre leur exemple.

Conclusion

En somme, la gestion du changement et la transformation en assurance qualité nécessitent une compréhension approfondie de modèles tels que la courbe de diffusion de l'innovation de Rogers et la philosophie Lean. En utilisant ces outils, les organisations peuvent favoriser l'innovation et l'amélioration continue, ce qui est essentiel pour réussir dans le paysage concurrentiel d'aujourd'hui.



(done) Entre assurer la qualité et le Kamikaze

January 18, 2024 10:31 AM

Entre assurer la qualité et le Kamikaze

Introduction

Dans le monde de l'assurance qualité, la plus grande épreuve à laquelle nous sommes confrontés est certainement la limite de l'assurance qualité. Assurer la qualité et éviter le kamikaze sont deux concepts très différents, mais tous deux essentiels au succès d'une entreprise.

Assurer la qualité

Assurer la qualité consiste à s'assurer que les produits ou services d'une entreprise répondent aux normes de qualité établies. Les professionnels de l'assurance qualité travaillent à différents niveaux pour s'assurer que les produits ou services répondent aux normes de qualité établies. Cela peut aider à garantir que les produits ou services d'une entreprise sont fiables et répondent aux besoins des clients.

Éviter le kamikaze

Éviter le kamikaze consiste à éviter les risques inutiles ou les comportements imprudents qui pourraient causer des dommages à l'entreprise ou à ses employés. Les professionnels du management des risques travaillent à différents niveaux pour identifier et atténuer les risques pour l'entreprise. Cela peut aider à garantir que l'entreprise fonctionne efficacement et évite les risques inutiles.

La limite de l'assurance qualité

La limite de l'assurance qualité est le point au-delà duquel les efforts supplémentaires pour assurer la qualité peuvent avoir des rendements décroissants. C'est le point où le coût et l'effort supplémentaires pour assurer la qualité dépassent les avantages potentiels. Il est important pour les entreprises de comprendre cette limite et de travailler pour l'équilibrer avec les risques potentiels.

Conclusion

Il est important de faire attention à ces deux concepts car ils sont tous deux importants pour le succès d'une entreprise. En équilibrant efficacement l'assurance qualité et la gestion des risques, les entreprises peuvent travailler pour assurer la qualité de leurs produits et services tout en évitant les risques inutiles.

(done) Les bénéfices de l'ingénierie du chaos

January 18, 2024 10:32 AM

Introduction

L’ingénierie du chaos est une discipline qui consiste à introduire délibérément des perturbations dans un système afin de tester sa résilience. Cette méthode est souvent utilisée dans les environnements informatiques pour tester la résilience des systèmes informatiques.

Qu’est-ce que l’ingénierie du chaos ?

L’ingénierie du chaos est une approche proactive pour découvrir les problèmes dans un système avant qu’ils ne causent des dégâts. Elle consiste à introduire délibérément des perturbations dans le système - comme le fait de tuer des processus, d’introduire des retards réseau, ou de simuler des défaillances de serveurs - pour observer comment le système réagit et pour s’assurer qu’il peut supporter ces conditions sans interruption de service.

Les bénéfices de l’ingénierie du chaos

Les bénéfices de l’ingénierie du chaos sont nombreux. Voici quelques-uns des principaux avantages :

1. **Identification des vulnérabilités** : L’ingénierie du chaos permet d’identifier les vulnérabilités des systèmes informatiques avant qu’elles ne soient exploitées par des attaquants malveillants.
2. **Amélioration de la résilience** : En identifiant les points faibles et en y remédiant, l’ingénierie du chaos permet d’améliorer la résilience des systèmes informatiques.
3. **Réduction des coûts** : En découvrant les problèmes plus tôt, les entreprises peuvent éviter les temps d’arrêt coûteux et les pertes de revenus associées à une défaillance du système.
4. **Amélioration de la confiance** : En démontrant que le système peut gérer des perturbations et des défaillances, l’ingénierie du chaos peut améliorer la confiance des parties prenantes dans le système.

Les risques de l’ingénierie du chaos

Cependant, il est important de noter que l’ingénierie du chaos peut également causer des problèmes si elle est mal exécutée. Si les perturbations sont trop importantes ou si elles sont mal ciblées, elles peuvent causer des dommages aux systèmes informatiques. Par conséquent, il est crucial de mettre en place des garde-fous appropriés et de s’assurer que les tests de chaos sont effectués dans un environnement contrôlé.

Conclusion

L’ingénierie du chaos est une pratique puissante qui peut aider les organisations à construire des systèmes plus résilients et fiables. En introduisant délibérément des perturbations dans le système et en apprenant de la manière dont il réagit, les organisations peuvent découvrir et corriger les problèmes avant qu’ils ne causent des dégâts, économisant ainsi du temps et de l’argent tout en améliorant la confiance dans le système.



(done) BLOG | Pourquoi éviter les tests est une meilleure pratique que tester !

January 18, 2024 10:32 AM

Introduction

Dans le domaine du développement de logiciels, les tests sont souvent considérés comme une étape essentielle pour assurer la qualité du code. Cependant, il existe des approches qui peuvent réduire la nécessité de tests intensifs sans compromettre la qualité.

Éviter les tests grâce à la programmation fonctionnelle

La programmation fonctionnelle est un style de programmation qui peut aider à réduire la nécessité de tests unitaires. En évitant les effets secondaires et en favorisant l'immuabilité, la programmation fonctionnelle peut rendre le code plus prévisible et plus facile à raisonner, réduisant ainsi la nécessité de tests unitaires.

Réduire le nombre de tests

Réduire le nombre de tests ne signifie pas négliger la qualité. Au contraire, cela signifie assurer la qualité de manière plus intelligente. Si un test peut valider plusieurs choses à la fois, il n'est pas nécessaire de créer et de maintenir de nombreux tests. Cela peut réduire la maintenance et le temps d'exécution des tests.

Les avantages de réduire le nombre de tests

Réduire le nombre de tests peut avoir plusieurs avantages. Cela peut réduire le temps et les ressources nécessaires pour maintenir et exécuter les tests. Cela peut également rendre le processus de développement plus agile, car moins de temps est consacré aux tests et plus de temps peut être consacré à l'écriture de code de qualité.

Conclusion

Les tests sont un élément important du développement de logiciels, mais ils ne sont pas la seule façon d'assurer la qualité. En adoptant des approches comme la programmation fonctionnelle et en réduisant intelligemment le nombre de tests, nous pouvons assurer la qualité du code tout en rendant le processus de développement plus efficace et plus agile.

(done) La leçon de la leçon

January 18, 2024 10:32 AM

Apprendre à apprendre dans l'assurance qualité du logiciel

Dans le domaine de l'assurance qualité du logiciel, l'apprentissage est une compétence essentielle. Cependant, il ne s'agit pas seulement d'apprendre à tester correctement, mais aussi d'apprendre à apprendre. Dans cet article, nous allons explorer comment nous pouvons améliorer notre capacité à apprendre et à nous adapter dans ce domaine en constante évolution.

Pourquoi apprendre à apprendre est important

Dans le monde du logiciel, les technologies et les pratiques évoluent rapidement. Pour rester à jour et maintenir la qualité de notre travail, nous devons être capables d'apprendre de manière efficace et autonome. Cela signifie non seulement acquérir de nouvelles connaissances, mais aussi être capable de remettre en question nos connaissances existantes et d'adapter nos méthodes de travail en conséquence.

Comment apprendre à apprendre

Voici quelques stratégies pour améliorer votre capacité à apprendre dans le domaine de l'assurance qualité du logiciel :

1. **Adoptez une mentalité de croissance** : Croyez en votre capacité à apprendre et à vous améliorer. Ne craignez pas l'échec, mais voyez-le comme une opportunité d'apprendre.
2. **Définissez vos objectifs d'apprentissage** : Qu'est-ce que vous voulez apprendre ? Pourquoi voulez-vous l'apprendre ? Avoir des objectifs clairs peut vous aider à rester motivé et concentré.
3. **Planifiez votre apprentissage** : Organisez votre temps et vos ressources de manière à favoriser l'apprentissage. Cela peut inclure la lecture de livres, la participation à des cours en ligne, la pratique de nouvelles compétences, etc.
4. **Pratiquez l'apprentissage actif** : Ne vous contentez pas de lire ou d'écouter passivement. Engagez-vous activement dans l'apprentissage en prenant des notes, en posant des questions, en discutant avec d'autres, etc.
5. **Recherchez des retours** : Demandez des retours sur votre travail et utilisez-les pour vous améliorer. Les retours peuvent venir de vos collègues, de vos mentors, ou même de vous-même à travers l'auto-évaluation.
6. **Prenez soin de vous** : L'apprentissage peut être exigeant mentalement et physiquement. Assurez-vous de prendre soin de votre santé en mangeant sainement, en faisant de l'exercice régulièrement, et en prenant le temps de vous détendre et de vous amuser.

Apprendre de nos erreurs

Dans la deuxième partie de cet article, nous allons explorer une autre signification du mot "leçon". En effet, une leçon peut aussi être une punition ou une conséquence. Dans ce contexte, l'idée est d'apprendre de nos erreurs et de transformer ces erreurs en un nouveau pouvoir acquis.

Apprendre de nos erreurs

Dans le domaine de l'assurance qualité du logiciel, les erreurs sont inévitables. Que ce soit une erreur de code, une mauvaise conception, ou une erreur de jugement, nous sommes tous susceptibles de faire des erreurs. Cependant, ce qui compte vraiment, ce n'est pas l'erreur elle-même, mais ce que nous en faisons.

Lorsque nous faisons une erreur, nous avons deux choix : nous pouvons soit laisser l'erreur nous décourager, soit la voir comme une opportunité d'apprendre et de grandir. En choisissant cette dernière option, nous pouvons transformer notre erreur en une leçon précieuse.

Transformer l'erreur en un nouveau pouvoir acquis

Chaque erreur est une occasion d'apprendre quelque chose de nouveau. Par exemple, une erreur de code peut nous apprendre une meilleure façon de structurer notre code. Une erreur de conception peut nous révéler une meilleure façon de répondre aux besoins des utilisateurs. Et une erreur de jugement peut nous aider à affiner notre processus de prise de décision.

En apprenant de nos erreurs, nous pouvons acquérir de nouvelles compétences et connaissances qui peuvent nous aider à améliorer la qualité de notre travail. De plus, en reconnaissant et en acceptant nos erreurs, nous pouvons développer une plus grande résilience et une meilleure capacité à faire face aux défis futurs.

En conclusion

une leçon peut être bien plus qu'une simple instruction ou un enseignement. Elle peut aussi être une punition ou une conséquence qui nous offre l'occasion d'apprendre et de grandir. En apprenant de nos erreurs et en transformant ces erreurs en un nouveau pouvoir acquis, nous pouvons continuer à améliorer la qualité de notre travail et à nous développer en tant que professionnels de l'assurance qualité du logiciel.

apprendre à apprendre est une compétence essentielle dans le domaine de l'assurance qualité du logiciel. En adoptant une mentalité de croissance, en définissant des objectifs d'apprentissage clairs, en planifiant votre apprentissage, en pratiquant l'apprentissage actif, en recherchant des retours, et en prenant soin de vous, vous pouvez améliorer votre capacité à apprendre et à vous adapter dans ce domaine en constante évolution.

(done) Une faute qui à coûtée 1M$ en a rapportée 1G$

January 18, 2024 10:32 AM

Introduction

Dans le monde du développement de logiciels, les erreurs peuvent être coûteuses. Cependant, avec une bonne assurance qualité, ces erreurs peuvent se transformer en opportunités d'apprentissage et de croissance.

L'importance de l'apprentissage à partir des erreurs

Chaque erreur est une occasion d'apprendre. Qu'il s'agisse d'un bug dans le code, d'une mauvaise conception ou d'une exigence mal comprise, chaque erreur nous donne l'occasion de comprendre ce qui a mal tourné et comment nous pouvons l'éviter à l'avenir.

Comment l'assurance qualité peut aider

L'assurance qualité joue un rôle crucial dans ce processus. Elle nous aide à identifier les erreurs, à comprendre leurs causes et à mettre en place des mesures pour les prévenir à l'avenir.

1. **Identification des erreurs** : L'assurance qualité nous aide à identifier les erreurs dès qu'elles se produisent. Cela nous permet de les corriger rapidement, minimisant ainsi leur impact.
2. **Comprendre les causes des erreurs** : Une fois l'erreur identifiée, l'assurance qualité nous aide à comprendre pourquoi elle s'est produite. Cela peut impliquer l'analyse du code, la revue des exigences ou la discussion avec l'équipe de développement.
3. **Prévention des erreurs futures** : En comprenant les causes des erreurs, nous pouvons mettre en place des mesures pour les prévenir à l'avenir. Cela peut inclure la formation de l'équipe de développement, la modification des processus de développement ou l'amélioration des outils de développement.

Transformer une erreur de 1M$ en un gain de 1G$

En apprenant de nos erreurs et en mettant en place des mesures pour les prévenir à l'avenir, nous pouvons transformer une erreur coûteuse en une opportunité de croissance. Par exemple, une erreur qui a coûté 1M$ à corriger pourrait nous aider à éviter une erreur similaire à l'avenir qui aurait pu coûter 10M$ ou plus. De plus, en améliorant nos processus et en augmentant la qualité de nos logiciels, nous pouvons augmenter la satisfaction de nos clients et potentiellement générer plus de revenus.

Conclusion

Dans le domaine du logiciel, les erreurs sont inévitables. Cependant, avec une bonne assurance qualité, nous pouvons transformer ces erreurs en opportunités d'apprentissage et de croissance. En fin de compte, une erreur qui coûte 1M$ pourrait nous aider à générer 1G$ de revenus à l'avenir.

(done) Prévenir versus subir

January 18, 2024 10:33 AM

Prévenir versus subir dans le domaine du logiciel

Introduction

Dans le domaine du logiciel, la différence entre prévenir et subir peut être la clé du succès ou de l'échec d'un projet. Prévenir signifie anticiper les problèmes avant qu'ils ne surviennent, tandis que subir signifie réagir aux problèmes après qu'ils se sont produits.

Prévenir

La prévention dans le domaine du logiciel implique une planification et une conception minutieuses, une gestion efficace des exigences, une assurance qualité rigoureuse et une attention constante à l'amélioration continue.

1. **Planification et conception** : Une bonne planification et conception peuvent aider à prévenir de nombreux problèmes avant même qu'ils ne surviennent. Cela comprend la définition claire des objectifs du projet, la compréhension des besoins des utilisateurs et la conception d'une architecture solide.
2. **Gestion des exigences** : Une gestion efficace des exigences peut aider à prévenir les malentendus et les erreurs. Cela comprend la définition claire des exigences, leur communication efficace à toutes les parties prenantes et leur suivi tout au long du projet.
3. **Assurance qualité** : Une assurance qualité rigoureuse peut aider à prévenir les défauts et les erreurs. Cela comprend la mise en place de processus de test rigoureux, la réalisation d'audits de qualité et la mise en place de mécanismes de feedback pour l'amélioration continue.
4. **Amélioration continue** : L'amélioration continue est un élément clé de la prévention. Cela implique l'évaluation constante des processus et des résultats, et la recherche de moyens de les améliorer.

Subir

Subir dans le domaine du logiciel signifie réagir aux problèmes après qu'ils se sont produits. Cela peut inclure la correction des bugs, la gestion des incidents et la résolution des problèmes de performance.

5. **Correction des bugs** : La correction des bugs est une partie inévitable du développement de logiciels. Cependant, une approche réactive à la correction des bugs peut entraîner des retards et des coûts supplémentaires.
6. **Gestion des incidents** : La gestion des incidents implique la réponse aux problèmes après qu'ils se sont produits. Cela peut inclure la résolution des problèmes de performance, la gestion des pannes de système et la réponse aux problèmes de sécurité.
7. **Résolution des problèmes de performance** : La résolution des problèmes de performance peut être un défi majeur dans le développement de logiciels. Cela peut impliquer l'optimisation du code, l'amélioration de l'efficacité des systèmes et la gestion des ressources.

Conclusion

Dans le domaine du logiciel, il est préférable de prévenir que de subir. Une approche proactive à la gestion de la qualité et à l'amélioration continue peut aider à prévenir de nombreux problèmes avant qu'ils ne surviennent, ce qui peut conduire à des logiciels de meilleure qualité, une plus grande satisfaction des utilisateurs et une meilleure rentabilité.

(done) C'est quoi l'assurance qualité ?

January 18, 2024 10:33 AM

C’est quoi au juste l’assurance qualité aujourd’hui ?

Introduction

L’assurance qualité (AQ) est un élément essentiel du développement de logiciels. Elle garantit que les logiciels sont développés conformément aux normes et aux exigences de qualité. Cela peut aider à réduire les coûts liés aux erreurs et aux défauts dans le logiciel, ainsi qu’à améliorer la satisfaction des clients.

Qu’est-ce que la qualité ?

La qualité est souvent perçue comme l’absence de défauts ou de problèmes dans un produit ou un service. Cependant, cette définition est assez réductrice et ne capture pas toute la complexité de ce qu’est réellement la qualité.

En réalité, la qualité est une mesure de l’excellence ou de l’état sans défaut d’un produit ou d’un service. Elle comprend plusieurs aspects, dont :

1. **La conformité aux spécifications** : Le produit ou le service répond-il aux exigences définies ?
2. **La fiabilité** : Le produit ou le service est-il fiable et fonctionne-t-il comme prévu sur une longue période ?
3. **L’utilisabilité** : Le produit ou le service est-il facile à utiliser et intuitif ?
4. **L’efficacité** : Le produit ou le service répond-il aux besoins de l’utilisateur de manière efficace et sans gaspillage de ressources ?
5. **La maintenabilité** : Le produit ou le service peut-il être facilement maintenu et mis à jour ?

Il est important de noter que la qualité n’est pas un attribut absolu, mais est souvent définie en fonction des attentes et des besoins de l’utilisateur ou du client. Ce qui peut être considéré comme de haute qualité pour un utilisateur peut ne pas l’être pour un autre.

Qu’est-ce que la méthodologie CARE ?

CARE (Collaborative Agile Reliable Efficacy) est une méthodologie de gestion qui intègre SAFe, Kanban, XP, tous pilotés par Lean. Elle met l’accent sur la collaboration, l’agilité, la fiabilité et l’efficacité, tout en plaçant la qualité au centre de tout.

Collaborative

CARE favorise la collaboration grâce à SAFe. Elle brise les silos organisationnels, favorisant une communication ouverte entre toutes les parties prenantes du projet. CARE s’efforce de créer un environnement qui facilite et valorise la collaboration à tous les niveaux de l’organisation.

Agile

L’agilité est assurée par Kanban pour visualiser le flux de travail. CARE adopte également DevOps, CI/CD, les tests continus et le shift-left. Ces principes permettent une intégration et une livraison continues, contribuant à l’agilité de l’approche.

Reliable

La fiabilité est maintenue par XP, qui assure la qualité grâce au cœur de CARE où se trouve le développement piloté par la qualité (Quality-Driven Development, QDD). La qualité est présente à chaque phase pour la prévention des problèmes, la mise en place d’alertes et la planification de réactions rapides.

Efficacy

L’efficacité est obtenue grâce à Lean, qui guide le processus avec une mentalité d’amélioration continue. Lean se concentre sur l’élimination de tout ce qui est sans valeur ajoutée dans le processus de développement. En éliminant ces éléments, Lean permet d’accélérer le processus de développement et d’augmenter l’efficacité.

Organigramme CARE

Voici un organigramme CARE proposé :

- **VP QA (TQM)** : S’assure que l’organisation dans son ensemble cultive bien la pratique de la qualité.
- **Directeur QA** : S’assure que le département spécifique cultive bien la pratique de la qualité.
- **Manager QA** : S’assure que l’équipe SAFe cultive bien la pratique de la qualité.
- **Superviseur QA** : S’assure que l’escouade Kanban cultive bien la pratique de la qualité.
- **Expert QA** : S’assure que l’unité XP3A cultive bien la pratique de la qualité.

Le coach QA est là pour guider et entraîner tous les rôles et contributeurs QA. Il doit assurer la pérennité de la transformation QA. Ce n’est pas un rôle éphémère comme l’exemple de remonter un département et fini merci bonsoir. Le coach c’est le joueur indispensable, le joueur clé, comme dans le sport, pour atteindre la satisfaction !

Conclusion

L’assurance qualité est un élément essentiel du développement de logiciels et la méthodologie CARE offre une approche complète pour gérer la qualité. En mettant l’accent sur la collaboration, l’agilité, la fiabilité et l’efficacité, et en plaçant la qualité au centre de tout, CARE peut aider les organisations à développer des logiciels de haute qualité qui répondent aux attentes des utilisateurs et des parties prenantes. C’est un objectif vers lequel toutes les organisations devraient tendre.

(done) La naissance de docker

January 18, 2024 10:33 AM

Salut à tous ! Aujourd'hui, on va parler d'un sujet qui me tient à cœur : la naissance de Docker. Oui, je sais, ça peut sembler un peu technique et ennuyeux, mais croyez-moi, une fois que vous aurez compris comment Docker est né, vous ne verrez plus jamais le développement de logiciels de la même manière. Alors, accrochez-vous, ça va être une sacrée aventure !

1. Ça marche sur ma machine

On a tous entendu cette phrase, n'est-ce pas ? Vous passez des heures à déboguer un problème, et quand vous demandez de l'aide, on vous dit : "Bizarre, ça marche sur ma machine". C'est un peu comme essayer de comprendre pourquoi votre gâteau n'a pas levé alors que vous avez suivi la recette à la lettre.

2. Alors expédiez votre machine

Et puis un jour, quelqu'un a eu une idée brillante : "Et si on pouvait simplement expédier notre machine ?". C'est un peu comme si vous pouviez emballer votre cuisine et l'envoyer à quelqu'un d'autre pour qu'il puisse cuisiner exactement le même gâteau.

3. Et voilà comment Docker est né

Docker est un peu comme une boîte magique qui vous permet d'emballer votre application avec tout ce dont elle a besoin pour fonctionner, et de l'envoyer à n'importe qui, n'importe où. Plus besoin de se soucier des dépendances, des versions de logiciels, ou des différences entre les systèmes d'exploitation. Avec Docker, si ça marche sur votre machine, ça marchera partout !

L'impact titanesque de Docker sur l'assurance qualité

Docker a non seulement simplifié le déploiement des applications, mais a également apporté des améliorations majeures dans le domaine de l'assurance qualité. Voici quelques exemples de l'impact de Docker :

1. Les branches de prévisualisation

Avec Docker, nous pouvons créer des branches de prévisualisation qui nous permettent de tester nos modifications dans un environnement isolé avant de les fusionner avec la branche principale. C'est un peu comme avoir une salle d'essai où vous pouvez essayer de nouveaux vêtements avant de les acheter.

2. Déploiement sur demande

Docker nous permet de configurer un déploiement sur demande sans altérer l'environnement existant. C'est un peu comme pouvoir construire une nouvelle maison à côté de l'ancienne sans avoir à la démolir.

3. Exécution parallèle de tests automatisés

Avec Docker, nous pouvons maintenant exécuter des tests automatisés en parallèle avec un nombre infini de déploiements de systèmes sous test (SUT). C'est un peu comme avoir une armée de robots à notre disposition pour tester notre logiciel.

4. Éviter l'installation d'outils avec des configurations différentes

Grâce à Docker, nous pouvons éviter d'installer des outils avec des configurations différentes sur nos postes. C'est un peu comme avoir une boîte à outils universelle qui

contient tous les outils dont nous avons besoin, peu importe le travail à accomplir.

Conclusion

Et voilà, c'est tout pour aujourd'hui ! J'espère que vous avez apprécié cette plongée dans le monde fascinant de Docker et de l'assurance qualité. N'oubliez pas, comprendre comment la naissance de Docker a changé le monde peut vous aider à apprécier encore plus la beauté et la puissance de cette technologie. Alors, à vos claviers, et faite du Docker !

***4 secrets de l'automatisation

January 18, 2024 10:33 AM

4 secrets de l'automatisation.pdf
22 Mar 2023, 11:54 PM

(done) D.O.W.N.T.I.M.E. : Les 8 péchés capitaux de l'effort

January 18, 2024 10:34 AM

Salut à tous ! Aujourd'hui, on va parler d'un sujet qui me tient à cœur : D.O.W.N.T.I.M.E., les huit péchés capitaux. Oui, je sais, ça peut sembler un peu technique et ennuyeux, mais croyez-moi, une fois que vous aurez compris ces erreurs, vous ne verrez plus jamais de la même manière. Alors, accrochez-vous, ça va être une sacrée aventure !

1. D pour Défauts

C'est un peu comme acheter un produit défectueux. Les défauts dans vos produits ou services peuvent entraîner des coûts supplémentaires et une insatisfaction des clients.

2. O pour Outreproduction

C'est un peu comme cuisiner pour une armée alors que vous n'avez que quelques invités. La surproduction peut entraîner des coûts supplémentaires et un gaspillage de ressources.

3. W pour Attente

C'est un peu comme attendre le bus sous la pluie. Les temps d'attente peuvent entraîner des retards et une baisse de productivité.

4. N pour Non-exploitation du talent

C'est un peu comme avoir un super-héros dans votre équipe, mais ne lui donner que des tâches de bureau. Le talent non utilisé peut entraîner une baisse de moral et une perte d'opportunités.

5. T pour Transport

C'est un peu comme faire un détour inutile sur le chemin du travail. Le transport inefficace peut entraîner des coûts supplémentaires et une perte de temps.

6. I pour Inventaire

C'est un peu comme avoir une armoire pleine de vêtements que vous ne portez jamais. Un inventaire excédentaire peut entraîner des coûts de stockage supplémentaires et une perte de ressources.

7. M pour Mouvement

C'est un peu comme faire des allers-retours inutiles dans votre maison. Le mouvement inutile peut entraîner une perte de temps et une baisse de productivité.

8. E pour Excès de traitement

C'est un peu comme repasser une chemise qui est déjà sans plis. Le traitement supplémentaire peut entraîner une perte de temps et de ressources.

Et voilà, c'est tout pour aujourd'hui ! J'espère que vous n'avez pas eu trop de D.O.W.N.T.I.M.E. N'oubliez pas, comprendre ces erreurs peut vous aider à éviter les pièges courants et à créer plus efficacement. Alors, à vos claviers, et que le meilleur code gagne !

(done) Les 10 erreurs les plus communes en automatisation : Un décompte chaud

January 18, 2024 10:34 AM

Salut à tous ! Aujourd'hui, on va parler d'un sujet qui me tient à cœur : les erreurs communes en automatisation. Oui, je sais, ça peut sembler un peu technique et ennuyeux, mais croyez-moi, une fois que vous aurez compris ces erreurs, vous ne verrez plus jamais l'automatisation de la même manière. Alors, accrochez-vous, ça va être une sacrée aventure !

1. Tomber amoureux d'une seule technologie

C'est un peu comme avoir un coup de cœur pour une seule chanson et l'écouter en boucle. Au bout d'un moment, vous risquez de passer à côté de plein d'autres morceaux géniaux. De la même manière, ne vous limitez pas à une seule technologie ou plateforme. Explorez et évaluez diverses options pour trouver celle qui convient le mieux à vos besoins spécifiques. N'ayez pas peur de sortir de votre zone de confort et d'essayer de nouvelles technologies ou de redécouvrir d'anciennes qui ont évolué. C'est bin beau selenium mais il y a autre chose, voyez par exemple dans les top Zaptest, AirTest Project, Eggplant ou Beastest qui sont beaucoup plus trippant que Selenium/Webdriver ou Cypress

2. Croire que l'entreprise peut automatiser sans la TI ou DevOps (QAOps)

C'est un peu comme essayer de faire un gâteau sans four. L'intégration de la technologie informatique est cruciale pour une automatisation réussie. Assurez-vous que votre équipe TI est impliquée dès le début.

3. Penser que l'automatisation est toujours la solution

L'automatisation peut être la meilleure option à long terme pour les processus métier et informatiques, mais les dirigeants ne peuvent pas simplement l'utiliser pour combler les lacunes d'un processus mal conçu. L'automatisation n'est pas destinée à compenser les défaillances des systèmes ou à différer le remplacement des systèmes. Comme la fameuse phrase que j'entends malheureusement trop souvent : "Nous allons maintenant, avec nos tests automatisés, réussir à respecter nos engagements de sprint"



Ce n'est qu'un exemple ridicule parmi tant d'autres !

4. Ne pas impliquer tous les intervenants et contributeurs

C'est un peu comme organiser une fête surprise sans inviter tous les amis. Incluez toutes les parties prenantes dans le processus pour assurer une intégration réussie et une adoption large de l'automatisation.

5. Ne pas consacrer suffisamment de temps aux tests

C'est un peu comme essayer de courir un marathon sans s'entraîner. Investissez du temps dans des tests approfondis pour identifier et résoudre les problèmes avant le déploiement complet.

6. Gaspiller des efforts sur des projets trop compliqués

C'est un peu comme essayer de construire une maison sans plan. Concentrez-vous sur des projets gérables et évitez ceux qui sont trop ambitieux ou compliqués dès le départ.

7. Traiter l'automatisation comme une simple réplication des tâches

C’est un peu comme copier les notes de quelqu’un d’autre sans vraiment comprendre le sujet. Considérez l’automatisation comme une opportunité d’améliorer et d’optimiser les processus, pas seulement de répliquer les tâches existantes.

8. Ne pas surveiller en postproduction

C’est un peu comme cuisiner un plat sans goûter. Continuez à surveiller et à ajuster vos systèmes automatisés après leur mise en œuvre pour assurer un fonctionnement optimal.

9. Utiliser les mauvaises métriques pour mesurer le succès

C’est un peu comme essayer de mesurer la température avec une règle. Choisissez des métriques pertinentes qui reflètent avec précision l’efficacité et la valeur ajoutée par l’automatisation.

10. Ne pas planifier et ne pas automatiser dès le début du cycle de vie du développement

C’est un peu comme essayer de construire une maison sans plan d’architecte. L’automatisation doit être planifiée et mise en œuvre dès le début du cycle de vie du développement. Cela permet de s’assurer que l’automatisation est intégrée de manière transparente dans le processus de développement et qu’elle peut apporter une valeur maximale.

Conclusion

Qu’auriez-vous ajouté en 11 ?
Et voilà, c’est tout pour aujourd’hui ! J’espère que vous avez apprécié ce décompte sur les erreurs les plus fréquentes de l’automatisation. N’oubliez pas, comprendre ces erreurs peut vous aider à automatiser plus efficacement et à créer des logiciels de haute qualité. Et rappelez-vous, l’automatisation n’est pas magique, ce n’est pas gratuit, mais c’est le plus bel investissement que vous puissiez faire pour votre entreprise. Alors, à vos claviers, et que le meilleur code gagne !

(done) Audit versus Inspection

January 18, 2024 10:34 AM

Audit versus Inspection : Un duel au sommet

Salut à tous ! Aujourd'hui, on va parler de deux concepts clés dans le monde du logiciel : l'audit et l'inspection. Vous êtes prêts ? Alors, c'est parti !

1. L'Audit

L'audit, c'est un peu comme un détective privé qui enquête sur un cas. Il fait partie du processus d'assurance qualité (QA) et ne peut pas faire partie de l'inspection. Il est généralement moins fréquent et est un processus proactif. C'est un processus complexe et prolongé qui suit des règles et des normes d'audit spécifiques. Il s'agit d'un processus systématique et structuré qui identifie les causes profondes des déficiences observées. C'est un processus qualitatif qui va souvent plus loin que l'inspection et se préoccupe de besoins plus larges.

2. L'Inspection

L'inspection, c'est un peu comme un contrôleur de qualité qui vérifie chaque produit qui sort de la chaîne de production. Elle fait partie du processus de contrôle de la qualité (QC) et peut faire partie de l'audit. Elle est généralement plus fréquente que l'audit et est un processus de détection. C'est un processus simple qui n'a pas de règles et de normes fixes - cela dépend de la nature de l'inspection. Elle est moins structurée et peut ne pas identifier les causes profondes des déficiences observées. C'est un processus quantitatif qui se concentre généralement sur un nombre plus restreint de caractéristiques.

L'audit et l'inspection sont en effet deux des nombreux "zigotos" qui font partie de la Qualimétrie !
Alors, qu'est-ce que la Qualimétrie exactement ?

La Qualimétrie est une discipline qui se concentre sur la mesure de la qualité, en particulier dans le domaine du logiciel. Elle utilise des outils comme SonarQube, un outil d'analyse statique de code, pour obtenir différentes mesures et informations sur le code source des applications. Ces mesures peuvent inclure la liste des violations des règles de codage, la dette technique, le taux de duplication de code dans l'application, le taux de couverture de code par les tests unitaires, et la complexité du code.

La Qualimétrie est basée sur l'idée que ce qui peut être mesuré peut être amélioré. En mesurant différents aspects de la qualité du logiciel, nous pouvons identifier les domaines qui nécessitent une amélioration et prendre des mesures pour améliorer ces domaines. C'est un peu comme un entraîneur de sport qui utilise des statistiques pour aider son équipe à s'améliorer.

Dans le cadre de la culture de la pratique de la qualité du logiciel, la Qualimétrie joue un rôle clé. Elle nous aide à comprendre comment nous faisons, où nous pouvons nous améliorer, et comment nous pouvons atteindre nos objectifs de qualité. Que ce soit par le biais d'audits, d'inspections, ou d'autres méthodes, la Qualimétrie est un outil précieux pour toute équipe de développement de logiciels.

Conclusion

Et voilà, c'est tout pour aujourd'hui ! J'espère que vous avez apprécié cette plongée dans le monde fascinant de l'audit, de l'inspection et de la Qualimétrie. N'oubliez pas, comprendre ces concepts peut vous aider à créer des logiciels de haute qualité. Alors, à vos claviers, et que le meilleur code gagne !

(done) Top 5 des leçons sur l’automatisation : Un voyage fascinant

January 18, 2024 10:34 AM

Salut à tous ! Aujourd’hui, on va parler d’un sujet qui me tient à cœur : les leçons sur l’automatisation des tests. Oui, je sais, ça peut sembler un peu technique et ennuyeux, mais croyez-moi, une fois que vous aurez compris ces leçons, vous ne verrez plus jamais le développement de logiciels de la même manière. Alors, accrochez-vous, ça va être une sacrée aventure !

1. Connaître votre cas d’automatisation

La première leçon est de connaître la valeur de votre cas de test. Il est important de comprendre pourquoi vous voulez automatiser un test et quel est l’impact de ce test sur votre produit. C’est un peu comme savoir pourquoi vous voulez aller à une fête - est-ce pour voir vos amis, pour danser, ou juste pour le buffet gratuit ?

2. Se concentrer sur les besoins de l’entreprise

La deuxième leçon est de toujours penser à la valeur ajoutée de l’automatisation pour votre entreprise. Ne vous contentez pas de penser à l’automatisation pour l’automatisation, mais pensez à ce qu’elle peut apporter à votre entreprise. C’est un peu comme acheter une nouvelle voiture - vous ne le faites pas juste pour le plaisir de conduire, mais parce qu’elle vous aide à aller d’un point A à un point B.

3. S’adapter au contexte

La troisième leçon est de toujours adapter votre approche d’automatisation au contexte de votre projet. Chaque projet est unique et nécessite une approche différente. C’est un peu comme choisir une tenue pour une soirée - vous ne porteriez pas la même chose pour un barbecue en plein air que pour un dîner formel.

4. Construire votre propre Stratégie et Politique de la culture de la pratique assurance qualité

La quatrième leçon est de construire votre propre Stratégie et Politique de la culture de la pratique assurance qualité. Il est important de comprendre vos besoins et de construire une stratégie qui répond à ces besoins. C’est un peu comme préparer un voyage - vous devez d’abord comprendre vos destinations et vos intérêts avant de pouvoir planifier un itinéraire qui vous convient.

5. Quand les choses sont floues, commencez par une preuve de concept

Expérimenter c’est la clé !
La cinquième leçon est de commencer par une preuve de concept lorsque les choses sont floues. C’est un excellent moyen de tester vos idées et de voir si elles fonctionnent avant de vous engager pleinement. C’est un peu comme faire un essai routier avant d’acheter une nouvelle voiture.

Conclusion

Et voilà, c’est tout pour aujourd’hui ! J’espère que vous avez apprécié cette plongée dans le monde fascinant de l’automatisation des tests. N’oubliez pas, comprendre ces cinq leçons peut vous aider à automatiser plus efficacement et à créer des logiciels de haute qualité. Alors, à vos claviers, et que le meilleur code gagne !
Il n'est pas toujours nécessaire d'avoir des blogs lourds, parfois c’est juste bon de voyager léger



(done) Validation, Surveillance et Vérification : Trois Amis Inséparables

January 18, 2024 10:35 AM

Salut à tous ! Aujourd'hui, on va parler de trois concepts clés dans le monde du logiciel : la validation, la surveillance et la vérification. Vous êtes prêts ? Alors, c'est parti !

1. La Validation

La validation, c'est un peu comme votre maman qui vérifie que vous avez bien mis votre écharpe avant de sortir quand il fait froid. Elle intervient avant une activité et donne des informations sur la capacité à livrer les résultats prévus. Par exemple, lors de la conception d'une combinaison de mesures de contrôle ou lorsque des modifications sont apportées aux mesures de contrôle mises en œuvre.

2. La Surveillance

La surveillance, c'est comme votre professeur qui vous regarde pendant que vous passez un examen. Elle intervient pendant une activité et fournit des informations pour agir dans un délai spécifié. Par exemple, lors de l'exécution d'une séquence planifiée d'observations ou de mesures pour évaluer si un processus fonctionne comme prévu.

3. La Vérification

La vérification, c'est comme votre patron qui vérifie que vous avez bien fait votre travail. Elle intervient après une activité et fournit des informations pour confirmer la conformité. Par exemple, lors de la confirmation périodique que le processus continue d'être aussi efficace que lorsqu'il a été validé pour la première fois.

Conclusion

Et voilà, c'est tout pour aujourd'hui ! J'espère que vous avez apprécié cette plongée dans le monde fascinant de la validation, de la surveillance et de la vérification. N'oubliez pas, comprendre ces trois concepts peut vous aider à créer des logiciels de haute qualité. Alors, à vos claviers, et que le meilleur code gagne !

Add label

(done) 7 principes de test ?

January 18, 2024 10:35 AM

Plongeons dans le monde fascinant des tests logiciels !

Salut à tous ! Aujourd'hui, on va parler d'un sujet qui me tient à cœur : les principes du test logiciel. Oui, je sais, ça peut sembler un peu technique et ennuyeux, mais croyez-moi, une fois que vous aurez compris ces principes, vous ne verrez plus jamais le développement de logiciels de la même manière. Alors, accrochez-vous, ça va être une sacrée aventure !

1. L'absence d'erreurs est une illusion

On commence fort avec ce premier principe. Vous pensez que votre logiciel est parfait parce qu'il n'a pas d'erreurs ? Détrompez-vous ! Même si votre logiciel ne présente aucune erreur, cela ne signifie pas qu'il répond aux besoins de vos utilisateurs. Et après tout, c'est bien ça le plus important, non ?

2. La présence d'erreurs

Alors, on sait que les tests peuvent nous montrer la présence d'erreurs. Mais attention, ils ne peuvent pas prouver leur absence. C'est un peu comme chercher une aiguille dans une botte de foin, on ne peut jamais être sûr à 100% de l'avoir trouvée.

3. Tester tôt

Vous savez ce qu'on dit : mieux vaut prévenir que guérir. Commencer les tests tôt dans le cycle de développement peut nous aider à identifier et à corriger les erreurs plus tôt. C'est un peu comme faire du jardinage, plus on s'occupe de ses plantes tôt, plus elles seront belles et en bonne santé.

4. Les tests exhaustifs sont impossibles

C'est un peu déprimant, mais c'est la vérité : on ne peut pas tester toutes les combinaisons et conditions. C'est comme essayer de compter toutes les étoiles dans le ciel, c'est juste impossible. Mais ne vous inquiétez pas, on peut toujours se concentrer sur les zones à haut risque et définir des priorités de test.

5. Regroupement des défauts

C'est un phénomène intéressant : les erreurs ont tendance à se regrouper dans certaines zones. C'est un peu comme si elles avaient leur propre club VIP. En se concentrant sur ces zones, on peut trouver et corriger un grand nombre d'erreurs.

6. Le paradoxe du pesticide

C'est probablement mon préféré. L'exécution répétée des mêmes tests ne permettra pas de trouver plus d'erreurs. C'est un peu comme utiliser le même pesticide encore et encore, à un moment donné, les insectes vont développer une résistance. Il est donc important de réviser et d'actualiser régulièrement les tests.

7. Les tests dépendent du contexte

Enfin, dernier mais non le moindre, les méthodes et outils de test dépendent du contexte spécifique. C'est comme essayer de visser un clou avec un tournevis, ça ne marchera tout simplement pas.

Conclusion

Et voilà, c'est tout pour aujourd'hui ! J'espère que vous avez apprécié cette plongée dans le

monde fascinant des tests logiciels. N’oubliez pas, comprendre ces sept principes de test peut vous aider à tester plus efficacement et à créer des logiciels de haute qualité. Alors, à vos claviers, et que le meilleur code gagne !

(done) Contrôle de la qualité, Assurance qualité et Culture de la pratique assurance qualité: Un TOUT !

January 18, 2024 10:35 AM

Introduction

Dans le domaine du logiciel, le contrôle de la qualité (QC), l'assurance qualité (QA) et la culture de la pratique assurance qualité sont des concepts clés pour garantir la fiabilité et l'efficacité des logiciels.

Contrôle de la qualité (QC)

Le contrôle de la qualité est un processus qui vise à identifier et à corriger les défauts dans les produits logiciels. Il se concentre sur l'examen de la qualité des produits finaux et du résultat final plutôt que sur les processus utilisés pour créer un produit. C'est un processus réactif et est de nature détective.

Assurance qualité (QA)

L'assurance qualité, en revanche, est une activité qui assure que les approches, techniques, méthodes et processus conçus pour les projets sont correctement mis en œuvre. Elle reconnaît les défauts dans le processus. L'assurance qualité est réalisée avant le contrôle de la qualité. Elle se concentre sur la prévention des défauts. C'est un processus proactif et est préventif par nature.

Culture de la pratique assurance qualité

La culture de la pratique assurance qualité est un concept plus large qui englobe à la fois le QC et le QA. Elle implique la promotion d'une compréhension profonde de la qualité à tous les niveaux de l'organisation et l'adoption de pratiques qui favorisent la qualité à chaque étape du processus de développement.

Les idées de Deming

W. Edwards Deming a proposé 14 points pour la gestion de la qualité totale. Ces points peuvent être appliqués par toute organisation pour mettre en œuvre plus efficacement la gestion de la qualité totale. Selon Deming, les organisations qui se concentrent sur l'amélioration de la qualité réduiront automatiquement les coûts.

Le rôle du coach QA

Un coach QA guide, soutient et rassemble une équipe pour posséder et améliorer collectivement la qualité grâce à la facilitation, l'éducation, l'expérimentation et la visualisation. Ils sont un défenseur passionné de la qualité.

Ce n'est pas la responsabilité du QA de tester ?

bien que le contrôle de la qualité soit crucial, il est important de noter que ce n'est pas la responsabilité du QA de tester. Le rôle du QA est plutôt de s'assurer que les processus appropriés sont en place pour permettre la création d'un produit de haute qualité. Cela peut inclure la mise en place de normes et de procédures, l'assurance que toutes les étapes du processus de développement sont correctement suivies, et la promotion d'une culture de qualité au sein de l'équipe.

En revanche, le contrôle de la qualité est généralement effectué par une équipe de testeurs dédiée. Leur rôle est d'identifier les bugs et les problèmes dans le produit avant qu'il ne soit livré. Cependant, dans une culture de qualité, tout le monde dans l'équipe a la responsabilité de la qualité et peut participer aux tests.

Si les tests ne sont pas automatisés, tout le monde peut exécuter les tests dans le cadre du contrôle de la qualité. Cela est possible grâce à un plan de test bien défini, généralement élaboré par les “3 amigos” - généralement un représentant du business, un développeur et un testeur.

Ces trois rôles travaillent ensemble pour définir les critères d’acceptation et pour comprendre comment le logiciel doit fonctionner. Le plan de test qui en résulte est clair, complet et compréhensible par tous les membres de l’équipe. Ainsi, même si les tests ne sont pas automatisés, n’importe qui dans l’équipe peut exécuter les tests pour assurer le contrôle de la qualité. Cela renforce l’idée que la qualité est la responsabilité de tous.

L'automatisation des tests solidifie le contrôle qualité et le rend plus efficace

Cependant, il est important de noter que l’automatisation des tests peut apporter de nombreux avantages, notamment l’efficacité, la fiabilité et la rapidité. Mais que les tests soient automatisés ou non, l’objectif reste le même : assurer la qualité du logiciel dont réduire le contrôle de la qualité à la machine plus qu’a l’humain, ce qui rend à la fois plus efficace le contrôle de la qualité.

Conclusion

En résumé, le QA et le QC jouent tous deux des rôles importants dans la garantie de la qualité, mais leurs responsabilités sont différentes. Le QA se concentre sur la prévention des défauts par l’amélioration des processus, tandis que le QC se concentre sur la détection des défauts dans le produit fini. Dans une culture de qualité, ces activités sont intégrées dans le processus de développement, avec tout le monde dans l’équipe qui prend la responsabilité de la qualité.

Comprendre la différence entre le contrôle de la qualité et l’assurance qualité, ainsi que leur rôle dans la culture de la pratique assurance qualité, est essentiel pour garantir la qualité du logiciel. En intégrant ces concepts et en adoptant les idées de Deming, nous pouvons tous contribuer à la création de logiciels de haute qualité.

(done) Under one roof

January 18, 2024 10:35 AM

Sous un même toit: Cultiver la culture de l'assurance qualité

Introduction

Dans le domaine du logiciel, l'expression "Under one roof" (sous un même toit) est souvent utilisée pour décrire une approche de développement de logiciel où tous les aspects du développement du produit logiciel sont réalisés par une seule équipe. Cela comprend la conception, l'interface utilisateur et l'expérience utilisateur (UI&UX), le développement d'applications web et mobiles, l'assurance qualité (QA), et le support.

Cette approche offre plusieurs avantages, notamment :

- **Gain de temps** : Il n'est pas nécessaire de chercher des compétences supplémentaires et d'impliquer d'autres partenaires de développement.
- **Économie d'argent** : Travailler avec une seule équipe sur toutes les parties du produit minimise de nombreux risques qui entraînent une perte d'argent.
- **Facilité de communication et de gestion** : Lorsqu'une seule équipe est affectée à un projet, il est plus facile de gérer et de diriger le projet.
- **Optimisation de l'intégration** : L'intégration du travail effectué par différents développeurs dans différentes parties du monde, mettant en œuvre différentes approches et pratiques de développement, est un défi.

En résumé, "Under one roof" signifie que toutes les activités de développement de logiciel sont intégrées dans le processus de développement, ce qui peut conduire à des produits logiciels plus fiables et efficaces.

Dans le monde de l'informatique, l'assurance qualité est un élément essentiel pour garantir la fiabilité et l'efficacité des logiciels. Dans cet article, nous allons explorer comment cultiver une culture d'assurance qualité "sous un même toit".

Comprendre l'assurance qualité

L'assurance qualité est un processus qui vise à prévenir les défauts et les erreurs dans les produits logiciels. Elle comprend des activités qui assurent la qualité des produits logiciels tout au long du cycle de vie du développement.

Cultiver une culture d'assurance qualité

Cultiver une culture d'assurance qualité signifie encourager chaque membre de l'équipe à prendre la responsabilité de la qualité du produit. Cela implique de promouvoir des pratiques telles que les revues de code, les tests unitaires, l'intégration continue et le déploiement continu.

Assurance qualité sous un même toit

"Sous un même toit" signifie que toutes les activités d'assurance qualité sont intégrées dans le processus de développement logiciel. Cela signifie que chaque développeur, testeur, chef de projet et utilisateur final joue un rôle dans l'assurance de la qualité du produit.

Conclusion

Cultiver une culture d'assurance qualité "sous un même toit" peut conduire à des produits logiciels plus fiables et efficaces. En comprenant et en intégrant l'assurance qualité dans chaque étape du processus de développement, nous pouvons tous contribuer à la création de logiciels de haute qualité.

(done) Test Coverage et Code Coverage: Deux Concepts Clés en Informatique

January 18, 2024 10:35 AM

Introduction

Dans le domaine de l'informatique, **Test Coverage** et **Code Coverage** sont deux concepts souvent confondus. Bien qu'ils soient liés, ils mesurent des aspects différents de la qualité du code et des tests. Comprendre ces deux concepts est essentiel pour assurer la qualité du logiciel.

Test Coverage

Le **Test Coverage** mesure la quantité de code qui est testée par rapport à la quantité de code qui n'est pas testée. Il est généralement exprimé en pourcentage. Par exemple, si 80% de votre code est testé, votre Test Coverage est de 80%. Le Test Coverage peut être utilisé pour évaluer la qualité des tests effectués sur un logiciel. Un Test Coverage élevé signifie que la plupart de votre code a été testé, ce qui peut aider à identifier et à corriger les bugs avant la mise en production.

Code Coverage

Le **Code Coverage**, d'autre part, mesure la quantité de code qui est exécutée lors des tests par rapport à la quantité de code qui n'est pas exécutée. Tout comme le Test Coverage, le Code Coverage est également exprimé en pourcentage. Un Code Coverage élevé signifie que la plupart de votre code est exécutée lors des tests, ce qui peut aider à assurer que votre code fonctionne comme prévu.

Conclusion

Bien que le Test Coverage et le Code Coverage soient deux concepts différents, ils sont tous deux essentiels pour assurer la qualité du code et des tests. En comprenant ces deux concepts, vous pouvez améliorer la qualité de votre code et de vos tests, ce qui peut conduire à un logiciel plus fiable et plus efficace.

(done) Unbiased procedures

January 18, 2024 10:36 AM

Les procédures impartiales dans la qualité logicielle

Dans le domaine de la qualité logicielle, l'importance des procédures impartiales ne peut être sous-estimée. Ces procédures garantissent que tous les aspects du logiciel sont testés de manière équitable et complète, sans préjugés ou favoritisme.

Qu'est-ce qu'une procédure impartiale ?

Une procédure impartiale est une méthode de test qui est libre de tout parti pris ou préférence. Elle vise à évaluer le logiciel sur la base de critères objectifs et mesurables, plutôt que sur des opinions ou des préférences personnelles.

Pourquoi les procédures impartiales sont-elles importantes ?

Les procédures impartiales sont essentielles pour assurer la qualité d'un logiciel. Elles permettent de détecter les défauts et les problèmes potentiels de manière objective, ce qui peut aider à améliorer la fiabilité et la performance du logiciel. De plus, elles contribuent à instaurer la confiance chez les utilisateurs, car ils peuvent être assurés que le logiciel a été testé de manière équitable et exhaustive.

Comment mettre en œuvre des procédures impartiales ?

La mise en œuvre de procédures impartiales nécessite une planification et une organisation minutieuses. Voici quelques étapes clés à suivre :

1. **Définir les critères de test** : Les critères de test doivent être clairement définis et basés sur des mesures objectives. Ils peuvent inclure des aspects tels que la performance, la fiabilité, la sécurité, et l'interopérabilité.
2. **Établir un plan de test** : Un plan de test détaillé doit être établi, décrivant les étapes à suivre pour chaque test. Le plan de test doit également préciser les ressources nécessaires, les délais, et les responsabilités de chaque membre de l'équipe.
3. **Exécuter les tests** : Les tests doivent être exécutés selon le plan de test, en veillant à respecter les critères de test définis. Tous les résultats des tests doivent être enregistrés de manière précise et détaillée.
4. **Analyser les résultats** : Les résultats des tests doivent être analysés pour identifier les défauts et les problèmes potentiels. Toute anomalie doit être documentée et communiquée à l'équipe de développement pour correction.
5. **Répéter le processus** : Le processus de test doit être répété régulièrement tout au long du cycle de développement du logiciel. Cela permet de s'assurer que tous les défauts sont détectés et corrigés de manière opportune.

En conclusion, les procédures impartiales jouent un rôle crucial dans l'assurance de la qualité logicielle. En mettant en œuvre ces procédures, les organisations peuvent améliorer la qualité de leurs logiciels, augmenter la satisfaction des utilisateurs, et renforcer leur réputation en matière de fiabilité et de performance.

(done) Smart Testing

January 18, 2024 10:36 AM

Comprendre le Smart Testing

Le Smart Testing est une approche de test logiciel qui se base sur l'analyse intelligente et l'automatisation. Il s'agit d'évaluer les tests en fonction de divers facteurs tels que la complexité du logiciel, la criticité pour l'entreprise, la fréquence d'utilisation, les zones possibles avec des défauts, etc. Le Smart Testing priorise le test des fonctionnalités et des fonctions de l'application logicielle qui sont plus impactantes et susceptibles d'avoir des défauts.

Quand mettre en œuvre le Smart Testing ?

Le Smart Testing peut être mis en œuvre dans des projets ayant des contraintes de temps, de ressources, de budget, etc. Les projets où l'analyse basée sur le risque peut être utilisée pour détecter les vulnérabilités aux attaques par injection SQL. Les tests de sécurité dans les environnements de Cloud Computing. Les nouveaux projets avec des facteurs de risque élevés comme le manque d'expérience avec les technologies utilisées, le manque de connaissance du domaine d'activité.

Processus de gestion des tests intelligents

Comprendre les étapes impliquées dans le processus de gestion des tests intelligents est crucial. L'identification des tests peut se faire par le biais d'ateliers de tests, de listes de contrôle, de brainstorming, d'entretiens, de la technique Delphi, de diagrammes de cause à effet, de leçons tirées de projets précédents, d'analyses de causes profondes, de contacts avec des experts du domaine et des experts en la matière.

Avantages du Smart Testing

Le Smart Testing offre plusieurs avantages. Il améliore la productivité et réduit les coûts. Il améliore l'opportunité de marché (Time to market) et la livraison à temps. Il améliore les performances du service. Il améliore la qualité car toutes les fonctions critiques de l'application sont testées. Il donne des informations claires sur la couverture des tests.

En conclusion, le Smart Testing est une approche efficace pour améliorer la qualité et l'efficacité du développement. En intégrant l'analyse et la validation à chaque étape du cycle, il est possible de détecter et de résoudre les problèmes plus tôt, ce qui peut conduire à un produit de meilleure qualité et à une plus grande satisfaction des utilisateurs.

(done) RISK-based testing

January 18, 2024 10:36 AM

Comprendre le RISK-based testing

Le RISK-based testing est une approche de test logiciel qui se base sur la probabilité de risque. Il s'agit d'évaluer le risque en fonction de la complexité du logiciel, de la criticité pour l'entreprise, de la fréquence d'utilisation, des zones possibles avec des défauts, etc. Le RISK-based testing priorise le test des fonctionnalités et des fonctions de l'application logicielle qui sont plus impactantes et susceptibles d'avoir des défauts.

Quand mettre en œuvre le RISK-based testing ?

Le RISK-based testing peut être mis en œuvre dans des projets ayant des contraintes de temps, de ressources, de budget, etc. Les projets où l'analyse basée sur le risque peut être utilisée pour détecter les vulnérabilités aux attaques par injection SQL. Les tests de sécurité dans les environnements de Cloud Computing. Les nouveaux projets avec des facteurs de risque élevés comme le manque d'expérience avec les technologies utilisées, le manque de connaissance du domaine d'activité.

Processus de gestion des risques

Comprendre les étapes impliquées dans le processus de gestion des risques est crucial. L'identification des risques peut se faire par le biais d'ateliers de risques, de listes de contrôle, de brainstorming, d'entretiens, de la technique Delphi, de diagrammes de cause à effet, de leçons tirées de projets précédents, d'analyses de causes profondes, de contacts avec des experts du domaine et des experts en la matière.

Avantages du RISK-based testing

Le RISK-based testing offre plusieurs avantages. Il améliore la productivité et réduit les coûts. Il améliore l'opportunité de marché (Time to market) et la livraison à temps. Il améliore les performances du service. Il améliore la qualité car toutes les fonctions critiques de l'application sont testées. Il donne des informations claires sur la couverture des tests.

En conclusion, le RISK-based testing est une approche efficace pour améliorer la qualité et l'efficacité du développement. En intégrant l'analyse et la validation à chaque étape du cycle, il est possible de détecter et de résoudre les problèmes plus tôt, ce qui peut conduire à un produit de meilleure qualité et à une plus grande satisfaction des utilisateurs.

(done) Assurer la qualité du code legacy de façon moderne et efficace

January 18, 2024 10:36 AM

Le code legacy, ou “ancien code”, est un défi majeur pour de nombreuses équipes de développement. Il s’agit d’un code qui ne fait généralement plus l’objet d’un développement actif et qui peut être difficile à comprendre et à modifier. Cependant, il est souvent essentiel pour le fonctionnement des systèmes existants. Dans cet article, nous allons explorer comment assurer la qualité du code legacy de manière moderne et efficace, sans nécessairement modifier le code lui-même.

Les défis du code legacy

Le code legacy présente plusieurs défis. Il peut être difficile à comprendre, surtout si les développeurs qui l’ont écrit ne sont plus disponibles. De plus, il peut être difficile à tester et à maintenir. Enfin, il peut être difficile à intégrer avec les nouvelles technologies.

Assurer la qualité du code legacy

Malgré ces défis, il est possible d’assurer la qualité du code legacy de manière moderne et efficace. Voici quelques stratégies :

1. **Tests de caractérisation** : Commencez par créer des tests de caractérisation pour comprendre le comportement actuel du système. Ces tests serviront de filet de sécurité pour les futurs changements. Les tests de caractérisation, également connus sous le nom de “Golden Master Testing”, sont un moyen de décrire (caractériser) le comportement réel d’un logiciel existant, et donc de protéger le comportement existant du code legacy contre les changements involontaires grâce aux tests automatisés.
2. **Tests unitaires** : Écrivez des tests unitaires pour les composants individuels du système. Cela aidera à identifier et à isoler les problèmes.
3. **Construction du système** : Assurez-vous que votre construction du système et tous les tests existants fonctionnent correctement avant de faire des modifications.
4. **Refactoring et injection de dépendances** : Refactorisez votre code pour le rendre plus testable. Utilisez l’injection de dépendances pour isoler les dépendances et les rendre plus faciles à simuler dans les tests.
5. **Cadres de simulation** : Utilisez des cadres avancés pour simuler les dépendances non injectées.
6. **Examen de la documentation** : Examinez la documentation existante pour comprendre le comportement prévu du système.
7. **Réécriture du code** : Ne réécrivez le code que lorsque c’est absolument nécessaire. La réécriture peut introduire de nouveaux bugs et doit être évitée si possible.
8. **Refactoring** : Essayez de refactoriser le code pour améliorer sa structure et sa lisibilité sans en changer le comportement.
9. **Cycles de révision** : Faites des modifications dans différents cycles de révision. Cela permet un meilleur suivi des modifications et facilite le débogage.
10. **Collaboration** : Collaborez avec d’autres développeurs. Ils pourraient avoir des idées ou de l’expérience qui peuvent aider.
11. **Nouveau code propre** : Tout nouveau code ajouté doit être propre et suivre de bonnes pratiques de codage.
12. **Recherche** : Faites des recherches supplémentaires sur les technologies et les cadres spécifiques utilisés dans le code legacy. Cela peut fournir des informations précieuses sur la façon de le tester efficacement.

Investir dans une API pour découper le front-end

Un autre aspect important à considérer lorsqu'on travaille avec du code legacy est l'investissement dans une API pour découper le front-end. Cela peut être un premier investissement crucial pour faciliter les tests et assurer la qualité.

En ayant une API clairement définie, vous pouvez séparer les préoccupations entre le front-end et le back-end de votre application. Cela signifie que vous pouvez tester le back-end indépendamment du front-end, ce qui peut rendre les tests plus gérables et plus efficaces.

De plus, en découplant le front-end, vous pouvez également rendre votre code plus modulaire et plus facile à maintenir. Cela peut également faciliter l'intégration de nouvelles technologies ou l'ajout de nouvelles fonctionnalités à l'avenir.

Il est important de noter que la mise en place d'une API pour découper le front-end peut nécessiter un investissement initial en termes de temps et de ressources. Cependant, les avantages à long terme en termes de qualité du code et d'efficacité des tests peuvent largement compenser cet investissement initial.

L'automatisation des tests avec des outils de reconnaissance d'image

Un aspect crucial à considérer lorsqu'on travaille avec du code legacy est l'automatisation des tests. Cependant, tous les outils d'automatisation ne sont pas égaux, et choisir le bon outil peut faire la différence entre le succès et l'échec.

Lorsqu'il s'agit de tester du code legacy, il peut être tentant d'utiliser des outils d'automatisation basés sur le code. Cependant, ces outils peuvent souvent se révéler plus problématiques qu'ils n'en valent la peine. Ils peuvent nécessiter une compréhension approfondie du code sous-jacent, ce qui peut être difficile avec du code legacy. De plus, ils peuvent introduire leurs propres bugs et complexités, ce qui peut finalement entraîner plus de problèmes qu'ils n'en résolvent.

C'est là qu'interviennent les outils de reconnaissance d'image. Contrairement aux outils d'automatisation basés sur le code, ces outils utilisent l'intelligence artificielle pour "voir" et interagir avec l'interface utilisateur de la même manière qu'un utilisateur humain. Cela peut être particulièrement utile pour tester des applications avec une interface utilisateur complexe, où l'écriture de tests automatisés basés sur le code peut être difficile.

De plus, comme ces outils utilisent la reconnaissance d'image, ils peuvent tester l'application telle qu'elle apparaît à l'utilisateur, plutôt que de se baser sur le code sous-jacent. Cela peut permettre de détecter des problèmes qui pourraient être manqués par des tests basés sur le code, tels que des problèmes d'interface utilisateur ou des problèmes de performance.

En conclusion

bien que le code legacy présente de nombreux défis, il est possible d'assurer sa qualité de manière moderne et efficace. En utilisant des techniques comme l'automatisation, les tests basés sur l'image, l'intégration continue et le refactoring progressif ou pas, il est possible de maintenir et d'améliorer la qualité du code legacy sans nécessairement tout changer, en autant que les changements soient judicieux.

(done) Pourquoi autant de mots pour un bug, car un bug c'est un bug !?

January 18, 2024 10:36 AM

Bug, bogue, défaut, anomalie, incident, faille, fail, erreur, glitch, dysfonctionnement, échec, bloque, obstrué, démantelé, dé configuré, lacune, omission, typo, non-conforme, non respect de la demande ou des critères, corruption, altération, pollution, scratch, bad, deadlock, intrusion, virus, contamination, faille, backdoor, bombe, malfunction, infraction, non standard, non uniforme, non conventionnel, non ux, non ui, mauvaise traduction, fraude, hack, mouchard, non accessibilité, non compatible, ticket/billet support, ...

Pourquoi autant de mots pour un bug, car un bug c’est un bug !?

Dans le monde de l’informatique, un bug est un terme couramment utilisé pour désigner un comportement inattendu d’un logiciel. Cependant, il existe une multitude de termes pour décrire ce que nous appelons communément un “bug”. Pourquoi cela ?

La diversité des bugs

Chaque terme que vous avez mentionné - bogue, défaut, anomalie, incident, faille, fail, erreur, glitch, dysfonctionnement, échec, bloque, obstrué, démantelé, dé configuré, lacune, omission, typo, non-conforme, non respect de la demande ou des critères, corruption, altération, pollution, scratch, bad, deadlock, intrusion, virus, contamination, faille, backdoor, bombe, malfunction, infraction, non standard, non uniforme, non conventionnel, non ux, non ui, mauvaise traduction, fraude, hack, mouchard, non accessibilité, non compatible, ticket/billet support - représente une nuance spécifique du concept de bug.

Par exemple, une “faille” peut être une vulnérabilité dans un logiciel qui peut être exploitée pour causer un préjudice, tandis qu’un “glitch” peut être un défaut mineur qui affecte l’apparence mais pas la fonctionnalité. Un “non-conforme” peut être un logiciel qui ne respecte pas les spécifications ou les normes établies.

Pourquoi tant de termes ?

La raison pour laquelle il existe tant de termes pour décrire un bug est due à la complexité et à la diversité des logiciels eux-mêmes. Chaque logiciel est unique et peut présenter des problèmes uniques qui nécessitent une terminologie spécifique pour être correctement identifiés et traités.

En fin de compte, bien que tous ces termes puissent être regroupés sous la catégorie générale de “bugs”, chaque terme offre une précision qui aide les développeurs et les utilisateurs à identifier, comprendre et résoudre les problèmes spécifiques qu’ils rencontrent.

Voici une liste de différents types de bugs, comment et pourquoi les utiliser et les catégoriser, ainsi qu’un exemple pour chacun :

1. **Défaut** : Une fonctionnalité qui ne répond pas aux exigences spécifiées. Exemple : Une fonctionnalité qui ne répond pas aux exigences du client. Il est utilisé pour décrire une fonctionnalité qui ne répond pas aux exigences spécifiées.
2. **Anomalie** : Un comportement qui n’était pas prévu par le concepteur du logiciel. Exemple : Un logiciel qui se bloque lorsqu’une certaine action est effectuée. Il est utilisé pour décrire un comportement qui n’était pas prévu par le concepteur du logiciel.
3. **Incident** : Un problème qui a un impact sur l’utilisateur final. Exemple : Un logiciel qui se bloque et empêche l’utilisateur de travailler. Il est utilisé pour décrire un problème qui a un impact sur l’utilisateur final.
4. **Faille** : Une vulnérabilité dans un logiciel qui peut être exploitée pour causer un préjudice.

Exemple : Une faille de sécurité qui permet à un attaquant d'accéder à des informations sensibles. Il est utilisé pour décrire une vulnérabilité dans un logiciel qui peut être exploitée pour causer un préjudice.

5. **Fail** : Un échec d'une fonctionnalité ou d'un processus. Exemple : Un processus qui échoue et entraîne une perte de données. Il est utilisé pour décrire un échec d'une fonctionnalité ou d'un processus.
6. **Erreur** : Une erreur de programmation. Exemple : Une erreur de syntaxe dans le code. Il est utilisé pour décrire une erreur de programmation.
7. **Glitch** : Un défaut mineur qui affecte l'apparence mais pas la fonctionnalité. Exemple : Un élément graphique qui ne s'affiche pas correctement. Il est utilisé pour décrire un défaut mineur qui affecte l'apparence mais pas la fonctionnalité.
8. **Dysfonctionnement** : Un problème qui empêche un logiciel de fonctionner correctement. Exemple : Un logiciel qui se bloque fréquemment. Il est utilisé pour décrire un problème qui empêche un logiciel de fonctionner correctement.
9. **Échec** : Un problème qui empêche un logiciel de remplir sa fonction. Exemple : Un logiciel de traitement de texte qui ne peut pas enregistrer un document. Il est utilisé pour décrire un problème qui empêche un logiciel de remplir sa fonction.
10. **Bloque** : Un problème qui empêche un utilisateur d'effectuer une action spécifique. Exemple : Un logiciel qui se bloque lorsqu'un utilisateur tente de télécharger un fichier. Il est utilisé pour décrire un problème qui empêche un utilisateur d'effectuer une action spécifique.
11. **Obstrué** : Un problème qui empêche un logiciel de fonctionner correctement. Exemple : Un logiciel qui se bloque lorsqu'un utilisateur tente de télécharger un fichier. Il est utilisé pour décrire un problème qui empêche un logiciel de fonctionner correctement.
12. **Démantelé** : Un logiciel qui a été décomposé ou démonté. Exemple : Un logiciel qui a été démonté pour l'analyse ou le débogage. Il est utilisé pour décrire un logiciel qui a été décomposé ou démonté.
13. **Dé configuré** : Un logiciel qui a été mal configuré. Exemple : Un logiciel qui ne fonctionne pas correctement en raison d'une mauvaise configuration. Il est utilisé pour décrire un logiciel qui a été mal configuré.
14. **Lacune** : Un manque ou une omission dans un logiciel. Exemple : Un logiciel qui manque d'une fonctionnalité importante. Il est utilisé pour décrire un manque ou une omission dans un logiciel.
15. **Omission** : Un élément qui a été oublié ou omis. Exemple : Une fonctionnalité qui a été oubliée lors de la conception d'un logiciel. Il est utilisé pour décrire un élément qui a été oublié ou omis.
16. **Typo** : Une erreur de frappe ou de grammaire. Exemple : Une erreur de frappe dans le code ou dans l'interface utilisateur d'un logiciel. Il est utilisé pour décrire une erreur de frappe ou de grammaire.
17. **Non-conforme** : Un logiciel qui ne respecte pas les spécifications ou les normes établies. Exemple : Un logiciel qui ne respecte pas les normes de l'industrie. Il est utilisé pour décrire un logiciel qui ne respecte pas les spécifications ou les normes établies.
18. **Non respect de la demande ou des critères** : Un logiciel qui ne répond pas aux exigences ou aux critères spécifiés. Exemple : Un logiciel qui ne répond pas aux exigences du client. Il est utilisé pour décrire un logiciel qui ne répond pas aux exigences ou aux critères spécifiés.
19. **Corruption** : Un problème qui cause la corruption des données. Exemple : Un logiciel qui corrompt les données lorsqu'il est utilisé. Il est utilisé pour décrire un problème qui cause la corruption des données.
20. **Altération** : Un changement non autorisé ou indésirable. Exemple : Un logiciel qui a été modifié sans autorisation. Il est utilisé pour décrire un changement non autorisé ou indésirable.
21. **Pollution** : Un problème qui cause la corruption des données. Exemple : Un logiciel qui

corrompt les données lorsqu'il est utilisé. Il est utilisé pour décrire un problème qui cause la corruption des données.

22. **Scratch** : Un problème qui cause la perte de données. Exemple : Un logiciel qui efface les données lorsqu'il est utilisé. Il est utilisé pour décrire un problème qui cause la perte de données.
23. **Bad** : Un problème qui cause un comportement indésirable. Exemple : Un logiciel qui se comporte mal lorsqu'il est utilisé. Il est utilisé pour décrire un problème qui cause un comportement indésirable.
24. **Deadlock** : Un état dans lequel deux ou plusieurs processus sont incapables de progresser parce que chacun attend que l'autre libère une ressource. Exemple : Deux processus qui attendent chacun que l'autre libère une ressource, ce qui entraîne un blocage. Il est utilisé pour décrire un état dans lequel deux ou plusieurs processus sont incapables de progresser.
25. **Intrusion** : Un accès non autorisé à un logiciel. Exemple : Un attaquant qui accède à un logiciel sans autorisation. Il est utilisé pour décrire un accès non autorisé à un logiciel.
26. **Virus** : Un logiciel malveillant qui peut causer des dommages. Exemple : Un logiciel qui est infecté par un virus. Il est utilisé pour décrire un logiciel malveillant qui peut causer des dommages.
27. **Contamination** : Un problème qui cause la corruption des données. Exemple : Un logiciel qui corrompt les données lorsqu'il est utilisé. Il est utilisé pour décrire un problème qui cause la corruption des données.
28. **Backdoor** : Un moyen d'accéder à un logiciel qui contourne les mesures de sécurité normales. Exemple : Un attaquant qui utilise une porte dérobée pour accéder à un logiciel. Il est utilisé pour décrire un moyen d'accéder à un logiciel qui contourne les mesures de sécurité normales.
29. **Bombe** : Un logiciel qui a été programmé pour causer des dommages à une date ou à un moment spécifique. Exemple : Un logiciel qui a été programmé pour effacer toutes les données à une date spécifique. Il est utilisé pour décrire un logiciel qui a été programmé pour causer des dommages à une date ou à un moment spécifique.
30. **Malfunction** : Un problème qui empêche un logiciel de fonctionner correctement. Exemple : Un logiciel qui se bloque fréquemment. Il est utilisé pour décrire un problème qui empêche un logiciel de fonctionner correctement.
31. **Infraction** : Un comportement qui viole les règles ou les politiques. Exemple : Un logiciel qui viole les politiques de confidentialité. Il est utilisé pour décrire un comportement qui viole les règles ou les politiques.
32. **Non standard** : Un logiciel qui ne suit pas les normes ou les standards établis. Exemple : Un logiciel qui ne suit pas les normes de l'industrie. Il est utilisé pour décrire un logiciel qui ne suit pas les normes ou les standards établis.
33. **Non uniforme** : Un comportement qui n'est pas cohérent ou uniforme. Exemple : Un logiciel qui se comporte de manière différente sur différents systèmes d'exploitation. Il est utilisé pour décrire un comportement qui n'est pas cohérent ou uniforme.
34. **Non conventionnel** : Un logiciel qui ne suit pas les conventions ou les pratiques courantes. Exemple : Un logiciel qui utilise une interface utilisateur non conventionnelle. Il est utilisé pour décrire un logiciel qui ne suit pas les conventions ou les pratiques courantes.
35. **Non UX** : Un logiciel qui ne suit pas les meilleures pratiques en matière d'expérience utilisateur. Exemple : Un logiciel qui a une interface utilisateur difficile à utiliser. Il est utilisé pour décrire un logiciel qui ne suit pas les meilleures pratiques en matière d'expérience utilisateur.
36. **Non UI** : Un logiciel qui ne suit pas les meilleures pratiques en matière d'interface utilisateur. Exemple : Un logiciel qui a une interface utilisateur mal conçue. Il est utilisé pour décrire un logiciel qui ne suit pas les meilleures pratiques en matière d'interface utilisateur.
37. **Mauvaise traduction** : Un logiciel qui a été mal traduit. Exemple : Un logiciel qui a des erreurs de traduction dans son interface utilisateur. Il est utilisé pour décrire un logiciel qui a

été mal traduit.

38. **Fraude** : Un logiciel qui a été utilisé pour commettre une fraude. Exemple : Un logiciel qui a été utilisé pour voler des informations sensibles. Il est utilisé pour décrire un logiciel qui a été utilisé pour commettre une fraude.
39. **Hack** : Un logiciel qui a été piraté. Exemple : Un logiciel qui a été modifié par un attaquant. Il est utilisé pour décrire un logiciel qui a été piraté.
40. **Mouchard** : Un logiciel qui surveille et enregistre les activités de l'utilisateur. Exemple : Un logiciel qui enregistre les frappes de l'utilisateur. Il est utilisé pour décrire un logiciel qui surveille et enregistre les activités de l'utilisateur.
41. **Non accessibilité** : Un logiciel qui n'est pas accessible à certains utilisateurs. Exemple : Un logiciel qui n'est pas accessible aux utilisateurs malvoyants. Il est utilisé pour décrire un logiciel qui n'est pas accessible à certains utilisateurs.
42. **Non compatible** : Un logiciel qui n'est pas compatible avec certains systèmes ou appareils. Exemple : Un logiciel qui ne fonctionne pas sur certains systèmes d'exploitation. Il est utilisé pour décrire un logiciel qui n'est pas compatible avec certains systèmes ou appareils.
43. **Ticket/Billet support** : Un problème signalé par un utilisateur qui nécessite une assistance. Exemple : Un utilisateur qui signale un problème avec un logiciel et demande de l'aide. Il est utilisé pour décrire un problème signalé par un utilisateur qui nécessite une assistance.
44. **Vulnérabilité** : Une faiblesse dans un logiciel qui peut être exploitée pour causer un préjudice. Exemple : Une vulnérabilité de sécurité dans un logiciel qui permet à un attaquant d'accéder à des informations sensibles. Il est utilisé pour décrire une faiblesse dans un logiciel qui peut être exploitée pour causer un préjudice.
45. **Dépréciation** : Un logiciel ou une fonctionnalité qui n'est plus pris en charge. Exemple : Une fonctionnalité qui a été dépréciée dans la dernière version d'un logiciel. Il est utilisé pour décrire un logiciel ou une fonctionnalité qui n'est plus pris en charge.
46. **Incompatibilité** : Un logiciel qui n'est pas compatible avec d'autres logiciels ou systèmes. Exemple : Un logiciel qui ne fonctionne pas correctement avec un autre logiciel. Il est utilisé pour décrire un logiciel qui n'est pas compatible avec d'autres logiciels ou systèmes.
47. **Performance** : Un logiciel qui ne fonctionne pas à un niveau de performance acceptable. Exemple : Un logiciel qui est lent ou qui se bloque fréquemment. Il est utilisé pour décrire un logiciel qui ne fonctionne pas à un niveau de performance acceptable.
48. **Problème de sécurité** : Un logiciel qui présente un risque pour la sécurité. Exemple : Un logiciel qui ne chiffre pas correctement les données sensibles. Il est utilisé pour décrire un logiciel qui présente un risque pour la sécurité.
49. **Problème de confidentialité** : Un logiciel qui ne protège pas correctement les informations privées. Exemple : Un logiciel qui partage des informations privées sans le consentement de l'utilisateur. Il est utilisé pour décrire un logiciel qui ne protège pas correctement les informations privées.
50. **Problème de compatibilité** : Un logiciel qui ne fonctionne pas correctement avec certains systèmes ou appareils. Exemple : Un logiciel qui ne fonctionne pas correctement sur certains systèmes d'exploitation. Il est utilisé pour décrire un logiciel qui ne fonctionne pas correctement avec certains systèmes ou appareils.
51. **Problème de synchronisation** : Un problème qui se produit lorsque plusieurs processus ne sont pas correctement synchronisés. Exemple : Un logiciel qui se bloque lorsque plusieurs utilisateurs tentent d'accéder à la même ressource en même temps.
52. **Problème de mémoire** : Un problème qui se produit lorsque le logiciel utilise trop de mémoire ou ne libère pas correctement la mémoire. Exemple : Un logiciel qui se bloque ou ralentit en raison d'une utilisation excessive de la mémoire.
53. **Problème de réseau** : Un problème qui se produit lors de la communication entre le logiciel et le réseau. Exemple : Un logiciel qui ne peut pas se connecter à Internet en raison d'un problème de configuration du réseau.
54. **Problème de base de données** : Un problème qui se produit lors de l'interaction du logiciel

avec une base de données. Exemple : Un logiciel qui ne peut pas récupérer les données de la base de données en raison d'une requête SQL mal formée.

55. **Problème de latence** : Un problème qui se produit lorsque le logiciel prend trop de temps pour répondre. Exemple : Un logiciel de streaming vidéo qui met du temps à charger les vidéos.
56. **Problème de chargement** : Un problème qui se produit lorsque le logiciel a du mal à charger des données ou des ressources. Exemple : Un jeu vidéo qui a du mal à charger les graphiques.
57. **Problème de rendu** : Un problème qui se produit lorsque le logiciel a du mal à afficher correctement les graphiques ou les interfaces utilisateur. Exemple : Un logiciel de modélisation 3D qui affiche des modèles de manière incorrecte.
58. **Problème de script** : Un problème qui se produit lorsque le logiciel a du mal à exécuter des scripts ou des commandes. Exemple : Un logiciel de traitement de texte qui ne peut pas exécuter une macro.
59. **Problème de compatibilité de version** : Un problème qui se produit lorsque le logiciel n'est pas compatible avec certaines versions d'un système d'exploitation ou d'un autre logiciel. Exemple : Un logiciel qui ne fonctionne pas correctement avec la dernière version de Windows.
60. **Problème de déploiement** : Un problème qui se produit lors de l'installation ou de la mise à jour du logiciel. Exemple : Un logiciel qui échoue lors de l'installation.
61. **Problème de licence** : Un problème qui se produit lorsque le logiciel a des problèmes avec la gestion des licences. Exemple : Un logiciel qui ne peut pas valider une clé de licence.
62. **Problème de documentation** : Un problème qui se produit lorsque la documentation du logiciel est incomplète ou incorrecte. Exemple : Un logiciel dont le manuel d'utilisation ne décrit pas correctement toutes les fonctionnalités.
63. **Problème de test** : Un problème qui se produit lorsque le logiciel échoue lors des tests. Exemple : Un logiciel qui échoue lors des tests d'intégration.
64. **Problème de configuration** : Un problème qui se produit lorsque le logiciel a été mal configuré. Exemple : Un logiciel qui ne fonctionne pas correctement en raison d'une mauvaise configuration.
65. **Problème d'intégration** : Un problème qui se produit lorsque le logiciel a du mal à fonctionner avec d'autres systèmes ou logiciels. Exemple : Un logiciel qui ne fonctionne pas correctement avec une base de données spécifique.
66. **Problème de compatibilité matérielle** : Un problème qui se produit lorsque le logiciel n'est pas compatible avec certains matériels. Exemple : Un logiciel de jeu qui ne fonctionne pas correctement avec certaines cartes graphiques.
67. **Problème de ressources** : Un problème qui se produit lorsque le logiciel utilise trop de ressources système. Exemple : Un logiciel qui utilise une grande quantité de mémoire, ce qui ralentit le système.
68. **Problème de stabilité** : Un problème qui se produit lorsque le logiciel se bloque ou se ferme de manière inattendue. Exemple : Un logiciel qui se bloque fréquemment sans raison apparente.
69. **Problème de scalabilité** : Un problème qui se produit lorsque le logiciel a du mal à gérer une augmentation de la charge de travail. Exemple : Un logiciel de serveur web qui ralentit considérablement lorsque le nombre de demandes augmente.
70. **Problème de fiabilité** : Un problème qui se produit lorsque le logiciel ne fonctionne pas de manière fiable. Exemple : Un logiciel qui produit des résultats différents lorsqu'il est exécuté avec les mêmes entrées.
71. **Problème de réactivité** : Un problème qui se produit lorsque le logiciel met trop de temps à répondre aux actions de l'utilisateur. Exemple : Un logiciel qui met du temps à répondre lorsque l'utilisateur clique sur un bouton.
72. **Problème de convivialité** : Un problème qui se produit lorsque le logiciel est difficile à

utiliser. Exemple : Un logiciel dont l'interface utilisateur est complexe et difficile à comprendre.

73. **Problème de fonctionnalité** : Un problème qui se produit lorsque une fonctionnalité du logiciel ne fonctionne pas comme prévu. Exemple : Un logiciel de traitement de texte qui ne peut pas enregistrer un document au format PDF.
74. **Problème de sauvegarde/restauration** : Un problème qui se produit lors de la sauvegarde ou de la restauration de données. Exemple : Un logiciel de sauvegarde qui échoue lors de la restauration des données.
75. **Problème de mise à jour** : Un problème qui se produit lors de la mise à jour du logiciel. Exemple : Un logiciel qui échoue lors de la mise à jour vers la dernière version.
76. **Problème de multithreading** : Un problème qui se produit lors de l'exécution de plusieurs threads en parallèle. Exemple : Un logiciel qui se bloque lors de l'exécution de plusieurs threads en parallèle.
77. **Problème de localisation** : Un problème qui se produit lorsque le logiciel n'est pas correctement adapté à une région ou à une langue spécifique. Exemple : Un logiciel qui affiche des dates au format américain au lieu du format local.
78. **Problème de gestion des exceptions** : Un problème qui se produit lorsque le logiciel ne gère pas correctement les exceptions. Exemple : Un logiciel qui se bloque lorsqu'une exception est levée.
79. **Problème de concurrence** : Un problème qui se produit lorsque plusieurs processus tentent d'accéder à une ressource partagée en même temps. Exemple : Un logiciel de base de données qui se bloque lors de l'accès concurrent à une même table.
80. **Problème de dépassement de capacité** : Un problème qui se produit lorsque le logiciel tente d'utiliser plus de ressources que ce qui est disponible. Exemple : Un logiciel qui tente d'utiliser plus de mémoire que ce qui est disponible sur le système.
81. **Problème de fuite de mémoire** : Un problème qui se produit lorsque le logiciel utilise de la mémoire mais ne la libère pas correctement. Exemple : Un logiciel qui utilise de plus en plus de mémoire au fil du temps sans la libérer.
82. **Problème de débordement de tampon** : Un problème qui se produit lorsque le logiciel écrit des données en dehors de la mémoire tampon allouée. Exemple : Un logiciel qui se bloque ou qui se comporte de manière imprévisible en raison d'un débordement de tampon.
83. **Problème de sérialisation** : Un problème qui se produit lorsque le logiciel a du mal à convertir les structures de données en un format qui peut être stocké ou transmis. Exemple : Un logiciel qui échoue lors de la sérialisation d'un objet complexe en JSON.
84. **Problème de désérialisation** : Un problème qui se produit lorsque le logiciel a du mal à convertir les données sérialisées en structures de données. Exemple : Un logiciel qui échoue lors de la désérialisation d'une chaîne JSON en objet.
85. **Problème de codage/encodage** : Un problème qui se produit lorsque le logiciel a du mal à convertir les données d'un format à un autre. Exemple : Un logiciel qui échoue lors de la conversion d'une chaîne de caractères en UTF-8.
86. **Problème de décodage** : Un problème qui se produit lorsque le logiciel a du mal à convertir les données d'un format à un autre. Exemple : Un logiciel qui échoue lors de la conversion d'une chaîne de caractères en UTF-8.
87. **Problème de gestion des dépendances** : Un problème qui se produit lorsque le logiciel a du mal à gérer ses dépendances. Exemple : Un logiciel qui échoue lors de l'installation d'une dépendance requise.
88. **Problème de versionnement** : Un problème qui se produit lorsque le logiciel a du mal à gérer les différentes versions d'un logiciel ou d'une dépendance. Exemple : Un logiciel qui échoue lors de la mise à jour vers une nouvelle version.
89. **Problème de journalisation** : Un problème qui se produit lorsque le logiciel a du mal à enregistrer les événements ou les erreurs dans un journal. Exemple : Un logiciel qui ne parvient pas à enregistrer un événement important dans le journal des événements.

90. **Problème de persistance des données** : Un problème qui se produit lorsque le logiciel a du mal à stocker les données de manière persistante. Exemple : Un logiciel qui échoue lors de l'enregistrement des données dans une base de données.
91. **Problème de récupération des données** : Un problème qui se produit lorsque le logiciel a du mal à récupérer les données stockées. Exemple : Un logiciel qui échoue lors de la récupération des données d'une base de données.
92. **Problème de gestion des erreurs** : Un problème qui se produit lorsque le logiciel a du mal à gérer les erreurs ou les exceptions. Exemple : Un logiciel qui se bloque lorsqu'une exception est levée au lieu de la gérer correctement.
93. **Problème de latence réseau** : Un problème qui se produit lorsque le logiciel a du mal à communiquer sur le réseau en raison de retards. Exemple : Un logiciel de chat qui a du mal à envoyer des messages en temps réel en raison de la latence du réseau.
94. **Problème de cryptographie** : Un problème qui se produit lorsque le logiciel a du mal à chiffrer ou à déchiffrer les données. Exemple : Un logiciel de messagerie qui ne peut pas déchiffrer un message chiffré en raison d'une clé de chiffrement incorrecte.
95. **Problème de compression** : Un problème qui se produit lorsque le logiciel a du mal à compresser ou à décompresser les données. Exemple : Un logiciel qui échoue lors de la décompression d'un fichier compressé.
96. **Problème de pagination** : Un problème qui se produit lorsque le logiciel a du mal à diviser le contenu en pages distinctes. Exemple : Un logiciel de traitement de texte qui affiche incorrectement la pagination d'un document.
97. **Problème de routage** : Un problème qui se produit lorsque le logiciel a du mal à diriger les demandes vers la bonne destination. Exemple : Un logiciel de routage qui envoie les paquets de données à la mauvaise destination.
98. **Problème de session** : Un problème qui se produit lorsque le logiciel a du mal à gérer les sessions utilisateur. Exemple : Un logiciel qui ne parvient pas à maintenir la session d'un utilisateur connecté.
99. **Problème de cache** : Un problème qui se produit lorsque le logiciel a du mal à gérer le cache. Exemple : Un logiciel qui affiche des données obsolètes en raison d'un problème de cache.
100. **Problème de parallélisme** : Un problème qui se produit lorsque le logiciel a du mal à exécuter des tâches en parallèle. Exemple : Un logiciel qui ne parvient pas à exécuter efficacement plusieurs tâches en parallèle.
101. **Problème de réplication** : Un problème qui se produit lorsque le logiciel a du mal à copier les données d'un endroit à un autre. Exemple : Un logiciel de base de données qui échoue lors de la réplication des données sur un autre serveur.
102. **Problème de sauvegarde** : Un problème qui se produit lorsque le logiciel a du mal à sauvegarder les données. Exemple : Un logiciel qui échoue lors de la sauvegarde des données sur un disque externe.
103. ETC.

OUFF!!! imaginez, on pourrait continuer !!!!

Qu'est-ce qu'un Bug ?

Un **bug** est un terme générique qui englobe tous les types de problèmes informatiques. C'est le mot unique qui désigne peu importe le type de problème. Donc un bug, c'est un bug, peu importe quel est son type.

Définition d'un Bug

Dans le domaine de l'informatique, un bug est un défaut dans un programme informatique qui provoque un résultat inattendu ou non intentionnel. Le terme "bug" est souvent utilisé pour désigner tout type de problème dans un logiciel, qu'il s'agisse d'un simple défaut visuel ou d'un problème majeur qui empêche le logiciel de fonctionner correctement.

Pourquoi “Bug” ?

Le terme “bug” a été utilisé pour la première fois dans ce contexte par les pionniers de l’informatique au milieu du 20ème siècle. L’origine exacte du terme est incertaine, mais il est généralement accepté qu’il a été emprunté au langage courant où il signifie un insecte. En effet, le premier “bug” documenté était un véritable insecte qui avait causé un dysfonctionnement dans un ordinateur.

Un Bug est un Bug

Peu importe la nature du problème, qu’il s’agisse d’une faille de sécurité, d’une erreur de programmation, d’un problème de performance, d’une mauvaise conception de l’interface utilisateur ou de tout autre problème imaginable, dans le langage courant de l’informatique, nous appelons cela un “bug”.

C’est un terme pratique car il nous permet de parler de problèmes dans le logiciel sans avoir à spécifier exactement de quel type de problème il s’agit. Cela dit, comme nous l’avons vu précédemment, il existe de nombreux types spécifiques de bugs, chacun avec ses propres caractéristiques et défis.

L’importance de traiter tous les bugs comme étant un bug

Dans le monde de l’informatique, il est essentiel de traiter tous les bugs comme étant un bug, peu importe leur type. Pourquoi cela ?

Égalité des bugs

Tous les bugs, quels que soient leur nature ou leur gravité, peuvent avoir un impact sur l’expérience de l’utilisateur. Un petit bug d’affichage peut sembler insignifiant, mais il peut frustrer l’utilisateur tout autant qu’un bug majeur qui fait planter le système. Par conséquent, tous les bugs méritent notre attention et doivent être traités avec sérieux.

Un bug n’est pas une autre entité

Il est important de noter qu’un type de bug n’est pas une autre entité. Par exemple, dans un système de suivi des problèmes comme Jira, un “bug” peut être un type d’issue, mais un “bug” n’est pas un autre type d’issue comme une “histoire” ou une “tâche”. Chaque type de bug est toujours un “bug”, peu importe sa spécificité.

Priorisation des bugs

Cela ne signifie pas que tous les bugs doivent être corrigés immédiatement. Dans la pratique, les ressources sont limitées et il est nécessaire de prioriser les bugs en fonction de leur impact sur l’utilisateur et sur le système. Cependant, même les bugs de faible priorité doivent être suivis et corrigés éventuellement.

Amélioration continue

Traiter tous les bugs comme étant un bug encourage une culture d’amélioration continue. Chaque bug est une occasion d’apprendre et d’améliorer le logiciel. En reconnaissant et en traitant tous les bugs, nous pouvons continuellement améliorer la qualité de notre logiciel et l’expérience de nos utilisateurs.

En conclusion

un bug est un bug, peu importe son type. Chaque bug est une occasion d’apprendre, d’améliorer et de grandir. En traitant tous les bugs avec le sérieux qu’ils méritent, nous pouvons continuellement améliorer notre logiciel et offrir la meilleure expérience possible à nos utilisateurs.

un bug est un bug, peu importe quel est son type. C’est le mot unique qui désigne peu

importe le type de problème dans le monde de l’informatique.
Donc si on vous parle par exemple d’une anomalie, d’une vulnérabilité ou un incident… c’est un “bug“ et il doit être traité comme un “bug”.

(done) Les secrets pour donner de la visibilité à votre travail en assurance qualité

January 18, 2024 10:37 AM

Salut à tous ! Aujourd'hui, on va parler d'un sujet qui me tient à cœur : comment donner de la visibilité à votre travail en assurance qualité. Que vous soyez analyste QA, automaticien QA, contrôleur QA ou coach QA, ce blog est pour vous. Alors, accrochez-vous, ça va être une sacrée aventure !

1. L'analyste QA : Le détective du logiciel

En tant qu'analyste QA, votre rôle est de trouver les bugs avant qu'ils ne trouvent les utilisateurs. Pour donner de la visibilité à votre travail, pensez à documenter vos découvertes et à partager régulièrement des rapports avec votre équipe. Utilisez des outils de suivi des bugs pour enregistrer les problèmes que vous trouvez et montrez comment votre travail contribue à améliorer la qualité du logiciel.

2. L'automaticien QA : L'architecte des tests

En tant qu'automaticien QA, vous construisez des tests automatisés pour vérifier la qualité du logiciel. Pour donner de la visibilité à votre travail, partagez les résultats de vos tests avec votre équipe et montrez comment ils aident à prévenir les régressions. Utilisez des outils de visualisation pour montrer l'évolution de la qualité du logiciel au fil du temps.

3. Le contrôleur QA : Le gardien de la qualité

En tant que contrôleur QA, vous veillez à ce que les normes de qualité soient respectées. Pour donner de la visibilité à votre travail, partagez régulièrement des rapports sur l'état de la qualité du logiciel. Montrez comment votre travail aide à maintenir un niveau de qualité élevé et à prévenir les problèmes avant qu'ils ne surviennent.

4. Le coach QA : Le mentor de l'équipe

En tant que coach QA, vous aidez votre équipe à comprendre et à appliquer les meilleures pratiques d'assurance qualité. Pour donner de la visibilité à votre travail, partagez vos connaissances et vos expériences avec votre équipe. Montrez comment votre coaching aide à améliorer les compétences de l'équipe et à augmenter la qualité du logiciel.

Et voilà, c'est tout pour aujourd'hui ! J'espère que vous avez apprécié cette plongée dans le monde fascinant de l'assurance qualité. N'oubliez pas, comprendre comment donner de la visibilité à votre travail peut vous aider à valoriser votre contribution et à améliorer la qualité du logiciel.

(done) CARE un méthodologie qui fonctionne vraiment

January 18, 2024 10:37 AM

Bonjour à tous ! Aujourd'hui, nous allons parler d'un sujet passionnant : CARE, une méthodologie qui fonctionne vraiment. Que vous soyez familier avec le Total Quality Management (TQM) ou que vous cherchiez à faciliter la transformation et/ou l'implémentation de votre organisation, ce blog est pour vous. Alors, préparez-vous, nous allons faire un voyage fascinant !

1. Qu'est-ce que CARE ?

CARE (**C**ollaborative **A**gile **R**eliable **E**fficacy) est une méthodologie de gestion qui intègre SAFe, Kanban, XP+3amigos, tous pilotés par Lean. CARE est conçu pour dépasser l'efficacité du "scrum" traditionnel et peut être démarré quel que soit l'état actuel de votre organisation.

2. Les quatre piliers de CARE

Collaboratif (Collaborative)

CARE favorise la collaboration grâce à SAFe. Elle brise les silos organisationnels, favorisant une communication ouverte entre toutes les parties prenantes du projet. CARE s'efforce de créer un environnement qui facilite et valorise la collaboration à tous les niveaux de l'organisation. SAFe remplace le SCRUM à plus haut niveau pour une structure plus flexible et évolutive.

Agile

L'agilité est assurée par Kanban pour visualiser le flux de travail. CARE adopte également DevOps, CI/CD, les tests continus et le shift-left. Ces principes permettent une intégration et une livraison continues, contribuant à l'agilité de l'approche.

Fiable (Reliable)

La fiabilité est maintenue par XP, qui assure la qualité grâce au cœur de CARE où se trouve le développement piloté par la qualité (Quality-Driven Development, QDD). La qualité est présente à chaque phase pour la prévention des problèmes, la mise en place d'alertes et la planification de réactions rapides.

Efficace (Efficacy)

L'efficacité est obtenue grâce à Lean, qui guide le processus avec une mentalité d'amélioration continue. Lean se concentre sur l'élimination de tout ce qui est sans valeur ajoutée dans le processus de développement. En éliminant ces éléments, Lean permet d'accélérer le processus de développement et d'augmenter l'efficacité.

3. Les rôles clés dans CARE

Dans CARE, chaque rôle a une importance cruciale pour assurer la qualité :

- **VP QA (TQM)** : S'assure que l'organisation dans son ensemble cultive bien la pratique de la qualité.
- **Directeur QA** : S'assure que le département spécifique cultive bien la pratique de la qualité.
- **Manager QA** : S'assure que l'équipe SAFe (par produit) cultive bien la pratique de la qualité.
- **Superviseur QA** : S'assure que l'escouade Kanban cultive bien la pratique de la qualité.
- **Expert QA** : S'assure que l'unité XP3A cultive bien la pratique de la qualité.

Coach QA : Le joueur clé, comme dans le sport, pour atteindre la satisfaction. Il est là pour guider et entraîner tous les rôles et contributeurs QA. Il doit assurer la pérennité de la

transformation QA.

L’équipe SAFe : Le rassemblement des escouades Kanban

L’équipe SAFe remplace le scrum mais à l’échelle du produit et les à-côtés, donc le complet. C’est le rassemblement des escouades Kanban. Avec des cérémonies comme le scrum mais plus espacées (monthly meeting).

L’escouade Kanban : Le rassemblement de tous les membres de livraison

L’escouade Kanban est l’équipe complète pour toutes les phases du cycle de vie DevOps. Elle se réunit régulièrement pour discuter de l’avancement du projet (weekly meeting). L’escouade s’assure de passer dans toutes les phases du cycle de vie et d’analyser le retour de la boucle DevOps. Cette équipe inclus les devs, les ops et l’unité XP3A.

L’unité XP3A : Le cœur, le cerveau et l'âme de l’équipe

L’unité XP3A est composée uniquement des représentants, donc l’expert Affaire, de l’expert Dev et de l’expert QA. Ils se réunissent quotidiennement pour discuter de l’avancement des tâches Kanban et s’assure qu’il rien ne bloque (Daily meeting).

XP3A est un mélange des 3 amigos et de XP. Le but est de définir et automatiser le plus possible tous les tests pendant la définition du besoin et de la solution pour ensuite piloter le reste du cycle de développement par ces tests. C’est du jumelage à trois plutôt qu’à deux comme le XP conventionnel.

Conclusion

Et voilà, c’est tout pour aujourd’hui ! J’espère que vous avez apprécié cette plongée dans le monde fascinant de CARE. N’oubliez pas, comprendre comment CARE fonctionne peut vous aider à améliorer la qualité de votre logiciel. Alors, à vos claviers, et que le meilleur code gagne !

(done) L’ère de l’intelligence artificiel (AI) : Un avenir prometteur pour l’assurance qualité

January 18, 2024 10:37 AM

Dans le monde dynamique de la technologie, l’intelligence artificiel est en train de révolutionner tous les aspects de notre vie, y compris l’assurance qualité. L’unité XP3A, combinée à l’intelligence artificiel, promet de remplacer une équipe de production à grande échelle. Comment est-ce possible? Jetons un coup d’œil.

L’unité XP3A : Le cœur, le cerveau et l’âme

L’unité XP3A est composée uniquement des représentants, donc l’expert Affaire, de l’expert Dev (designer) et de l’expert QA. C’est du jumelage à trois (amigos) plutôt qu’à deux comme le XP conventionnel.

L’intelligence artificiel et l’assurance qualité

Le travail se situe principalement dans la partie de faire les prompts intelligence artificiel selon la définition du besoin et solution pour faire générer le modèle de l’application avec l’automatisation des tests intégrés. Par la suite, l’intelligence artificiel continue à faire l’intégration selon le modèle et déploie l’application de qualité car nous avons assuré la qualité au début dans les prompts.

En conclusion

L’intelligence artificiel, combinée à l’unité XP3A, a le potentiel de transformer radicalement l’assurance qualité. En automatisant les tests et en intégrant l’intelligence artificiel dès le début, nous pouvons assurer une qualité supérieure et une efficacité accrue. L’avenir de l’assurance qualité est prometteur grâce à l’intelligence artificiel.
Ce sera bon pour les experts analytiques, marketing, designer, etc.

Nous serons au cœur du client/utilisateur, sur les tactiques commerciales et envelopper de la vision de l'entreprise.

Bon le côté moins lumineux qui devra être recyclé est pour les profils de développeurs traditionnels, les contributeurs Agile, les gestionnaires, les autres stakeholders, etc.



(done) Vérifier la conformité de l'organisation aux standards internationales ISO

January 18, 2024 10:37 AM

Un guide pour l’excellence et la conformité dans le domaine du logiciel ?

L’Organisation Internationale de Normalisation (ISO) est un phare dans le monde des affaires, guidant les entreprises vers la standardisation, la régulation et l’assurance qualité. Mais qu’est-ce que l’ISO exactement, et pourquoi est-elle si importante ? Plongeons dans le monde fascinant de l’ISO !

L’ISO : Un organisme de normalisation mondial

L’ISO est un organisme indépendant et international qui développe des réglementations pour aider diverses entités à satisfaire les demandes des consommateurs. Il fournit essentiellement une formule pour faire certaines choses. Les normes internationales assurent la sécurité, l’efficacité et la qualité des produits, des systèmes et des services.

Un monde de normes

Avec plus de 19 000 types de normes et 1 000 nouvelles normes développées chaque année, l’ISO couvre un large éventail de secteurs. Voici quelques-unes des normes ISO les plus courantes :

- **ISO 9000 / 9001** – Systèmes de gestion de la qualité
- **ISO/IEC 27000** – Technologie de l’information
- **ISO 14000** – Systèmes de gestion environnementale
- **ISO 22000** – Normes de sécurité alimentaire
- **ISO 45001** – Santé et sécurité
- **ISO 50001** – Gestion de l’énergie
- **ISO 26000** – Responsabilité sociale
- **ISO 31000** – Gestion des risques
- **ISO 13485** – Dispositifs médicaux
- **ISO 37001** - Système de gestion anti-corruption

L’ISO dans le domaine du logiciel

Dans le domaine du logiciel, l’ISO a développé plusieurs normes pour assurer la qualité et l’efficacité des processus de développement de logiciels. Par exemple, l’ISO/IEC 9126, maintenant remplacée par l’ISO 25010, définit un modèle de qualité logicielle. De plus, l’ISO/IEC 5055:2021 est une norme ISO pour mesurer la structure interne d’un produit logiciel sur quatre facteurs d’affaires critiques : la sécurité, la fiabilité, l’efficacité des performances et la maintenabilité.

L’importance de l’ISO

Voici des exemples d'avenues qu’ISO explore dans l’amélioration continue

- Éliminer le papier et les formulaires Word et Excel qui créés des silos.
- Maximiser tout le potentiel de votre gestion documentaire.
- Optimiser la gestion de vos non-conformités et actions correctives.
- Gérer les risques, les projets et les programmes de façon intégrée.
- Mettre en place des parcours de compétences pour les employés et prestataires externes.
- Monter un environnement afin de mettre en valeur tout l’expertise et le savoir-faire dans votre organisation.
- Faire remonter en amont vos données statistiques pertinentes afin de démontrer l’importance de la Qualité au sein de votre organisation.

- Assurer une performance optimale de vos équipements
  Sujets de bases
- Gestion documentaire
- Gestion des connaissances
- Formations & Compétences
- Formations en ligne
- Non-conformités & Actions
- Audits & Inspections
- Analyse des risques
- Gestion des équipements
- Planification
- Gestion des fournisseurs
- Surveillance & Mesurage
- Revue de performance
- Tableau de bord
  Bénéfices visés
- Élimination des silos pour une meilleur performance organisationnelle.
- Partage accru des informations avec les dirigeants, les employés et les prestataires externes.
- Participation élargie au maintien de votre système qualité & amélioration continue.
- Renforcement d'une culture de la qualité et de l'amélioration continue.
- Captation des expertises et du savoir-faire chez vos employés les plus expérimentés.
- Transfert facilité des compétences auprès des nouveaux employés.
- Offrir à tous une vue en temps réel sur l'ensemble des activités et tâches en cours.
- Production sans effort de vos statistiques qualité et des indicateurs de performance.

Lorsqu'une entreprise se présente comme étant certifiée ISO, cela signifie qu'elle satisfait aux exigences d'une norme particulière. Les normes ISO offrent un moyen pour les entreprises d'améliorer les processus, d'augmenter les normes de sécurité, et d'améliorer la qualité et la cohérence dans un certain nombre de domaines. Elles peuvent être utilisées pour renforcer les processus et les procédures et améliorer la gestion de la qualité globale tout en offrant un moyen d'aider à réduire les coûts en améliorant les procédures.

En conclusion

L'ISO est plus qu'un simple ensemble de normes. C'est un outil précieux pour toute entreprise cherchant à améliorer ses processus et à assurer la qualité de ses produits et services. En adoptant les normes ISO, les entreprises peuvent non seulement améliorer leur efficacité et leur productivité, mais aussi gagner la confiance de leurs clients et se démarquer dans un marché de plus en plus concurrentiel. Alors, êtes-vous prêt à embarquer dans le voyage ISO ?

(done) Pourquoi la collaboration est essentielle dans le domaine de l’assurance qualité logicielle

January 18, 2024 10:38 AM

Dans le monde rapide et en constante évolution de l’assurance qualité logicielle, la collaboration est plus importante que jamais. Voici pourquoi.

Introduction

L’assurance qualité logicielle est un domaine complexe et multidimensionnel. Elle ne se limite pas aux tests - elle englobe toutes les activités qui garantissent la qualité du logiciel. Dans cet article, nous allons explorer pourquoi la collaboration est si essentielle dans ce domaine.

1. Définition de l’assurance qualité logicielle

L’assurance qualité logicielle est l’ensemble des activités qui garantissent la qualité du logiciel. Cela comprend la planification, le suivi, l’assurance et le contrôle de la qualité. Les principes d’assurance de la qualité, les facteurs de qualité, les modèles d’affaires et les coûts et bénéfices de la qualité sont tous des éléments clés de l’assurance qualité logicielle.

2. L’importance de la collaboration dans l’assurance qualité logicielle

La collaboration est essentielle pour la réussite des projets d’assurance qualité logicielle. Une bonne collaboration peut conduire à une meilleure qualité du logiciel, à une livraison plus rapide et à une meilleure satisfaction du client. Par exemple, lors du développement du logiciel XYZ, une collaboration efficace entre les équipes de développement et d’assurance qualité a permis de réduire les bugs de 30%.

3. Les conséquences d’une mauvaise collaboration

Une mauvaise collaboration peut avoir des conséquences désastreuses. Le non-respect des délais, les clients insatisfaits et les logiciels de mauvaise qualité sont autant de conséquences possibles. Par exemple, le projet ABC a échoué en raison d’un manque de collaboration entre les équipes, ce qui a entraîné de nombreux retards et une qualité de logiciel médiocre.

4. Comment améliorer la collaboration

Il existe de nombreuses façons d’améliorer la collaboration dans les équipes d’assurance qualité logicielle. Les techniques de communication, les outils de collaboration et les pratiques de travail en équipe peuvent tous contribuer à améliorer la collaboration. Il est également important de mettre en place les pratiques appropriées à un projet spécifique dès le début.

Conclusion

La collaboration est essentielle dans le domaine de l’assurance qualité logicielle. En travaillant ensemble, les équipes peuvent produire des logiciels de meilleure qualité, respecter les délais et satisfaire les clients. N’oubliez pas : ensemble, nous sommes plus forts.

(done) Le TMMi et CMMi c'est pour qui ?

January 18, 2024 10:38 AM

Bienvenue dans le monde de TMMi et CMMi !

Bonjour à tous ! Aujourd'hui, nous allons explorer deux modèles de référence fascinants qui transforment la façon dont les organisations gèrent leurs processus : le **TMMi (Test Maturity Model Integration)** et le **CMMi (Capability Maturity Model Integration)**. Préparez-vous à un voyage passionnant à travers ces modèles qui sont conçus pour vous aider à atteindre l'excellence dans vos processus organisationnels.

TMMi - Votre allié pour des tests logiciels de qualité

Le **TMMi** est comme un ami fidèle qui vous guide à travers les différentes étapes de maturité des tests logiciels. Que vous soyez au stade initial où les tests sont ad hoc et non gérés, ou que vous soyez à un niveau plus avancé où vous optimisez et prévenez les défauts, le TMMi est là pour vous aider. Il est destiné à toutes les organisations qui cherchent à évaluer et à améliorer leurs processus de test de manière rentable. Alors, si vous cherchez à améliorer vos tests logiciels, le TMMi pourrait être votre meilleur allié !

CMMi - Le guide pour un développement de produits réussi

D'autre part, le **CMMi** est comme un mentor expérimenté qui vous aide à appréhender, évaluer et améliorer vos activités d'ingénierie. Développé par le prestigieux Software Engineering Institute de l'université Carnegie-Mellon, le CMMi est largement utilisé par les entreprises d'ingénierie informatique et les industriels pour évaluer et améliorer leurs propres développements de produits. Si vous cherchez à améliorer vos processus de développement de produits, le CMMi est l'outil qu'il vous faut !

Pour qui sont ces modèles ?

Ces deux modèles sont destinés aux organisations (grandes ou petites) qui cherchent à améliorer leurs processus. Que vous soyez un chef de projet, un membre d'une équipe de développement de logiciels, un responsable de la qualité ou un gestionnaire de processus, ces modèles peuvent vous offrir une structure et des directives pour l'amélioration continue, l'optimisation des performances et l'alignement avec les objectifs commerciaux.

En conclusion, le TMMi et le CMMi sont plus que de simples modèles de référence. Ils sont des guides précieux pour toute organisation qui cherche à améliorer ses processus et à atteindre l'excellence. Alors, êtes-vous prêt à embarquer dans ce voyage passionnant vers l'amélioration des processus avec le TMMi et le CMMi ? Restez à l'écoute pour plus d'informations sur ces modèles passionnants !

(done) Matrice de traçabilité des exigences

January 18, 2024 10:38 AM

Introduction

Imaginez que vous êtes un chef de projet chargé de développer une nouvelle application mobile. Vous avez une liste d'exigences fournies par le client, mais comment pouvez-vous vous assurer que toutes ces exigences sont prises en compte pendant le développement et les tests ? C'est là qu'intervient la **matrice de traçabilité des exigences** (RTM), un outil essentiel qui permet de suivre et de vérifier l'intégralité des exigences tout au long du cycle de développement.

Qu'est-ce que la Matrice de Traçabilité des Exigences ?

Une RTM est une table qui lie les exigences aux cas de test pour assurer que toutes les exigences sont couvertes par des tests appropriés. Elle est utilisée pour conserver un enregistrement des exigences et assurer leur respect. C'est un outil précieux pour les gestionnaires de projet, car il leur permet de visualiser clairement le statut de chaque exigence et de s'assurer qu'aucune n'est oubliée.

Exemple : Supposons que l'une des exigences de votre application mobile soit "L'application doit permettre aux utilisateurs de se connecter via Facebook". Dans votre RTM, vous lieriez cette exigence à un cas de test spécifique, comme "Vérifier que l'utilisateur peut se connecter via Facebook". De cette façon, vous pouvez facilement voir si cette exigence a été testée et validée.

Types de Matrice de Traçabilité

Il existe trois types de matrice de traçabilité :

1. **Traçabilité en aval** : Elle assure la traçabilité des exigences aux cas de test. Cela permet de s'assurer que chaque exigence est couverte par un ou plusieurs cas de test, garantissant ainsi que le produit final répondra aux attentes du client.
2. **Traçabilité en amont** : Elle assure la traçabilité des cas de test aux exigences. Cela permet de s'assurer que chaque cas de test est lié à une exigence spécifique, évitant ainsi le développement de fonctionnalités non nécessaires ou non spécifiées.
3. **Traçabilité bidirectionnelle** : Elle assure une traçabilité à la fois en amont et en aval. Cela permet de s'assurer que chaque exigence est couverte par un cas de test et que chaque cas de test est lié à une exigence spécifique.

Exemple : Dans le cadre du développement de votre application mobile, vous pourriez avoir un cas de test pour vérifier que les utilisateurs peuvent se connecter via Facebook. Dans une matrice de traçabilité en aval, ce cas de test serait lié à l'exigence correspondante. Dans une matrice de traçabilité en amont, l'exigence serait liée au cas de test. Dans une matrice de traçabilité bidirectionnelle, les deux seraient liés.

Importance de la Matrice de Traçabilité

L'objectif principal de chaque testeur est de bien comprendre les exigences du client et de s'assurer que le produit développé est à la hauteur et aussi exempt de défauts que possible. Un RTM est une feuille de travail qui contient toutes les informations relatives aux exigences du client. Elle permet de s'assurer que toutes les exigences sont prises en compte et testées, ce qui contribue à la réussite du projet.

Exemple : En utilisant une RTM dans le développement de votre application mobile, vous pouvez vous assurer que toutes les exigences sont prises en compte et que chaque exigence

est couverte par un ou plusieurs cas de test. Cela vous permet de vérifier que toutes les fonctionnalités sont correctement mises en œuvre et testées avant le lancement de l'application.

Comment créer une Matrice de Traçabilité des Exigences ?

La création d'une RTM commence par la collecte de toutes les exigences du projet. Ces exigences peuvent provenir de diverses sources, telles que les documents de spécification des exigences, les entretiens avec les clients ou les parties prenantes, et les sessions de brainstorming de l'équipe de projet.

Une fois que toutes les exigences ont été collectées, elles sont inscrites dans la RTM. Chaque exigence est ensuite liée aux cas de test appropriés. Cela peut être un processus manuel, mais il existe également des outils logiciels qui peuvent automatiser une partie de ce processus.

Exemple : Pour créer une RTM pour votre application mobile, vous commencerez par recueillir toutes les exigences du projet. Vous pourriez le faire en organisant des sessions de brainstorming avec votre équipe, en menant des entretiens avec les clients ou en examinant les documents de spécification des exigences. Une fois que vous avez recueilli toutes les exigences, vous les inscrivez dans la RTM et les liez aux cas de test appropriés.

Conclusion

La matrice de traçabilité des exigences est un outil précieux pour tout gestionnaire de projet. Elle permet de garantir que toutes les exigences sont prises en compte et testées, ce qui contribue à la réussite du projet. En comprenant comment créer et utiliser une RTM, vous pouvez améliorer l'efficacité de votre gestion de projet et augmenter les chances de succès de votre projet.

(done) Le testeur Agile versus Coach en culture de la pratique assurance qualité du domaine logiciel

January 18, 2024 10:38 AM

Introduction

Dans le monde du développement logiciel, deux rôles clés sont souvent discutés : le testeur Agile et le Coach en culture de la pratique assurance qualité. Ces deux rôles sont essentiels pour assurer la qualité du logiciel, mais ils ont des approches et des responsabilités différentes.

Le testeur Agile

Le testeur Agile est un membre clé de l'équipe de développement. Il ne se contente pas de tester le logiciel, mais il a la responsabilité de guider les spécialistes et les experts pour qu'ils vérifient, valident et testent eux-mêmes. Par exemple, dans une équipe de développement, le testeur Agile pourrait organiser des ateliers de formation pour enseigner aux développeurs comment effectuer leurs propres tests unitaires. Cela permet de s'assurer que les problèmes sont détectés et corrigés dès le début du processus de développement.

Le Coach QA

Le Coach QA joue un rôle crucial dans une équipe Agile. Il ne se limite pas à travailler avec le testeur Agile, mais interagit avec tous les membres de l'équipe. Voici comment le Coach QA contribue à l'équipe Agile :

1. **Fluidifier les échanges** : Le Coach QA aide à fluidifier les échanges au sein de l'équipe. Il facilite la communication et assure que tout le monde est sur la même longueur d'onde. Par exemple, le Coach QA pourrait organiser des réunions régulières pour discuter des problèmes de qualité et pour s'assurer que tout le monde comprend les objectifs de qualité de l'équipe.
2. **Monter en compétence** : Le Coach QA fait monter en compétence sur le métier du test l'équipe QA. Il partage ses connaissances et son expertise pour aider l'équipe à améliorer ses compétences en matière de test. Par exemple, le Coach QA pourrait organiser des formations sur les dernières techniques de test ou sur l'utilisation de nouveaux outils de test.
3. **Gérer les anomalies** : Le Coach QA peut prendre le rôle de defect manager afin de s'assurer du bon suivi des anomalies de manière transverse. Par exemple, le Coach QA pourrait mettre en place un système de suivi des bugs pour s'assurer que tous les bugs sont correctement documentés et suivis.
4. **Apporter des feedbacks** : En tant que "premier utilisateur", le Coach QA apporte une valeur ajoutée grâce à ses feedbacks. Son retour est essentiel à la réactivité des parties prenantes du projet et à la maîtrise des coûts. Par exemple, le Coach QA pourrait fournir des retours précoces sur les nouvelles fonctionnalités, permettant à l'équipe de développement de faire des ajustements avant que le produit ne soit livré aux utilisateurs.

Conclusion

En fin de compte, le testeur Agile et le Coach en culture de la pratique assurance qualité jouent tous deux un rôle essentiel dans la garantie de la qualité du logiciel. Bien qu'ils aient des approches différentes, leur travail est complémentaire et contribue à la réussite globale de tout projet de développement logiciel.

(done) L'importance de la revue et de la validation en assurance qualité du logiciel

January 18, 2024 10:39 AM

Dans le monde d'aujourd'hui, où l'information est à portée de main, il est essentiel de comprendre l'importance de la revue et de la validation. Que ce soit dans le domaine de la recherche, de l'ingénierie, de la médecine ou de la gestion de projet, la revue et la validation sont des étapes cruciales pour garantir la qualité et l'exactitude.

Comprendre les termes

Avant de plonger dans le sujet, il est important de comprendre la différence entre certains termes couramment utilisés dans ce contexte :

- **Vérifier** : Consiste à s'assurer que les choses sont faites conformément à ce qui avait été défini. Par exemple, dans le développement de logiciels, la vérification pourrait impliquer de s'assurer que le code est écrit selon les normes de codage de l'entreprise.
- **Valider** : Consiste à s'assurer que les résultats sont bien atteints. Par exemple, dans le développement de logiciels, la validation pourrait impliquer de s'assurer que le produit fini répond aux attentes du client.
- **Approuver** : Donner son accord formel à quelque chose. Par exemple, un manager peut approuver un plan de projet.
- **Tester** : Évaluer les performances ou la qualité de quelque chose dans certaines conditions. Par exemple, dans le développement de logiciels, tester pourrait impliquer de vérifier un nouveau logiciel pour détecter les bugs.
- **Essayer** : Faire quelque chose pour voir si cela fonctionne ou convient. Par exemple, dans le développement de logiciels, essayer pourrait impliquer d'adopter une nouvelle méthode de travail pour voir si elle améliore la productivité.
- **Réviser** : Examiner quelque chose avec l'intention de corriger ou d'améliorer. Par exemple, dans le développement de logiciels, réviser pourrait impliquer de revoir un document pour corriger les erreurs grammaticales.

Le trios inséparables clés de l'assurance qualité du logiciel

1. **Planification** : Toujours s'assurer que ce que nous allons faire est la bonne chose à faire. Par exemple, avant de commencer à coder, un développeur de logiciels doit planifier comment il va aborder le problème et quels outils il va utiliser.
2. **Vérification** : Toujours s'assurer que ce que nous avons fait est le résultat attendu. Par exemple, après avoir écrit un morceau de code, un développeur de logiciels doit vérifier que le code fait ce qu'il est censé faire.
3. **Validation** : Toujours s'assurer que ce que nous avons livré est satisfaisant. Par exemple, après avoir livré un produit logiciel, un développeur de logiciels doit s'assurer que le client est satisfait du produit.

Exemples concrets

Pour illustrer ces concepts, prenons l'exemple d'une entreprise de développement de logiciels.

- Lors de la **planification**, l'équipe décide de développer une nouvelle fonctionnalité pour leur application. Ils définissent les objectifs de cette fonctionnalité et déterminent la meilleure façon de l'implémenter.
- Pendant la phase de **vérification**, l'équipe s'assure que le code pour la nouvelle fonctionnalité est écrit correctement et qu'il ne contient pas d'erreurs. Ils peuvent utiliser des outils de vérification de code pour aider à cette étape.
- Enfin, lors de la **validation**, l'équipe s'assure que la nouvelle fonctionnalité fonctionne comme prévu et répond aux besoins des utilisateurs. Ils peuvent recueillir des

commentaires des utilisateurs pour aider à cette étape.

Conclusion

La revue et la validation sont des étapes cruciales dans de nombreux processus. Elles garantissent que le travail est effectué correctement et que le produit final est de la plus haute qualité possible. En comprenant ces termes et en les appliquant correctement, nous pouvons améliorer la qualité de notre travail et la satisfaction de nos clients.

(done) "FLAKY" : Origine, Problèmes et Solutions

January 18, 2024 10:39 AM

L’automatisation des tests est un élément crucial du développement logiciel. Cependant, elle a ses propres défis. L’un d’eux est le phénomène des tests “FLAKY”. Le terme “FLAKY” a été popularisé en raison des faiblesses de Selenium et WebDriver. En moyenne, lors des exécutions de tests, seulement 6 tests sur 10 passaient. Pour pallier ce problème, les développeurs ont dû recourir à des solutions de contournement, comme relancer le même test 10 fois pour s’assurer de sa réussite ou de son échec.

Qu’est-ce qu’un test “FLAKY” ?

Un test “FLAKY” est un test qui, lorsqu’il est exécuté plusieurs fois de suite, passe parfois et échoue parfois, sans qu’aucun changement n’ait été apporté au code. Par exemple, imaginez un test qui passe 6 fois sur 10. Vous ne savez pas si les 4 échecs sont dus à un véritable bug ou à un problème avec le test lui-même.

Pourquoi les tests deviennent-ils “FLAKY” ?

Il existe plusieurs raisons pour lesquelles un test peut devenir “FLAKY”. Parmi les plus courantes, citons :

- **Des tests mal écrits** : Un test mal conçu peut donner des résultats incohérents. Par exemple, si un test dépend de l’état d’un autre test ou si les tests ne sont pas correctement isolés.
- **Des problèmes de synchronisation** : Si votre test dépend du timing ou de l’ordre d’exécution, il peut devenir “FLAKY”. Par exemple, si votre test suppose que certaines actions se produiront dans un ordre spécifique ou dans un certain délai.
- **La dépendance à l’ordre des tests** : Si l’ordre d’exécution des tests affecte leurs résultats, ils peuvent devenir “FLAKY”. Par exemple, si un test échoue lorsqu’il est exécuté après un certain test, mais réussit lorsqu’il est exécuté seul.
- **La concurrence** : Si plusieurs tests sont exécutés en parallèle et qu’ils interfèrent entre eux, ils peuvent devenir “FLAKY”.
- **Le choix du framework** : C'est également crucial pour éviter les tests “FLAKY”. Selenium a longtemps été le framework de référence pour l’automatisation des tests. Cependant, il présente certaines limites qui peuvent conduire à des tests “FLAKY”. Par exemple, Selenium nécessite des compétences en programmation pour écrire et maintenir des scripts de test. De plus, même de petits changements de code peuvent casser les tests Selenium, nécessitant du temps de développement pour les réparer. C’est pourquoi de nombreux développeurs et équipes de test cherchent des alternatives à Selenium.

Comment résoudre le problème des tests “FLAKY” ?

Heureusement, il existe plusieurs stratégies pour réduire la “FLAKINESS” des tests :

- **Isoler les tests** : Assurez-vous que chaque test est indépendant des autres. Cela signifie qu’il ne doit pas dépendre de l’état créé par d’autres tests.
- **Nettoyer correctement** : Assurez-vous que chaque test nettoie correctement après lui. Cela signifie qu’il doit supprimer toute donnée ou état qu’il a créé pendant son exécution.
- **Utiliser des outils de prédiction de la “FLAKINESS”** : Ces outils peuvent aider à identifier les tests qui sont susceptibles de devenir “FLAKY”.
- **Bien sur le premier point** : Ne pas utiliser Selenium, si possible.

En conclusion, les tests “FLAKY” sont un défi dans l’automatisation des tests. Cependant, avec une bonne compréhension du problème et les bonnes stratégies, il est possible de réduire leur impact et de maintenir la confiance dans vos tests automatisés.

***Comment écrire un récit de l'utilisateur (user story) en symbiose avec l'assurance qualité et de façon efficace ?

January 18, 2024 1:36 PM

Les user stories sont une technique de description des besoins d'un utilisateur dans un langage simple et compréhensible pour tous les membres de l'équipe de développement. Les user stories ne sont pas des exigences détaillées, mais plutôt une description du résultat souhaité1. Les user stories s'intègrent parfaitement aux frameworks Agile comme Scrum et Kanban.

Pour écrire une user story efficace, il est important de bien comprendre les besoins des utilisateurs et d'identifier le profil de l'utilisateur final en cernant votre public cible. Des profils "Persona" peuvent vous aider à comprendre le problème que le personnage veut voir résolu en utilisant le produit, et ce, à l'aide d'une conception visuelle.

En ce qui concerne l'assurance qualité, il est important de s'assurer que les user stories sont claires et précises pour éviter toute confusion ou malentendu entre les membres de l'équipe1. Il est également important de s'assurer que les user stories sont testables et mesurables pour garantir leur efficacité.

Dans "User Story" nous oublions souvent le mot "utilisateur" (user) et pourtant ce qui est le plus important du "récit de l'***utilisateur***" !

Pourquoi nous avons la convention d'une STORY dans la stratégie ?... Simplement parce que la documentation/couverture/validation est l'essence même de la définition du besoin et de la solution (le DoD/ l'Acceptation)

Comment écrire un récit de l'utilisateur en symbiose avec la qualité et de façon efficace ?

Les "user stories" font partie d'une approche agile qui permet de passer de l'écriture des exigences à la discussion. Toutes les user stories agiles incluent une ou deux phrases écrites et, plus important encore, une série de conversations sur la fonctionnalité souhaitée.

Une description courte et simple d'une fonctionnalité racontée du point de vue de la personne qui souhaite la nouvelle fonctionnalité, généralement un utilisateur ou un client du système. Les "user stories" suivent généralement un modèle simple :

En tant que < utilisateur (type) >, je veux < objectif > pour que < raison >.

C'est quoi la dif. entre AsISo et Gerkins ?

Certains seront surpris, mais le but est le même

×

c'est juste une façon différente de dire le besoin

	AsISo	Gerkins
	AS who (include the context) **I WANT** objective **SO** reason	**GIVEN** context (include the who) **WHEN** action **THEN** result **EXIST**
exemple	**AS** an utilisateur **I WANT** convertir l'unité de température à partir de Fahrenheit	**GIVEN** fahrenheit **WHEN** convertir **THEN** Celsius **EXIST**

	SO que je puisse lire la température en Celsius	
Point de vu	Le besoin utilisateur	Le besoin fonctionnel

Par contre à NOTER !

AsISo est pour une fonctionnalité (feature), beaucoup vont penser que c'est pour une story !!! l'important est d'avoir **Qui, Quoi, Pourquoi** **"As a <user or stakeholder type>, I want <some software feature**>, So that <some business value>" or "In order to <achieve some business value>, As a <stakeholder type>, I want <some new system **feature**>"
Gerkin est pour un scenario l'important est d'avoir **Contexte, Action, Attente(s)** **Arrange, Act, Assert** **Contexte peut être Setup, Precondition, Prerequisite, etc.**

Mais alors comment cela peut nous aider à écrire une story ?

La réponse est par la définition du besoin, définition de la solution, ainsi les spécifications et sa couverture testable.

…….

Bon ajoutons la relation avec la qualité

Si nous voulons ajouter un validation d'un comportement inverse comme dans l'exemple... Qu'arrivera t-il si l'utilisateur est déjà en celsius, par exemple ?

Option A (avec une épopée)

Est-ce qu'il est bon de faire un autre story, donc un test/une validation par story et partir d'une épopée ?

Certains seront surpris, mais oui nous pourrions livrer seulement la conversion fahrenheit à celsius "happy path" et une autre story pour l'éligibilité avant la conversion.

Même que découper une story de cette façon nous simplifie la vie de beaucoup, c'est le concept INVEST et donc le plus petit format (unitaire) testable.

- ici on ne se pose pas la question de la couverture, car une story = un test
- Plus, facile a estimer, car c'est très claire
- etc.

De cette façon, nous utilisons les "epic" en tant qu'exigence, ce qui prends tout son sens du pourquoi une "epic" !

Maintenant, comment écrit-on une "epic" ?

Et bien, c'est simple, les stories sont la définition de l'épopée, pas besoin d'un roman, juste un titre est généralement suffisant.

Si on part avec le "**SO** convertir en Celsius" ou le " **WHEN TO** convertir en celsius", nous pouvons, par exemple, nommer l'épopée : "Convertir en celsius"

Option B (sans épopée)

Bon, dans la réalité, si on fait l'option A, un dev sufi pour réaliser une epic complète !

Nous voulons faire ça moins unitairement et plus au niveau de la "feature", donc plusieurs scenarios pour une fonctionnalité.
Nous nous retrouvons ici dans les situations les plus communes... avoir moins de stories... souvent le centre d'une exigence est la story et un découpe par la suite la story au besoin selon l'estimé trop grande

Donc on voudrait être moins granulaire...

Cela augmente le risque de ne pas pouvoir tester complètement la story ou...

Nous devons nous assurer de couvrir tout les scenarios possibles dans la story

todo

Quelle est la différence entre :

- Définition du besoin et de la solution
- Critère d'acceptation
- UAT
- Définition de terminer (DoD)
- Plan
- Couverture
- Test
- Conditions de succès
- Description/Sommaire

Certains seront surpris, mais le but est le même



Ce sont tous des façons différentes d'exprimer le même besoin/solution

Une équipes mature pourrait utiliser juste des récits (sans épopée) avec un simple titre par exemple "convertir celcius", rien d'autre sauf un lien d'un plan de test. Pourtant, tout est complet, pas de confusion et on peut faire le développement zero bug piloté par les tests.

Voyons ensemble le SDLC en lien avec une story

todo

Maturité (le delta en assurance qualité logicielle)

January 18, 2024 1:37 PM

Le delta en assurance qualité logicielle fait référence à la différence entre l'état actuel de l'assurance qualité et l'état souhaité. Il peut être utilisé pour mesurer l'efficacité des processus d'assurance qualité et pour identifier les domaines qui nécessitent des améliorations.

La maturité en assurance qualité logicielle fait référence au niveau de maturité des processus d'assurance qualité dans une organisation. Elle peut être mesurée à l'aide de modèles de maturité tels que le modèle Capability Maturity Model Integration (CMMI). Les organisations peuvent utiliser ces modèles pour évaluer leur niveau de maturité actuel et identifier les domaines qui nécessitent des améliorations.

ref. : Critères d'évaluation de maturité des professionnels QA

Meilleures pratiques en automatisation

January 18, 2024 2:08 PM

Cette page résume bien les principes qui sont à la base de notre pratique.

Table des matières

- 1. Appliquer les meilleures pratiques du développement à l'automatisation
- 2. "Testeurs" membres à part entière de l'équipe de réalisation
- 4. Tester le plus tôt et le plus souvent possible
-
- 5. Choisir le meilleur coffre d'outils d'automatisation
- 6. Éviter le plus possible de tester via le UI (interface utilisateur)
- 7. Diviser les efforts de tests automatisés
- 8. Des données complètes et de qualité
- 9. Maximiser votre investissement

1. Appliquer les meilleures pratiques du développement à l'automatisation

Il faut aborder l'automatisation de la même façon que le développement pur et dur. Ce qui est bon pour le développement l'est pour l'automatisation. L'automatisation n'est qu'une autre forme de développement. plutôt que de développer des fonctionnalités, nous développons des validations. Le design, la structure, le découpage, la revue de code sont tous des incontournables à appliquer à l'automatisation des tests.

2. "Testeurs" membres à part entière de l'équipe de réalisation

Chez Askida, développant en mode Agile, les "testeurs" sont des membres à part entière des équipes de développement. Ainsi ils sont présents dès les débuts des projets. Les équipes de développement vont donc travailler de pair avec les différents membres de l'équipe de la pratique qualité Askida.

Chez Askida, les "testeurs" se définissent en 3 grands rôles:

Rôle	Responsabilités
Stratège en pratique qualité (Senior QA)	• Étude du contexte d'automatisation • Définition & élaboration l'approche de test • Mise en œuvre de la stratégie • Supervision et mentorat de l'équipe de tests • Spécialiste conseil • Coordination des différents intervenants • Analyse des résultats de tests • Formations spécialisées • Spécialistes automatisation & orchestration
Spécialistes automatisation & orchestration (Analyste en AQ)	• Définition & mise en place des plans de test • Définition & mise en place des scénarios de tests • Définition & mise en place des jeux de données • Sélection, définition et mise en place des méthodes d'automatisation • Entretien des plans de tests • Gestion des anomalies • Interprétation des résultats de tests
Scripteurs	• Mise en place des outils de scripting • Création et entretien des scripts • Déploiement en intégration continue des scripts

1. Automatiser les tests à valeurs ajoutées

   Vous ne pouvez pas automatiser tout et en fait vous ne devriez pas. Les tests manuels, lorsque judicieusement sélectionnés apportent de la valeur à l'ensemble des tests qui seront faits dans un projet.

   Il est donc primordial de s'assurer que les tests qui seront automatisés ont une réelle valeur:

2. Quels sont les composantes du système testé qui sont les plus fragiles? qui vous empêche de dormir? Ce sont certainement ces composantes qui auraient tout intérêt à être automatisées en premier lieu.
3. Qu'est-ce qui a tendance à être monotone et répétitif ? Ces tests ont tendance à être où les erreurs arrivent dans le test de manuel, qui n'a rien à voir avec l'habileté(la compétence) de vos testeurs et tout pour faire avec l'avis(esprit) humain quand affronté(confronté) avec aussi une grande partie de la même chose.
4. Est-ce un "one shot deal"? qu'est-ce qui perdurera dans le temps?

   Quand vous vous demandez où commencer par l'automatisation, ayez sur ces réflexions d'abord. Les tests de régression sont un bon exemple de ceci.

- Les cas de tests répétitifs
- Les cas de tests les plus sujets aux erreurs humaines (« error-prone »)
- Les cas de tests qui requièrent plusieurs jeux de données ou des jeux de données complexes.
- Les cas de tests qui réfèrent à des fonctionnalités stratégiques
- Les cas de tests qui sont difficilement réalisables manuellement
- Les tests qui s'exécutent sur plusieurs appareils ou logiciels ayant des configurations complexes & multiples
- Les cas de test qui demandent des efforts considérables, lorsqu'exécutés manuellement.

4. Tester le plus tôt et le plus souvent possible

Plus les tests sont exécutés tôt dans le processus, moins les anomalies détectées seront dispendieuses à corriger.

The earlier you test, the more you can test; the more bugs you'll find, and the more time the QA team will have to find fixes. It's simple!

By testing frequently, you'll also reduce the potential for disconnect between the coding and testing phases of your application lifecycle. Frequent testing means feedback will be going back and forth between your testing and development teams on a regular basis. They'll end up working much more closely together than before, and it will move the whole development lifecycle towards a continuous integration method of working.

5. Choisir le meilleur coffre d'outils d'automatisation

- Un ou plusieurs outils?
- Un outil qui répond à mes besoins actuels et futurs idéalement (plateforme de développement, teste une application web? .net? Java? Mobile? Quels sont les OS qui doivent être testés, etc.)
- Un outil flexible et que l'équipe de tests est en mesure de maitriser
- Un outil qui permettra de rédiger les tests de manière modulaire, réutilisable et flexible

You obviously need to make sure you're working with a testing tool which meets the requirements of your QA team.

More than that, though, you should opt for a tool that integrates with the rest of the tools used across your organization. Having an integrated set of tools promotes a common mindset and language across the company. It will unify departments under one overarching goal: to satisfy your customers.

It means that teams will understand each other even when their work is very different, and this will help considerably when they need to work together. Consider testing and marketing, for example. Both need to understand the end user; marketing will have a very different perspective than testers, but the feedback they get from customers can offer invaluable contributions to the feedback loop already established between testing and development.

...

When your tools talk to each other, your teams talk to each other, and that's best practice across all departments.

6. Éviter le plus possible de tester via le UI (interface utilisateur)

Privilégier une approche de tests où le moins de tests possible dépendent du UI. Les tests via le UI sont très dispendieux à développer et à entretenir. L'interface utilisateur change fréquemment et impacte alors les cas de tests qui se basent dessus. De plus, le développement du UI se termine souvent très près de la livraison et il reste donc très peu de temps pour réaliser les tests automatisés si nous devons en attendre la livraison par l'équipe de développement.

7. Diviser les efforts de tests automatisés

Allez chercher les meilleurs de vos spécialistes ! Selon les tests qui doivent être automatisés, acceptez de les diviser par compétences et connaissances. Attribuer l'automatisation des tests par spécialité est bénéfique. Tous n'ont pas les mêmes compétences. Ainsi la courbe d'apprentissage sera moins abrupte.

8. Des données complètes et de qualité

- Créer des jeux de données de tests structurés
- Gérer les données de tests à même les tests automatisés

9. Maximiser votre investissement

- Faites appel à des experts dans le domaine. Ils vous orienteront et vous garderont sur la bonne route tout au long du projet.
- N'automatisez pas le chaos ! Assurez-vous que vos processus de tests sont définis, clairs et maitrisés. Sinon l'automatisation ne fera qu'amplifier la situation.
- Approchez la démarche de manière itérative et évolutive. Tout ne doit pas être fait au jour 1.

Add label

Related pages

Conventions Alithya GoTest - CONVENTIONS Alithya - Scripting Best Practices / Standards (automation)

Conventions Alithya GoTest - CONVENTIONS Alithya - Scripting Best Practices / Standards (automation)

Pratique Assurance Qualité

Often read together

Definition Of Done

Definition Of Done

Oracle Cloud

Often read together

Test strategy automation

Test strategy automation

Oracle Cloud

Often read together

Proposition d'ajouter des points qui impliqueraient un CI/CD:

- Stratégies de déclenchement de tests automatisés (PR, Merge, Déploiement)
- Bonnes pratiques pour écrire des tests unitaires efficaces
- Tests de systèmes intégrés (System Integration Tests ou SIT)
- Automatisation d'UAT (débat et discussion de la valeur ajoutée de ce sujet)

Mise en place de l'Automatisation

January 18, 2024 2:10 PM

